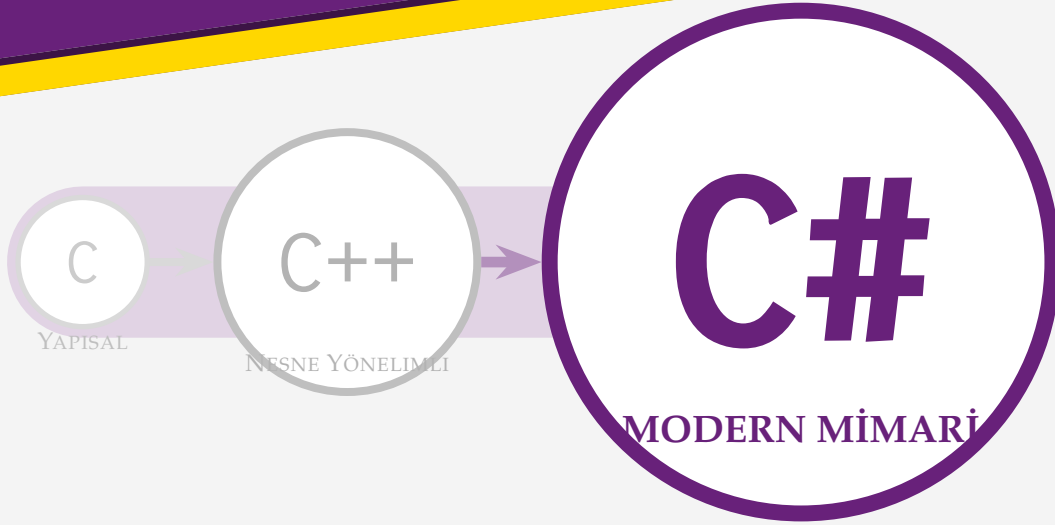


C# ve Modern Yazılım Mimarisi: Nesneden Mikroservice

Koleksiyonlar, LINQ, Tasarım Desenleri, Kurumsal Yazılımlar, Varlık Çerçevesi



Elektronik Yük. Müh.

İlhan ÖZKAN

Üçüncü Sürüm, Mayıs 2026, Ankara

C# ve Modern Yazılım Mimarisi: Nesneden Mikroservise

Yazar: Elektronik Yük. Müh. İlhan ÖZKAN

© Mayıs 2026, İlhan ÖZKAN. Tüm hakları saklıdır.

Bu kitap kâr amacı güdülmeden hazırlanmıştır ve **referans gösterilerek** her yerde kullanılabilir. C# programlama dilinin eğitimini veren ve alan tüm kişiler için serbestçe kullanılabilir.

Bu kitabın bir kısmı veya tamamı, **yazarın önceden yazılı izni alınarak** elektronik, mekanik, fotokopi veya herhangi bir kayıt sistemi ile çoğaltılabilir, yayımlanabilir veya depolanabilir.

ISBN: 978-625-92164-1-6

Baskı Tarihi: Mayıs 2026

Baskı Yeri: Ankara, Türkiye

Önsöz

Ülkemizde her alanda olduğu gibi yazılım alanında da bir terminoloji karmaşasının yaşandığı aşikardır. Son dönemde yazılanlar adeta çeviri yapan programların bir gecede ürettiği çıktılar birleştirilerek hazırlanmaktadır. Birçok programlama dili kullanarak yazılım geliştiriyor olmama rağmen, bu kitapların içeriğini anlama konusunda oldukça zorlanıyorum.

Bu kitapla birlikte tarihsel gelişimle birlikte programlama mantığının yapısal programlamaya geçişi sürecini de içerecek şekilde yazılımın gelişme süreci anlatılmıştır.

Yapısal programlamanın en çok uygulandığı Pascal dilidir. Pascal diline yakın zamanda ortaya çıkan C dili, kısa sürede sistem dili olarak tarihte yerini almıştır. Günümüzde sistem programlama ve yüksek hız gerektiren gömülü sistemlerde vazgeçilmez bir dil olmuştur.

Daha sonra C dilinden etkilenecek olan Java, C# gibi birçok nesne yönelimli programlama dili ve yapısal programlama üzerine kurulmuş dillerdir. Birçok modern programlama diline esin kaynağı olan bu dilin özellikle bilgisayar mühendisleri tarafından bilinmesi gerekir.

C# programlama dili ise Java gibi bir ara dil oluşturarak çalıştırılabilir kodun her platformda derlenmeden çalıştırılmasını hedefleyen ve programcının kurumsal yazılımları hızlıca geliştirmek için istenilen bileşenleri hazır olarak sunan kütüphanelere sahip bir saf nesne yönelimli programlama dilidir. Yüksek performans isteyen kurumsal yazılım projelerde çokça kullanılmaktadır.

Nesne yönelimli programlama mantığını oluşturan kavramların programcıların kafasında aynı ışığı yakması için bu kitap hazırlanmıştır. Bu dilin eğitimini verecek tüm kişiler için serbestçe kullanılabilir.

Bu kitabın güncel sürümüne <https://github.com/HoydaBre/csharp> adresinden erişilebilir. Bu kitabı yazmaktaki amacım, Türkçe düşünen ve konuşan öğrencilerimize kavramları doğru bir şekilde aktarmaktır. Katkılarınızı bekler iyi çalışmalar dilerim.

Elektronik Yüksek Mühendisi, İlhan ÖZKAN
ilhanozkan[at]outlook.com
[linkedin.com/in/ilhanozkan](https://www.linkedin.com/in/ilhanozkan)

Sürümler

İkinci Sürüm

İlk sürümden farklı olarak aşağıdaki değişiklikler yapılmıştır¹.

- ◊ Sınıf ve Nesne bölümü sadeleştirildi.
- ◊ Temsilciler ve Olaylar kısmı yeni bölüm yapıldı.
- ◊ Varlık Çerçevesi (Entity Framework) bölümü eklendi.
- ◊ Katmanlı Mimari bölümü sadeleştirildi ve örneği eklendi.
- ◊ Aşırı yükleme kavramı fonksiyonlar altına alındı.

Üçüncü Sürüm

İkinci sürümden farklı olarak²;

- ◊ Her bölüme düşün ve çöz kısmı eklendi.
- ◊ Kodlar renklendirildi. Daha rafine hale getirildi.
- ◊ Referans Tipler başlığı ve Değişken Tanımlama içeriği yenilendi.
- ◊ Foreach başlığı, Diziler bölümü sonuna eklendi.
- ◊ İsim Uzayları, Nesne Yönelimli Programlama altına taşındı.
- ◊ Genişleme Yöntemleri, Nesne Yönelimli Programlama altına taşındı.
- ◊ Yapılar başlığı yenilendi.
- ◊ Öznitelikler ardına Yansıma bölümü eklendi.
- ◊ Performans ve Bellek Optimizasyon Jenerikleri eklendi.
- ◊ İleri Seviye Jenerik Mimariler eklendi. Altında iş parçacığı (thread) yönetimi bulunmaktadır.
- ◊ Mikroservis Mimarisi başlığı eklendi.
- ◊ Bellek Yönetimi ve Denetimi bölümü eklendi.
- ◊ Birim Testi ve Test Güdümlü Geliştirme bölümü eklendi.

¹Haziran 2025 tarihinde yayımlanmıştır.

²Nisan 2026 tarihinde yayımlanmıştır.

İçindekiler

Önsöz	i
Sürümler	ii
1 Temel Bilgisayar Kavramları	1
1.1 Mühendis ve Teknisyen	1
1.2 Bilgisayara Niçin İhtiyaç Duyulur?	1
1.3 Veri ve Bilgi	1
1.4 En Basit Bilgisayar	1
1.5 Emri Alma, Çözme ve İcra Çevrimi	3
1.6 İşlemci ve Bellek İş Birliğinin Temeli	4
1.6.1 İşlem Nerede Yapılır?	4
1.6.2 Veri Nerede Durur?	4
1.7 İşlemci Tasarlama Stratejileri	5
1.8 Onaltılı Sayı Sisteminin Kullanılması	5
1.9 Program ve Programlama	6
1.10 Program Türleri	6
1.11 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri	7
1.12 Düşün ve Çöz	7
2 Temel Yazılım Kavramları	9
2.1 Düşük Düzey Programlama	9
2.2 Genel Amaçlı Yüksek Düzey Diller	10
2.3 Arapsaçı Kod	10
2.4 Değişken	12
2.5 Fonksiyon	13
2.6 Yapısal Programlama	14
2.7 Yapısal Programlama Genel Çerçevesi	14
2.8 Nesne Yönelimli Programlama İhtiyacı	15
2.9 Nesne Yönelimli Programlama	16
2.10 Nesne Yönelimli Programın Genel Çerçevesi	16
2.11 Soyutlama Merdiveni	18
2.12 Derleyici ve Yorumlayıcı	19
2.12.1 Derleyici	19
2.12.2 Yorumlayıcı	19
2.13 Düşün ve Çöz	19
3 Derleyiciler ve Çalıştırma Ortamı	21
3.1 Derleyici ve .NET Framework	21
3.2 Ortak Dil Tarifnamesi	22
3.3 Ortak Tip Sistemi	22
3.4 Derleme ve İcra Süreci	23
3.5 Ortak Ara Dil Kodu	24
3.6 .NET Framework Kurulumu	25
3.7 Entegre Geliştirme Ortamı	26
3.8 Çevrimiçi Derleyiciler	26
3.9 Derleme Zamanı	26
3.10 Hatalar	27
3.11 C# Projesi Oluşturma ve Çalıştırma	27
3.11.1 .NET Core CLI ile Proje Oluşturma	27
3.11.2 Visual Studio ile Proje Oluşturma	28
3.12 Düşün ve Çöz	29
4 C# Programlama Diline Giriş	31

4.1	Söz Dizim Kuralları	31
4.1.1	Kaynak Kodu Oluşturan Karakterler	31
4.1.2	Anahtar Kelimeler	31
4.1.3	Açıklamalar	32
4.2	Değişken Tanımlama	33
4.2.1	Değer Tipler	33
4.2.2	Referans Tipler	34
4.2.3	Değişken Bildirimi ve Tanımlaması	35
4.2.4	Değişken Kapsamı	35
4.2.5	Değişken Kimliklendirme Kuralları	35
4.3	Değişmezler	36
4.4	Yerel Sabitler	37
4.5	Anonim Tipler	38
4.6	Dinamik Değişkenler	39
4.7	Boş Olabilir Değer Tipler	39
4.8	Numaralandırma Tipi	40
4.9	Yazdığımız Kod Nasıl İcra Edilir?	41
4.10	Temel İşleçler	41
4.10.1	Aritmetik İşleçler	41
4.10.2	Tekli İşleçler	42
4.10.3	İlişkisel İşleçler	43
4.10.4	Bit Düzeyi İşleçler	43
4.10.5	Mantıksal İşleçler	44
4.10.6	Atama işleçleri	45
4.10.7	İşleç Öncelikleri	46
4.11	Kayan Noktalı Sayı Hesapları	47
4.12	Düşün ve Çöz	48
5	Giriş Çıkış İşlemleri	49
5.1	Modüler Programlama	49
5.2	Konsola Veri Yazma	49
5.3	Metinler	50
5.3.1	Metinleri Tanımlama	50
5.3.2	Tam Metin Değişmezleri	51
5.3.3	Temel Metin İşlemleri	51
5.3.4	Metin Enterpolasyonu	52
5.3.5	String Sınıfı	53
5.4	Konsoldan Veri Okuma	54
5.4.1	Temel Okuma Yöntemler	54
5.4.2	Veri Okuma ve Tip Dönüşümü	54
5.4.3	Veri Okuma ve Tip Dönüşümünde Profesyonel Yaklaşım	54
5.4.4	Klavyeden Basılan Tuşu Okuma	54
5.4.5	Konsoldan Veri Okurken Neye Dikkat Etmeli?	54
5.5	Konsola Yazılan Değerleri Biçimlendirme	54
5.5.1	Sayı Biçimlendirme	54
5.5.2	Zaman Biçimlendirme	56
5.6	Düşün ve Çöz	57
6	Kontrol İşlemleri	59
6.1	Ardışık İşlem ve Kontrol İşlemleri	59
6.2	Akış Diyagramları	59
6.3	Duruma Göre Seçimler	60
6.3.1	If Talimatı	60
6.3.2	If-Else Talimatı	62
6.3.3	Sarkan Else	64
6.3.4	Switch Talimatı ve İfadesi	65
6.3.5	Üçlü Koşul İşleci	67
6.4	İlişkisel Döngüler	67
6.4.1	Sayaç Kontrollü Döngüler	68
6.4.2	While Talimatı	69
6.4.3	Do-While Talimatı	69

6.4.4	For Talimatı	70
6.4.5	İç İçe Döngü Kodu	73
6.4.6	Gözcü Kontrollü Döngüler	73
6.4.7	BigInteger Veri Tipi	74
6.5	Dallanmalar	75
6.5.1	Continue ve Break Talimatları	75
6.5.2	Return Talimatı	77
6.5.3	Goto Talimatı	77
6.6	Düşün ve Çöz	77
7	Diziler	79
7.1	Dizi Nedir	79
7.2	Dizi Tanımlama	79
7.3	Dizi Elemanlarını Değiştirme	80
7.4	Dizi Üzerinde Yinelemeler	80
7.5	Dizileri Kopyalama	83
7.6	Dizileri Karşılaştırma	83
7.7	Array Sınıfı Yöntemleri	83
7.8	Çok Boyutlu Diziler	84
7.9	Pürüzlü Diziler	86
7.10	Foreach Talimatı	86
7.10.1	Foreach Neden Tercih Edilir?	87
7.10.2	IEnumerable ve IEnumerator	87
7.11	Dizilerde Dilimleme	87
7.11.1	Index Yapısı	87
7.11.2	Range Yapısı	87
7.12	Düşün ve Çöz	88
8	Nesne Yönelimli Programlama	89
8.1	Nesne Yönelimli Programlama Paradigması	89
8.2	Sınıf ve Nesne Nedir? Nasıl Tanımlanır?	89
8.3	Erişim Değiştiricileri	91
8.4	Yöntemler	92
8.4.1	Yöntem Tanımı Nasıl Yapılır	93
8.4.2	Nesneye Mesaj Verme	94
8.4.3	Yöntemlere Değerle Parametre Geçirme	94
8.4.4	Yöntemlere Referansla Parametre Geçirme	96
8.4.5	Yöntem Parametrelerini Çıktı Olarak Kullanma	96
8.4.6	Varsayılan Argüman	97
8.4.7	Özyinelemeli Yöntemler	97
8.4.8	Yöntem Parametresi Olarak Diziler	98
8.4.9	Yöntemlerin Aşırı Yüklenmesi	99
8.4.10	Çağrı Kuralı	100
8.5	Nesne Yapıcısı	101
8.5.1	Yapıcı Tanımlama	101
8.5.2	Yapıcı Aşırı Yükleme	102
8.5.3	this Anahtar Kelimesi	103
8.5.4	Yapıcı Zincirleme	104
8.5.5	Yapıcılarda Varsayılan Argümanlar	105
8.5.6	Birincil Yapıcı	106
8.5.7	required Anahtar Kelimesi	106
8.6	Soyut, Somut ve Bilgi Gizleme	107
8.7	Kapsülleme	107
8.8	Nesne Başlatıcıları	110
8.9	init Erişim Belirleyicisi ile Esnek Değişmezlik	110
8.10	Miras Alma	111
8.10.1	Miras Alınamayanlar	116
8.10.2	protected Erişim Değiştiricisi	116
8.10.3	Gölgeleme	117
8.11	Geçersiz Kılma	118
8.12	Statik Üyeler ve Statik Sınıf	118

8.13	İşleçlerin Aşırı Yüklenmesi	119
8.14	Sabit Alan Değişkenleri ve Yalnız Okunabilir Nesneler	120
8.15	Değiştirilemez Nesneler	122
8.16	Arayüzler	122
8.17	Sonlandırıcılar	124
8.18	Genişleme Yöntemleri	125
8.19	Sınıf Çeşitleri	125
8.19.1	Soyut Sınıf	125
8.19.2	Somut Sınıf	126
8.19.3	Taban Sınıf	126
8.19.4	Türemiş Sınıf	126
8.19.5	Kök Sınıf	126
8.19.6	Yaprak Sınıf	126
8.19.7	Kısır Sınıf	126
8.19.8	Tekil Sınıf	127
8.19.9	Yardımcı Sınıf	127
8.19.10	Kısmi Sınıf	127
8.20	Nesnelerle İlgili İşleçler	128
8.20.1	Tip Kontrol İşleci	128
8.20.2	Güvenli Tip Dönüşüm İşleci	128
8.20.3	Eşitlik İşleci	128
8.21	İsim Uzayları	129
8.21.1	İsim Uzayı Nedir?	129
8.21.2	Hiyerarşik İsim Uzayları	129
8.21.3	Dosya Kapsamlı İsim Uzayları	130
8.21.4	İsim Uzaylarında Takma İsim	130
8.21.5	Standart .NET İsim Uzayları	130
8.22	Düşün ve Çöz	130
9	Temsilciler, Lambda İfadeleri ve Olaylar	132
9.1	Temsilciler	132
9.2	Lambda İfadeleri	133
9.2.1	Lambda İfadesi Nedir?	133
9.2.2	Lambda Nasıl Tanımlanır?	133
9.2.3	Temsilcilerle Lambda Kullanımı	133
9.2.4	Yerleşik Temsilciler	134
9.2.5	Lambda İfadelerinin Gücü	135
9.2.6	Anonim Değişkenlerle Lambda İfadeleri	135
9.3	Olaylar	135
9.3.1	Olay Tanımlama ve Olay Tetiklendiğinde Tepki Verme	136
9.3.2	Yerleşik Olaylar	137
9.3.3	Olaylara Tepki Vermek İçin Lambda İfadeleri	139
9.3.4	Modern ve Standart Olay Kullanımı	140
9.4	Düşün ve Çöz	141
10	Sınıflar Arası İlişkiler ve SOLID İlkeleri	143
10.1	Unified Modeling Language	143
10.2	Sınıf Diyagramları	144
10.3	Sınıflar Arası İlişkiler	145
10.3.1	Genelleştirme İlişkisi	145
10.3.2	Bağımlılık İlişkisi	145
10.3.3	Sahiplik ve Ortaklık İlişkisi	146
10.3.4	Kalıtım Yerine Kompozisyon İlkesi	148
10.3.5	Biriktirme İlişkisi	149
10.4	Sınıf Tasarlama İlkeleri	150
10.4.1	Tek Sorumluluk İlkesi	150
10.4.2	Açık-Kapalı İlkesi	151
10.4.3	Liskov İkame İlkesi	152
10.4.4	Ara Yüzleri Ayırma İlkesi	152
10.4.5	Bağımlılıkları Ters Çevirme İlkesi	153
10.5	Düşün ve Çöz	153

11 Yapılar, Birlikler, Öznitelikler ve Yansıma	155
11.1 Yapılar	155
11.1.1 Yapı Tanımlama ve Temel Yapı	155
11.1.2 Yapı Öğelerine Erişim	155
11.1.3 Yapılarda Yapıcılar	156
11.1.4 Yapılarda Ara Yüz Gerçekleştirme	156
11.1.5 Yapı ve Sınıf Arasındaki Farklar	157
11.1.6 Değer Tipi Davranışı	157
11.1.7 Ne zaman Yapı Kullanmalı?	157
11.1.8 Yalnız Okunabilir Modern Yapı	157
11.2 Birlikler	158
11.3 record Anahtar Kelimesi	159
11.4 Öznitelikler	159
11.4.1 Hazır Öznitelikler	159
11.4.2 Kendi Özniteliğimizi Tanımlama	159
11.4.3 Çalışma Zamanında Özniteliklere Erişme	160
11.4.4 Öznitelikler Nerede Kullanılır?	160
11.5 Yansıma	160
11.5.1 Temel Giriş Noktası Type Sınıfı	160
11.5.2 Çalışma Zamanında Sınıfı Keşfetmek	161
11.5.3 Dinamik Nesne Oluşturma ve Yöntem Çağırma	161
11.5.4 Mahrem Alanlara Müdahale	161
11.5.5 Yansıma Nerede Kullanılır?	161
11.6 Düşün ve Çöz	162
12 İstisna Yönetimi	163
12.1 Yapısal Programlamada Hata Yönetimi	163
12.2 İstisna Yönetimi	163
12.3 Standart İstisna Tipleri	165
12.4 İstisna Filtreleri	168
12.5 Kullanıcı Tanımlı İstisnalar	168
12.6 finally Bloğu	169
12.7 İstisnayı Tekrar Fırlatma	170
12.8 Yığın Geri Sarma	170
12.9 Özel Durum İşleme Kuralları	171
12.10 Düşün ve Çöz	172
13 Tip Dönüşümleri ve Göstericiler	173
13.1 Tip Dönüşümleri	173
13.2 Örtük Dönüşümler	173
13.3 Açık Dönüşümler	173
13.4 Kullanıcı Tanımlı Dönüşümler	175
13.5 Yardımcı Sınıflarla Dönüşümler	175
13.6 Kutulama ve Kutudan Çıkarma	176
13.6.1 Kutulama	176
13.6.2 Kutudan Çıkarma	176
13.7 Göstericiler	176
13.7.1 Gösterici Tipi Bildirme	177
13.7.2 Gösterici Aritmetiği	179
13.7.3 Referanstan Çıkarma İşleci	179
13.7.4 Üyelere Erişim	180
13.8 Düşün ve Çöz	180
14 Jenerikler, Koleksiyonlar ve İndisleyiciler	182
14.1 ArrayList	182
14.2 Jenerik Programlama	182
14.2.1 Jenerik Sınıf Tanımlama	183
14.2.2 Jenerik Kısıtlamalar	183
14.2.3 Jenerik Yöntemler	184
14.2.4 C++ Şablonları ve C# Jenerikleri	184
14.2.5 Numaralandırıcılar ve Koleksiyonlar	184

14.2.6	Yineleyiciler	185
14.2.7	yield Anahtar Kelimesi	186
14.3	Koleksiyonlar ve Veri Yapıları	189
14.3.1	Dinamik Dizi List<T>	189
14.3.2	Sözlük Dictionary<TKey, TValue>	189
14.3.3	Benzersiz Küme HashSet<T>	190
14.3.4	Sıralı Küme SortedSet<T>	190
14.3.5	Yığın Stack<T>	191
14.3.6	Kuyruk Queue<T>	191
14.3.7	Bağlı Liste LinkedList<T>	191
14.3.8	Özet Tablo	191
14.4	Performans ve Bellek Optimizasyon Jenerikleri	191
14.4.1	Lazy<T>	192
14.4.2	LazyInitializer	192
14.4.3	İş Parçacığı Yönetimi ve İzole Veri Yapıları	193
14.4.4	Performans Notları	196
14.5	İleri Seviye Jenerik Mimariler	196
14.5.1	Nullable<T>	197
14.5.2	ValueTuple<T1, T2, ...>	197
14.5.3	EqualityComparer<T>	198
14.5.4	IProgress<T>	198
14.5.5	Comparer<T> ve IComparable<T>	199
14.5.6	Özet	200
14.6	İndisleyiciler	200
14.7	Düşün ve Çöz	201
15	LINQ Sorguları	202
15.1	LINQ Nedir?	202
15.1.1	Sorgu Şeklinde	202
15.1.2	Akıcı (fluent) Şekilde	202
15.2	IEnumerable ve IEnumerator Arayüzleri	203
15.3	IQueryable Arayüzü	203
15.4	LINQ Genişleme Yöntemleri	203
15.4.1	Where	203
15.4.2	Select	203
15.4.3	OrderBy ve OrderByDescending	204
15.4.4	Group By	204
15.4.5	First ve Last	205
15.4.6	FirstOrDefault ve LastOrDefault	205
15.4.7	Single	205
15.4.8	SingleOrDefault	206
15.4.9	Sum, Min, Max ve Average	206
15.4.10	Any ve All	206
15.4.11	Count	206
15.4.12	FindAll	206
15.4.13	Distinct	206
15.4.14	Intersect, Except ve Union	207
15.4.15	OfType	208
15.4.16	SelectMany	209
15.4.17	Join	209
15.4.18	Skip ve Take	210
15.4.19	Range ve Repeat	211
15.4.20	Contains	211
15.4.21	Zip	211
15.4.22	Aggregate	211
15.5	Let Anahtar Kelimesi	211
15.6	Düşün ve Çöz	212
16	Dosyalar, Akışlar ve Serileştirme	213
16.1	Akış Nedir?	213
16.2	Dosyalar ve Klasörler	213

16.3	Veri Akışları	213
16.4	Dosya İşlemleri	214
16.4.1	Dosya ve Klasör Bilgisi	214
16.4.2	Yol Yönetimi	215
16.4.3	Dosya Oluşturma	215
16.5	Metin Dosyalarını Toptan Okuma ve Yazma	216
16.5.1	Büyük Metin Dosyaları	216
16.6	Akışlar ile Dosya Yazma ve Okuma	216
16.6.1	Dosyaya Yazma	217
16.6.2	Dosyadan Okuma	217
16.6.3	İkili Dosyalar	217
16.7	Bellek Akışları	218
16.8	Serileştirme ve Seriden Çıkarma	219
16.8.1	JSON Serileştirme	219
16.8.2	XML Serileştirme	219
16.8.3	Binary (İkili) Serileştirme	220
16.8.4	Serileştirme Kontrol Mekanizmaları	220
16.8.5	Özet	220
16.9	Düşün ve Çöz	220
17	Veri Tabanı İşlemleri	222
17.1	Veri Tabanı Nedir?	222
17.2	Bağlantı Metni	222
17.3	İlişkisel Veri Tabanı İşlemleri	223
17.3.1	Veri Tabanında Tablolar Oluşturma	223
17.3.2	Veri Tabanından Veri Okuma	225
17.3.3	Sorgu Parametreleri	225
17.3.4	Veritabanı İşlemleri	225
17.4	Veri Kaynaklarına Erişim Yöntemleri	227
17.5	NoSQL Veri Tabanı	229
17.6	Düşün ve Çöz	230
18	Tasarım Desenleri	232
18.1	Desen Nedir?	232
18.2	Nesne İmalatı ile İlgili Desenler	232
18.2.1	Soyut Fabrika	233
18.2.2	Fabrika Yöntemi	235
18.2.3	Tekil Nesne	236
18.2.4	Kurucu	238
18.2.5	Prototip	240
18.3	Yapısal Desenler	241
18.3.1	Önyüz	241
18.3.2	Adaptör	244
18.3.3	Bileşik	246
18.3.4	Süsleyici	249
18.3.5	Sinek Sıklet	252
18.3.6	Vekil	254
18.3.7	Köprü	256
18.4	Davranışla İlgili Desenler	258
18.4.1	Sorumluluk Zinciri	258
18.4.2	Komut	260
18.4.3	Yineleyici	263
18.4.4	Gözlemci	265
18.4.5	Strateji	267
18.4.6	Şablon Yöntem	270
18.4.7	Ziyaretçi	272
18.4.8	Hatıra	274
18.4.9	Ara Bulucu	278
18.4.10	Durum	281
18.4.11	Yorumlayıcı	285
18.5	MVC Mimari Deseni	289

18.5.1 MVC Deseni Temel Yapısı	289
18.5.2 Gözlemci ile MVC Deseni İçin Örnek Proje	291
18.6 Düşün ve Çöz	301
19 Kurumsal Yazılımlar	302
19.1 Kurumsal Yazılım İçin Gereklikler	302
19.1.1 İş Analizi ve Gereksinim Analizi	302
19.1.2 Gereksinim Analizi Adımları	302
19.1.3 İyi Bir Gereksinimin Özellikleri	303
19.1.4 Gereksinim Analizi ve Modelleme Yol Haritası	303
19.1.5 Kurumsal Uygulama Geliştirmenin Temel Bileşenleri	304
19.2 Yazılım Geliştirme Modelleri	305
19.2.1 Şelale Modeli	305
19.2.2 Çevik Model	305
19.3 Modern Kurumsal Yazılımların Özellikleri	307
19.4 Katmanlı Mimari	307
19.4.1 Yekpare Uygulamalar	307
19.4.2 İki Katmanlı Mimari	308
19.4.3 Üç Katmanlı Mimari	310
19.4.4 Çok Katmanlı Mimari	312
19.4.5 Katmanların Tasarlanması	313
19.5 Bileşen Kullanımı	318
19.5.1 Sınıf Kütüphaneleri ve DLL Yapısı	318
19.5.2 NuGet ve Paket Yönetimi	318
19.5.3 Arayüzler ile Gevşek Bağlılık	318
19.5.4 Bağımlılık Enjeksiyonu	318
19.5.5 Kullanıcı Arayüzü Bileşenleri	318
19.6 Dağıtık Programlama	318
19.6.1 Dağıtık Programlama Teknolojileri	319
19.6.2 Tekdüzen Kaynak Bulucu ve Port	319
19.6.3 Web API ve RESTful Mimari	320
19.6.4 .NET Core RESTful Web API Örneği	320
19.6.5 Yüksek Performanslı gRPC Uygulama Örneği	324
19.6.6 RESTful Web API ve gRPC Seçim Kriterleri	325
19.7 Modern Dağıtık Programlama: Mikroservis Mimarisi	325
19.7.1 Mimari Katmanlar ve Bileşenler	326
19.7.2 Temel Prensipler ve Avantajlar	326
19.7.3 Mikroservislerde Haberleşme	326
19.7.4 Mikroservislerde Veri Yönetimi ve Tasarım Desenleri	326
19.7.5 Operasyonel Zorluklar	327
19.8 Örnek Kurumsal Uygulama	327
19.9 Düşün ve Çöz	330
20 Zaman Uyumsuz Programlama	331
20.1 Eş Zamansız Programlamanın Temelleri	331
20.1.1 Eş Zamanlı ve Eş Zamansız Arasındaki Fark	331
20.1.2 G/Ç Bağımlı ve İşlemci Bağımlı İşlemler	331
20.1.3 .NET'te Eş Zamansız Programlamanın Tarihi	331
20.2 Görev Tabanlı Programlama	331
20.2.1 Task Sınıfı	332
20.2.2 Task<T> Sınıfı	332
20.2.3 Görev Oluşturma Yöntemleri	332
20.2.4 Görev Durumları	332
20.3 async ve await Anahtar Kelimeleri	333
20.3.1 Temel Kullanım	333
20.3.2 Derleyici Dönüşümü	333
20.3.3 async Yöntem İmzaları	333
20.3.4 Birden Fazla Görevi Bekleme	334
20.3.5 ConfigureAwait(false) ile Bağlam Yönetimi	335
20.4 İptal Belirteci	335
20.4.1 Temel İptal Mekanizması	335

20.4.2	Manuel İptal	336
20.4.3	Bağlantılı İptal Kaynakları	337
20.5	İstisna Yönetimi Eş Zamansız Bağlamda	337
20.5.1	Temel İstisna Yakalama	337
20.5.2	Birden Fazla Hatayı Toplama	338
20.5.3	async void ve İstisna Tuzağı	339
20.6	ValueTask<T> ile Performans Optimizasyonu	339
20.6.1	Ne Zaman ValueTask<T> Kullanılmalı?	339
20.7	Paralel Programlama	340
20.7.1	Eş Zamansız ve Paralel Arasındaki Fark	340
20.7.2	Parallel.For ve Parallel.ForEach	340
20.7.3	PLINQ	341
20.7.4	İş Parçacığı Güvenliği ve Paylaşılan Veri	342
20.8	Düşün ve Çöz	343
21	Bellek Yönetimi ve Kaynak Denetimi	345
21.1	.NET'te Bellek Modeli	345
21.1.1	Yığın ve Öbek Bellek	345
21.1.2	Değer Tipleri ve Referans Tipleri	345
21.1.3	.NET Çöp Toplayıcı Kuşakları	346
21.1.4	stackalloc ile Yığın Tahsisi	346
21.2	Yığın Tabanlı Bellek Dilimi	346
21.2.1	Dizi Üzerinde Dilim Oluşturma	347
21.2.2	Metin İşlemede Span<T> ve Sıfır Tahsis	347
21.2.3	stackalloc ile Span<T> Birlikte Kullanımı	348
21.2.4	Span<T> Kısıtlamaları	348
21.3	Öbek Uyumlu Bellek Dilimi	349
21.3.1	Span<T> ve Memory<T> Karşılaştırması	349
21.3.2	Temel Kullanım	349
21.3.3	Memory<T> ile Büyük Veri İşleme	350
21.4	Dizi Havuzu ile Tahsis Optimizasyonu	350
21.4.1	Temel Kullanım	351
21.4.2	IDisposable Benzeri Sarmalayıcı ile Güvenli Kullanım	351
21.4.3	Özel Havuz Oluşturma	352
21.4.4	Performans Karşılaştırması	352
21.5	Yönetilmeyen Kaynakların Serbest Bırakılması	353
21.5.1	Temel IDisposable Uygulaması	353
21.5.2	Hem Yönetilen Hem Yönetilmeyen Kaynaklar	354
21.5.3	Modern using Bildirimi	355
21.5.4	Hata Durumunda Kaynak Güvenliği	356
21.6	Eş Zamansız Kaynak Temizleme	356
21.6.1	Temel Uygulama	356
21.6.2	Hem IDisposable Hem IAsyncDisposable Uygulama	357
21.6.3	IDisposable ve IAsyncDisposable Karşılaştırması	358
21.7	Bellek Sızıntısı Tespiti ve Profilleme	359
21.7.1	Yaygın Bellek Sızıntısı Kalıpları	359
21.7.2	Bellek Kullanımını İzleme	360
21.7.3	Profilleme Araçları	361
21.7.4	Bellek Yönetimi En İyi Uygulamaları	361
21.8	Düşün ve Çöz	361
22	Birim Testi ve Test Güdüllü Geliştirme	363
22.1	Test Piramidi ve Test Türleri	363
22.1.1	Test Piramidi	363
22.1.2	İyi Bir Birim Testin Özellikleri — FIRST	363
22.2	xUnit.net ile Birim Testi	363
22.2.1	Proje Kurulumu	364
22.2.2	Test Edilecek Sınıf	364
22.2.3	İlk xUnit Testi — [Fact]	364
22.2.4	Test İsimlendirme Kuralı	365
22.2.5	Parametrelili Test — [Theory] ve [InlineData]	365

22.2.6	[MemberData] ile Karmaşık Veri Setleri	366
22.2.7	xUnit Doğrulama Yöntemleri	366
22.2.8	Test Yaşam Döngüsü — Oluşturucu ve IDisposable	367
22.3	NUnit ile Birim Testi	368
22.3.1	NUnit Kurulumu	368
22.3.2	Temel NUnit Testi	368
22.3.3	NUnit Akıcı Doğrulama — Assert.That	369
22.3.4	xUnit ve NUnit Karşılaştırması	369
22.4	Moq ile Bağımlılık Taklidi	369
22.4.1	Taklit İçin Arayüz Tasarımı	370
22.4.2	Moq Kurulumu ve Temel Kullanım	371
22.4.3	Yöntem Çağrılarını Doğrulama — Verify	372
22.4.4	Moq — Gelişmiş Kullanım	373
22.5	Eş Zamansız Kod Testi	373
22.5.1	Eş Zamansız Üretim Kodu	373
22.5.2	Eş Zamansız Test Yöntemleri	374
22.6	Test Güdümlü Geliştirme — TDD	375
22.6.1	TDD ile Alışveriş Sepeti Geliştirme	375
22.7	Test Organizasyonu ve En İyi Uygulamalar	377
22.7.1	Test Projesi Klasör Yapısı	377
22.7.2	Testleri Kategorilere Ayırma	378
22.7.3	Temiz Test Kodu İlkeleri	378
22.7.4	Kod Kapsama Analizi	379
22.8	Düşün ve Çöz	379
23	Varlık Çerçevesi	381
23.1	Nesne-İlişkili Eşleme	381
23.2	Veri Tabanı Bağlamı	381
23.2.1	Önden Yükleme	382
23.2.2	Tembel Yükleme	383
23.2.3	Açıkça Yükleme	383
23.2.4	Varlıkları Yüklerken Değişiklik Takibini Kapatma	383
23.3	Önce Kod	384
23.4	Önce Kod Kimliklendirme Kuralları	387
23.4.1	Var Olan Bir Kimliklendirme Kuralını Geçersiz Kılma	387
23.4.2	Birincil Anahtar Kuralı	388
23.4.3	Veri Tipi Keşfi	388
23.4.4	Ondalık Kimliklendirme Kuralı	391
23.4.5	İlişki Kuralı	392
23.4.6	Yabancı Anahtar Kimliklendirme Kuralı	392
23.5	Önce Kod Veri Öz nitelikleri	393
23.5.1	[Column] Öz niteliği	393
23.5.2	[Required] Öz niteliği	393
23.5.3	[MaxLength] ve [MinLength] Öz nitelikleri	393
23.5.4	[ForeignKey(string)] Öz niteliği	393
23.5.5	[InverseProperty(string)] Öz niteliği	394
23.5.6	[ComplexType] Öz niteliği	394
23.5.7	[Range(min,max)] Öz niteliği	395
23.5.8	[NotMapped] Öz niteliği	395
23.5.9	[Table] Öz niteliği	395
23.5.10	[Index] Öz niteliği	396
23.5.11	[Key] Öz niteliği	396
23.5.12	[StringLength(int)] Öz niteliği	396
23.5.13	[DatabaseGenerated] Öz niteliği	396
23.6	Mevcut Veri Tabanı ile Önce Kod	397
23.7	Veri Göçü	399
23.8	Düşün ve Çöz	410
Dizin		412

Şekiller Listesi

1.1	Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği	2
1.2	Frekans 1 olan Kare Dalga İşareti	3
1.3	Basit Bir İşlemcinin Yapısı	3
1.4	İşlemcinin Emri Alma, Çözme ve İcra Çevrimi	4
2.1	C# Kodu Temel Yapısı	17
3.1	Ortak Tip Sistemi (CTS)	22
3.2	Klasik Derleme Süreci	23
3.3	.NET Framework Derleme Süreci	23
3.4	Derlenmiş bir Çalıştırılabilir Uygulamanın Çalışma Süreci	24
6.1	Akış Diyagramları Genel Şekilleri	59
6.2	Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı	60
6.3	If Talimatı İcra Akışı	60
6.4	Bir Sayının İşaretini Bulan Akış Diyagramı	62
6.5	If-Else Talimatı İcra Akışı	62
6.6	Bir Sayının İşaretini Bulan If-Else Akış Diyagramı	63
6.7	Switch Talimatı İcra Akışı	65
6.8	Sayaç Kontrollü Döngü İcra Akışı	68
6.9	While Talimatı İcra Akışı	69
6.10	Do-While Talimatı İcra Akışı	70
6.11	For Talimatı İcra Akışı	71
6.12	While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı	75
6.13	For Döngüsünde Break ve Continue Talimatları İcra Akışı	76
8.1	Yöntem Çağrısı Sırasında Yığın Bellek	95
8.2	Okul Projesi Sınıfları UML Diyagramı	114
8.3	Kalıtım Uygulanmış Okul Projesi Sınıfları UML Diyagramı	116
10.1	Örnek Bir Sınıf Diyagramı	144
10.2	Sınıf Diyagramlarında Arayüz Gerçekleştirme	144
10.3	Sınıf Diyagramlarında Sınırlama ve Çokluk	144
10.4	Genelleştirme İlişkisi Örneği	145
10.5	Bağımlılık İlişkisi Örneği	145
10.6	Sahiplik ve Ortaklık İlişkisi Örneği	147
10.7	Biriktirme İlişkisi Örneği	149
17.1	SQLite Veritabanı	224
17.2	Microsoft SQL Server LocalDB Üzerinde Veri Tabanı Oluşturma	228
18.1	Soyut Fabrika Deseni UML Diyagramı	233
18.2	Fabrika Yöntemi Deseni UML Diyagramı	235
18.3	Tekil Deseni UML Diyagramı	236
18.4	Kurucu Deseni UML Diyagramı	238
18.5	Prototip Deseni UML Diyagramı	240
18.6	Önyüz Deseni UML Diyagramı	241
18.7	Adaptör Deseni UML Diyagramı	244
18.8	Bileşik Deseni UML Diyagramı	246
18.9	Süsleyici Deseni UML Diyagramı	249
18.10	Sinek Sıklet Deseni UML Diyagramı	252
18.11	Vekil Deseni UML Diyagramı	254
18.12	Köprü Deseni UML Diyagramı	256
18.13	Sorumluluk Zinciri Deseni UML Diyagramı	258
18.14	Komut Deseni UML Diyagramı	260

18.15Yineleyici Deseni UML Diyagramı	263
18.16Gözlemci Deseni UML Diyagramı	265
18.17Strateji Deseni UML Diyagramı	267
18.18Şablon Yöntem Deseni UML Diyagramı	270
18.19Ziyaretçi Deseni UML Diyagramı	272
18.20Hatıra Deseni UML Diyagramı	275
18.21Ara Bulucu Deseni UML Diyagramı	278
18.22Durum Deseni UML Diyagramı	281
18.23Yorumlayıcı Deseni UML Diyagramı	286
18.24ProgressBarView Kullanıcı Kontrolü	297
18.25PictureBox Kullanıcı Kontrolü	298
18.26WinFormApp1 Uygulamasında Form1	300
18.27WinFormApp1 Uygulamasının Çalışması	301
19.1 Yekpare Uygulama	308
19.2 İstemci-Sunucu Mimarisi	309
19.3 Üç Katmanlı Mimari	310
19.4 Çok Katmanlı Mimari	313
19.5 Niteliksel (İşlevsel Olmayan) Bileşenler	314
19.6 Web API'ye Tarayıcı Üzerinden Erişim	323
19.7 Web API ile Kitap Controller'a Tarayıcı Üzerinden Erişim	324
22.1 Test Piramidi	363
22.2 TDD Döngüsü: Kırmızı → Yeşil → Yeniden Düzenle	375
23.1 CodeFirst.sqlite Veritabanında EF Core Tarafından Oluşturulan Tablolar	387
23.2 EF Core Veri Tipi Keşfi	390
23.3 Veritabanında Ölçüm için EF Core Tarafından Oluşturulan Tablo	391
23.4 SQLServerMevcut.mdf Veritabanında Önceden Oluşturulan İller Tablosu	398
23.5 Göç Sonrası Veri Tabanı	408

Tablolar Listesi

1.1	Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri	2
1.2	Onaltılı Sayı Rakamları ve İkili ve Onlu Karşılıkları	5
1.3	6800 Emir Seti Örneği	6
1.4	Yazılımlar ve Farkları	7
2.1	Tam Sayılar ve Sayı Sınırları	12
2.2	Kayan Noktalı Sayılar ve Sınırları	13
2.3	İki İle Çarpma Fonksiyonu	13
2.4	Soyutlama Merdiveni	18
2.5	Derleyici ve Yorumlayıcı Farkı	19
4.1	Temel Anahtar Kelimeler	31
4.2	Bağlamsal Anahtar Kelimeler	32
4.3	Tam Sayı Veri Tipleri ve Özellikleri	33
4.4	Kayan Noktalı ve Ondalık Veri Tipleri	33
4.5	Mantıksal, Karakter ve Temel Referans Tipleri	34
4.6	Referans Tipler ve Özellikleri	34
4.7	Değer Tipler İçin Değişmez Biçimleri	36
4.8	Aritmetik İşleçler	42
4.9	Tekli İşleçler	42
4.10	İlişkisel İşleçler ve Mantıksal Karşılıkları	43
4.11	Bit Düzeyi İşleçler	44
4.12	Mantıksal İşleçler ve Kısa Devre Özelliği	45
4.13	Atama İşleçleri	46
4.14	İşleç Öncelik Hiyerarşisi	46
5.1	Kaçış Dizileri ve Unicode Karşılıkları	52
7.1	Array Sınıfı Yöntemleri	83
8.1	Erişim Belirleyicileri ve Erişim Sınırları	92
8.2	new ile override Karşılaştırması	119
8.3	Aşırı Yüklenebilecek İşleçler	120
10.1	Niçin Kompozisyon Kullanılmalı?	148
11.1	Yapı ve Sınıf Karşılaştırması	157
12.1	Yaygın Sistem İstisnaları	166
13.1	Örtük ve Açık Dönüşüm Karşılaştırması	174
14.1	C++ Şablonu ve C# Jenerik Karşılaştırması	184
14.2	Koleksiyonlar Özet Tablosu	192
14.3	Thread ve Task Karşılaştırması	193
14.4	Veri Saklama ve Taşıma Yapıları Karşılaştırması	195
14.5	Asenkron Hata Yönetimi Yöntemleri	196
14.6	Ertelenmiş Başlatma ve Veri Saklama Yapıları	197
14.7	ValueTuple Teknik Karşılaştırması	197
14.8	Kritik Jenerik Yapıların Teknik Analizi	200
16.1	SearchOption Seçenekleri ve Karşılaştırması	214
16.2	FileMode Seçenekleri ve Davranışları	215
16.3	Serileştirme Formatlarının Teknik Analizi	220
19.1	İş Analizi-Gereksinim Analizi Karşılaştırması	302

19.2 Gereksinim Türleri	303
19.3 HTTP Yöntemlerinin RESTful Servislerde Kullanımı	320
19.4 Karşılaştırma ve Seçim Kriterleri	325
20.1 G/Ç Bağımlı ve İşlemci Bağımlı İşlemler	331
20.2 Task Durum Geçişleri	332
20.3 async Yöntem Dönüş Tipleri	334
20.4 Task<T> ve ValueTask<T> Karşılaştırması	340
20.5 Eş Zamansız ve Paralel Programlama Karşılaştırması	340
21.1 Yığın ve Öbek Bellek Karşılaştırması	345
21.2 Çöp Toplama Kuşakları ve Özellikleri	346
21.3 Span<T> ve Memory<T> Karşılaştırması	349
21.4 Klasik Dizi ve ArrayPool Karşılaştırması (100.000 iterasyon, 1 KB dizi)	353
21.5 IDisposable ve IAsyncDisposable Karşılaştırması	358
21.6 .NET Bellek Profilleme Araçları	361
21.7 Bellek Yönetimi En İyi Uygulamaları	362
22.1 Test Türleri Karşılaştırması	363
22.2 Test İsimlendirme Örnekleri	365
22.3 xUnit.net ve NUnit Karşılaştırması	370
23.1 EF Core Tür Keşfi ve Yapılandırma Hiyerarşisi	391

1. Bölüm

Temel Bilgisayar Kavramları

1.1 Mühendis ve Teknisyen

Mühendis (**engineer**), ihtiyaç duyulan ürünü; ekonomik (**cost**), güvenli (**safe**) ve kullanışlı (**functional**) olarak zamanında (**on time**) ortaya koyan kişilerdir. Bunu yapmak için; keşif (**invent**), tasarım (**design**), analiz (**analysis**), uygulama (**build**) ve sinama (**test**) yaparlar ¹.

Teknisyenler (**technician**) ise genellikle laboratuvarında teknik ekipmanlarla ilgilenen veya bu ekipmanlar ile pratik çalışmalar yapan kişilerdir. Genel olarak amacı ekipmanın bakım ve kullanımına ilişkindir.

Konumuz yazılım olduğundan, yazılım mühendisleri istenilen bilgisayar yazılımını ekonomik, güvenli ve kullanışlı olarak zamanında ortaya koyan kişilerdir.

Programcılar ise teknisyenlere karşılık gelir. Yani geliştirme aşamasında bazı yazılım kısımlarının kodlanmasını veya geliştirilen yazılımın çalışır tutulmasını sağlarlar.

1.2 Bilgisayara Niçin İhtiyaç Duyulur?

Bir çiftçi niçin traktöre ihtiyaç duyar? Bir berber niçin elektrikli tıraş makinesi kullanır? Benzer şekilde bir mühendis niçin bilgisayara ihtiyaç duyar? İşte bütün bu soruların cevabı yapılacak işleri daha kısa sürede yapmaktır. Yani bütün araç gereç ve makinalara olan ihtiyacımız işlerimizi **daha hızlı** (**speed**) yapabilmek içindir. Bütün makineler gibi bilgisayar da veriden hızlıca bilgi elde etmemizi ve tekrarlanan hesaplamaları hatasız yapmamızı hızlıca sağlayan makinelerdir.

Veri sorumlusu, veri hazırlama kontrol işletmeni gibi unvanlar bu sebeple ortaya çıkmıştır. Bu unvanlara ihtiyaç duyulmasının sebebi verinin belli bir şekle uygun olarak bilgisayara girilmesi ve işlenen verinin saklanması ve raporlanmasının bilgisayar dünyasında önemli bir yer tutmasıdır. Bilgisayarlar ilk zamanlarda daha çok cebirsel ifadeleri hatasız ve hızlı bir şekilde yapmak için kullanılmışlardır. Bunu daha sonradan ALGOL (**ALGO**ritmic **L**anguage) adını alacak IAL (**I**nternational **A**lgebraic **L**anguage) ya da FORTRAN (**FOR**mula **TRAN**slation) dilinden de anlayabiliriz.

1.3 Veri ve Bilgi

Veriler (**data**), genellikle bir ölçüm ya da sayım sonucu elde edilen ve hakkında az şey bilinen varlıklardır. Örneğin ölçülen hava sıcaklığı, derse giren öğrenci sayısı, tartılan sebzenin ağırlığı gibi şeyler.

Bilgi (**information**) ise verinin işlenerek ortaya çıkan anlamlı verilerdir. Örneğin ortalama hava sıcaklığı, ölçülen yerde bir ölçüm yapmaya gerek kalmadan sıcaklığı yaklaşık olarak bilmeye yarayan bir bilgidir. Bir sınıftaki not ortalaması, o sınıftan rastgele seçilen bir öğrencinin notunu yaklaşık belirleyen bir bilgidir. Kısacası bilgi, işlenmiş veridir

1.4 En Basit Bilgisayar

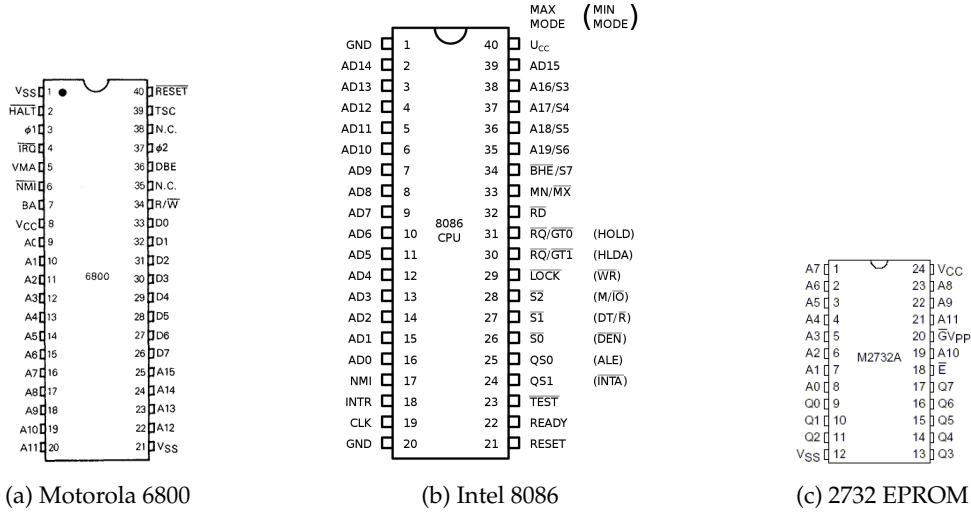
En basit bilgisayar üç bileşenden oluşur;

1. Merkezi İşlem Birimi CPU (**C**entral **P**rocessing **U**nit): Kısaca **işlemci** (**processor**) olarak adlandırılır.
2. Bellek (**memory**): İşlemcinin işleyeceği **emirleri** (**instruction**) ve verileri barındırır.
3. Yol (**bus**): İşlemci ile belleği birbirine bağlayan elektrik yollarıdır.

Hem belleğin hem de işlemcinin adres (**address**) veri (**data**) bacakları bulunur. Bu bacakları karşılıklı olarak yollar (**bus**) birbirine bağlar. Bunların yanında hem işlemcinin hem de belleğin okuma (**read**), yazma (**write**) bacakları bulunur. Ayrıca işlemciye özel kesme (**interrupt**) veya kristal bağlanan bacakları gibi kontrol (**control**) bacakları da bulunur. İşlemci ve bellek, entegre devre (**integrated circuit**) kısaca **yonga** şeklinde üretilirler.

Şekil 1.1'de görüldüğü üzere 6800 işlemcisinin 15 adet adres bacağı, 8 adet veri bacağı bulunmaktadır. 8086 işlemcisinin ise 20 adet adres bacağı bulunmakta olup bu bacakların 16 adedi aynı zamanda veri bacağı olarak kullanılmaktadır. İşlemciye karşılık olarak kullanılacak 2732 belleğin ise 12 adet adres bacağı, Q ile gösterilen 8 adet veri bacağı bulunmaktadır.

¹www.bls.gov/ooh/architecture-and-engineering



Şekil 1.1: Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği

Görüldüğü üzere işlemci ile bellek arasında bağlantı sağlayan elektrik yolları (bus) adres veya veri taşıır. Bu yolların veri taşıyanlarına veri yolu (data bus), adres taşıyanına ise adres yolu (address bus) adı verilir. Veri yolu; hem işlemci tarafından belleğe veri yazmak veya bellekten işlemciye taşınacak veriler için kullanıldığından çift yönlüdür. Adres yolu ise belleğin hangi bölgesindeki veriye ulaşılacağını belirleyen hatlar olup tek yönlüdür. 8086 işlemcisinde AD ile işaretli veri ve adres için kullanılan ortak yolların ne zaman adres ne zaman veri göstereceği S0 bacağından belli olur.

Bilgisayarlar elektriklerle çalışan aletlerdir. İşlemci ile bellek arasında bir hatta elektrik varsa 1 yoksa 0 olarak anımlandırılır. Bunlar ikili (binary) sayı sisteminin rakamlarıdır ve kısaca bit (Binary DigiT) olarak adlandırılır. Dolayısıyla; bir elektrik hattının taşıyacağı veri, bir haneli ikili sayı ile, iki elektrik hattının taşıyacağı veri, iki haneli ikili sayı ile ... ifade edilebilir.

İşlemcinin veri yolunun genişliği kelime uzunluğudur (word length). ve işlemcinin aynı anda işlem yaptığı bit sayısını da verir. Sekiz (8) bitlik veriye bayt (byte) adı verilir ve bellek miktarı genellikle bayt olarak ölçülür.

İşlemcinin veri yolu, 8 bit ise BYTE işlemci, 16 bit ise WORD işlemci, 32 bit ise DWORD (Double WORD) işlemci, 64 bit ise QWORD (Quad WORD) işlemci olarak adlandırılır.

İşlemcinin adreslenebilir bellek miktarı ise işlemcinin adres hatlarının sayısı yani adres yolu ile belirlenir. Adres yoluna konulan her bir adres ile 1 bellek baytı adreslenir. Adres yolu ne kadar geniş ise adreslenebilir bellek miktarı da o kadar artar. Örneğin 64 bit adres bacağına sahip işlemci $2^{64}=4.294.967.296$ kadar bellek baytını adresler.

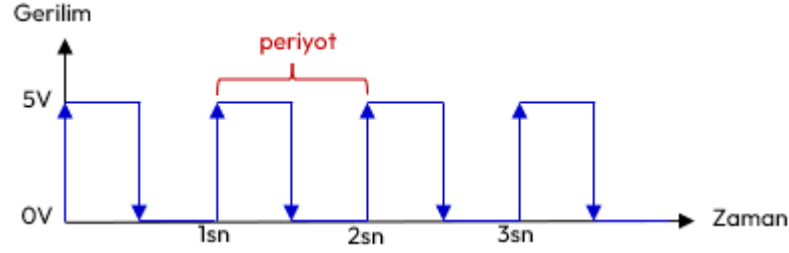
İşlemci	Veri Yolu	Adres Yolu	Adreslenebilir Bellek	Kilo bayt	Mega bayt	Giga bayt
Motorola 6802	8 Bit	16 Bit	$2^{16}=65.536$	64		
Intel 8086	16 Bit	20 Bit	$2^{20}=1.048.576$	1024	1	
Intel 80286	16 Bit	24 Bit	$2^{24}=16.777.216$		16	
Intel Pentium	32 Bit	32 Bit	$2^{32}=4.294.967.296$		4096	4
Intel i7	64 Bit	36 Bit	$2^{36}=68.719.476.736$			64

Tablo 1.1: Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri

Bilgisayarlarda kablodaki gerilimin değeri genelde 5V, 3.3V, 2.8V, 2V, 1.5V veya 1V gibi değerler alır. Programcılar için gerilimin ne olduğu değil varlığı önemlidir. Gerilim değerin yüksek olması fazla akım çekme ve ısınma anlamına gelir. Bu nedenle günümüzde daha düşük gerilimle çalışan işlemciler tasarlanmaktadır.

İşlemciler kare dalga olarak oluşturulmuş elektriksel bir işareti saat (clock) olarak kullanır. Şekil 1.2'de gösterilen bu elektrik işareti işlemciye φ ya da CLK bacaklarından uygulanır. Saniyede 1 kez bir dalga üreten işaretin frekansı $1/1\text{sn} = 1 \text{ Hertz} = 1 \text{ Hz}$. olarak hesaplanır. Bir mikro saniyede bir kez dalga üreten işaretin frekansı; $1/(1\mu\text{s}) = 1/(10^{-6}\text{s}) = 10^6 \text{ Hertz} = 1.000.000 \text{ Hertz} = 1 \text{ MHz}$ olarak hesaplanır.

İşlemciye uygulanan saatin frekansı ne kadar yüksek olursa işlemci o kadar hızlanır. Ancak işlemcilerin de tasarımından ötürü karşılayabileceği belli bir frekans değeri bulunmaktadır. Günümüzde 5 GHz frekansını karşılayan işlemciler bulunmaktadır.

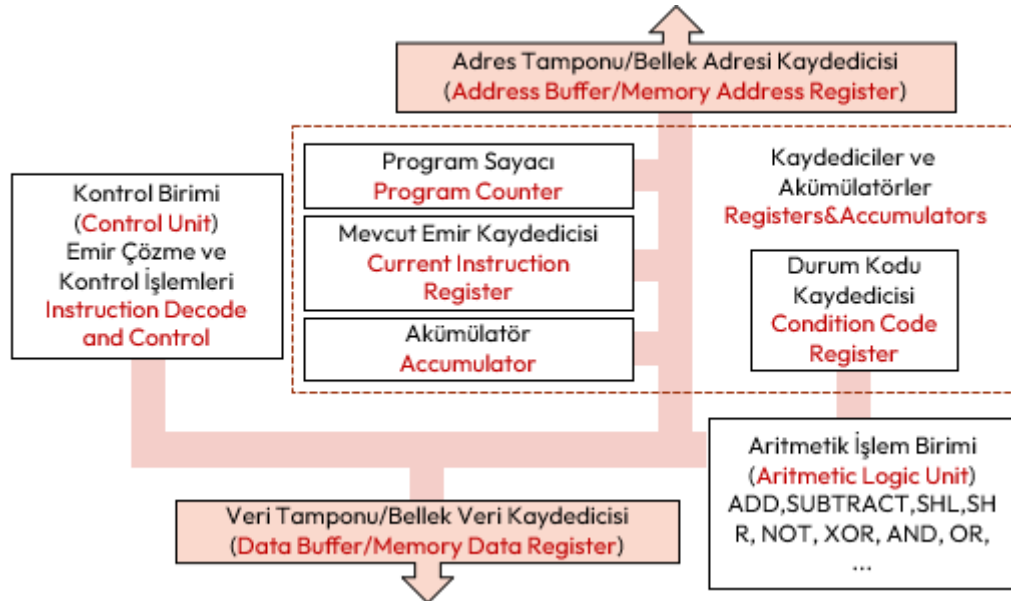


Şekil 1.2: Frekansı 1 olan Kare Dalga İşareti

1.5 Emri Alma, Çözme ve İcra Çevrimi

Basit bir işlemcideki bileşenleri aşağıdaki şekilde açıklayabiliriz;

- ◊ **Kaydediciler (register)**, işlemci içerisinde **çok hızlı ve geçici adres veya veri saklayan** alanlardır.
- ◊ **Akümülatörler (accumulator)** ise bellekten alınan verileri saklamak veya işlenen sonuçları saklamak için en sık kullanılan kaydedicilerdir.
- ◊ **Program sayacı PC (program counter)**, işlemcinin icra edeceği **emri (instruction)** takip için kullanılır. Getirilecek bir sonraki emrin bellek adresini içerir.
- ◊ **Mevcut Emir kaydedicisi CIR (current instruction register)** işlemcinin o an icra ettiği emirdir.
- ◊ **Koşul kodu kaydedicisi (condition code register)**, herhangi bir işlemin durumunu gösteren farklı bayraklar (**flag**) içeren kaydedicidir.
- ◊ **Aritmetik İşlem Birimi ALU (arithmetic logic unit)**, işlemcinin toplama çıkarma ve karşılaştırma yapan birimidir.
- ◊ **Kontrol Birimi CU (control unit)** Orkestra şefi gibidir, emirleri çözer ve yorumlar ve gerekli bileşenlere sinyal gönderir.
- ◊ **Veri Yolları (bus)**, verinin bileşenler arasında taşındığı birden çok elektrik hattıdır.



Şekil 1.3: Basit Bir İşlemcinin Yapısı

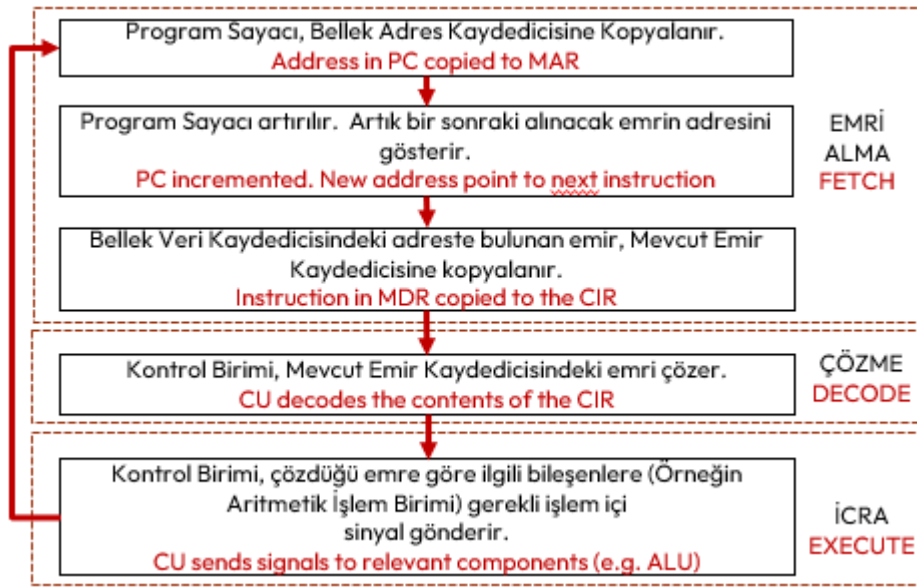
Bu işlemci tasarımı aynı zamanda **depolanmış program (stored program)** kavramı olarak adlandırılır ve **Von Neumann Mimarisi** olarak da bilinir. *John Von Neumann* tarafından 1946 ila 1949 yılları arasında tasarlanan ve Şekil 1.3'de verilen bu mimari günümüzde modern bilgisayarlarda hala kullanılmaktadır.

Basit bir bilgisayara elektrik verildiğinde, işlemci aşağıdaki üç adımlık **emir alma (fetch)**, **çözme (decode)** ve **icra (execute)** çevrimini elektrik kesilene kadar sürekli tekrarlanır². Bu çevrime **emir çevrimi (instruction cycle)** adı da verilir.

1. **Emri Alma (fetch)**: Bu aşamada program sayacının (**program counter**) gösterdiği adresteki emir (**instruction**) bellekten okunur:
 - (a) Program sayacı, bir sonraki emrin adresini tutar.
 - (b) Bu adres **bellek adres kaydedicisine (memory address register)** kop-

²<https://www.youtube.com/watch?v=ByllwN8q2ss>

- yalanır.
- (c) Talimat bellekten çağrılır ve **bellek veri kaydedicisine** (**memory data register**), oradan da **mevcut emir kaydedicisine** (**current instruction register**) aktarılır.
 - (d) Program Sayacı, bir sonraki emri işaret etmek için bir artırılır.
2. **Çözme** (**decode**): Kontrol birimi (control unit) tarafından emrin ne anlama geldiği ve buna bağlı nelerin yapılacağını (örneğin ulaşılacak bellek adresi değiştirilir) belirler:
- (a) Mevcut emir kodu (**current instruction code**), ne yapılacağını belirten kısım yani işlem kodu (**opcode**) ve hangi veriyle yapılacağını belirten işlenen (**operand**) kısımdan oluşur.
 - (b) Kontrol birimi, varsa emrin işaret ettiği veriyle işlemin gerçekleşmesi için gerekli donanım parçalarını hazırlar.
3. **İcra** (**execute**): Çözülen emir koduna bağlı olarak işlemcinin bazı bileşenleri çalıştırılır:
- (a) Eğer bir matematiksel işlem veya mantıksal karşılaştırma gerekiyorsa, bu görev aritmetik işlem birimini ALU (**Aritmetic Logic Unit**) tarafından yapılır. İşlem sonuçları ortaya çıkan değerler ise akümülatörlere (**accumulator**) veya bellek veri kaydedicisi (**memory data register**) üzerinden belleğe yazılır.
 - (b) Bazı durumlarda ise kaydedici (**register**) içerikleri değiştirilir.



Şekil 1.4: İşlemcinin Emri Alma, Çözme ve İcra Çevrimi

Burada çözülecek emirlerin (**instruction**) her biri bir ikili sayıya karşılık gelir. Bu sayılara **emir kodu** (**instruction code**), **makine kodu** (**machine code**) veya **işlem kodu** (**opcode**) olarak da bilinirler. İşlemciler tüm sayıları emir kodu olarak anlamaz. Yani sınırlı sayıda sayı emri tanırlar. Tablo 1.1'de verilen Motorola 6802 işlemcisi 72 farklı emri, Intel 8086 işlemcisi ise 116 farklı emri tanıır. Çözülebilecek tüm emirlerin oluşturduğu kümeye **emir seti** (**instruction set**) adı verilir.

1.6 İşlemci ve Bellek İş Birliğinin Temeli

Bir bilgisayarın çalışma mantığını anlamak için iki temel soruya yanıt vermemiz gerekir: "Veri nerede durur?" ve "İşlem nerede yapılır?"

§1.6.1 İşlem Nerede Yapılır?

İşlemci yani CPU (Central Processing Unit), bilgisayarın "beyni" veya bir mutfağın "şefi" gibidir. Yazdığınız kodlardaki tüm matematiksel hesaplamalar, mantıksal karşılaştırmalar ve karar verme süreçleri burada gerçekleşir. Ancak işlemcinin kendi içinde veri saklama kapasitesi çok sınırlıdır; o sadece önüne gelen işi o an yapmakla görevlidir.

§1.6.2 Veri Nerede Durur?

Bellek RAM (Random Access Memory), bilgisayarın **çalışma masası**'dır. Programınız çalışırken kullandığınız değişkenler, diziler ve nesneler geçici olarak burada tutulur. İşlemcinin bir veriyi işlemesi için, o verinin önce bellekten çağrılması gerekir. Bellek hızlıdır ancak "geçicidir"; bilgisayar kapandığında içindeki tüm veriler silinir.

Bu ilişkiyi daha somutlaştırmak için bir mutfağı hayal edelim:

- ◊ İşlemci (CPU): Yemek yapan aşçıdır. İşlemi (doğrama, pişirme) o yapar.
- ◊ Bellek (RAM): Aşçının önündeki kesme tahtası veya tezgâhtır. O an kullanılan malzemeler (veriler) burada durur.
- ◊ Sabit Disk (HDD/SSD): Mutfağın kileri veya buzdolabı gibidir. Tüm malzemeler orada saklanır (kalıcı depolama).

Programınızı çalıştırdığınızda, verileriniz kalıcı depolamadan (disk) alınır ve belleğe (RAM) yerleşir. İşlemci (CPU) ise bellekte hazır bekleyen bu verileri sırayla alır, işler ve sonucu tekrar belleğe yazar.

1.7 İşlemci Tasarlama Stratejileri

İşlemciler, aşağıda verilen üç strateji ile tasarlanırlar;

1. **İndirgenmiş Emir Setli İşlemciler** olan RISC (**Reduced Instruction Set Computing**) İşlemci: Az sayıda emir alan yüksek hızlı işlemci tasarımı. RISC işlemciler az sayıda emre sahip olduğundan emrin çözülmesi ve icrası da kısa zaman almaktadır. Bu da çalıştırılacak kodu büyütür.
2. **Karmaşık Emir Setli İşlemciler** olan CISC (**Complex Instruction Set Computing**) İşlemci: Çok fazla emri anlayan karmaşık nispeten yavaş işlemci tasarımı. CISC işlemciler, çok sayıda emir tanıyabildiğinden emrin çözülmesi ve icra edilmesi nispeten daha uzun sürer. Ancak yüksek düzey dillerin daha kolay makine koduna çevrilmesini kolaylaştırır.
3. **Özel İşlemci**: Bunlar grafik kartları ve yapay zekâ gibi belli amaçlar için tasarlanmış işlemcilerdir.

Genel olarak işlemcilerde emir çevrimi (**instruction cycle**) 1 saat periyodunda tamamlanmaz. RISC işlemciler için bu çevrim, birçok emir için 1 saat periyodunda tamamlanır. CISC işlemciler için ise birden fazla saat periyodu gerekir. Çünkü CISC işlemcilerin emir seti daha geniş olduğundan emrin çözülmesi daha fazla vakit alır. Bu nedenle RISC işlemciler CISC işlemcilere göreceli olarak daha hızlıdır.

Günümüzde ticari olarak satılan işlemcilerin çoğu ortak emir seti kullanmaya başlamışlardır;

- ◊ Masaüstü bilgisayarların birçoğu Intel ve AMD Marka işlemciler kullanır ve bunlar IA-16, IA-32, x86-16, x86-64, AMD64, ... emir setleri ve eklentilerini desteklemektedir.
- ◊ Bilinen birçok telefon işlemcisi Acorn Risc Machine-ARM tabanlı işlemciler kullanır. Bu işlemcilerde ARMv7, ARMv8, ... emir setleri ve eklentileri kullanılmaktadır.

1.8 Onaltılı Sayı Sisteminin Kullanılması

İşlemci ile bellek arasında verilerin elektrik yolları (**bus**) ile taşındığı anlatılmıştı. Veri taşıyan elektrik hattı sayısı işlemci ile bellek arasında taşınabilecek verinin sınırlarını da belirler. Bir elektrik hattının taşıyacağı veri bir bit ile (0 veya 1) gösterilebilir. İki elektrik hattının taşıyacağı veri ise iki bit ile (00, 01, 10 ve 11) olmak üzere 2^2 kadar farklı gösterilebilir. Benzer şekilde n adet elektrik hattının taşıyacağı veri 2^n farklı alternatifte veri olacaktır.

Bit sayısı arttıkça ikili sayının okunup yazılması da çok zor olacaktır. Günümüzde 64 bitlik hatta 1024 bitlik veri yoluna sahip bilgisayarlar bulunmaktadır. Bunun üstesinden gelmek için (**hexadecimal**) sayı sistemi kullanılır. Çünkü her 4 elektrik hattının verisi onaltılık sayı sisteminin bir rakamına karşılık gelir.

Onaltılı	İkili	Onlu	Onaltılı	İkili	Onlu
0	0000	1	8	1000	8
1	0001	2	9	1001	9
2	0010	3	A	1010	10
3	0011	4	B	1011	11
4	0100	5	C	1100	12
5	0101	6	D	1101	13
6	0110	7	E	1110	14
7	0111	8	F	1111	15

Tablo 1.2: Onaltılı Sayı Rakamları ve İkili ve Onlu Karşılıkları

Kod içerisinde onaltılık sayıların rakamlarının diğer sayı sistemindeki rakamlarla karışmaması için sonuna **h** konularak **113Ah** şeklinde veya çoğunlukla başına **0x** konularak **0x1133** şeklinde kullanılır. Örnek olarak ikili tabanda verilen 64 bitlik 0001 1010 1011 1101 0101 0100 1001 1010 0011 0010 0100 0110 1001 1101 1100 1111 sayısının onaltılı tabanda karşılığı oldukça kısa (16 hane) olarak 1ABD 549A 3246 9DCF şeklindedir.

Onaltılı sayının A,B,C,D,E ve F rakamları büyük harf olacağı gibi küçük harf de olabilir. Anlam olarak bir şey değişmez.

Bilgisayar dünyasında ikili sayıları onaltılı sayı sistemiyle ifade etmek çoğunlukla zorunluluktur. Zamanla standart hale gelmiştir.

1.9 Program ve Programlama

Belli bir işlemi gerçekleştirmek üzere, işlemciye verilen bir dizi emre (instruction), **bilgisayar programı** (**computer program**) denir. Bu bir dizi emrin, işlemcinin çalıştıracağı şekilde hazırlanması işlemine **programlama** (**programming**) denir. Programlama **emir kodları** (**instruction code**) ile yapıldığından bu programın dili, **makine dili** (**machine language**) olarak adlandırılır³.

Her işlemci üreticisi kullandığı emir seti farklı olduğundan yani her bir emre farklı makine kodu verdiğinden programlama oldukça zorlu ve teknik bilgi gerektirmekteydi. Burada daha önce yazılmış bir programın ne yaptığını anlamak için her defasında bu listelere defalarca bakmak gerekiyordu. Programın okunurluğunu artırmak için her bir emre, hatırlatıcı **sembolik isimler** (**mnemonic**) verilmeye başlandı. Tablo 1.3'de 6802 işlemcisinin birkaç sembolik ismi verilmiştir.

Sembolik İsim (Mnemonic)	Emir Kodu	Açıklama
ABA	1B	ADD A to B → A
INCA	4C	INCREMENT A
INCB	5C	INCREMENT B
SBA	10	SUBTRACT B from A → A
LDA	96	LOAD M to A, M: Memory
LDB	D6	LOAD M to B, M: Memory

Tablo 1.3: 6800 Emir Seti Örneği

Bu sembolik isimlerle yazılan program, **montaj kodu** (**assembly code**) olarak adlandırılır. Montaj kodları bir metin dosyasına yazılarak bir dosyaya saklanır. Bu dosyalar montaj dosyaları olarak adlandırılır. Montaj dosyalarını makine diline çeviren programlar yazılmıştır. Bu programlar ilk **derleyicilerdir** (**compiler**). Bu derleyiciler, sembolik isimlerle yazılmış program kodlarını makine koduna çeviren programlardır.

Aşağıda x86 emir seti kullanan Linux işletim sisteminde örnek montaj kodu örneği verilmiştir; Örnekte, genel amaçlı CL kaydedicisine koyulan sayıya kadar 2 ve 3'e bölünebilen sayıların toplamını bulup yine genel amaçlı DX kaydedicisine konulmaktadır⁴.

```
section .text ; programı oluşturan emir kodları buradan başlar
global _start

_start:
    mov dx, 0 ; dx 0 olsun. Toplamı tutacaktır.
    mov cl, 99 ; 99 kez tekrarlanacak.
    sum:
    mov ax, cx ; cx, ax e kopyalanır. Bölme işlemi için.
    mov bl, 3 ; bl 3 olsun.
    div bl ; ax/bl.
    cmp ah, 0 ; Kalan olup olmadığı kontrol ediliyor.
    jne cont ; Kalan yoksa, 'cont' etiketine git.
    mov bl, 2 ; Sonra bl 2 olsun.
    div bl ; ax/bl
    cmp ah, 0 ; Kalan olup olmadığı kontrol ediliyor.
    jne cont ; Kalan yoksa, 'cont' etiketine git.
    add dx, c ; Değilse, cx deki sayı 2 ve 3'e bölünür. Toplama ekle.
cont:
    loop sum ; Bir sonraki sayı için 'sum' etiketine dön.
exit:
    mov eax, 1 ; Artık programdan çıkıyoruz.
    mov ebx, 0 ; Return 0.
    int 80h ; linux çekirdeğini çağır. (soft interrupt 80h!).
```

1.10 Program Türleri

Geliştirdiğimiz programlar üç ana grupta incelenebilir;

³ISO/IEC 2382:2015. ISO. 2020-09-03. [Software includes] all or part of the programs, procedures, rules, and associated documentation of an information processing system

⁴<https://github.com/armut/x86-Linux-Assembly-Examples>

- ◊ **Sistem yazılımları**, bilgisayar donanımını yöneten ve uygulama yazılımları için bir platform sunan temel katmandır; işletim sistemleri (Windows, Linux) ve sürücüler bu grubun en net örnekleridir.
- ◊ **Uygulama yazılımları**, son kullanıcının belirli görevleri (metin yazma, internette gezinme, oyun oynama) yerine getirmesi için tasarlanmış, donanımdan ziyade kullanıcı odaklı araçlardır.
- ◊ **Gömülü yazılımlar** ise genel amaçlı bilgisayarlardan ziyade; beyaz eşyalar, otomobiller veya endüstriyel robotlar gibi belirli bir donanımı kontrol etmek üzere o cihazın içine hapsedilmiş, genellikle kısıtlı kaynakla çalışan ve yüksek kararlılık gerektiren özel kod parçalarıdır.

Aşağıdaki tabloda farkları yer almaktadır;

Özellik	Sistem Yazılımları	Uygulama Yazılımları	Gömülü Yazılımlar
Ana Hedef	Donanımı yönetmek	Kullanıcıya hizmet etmek	Cihazı çalıştırmak
Kullanıcı Etkileşimi	Arka planda çalışır	Doğrudan etkileşim kurar	Genellikle görünmezdir
Donanım Bağımlılığı	Çok yüksek	Düşük (Platform bağımlı)	Tam bağımlı
Örnek	Windows 11, macOS	Microsoft Word, Chrome	Çamaşır makinesi kontrolcüsü

Tablo 1.4: Yazılımlar ve Farkları

1.11 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri

Genel amaçlı yani herkesin kendi amacına göre kullanacağı bilgisayarlar, veri girişini sağlayan klavye, verileri görüntüleneceği monitör, işlenen verilerin elektrik kesildiğinde saklanacağı disk ve benzeri donanımlara sahip olmalıdır.

Genel amaçlı bilgisayarın kullanıcı isteklerine cevap verebilmesi için;

1. Çeşitli sistem yazılımlarına sahiptirler. İşte bu programlar bütününe işletim sistemi (**operating system**) adı verilir. DOS, Windows, Unix, Linux, MacOS, ChromeOS, OS/2 bunların en çok bilinenleridir.
2. Elektrik verildiğinde klavye, monitör ve disk olup olmadığı kontrol eden **BIOS (Basic Input Output System)** programı koşturulur. BIOS gerekli kontrollerden sonra bilgisayara işletim sistemini yükler. İşletim sistemi yüklendikten sonra, işletim sistemi kullanıcının vereceği komutları bekler.
3. Kullanıcı işletim sistemine vereceği komutlarla yeni bir program derleyebilir, derlenmiş bir programı çalıştırabilir.
4. Kullanıcının vereceği komutlar grafik olmayan işletim sistemlerinde (DOS, Unix, Linux gibi) komut yazılarak verilir. Komut yazılan bu ekrana **konsol (console)** adı verilir. Linux/MacOSX işletim sistemlerinde buna **terminal**, Windows işletim sisteminde ise **komut istemi (command prompt)** adı verilir. Performans için sunucularda (**server**) grafik arayüzleri tercih edilmez **konsol üzerinden sistem yönetilir**.
5. Grafik arayüzlerine sahip işletim sistemlerinde (Windows ve MacOS İşletim Sistemleri ile KDE, Mate, GNome, Cinnamon, LXQt, XFce ve Deepin gibi grafik arayüzleri içeren Linux İşletim Sistemleri) fare tıklamasıyla çoğu komut verilebilmektedir.

İşletim sistemlerinin yaygınlaşması ile kullanıcının bilgisayarı donanım bilmeden yapabilmesi sağlanmıştır.

Günümüzdeki işletim sistemlerinin birkaçı hariç tamamında temel programlama dili olarak C dili kullanılmaktadır.

1.12 Düşün ve Çöz

- ◊ **Düşün:** Bir mühendis tasarım sürecinde ne yapar, bir teknisyen ise ne yapar? Gerçek bir yazılım projesinden (örneğin bir banka uygulaması) hareketle her ikisinin görev dağılımını somutlaştırınız. Bu iki rol birbirinin yerine geçebilir mi?
- ◊ **Düşün:** CPU içindeki kaydedicilerin (**register**) hız ve kapasite açısından ana bellekten (RAM) farkı nedir? Tüm verilerimizi kaydedicilerde tutmak neden mümkün değildir? Bu kısıtlama işlemci mimarisini nasıl şekillendirir?
- ◊ **Düşün:** Von Neumann mimarisinde veri ve komutların aynı yolu (bus) paylaşması 'Von Neumann darboğazı' olarak bilinir. Bu darboğazın gerçek sistemlerde nasıl aşıldığını araştırınız. Ön bellek (cache) mimarisi bu soruna çözüm olarak nasıl konumlanır?
- ◊ **Düşün:** RISC işlemciler neden daha az komut içerir? ARM işlemcilerin mobil cihazlarda bu denli yaygınlaşmasının temel nedeni performans mı, yoksa güç tüketimi mi? İkisi arasındaki dengeyi tartışınız.
- ◊ **Düşün:** Onaltılı (**hexadecimal**) sayı sistemi neden bilgisayar dünyasında standart haline gelmiştir? İkili sistemin doğrudan kullanılması yerine neden onaltılıya ihtiyaç duyulur? Bellek adreslerinin onaltılıyla

gösterilmesinin programcıya faydası nedir?

- ◇ **Düşün:** Günümüzde bir akıllı telefonun işlemcisi (örneğin Apple M-serisi veya Qualcomm Snapdragon) ile 1970'lerin Motorola 6800 işlemcisi arasındaki temel mimari farklar nelerdir? Emir seti mimarisi açısından değerlendiriniz.
- ◇ **Düşün:** Bir işletim sisteminin var olmasının temel nedeni nedir? İşletim sistemi olmadan bir bilgisayarın kullanılabilmesi mümkün müdür? Gömülü sistemlerde işletim sistemi yerine doğrudan donanım programlamasının ne zaman tercih edildiğini araştırınız.
- ◇ **Çöz:** 24-bit adres yoluna ve 16-bit veri yoluna sahip bir işlemcinin adresleyebileceği maksimum bellek miktarını MB cinsinden hesaplayınız.
- ◇ **Çöz:** 16-bit adres yoluna sahip bir mikrodenetleyicinin adresleyebileceği maksimum bellek miktarını KB cinsinden hesaplayınız.
- ◇ **Çöz:** Bir bilgisayarın 8 GB RAM'i tam olarak adresleyebilmesi için adres yolunun (address bus) en az kaç bit olması gerektiğini hesaplayınız ve sonucu doğrulayınız.
- ◇ **Çöz:** İkili tabanda 1010 1100 0011 1111 sayısını onaltılı tabana dönüştürünüz. Ardından tersine çevirerek onaltılı 2F8A sayısını ikili tabana dönüştürünüz.
- ◇ **Çöz:** Bir işlemci 500 MHz saat frekansı ile çalışmaktadır. Her komut ortalama 4 saat çevriminde tamamlanıyorsa saniyede kaç komut icra edebilir? Aynı işlemci 2 GHz'e çıkarılırsa fark ne olur?

2. Bölüm

Temel Yazılım Kavramları

2.1 Düşük Düzey Programlama

1642 Yılında *Blaise Pascal* tarafından icat edilen Pascaline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 Yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 Yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar** (**punch card**) kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 Yılında *Charles Babbage* fark makinesi olan analitik makineyi icat etmiştir. Buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Byron*, Babbage ile beraber çalışmıştır. Byron, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi Bernoulli sayılarını üretmek için kullandı. Bu algoritmanın o dönemdeki "Analitik Makine" için devrimsel bir gelişmedir. Çünkü şartlı dallanma ve döngü mantığı içerir. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir.

Augusta Ada Byron, **ilk programcı** olarak kabul edilir.

1854 Yılında *George Boole* tarafından ikili sayı sistemini geliştirmiş bugün bilgisayarlarımızın kullandığı **bool cebiri** üzerine çalışmalar yapmıştır.

1890 Yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabilirdiği bir elektro mekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. Hollerith başarılı olunca bir şirket kurdu ve daha sonra üç firma birleşerek 1924 yılında adını **IBM** olarak değiştirdi.

1941 Yılında *Konrad Zuze* Z3 isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 Yılında *Howard Aitken*, IBM ile iş birliği yaparak **MARK-I**'i yaptı. Bilgiler, delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları, 18 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1943-1945 Yılları arasında *Konrad Zuze*, ilk yüksek düzey programlama dili olan Plankalkül'ü, Z1 bilgisayarı için geliştirmiştir.

Emir kodlarını (**instruction code**) ve sembolik isimleri (**mnemonic**) içermeyen programlama dilleri, **yüksek düzey programlama dili** (**high-level programming language**) olarak adlandırılır. Yüksek düzey programlama dilleri çoğunlukla İngilizce sözcükleri kullanır.

1946 Yılında *John Von Neumann* önderliğinde Pensilvanya Üniversitesinde **ENIAC** (Elektronik Sayısal Hesaplayıcı ve Doğrulamalı) isimli sayısal elektronik bilgisayar tamamlandı. Askeri amaçla üretildi ve top mermilerinin menzillerini hesaplamak için kullanıldı. Neumann tarafından geliştirilmiş bu mimari (**Stored-program computer and Universal Turing machine** § **Stored-program computer**), günümüzdeki bilgisayarlarda hala kullanılmaktadır. 18,000 adet elektronik tüp kullanılan ENIAC; 150 kW gücünde idi ve 50 ton ağırlığıyla 167 m² yer kaplıyordu. Saniyede 5.000 toplama işlemi yapabiliyordu. MARK-I bilgisayarı bin kat daha hızlıydı.

Yukarıda anlatılan bilgisayarlar, bir önceki bölümde anlatılan emir kodu (**instruction code**) veya sembolik isimlerle (**mnemonic**) yazılan programlar ile programlanmaktadır. Emir kodu ve sembolik isimlerle yapılan ve bilgisayar mimarisini bilmeyi gerektiren programlamaya **düşük düzey programlama** (**low-level programming**) olarak adlandırılır. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan matematikçiler bu programlama türünü kullanır.

Düşük düzey programlamanın yapılabilmesi için hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir.

2.2 Genel Amaçlı Yüksek Düzey Diller

Sembolik isimler veya makine kodu yerine bildiğimiz konuşma diline yakın talimatlarla metin olarak yazılan programlama dilleri, **yüksek düzey programlama dilleri** (**high-level programming language**) olarak adlandırılır. Yani emir kodu (**instruction code**) ve sembolik isim (**mnemonic**) kullanmayan programlama dilleri yüksek düzey programlama dilleridir.

Yüksek düzey dillerde **talimatlar** (**statement**) olarak yazılan programlar yine kendine has derleyiciler tarafından makine diline çevrilir. Her talimat, genellikle birden fazla emirden (**instruction**) oluşur.

Yüksek seviyeli bir dilde talimatlar, yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (**logical sequence**).

1954 Yılında **FORTRAN** (**FORmula TRANslation**), *John Backus* liderliğinde IBM 704 için geliştirildi. FORTRAN, **READ**, **WRITE**, **IF** gibi 32 farklı talimata (**statement**) sahiptir. İlk genel amaçlı, yüksek düzey bir programlama dilidir. Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN-II programı verilmiştir¹. Görüldüğü üzere talimatlar kısa öz ve anlaşılırdır.

```
C AREA OF A TRIANGLE - HERON'S FORMULA
C INPUT - CARD READER UNIT 5, INTEGER INPUT
C OUTPUT -
C INTEGER VARIABLES START WITH I,J,K,L,M OR N
READ(5,501) IA,IB,IC
501 FORMAT(3I5)
IF (IA) 701, 777, 701
701 IF (IB) 702, 777, 702
702 IF (IC) 703, 777, 703
777 STOP 1
703 S = (IA + IB + IC) / 2.0
AREA = SQRT( S * (S - IA) * (S - IB) * (S - IC) )
WRITE(6,801) IA,IB,IC,AREA
801 FORMAT(4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
$13H SQUARE UNITS)
STOP
END
```

1956 Yılında ticari amaçlarla kullanılabilen ve seri halde üretimi yapılan ilk bilgisayar **UNIVAC-I** oldu. UNIVAC, giriş-çıkış birimleri manyetik bant idi ve bir yazıcıya sahipti. UNIVAC 1951-1959 arasında üretilen bilgisayarlarda vakum tüpleri kullanıldı. Bu tüpler bir ampul büyüklüğünde, çok fazla enerji harcamakta ve çok fazla ısı yaymakta idiler. Veri ve programlar manyetik teyp ve tambur gibi bilgi saklama araçlarıyla saklandı. Veriler ve programlar bilgisayara delgi kartları ile yükleniyordu.

1959 yılı sonrasında üretilen bilgisayarlarda transistörler (yaklaşık 10 bin adet) kullanıldı. Bu yıl ve sonrasında transistör içeren **ikinci kuşak bilgisayarlar** hayatımıza girmiştir.

2.3 Arapsaçı Kod

1955-1959 Yılları arasında **FLOW-MATIC B0** (**Business Language Version 0**) Dili, *Grace Hopper* tarafından UNIVAC-I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 programlama dilinde örnek bir program verilmiştir².

```
(0) INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1) COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2) TRANSFER A TO D .
(3) WRITE-ITEM D .
(4) JUMP TO OPERATION 8 .
```

¹https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f

²<https://gist.github.com/marcgg/f9f2da31a973ba319809>

- (5) **TRANSFER** A TO C .
- (6) **MOVE** UNIT-PRICE (B) TO UNIT-PRICE (C) .
- (7) **WRITE**-ITEM C .
- (8) **READ**-ITEM A ; **IF** **END** OF **DATA** **GO TO** OPERATION 14 .
- (9) **JUMP** TO OPERATION 1 .
- (10) **READ**-ITEM B ; **IF** **END** OF **DATA** **GO TO** OPERATION 12 .
- (11) **JUMP** TO OPERATION 1 .
- (12) **SET** OPERATION 9 TO **GO TO** OPERATION 2 .
- (13) **JUMP** TO OPERATION 2 .
- (14) **TEST** **PRODUCT**-NO (B) AGAINST ZZZZZZZZZZ ; **IF** **EQUAL** **GO TO** OPERATION 16 ;
OTHERWISE **GO TO** OPERATION 15 .
- (15) **REWIND** B .
- (16) **CLOSE**-OUT FILES C ; D .
- (17) **STOP** . (**END**)

Görüldüğü üzere bu tarihlere kadar yazılan programlarda bir sürü **GO TO** ya da **JUMP TO** talimatı (**statement**) bulunmaktadır. Bu durumda programın ne yaptığı konusunda fikir yürütmeye çalıştığımızda **okunaklılık oldukça azdır**. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod** (**spaghetti code**) adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

İşin doğası gereği **makine diline çevrilecek tüm kaynak kodlar gözle okunmalıdır!**

1958 Yılında **LISP** programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş ikinci yüksek düzey programlama dilidir;

```
(format t "Adinizi Giriniz: ")
(setq name (read))
(princ "Merhaba ")
(write name)
```

1959 Yılında **COBOL** (**Common Business-Oriented Language**) programlama dili, CODASYL komitesi tarafından İngilizceye benzer ikinci yüksek düzey bir dil olarak geliştirilmiştir;

```
IDENTIFICATION DIVISION.
PROGRAM-ID. GREET.
```

```
DATA DIVISION.
WORKING-STORAGE SECTION.
01 USER-NAME PIC A(30).
```

```
PROCEDURE DIVISION.
DISPLAY "Enter your name: ".
ACCEPT USER-NAME.
DISPLAY "Hello, " USER-NAME "!".
STOP RUN.
```

1958 Yılında daha sonra **ALGOL** (**ALGOritmic Language**) adını alan **IAL** (**International Algebraic Language**) programlama dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok dile önderlik etmiştir. Bu dil ile birlikte kod blokları (**code block**), iç içe fonksiyon (**nested function**), değişken kapsamı (**scope**) ve dizi (**array**) kavramları programcının hayatına girmiştir. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur;

```
BEGIN
FILE F (KIND=REMOTE);
EBCDIC ARRAY E [0:11];
REPLACE E BY "HELLO WORLD!";
WRITE (F, *, E);
END.
```

1958 Yılındaki FORTRAN diline diline çıktı üretmeyen fonksiyonlar için **SUBROUTINE**, değer üreten fonksiyonlar için **FUNCTION**, **CALL** ve **RETURN** özellikleri eklenmiş ve **FORTRAN-II** adını almıştır.

Benzer şekilde 1958 ile 1969 yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için **PROCEDURE** ve **BEGIN-END** arasında kod bloğu özellikleri eklenmiştir.

1966 Yılındaki FORTRAN-IV diline **INTEGER**, **REAL**, **DOUBLE PRECISION**, **COMPLEX** ve **LOGICAL** veri tipleri (**data type**) ile Mantıksal **IF**, aritmetik (üç yollu) **IF** ve **DO** döngüsü talimatları (**statement**) eklenmiştir.

1964 yılında Dartmouth College’da John G. Kemeny ve Thomas E. Kurtz tarafından geliştirilen **BASIC** (**B**eginner’s **A**ll-purpose **S**ymbolic **I**nstruction **C**ode) öğrenilmesi ve kullanılması kolay olacak şekilde tasarlanmış bir programlama dilidir. Bilimsel olmayan alanlardaki öğrencilerin ve yeni başlayanların bilgisayar kullanabilmesini sağlamak amacıyla İngilizce benzeri komutlarla tasarlanmıştır. Karmaşık donanım detaylarını gizleyerek programcının mantığa odaklanmasına izin verir. Kodlar genellikle satır satır gerçek zamanlı olarak çalıştırılır, bu da hataların anında bulunmasını kolaylaştırır. 1980’li yılların başında kadar ev bilgisayarlarının (Apple II, Commodore 64, IBM PC gibi) varsayılan dili haline gelmiştir;

```
10 REM A simple program to greet the user
20 CLS ' Clears the screen (command might vary by implementation)
30 PRINT "What is your name? ";
40 INPUT NAME$ ' The '$' indicates a string variable
50 PRINT "Hello, "; NAME$; "!"
60 GOTO 30 ' Jumps back to line 30 to run the program again
70 END ' Terminates the program
```

2.4 Değişken

Değişkenler (**variable**) aslında cebirde kullandığımız değişkenlerdir. Cebir (**algebra**) işlemlerinde doğru-
dan problemde verilen sayıları değil, onları temsil eden değişkenleri kullanırız.

$$3x^2 + 2x + 10$$

$$c^2 = a^2 + b^2$$

$$c = 2\pi r$$

Yukarıdaki cebirsel ifadelerin ilkinde bir polinom, ikincisinde ise bir dik üçgenin kenar uzunlukları arasındaki ilişki verilmiştir. Bu ifadelerde x , a , b ve r **bağımsız değişken**, c ise **bağımlı değişkendir**. Örneklerdeki 3,2,10,2 ve π ise **sabit** (**constant**) sayılardır.

Değişkenler, bilgisayarlarda bir miktar bellek bölgesini işgal eder ve bu bellek bölgesinde çeşitli biçimlerde tutulurlar.

Tam sayılar, ikili (**binary**) sayı olarak bellekte tutulur. Tam sayıyı ifade eden ikili sayının en anlamlı biti, işaret (**sign**) biti olarak adlandırılır ve bu bitin değeri 1 ise sayı negatif, 0 ise sayı pozitiftir. Tablo 2.1’de en çok kullanılan tam sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
8 Bit	1 bayt	-127 ile +128 arasında veya 0 ile 255 arasında
16 bit	2 bayt	-32.767 ile +32.768 arasında veya 0 ile 65.535 arasında
32 bit	4 bayt	-2.147.483.648 ile +2.147.483.647 arasında veya 0 ile 4.294.967.295 arasında
64 bit	8 bayt	-9.223.372.036.854.775.808 ile 9.223.372.036.854.775.807 arasında veya 0 ile 18.446.744.073.709.551.615 arasında

Tablo 2.1: Tam Sayılar ve Sayı Sınırları

n bitlik bir tamsayı işaretli (**signed**) olarak kullanılacaksa değer aralığı -2^{n-1} ile $2^{n-1} - 1$ arasında olacaktır. Aynı bit sayısına sahip sayı, işaretli (**unsigned**) olarak kullanılacak ise 0 ile $2^n - 1$ aralığında değer alabilecektir.

Gerçek sayılar, kayan noktalı sayı (**floating point number**) olarak biçimlendirilir. Kayan noktalı sayı kavramı ilk olarak 1914 yılında Leonardo Torres Quevedo tarafından Charles Babbage’ın analitik makinesine kullanılmak üzere yayınlanmıştır.

$$12345 = 1234,5 \times 10^1 = 123,45 \times 10^2 = 12,345 \times 10^3 = 1,2345 \times 10^4 = 0,12345 \times 10^5$$

Yukarıda ondalık tabanda verilen gerçek bir sayıya ilişkin kesir noktasının kaydırılması ile aynı sayı ifade edildiği bir örnek verilmiştir. Kesir noktası sola kaydırıldığında sayı, 10 ile çarpılmış, sağa kaydırıldığında ise sayı 10’a bölünmüş olur. Kayan noktalı sayının üs kısmı dışında kalan kısmı taban veya **kesir** (**fraction**) olarak adlandırılır. Tabanın kesir noktası sağında kalan hanelerine ise **mantis** (**mantissa**) adı verilir. Tablo 2.2’de en

çok kullanılan kayan noktalı sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir³.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
32 bit	4 bayt	$1,2 \times 10^{-38}$ ile $3,4 \times 10^{+38}$ arasında tek hassasiyetli gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 8 bit üs ve geri kalan 23 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.
64 bit	8 bayt	$2,3 \times 10^{-308}$ ile $1,7 \times 10^{+308}$ arasında çift hassasiyetli gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 11 bit üs ve geri kalan 52 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.

Tablo 2.2: Kayan Noktalı Sayılar ve Sınırları

1938 yılında *Konrad Zuse*, ilk ikili (**binary**) programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır.

Tam sayılarda olduğu gibi **kayan noktalı sayılar da bilgisayarlarda ikili (binary) sayı tabanında tutulur.**
Örneğin ondalık tabanda 50,25 sayısı ikili tabanda 1.1001001×2^5 olarak ifade edilir.

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi (data type)** belirler. Değişkenlerin veri tipine kısaca **değişken tipi (variable type)** adı da verilir. Veri tiplerine verilen adlar dilden dile değişir; Örneğin PASCAL dilinde 8 bitlik tam sayıya **bayt** denilirken, C, C++ ve C# dilinde **char** adı verilir. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

2.5 Fonksiyon

Matematikte **fonksiyon (function)**, bir kümenin bir veya daha fazla kümenin elemanını tek bir kümenin elemanına eşleyen **ilişkidir (relationship)**.

Girdi	İlişki	Çıktı
1	x2	2
2	x2	4
3	x2	6
4	x2	8
...		

Tablo 2.3: İki İle Çarpma Fonksiyonu

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematikte aşağıdaki şekilde **ifade (expression)** edilir.

$$f(x) = 2x$$

Burada f, fonksiyon anlamında x ise girdi anlamında olup **parametre** olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x$$

Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$x \in R, f(x) = 2x + 33$$

Bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan f(x) değerine eşleyen bir ifadedir. Yani x=1.0 argümanını f(1.0)=5.0 reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp **döndürür (return)**.

$$g(a, b) = 2a + b$$

Bu fonksiyon ise reel sayı kümesindeki her a elemanı ile b elemanını, hesaplanan g(a,b) değerine eşleyen bir ifadedir. Yani a=1,0 ve b=1,0 argümanlarını g(1,0,1,0) fonksiyonu yine reel sayı kümesinden 3,0 reel sayısına eşler. Kısaca bu fonksiyon, 3,0 reel sayısını hesaplayıp döndürür (**return**).

Örneklerdeki x, a ve b parametre olarak adlandırılır. f(1,0) ifadesindeki 1,0 ise parametrenin o an andığı değeri belirtir ve **gerçek parametre (actual parameter)** ya da **argüman (argument)** olarak adlandırılır.

³IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1–84.

$$f(3.0) + g(3.0, 2.0) + f(2.0) + g(1.0, 3.0)$$

Fonksiyonlar ifade olarak tanımlandıktan sonra yukarıdaki gibi tekrar tekrar **çağrılabilirler (call)**.

2.6 Yapısal Programlama

1970'li yıllara kadar her ne kadar çıktı üretmeyen fonksiyonlar için **SUBROUTINE**, birden fazla çağrı noktası olabilen **COROUTINE** ve **FUNCTION** kullanılıyor olsa da hala **GO TO** ve **JUMP TO** ifadeleri kullanılmakta ve kafa karışıklığına devam etmekteydi.

1968 Yılında *Edsger W. Dijkstra* önderliğinde **GO TO** ve **JUMP TO** ifadeleri zararlı olarak ilan edilmiştir.

1970 Yılında *Niklaus Wirth* tarafından **Pascal** dili geliştirildi. Bu dil ilk geliştirilen yapısal programlama dilidir. **Yapısal programlamada (structural programming)** verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (**if**, **for**, **do**, **repeat**) birbirinden ayrılmıştır. Pascal, kendi kendini çağıran **özyinelemeli (recursive)** fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri (pointer)** desteklemektedir⁴. Aşağıda bir sayısı konsoldan alan ve faktöriyelini hesaplayıp sonucu yazdıran bir program örneği verilmiştir;

```
program FaktoriyelHesaplama;
{ Faktöriyel hesaplayan fonksiyon }
function Faktoriyel(n: integer): longint;
var {Fonksiyon terelindeki değişken tanımları}
i: integer;
sonuc: longint;
begin
sonuc := 1;
for i := 1 to n do
sonuc := sonuc * i;
Faktoriyel := sonuc; { Pascal'da sonuç fonksiyon ismine atanarak döndürülür }
end;

var {Ana fonksiyon yerel değişkenleri}
sayi: integer;
begin {Ana Program Başlangıcı}
writeln('Faktoriyeli hesaplanacak sayiyi giriniz: ');
readln(sayi);
if sayi < 0 then
    writeln('Negatif sayıların faktoriyeli tanımsızdır.')
else
    writeln(sayi, ' != ', Faktoriyel(sayi)); //Fonksiyon Çağrılıyor
end. {Ana Program Bitişi}
```

Yapısal programlamada programlama, fonksiyonların birbirini çağırmasıyla yapılır. **Yapısal programlamada veri ile veriyi işleyen yapılar birbirinden ayrıldığından arapsaçı programlamanın aksine okunaklılık çok yüksektir.**

2.7 Yapısal Programlama Genel Çerçevesi

1. Programın başladığı yeri belirten bir **ana fonksiyon (main function)** tanımlanır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar olarak bu ana fonksiyonda yazarız.
2. Her fonksiyonda ilk önce **veri yapıları (data structure)** tanımlanır. Veri yapısı;
 - (a) En basit veri yapısı **char**, **integer**, **float** ve **double** gibi **ilkel veri tiplerinden (primitive data type)** olabilen **değişken (variable)**,
 - (b) İlkel veri tiplerinden meydana gelen **dizi (array)**,
 - (c) İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar (structure)**,
 - (d) Ya da dinamik olarak yani çalışma anında (**run-time**) birbirine bağlanmış verilerden oluşan **bağlı liste (linked list)** olabilir.
3. Her fonksiyonda tanımlanan veri yapılarının ardından bu yapıları işleyecek **kontrol yapıları (control structure)** tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe **if**, **if-else**, **switch**, **while**,

⁴Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

`do-while`, `for`, `continue`, `break`, `goto` ve `return` talimatlarından (**statement**) oluşur.

Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon nokta ile biten **BEGIN-END**. bloğudur. C dilinde ana fonksiyon, `main()` fonksiyonudur.

C Programlama Dili, *Dennis M. Ritchie* tarafından Bell Laboratuvarlarında UNIX işletim sistemini geliştirmek için geliştirilen genel amaçlı, üst düzey bir dildir. C, ilk olarak 1972'de DEC PDP-11 bilgisayarında çalıştırılmıştır. *Ken Thompson* tarafından geliştirilen B adlı daha eski bir dilden gelişmiştir. 1978 yılında *Brian Kernighan ve Dennis Ritchie*, günümüzde K&R standardı olarak bilinen C'nin ilk kamuya açık kılavuzunu hazırlamışlardır.

C programlama dili; Öğrenmesi kolaydır, **yapısal programlama dilidir**, hızlı ve verimli programlar üretir, assembly dili desteği ile **düşük düzey programlamayı** (**low-level programming**) da destekler ve standart başlık dosyaları sayesinde çeşitli bilgisayar platformlarında derlenebilir. Bu özellikleri nedeniyle neredeyse tüm işletim sistemlerinin çekirdeği C dilinde yazılmıştır ve sistem dili olarak da adlandırılmaktadır. Yukarıda verilen Pascal yapısal programının C dilinde kodlanmış hali aşağıda verilmiştir;

```
#include <stdio.h>
/* Faktöriyel hesaplayan fonksiyon */
long Faktoriyel(int n) {
    int i; /* Fonksiyon yerelindeki değişken tanımları */
    long sonuc = 1;
    for (i = 1; i <= n; i++) {
        sonuc = sonuc * i;
    }
    return sonuc; /* Pascal'daki 'Faktoriyel := sonuc' ifadesinin C karşılığı */
}

int main() { /* Ana Fonksiyon Başlangıcı */
    int sayi; /* Ana Fonksiyon Yerel Değişkenleri */
    printf("Faktöriyeli hesaplanacak sayiyi giriniz: ");
    scanf("%d", &sayi);
    if (sayi < 0) {
        printf("Negatif sayıların faktöriyeli tanımsızdır.\n");
    } else {
        /* Fonksiyon çağrılıyor ve sonuç ekrana yazdırılıyor */
        printf("%d! = %ld\n", sayi, Faktoriyel(sayi));
    }
    return 0;
} /* Ana Fonksiyon Bitişi */
```

2.8 Nesne Yönelimli Programlama İhtiyacı

C++ programlama dili, C diline yapılan nesne yönelimli programlama OOP (**object-oriented programming**) eklentileri yapılarak geliştirilmiş bir programlama dilidir. 1979 senesinde bir Danimarkalı bilgisayar bilim adamı olan *Bjarne Stroustrup*, sonradan C++ olarak bilinecek olan **C with Classes** üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı.

Yapısal programlamada talimatlar (**statement**) art arda koda yazılarak programlama yapılı ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya ise emreden programlama (**imperative programming**) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler. Emreden paradigmanın bir örneği olan yapısal programlamada zamanla aşağıdaki problemler ortaya çıkmıştır;

- ◊ Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. Değişim yönetimi (**change management**) çok zordur ve uzun zaman alır.
- ◊ Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda hataların izini sürmek (**error handling**) işi zorlaştırır.
- ◊ Yapısal programlamada, gösterici (**pointer**) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun güvenli (**safe code**) olarak çalışmasının incelenmesi çok zordur.
- ◊ Sürekli olarak benzer projelerde aynı kodları tekrar yazmak (**duplicate code**) gereklidir.
- ◊ Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır.

Bu nedenle 1980'li yıllarda birçok yazılım projesi başarısız (**fail**) olmuştur.

Bir yazılım; Kullanıcı ihtiyaçlarının karşılanamaması, Öngörülen bütçenin aşılması ve Zamanında teslim edilememesi durumlarında başarısız olur. Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillerindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi.

Smalltalk, 1972 yılında Xerox Park şirketinde Alan Kay önderliğinde Dan Ingalls, Adele Goldberg, Ted Kaehler, Diana Merry, Scott Wallace ve Peter Deutsch tarafından geliştirilmiş bir dildir. Smalltalk dilinde nesneler (**object**) temel yapıtaşlarıdır. Nesneler;

- ◊ Durum (**state**) ve davranışlara (**behavior**) sahiptir.
- ◊ Programlama, nesnelerin birbirlerine ileti göndermesiyle (**message-passing**) yapılır.
- ◊ Nesnelerin kendi ya da bir başka nesnenin davranış ve durumlarını öğrenebilme yeteneği yani yansıma (**reflection**) özelliği vardır.

Yapısal programlamanın içinde olduğu emreden paradigmanın (**imperative programming**) aksine, **nesne yönelimli programlama** (OOP) paradigmasında programlama, nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılır.

C++, nesne yönelimli programlama kavramlarına sahip C programlama dilinin bir uzantısıdır. Bu nedenle **saf** nesne yönelimli programlama dili değildir. C# programlama dili, Anders Hejlsberg, Scott Wiltamuth ve Peter Golde tarafından Microsoft Şirketinde geliştirilip ilk olarak Temmuz 2000'de dağıtılmıştır. C# dilindeki karma (#) karakteri, C++ diline de eklenti (++) yapılarak dört adet + karakterinin bir araya gelmesi anlamındadır.

2016 yılına kadar kapalı kaynak (**closed source**) olarak .NET platformunun bir parçası olan bu dil 2016 yılında açık kaynak (**open source**) haline gelmiştir. Şu an için Windows, Linux ve macOS işletim sistemleri için ücretsiz ve açık kaynaklı bir **yazılım çerçevesi** (**framework**) olarak kullanıcılara sunulmuştur. C# programlama dili birden fazla paradigmayı destekleyen genel amaçlı (**general purpose**), **saf nesne yönelimli** üst düzey programlama dilidir (**high level programming language**).

2.9 Nesne Yönelimli Programlama

Yapısal programlamada talimatlar (**statement**) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya emreden programlama paradigması (**imperative programming**) adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.

Emreden programlama (**imperative programming**) bir örneği olan yapısal programlamada talimatlar (**statements**) art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Kodlar büyüdükçe sorunlar artmış ve 2.8 başlığı altında anlatılan problemler ortaya çıkmıştır. Bu yıllarda Simula ve Smalltalk yaklaşımı yazılımcıların imdadına yetişmiştir.

Bu dillerde nesneler programın (**objects**) temel yapıtaşlarıdır. Nesneler; durum (**state**) ve davranışlara (**behavior**) sahiptir. Programlama, nesnelerin birbirlerine ileti göndermesiyle (**message-passing**) yapılır. Emreden paradigmanın aksine **nesnelerin birbirlerine ileti göndermesi** bakış açısıyla yapılan programlamaya **nesne yönelimli programlama** (**object oriented programming**) paradigması adı verilir.

C# programlama dili, C++ ve C dilinden daha fazla kütüphaneye sahiptir. Ayrıca bu dil, Java gibi derlenmiş kodun Windows ve Linux ve gibi platformlar arasında taşındığında çalışabilmesi (**interoperability**) için .NET altyapısı üzerinde çalışır. Birçok kavramı bir anda özümsemek gerektiğinden **öğrenme eğrisi diktir**.

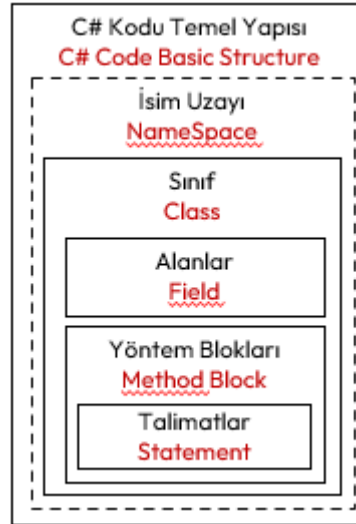
2.10 Nesne Yönelimli Programın Genel Çerçevesi

C# dili, Java dili gibi **saf nesne yönelimli bir dildir**. Nesne olmadan program yazılamaz. Nesne yönelimli programlamada;

- ◊ Nesneler, durum (**state**) ve davranışlara (**behavior**) sahiptir.
- ◊ Kodlaması yapılacak iş, nesnelerin birbirlerine ileti göndermesiyle (**message-passing**) yapılır.

2.7 Başlığında yapısal programın genel çerçevesi anlatılmıştır. Nesne yönelimli programlamada ise imal edilecek nesnelerin **hangi durum ve davranışa sahip olacağı** programcı tarafından kullanıcı tanımlı **sınıflarla** (**class**) belirlenir. Bu nedenle **nesne kullanan her C# programı en az bir sınıf içerir!** Kodun genel yapısı aşağıda verilmiştir;

```
1 using System; // System adındaki isim uzayındaki bileşenleri kullanacağız!.
2 namespace ÖrnekUygulama // Örnek Uygulama adında bir isim Uzayı
```



Şekil 2.1: C# Kodu Temel Yapısı

```

3  { // Blok Baş1
4      class Sınıf // "Sınıf" adında bir Sınıf.
5      {
6          // Alan Değişkenleri (fields):
7          int yaş = 5;
8
9          // Özellikler (properties):
10         public int Yaş
11         {
12             get { return yaş; }
13             set { yaş = value; }
14         }
15         public string Adi { get; set; }
16
17         // Davranışlar/Yöntemler (methods):
18         public void BilgileriYazdırYöntemi()
19         {
20             Console.WriteLine($"Yaş: {Yaş}, Adı: {Adi}"); //Talimat (statement)
21         }
22     }
23 } // Blok Sonu

```

Bir sınıf (**class**) her zaman bir **namespace** ile belirtilen bir **isim uzayında** olmak zorunda değildir. Ama **sınıflar**;

- ◊ Durum verisini tutacak **alan değişkenlerine** (**field**),
- ◊ Nesne dışına kontrollü açılan **Özelliklere** (**property**) ve
- ◊ Davranışları gösteren **yöntemlere** (**method**) sahip olabilir.

Yöntemler yapısal programlamadaki fonksiyonlar gibi tanımlanır ve içinde bir dizi **talimat** (**statement**) yer alır.

Yapısal programlamada ilk çalışacak olan ana fonksiyondur (**main function**). Yani program ana fonksiyonla başlar. Nesne yönelimli programlamada ise programın başladığı yer için sınıfta statik bir ana fonksiyon yani **static void Main(string[] args)** tanımlanır. Program çalışmaya bu yöntemi icra ederek başlar. Böylece derleme sonrasında icra edilebilir (**executable**) bir **.NET uygulaması** (**application**) ortaya çıkar. Bu yöntemin olmadığı kodlar **sınıf kütüphanesi** (**class library**) olarak adlandırılır.

Bir proje birden fazla kod dosyasından oluşabilir. Her bir sınıf ayrı bir dosyada yer alabilir. Tüm projeyi oluşturan bu kodlarının tamamına **kod tabanı** (**codebase**) adı verilir. Aşağıda yukarıdaki sınıftan nesne imal edip kullanan bir konsol uygulaması verilmiştir;

```

using System; // System adındaki isim uzayındaki bileşenleri kullanacağız!.
namespace ÖrnekUygulama // ÖrnekUygulama adında bir isim Uzayı

```

```

{
    class Sınıf // "Sınıf" adında bir sınıf
    {
        // Alan Değişkenleri (fields):
        int yaş = 5;
        // Özellikler (properties):
        public int Yaş
        {
            get { return yaş; }
            set { yaş = value; }
        }
        public string Adi { get; set; }

        // Metotlar (methods):
        public void BilgileriYazdırYöntemi()
        {
            Console.WriteLine($"Yaş: {Yaş}, Adı: {Adi}");
        }
    }
    class Program // Programı başlatacak Main yöntemini içeren sınıf
    {
        static void Main(string[] args) // Programın başlangıç noktası
        {
            Sınıf nesne = new Sınıf(); // Sınıftan "nesne" adlı nesne imal edildi
            nesne.Yaş = 10; // nesnenin Yaş özelliğine değer atama
            nesne.Adi = "Ali"; // nesnenin Adi özelliğine değer atama
            nesne.BilgileriYazdır(); // nesnenin yöntemini çağırarak nesneye ileti-gönderme
        }
    }
}

```

2.11 Soyutlama Merdiveni

Yazılım dünyasında soyutlama merdiveni (**abstraction ladder**), bir programlama dilinin bilgisayar donanımına (işlemci ve bellek) ne kadar yakın veya insan diline (mantık ve söz dizimi) ne kadar benzer olduğunu gösteren bir hiyerarşidir.

Seviye	Tanım	Örnekler	Donanım Yakınlığı
Donanım	Elektrik sinyalleri, kapılar ve transistörler.	İşlemci (CPU), RAM	%100
Makine Dili	İşlemcinin doğrudan anladığı 0 ve 1'ler (binary).	01101010...	Çok Yüksek
Assembly	Emir kodunun (instruction) kısa kelimelerle temsilidir.	x86 ASM, ARM ASM	Yüksek
Düşük Düzey	Donanım üzerinde tam kontrol, manuel bellek yönetimi.	C, Rust	Orta-Yüksek
Yüksek Düzey	Nesne yönelimli yapılar, otomatik bellek yönetimi.	C#, Java, C++	Orta-Düşük
Ultra Yüksek	İnsan diline en yakın, yüksek soyutlama seviyesi.	Python, Ruby	Düşük

Tablo 2.4: Soyutlama Merdiveni

Merdivende tırmandıkça veya indikçe iki temel değişken arasında bir takas (**trade-off**) gerçekleşir.

C, Assembly Tarafına Doğru İlerledikçe:

- ◆ **Performans Artar:** Kod doğrudan donanıma hitap eder, aracı katmanlar azalır.
- ◆ **Kontrol Artar:** Belleğin her bir baytını siz yönetirsiniz.
- ◆ **Zorluk Artar:** Yazması ve hata ayıklaması (**debugging**) çok daha zordur.

C#, Python Tarafına Doğru İlerledikçe:

- ◆ **Verimlilik Artar:** Az kodla çok iş yapılır (Üretkenlik).
- ◆ **Güvenlik Artar:** Bellek yönetiminde çöp toplama GC (**garbage collector**) gibi süreçler dil tarafından

otomatik yapılır.

- ♦ **Taşınabilirlik Artar:** Kodunuzun farklı işletim sistemlerinde ve işlemcilerle çalışması daha kolaydır.

Soyutlamayı anlamak için araba örneğini düşünelim:

- ♦ **Düşük Düzey (C/Assembly):** Arabanın motorunu, pistonlarını ve yakıt enjeksiyonunu doğrudan yönetmektir. Arabanın nasıl çalıştığını bilmeniz gerekir.
- ♦ **Yüksek Düzey (C#/Python):** Sadece direksiyon, gaz ve freni kullanmaktır. Motorun içindeki karmaşık detaylar sizden soyutlanmıştır. Siz "hızlan" (gaz) dersiniz, alt katmanlar bunu motorun anlayacağı dile çevirir.

2.12 Derleyici ve Yorumlayıcı

Programlama dilleri ile yazdığımız kodlar insan tarafından okunabilir (**high-level**) metinlerdir. Ancak bilgisayarın beyni olan işlemci, sadece "0" ve "1"den oluşan makine dilini anlar. Bu iki dünya arasındaki çeviriyi yapan iki temel araç vardır. Bunlar derleyici (compiler) ve yorumlayıcıdır (interpreter).

§2.12.1 Derleyici

Derleyici (compiler), yazdığınız kaynak kodun tamamını bir kerede okur ve bu kodun karşılığı olan bir icra edilebilir ikili dosya (Windows'ta .exe gibi) oluşturur. Derleme işlemi çalıştırılmadan önce bir kez yapılır. Derleme sonunda oluşan icra edilebilir dosya, işletim sistemi ve CPU tarafından defalarca çalıştırılabilir.

Derleme işlemi, tüm kitabı bir kerede çevirip yeni bir dilde matbaaya basan bir tercüman gibidir. Program, **bir kez derlendikten sonra çok hızlı çalışır**. Kaynak kodun paylaşılmasına gerek kalmadan **sadece icra edilebilir ikili dosya dağıtılabilir**. Aynı işletim sistemi ve aynı emir setini kullanan işlemcilerde doğrudan çalışır. Ancak kaynak kodda **küçük bir değişiklik yapılırsa bile tüm programın tekrar derlenmesi gerekir**. **Hatalar ancak derleme işlemi bittiğinde topluca görülür**.

§2.12.2 Yorumlayıcı

Yorumlayıcı (interpreter), kaynak kodu satır satır okur ve her satırı okuduğu anda işlemciye ileterek çalıştırır. Ortada kalıcı bir çıktı dosyası oluşmaz; **programın çalışması için yorumlayıcının her zaman aktif ve çalışıyor olması gerekir**. Yorumlayıcı, bir konuşmacının söylediklerini eş zamanlı olarak dinleyiciye çeviren bir simültane tercüman gibidir. Kod yazılırken **hatalar anında tespit edilebilir**. Program, yorumlayıcısının olduğu her işletim sisteminde (Windows, Linux, macOS) **değişiklik yapmadan çalışabilir**. Böylece platform bağımsızlığı sağlanır. Ancak her kod satırı çalışma anında işlemcinin anladığı emirlere çevrildiği için **derlenmiş programlara göre daha yavaş çalışır**.

Özellik	Derleyici (Compiler)	Yorumlayıcı (Interpreter)
İşleyiş	Kodun tamamını bir kerede çevirir.	Kodun her satırını sırayla çevirir.
Hız	Çalışma anında daha hızlıdır.	Çalışma anında daha yavaştır.
Hata Denetimi	Derleme sonunda tüm hataları gösterir.	İlk hatada durur ve hata düzeltilene kadar ilerlemez.
Örnek Diller	C, C++, Rust	Python, PHP, Ruby

Tablo 2.5: Derleyici ve Yorumlayıcı Farkı

2.13 Düşün ve Çöz

- ♦ **Düşün:** Arapsaçı kod (**spaghetti code**) neden okunması ve bakımı zor bir koda yol açar? 1950'lerin GOTO ağırlıklı kodlarından günümüz nesne yönelimli mimarilerine geçişte hangi kavramsal sıçramalar yaşandı?
- ♦ **Düşün:** Yapısal programlamada fonksiyonlar ile nesne yönelimli programlamadaki yöntemler (**method**) arasındaki en temel kavramsal fark nedir? Bir sıralama algoritması yazarken her iki yaklaşım nasıl farklı bir tasarım ortaya çıkarır?
- ♦ **Düşün:** Assembly dilinde yazılan bir program neden yüksek seviyeli dile göre daha hızlı çalışabilir? Ancak günümüzde sistem programlaması dışında neden tercih edilmemektedir? Modern derleyicilerin optimizasyon yetenekleri bu soruya nasıl etki eder?
- ♦ **Düşün:** Veri (**data**) ile bilgi (**information**) arasındaki farkı bir hastane bilgi sistemi senaryosu üzerinden açıklayınız. Ham hasta verisi nasıl anlamlı bir tıbbi rapora dönüşür?
- ♦ **Düşün:** Algoritma tasarımında yalancı kod (**pseudocode**) kullanmak neden doğrudan koda başlamakta daha avantajlıdır? Bir ekip projesinde pseudocode'un iletişim aracı olarak işlevi nedir?
- ♦ **Düşün:** Modüler programlama büyük bir yazılım projesinde farklı ekiplerin paralel çalışmasını nasıl mümkün kılar? Modüllerin birbirinden bağımsız test edilebilmesinin önemi nedir?
- ♦ **Düşün:** Simula ve Smalltalk dilleri nesne yönelimli programlamanın öncüleridir. Bu dillerin C++'a ve

oradan C#'a aktardığı en kritik kavramlar nelerdir? 'Sınıf' (**class**) fikri neden devrimci sayılır?

- ◇ **Düşün:** Yapısal programlama ile nesne yönelimli programlama arasında seçim yaparken hangi faktörler belirleyicidir? Her ikisinin de güçlü olduğu kullanım alanlarına birer örnek veriniz.
- ◇ **Çöz:** Bir öğrencinin vize (40 puan üzerinden) ve final (60 puan üzerinden) notlarını girdi olarak alan, geçme notunu 50 kabul eden ve sonucu 'Geçti' / 'Kaldı' olarak ekrana basan algoritmanın yalancı kodunu (**pseudocode**) yazınız.
- ◇ **Çöz:** Bir sayının asal olup olmadığını kontrol eden algoritmanın akış diyagramını (**flowchart**) çiziniz. Algoritmanın zaman karmaşıklığını (**time complexity**) belirtiniz ve daha verimli hale getirmenin bir yolunu önerin.
- ◇ **Çöz:** Kullanıcının girdiği üç tam sayıdan en büyüğünü bulan algoritmanın adımlarını mantıksal sırayla listeleyiniz. Algoritmanızı dört sayı için genelleştiriniz.
- ◇ **Çöz:** Fibonacci serisinin ilk 10 terimini hesaplayan algoritmanın yalancı kodunu ve akış diyagramını hazırlayınız.
- ◇ **Düşün:** "Günümüzde kullandığımız yapay zeka araçları kod yazabiliyor olsa da, neden hala bir programcının 'Algoritmik Düşünme' yeteneğine sahip olması zorunludur? Kod yazmak ile problem çözmek arasındaki fark nedir?"

3. Bölüm

Derleyiciler ve Çalıştırma Ortamı

3.1 Derleyici ve .NET Framework

Genel amaçlı bilgisayarların çok kullanılmasıyla birlikte metin editörleri de (Windows Not Defteri, Mac TextEdit, Notepad++, Epsilon, EMACS, nano, vim veya vi) işletim sistemlerinin bir parçası olmuştur. Metin editörlerinde yazılan programlar ise **derleyiciler (compiler)** tarafından makine koduna çevrilerek **icra edilebilir (executable)** dosya oluşur. Bu dosya, işletim sisteminin yardımıyla işlemci tarafından çalıştırılır.

C# dilinde programlama öğrenmeye başlamak için;

1. İlk adım, programı yazabileceğiniz bir metin editörü kullanmak ve yazdığınız programa ilişkin kaynak kodları *.cs uzantısı ile dosya olarak saklamaktır. Tüm kaynak kodlar doğal olarak insan tarafından okunabilen metin dosyasıdır.
2. İkinci adım ise **.NET Yazılım Çerçevesinin (.NET Framework, kısaca .NET)** uygun sürümünü işletim sisteminizde **(operating system)** kurmaktır.
3. Üçüncü adım ise yazdığınız kodu derleyerek çalışabilen bir icra edilebilir dosyayı oluşturup çalıştırmaktır. C# derleyicisi, kaynak dosya içerisinde yazılı C# talimatlarını **(statement)** .NET'in tarafından icra edilecek bir ara dosyaya dönüştürür.

C# dilinde derleme sonrasında .NET platformu tarafından çalıştırılacak **ara kod** elde edilir. .NET platformu, her işletim sistemi için ayrı kurulur. Amacı derlenen bu ara kodu, **işletim sistemi ve işlemciden bağımsız olarak her yerde çalıştırmaktır!**

.NET platformu, çalışan uygulamalarına çeşitli hizmetler sağlayan bir **çalışma ortamıdır (run-time)**. İki ana bileşenden oluşur:

- ♦ **Temel Sınıf Kütüphanesi BCL (Base Class Library)**: Yazılımcıların kendi uygulamalarını geliştirirken esas alacağı, çağırabileceği, test edilmiş ve yeniden kullanılabilir bir kütüphanedir. Bir başka adı **.NET sınıf kütüphanesidir (.NET Class Library)**.
- ♦ **Ortak Dil Çalışma Ortamı CLR (Common Language Runtime)**: .NET için derlenmiş uygulamaları çalıştıran ve yöneten motordur **(engine)**. İş parçacığı **(thread)** yönetimi, çöp toplama GC **(garbage collector)**, veri tipi güvenliği **(type safety)**, istisna yönetimi **(exception handling)** ve daha fazlasını içeren hizmetler sağlar.

Bu bileşenler, çalışan uygulamalar için aşağıdaki hizmetleri sunar:

- ♦ **Bellek yönetimi**: Çalışma ortamı (CLR), birçok programlama dilinde, programcılar bellek ayırmayı ve serbest bırakmayı ve nesne ömrünü işlemekten sorumludur.
- ♦ **Ortak tip sistemi CTS (Common Type System)**: Geleneksel programlama dillerinde temel tipler derleyici tarafından tanımlanır, bu da başka dillerde yazılan uygulamaların ortak çalışabilirliği karmaşıktır. .NET, temel tipleri ortak tip sistemi tarafından tanımlanır ve .NET ile çalışan tüm dillerde ortaktır.
- ♦ **Kapsamlı bir sınıf kütüphanesi**: Programcılar, sık kullanılan alt düzey programlama işlemlerini işlemek için çok miktarda kod yazmak yerine, Temel sınıf kütüphanesinden (BCL) veri tipi ve üyeleri olan kolay erişilebilir bir kitaplık kullanır.
- ♦ **Yazılım geliştirme çerçeveleri**: .NET, web uygulamaları için ASP.NET, veri erişimi için ADO.NET, hizmet odaklı uygulamalar için WCF **(Windows Communication Foundation)** gibi yazılım geliştirme çerçevelerine sahiptir.
- ♦ **Birlikte çalışabilirlik (Interoperability)**: .NET altyapısı için uygulama geliştirmede kullanılan olan tüm diller (C#, VB.NET, F#, ...) derlendikten sonra ortak ara dile olan CIL **(Common Intermediate Language)**/MSIL **(Microsoft Intermediate Language)** koduna çalışma ortamı tarafından dönüştürülür. Bu özellik ile, bir dilde yazılan yöntemler diğer diller tarafından erişilebilir olur. Böylece programcılar tercih ettiği dilde uygulama oluşturmaya odaklanırlar.
- ♦ **Sürüm uyumluluğu**: Nadir özel durumlar dışında belirli bir .NET sürümü kullanılarak geliştirilen uygulamalar, daha sonraki sürümde değişiklik yapılmadan çalışır.
- ♦ **Yan yana çalıştırma**: .NET, aynı bilgisayarda ortak dil çalışma ortamının birden çok sürümünün var

olmasına izin vererek sürüm çakışmalarını çözmeye yardımcı olur.

- ◊ **Çoklu sürüm desteği:** Geliştiriciler, standart bir .NET sürümü tarafından desteklenen, birden çok sınıf kütüphanesi oluşturabilir.

3.2 Ortak Dil Tarifnamesi

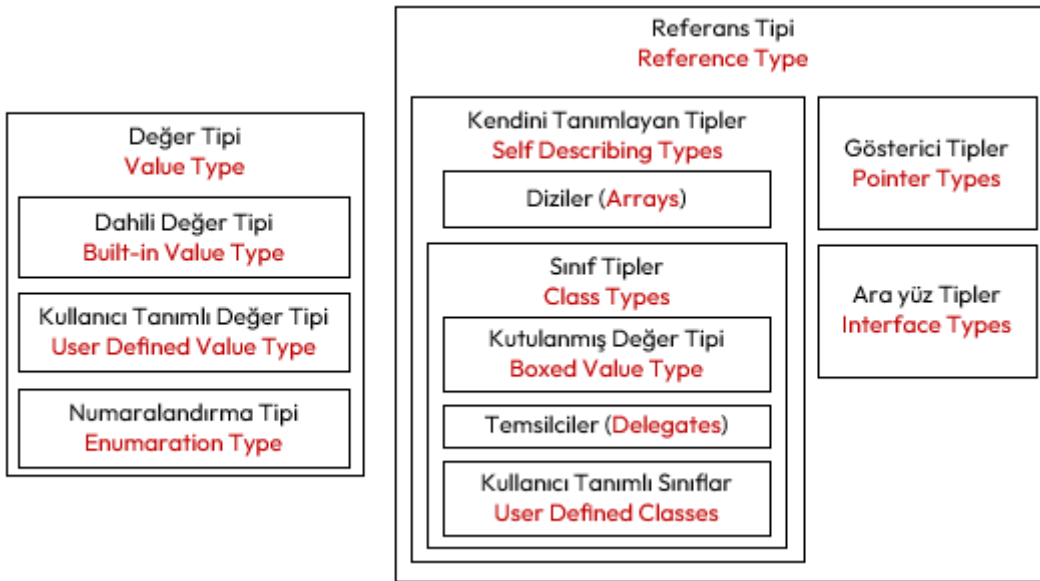
Ortak Dil Tarifnamesi CLS (**Common Language Specification**) programlama dillerinin nesne modellemesinden kaynaklanan farklılıklarını bertaraf etmek için geliştirilmiş standartlar dizisidir. Bunlar aşağıda belirtilen başlıklarda toplanmaktadır;

- ◊ Yapı (**structure**) standardı.
- ◊ Ortak ara dil (CIL) standardı, programlama dili ne olursa olsun, bu diller herkes tarafından bilinecek bir ara koda dönüştürülür ya da derlenir.
- ◊ Veri tipi standardı CTS (**Common Type System**) aşağıda anlatılmıştır.
- ◊ Olay (**event**) standartları.
- ◊ Verilerin bellekteki yerleşimine (**persistence**) ilişkin standartlardır. Aynı blok içinde tanımlanan **char**, **int**, **double** değişkenlerinin tanımlandığı örneği göz önüne alacak olursak; C dilinde, belleğe ilk önce **char**, sonra **int**, daha sonra **double** yerleşir (**row major**). Pascal dilinde ise ilk önce **double**, sonra **int**, son olarak da **char** yerleşir (**column major**).

İşte bu tip değişken yerleşimlerinden kaynaklanacak problemleri gidermek için bu standartlar oluşturulmuştur.

3.3 Ortak Tip Sistemi

Ortak Tip Sistemi (CTS), dillerdeki veri tipleri arasındaki farklılıkların bertaraf edilmesine yönelik standartlardır. Bu sistem ile bir dilde 4 bayt, diğer bir dilde 8 bayt olan gerçek sayı tanımlaması gibi karmaşıklıklar giderilmiştir.



Şekil 3.1: Ortak Tip Sistemi (CTS)

CTS, aşağıdaki veri tiplerini desteklemektedir:

1. Değer Tipleri (**value type**)
 - (a) Dahili değer tipleri (**built-in value type**): **bool**, **int**, **byte**, **double**, **char** gibi veri tipleridir.
 - (b) Kullanıcı tanımlı değer tipleri (**user-defined value type**): **struct** gibi kullanıcı tarafından, yerleşik değer tiplerinin çeşitli şekillerde bir araya getirilip tek bir isim altında toplanmasıyla oluşan tiplerdir.
 - (c) Numaralandırma (**enumeration**): Bir kümeye ait elemanların her birine bir numara atanarak kod okunabilirliğini yükseltmek için tanımlanan **enum** tipleridir.
2. Referans Tipler (**reference type**)
 - (a) Gösterici tipler (**pointer type**)
 - (b) Arayüz tipler (**interface type**)
 - (c) Kendini tanımlayan tipler (**self-description type**)
 - (d) Diziler (**array**): Tek boyutlu, çok boyutlu ya da dizilerin dizisi (**jagged array**) gibi diziler.
 - (e) Sınıflar (**class**): **object** ve **string** gibi ön tanımlı sınıflar.

- (f) Kullanıcı tanımlı sınıflar (**user-defined class**)
- (g) Doğrudan ulaşımı engellenmiş ya da kutulanmış değer tipler (**boxed value types**)
- (h) Temsilciler (**delegate**): Yöntemleri referans gösteren tipler.

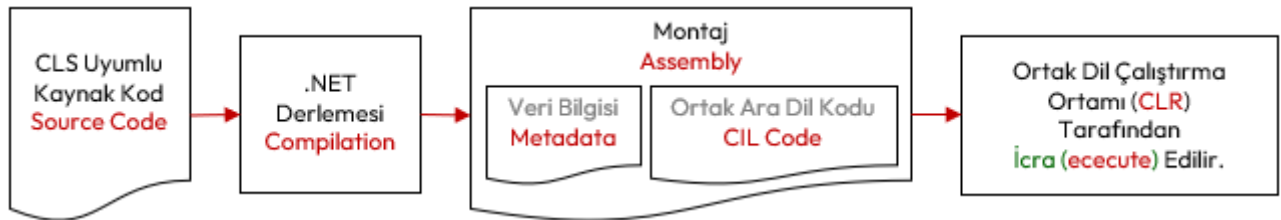
3.4 Derleme ve İcra Süreci

C ve C++ gibi klasik yazılım geliştirme araçlarıyla derlenen bir uygulama aşağıda gösterildiği gibi bir süreçten geçer. Yani kaynak kod derlenir ve doğrudan işletim sistemi tarafından işlemciye hedef gösterilerek çalıştırılacak şekilde işlemcinin emir kodlarından (**instruction code**) oluşan çalıştırılabilir kod (**executable code**) elde edilir (Şekil 3.2). Klasik derlemede sonucunda oluşan bu dosyaya, **ikili çalıştırılabilir doğal kod** (**binary executable native code**) veya kısaca **doğal kod** (**native code**) adı verilir.



Şekil 3.2: Klasik Derleme Süreci

.NET altyapısında ise kaynak kod derlenir ve çalıştırma ortamı (CLR) tarafından icra edilecek şekilde **ortak ara dil** (CIL) ve **veri bilgisinden** (**metadata**) oluşan bir dosya elde edilir. Derlenmiş bu .NET uygulamasına, **montaj** (**assembly**) adı verilir (Şekil 3.3). Bu isim, 1.9 başlığında anlatılan sembolik isimlerle yazılan kodlarla karıştırılmamalıdır.



Şekil 3.3: .NET Framework Derleme Süreci

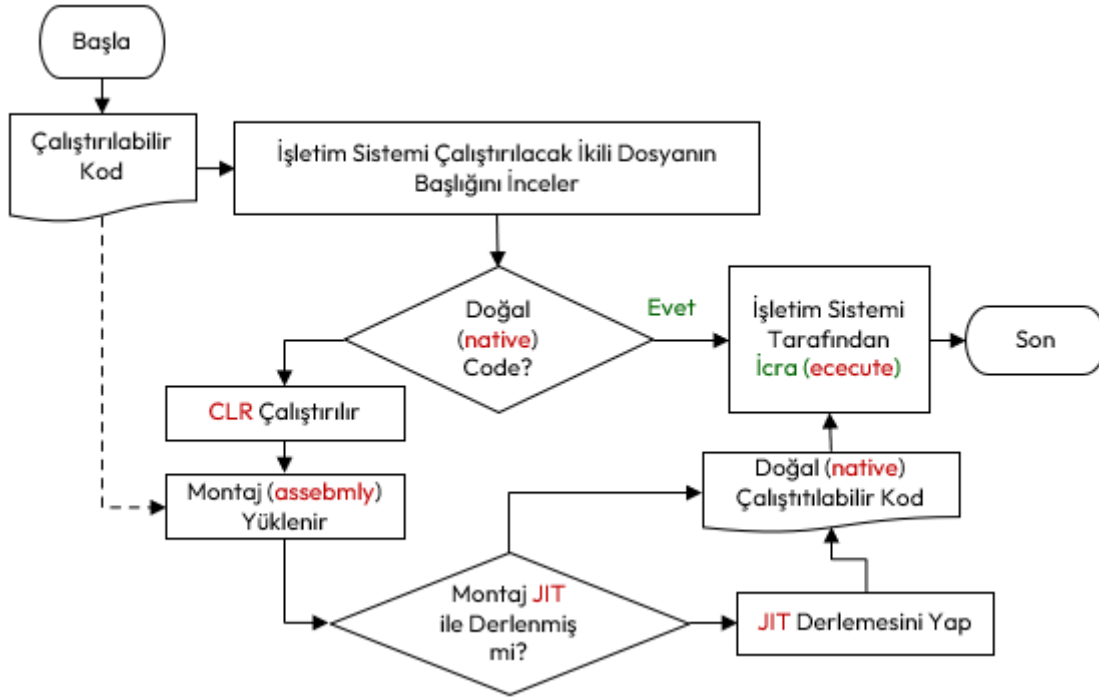
.NET altyapısında çalışacak kaynak kod, ortak dil tarfnamesinde (CLS) belirtilen standartlara uymak zorundadır. İçinde ara dil (CIL) kodu olan bu **montaj** (**assembly**) uygulama, işlemcinin anladığı emir kodları (**instruction code**) ile hiçbir ilgisi yoktur ve işletim sistemine bağımlı değildir. .NET platformu için yapılmış derleme işlemine, **yönetilmiş derleme** (**managed compilation**) adı verilir. Bunun yanında CLS uyumlu olarak hazırlanmış kaynak kodlara **yönetilmiş kod** (**managed code**) adı verilir. CLS uyumlu hazırlanmayan kodlara ise **yönetilmemiş kod** (**unmanaged code**) adı verilir. **Yönetilmiş kodlar CLR tarafından, yönetilmemiş kodlar ise işletim sistemi tarafından çalıştırılır.**

Peki işletim sistemi; Çalıştırılabilir (**executable**) uygulamayı işlemcinin mi yoksa .NET platformunun mu çalıştıracağını nasıl anlar? Windows işletim sisteminde çalıştırılabilir uygulamalar *.exe uzantısına sahiptir. Windows, derlenmiş bir çalıştırılabilir uygulamayı Şekil 3.4'deki gibi yapar. Klasik derleme sonucunda oluşan doğal kod (**native code**), işletim sistemi tarafından doğrudan belleğe yüklenerek çalıştırılır. .NET platformu için derlenen montaj (**assembly**) uygulamanın çalışması için, ortak dil çalıştırma ortamının (CLR) işletim sisteminde çalışıyor olması gerekir. Çalışıyor olan CLR, ilgili montaj uygulamasını belleğe yükler ve ani derleyiciler olan **JIT Compiler** (**Just-in-time Compiler**) tarafından derlenerek işlemcinin anlayacağı **doğal kod** (**native code**) elde edilir. Burada oluşan doğal kod, CLR kontrolünde, işletim sistemi tarafından çalıştırılır. İcra edilebilir bu doğal kod, her platformda farklıdır. Windows işletim sisteminde .exe, Linux işletim sisteminde ise uzantısız bir dosyadır.

Ortak dil çalışma ortamı (CLR), ortak dil altyapısı (CLI) standartlarına uyumlu olarak derlenmiş ve kurulu olduğu işletim sisteminin kaynaklarını kullanarak oluşturulan CIL kodunun doğru bir şekilde çalıştırılmasını sağlayan altyapıdır.

Microsoft tarafından Visual Basic, Visual Java, Visual C++ dillerinde küçük değişiklikler yapılarak CLS uyumlu hale getirilmiştir. Bu diller .NET dilleri olarak da adlandırılır. Böylece yazılımcı takımında dil farklılıklarından doğan geliştirme problemlerine de çözüm bulunmuştur. Böylece yazılımcıların .NET mimarisine geçmek için harcaacağı eğitim ve zamandan tasarruf sağlamıştır.

Ortak dil çalıştırma ortamının montaj uygulamayı belleğe yükleyip, ani derleyicilerden geçirerek işlem-



Şekil 3.4: Derlenmiş bir Çalıştırılabilir Uygulamanın Çalışma Süreci

cinin çalıştıracağı doğal kod (**native code**) elde edilir. Bazı durumlarda içinde bulunulan işletim sistemine uygun olarak montaj (**assembly**) uygulaması bu doğal koda önceden çevrilir. Buna **vaktinden önce doğal derleme** NAOT (**Native Ahead-Of-Time**) adı verilir. Bu durum için .NET derleyicileri optimize edilmemiştir. Bazı test senaryoları için kullanılabilir.

3.5 Ortak Ara Dil Kodu

Yapısal programlama ve fonksiyon kavramının bilgisayarlarda kullanılmasıyla birlikte işlemcilerde de değişiklikler olmuştur. Basit bir işlemci 1.5 başlığında anlatılan işlemciye yeni kaydediciler de eklenmiştir;

- ◊ Veriler ile icra edilen kod farklı bellek bölgesinde bulunur. Bu nedenle verileri işaret eden **veri adresleri kaydedicisi** (**data memory register**) işlemciye eklenmiştir.
- ◊ **Yığın bellek kaydedicisi** (**stack register**), fonksiyon çağrıları sırasında mevcut işlemci durumu ve yerel değişkenler gibi bazı verileri depolamak için kullanılan belleği gösteren (**stack pointer**) adresi tutan kaydedici işlemciye eklenmiştir.
- ◊ **Bayrak kaydedicisi** (**flag register**), işlemcinin durumu veya sıfır (**zero**) bayrağı, taşıma (**carry**) bayrağı, işaret (**sign**) bayrağı, taşma (**overflow**) bayrağı, eşlik (**parity**) bayrağı, yardımcı taşıma (**auxiliary carry**) bayrağı ve kesme etkinleştirme (**interrupt enable**) bayrağı gibi çeşitli işlemlerin sonucunu tutmak için kullanılan özel bir kaydedici işlemciye eklenmiştir.
- ◊ **Genel amaçla kullanılan kaydediciler** (**general purpose registers**) işlemciye eklenmiştir.

Eklenen bu kaydediciler ile doğal koda (**native code**) **derlenen her bir program**, dört bellek bölgesinden (**segment**) oluşur;

1. **Kod bellek** (**code segment**) ya da kaynak koda ithafla **metin bellek** (**text segment**) icra edilecek emirleri içeren bellektir.
2. Programın işleyeceği verileri tutan **veri belleği** (**data segment**).
3. Yığın bellek (**stack segment**), fonksiyon çağrıları sırasında kullanılan bellektir. Bu bellek, son giren veri ilk çıkar LIFO (**last in first out**) mantığıyla çalışır. Çünkü;
 - ◊ **Çağrıdan önce** mevcut işlemci durumu ve yerel değişkenler gibi bazı verileri depolamak ve
 - ◊ Fonksiyondan **dönülünce** işlemciyi ve çağrı ortamını eski hale getirmek için kullanılır.
4. Programın icrası sırasında dinamik olarak eklenip çıkarılan veriler için hurdalık gibi kullanılan **öbek bellek** (**heap segment**).

Klasik derlenmiş doğal kod (**native code**) dosyaları gibi, .NET altyapısında da ortak dil kodu (**CIL code**) da sanki sanal bir işlemci varmış gibi **bu dört bellek bölgesiyle çalışacak şekilde üretilir**. Çalıştırma ortamı (CLR) tarafından içinde bulunduğu işletim sistemine ait işlemcinin emir kodlarına JIT ile derlenir.

Ayrıca ortak dil tarifiğinde (CLS) belirtilen standartlara uymak zorunda olan birçok programlama dili bulunmaktadır; VB.NET, C#, VC++.NET, J#, F#, Jscript.NET, WindowsPowerShell, Iron phyton, Iron Ruby. Bu dillerde yazılan kodlar derlenerek ortak ara dile (CIL) dönüştürülür.

3.6 .NET Framework Kurulumu

Çerçeve yapının son sürümü, resmi sitesinden indirilerek kurulur ¹. Kurulum tamamlandıktan sonra konsoldan aşağıdaki komut yazılarak kurulum teyit edilebilir;

```
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. Tüm hakları saklıdır.
C:\Users\ILHANOZKAN>dotnet
```

```
Usage: dotnet [options]
Usage: dotnet [path-to-application]
```

```
Options:
-h|--help          Display help.
--info             Display .NET information.
--list-sdks        Display the installed SDKs.
--list-runtimes    Display the installed runtimes.
```

```
path-to-application:
The path to an application .dll file to execute.
C:\Users\ILHANOZKAN>
```

Sonrasında **PATH** değişkenine C# Derleyicisinin yolu (C:\Windows\Microsoft.NET\Framework\v4.0.30319\) eklenir. Daha sonra kurulum, konsolda **csc -help** komutu yazılarak kontrol edilebilir. *Düzeltilir*

En sonunda metin editörü ile ilk kaynak kodumuzu oluşturmak için komut satırında **notepad hello.cs** komutu yazılır. Eğer böyle bir dosya yoksa editör bu dosyayı oluşturmak için onay ister. Açılan metin editörü penceresine aşağıdaki ilk C# programı yazılır ve **hello.cs** olarak dosyaya kaydedilir.

```
using System;
using System.IO;
namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Bu kodu derlemek için **csc hello.cs** komutu konsolda yazılır. Artık .NET CIL kodumuzu barındıran **hello.exe** adlı **montaj (assembly)** uygulaması hazır. Derlenen programı çalıştırmak için yine aynı komut satırında **hello.exe** yazarak CLR tarafından icra edilebilir.

```
C:\Users\ILHANOZKAN>notepad hello.cs
```

```
C:\Users\ILHANOZKAN>csc hello.cs
Microsoft (R) Visual C# Compiler version 4.8.9232.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
This compiler is provided as part of the Microsoft (R) .NET Framework,...
```

```
C:\Users\ilhan>hello.exe
Hello, World!
```

```
C:\Users\ILHANOZKAN>
```

Bazı durumlarda bir projede sadece sınıflar bulunur. Bu tip projelere sınıf kütüphanesi denir ve der-

¹<https://dotnet.microsoft.com/en-us/download>

lendiğinde *.dll dosyası oluşur. Bunlar derlenmiş olarak başka projelerde referans gösterilerek kullanılabilir. İleride örnekleri verilecektir.

3.7 Entegre Geliştirme Ortamı

Şimdiye kadar anlatılan kod yazma (**implementation**), derleme (**compile**), icra (**execute**) ya da koşma (**run**) ve izleme (**trace**) gibi süreçlerin tamamını tek bir çatı altında yürütmemizi sağlayan programlar, entegre geliştirme ortamı IDE (**Integrated Development Environment**) olarak adlandırılır. C# programlama dili için en çok kullanılan entegre geliştirme ortamları aşağıda verilmiştir;

- ◊ **Visual Studio**, Microsoft tarafından geliştirilen, C dilinde yazılmış, güçlü, yüksek performanslı uygulamalar oluşturmak için kullanılabilen bir geliştirme ortamıdır. Yalnızca Windows'ta çalışır. Visual Studio, kod tamamlama (**intellisense**), kullanıcı ara yüzü UI (**user interface**) hazırlama desteği ve hata ayıklama (**debugger**) ve birçok eklentisi (**plug-in**) olan muazzam özelliklere sahiptir. Bu geliştirme ortamının kurulumu dipnotta verilmiştir.
- ◊ **Visual Studio Code**, Microsoft tarafından geliştirilen açık kaynaklı bir geliştirme ortamıdır. Windows, macOS ve Linux gibi işletim sistemlerinde çalışır. Git sürüm kontrol (**version control**) sistemleriyle çok iyi çalışır. Ayrıca, akıllı kod tamamlamanın dikkate değer özellikleriyle birlikte gelir.
- ◊ **Xcode**, MacOSX işletim sisteminde yazılım geliştirmek için kullanılan bir entegre geliştirme ortamıdır.
- ◊ **JetBrains Rider**, Windows, Mac OS X ve Linux'ta .NET ve .NET Core uygulamalarını destekler. Rider'ın hızlı performansı, geniş platform ve çalışma zamanı desteği, sürüm kontrol sistemleriyle sorunsuz entegrasyonu ve kapsamlı derleme çözümleme (**decompile**) özellikleri öne çıkan özelliklerinden bazılarıdır.
- ◊ **MonoDevelop**, .NET dilleri için oluşturulmuş açık kaynaklı bir geliştirme ortamıdır. Yazılım geliştirme için Mono framework ve .NET framework'ü kullanır. MonoDevelop ile MacOSX, Windows ve Linux dahil olmak üzere çeşitli işletim sistemleri için masaüstü ve web uygulamalarını verimli bir şekilde oluşturabilirsiniz.
- ◊ **.NET Core CLI**, .NET Core uygulamalarını geliştirmek, oluşturmak, çalıştırmak ve yayınlamak için komut satırı arayüzüdür (**command line interface**). Bu arayüz adından da anlaşılacağı üzere grafik ara yüzünden yoksundur.
- ◊ Entegre geliştirme ortamları metin dosyası olarak tutulan kaynak kodları programcıya renklendirerek gösterir. Bu da programcının kodu anlamasını daha da kolaylaştırır. Ancak kodu dosyaya yine sade metin olarak kaydeder.

3.8 Çevrimiçi Derleyiciler

Bütün bu entegre geliştirme ortamlarının yanında çevrim içi olarak da derleme yapılan web uygulamaları da bulunmaktadır. Bunlardan birkaçı aşağıda verilmiştir;

- ◊ https://www.onlinegdb.com/online_csharp_compiler
- ◊ <https://www.tutorialspoint.com/csharp/index.htm>
- ◊ <https://onecompiler.com/csharp>

3.9 Derleme Zamanı

Derleyici programının derleme işlemine başlayıp bitirildiği sürece **derleme zamanı** (**compile-time**) denir. Bu süreç başarısızlıkla da sonuçlanabilir ve eğer derleme işleminde hata meydana gelirse programcı hata mesajları ile uyarılır.

Bir derleyici program, kaynak dosyayı makine diline çevirme çabasında, kaynak dosyanın C# dilinin söz dizimi kurallarına uygunluğunu da denetler. Eğer dilin kurallarına uyulmamışsa, derleyici bu durumu bildiren bir ileti de vermek zorundadır.

```
using System;
using System.IO;
namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!")
        }
    }
}
```

Yukarıda hatalı yazılmış bir C# programı derlendiğinde aşağıdaki hata alınacaktır.

```
C:\Users\ILHANOZKAN>csc.exe hello.cs
Microsoft (R) Visual C# Compiler version 4.8.9232.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.
```

This compiler is provided as part of the Microsoft (R) .NET Framework,...

```
hello.cs(10,47): error CS1002: ; bekleniyor
```

```
C:\Users\ILHANOZKAN>
```

3.10 Hatalar

Programcı tarafından kodlama yapılırken genellikle aşağıdaki üç tip hata yapılır;

1. **Derleme zamanı hataları (compile-time error)**: Bu tip hatalar genelde kullanılan dilin yazım kurallarına (**syntax**) uyulmadığından, talimatların (**statement**) yanlış yazılmasından ya da programcının kod metninde uygun olmayan karakterlerin kullanılmasından kaynaklanır.
2. **Çalışma zamanı hataları (run-time error)**: Kaynak kod, kurallara uygun olarak yazılmıştır ve herhangi bir yazım hatası bulunmaz. Bu tip hatalara en iyi örneklerden birisi sıfıra bölme hatasıdır. Taşma hataları da bu hatalardandır. Daha sonra değinilecektir.
3. **Mantıksal hatalar (logical error)**: Programcının çözüm için gerekli adımların oluşturulmasında, çözüm yönteminin yanlış olmasından ya da yanlış işleçler (**operator**) kullanmasından kaynaklanır. Örneğin bir büyüktür (>) işleci yerine küçüktür (<) işleci kullanıldığında ne bir yazım hatası ne de bir çalışma zamanı hatası ortaya çıkar. Fakat program kendisinden istenilen davranışları yerine getirmez ve uygun çıktıları üretmez. Faktöriyel hesaplanırken $0! = 1$ yerine $0! = 0$ kabul etmek buna örnek verilebilir.

3.11 C# Projesi Oluşturma ve Çalıştırma

§3.11.1 .NET Core CLI ile Proje Oluşturma

Konsolda komut satırı kullanılarak bir proje oluşturmak için aşağıdaki adımlar takip edilir:

1. Çalıştığımız herhangi bir klasörde yeni proje için konsolda `mkdir konsolprojesi` komutu ile yeni klasör oluşturulur.
2. Ardından oluşturulan klasöre girilir (`cd konsolprojesi`).
3. Yeni bir proje oluşturmak için `dotnet -new console` komutu çalıştırılır.
4. İki adet dosya oluşacaktır: Biri proje dosyası (`konsolprojesi.csproj`), diğeri ise `Program.cs` dosyasıdır.
5. Bir de projeye ilişkin amaç dosyaların olacağı `obj` adlı bir klasör de oluşur.

Oluşan Projenin Klasör Yapısı Aşağıda verilmiştir;

```
konsolprojesi/
├── Program.cs
├── konsolprojesi.csproj
├── obj/
│   ├── konsolprojesi.csproj.nuget.dgspec.json
│   ├── konsolprojesi.csproj.nuget.g.props
│   ├── konsolprojesi.csproj.nuget.g.targets
│   ├── project.assets.json
│   └── project.nuget.cache
```

Ana programımız olan `Program.cs` dosyası da en basit haliyle aşağıdaki gibi oluşur;

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

Ayrıca XML (**Extensible Markup Language**) olarak oluşan proje dosyasının (`konsolprojesi.csproj`) içeriği aşağıdaki gibidir;

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
```



```
</PropertyGroup>
</Project>
```

Oluşan projede **ana programı çalıştırmak için** proje klasöründe **dotnet run** komutu çalıştırılır.

```
C:\Users\ILHANÖZKAN\konsolprojesi>dotnet run
Hello, World!
```

Çalışma sonrasında proje klasörü aşağıdaki şekilde değişir;

```
konsolprojesi/
├── Program.cs
├── konsolprojesi.csproj
├── obj/
│   ├── Debug/
│   │   └── net10.0/
│   │       └── ...
│   ├── konsolprojesi.csproj.nuget.dgspec.json
│   ├── konsolprojesi.csproj.nuget.g.props
│   ├── konsolprojesi.csproj.nuget.g.targets
│   ├── project.assets.json
│   └── project.nuget.cache
├── bin/
│   └── Debug/
│       └── net10.0/
│           ├── konsolprojesi.deps.json
│           ├── konsolprojesi.dll
│           ├── konsolprojesi.exe
│           ├── konsolprojesi.pdb
│           └── konsolprojesi.runtimeconfig.json
```

Aslında icra edilen **./bin/Debug/net10.0/konsolprojesi.exe** ve dolaylı olarak **konsolprojesi.dll** montaj dosyalarıdır. Geri kalan dosyalar çalışma ortamı ile ilgili ayarlardır.

Çalıştırma değil de sadece derleme yapılacak ise iki tip derleme yapılabilir;

- ◊ Hata ayıklama (**debug**) amacıyla yapılacak ise **dotnet build** komutu ile derlenir.
- ◊ Yayım (**release**) amacıyla yapılacak ise **dotnet build -c Release** komutu ile derleme yapılır.

Bu adımlar MacOSX ve Linux işletim sistemleri üzerinde .NET yüklü ise aynen çalışır.

§3.11.2 Visual Studio ile Proje Oluşturma

Windows işletim sisteminde çalışan Visual Studio, kod geliştiriciler için en çok tercih edilen geliştirme ortamlarından biridir. İngilizce dile sahip Visual Studio programında sırasıyla şu adımlar takip edilir;

1. Programı ilk defa çalıştırıyorsanız, karşılama sayfasından **Create a new project** seçiniz. Eğer program açık ise **File** menüsüne giderek **File→New Project** seçiniz.
2. Proje tipi olarak **Console Application** seçilir.
3. **Project Name**, **Location** ve **Solution Name** belirlenerek ilk proje oluşturulmaya başlanır.
4. Projeye oluşturma sihirbazının **Additional Information** sayfasında **Do not use top-level statements** kutucuğu varsayılan olarak *seçili değildir*. Bu durumda projemiz yeni proje şablonuyla oluşur.
5. **Solution Explorer** üzerinde **Program.cs** dosyası açılır.

C#10+ ile birlikte **yeni proje şablonuyla** oluşan uygulamalar için **Program.cs** dosyası aşağıdaki şekilde oluşur;

```
1 // See https://aka.ms/new-console-template for more information
2 Console.WriteLine("Hello, World!");
```

Yeni proje şablonu ile oluşturulan projelerde **Program.cs** kodunun başında birinci satırdaki açıklama her zaman bulunur. Eski proje şablonuyla proje oluşturulmak istenirse yukarıda anlatılan 3. adımdaki *Additional Information* sayfasında **Do not use top-level statements** kutucuğu *seçili bırakılır*. Bu durumda **Program.cs** alışık olduğumuz eski şekilde oluşur;

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
```

```

7         Console.WriteLine("Hello, World!");
8     }
9 }
10 }

```

Burada ilk oluşturduğumuz yeni tip `Program.cs` dosyasında yazılan kodları, ikinci oluşan eski tip `Program.cs` dosyasındaki `static void Main(string[] args)` yönteminin içerisinde yazılıyor gibi kabul edebilirsiniz. Yeni proje şablonuyla yazılan değişkenlerin "yerel değişken" değil, aslında gizli bir **Main yönteminin içindeki değişkenlerdir**, bu yüzden ana programın parametrelerine `args` parametresine hala erişilebilir.

Yeni proje şablonuyla oluşturulan `Program.cs` dosyasına aşağıdaki yönergeler uygulamaya örtük olarak eklenir. Bu kütüphanelerdeki sınıfları projemizde doğrudan kullanabiliriz;

```

◇ using System;
◇ using System.IO;
◇ using System.Collections.Generic;
◇ using System.Linq;
◇ using System.Net.Http;
◇ using System.Threading;
◇ using System.Threading.Tasks;

```

Visual Studio menüsünde, **hata ayıklama** için **Debug**→**Start Debugging** tıklanır veya **F5** tuşuna basılarak hata ayıklama modunda program çalıştırılır. Ya da hata ayıklamadan derleme yapmak için **Debug**→**Start Without Debugging** tıklanarak veya **Ctrl + F5** tuşlarına basarak program doğrudan çalıştırılabilir.

Visual Studio ile oluşturulan projelerde de komut satırından oluşturulan projeler gibi aynı klasör yapısında dosyalar oluşur.

Kitap içerisinde oluşturulan projelerde aksi belirtilmedikçe C#6+ sonrası kullanılan **yeni tip proje şablonu kullanılacaktır!**

3.12 Düşün ve Çöz

- ◇ **Düşün:** .NET'in Ortak Ara Dili (CIL/MSIL) ile Java'nın bytecode'u arasındaki benzerlik ve farklar nelerdir? 'Write once, run anywhere' felsefesi her iki platformda nasıl hayata geçirilir?
- ◇ **Düşün:** JIT (**Just-In-Time**) derleme ile AOT (**Ahead-Of-Time**) derleme arasındaki temel fark nedir? .NET'in ReadyToRun ve NativeAOT özellikleri hangi kullanım senaryolarında tercih edilir?
- ◇ **Düşün:** Derleme zamanı hatası (**compile-time error**) ile çalışma zamanı hatası (**run-time error**) arasındaki fark nedir? Hangisi daha tehlikelidir ve neden? Bir örnek üzerinden açıklayınız.
- ◇ **Düşün:** Ortak Dil Tarifnamesi (CLS) ve Ortak Tip Sistemi (CTS) olmadan C# ile F# veya VB.NET'in bir arada çalışması neden mümkün olmaz? Dil bağımsızlığının kurumsal yazılım projelerindeki önemi nedir?
- ◇ **Düşün:** Çöp toplayıcı GC (**garbage collector**) bellek yönetimini nasıl yapar? Çöp toplayıcının olmadığı bir ortamda (örneğin C/C++'da) bellek sızıntısına (**memory leak**) nasıl yol açılır? C#'ta yine de bellek sızıntısı oluşabilir mi?
- ◇ **Düşün:** Visual Studio ile VS Code arasında bir C# projesi geliştirirken hangisini tercih edersiniz ve neden? Ticari bir proje için IDE seçimini etkileyen faktörler nelerdir?
- ◇ **Düşün:** .NET 8 ile .NET Framework 4.x arasındaki temel farklar nelerdir? Eski bir .NET Framework uygulamasını modern .NET'e taşımanın (migration) önünde hangi engeller bulunabilir?
- ◇ **Çöz:** C# kaynak kodunun .NET çalışma ortamında icra edilmesine kadar geçen adımları sırasıyla açıklayan bir akış diyagramı çiziniz: `.cs` dosyası→CIL→JIT→makine kodu→çalışma.
- ◇ **Çöz:** Komut satırından (CLI) yeni bir C# konsol uygulaması oluşturan, derleyen ve çalıştıran komutları adım adım yazınız. Her komutun ne yaptığını açıklayınız.
- ◇ **Çöz:** Aşağıdaki kodda kaç tür hata bulunabilir? Her hatayı sınıflandırınız (söz dizimi/mantık/çalışma zamanı): `int x = 10; int y = 0; Console.WriteLine(x / y);`
- ◇ **Çöz:** `using System;` direktifinin olmadığı bir projede `Console.WriteLine()` çağrısının neden derleme hatası verdiğini açıklayınız. Bunu çözmek için iki farklı yolu gösteriniz.
- ◇ **Çöz:** `dotnet run` ile `dotnet build + dotnet ./bin/.../app.dll` komutları arasındaki farkı açıklayınız. Hangi senaryoda hangisi tercih edilir?
- ◇ **Düşün:** C# kodu neden doğrudan işlemcinin anlayacağı makine koduna değil de önce bir 'Ara Dil' (CIL) koduna dönüştürülür? Bu durumun platform bağımsızlığıyla ilgisi nedir?

- ◇ **Düşün:** Visual Studio'da 'Hata Ayıklama' (**debug**) modu ile 'Hata Ayıklamadan Başlat' arasındaki temel fark, programın çalışma hızı ve geliştiriciye sunduğu bilgiler açısından nelerdir?
- ◇ **Düşün:** Yeni C# proje şablonlarında neden **static void Main** metodunu görmüyoruz? Arka planda derleyici bu kodu bizim yerimize nasıl oluşturuyor?

4. Bölüm

C# Programlama Diline Giriş

4.1 Söz Dizim Kuralları

C# dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizimi kullarına** (**syntax**) uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

§4.1.1 Kaynak Kodu Oluşturan Karakterler

C# dilinde de kaynak kodu oluşturan karakterler ön tanımlı olarak **UTF-8 karakterleridir**. Ancak talimatları (**statement**) oluşturan anahtar kelimeler (**keyword**) İngiliz alfabesindeki karakterlerden oluşur.

- İngiliz alfabesindeki büyük harfler:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
- İngiliz Alfabesindeki küçük harfler:
a b c d e f g h i j k l m n o p q r s t u v w x y z
- Rakamlar:
0 1 2 3 4 5 6 7 8 9
- Özel Karakterler:
< > . , _ () ; \$: % [] # ? ' & { } " ^ ! * / | - ~ +
- Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk** (**white space**) karakterleri;
 - Boşluk (**space**)
 - Yeni satır (**new line**)
 - Satır başı (**carriage return**)
 - Geri (**backspace**)
 - Sekme (**tab**)

İleride anlatacağımız kimliklendirme için kullanılacak kelimeler için her türlü karakter kullanılır!

§4.1.2 Anahtar Kelimeler

Anahtar kelimeler (**keywords**), programlamada kullanılan ve C# derleyicisi tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimelerdir** (**reserved word**). Bu kelimeler, C# söz diziminin bir parçası olan önceden tanımlanmış sözcüklerdir ve talimatların (**statement**), bir parçasıdır. Başka hiçbir amaç için veya kimliklendirmede kullanılamazlar. Aşağıda anahtar kelimelerin listesi verilmiştir ¹;

Veri Tipleri	Erişim Belirleyiciler	Kontrol Yapıları	Diğerleri
bool	public	if	base
byte	private	else	this
char	protected	switch	new
decimal	internal	case	return
double	static	for	try
float	readonly	foreach	catch
int	virtual	while	finally
long	override	break	throw
object	abstract	continue	using
string	sealed	yield	lock

Tablo 4.1: Temel Anahtar Kelimeler

- ◇ Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından **her zaman küçük harflerle yazılırlar**.
- ◇ C# dilinde **kod bloğu** (**code block**) oldukça fazla kullanılır. Süslü parantez açtıktan ({) sonra kapanana (}) kadar yazılanlar **kod bloğu** (**code block**) olarak adlandırılır. **Kod bloklarının sonrasinda noktalı virgül kullanılmaz!**

¹<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/>

- ◊ **Talimatlar**, yapılması gerekeni kısa ve net olarak belirten **mantıksal cümlelerdir** (**logical sequences**) ve bu blokların içinde yer alırlar. Talimatlar ve **noktalı virgül karakteri** (;) ile biter.
- ◊ Bu kurallara uyulması kaydıyla her türlü boşluk, yeni satır ve sekme gibi beyaz boşluk (**white space**) karakterleri istenildiği gibi kullanılabilir. Bundan dolayı kodlama **gelişigüzel** (**free format**) yapılabilir!

C# dilinde anahtar kelime olmayıp bağlam (**context**) olarak kullanılan başka anahtar kelimeler de vardır. **Bağlam anahtar kelimeleri** (**contextual keywords**) olarak adlandırılan bu kelimeler, derleyici için sadece belirli kod blokları içinde özel bir anlam taşırlar. Bu sayede, bu kelimeleri değişken kimliği (**identifier**) olarak kullanmanıza teknik olarak izin verilir, ancak kod okunabilirliği açısından bu pek önerilmez. Bu kelimelerin çeşitli üstünlükleri vardır;

- ◊ **Esneklik**: Normal anahtar kelimeler (örneğin **class** veya **int**) asla değişken kimliği olamazken, bağlamsal anahtar kelimeleri değişken kimliği yapabilirsin. Örneğin: **var get = 5;** geçerli bir koddur.
- ◊ **LINQ Baskınlığı**: Bağlamsal kelimelerin en yoğun kullanıldığı yer LINQ (**Language Integrated Query**) ifadeleridir. Buradaki **from**, **select** gibi kelimeler sadece sorgu içinde özel anlam taşır. İleride 15.1 başlığında anlatılacaktır.
- ◊ **Modern C# Etkisi**: **init**, **record**, **nint** ve **with** gibi kelimeler C#9.0+ sürümlerinde eklenmiştir.

Kategori	Anahtar Kelimeler
Özellik (property) & İndisleyici (indexer) LINQ Sorguları	get, set, init, value select, from, where, join, group, orderby, ascending, descending, into, on, let, equals
Zaman Uyumsuz (asenkron) Programlama Veri Modelleme Olay ve Kapsam Desen Seçme (pattern matching)	await, async, yield record, partial, var, dynamic, nint, nuint add, remove, global, when, managed, unmanaged not, and, or, with

Tablo 4.2: Bağlamsal Anahtar Kelimeler

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, işlemciye gönderilen birden fazla emir kodundan (**instruction code**) oluşur. C# uygulaması, .Net platformu için derlendiğinden işlemcinin emir kodları yerine CIL kodunu kullanır.

§4.1.3 Açıklamalar

C# programlama dilinde kod blokları (**code block**) kullanılarak yazılan kodlar birbirinden ayrılır. İstenildiği kadar iç içe blok yazılabilir. Ayrıca programcı kodu yazarken, **açıklama** (**comment**) yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunulan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü karakteri (//) kullanılır.
2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama /* ile */ karakterleri arasına alınır.

Aşağıda açıklama eklenmiş kod örneği bulunmaktadır.

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Merhaba!"); //Talimat ; ile biter.
/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan C# programıdır.
İlhan ÖZKAN, Ocak 2025
*/
{ // Bi Kod Bloğu Başlangıcı
    { // İç Kod Bloğu Başlangıcı
        //iç bloğa ait girinti...
        Console.ReadLine(); //Konsoldan Satır Okuma Talimatı.
        //...
    } //İç Kod Bloğu Bitişi
} // Bir Kod Bloğu Bitişi
```

C# dilinde kodun bir başka programcı tarafından anlaşılması için bir kod bloğu içindeki açıklama ve talimatları bloktan itibaren girintili olarak yazarız. Kodu bu şekilde yazmamızın derleyici açısından hiçbir önemi yoktur.

Derlemenin ilk aşamasında bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır.

4.2 Değişken Tanımlama

Değişkenler veri yapılarının temel öğeleridir. Değişkenin (**variable**) ne olduğu 2.4 başlığı altında verilmiştir. Değişkenler, belleğin belli bir bölgesinde yer kaplar ve çeşitli biçimde verilere ilişkin değerleri tutarlar.

Değişkenler tanımlanırken hangi veri tipinden (**data type**) olduğu belirtilir. Bu aynı zaman bellekte ayrılacak yer miktarını ve tutacağı verinin ikili biçimini belirler.

§4.2.1 Değer Tipleri

Değer tipleri (**value type**), veriyi doğrudan yığın bellek (**stack segment**) bölgesinde tutan veri tipleridir. Bir değer tipi değişkeni oluşturduğunuzda, **değişkenin kendisi verinin gerçek değeri olarak hareket eder**.

Değer tiplerin en temel özellikleri:

- ❖ **Doğrudan Erişim:** Değişkene eriştiğinizde **doğrudan değere ulaşırsınız**.
- ❖ **Kopyalama Davranışı:** Bir değer tipi değişkenini başka bir değişkene atadığınızda, değer bir kopyası oluşturulur. İki değişken birbirinden bağımsız hale gelir.
- ❖ **Bellek Yönetimi:** Yığın (**stack**) üzerinde tutuldukları için oldukça hızlıdır ve **kapsamdan (scope) çıktıkları anda bellekten otomatik olarak silinirler**.
- ❖ **Boş Olamaz:** Normal şartlarda **değer tiplerin mutlaka bir değeri vardır**. Değeri yok yani **boş (null)** denilemez! Ancak ileride anlatılacak **Nullable<T>** ile değer tipler için bu mümkün hale gelir.

Tamsayı Değer Tipleri

Tip	Boyut (Bit)	.NET Tipi	Değer Aralığı
İşaretili Tam Sayılar			
sbyte	8	SByte	-128 ile 127
short	16	Int16	-32.768 ile 32.767
int	32	Int32	-2.147.483.648 ile 2.147.483.647
long	64	Int64	-9.223.372.036.854.775.808 ile 9.223.372.036.854.775.807
İşaretsiz Tam Sayılar			
byte	8	Byte	0 ile 255
ushort	16	UInt16	0 ile 65.535
uint	32	UInt32	0 ile 4.294.967.295
ulong	64	UInt64	0 ile 18.446.744.073.709.551.615
Platform Bağımlı Tam Sayılar			
nint	32/64	IntPtr	Platforma göre değişir
nuint	32/64	UIntPtr	Platforma göre değişir

Tablo 4.3: Tam Sayı Veri Tipleri ve Özellikleri

Bir tam sayı veri tipinin, hem pozitif hem de negatif değerleri tutabileceğini düşünüyorsak **işaretili (signed)** veri tiplerinden birini seçmeliyiz. Örneğin 8 bitlik **sbyte** veri tipinde en anlamlı olan 8. bitini, işaret biti olarak kullanılmasını sağlar. Böylece -128 ile +127 arasındaki sayılar değer olarak kullanılabilir.

Benzer şekilde bir tam sayı veri tipinin, sadece pozitif değerleri tutabileceğini düşünüyorsak **işaretsiz (unsigned)** bir veri tipini seçmeliyiz. Örneğin 8 bitlik **byte** veri tipinde en anlamlı olan 8. bitinin de sayı olarak kullanılmasını sağlar. Böylece 0 ile 255 arasındaki sayılar değer olarak kullanılabilir.

C# 9.0+ ile gelen platform bağımlı tam sayılar, aslında bellek adreslerini değer olarak tutarlar. Dolayısıyla işlemcinin desteklediği adres genişliğine göre bellek boyutu değişir.

Kayan Noktalı Sayı Değer Tipleri

Kayan noktalı (floating-point) sayı tipleri, hassasiyet ve kullanım amaçlarına göre üç ana kategoriye ayrılır. Burada **decimal** veri tipinin kullanımı finansal hesaplamalar için çok kritiktir.

Tip	Boyut	Hassasiyet	.NET Tipi	Sonsuzluk/NaN
float	32 Bit	6-9 basamak	Single	Destekler
double	64 Bit	15-17 basamak	Double	Destekler
decimal	128 Bit	28-29 basamak	Decimal	Desteklemez

Tablo 4.4: Kayan Noktalı ve Ondalıklı Veri Tipleri

Diğer Değer Tipleri

Tip	Boyut	.NET Tipi	Açıklama
bool	8 Bit (1 Byte)	Boolean	true veya false değerlerini alır.
char	16 Bit (2 Byte)	Char	Tek bir Unicode karakteri tutar.
object	32/64 Bit	Object	Tüm tiplerin türediği kök tiptir.
dynamic	32/64 Bit	Object	Çalışma zamanında tipi belirlenen değişken.
string	Değişken	String	Karakter dizilerini (metin) tutar.
struct	Değişken	Kullanıcı Tanımlı	Değer tiplerden oluşan yapılar
enum	int kadar	Kullanıcı Tanımlı	İsimlendirilmiş tam sayılar

Tablo 4.5: Mantıksal, Karakter ve Temel Referans Tipleri

Tabloda;

- ♦ Mantıksal olarak true ya da false tutabilen bool değer tipi, şart ifadelerinde (expression) kullanılır.
- ♦ Unicode UTF-16 karakterini tutan 16 bitlik char değer tipi. Karakterler '\u0000' ile '\uFFFF' arasında değer alır. Değeri sıfır olan '\u0000' karakterine yani '\0' karakterine boş karakter (null character) olarak adlandırılır.
- ♦ İleride yeri geldikçe göreceğimiz yapı (struct) ve numaralandırma (enum) tipleri de değer tipleridir.
- ♦ İleride göreceğimiz tuple bir değer tipidir, ancak basit bir veri tipi değildir.

Değer tipleri doğrudan yığın bellek (stack) üzerinde yaşadığı için, bu verilere erişim ve silme işlemleri mikro saniyeler düzeyinde gerçekleşir. **Değer tiplerde bellek erişim hızı çok yüksektir.** Bu, C#'ın neden bu kadar yüksek performanslı bir dil olduğunun temel sebeplerinden biridir.

§4.2.2 Referans Tipleri

Referans tipleri (reference type), verinin kendisini değil, verinin bellekte bulunduğu adresi (referansı) tutan tiplerdir.

En temel özellikleri:

- ♦ **Bellek Yapısı:** Verinin asıl içeriği öbek bellek (heap) bölgesinde saklanır; bu verinin adresi ise yığın bellek (stack) bölgesindeki değişkende tutulur.
- ♦ **Kopyalama Davranışı:** Bir referans tipi değişkenini başka bir değişkene atadığınızda, verinin kendisi kopyalanmaz. Sadece adresi kopyalanır. Sonuç olarak iki değişken de bellekteki aynı nesneyi gösterir.
- ♦ **Varsayılan Değer:** Olmayan nesneleri gösteren referans değişkenler varsayılan olarak boş referanstır (null reference). Bu nedenle referans tiplerinin varsayılan değeri null'dır.
- ♦ **Yaşam Döngüsü:** Öbek bellek (heap) üzerindeki nesneler, artık hiçbir değişken tarafından referans gösterilmediklerinde **çöp toplayıcı GC (garbage collector)** tarafından temizlenir.

Bu tipler ilerleyen bölümlerde yeri geldikçe detaylı olarak anlatılacaktır.

Referans Tipi	Açıklama
class	Kullanıcı tanımlı sınıflar (Örnek olarak string, Form, List).
interface	Sınıfların uygulaması gereken sözleşmeleri tanımlar.
delegate	Yöntem referanslarını (göstericilerini) tutan tiplerdir.
record	Veri odaklı nesneler için kullanılır (C# 9.0+).
dynamic	Tip kontrolünün çalışma zamanında yapıldığı özel referans tipi.
object	Referans tipler de dahil tüm tiplerin en tepedeki kök sınıfıdır.
T[], T[,], ve T[,]	Dizi tipleri.

Tablo 4.6: Referans Tipler ve Özellikleri

Bir referans tipi öbek (heap) bellekte yer tahsisi ancak çalışma zamanında (run-time) oluşturabiliriz. Bellek tahsisi için **new işleci (new operator)** kullanılır. Ardından varsayılan durumlar için **yapıcılar (constructor)** çalıştırılır. Bu tiplerin yapıcıları, öbek bellekte yer ayrıldıktan sonra nesnenin durumlarını yani verilerini düzenler ve istenilen verilerle imal edilmelerini sağlar.

§4.2.3 Değişken Bildirimi ve Tanımlaması

Bildirim (declaration), derleyiciye bir değişkenin adını ve veri tipini tanıtmaktır. Değişken bildiriminde, eğer tipleri için yığın bellekte (**stack**) yer tahsis edilir, ancak referans tipleri için sadece bir gösterici (**pointer**) oluşturulur ve değeri **null** olur. Değişken bildirimine iki örnek verelim;

- ◊ Değer Tip Örneği: örnek olarak `int sayi`; talimatı (statement) yazdığımızda derleyiciye şu mesajı veririz: "Benim `sayi` adında bir değişkenim olacak ve bu `int` tipinde veri tutacak."
- ◊ Referans Tip Örneği: örnek olarak `Oğrenci ali`; talimatı (statement) yazdığımızda derleyiciye şu mesajı veririz: "Benim `Oğrenci` sınıfından imal edilecek bir nesneye `ali` adında referansım olacak ve `ali` henüz bir `Oğrenci` tipinde bir nesneyi göstermiyor. Çünkü değeri `null`"

Değişkenlerin veri tipine (**data type**) bağlı olarak benzersiz bir **kimlik (identifier)** verilerek **bildirilmesi (declaration)** gerekir!

```
byte yas; // veri tipi "char" ve kimliği "yas" olan değişken bildirimi
int kat; // veri tipi "int" ve kimliği "kat" olan değişken bildirimi
float kilo; // veri tipi "float" ve kimliği "kilo" olan değişken bildirimi
decimal para; // veri tipi "decimal" ve kimliği "para" olan değişken bildirimi
```

Tanımlama (definition), bildirimi yapılmış bir değişkene ilk değerinin atanması veya (referans tipleri için) bellekte gerçek bir nesnenin oluşturulması sürecidir. Değer tipe değer atanır veya referans tipe öbek bellek (**heap**) bölgesinde nesne için yer tahsis edilir. Referans tipe bellekte yer tahsis edildikten sonra varsayılan durumlar için **yapıcılar çalıştırılır**. Değişken tanımlamaya, **başlatma (initialization)** adı vermek yaygın bir kullanımdır. Değişken tanımlamasına iki örnek verelim;

- ◊ Değer Tip Örneği: örnek olarak `int sayi=10`; talimatı (statement) yazdığımızda derleyiciye şu mesajı veririz: "Benim `sayi` adında bir değişkenim olacak ve bu `int` veri tipinde 10 verisini tutacak."
- ◊ Referans Tip Örneği: örnek olarak `Öğrenci ali = new Öğrenci()`; talimatı (statement) yazdığımızda derleyiciye şu mesajı veririz: "`Öğrenci` sınıfından bir nesne imal et ve `ali` adında referansım onu göstertsin."

Değişkenleri kullanabilmek için bildirim yapılması yeterlidir. Ancak ilk değer verilerek de değişkenler başlatılabilir. Böylece bildirim de yapılmış olunur. Şimdi yukarıda bildirimi yapılan değişkenleri başlatan bir örneği yenileyebiliriz.

```
byte yas=45; // veri tipi "char" ve kimliği "yas" olan değişken 45 değeriyle başlatıldı
int kat=1; // veri tipi "int" ve kimliği "kat" olan değişken 1 değeriyle başlatıldı
float kilo=60.0F; // veri tipi "float" ve kimliği "kilo" olan değişken 60.0 değeriyle başlatıldı
decimal para=200m; // veri tipi "decimal" ve kimliği "para" olan değişken 200 değeriyle başlatıldı
```

Bildirim (declaration) derleyiciye böyle bir değişken veya nesne var demektir. Bunu ileride daha detaylı göreceğiz. Şimdilik tanımlama gibi kabul edebilirsiniz. **Tanımlama, (definition)** derlemenin **bağlama (link)** ile birlikte sorunsuz olabilecek şekilde eksiksiz yapılan bir işlemdir.

§4.2.4 Değişken Kapsamı

Değişkenin kapsamı (variable scope); değişkenin bildiriminin (**variable declaration**) yapılabileceği, değişkene erişilebileceği (**access**), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır. Değişkenin kapsamı, tanımlandığı kod bloğu (ve varsa bu blok içindeki iç bloklar) ile sınırlıdır.

4.2.3 Başlığında bildirimi yapılan `yas`, `kat`, `kilo` ve `para` değişkenleri ana fonksiyon olan `Program.cs` içinde tanımlandığından, tanımlanan değişkenler sadece program içinde geçerlidir. Yani değişkenin kapsamı (**scope**), bildirimi yapıldıktan sonra dosya sonuna kadardır. **Aynı kapsam içinde bildirilmiş bir değişken kimliği, bir başka değişkene verilemez!**

Süslü parantez ile tanımlanan **her kod bloğu (code block)** içinde de **değişken bildirimi yapılabilir**. Bu şekilde tanımlanan değişkenin kapsamı, **kod bloğu ile sınırlıdır**. Yani bildirimi yapıldıktan sonra değişken blok sonuna kadar geçerlidir.

Yöntemlerin de en az bir kod bloğu vardır. Bu kod blokları içinde tanımlanan değişkenlere **yerel değişken (local variable)** adı verilir. Yerel değişkenlerin kapsamı da fonksiyon bloğu ile sınırlıdır.

§4.2.5 Değişken Kimliklendirme Kuralları

C# programlama dilinde **değişkenlerin kimliklendirilmesinde (identifier definition)** dört temel kural vardır;

1. **Kimliklendirmede anahtar kelimeler kullanılamaz!**
2. **Kimliklendirme asla rakamla başlayamaz!**
3. Kimliklendirmede yukarıdaki iki kurala uyacak şekilde herhangi bir karakter kullanılabilir.
4. Aynı kapsam (**scope**) içinde **aynı kimlik iki farklı değişkene verilemez!**

Aşağıda kimliklendirme örnekleri verilmiştir;

```
//Tamsayı Değişken Bildirimleri:
byte yaş;
int tamsayı;
long uzunTamsayı;
//Gerçek Sayı Değişken Bildirimleri:
float oran;
double gerceksayı;
/*Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir. */
int kat1, kat2, kat3;
float ölçek1, ölçek2;
decimal alınanÜcret, paraÜstü;
```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılığı artırmak için **Deve Yazımı** (**camel case**) şeklinde olarak kimlik verilir. Bu kimliklendirme türünde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcılarını işini kolaylaştırmak için ahlaki olarak tercih edilir. Aşağıdaki buna ilişkin bir örnek verilmiştir.

```
int sayaç;
int öğrenciBoyu=180;
int asansörünOlduğuKat=-1;
float asansörAğırlığı=300.0F;
byte öğrenciYaşı;
char ilkHarf='A';
double ortalamaNot=0.0;
```

4.3 Değişmezler

Programcı, kodlama yaparken bazı **değişmezleri** (**literal**) ister istemez kullanır. Bu değişmezler, derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynıdır. Değişkenlere verilen **ilk değerler** (**initial value**) ile **katsayılar** (**coefficient**) en çok kullanılan değişmezlerdir. Örnek olarak **int sayı = 10;** talimatında **sayı** bir değişkendir, **10** ise bir tam sayı değişmezidir (**integer literal**).

Değişkenlerin veri tiplerine göre değişmezler çeşitli biçimlerde yazılabilir. Aşağıdaki tabloda örnekleri verilmiştir;

Veri Tipi	Değişmez(literal) Örneği	Notlar / Sonekler
Tam Sayılar		
int	100, 0b110, 0x1A	Varsayılan tam sayı tipidir.
uint	100u, 100U	u veya U soneki alır.
long	100l, 100L	l veya L soneki alır (L önerilir).
ulong	100ul, 100UL	ul, uL, UL veya UL alabilir.
byte / sbyte	(byte)10, (sbyte)-5	Doğrudan soneki yoktur, cast edilir.
short / ushort	(short)500	Doğrudan soneki yoktur, cast edilir.
Kayan Noktalı ve Ondalıklı Sayılar		
double	1.5, 1.5d, 1e2	Varsayılan ondalıklı tiptir.
float	1.5f, 1.5F	f veya F soneki zorunludur.
decimal	1.5m, 1.5M	m veya M soneki zorunludur.
Diğer Değer Tipleri		
bool	true, false	Sadece bu iki anahtar kelime.
char	'A', '\n', '\x0041'	Tek tırnak ve kaçış dizileri kullanılır.
enum	Gunler.Pazartesi	Tanımlanan enum üyesi kullanılır.

Tablo 4.7: Değer Tipleri İçin Değişmez Biçimleri

Türkçe **1.025,75** olarak yazılan bir gerçek sayı, İngilizce **1,025.75** olarak yazılır. Türkçede bindelik ayracı nokta ve ondalık ayracı virgül iken, İngilizcede bindelik ayracı virgül, ondalık ayracı noktadır. Kodlama İngilizce olarak yapıldığından **gerçek sayılar İngilizcedeki gibi yazılır.**

Aşağıda bazı değişmezlerle ilişkin örnek verilmiştir;

```
uint işaretliTamsayı = 5U;
int tamsayı = -5;
sbyte işaretliBayt = -120;
decimal ondalık = 30.5M;
double çiftİncelikliKayanNoktalıSayı = 30.5D;
float kayanNoktalıSayı = 30.5F;
long uzunTamsayı = 5L;
ulong işaretliUzunTamsayı = 5UL;
byte bayt = 127;
short kısaTamsayı = -12;
ushort işaretliKısaTamsayı = 220;
bool evetHayır = true;
bool doğruYanlış = false;
char karakter1 = 'ö'; // Karakter değişmezleri tek tırnak içinde yazılır.
char karakter2 = '\u006A'; // Tek tırnak içinde UTF-16 kodlamasına göre
char karakter3 = '\x0013';
char karakter4 = 'Ş';
/*
Metin değişmezler çift tırnak arasında yazılır.
Metin içinde;
Çift tırnak kullanmak için \" yazılır.
Bir sonraki satıra geçmek için \ karakteri kullanılır.
\ karakteri kullanmak için \\ yazılır.
*/
string metin = "hello, this is a string literal";
/*
Yukarıdaki metin değişmesinde birçok kural vardır.
Onları gözardı etmek için @ karakteri ile başlayan tam metin (verbatim string) değişmesi
→ kullanılır.
Tam metinlerde sadece çift tırnak için iki adet "" kullanılır.
*/
string tammetin = @"İlhan
Özkan \ """;
```

Aşağıdaki kod örneğinde programcı, 3F, 3.14F, 3.14F ve 2.0F olmak üzere dört farklı değişmez kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir.

```
float yariCap = 3F;
float daireninAlani = 3.14F * yariCap * yariCap;
float daireninCevresi = 2.0F * 3.14F * yariCap;
```

Kodu 0 olan karakter ise **boş karakter (null character)** olarak adlandırılır.

4.4 Yerel Sabitler

Değişmezlerin (**literal**) çokça kullanılması derleme sonrası üretilen icra edilebilir montaj kodu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye yerel değişkenler **sabitler (const)** olarak tanımlanabilir. Bunun için **const** anahtar kelimesi sıfat olarak kullanılır ve bu değişken **yerel sabit** adını alır. Başlatması yapıldıktan sonra yerel sabitler ve değiştirilemezler. Sabit değişkenler etik olarak büyük harfle kimliklendirilir.

```
const float PI=3.14F; //Yerel sabiti başlatma yapmazsak daha sonra değiştiremeyiz!
float yariCap = 3F;
float daireninAlani = PI * yariCap * yariCap;
float daireninCevresi = 2.0F * PI * yariCap;
```

Örnekteki aynı yerel sabiti PI birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğin-

den bellekten tasarruf edilmiş olunur. Bildirim sırasında `const` sıfatı kullanılan yerel sabit için bildirim yapılmalı değişken başlatılmalıdır. Bir sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- ◊ **Gelişmiş Kod Okunabilirliği:** Bir değişkene `const` sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- ◊ **Gelişmiş Veri Tipi Güvenliği:** `const` kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- ◊ **Gelişmiş Optimizasyon:** Derleyiciler, değerlerinin programın icrası sırasında değişmeyeceğini bildikleri için yerel sabitleri daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- ◊ **Daha İyi Bellek Kullanımı:** Değişkenlere `const` sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- ◊ **Gelişmiş Uyumluluk:** Değişkenlere `const` sıfatı vermek, kodunuzdaki yerel sabitleri kullanan diğer kodlar ve API'lerle daha uyumlu hale getirebilir.
- ◊ **Gelişmiş Güvenilirlik:** `const` kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

4.5 Anonim Tipler

Değişkenler veri tipi verilmeden de tanımlanabilir. Bu durumda veri tipi değişkene verilen ilk değer değişiminden (*literal*) çıkarılır. Bu tip değişkenlere **anonim tip** (*anonymous type*) adı verilir. Anonim tipler `var` anahtar kelimesiyle tanımlanır ve **statik tip denetimini korur** yani tip denetimi derleme zamanı (*compile-time*) yapılır. Aşağıda değer tipler için bir örnek verilmiştir;

```
var pi=3.14F; //pi:float
var sayac=0; //sayac:int
var durum=false; //durum:bool
var gercek=10.5; //gercek:double
var ondalıklı=100000.0m; //ondalıklı:decimal
```

Aşağıda verilen örneklerdeki **anonim tipler** (*anonymous type*) aslında referans tiptir. Referans tipler ileride detaylı anlatılacaktır. Örnek olarak `var anon=new { x = 1, y = 2 }` talimatını göz önüne alalım;

- ◊ Önceden bir `class` veya `struct` tanımlaması yapmadan, sadece o anlık ihtiyacımızı karşılayacak geçici bir nesne öbek bellekte (*heap*) oluşturuyoruz. Oluşturulan nesne **isimsizdir**. Yazdığımız kod tarafında bu tipe verebileceğiniz bir isim yoktur. Bu yüzden `var` kullanmak zorunludur.
- ◊ Oluşturulan nesne salt okunurdur (*read-only*). Oluşturulan bu nesnenin özellikleri de (x ve y) daha sonra değiştirilemez. Yani `anon.x = 5`; yazarsanız derleme hatası alırsınız.
- ◊ Anonim tipler arka planda birer `class` olarak oluşturulur, dolayısıyla belleğin öbek bellek (*heap*) bölgesinde yaşarlar.

```
var anon = new { Value = 1 };
Console.WriteLine(anon.Value); //1
//Console.WriteLine(anon.Id); Derleme Hatası Olur!
var anon2 = new { x = 1, y = 2 };
// anon.x == 1
// anon.y == 2
int a = 10;
int b = 20;
var anon3 = new { a, b };
// anon3.x == 10
// anon3.y == 20
var anon4 = new { adı="ilhan", yaşı=50 }; //Anonim Tipler ileride anlatılacaktır.
// anon4.adı == "ilhan"
// anon4.yaşı == 50
```

Anonim tiplerin eşitliği de sorgulanabilir;

```
var anon = new { Foo = 1, Bar = 2 };
var anon2 = new { Foo = 1, Bar = 2 };
var anon3 = new { Foo = 5, Bar = 10 };
var anon3 = new { Foo = 5, Bar = 10 };
var anon4 = new { Bar = 2, Foo = 1 };
// anon.Equals(anon2) == true
// anon.Equals(anon3) == false
// anon.Equals(anon4) == false (anon ve anon4 farklı tiplerdir)
```

Anonim tiplerin içerdiği özellikler aynı ise aynı olarak kabul edilir;

```
var anon = new { Foo = 1, Bar = 2 };
var anon2 = new { Foo = 7, Bar = 1 };
var anon3 = new { Bar = 1, Foo = 3 };
var anon4 = new { Fa = 1, Bar = 2 };
// anon ve anon2 aynı tiptir
// anon ve anon3 farklı tiptir (Bar ve Foo farklı sırada olduğundan)
// anon ve anon4 farklı tiplerdir (özellik isimleri farklı olduğundan)
```

4.6 Dinamik Değişkenler

Dinamik değişkenler (**dynamic variables**), anonim değişkenlerin aksine, değişkenlerin tipi dinamik olarak derleme zamanı yerine çalışma zamanında (**run-time**) veri tipi kontrolü yapılan değişken tiptir.

dynamic anahtar kelimesi, dilin statik yani derleme zamanı (**compile-time**) yapısından dinamik yani çalışma zamanı (**run-time**) yapısına geçtiği sihirli bir köprüdür. C#4.0+ ile gelen bu özellik, derleyiciye şu mesajı verir: "Bu değişkenle ne yapacağıma şimdi karışma, çalışma zamanında (**run-time**) bakarız."

```
dynamic val = "Merhaba";
Console.WriteLine(val.Id); /* Derlenir, ama çalışma anında istisna olur! Çünkü ID adlı bir
→ özelliği bulunmamaktadır */
```

Normalde C# derleyicisi, bir değişken üzerinde yöntem çağırdığınızda o yöntemin var olup olmadığını derleme aşamasında kontrol eder. Ancak **dynamic** olarak tanımlanan bir değişkende bu kontrol atlanır;

- ♦ Yanlış bir yöntem çağırırsanız bile kod derlenir, ancak program çalışırken hata (**RuntimeBinderException**) verir. Bu nedenle **derleme zamanı güvenliği yoktur**.
- ♦ Bir **dynamic** değişken önce **int**, sonra **string** tutabilir. Yani **tip değişebilir!**
- ♦ Çalışma zamanında **object** gibi davranır ama üzerindeki her işlem dinamik olarak çözülür.

Aşağıda buna ilişkin bir örnek verilmiştir;

```
dynamic veri = 10;
Console.WriteLine(veri.GetType()); // Çıktı: System.Int32
veri = "Merhaba C#"; //Tip Değişikliği!
Console.WriteLine(veri.GetType()); // Çıktı: System.String
dynamic nesne = "C# Kitabı";
try {
    // string sınıfında 'ToUpper' vardır, sorunsuz çalışır.
    Console.WriteLine(nesne.ToUpper());
    // 'Yuz' diye bir metod yoktur. Derleme hatası ALMAZSINIZ. Çünkü derleyici Yuz() diye bir
    → metodun olup olmadığını kontrol etmez!
    // Ancak çalışma zamanında PROGRAM ÇÖKER.
    nesne.Yuz();
}
catch (Exception ex) {
    Console.WriteLine("Hata: " + ex.Message);
}
```

4.7 Boş Olabilir Değer Tipler

Değer tiplerin alabileceği alt ve üst sınırlar bellidir. Ancak bir değer tipe, değeri yok anlamında **boş** (**null**) atayamazsınız. Bunu sağlamak için değişkeni **boş olabilir** (**nullable**) şekilde aşağıdaki gibi tanımlama yapılır;

```
Nullable<int> i1 = null;
//ya da kısaca veri tipi sonunda ? karakteri kullanılır!
int? i2 = null;
//ya da
var i3 = (int?)null;
```

Boş olabilir (**nullable**) değişkeni aşağıdaki şekilde **null** olmayan bir değerle tanımlayabilirsiniz.

```
//Nullable<int> i = 0;
//Ya da kısaca:
int? i = 0;
```

Boş olabilen değişkenlerde **null denetimi** ve **varsayılan değer atama** işlemlerini tek bir satıra indiren **null birleştirme işleci** (**null coalescing operator**) (**??**) vardır;

```
string isim = GetUser() != null ? GetUser() : "Anonim"; //Eski yöntem.
string isim = GetUser() ?? "Anonim"; //Yeni Yöntem: null coalescing operator
```

Birden fazla `null` ihtimalini ardı ardına kontrol etmek için bu işleci zincirleyebilirsiniz. Bu, "bulabildiğin ilk geçerli değeri al" mantığıdır.

```
string adres = evAdresi ?? isAdresi ?? "Adres Bilgisi Yok";
```

C#8.0+ ile gelen **null atama işleci** (`??=`), bir değişken eğer `null` ise ona bir değer atamak için kullanılır. Eğer değişken zaten bir değere sahipse, atama işlemi gerçekleşmez;

```
List<int> sayilar = null;
// Eger sayilar null ise yeni bir liste olustur ve ata
sayilar ??= new List<int>();
sayilar.Add(1);
```

`null` birleştirme işleci (`??`) sadece kodunuzu kısaltmaz, aynı zamanda `null reference exception` hatalarının önüne geçmek için en güçlü savunma hatlarınızdan biridir!

Ayrıca boş olabilir (**nullable**) bir veri tipini değer tipe aşağıdaki şekilde atayabiliriz.

```
int? i = 10;
int j = i ?? 0; // null ise 0 - null coalescing operator
int j = i.GetValueOrDefault(0); // null ise 0
int j = i.HasValue ? i.Value : 0; // null ise 0 - conditional tenary operator
```

4.8 Numaralandırma Tipi

Numaralandırma (**enumeration**) olarak adlandırılan **enum**, bir grup tam sayı ile ilişkilendirilmiş değişmezden (**literal**) oluşan numaralandırılmış kullanıcı tanımlı bir değer veri tipidir.

Bir **enum** tam sayı veri tiplerinin (**byte**, **sbyte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**) herhangi birinden türetilir. Varsayılan olarak **int** veri tipi altta yatar ve **enum** tanımında veri tipi belirterek değiştirilebilir;

```
enum MedeniHal { Belirtilmemiş, Evli, Bekar, Boşanmış, Dul }; //Ön tanımlı olarak int
enum Günler:byte { Pazartesi = 1, Salı = 2, Çarşamba = 3, Perşembe = 4, Cuma = 5 }
enum Aylar:int { Ocak = 1, Şubat = 2, Mart = 3, Nisan = 4, Mayıs = 5, Haziran = 6, Temmuz = 7,
    → Ağustos = 8, Eylül = 9, Ekim = 10, Kasım = 11, Aralık = 12 }
enum Renkler:uint { Kırmızı = 1, Mavi = 2, Yeşil = 3, Sarı = 4, Beyaz = 5, Siyah = 6 }
```

```
byte işeBaşlananGün=1; // Pazartesi
Günler işbaşı = Günler.Pazartesi;
Aylar ramazan = Aylar.Mart;
Renkler arabanınRengi = Renkler.Kırmızı;
```

Numaralandırmalar bir tam sayının her bir bitini temsil edecek şekilde bayrak (**flag**) olarak kullanılabilir. Burada **int** veri tipinin 32 bitten oluştuğu unutulmamalıdır!

```
[Flags]
enum Bayraklar
{
    //None = 0
    BirinciDurum = 1, //1.Bit
    İkinciDurum = 2, //2.Bit
    ÜçüncüDurum = 4, //3.Bit
    DördüncüDurum = 8, //4.Bit
    BeşinciDurum = 16 //5.Bit
}

[Flags]
enum FlagsEnum
{
    None = 0,
    Option1 = 1,
    Option2 = 2,
    Option3 = 4,
    Default = Option1 | Option3,
    All = Option1 | Option2 | Option3,
}
```



```

var İkiBayrakHavada = Bayraklar.BeşinciDurum | Bayraklar.ÜçüncüDurum;
Console.WriteLine(İkiBayrakHavada); //ÜçüncüDurum, BeşinciDurum
var ÜçüncüBayrakHavada = Bayraklar.ÜçüncüDurum;
Console.WriteLine(ÜçüncüBayrakHavada); //ÜçüncüDurum
var BaşkaİkiBayrakHavada = Bayraklar.BirinciDurum | Bayraklar.İkinciDurum;
Console.WriteLine(BaşkaİkiBayrakHavada); //BirinciDurum, İkinciDurum

```

4.9 Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı CIL koduna çevrilen programımız aslında yazdığımız talimatları (**statement**) sırasıyla çalıştırır. Aşağıdaki örnek programı göz önüne alalım;

```

1 // See https://aka.ms/new-console-template for more information
2 float boy = 1.80F;
3 float kilo = 100.0F;
4 float bki;
5 float boyKare;
6 boyKare = boy * boy;
7 bki = kilo / boyKare;
8 boy = 1.50F;

```

Yapısal programlamada program, ana fonksiyondan başlar. Yeni proje şablonu ile oluşturulan projelerde **Program.cs** ana fonksiyonu kodladığımız yerdir. Bu nedenle icra, **Program.cs** içindeki ilk talimatla başlar sırayla icra edilir.

1. Satırda, .Net6+ sonrasında yeni proje şablonuyla oluşturulan yeni konsol uygulamasının açıklama satırı bulunur.
2. İcra edilecek ilk talimat 2. satırdaki talimattır. **boy** değişkeni **1.80F** değeri ile başlatılır.
3. Saha sonra 3. satırdaki talimat icra edilir. **kilo** değişkeni **100.0F** değeri ile başlatılır.
4. Ondan sonra 4. satırdaki talimat icra edilir. **bki** değişkeni bildirilir.
5. Daha sonra 5. satırdaki talimat icra edilir. **boyKare** değişkeni bildirilir.
6. Ondan sonraki talimat 6. satırdaki talimattır. **boy** ile **boy** değişkeni çarpılır ve sonuç **boyKare** değişkenine atanır. Artık **boyKare** değişkeninin değeri bu noktadan sonra **3.24** olmuştur.
7. Daha sonra 7. satırdaki talimat icra edilir. **kilo** değişkeni **boyKare** değişkenine bölünür (100,0/3,24) ve sonuç **bki** değişkenine atanır. Artık **bki** değişkeninin değeri bu noktadan sonra **30.86419753** olmuştur.
8. Son olarak 8. satırdaki talimat icra edilir. **boy** değişkenine **1.50** atanmıştır. Bu atama sonrası **boyKare** ve **bki** değişkenlerinin değeri değişmemiştir. Çünkü bu değişkenleri değiştiren 6. ve 7. satırdaki talimatlar icra edilmiştir.

4.10 Temel İşleçler

İşleçler (operator) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen (operand)** adı verilir. Yüksek düzey dillerde de aritmetik işleçlerin yanında birçok işleç de bulunur.

En çok kullanılan işleç, atama (=) işlecidir.
Atama işleci, sağdaki işleneni soldakine atar.
 Bu nedenle kodlama yapılırken **2+2=x** veya **a+b=x** yazılamaz!

§4.10.1 Aritmetik İşleçler

C# programlama dilinde **aritmetik işleçlerin (arithmetic operator)** çalışma şekli işlenenlerin (**operand**) veri tipine göre değişir. Bu işleçler Tablo 4.8’de verilmiştir. Bu işleçler **int**, **uint**, **long** ve **ulong** tiplerini işlenen olarak alır. İşlenenler diğer tam sayı tipleri olduğunda (**sbyte**, **byte**, **short**, **ushort** veya **char**) değerleri bir işlemin sonuç tipi olan **int** tipine dönüştürülür.

Aritmetik işleçlere (arithmetic operator) ilişkin örneği aşağıdaki görebilirsiniz;

```

int a = 9, b = 4, res;

res = a + b; // işlem sonucu res=13
res = a - b; // işlem sonucu res=5
res = a * b; // işlem sonucu res=36

```


İşleç	Adı	Açıklama
+	Toplama	İki işleneni toplar. Sayısal olmayan bazı referans tiplerde (örn. <code>string</code>) birleştirme yapar.
-	Çıkarma	Soldaki işlenenden sağdakini çıkarır.
*	Çarpma	İki işleneni çarpar.
/	Bölme	Soldaki işleneni sağdakine böler. Her iki işlenen tamsayı ise sonuç tamsayıdır (ondalık kısım atılır).
%	Modül	Soldaki işlenenin sağdakine bölümünden kalan değeri verir.

Tablo 4.8: Aritmetik İşleçler

```
res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
/* Toplama işlecinin işlenenleri tamsayı olduğundan, bölme sonucundaki tam kısım bölme
   ↳ sonucudur.*/
res = (int)(a / 4.3);
/* İşlem sonucu double olacaktır. Bu durumda res değişkeni de int olduğundan sonucu int türüne
   ↳ çevirmemiz gerekir. res=9/4.3=9.0/4.3=2.0930= 2 tam 0.0930=2 */
res = a % b; // işlem sonucu res=9%4= 2 tam 1/4=1
```

```
float fa = 6.6F, fb = 2.2F, fres;
fres = fa / fb; // işlem sonucu res=6.6/2.2=3.00
```

Tam sayılarda bölme işlemi, işlemcinin en basit yetenekleriyle çözülür. Aşağıda buna ilişkin örnek verilmiştir.

```
int a = 19 / 2; /* a = 2*9+1 = 9 */
int b = 18 / 2; /* b = 9 */
int c = 255 / 4; /* c = 4*63+3 = 63 */
int d = 45 / 4; /* d = 4*11+1 = 11 */
double aa = 19 / 2.0; /* aa = 9.5 */
double bb = 18.0 / 2; /* bb = 9.0 */
double cc = 255 / 2.0; /* cc = 127.5 */
double dd = 45.0 / 4; /* dd = 11.25 */
double ee = 45 / 4; /* ee = 11 çünkü bölme tamsayılar arasında yapılmıştır */
```

§4.10.2 Tekli İşleçler

Tekli işleçler (**unary operator**) sadece **bir işlenen üzerinde işlem yapar**. Özellikle bellek yönetimi ve mantıksal akış bölümlerinde sıkça karşımıza çıkar.

İşleç	Adı	Açıklama
+	Tekli Artı	İşlenenin işaretini pozitif yapar (genellikle varsayılan durumdur).
-	Tekli Eksi	İşlenenin işaretini tersine çevirir (x ise $-x$, $-x$ ise x yapar).
++	Artırım	İşlenenin değerini 1 artırır. Önek (prefix) ise önce artırır sonra değeri döndürür; sonek (postfix) ise önce değeri döndürür sonra artırır.
--	Eksiltme	İşlenenin değerini 1 eksiltir. Önek ve sonek kullanım kuralları artırım işleci ile aynıdır.
!	Mantıksal Değil	bool değerini tersini alır (true ise false yapar).
(T)	Tip Dönüşümü	İşleneni parantez içinde belirtilen veri tipine (casting) dönüştürür.
&	Adres Operatörü	unsafe kod bloklarında değişkenin bellek adresini döndürür.
*	Dolaylı Erişim	unsafe kod bloklarında göstericinin (pointer) işaret ettiği değeri döndürür.
sizeof	Bellek Boyutu	sizeof temel veri tiplerin bellek boyutunu geri döndürür.

Tablo 4.9: Tekli İşleçler

Bunların bir kısmı konusu geldikçe kullanımı anlatılacaktır. Aşağıda tekli işleçlerin kullanımına ilişkin örnek program verilmiştir;

```
int a = 5, b = 5, res;
// 1. Tekli Eksi (Unary Minus)
res = -a; // res = -5
// 2. Önce-Artırım (Pre-Increment)
// a önce 6 olur, sonra 2 ile toplanır.
res = ++a + 2;
```

```

/* Sonuç: res = 8, a = 6 */
// 3. Sonra-Artırım (Post-Increment)
// b'nin mevcut değeri (5) ile 2 toplanır, sonra b 6 olur.
res = 2 + b++;
/* Sonuç: res = 7, b = 6 */

a = 5; b = 5;
// 4. Önce-Eksiltme (Pre-Decrement)
// a önce 4 olur, sonra 2 ile toplanır.
res = --a + 2;
/* Sonuç: res = 6, a = 4 */
// 5. Sonra-Eksiltme (Post-Decrement)
// b'nin mevcut değeri (5) ile 2 toplanır, sonra b 4 olur.
res = 2 + b--;
/* Sonuç: res = 7, b = 4 */

// 6. Mantıksal Değil (!)
// C#'ta ! işleci sadece bool ile çalışır. C dilindeki gibi 0/1 (int) kabul etmez.
bool mantiksalA = true;
bool mantiksalB = false;
bool resBool;
resBool = !mantiksalA; // resBool = false
resBool = !mantiksalB; // resBool = true

// 7. Bellek Boyutu (sizeof)
// sizeof C#'ta genellikle temel veri tipleriyle kullanılır.
int boyutInt = sizeof(int); // sonuç: 4
int boyutChar = sizeof(char); // sonuç: 2 (C#'ta char Unicode'dur!)

// Değişkenler için sizeof ancak 'unsafe' bloklarda veya Marshal.SizeOf yöntemiyle
→ kullanılabilir. İleride Anlatılacaktır.

```

Ş4.10.3 İlişkisel İşleçler

İlişkisel işleçler (**relational operator**), işlenenlerin arasındaki ilişkiyi verirler. İlişkisel İşleçler, ileride göreceğimiz karar yapılarının (**if**, **switch**) ve döngülerin (**while**, **for**) temelini oluşturur. Bu işleçler, iki değeri karşılaştırıp sonucu her zaman bir mantıksal değer olan **bool** veri tipinde döndürürler.

İşleç	Adı	Açıklama
==	Eşittir	Sol ve sağdaki işlenenler birbirine eşitse true döner.
!=	Eşit Değildir	İşlenenler birbirine eşit değilse true döner.
>	Büyüktür	Soldaki işlenen sağdakinden büyükse true döner.
<	Küçüktür	Soldaki işlenen sağdakinden küçükse true döner.
>=	Büyük veya Eşit	Soldaki işlenen sağdakinden büyük veya ona eşitse true döner.
<=	Küçük veya Eşit	Soldaki işlenen sağdakinden küçük veya ona eşitse true döner.

Tablo 4.10: İlişkisel İşleçler ve Mantıksal Karşılıkları

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```

int a = 5, b = 6;
bool res;
res = a < b; // Küçüktür: a, b'den küçük olduğu için sonuç true
res = a <= b; // Küçük veya Eşit: a, b'den küçük olduğu için sonuç true
res = a > b; // Büyüktür: a, b'den büyük olmadığı için sonuç false
res = a >= b; // Büyük veya Eşit: a, b'den büyük veya eşit olmadığı için sonuç false
res = a == b; // Eşittir: a ve b değerleri farklı olduğu için sonuç false
res = a != b; // Eşit Değildir: a ve b farklı olduğu için sonuç true

```

Ş4.10.4 Bit Düzeyi İşleçler

Bit düzeyi işleçler (**bitwise operator**) özellikle ikilik tabandaki **tam sayılarda karşılıklı bitler üzerinde yapılan işlemlerdir**. Aslında bu işlemler, işlemci üzerinde çarpma, toplama, bölme ve çıkarma işlemlerinin temelini oluşturur. Bu işleçler genellikle düşük seviyeli programlama, protokol geliştirme, kriptografi ve per-

formans optimizasyonu gibi alanlarda kullanılır. Aşağıda tam sayılar üzerinde yapılan bit düzeyi işlemler

İşleç	Adı	Açıklama
&	VE (AND)	Karşılıklı iki bit de 1 ise sonuç 1, diğer durumlarda 0 olur.
	VEYA (OR)	Karşılıklı bitlerden en az biri 1 ise sonuç 1, ikisi de 0 ise 0 olur.
^	ÖZEL VEYA (XOR)	Bitler birbirinden farklıysa 1, aynıysa 0 olur.
~	DEĞİL (NOT)	Tekli işleçtir. Tüm bitleri tersine çevirir (0 → 1, 1 → 0).
<<	Sola Kaydırma	Bitleri belirtilen sayı kadar sola kaydırır. Sağdan 0 ile doldurur.
>>	Sağa Kaydırma	Bitleri belirtilen sayı kadar sağa kaydırır.

Tablo 4.11: Bit Düzeyi İşleçler

gösterilmiştir;

```
// C#'ta ikili sayıları 0b öneki ile tanımlarız.
byte a = 0b0000_0110; // 6
byte b = 0b0000_0010; // 2

// 1. Bit Düzeyi VE (AND)
// Karşılıklı her iki bit de 1 ise sonuç 1 olur.
byte bitwiseAnd = (byte)(a & b); // 0000010 (2)
// 2. Bit Düzeyi VEYA (OR)
// Karşılıklı bitlerin biri 1 ise sonuç 1 olur.
byte bitwiseOr = (byte)(a | b); // 00000110 (6)
// 3. Bit Düzeyi ÖZEL VEYA (XOR)
// Karşılıklı bitler birbirinden farklı ise sonuç 1 olur.
byte bitwiseXor = (byte)(a ^ b); // 00000100 (4)
// 4. Bit Düzeyi DEĞİL (NOT)
// Tüm bitleri tersine çevirir.
// Not: ~b işlemi sonucu int döneceği için byte'a cast edilir.
byte bitwiseNot = (byte)(~b); // 11111101 (253)
// 5. Sola Kaydırma (Left Shift)
// Bitleri 2 basamak sola kaydırır (Sayıyı 2^2 ile çarpar).
byte leftShift = (byte)(b << 2); // 00001000 (8)
// 6. Sağa Kaydırma (Right Shift)
// Bitleri 1 basamak sağa kaydırır (Sayıyı 2^1'e böler).
byte rightShift = (byte)(b >> 1); // 00000010 (2)
```

İşaretsiz sağa kaydırma (unsigned right shift) işleci (>>), C# 11 ile birlikte dilimize eklenen modern ve oldukça teknik bir işleçtir. Standart sağa kaydırma işlecinin (>) aksine, sayının işaretine (pozitif veya negatif olmasına) bakmaksızın en soldaki boşlukları her zaman 0 ile doldurur. C# dilinde iki tür sağa kaydırma vardır:

1. Aritmetik Kaydırma (>>): Sayı negatifse, en soldaki bit (işaret biti) 1 olduğu için boşalan yerleri 1 ile doldurur. Bu, sayının işaretini korur.
2. Mantıksal/İşaretsiz Kaydırma (>>>): Sayı ne olursa olsun, en soldaki boşalan bitleri her zaman 0 ile doldurur.

```
// -8 sayısının ikilik (binary) temsili (32-bit):
// 11111111 11111111 11111111 11111000
int sayi = -8;
// 1. Standart Sağa Kaydırma (>>): İşaret korunur
int aritmetik = sayi >> 2;
// Sonuç: 11111111 11111111 11111111 11111110 (-2)
// 2. İşaretsiz Sağa Kaydırma (>>>): Sol taraf 0 ile dolar
int mantiksal = sayi >>> 2;
// Sonuç: 00111111 11111111 11111111 11111110 (1,073,741,822)
```

§4.10.5 Mantıksal İşleçler

Mantıksal ifadeler, doğru (**true**) ve yanlış (**false**) olabilen ifadelerdir. C# dilinde sıfırdan farklı bir tam sayı ifade **true**, sıfır olan tam sayı yanlış **false** olarak değerlendirilir. **Mantıksal işleçlere** (**logical operator**) işlenen olarak giren ifadeler öncelikle **bool** veri tipine dönüştürülür.

Bu işleçlerin çalışma şekli doğruluk tablosundaki gibidir ancak derleyici optimizasyonu gereği, ifadenin bir kısmı işlenmediğinden, **koşullu olarak çalışırlar**. Buna **kısa devre** adı verilir. Derleyici, sonucun kesinleştiği noktada işlem yapmayı bırakır. Örneğin (**a == 5 && b == 10**) ifadesinde **a** değeri 5 değilse, **b**'nin ne

İşleç	Mantıksal İşlem ve Kısa Devre Davranışı
&&	Koşullu Mantıksal VE: Her iki işlenen de true ise true döner. Soldaki işlenen false ise sağdaki ifade değerlendirilmez (Kısa Devre).
	Koşullu Mantıksal VEYA: İşlenenlerden en az biri true ise true döner. Soldaki işlenen true ise sağdaki ifade değerlendirilmez (Kısa Devre).
!	Mantıksal DEĞİL: İşlenenin mantıksal durumunu tersine çevirir. true değerini false , false değerini true yapar.

Tablo 4.12: Mantıksal İşleçler ve Kısa Devre Özelliği

olduğuna bakmaya gerek duymaz. Bu durum sadece hız kazandırmaz, aynı zamanda olası hataları da önler:

```
int a=10, b=10;
if (a == 5 && b == 10) // a 5 den farklı olduğu için b==10 kontrol edilmez!
{
    // Buradaki işlemler yapılmaz!...
}
```

C# dilinde tekli & ve | işleçleri de mantıksal ifadelerle kullanılabilir. Ancak bunlar **kısa devre yapmazlar**; yani sol taraf ne olursa olsun sağ tarafı da kontrol ederler. Aşağıdaki kod parçasında && yerine & kullanılsaydı programın davranışı nasıl değişirdi?

```
string metin = null;
if (metin != null && metin.Length > 0) // Sorunsuz çalışır.
{
    Console.WriteLine("Metin dolu.");
}
if (metin != null & metin.Length > 0) // PROGRAM ÇÖKER!
```

Verilen kodda & işleci kısa devre yapmadığı için **metin**, **null** olsa bile sağdaki **metin.Length** ifadesini hesaplamaya çalışır ve **NullReferenceException** hatasına sebep olur.

Aşağıda verilen ilişkisel ve mantıksal işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// 1. Mantıksal VE (&&)
res = (a != 0) && (b != 0); // true
// 2. Mantıksal VEYA (||)
res = (a > 0) || (b > 10); // true
// 3. Mantıksal DEĞİL (!)
bool durum = (a == 5); // true
res = !durum;           // false
```

§4.10.6 Atama işleçleri

Atama işleçleri (**assignment operator**) en çok kullanılan işleçlerdir.

Atama işleçleri her zaman sağdaki değeri sola atar!

Dolayısıyla soldaki işlenen, atama yapılacak uygun veri tipinde bir değişken (**variable**) olmak zorundadır.

Aşağıda verilen atama işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// 1. Temel Atama
int a = 10; // a değişkenine 10 değişmezi (literal) atanıyor.
// 2. Aritmetik Bileşik Atama İşleçleri
a += 10; // a = a + 10; (a = 20)
a -= 10; // a = a - 10; (a = 10)
a *= 10; // a = a * 10; (a = 100)
a /= 10; // a = a / 10; (a = 10)
a %= 10; // a = a % 10; (a = 0)
// 3. Bit Düzeyi Bileşik Atama İşleçleri
int b = 0b0101; // 5
b &= 0b0011;    // b = b & 3; (b = 1 olur)
b |= 0b0010;    // b = b | 2; (b = 3 olur)
b ^= 0b0001;    // b = b ^ 1; (b = 2 olur)
// 4. Kaydırma Bileşik Atama İşleçleri
int c = 4;
```

İşleç	Adı	Örnek	Açıklama
=	Temel Atama	x = 5	Sağdaki değeri soldaki değişkene aktarır.
+=	Toplayarak Atama	x += 3	x = x + 3 ile aynıdır.
-=	Çıkararak Atama	x -= 2	x = x - 2 ile aynıdır.
*=	Çarparak Atama	x *= 4	x = x * 4 ile aynıdır.
/=	Bölerek Atama	x /= 2	x = x / 2 ile aynıdır.
%=	Modül Alarak Atama	x %= 3	x = x % 3 ile aynıdır.
&=	Bit Düzeyi VE ile Atama	x &= 1	x = x & 1 ile aynıdır.
=	Bit Düzeyi VEYA ile Atama	x = 1	x = x 1 ile aynıdır.
^=	Bit Düzeyi ÖZEL VEYA ile Atama	x ^= 1	x = x ^ 1 ile aynıdır.
<<=	Bit Düzeyi Sola Kaydırma ile Atama	x <<= 1	x = x << 1 ile aynıdır.
>>=	Bit Düzeyi Sağa Kaydırma ile Atama	x >>= 1	x = x >> 1 ile aynıdır.
>>>=	Bit Düzeyi İşaretsiz Sağa Kaydırma ile Atama	x >>>= 1	x = x >>> 1 ile aynıdır.
??=	Null Atama (C# 8.0+)	x ??= 5	x null ise 5 değerini atar.

Tablo 4.13: Atama İşleçleri

```
c <<= 1; // c = c << 1; (c = 8)
c >>= 2; // c = c >> 2; (c = 2)
// 5. Null-Coalescing Assignment (C# 8.0+)
string metin = null;
metin ??= "Varsayılan Değer"; // metin null olduğu için "Varsayılan Değer" atandı.
```

§4.10.7 İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C programlama dilinde yazılan ifadelerde de **işleç öncelikleri** (**operator precedence**) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez içerisine alınır. En içteki parantez en öncelikli olarak gerçekleştirilir.

Kategori	İşleçler	Birleşme	Öncelik
Temel	x.y, f(x), a[i], x++, x--, new, sizeof	Soldan Sağa	1 (En Yüksek)
Tekli	+x, -x, !x, x, ++x, --x, (T)x	Sağdan Sola	2
Çarpma/Bölme	*, /, %	Soldan Sağa	3
Toplama/Çıkarma	+, -	Soldan Sağa	4
Bit Düzeyi Kaydırma	<<, >>	Soldan Sağa	5
İlişkisel	<, >, <=, >=, is, as	Soldan Sağa	6
Eşitlik	==, !=	Soldan Sağa	7
Mantıksal VE	& (Bit Düzeyi/Mantıksal)	Soldan Sağa	8
Mantıksal XOR	^ (Bit Düzeyi/Mantıksal)	Soldan Sağa	9
Mantıksal VEYA	(Bit Düzeyi/Mantıksal)	Soldan Sağa	10
Koşullu VE	&& (Kısa Devre)	Soldan Sağa	11
Koşullu VEYA	(Kısa Devre)	Soldan Sağa	12
Null Birleştirme	??	Sağdan Sola	13
Atama	=, +=, -=, *=, /=, %=, ??=	Sağdan Sola	14 (En Düşük)

Tablo 4.14: İşleç Öncelik Hiyerarşisi

Başka işleçler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır. Aşağıda işleçlerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```
int a = 10, b = 5;
// 1. Örnek: Çıkarma ve Toplama Aynı Önceliklidir (Soldan Sağa)
a = a - b + 2; // a = 10 - 5 + 2 -> 5 + 2 = 7
// 2. Örnek: Bölme, Toplamadan Önceliklidir
// a = 7 / 5 + 2
// Not: 7/5 tam sayı bölmesi olduğu için 1.4 değil 1 olur.
// a = 1 + 2 = 3
a = a / b + 2; //3
// 3. Karmaşık Örnek: Hiyerarşik Çözüm
// Öncelik: ( ) > * , / > + , -
a = 2 * 10 - 20 / 2 + 3 + (12 - 10); //15
```

```
/* İfadenin çözüm sırası:
1. Parantez içi: (12 - 10) = 2
2. Çarpma: 2 * 10 = 20
3. Bölme: 20 / 2 = 10
4. Çıkarma (Soldan): 20 - 10 = 10
5. Toplama: 10 + 3 = 13
6. Son Toplama: 13 + 2 = 15
*/
```

4.11 Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu 2.4 başlığında anlatılmıştı. Kodlarken onluk tabanda değerler verdiğimiz kayan noktalı sayılar, aslında arka planda ikilik tabanda tutulur. Bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```
float total = 0.0F;
for(float a = 0.0F; a == 2.0F; a += 0.01F)
    total += a;
Console.WriteLine($"Toplam:{total}"); //Program İcrası Asla Buraya Ulaşmaz!
```

Acemi programcı bunun 0, 0.01, 0.02, 0.03, gibi artarak, 1.97, 1.98, 1.99 aralığındaki her bir sayıyı toplayacağını ve 199.0 sonucunu, yani matematiksel olarak doğru cevabı vereceğini varsayar. Bu kodda doğru yürümeyen iki şey olur:

1. Birincisi yazıldığı haliyle program asla sonuçlanmaz. `a` asla 2'ye eşit olmaz ve döngü asla sonlanmaz.
2. İkincisi ise döngü mantığını `a < 2` şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam 199'dan farklı bir şey olur. IEEE-754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık 201'e ulaşır.

Bunun olmasının nedeni, kayan noktalı sayılarının **onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir**. Bu duruma aşağıdaki klasik örnek verilir;

```
double a = 0.1;
double b = 0.2;
double c = 0.3;
if(a + b == c)
    Console.WriteLine("Bu satır İcra Edilmez!"); //IEEE754 standardında işlem yaparsak.
else
    Console.WriteLine("Bu satır İcra EDİLİR!"); // Kayan Noktalı Sayılar İkili Sayı Sisteminde
    ↳ Tutulduğundan Yaklaşık Olarak Hesaplanır.
```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki 1/3 sayısı gibi 0.333333333333333... şeklinde kesin olmayan biçimde depolanması gerektiğindendir);

```
double a = 0.1;
double b = 0.2;
double c = 0.3;
double toplam = a + b;

// Karşılaştırma sonucu şaşırtıcı olacaktır!
bool esitMi = (toplam == c);
Console.WriteLine($"a (0.1) : {a:R}"); // 'R' formatı tam hassasiyeti gösterir
Console.WriteLine($"b (0.2) : {b:R}");
Console.WriteLine($"a + b : {toplam:R}");
Console.WriteLine($"c (0.3) : {c:R}");
Console.WriteLine($"\\n(0.1 + 0.2) == 0.3 sonucu: " + esitMi);
// Aradaki çok küçük farkı görelim:
double fark = toplam - c;
Console.WriteLine($"Aradaki fark: {fark:R}");

/*Program Çıktısı:
a (0.1) : 0.1
b (0.2) : 0.2
a + b : 0.30000000000000004
```



```
c (0.3) : 0.3
```

```
(0.1 + 0.2) == 0.3 sonucu: False
Aradaki fark: 5.551115123125783E-17
*/
```

4.12 Düşün ve Çöz

- ◇ **Düşün:** C#'ta değer tipler (**value types**) ile referans tipler (**reference types**) bellek modelinde nasıl farklılaşır? Yığın(**stack**) ve öbek (**heap**) kavramlarını bir araba ve arabanın kopyası metaforuyla açıklayınız.
- ◇ **Düşün:** **int**, **long**, **float** ve **double** arasındaki seçimi etkileyen faktörler nelerdir? Finansal hesaplamalarda **decimal** tipinin tercih edilmesinin teknik nedeni nedir? Ondalık sayı hesaplamalarında kayan nokta hatalarına somut bir örnek veriniz.
- ◇ **Düşün:** C#'ta **var** anahtar kelimesiyle tanımlanan değişkenin tipi derleme zamanında mı çalışma zamanında mı belirlenir? **dynamic** ile **var** arasındaki temel fark nedir? Hangi durumda hangisi tercih edilmelidir?
- ◇ **Düşün:** Boş olabilir değer tipleri (**Nullable<T>** veya **int?**) neden C#'a eklenmiştir? Bir veritabanı sütununun 'henüz girilmemiş' değerini temsil etmek için neden düz **int** yeterli değildir?
- ◇ **Düşün:** C#'taki işleç önceliği (**operator precedence**) kuralları bilinmediğinde nasıl beklenmedik sonuçlar çıkabilir? Parantez kullanmanın hem doğruluk hem de okunabilirlik açısından önemi nedir?
- ◇ **Düşün:** Numaralandırma tipi (**enum**) yerine **string** sabitleri kullanmanın yaratacağı sorunları tartışınız. Enum kullanımı kodun bakım kolaylığını nasıl artırır?
- ◇ **Düşün:** C#9+ ile gelen kayıt tipleri (**record**) ve değiştirilemez nesneler (**immutable objects**) kavramı neden modern yazılım mimarisinde önem kazanmaktadır? İş parçacığı-güvenli (**thread-safe**) olması açısından faydası nedir?
- ◇ **Çöz:** **int**, **double**, **bool**, **char** ve **string** tiplerinden birer değişken tanımlayınız. Her birinin bellekte kapladığı alanı belirtiniz ve her birine uygun bir gerçek hayat senaryosu yazınız.
- ◇ **Çöz:** Aşağıdaki ifadelerin C#'ta ürettiği değerleri hesaplayınız:
a) **17 % 5** b) **(int)(7.9)** c) **2 < 3** d) **0xFF & 0x0F** e) **true && (5 > 3)**
- ◇ **Çöz:** Bir dikdörtgenin alanını ve çevresini hesaplayan, enini ve boyunu kullanıcıdan alan C# programını yazınız. Girilen değerlerin sıfırdan büyük olup olmadığını kontrol ediniz.
- ◇ **Çöz:** Bir sıcaklık değerini Celsius'tan Fahrenheit'a ve Kelvin'e dönüştüren programı yazınız. Mutlak sıfır (-273.15°C) altında giriş yapılmasını engelleyiniz.
- ◇ **Çöz:** C#'ta **int.MaxValue** değerine 1 eklendiğinde ne olur? Taşma (**overflow**) durumunu **checked** anahtar kelimesiyle nasıl yakalarsınız? Deneyi yapıp sonucu açıklayınız.
- ◇ **Düşün:** Neden **if (0.1 + 0.2 == 0.3)** ifadesi **false** sonucunu verir? Bu durumun donanımsal sebebi nedir?
- ◇ **Düşün:** Değişken isimlendirirken neden Türkçe karakter kullanabiliyoruz ama anahtar kelimeler (**if**, **while** vb.) sadece İngilizce karakterlerden oluşmak zorundadır?
- ◇ **Düşün:** UTF-8 karakter seti desteği, kaynak kod yazarken bize ne gibi avantajlar sağlar?

5. Bölüm

Giriş Çıkış İşlemleri

5.1 Modüler Programlama

C# programlama dili, aynı zamanda modüler bir programlama dilidir. Oluşturulan ve önceden test edilip doğrulanmış modüller olan **sınıf kütüphaneleri** (**class library**) yazacağımız kodlara **using** yönergesi ile dahil edilir.

using Kütüphane-Adı; //Bu yönergeler de talimattır ve noktalı virgül ile biter.

Modüllere ayırmanın **soyutlama** (**abstraction**), **değişim yönetimi** (**change management**) ve **yeniden kullanım** (**reusing**) gibi üstünlükleri vardır. Modüllere ayrılan bir kodun bakımı da daha kolaydır. Ancak daha fazla fonksiyonun çağırılması, işlemciyi yorar. Çünkü her fonksiyon çağırısında işlemcinin mevcut durumu ve kaydediciler belleğe itilir ve fonksiyondan geri döndüğünde eski duruma dönmek için bu veriler tekrar geri çekilir. İşlemciye yüklenen bu **çağırma yükü** (**calling overhead**) günümüz bilgisayarlarında oldukça ihmal edilebilir seviyededir. Ancak modern derleyiciler **satır içi genişletme** (**JIT Inlining**) yöntemiyle **küçük fonksiyonları doğrudan ana koda gömerler ve böylece bu yükü sıfıra indirirler**.

Ancak mili saniyeler bazında yapılması gereken zaman kritik işler gerçekleştirmek için bu durum göz önüne alınmalıdır.

C# programlama dilinde en çok kullanacağımız modüllerden biri de giriş çıkış IO (**input output**) işlemlerinin tanımlı olduğu **System**¹ ve **System.IO**² sınıf kütüphaneleridir. Bu kütüphaneler, yeni proje şablonunda **örtük olarak koda dahil edilmiştir**. Böylece kodumuza ek bir **using** yönergesi yazmadan konsoldan bir şey okumak ve konsola bir şey yazmak için **Console** sınıfını doğrudan kullanırız. Bunların yanında **Math**, **Random**, **string** gibi birçok sınıf kullanır hale geliriz.

```
/* Bu kütüphaneler örtük olarak tanımlıdır!
using System;
using System.IO;
*/
Console.Write("Merhaba "); // Konsola "Merhaba" yazar ve imleci son yazılan karakter sonrasına
→ taşır.
```

5.2 Konsola Veri Yazma

Konsoldan veri okumak ve konsola veri yazmak için **Console** sınıfı kullanılır. Sınıfın **Write** yöntemi ile imlecin (**cursor**) bulunduğu yerden itibaren parametre olarak girilen veriler konsola yazılır ve imleç son yazılan karakterden sonrasına taşınır. Sınıfın **WriteLine** yöntemi kullanılarak veriler yazıldıktan sonra son yazılan karakterden sonra imleç bir alt satıra inerek satır başı yapar.

```
Console.Write("Merhaba ");
Console.WriteLine("İlhan");
int yaş = 50;
Console.WriteLine(yaş);
float kilo = 99.5f;
Console.WriteLine(kilo);
decimal para = 545.5M;
Console.WriteLine(para);
/*Program Çıktısı:
Merhaba İlhan
50
99,5
545,5
*/
```

Write ve **WriteLine** yönteminin ilk argümanı metin olmak zorunda değildir. Ancak bu yöntemlerin ilk argümanı, **biçimlendirme metni** (**format string**) olarak kullanılabilir;

¹<https://learn.microsoft.com/en-us/dotnet/api/system>

²<https://learn.microsoft.com/en-us/dotnet/api/system.io>

- ◇ Biçimlendirme metninin içinde süslü parantez kullanılarak izleyen argümanların indisi verilir.
- ◇ İzleyen her bir argümanların indisi sıfırdan başlar ve sırasıyla devam eder.
- ◇ Metin içindeki süslü parantezle belirtilen yere ilgili argümanın değeri yazılır.
- ◇ Son olarak oluşturulan metin konsola yazılır.

```

1 // See https://aka.ms/new-console-template for more information
2 int sayi1 = 20, sayi2 = 30;
3 Console.WriteLine("1.Sayılar:{0}, {1}",sayi1,sayi2);
4 int toplam = sayi1 + sayi2;
5 Console.WriteLine("2.Sayıların Toplamı:{0}", toplam);
6 Console.WriteLine("3.Toplamı {0} olan sayılar: {1}, {2}", toplam, sayi1, sayi2);
7 Console.WriteLine("4.Toplam: {0}, toplam({0}) şeklinde de yazılabilir", toplam);
8 /*Program Çıktısı:
9 1.Sayılar:20, 30
10 2.Sayıların Toplamı:50
11 3.Toplamı 50 olan sayılar: 20, 30
12 4.Toplam: 50, toplam(50) şeklinde de yazılabilir
13 */

```

Bu kodu şu şekilde açıklayabiliriz;

- ◇ 3. Satırdaki `WriteLine("1.Sayılar:{0}, {1}",sayi1,sayi2);` talimatı ile `"1.Sayılar:{0}, {1}"` metnindeki `{0}` görülen yere izleyen **ilk argümanın değeri** konur, ardından `{1}` görülen yere izleyen **ikinci argümanın değeri** konur. Sonuç olarak oluşan metin `"1.Sayılar:20, 30"` olacaktır. Bu durumda sonuç metin konsola yazdırılır ve imleç bu metin yazdırıldıktan sonra satır başı yaptırılır.
- ◇ 5. Satırdaki `WriteLine("2.Sayıların Toplamı:{0}", toplam);` talimatı ile `"2.Sayıların Toplamı:{0}"` metnindeki `{0}` görülen yere izleyen **ilk argümanın değeri** konur. Sonuç olarak oluşan metin, konsola yazdırılacak metindir ve `"2.Sayıların Toplamı: 50"` şeklinde olacaktır. Bu durumda sonuç metin konsola yazdırılır ve imleç bu metin yazdırıldıktan sonra satır başı yaptırılır.
- ◇ 6. satırda da benzer şekilde bir konsola yazdırma yapılacaktır.
- ◇ 7. satırda da benzer şekilde bir konsola yazdırma yapılacaktır. Burada, aynı indisteki argümanın biçimlendirme metni içerisinde istenildiği yere, istenildiği kadar ve istenildiği sırada konulduğu gözden kaçırılmamalıdır.

5.3 Metinler

Metinleri (**string**) kod içerisinde işlemek için aslında karakter dizileri kullanılır. Ancak, daha yaygın uygulama olarak bir metin değişkeni bildirmek için **string** anahtar sözcüğünü kullanılır. Bu anahtar kelime, **System.String** sınıfı için bir takma addır.

§5.3.1 Metinleri Tanımlama

Metin değişmezleri (**string literal**) kod içerisinde **çift tırnak** (") karakterleri arasında yazılır. Aslında bu metinler UTF-16 karakterlerinden oluşur. C# dilinde karakterler için **char** veri tipi kullanılır. Karakter değişmezleri (**character literal**) ise **tek tırnak** (') arasında yazılır;

```

1 string adı, soyadı;
2 adı = "İlhan";
3 soyadı = "ÖZKAN";
4 Console.WriteLine("Adı ve Soyadı : {0} {1}", adı, soyadı);
5 char karakter1 = 'h';
6 char karakter2 = '\u006A';// Tek tırnak içinde UTF-16 kodlamasına göre
7 char karakter3 = '\x0013';
8 char karakter4 = '$';
9 Console.WriteLine("{0} {1} {2} {3}", karakter1, karakter2, karakter3, karakter4);
10 char[] harfler = { 'A', 'B', 'C', 'Ç', 'D', 'E', 'F' }; //Karakter Dizisi
11 Console.WriteLine(harfler); //Her bir dizi elemanı ayrı ayrı yazılır!
12 string selamlama = new string(harfler); //Karakter disizi metne çevrildi (cast)
13 Console.WriteLine("Alfabe: {0}", selamlama);
14 /*Program Çıktısı:
15 Adı ve Soyadı : İlhan ÖZKAN
16 h j $
17 ABCÇDEF
18 Alfabe: ABCÇDEF
19 */

```

Metin değişmezleri içinde çift tırnak karakteri koymak için (\") yazılır. Benzer şekilde tek tırnak karakteri değişmezi için de tek tırnak arasında (') yazılır;

```
/*
Metin değişmezler çift tırnak arasında yazılır.
Metin değişmezi içinde çift tırnak kullanmak için \" olarak yazılır.
Bir sonraki satıra geçmek için \ karakteri kullanılır.
Metin değişmezi içinde \ karakteri kullanmak için \\ yazılır.
*/
string metin = "Merhaba \"İlhan\"";
Console.WriteLine(metin);
/*Program Çıktısı:
Merhaba "İlhan"
*/
```

§5.3.2 Tam Metin Değişmezleri

Bir önceki örnekte metin değişmezinde birçok kural vardır. Onları gözardı etmek için **tam metin işleci** (@) kullanılır. Metnin başına bu işleç konularak **tam metin** (*verbatim string*) değişmesi tanımlanabilir. Tam metnin içinde sadece çift tırnak karakteri için iki adet çift tırnak karakteri (") kullanılır;

```
string filename1 = @"c:\documents\files\u0066.txt"; // Tam metin (verbatim string)
string filename2 = "c:\\documents\\files\\u0066.txt"; // Sıratan metin (string)
Console.WriteLine(filename1);
Console.WriteLine(filename2);
string tammetin= @"İlhan
Özkan \ " ";
Console.WriteLine(tammetin);
/*Program Çıktısı:
c:\documents\files\u0066.txt
c:\documents\files\u0066.txt
İlhan
Özkan \ "
*/
```

§5.3.3 Temel Metin İşlemleri

Metinlerin karakter sayısı, aynı zamanda **uzunluğudur**. Metin uzunluğu için **Length** özelliğini kullanırız;

```
string str = "İlhan Özkan";
int length = str.Length;
Console.WriteLine(length); //11
```

Metinler, **String.Format()** yöntemini kullanarak biçimlendirme metni (*format string*) ile bir veya daha fazla öğeyi tek bir metinde birleştirebiliriz. Burada ilk argüman biçimlendirme metnidir. **Console.Write()** yöntemi gibi çalışır. Konsola yazdırılan metin gibi biçimlendirilmiş yeni bir metin oluşturur;

```
string yenimetin=string.Format("Yaşım: {0} ve Adım: {1}", 45, "İlhan");
Console.WriteLine(yenimetin); //Yaşım:45 ve Adım:İlhan
```

Bunların yanında metinler toplama işleci (+) ile birleştirilebilir. Bunun için **String.Concat()**, **String.Join()** ve **String.Format()** yöntemleri de kullanılabilir;

```
string s1 = "İlhan";
string s2 = "Özkan";
string s3 = s1 + " " +s2; //"İlhan Özkan"
string s4 = string.Concat(s1, " ", s2); //"İlhan Özkan"
string s5 = string.Join(" ", s1, s2); //"İlhan Özkan"
string s6 = string.Format("{0} {1}", s1, s2); //"İlhan Özkan"
string s7 = string.Concat(s1, " ", s2); //"İlhan Özkan"
string s8 = string.Join(" ", new string[] { s1, s2 }); //"İlhan Özkan"
string s9 = $"{s1} {s2}"; //"İlhan Özkan" //Enterpolasyon İleride anlatılacaktır!
```

Metinler içerisindeki UTF-8 karakterleri 4 karakterli onaltılık sayı olarak da verilebilir;

```
string s = "\u0130İlhan"; //"İlhan"
```

Metinlere ilk değer çift tırnak karakterleri içerisinde verilir. Peki metin çift tırnak karakteri içeriyorsa ters bölü işareti ile birlikte çift tırnak karakteri (\") kullanılır. **Kaçış dizileri** (*escape sequences*), metinsel ifadeler (*string*) içinde **doğrudan yazılamayan** veya **özel bir anlam taşıyan karakterleri** temsil etmek için

kullanılır. Bunlar Tablo 5.1’de verilmiştir. C# dilinin Unicode desteği, çok dilli uygulamalar geliştirmedeki gücünü gösterir.

Dizi	Karakter Adı	Unicode	Açıklama / Örnek
'	Tek Tırnak	0x0027	Karakter sabitlerinde kullanılır.
"	Çift Tırnak	0x0022	Metin (<i>string</i>) içinde tırnak yazdırmak için.
\\	Ters Bölü	0x005C	Dosya yollarında yaygın kullanılır.
\0	Boş (<i>null</i>) karakter	0x0000	Bellek sonlandırıcı karakter.
\a	Uyarı (<i>alert</i>)	0x0007	Sistem bip sesini tetikler.
\b	Geri (<i>backspace</i>)	0x0008	İmleci bir karakter geri kaydırır.
\e	Kaçış (<i>escape</i>)	0x001B	ESC karakteri.
\f	Yeni Sayfa (<i>form feed</i>)	0x000C	Sayfa başı (yazıcılar için).
\n	Yeni Satır (<i>new line</i>)	0x000A	İmleci bir alt satırın başına indirir.
\r	Satır Başı (<i>carriage return</i>)	0x000D	İmleci mevcut satırın başına döndürür.
\t	Yatay sekme (<i>horizontal tab</i>)	0x0009	Yatayda bir sekme, 8 karakter ve katlarına kadar boşluk bırakır.
\v	Dikey Sekme (<i>vertical tab</i>)	0x000B	Dikey sekme, aynı sütunda alt satıra imleci taşır.
\u	Unicode (UTF-16)	\uHHHH	4 haneli onaltılık kod: \u00E7 (ç)
\U	Unicode (UTF-32)	\U00HHHHHH	8 haneli onaltılık kod: \U0000015E (§)
\x	Onaltılık Karakter	\xH[H][H][H]	Değişken uzunluklu onaltılık kod. \x015E = \x15E (§)

Tablo 5.1: Kaçış Dizileri ve Unicode Karşılıkları

Ayrıca metni belirlenen bir aralıkta sağa ya da sola yaslama (*padding*) da yapılabilir;

```
string s = "\u0130İlhan \'61\'";
Console.WriteLine("\u0130İlhan \'61\'"); //İlhan '61'
string paddedLeft = s.PadLeft(10, ' '); //Sola boşluk ekle
Console.WriteLine(paddedLeft); //      İlhan
string paddedRight = s.PadRight(10, '+'); //Sağa + karakteri ekle
Console.WriteLine(paddedRight); //İlhan+++++
```

Bit tam sayıyı da metin olarak herhangi bir tabana *Convert()* yöntemi çevirebiliriz;

```
Int32 Number = 252;
Console.WriteLine(Convert.ToString(Number, 2)); //11111000
Console.WriteLine(Convert.ToString(Number, 8)); //370
Console.WriteLine(Convert.ToString(Number, 10)); //252
Console.WriteLine(Convert.ToString(Number, 16)); //FC
```

Değer tipler de *ToString()* yöntemi ile metne çevrilebilir. Burada metne dönüştürülecek veri tipine özel biçimlendirme metni içinde çeşitli karakterler (0,d,M,y,H,m,s) olduğuna dikkat ediniz;

```
int intValue = 31;
Console.WriteLine(intValue.ToString("000")); //031
Console.WriteLine(intValue.ToString("Değer 0 dır.")). //Değer 31 dır.
double doubleValue = 10.456;
Console.WriteLine(doubleValue.ToString("0.00")); //10,46
Console.WriteLine(doubleValue.ToString("00")); //10
Console.WriteLine(doubleValue.ToString("Değer 0.0 dır.")). //Değer 10.5 dır.
DateTime currentDate = DateTime.Now; // 14/05/2025 10:38:03
Console.WriteLine(currentDate.ToString("dd-MM-yyyy HH:mm:ss")); // "14-05-2025 10:38:03"
Console.WriteLine(currentDate.ToString("dd-MM-yyyy")); // "14-05-2025"
Console.WriteLine(currentDate.ToString("dd-MMMM-yyyy HH:mm:ss")); // "14-Mayıs-2025 10:38:03"
```

§5.3.4 Metin Enterpolasyonu

Metin enterpolasyonu (*string interpolation*), bir metin ifadesinin başına dolar (\$) işleci konulduğunda, o metin artık **enterpolasyonlu metin** olur.

Enterpolasyon işleci (\$) sayesinde, metin içinde **süslü parantezler kullanılarak**;

1. matematik işlemleri veya mantıksal işlemler kodlanabilir.

2. doğrudan kapsamdaki (scope) değişkenler, isimleri ile kodlanabilir.

```
//METİN BİRLEŞTİRME:
string ad = "Mustafa";
int yas = 25;
//1. Klasik Yöntem (+ operatörü ile)
string metin1 = "Merhaba " + ad + ", yaşıınız " + yas + ".";
//2. String.Format Yöntemi
string metin2 = string.Format("Merhaba {0}, yaşıınız {1}.", ad, yas);
//3. MODERN YÖNTEM (String Interpolation)
string metin3 = $"Merhaba {ad}, yaşıınız {yas}.";
Console.WriteLine(metin3); //Merhaba Mustafa, yaşıınız 25.

//MATEMATİKSEL İŞLEMLER:
int sayi1 = 10, sayi2 = 20;
Console.WriteLine($"Toplam: {sayi1 + sayi2}"); //Toplam: 30
Console.WriteLine($"Sonuç: {(sayi1 > sayi2 ? "Büyük" : "Küçük")}"); //Toplam: 30

double fiyat = 1250.50;
Console.WriteLine($"Fiyat: {fiyat:C2}"); // Para birimi (Currency) -> $1.250,50
DateTime bugun = DateTime.Now;
Console.WriteLine($"Tarih: {bugun:dd/MM/yyyy}"); // 11/04/2026

Eğer metniniz hem kaçış dizilerini (ters bölü gibi) içermesin hem de enterpolasyon özelliğini kullansın istiyorsanız, tam metin işleci ($) ile birlikte metin enterpolasyonu işlecini (@) birleştirerek ($@) kullanabilirsiniz;

// See https://aka.ms/new-console-template for more information
string klasor = "Kullanıcılar";
string yol = $"C:\\{klasor}\\Belgelerim";
Console.WriteLine(yol); //C:\\Kullanıcılar\\Belgelerim
```

C#6.0 ile gelen metin enterpolasyonu, arka planda **string.Format** fonksiyonuna dönüştürülür.

§5.3.5 String Sınıfı

Programlama dünyasının en temel taşlarından biri olan **String** sınıfı, metinsel veriler üzerinde işlem yapmamızı sağlayan devasa bir araç kutusu gibidir. İster Java, ister C# kullanıyor olun; bu yöntemler mantık olarak birbirine çok benzer.

Bilgi Alma Yöntemleri

Bu yöntemler, metnin içeriğini değiştirmeden size metin hakkında bilgi verir.

- ◊ **length()**: Metnin toplam karakter sayısını (uzunluğunu) döndürür.
- ◊ **charAt(index)**: Belirtilen indisteki karakteri verir. (Unutmayın, indisler her zaman 0'dan başlar!)
- ◊ **indexOf(keyword)**: Aranılan bir karakterin veya alt metnin başladığı ilk indisi verir. Bulamazsa -1 döner.

Metin Düzenleme ve Değiştirme

String'ler genellikle değiştirilemez (immutable) olduğu için, bu yöntemler mevcut metni değiştirmek yerine size yeni bir metin döndürür.

- ◊ **toLowerCase()** / **toUpperCase()**: Tüm harfleri küçük veya büyük harfe çevirir.
- ◊ **trim()**: Metnin başındaki ve sonundaki gereksiz boşlukları temizler.
- ◊ **replace(eski, yeni)**: Metin içindeki belirli bir parçayı başka bir parçayla değiştirir.
- ◊ **substring(başlangıç, bitiş)**: Metnin belirli bir aralığını kesip almanızı sağlar.

Kontrol ve Karşılaştırma

Metnin içeriğinde ne olduğunu sorgulamak için kullanılır.

- ◊ **contains(sequence)**: Metnin içinde belirli bir kelime veya harf dizisi geçip geçmediğini kontrol eder (true/false).
- ◊ **startsWith(prefix)** / **endsWith(suffix)**: Metnin belirli bir ifadeyle başlayıp başlamadığını kontrol eder.

- ◇ `equals(otherString)`: İki metnin birbirine tıpatıp aynı olup olmadığını kontrol eder. (Büyük/küçük harf duyarlıdır).

Metni Parçalara Ayırma

Özellikle veri işleme süreçlerinde hayat kurtarır. `split(delimiter)`: Metni belirli bir ayırıcıya (örneğin virgül veya boşluk) göre bölerek bir diziye (`array`) dönüştürür. Eğer performansın kritik olduğu ve sürekli metin birleştirdiğiniz bir döngüdeyseniz, standart `String` yerine `StringBuilder` sınıfını kullanmak uygulamınızı ciddi anlamda hızlandıracaktır.

5.4 Konsoldan Veri Okuma

Konsoldan veri okuma, kullanıcı ile program arasındaki etkileşimin ilk adımıdır. C# dilinde bu işlem için `System` isim uzayının (`namespace`) altındaki `Console` sınıfı kullanılır.

§5.4.1 Temel Okuma Yöntemler

Konsoldan veri okumak için üç ana yöntemimiz vardır;

1. `string? ReadLine()`; yöntemi, kullanıcı **tamam** (**enter**) tuşuna basana kadar yazdığı tüm satırı okur.
2. `int Read()`; yöntemi, kullanıcının girdiği ilk karakterin ASCII/Unicode sayısal değerini döndürür.
3. `ConsoleKeyInfo ReadKey()`; yöntemi, kullanıcının bastığı tek bir tuşu (Ctrl, Alt, Harf vb.) yakalar. Genellikle programı bekletmek için kullanılır.

§5.4.2 Veri Okuma ve Tip Dönüşümü

`Console.ReadLine()` her zaman bir `string` döndürür. Eğer kullanıcıdan sayısal bir veri alıp üzerinde matematiksel işlem yapacaksanız, bu veriyi uygun türe dönüştürmeniz (**casting**) şarttır. Bunun için veri tipine özgü `Parse()` yöntemi kullanılır.

```
Console.Write("Lütfen yaşıınızı giriniz: ");
string girdi = Console.ReadLine();
// string'i int'e dönüştürme (Parse yöntemi)
int yas = int.Parse(girdi);
Console.WriteLine($"Gelecek yıl {yas + 1} yaşında olacaksınız.");
```

§5.4.3 Veri Okuma ve Tip Dönüşümünde Profesyonel Yaklaşım

Eğer kullanıcı sayı yerine harf girerse `int.Parse()`, programı çökertecektir, ancak `TryParse` güvenli bir çıkış sunar.

```
Console.Write("Bir sayı girin: ");
if (int.TryParse(Console.ReadLine(), out int sonuc)) {
    Console.WriteLine($"Girdiğiniz sayının karesi: {sonuc * sonuc}");
} else {
    Console.WriteLine("Hatalı giriş! Lütfen sadece sayı giriniz.");
}
```

§5.4.4 Klavyeden Basılan Tuşu Okuma

Örnek olarak klavyeden girilen karakteri ekranda göstermeden (yıldız basarak) veri okumak isterseniz `ReadKey()` kullanmalısınız.

```
Console.Write("$ifre: ");
ConsoleKeyInfo tus = Console.ReadKey(true); // true argümanı karakteri ekrana yazma demektir.
char karakter = tus.KeyChar; //Klavyeden basılan tuşu öğrendik.
```

§5.4.5 Konsoldan Veri Okurken Neye Dikkat Etmeli?

- ◇ Kullanıcı hiçbir şey yazmadan Enter tuşuna basarsa `Console.ReadLine()` ne döndürür? `null` mu yoksa boş bir metin ("") mı? Bu durum `int.Parse()` yöntemini nasıl etkiler?
- ◇ C#10+ ile gelen boş olabilir referans tip (**nullable reference types**) tanımıyla bu yöntem artık `string?` yani boş olabilir metin olarak işaretlenmiştir. İleride anlatılacaktır.

5.5 Konsola Yazılan Değerleri Biçimlendirme

§5.5.1 Sayı Biçimlendirme

C#6+ sonrasında programlarında ham veriyi olduğu gibi yazdırmak yerine, son kullanıcının anlayabileceği bir forma sokmak (para birimi, yüzde veya belirli bir hassasiyet derecesi gibi) hayati önem taşır. Bu işlem için en modern yöntem metin interpolasyonu (\$) içinde kullanılan **biçim belirleyicilerdir** (**format specifier**).

Tam sayılarda genellikle başına sıfır ekleme veya para birimi gibi gösterimler tercih edilir;

1. **D[n]**: Belirtilen genişliğe kadar başına 0 ekler. `int i=42;` için `{i:D5}` konsola `00042` yazar.
2. **C[n]**: Yerel para birimi sembolünü ekler. `int i=42;` için `{i:C2}` konsola `$42,00` yazar.
3. **R**: `C#7.0+` ile birlikte sayının metne dönüştürülüp tekrar sayıya çevrildiğinde değer kaybı yaşanmamasını sağlayan "tam hassasiyet" formatıdır.

```
double a = 0.1;
double b = 0.2;
double toplam = a + b;
Console.WriteLine($"Varsayılan: {toplam}"); // Çıktı: 0,3
Console.WriteLine($"Round-trip: {toplam:R}"); // Çıktı: 0.30000000000000004
```

4. **X**: Sayıyı onaltılık tabana çevirir. `int i=254;` için `{i:X}` konsola `FE` yazar.

Kayan noktalı sayılarda en büyük ihtiyaç, virgülden sonraki basamak sayısını kontrol etmektir. Bunu üç şekilde yapabiliriz;

1. **F[n]**: Sabit basamak sayısı. `float f=12.765F;` için `{f:F2}` konsola `12.77` yazar.
2. **N[n]**: Binlik ayrıçlı gösterim. `float f=12.765F;` için `{f:N1}` konsola `12.8` yazar.
3. **P[n]**: Yüzde gösterimi. Sayıyı 100 ile çarpar ve % ekler. `float f=0.25F;` için `{f:P0}` konsola `%25` yazar.
4. **0.##**: Özel Biçimlendirme. Standart formatlar yetersiz kalırsa, kendi şablonunuzu oluşturabilirsiniz. Özel biçimlendirmede;

- ◊ **0** (Sıfır): Basamakta rakam yoksa mutlaka 0 koyar. `float f=12.7F;` için `{f:0.00}` konsola `12.70` yazar.
- ◊ **#** (Kare): 0 basamakta rakam varsa koyar, yoksa boş bırakır. `float f=12.7;` için `{f:0.##}` konsola `12.7` yazar.

```
double maas = 25500.758;
int kod = 123;
Console.WriteLine($"Maaş Bilgisi (Para Birimi): {maas:C2}"); //$25,500.76
Console.WriteLine($"Maaş Bilgisi (Sade): {maas:F2}"); //25500.76
Console.WriteLine($"Ürün Kodu (5 Haneli): {kod:D5}"); //00123
float f = 12.7654765476547F;
// String Interpolation kullanarak iki hane (precision) sınırlaması:
// 'F2' -> Fixed-point format, virgülden sonra 2 hane.
Console.WriteLine($"Sayının iki haneli hali: {f:F2}"); //12.77
// Alternatif olarak '#' kullanımı (varsa gösterir, yoksa boş bırakır):
Console.WriteLine($"Alternatif biçimlendirme: {f:0.##}"); //12.77
```

Bahsedilen biçimlendirmeler veri tiplerinin `ToString()` yönteminde de kullanılır;

```
float oran = 0.752f;
int sayi = 255;
string sonuc1 = oran.ToString("P1"); // %75,2 (Yüzde formatı)
string sonuc2 = sayi.ToString("D5"); // 00255 (5 haneye tamamla)
string sonuc3 = sayi.ToString("X"); // FF (Onaltılık taban)
```

Metin **enterpolasyonu olmadan** verileri belirli bir sütun genişliğine yayabilir ve sağa/sola yaslayabilirsiniz. Bu, özellikle konsolda tablo oluştururken hayat kurtarır.

```
// Pozitif sayı (10): 10 karakter genişliğinde SAĞA yasla
// Negatif sayı (-10): 10 karakter genişliğinde SOLA yasla
Console.WriteLine("{0,-15} | {1,10}", "Ürün Adı", "Fiyat");
Console.WriteLine("{0,-15} | {1,10:C2}", "Elma", 15.50);
Console.WriteLine("{0,-15} | {1,10:C2}", "Bilgisayar", 25000.00);
/*Program Çıktısı:
Ürün Adı      |      Fiyat
Elma          |     $15.50
Bilgisayar    |    $25,000.00
*/
```

Ancak burada izleyen argüman için **yanlış indis numarası verme riski vardır!** Metin enterpolasyonu her zaman daha iyiyse, neden hala `string.Format()` veya indisli yapıyı bilmeliyiz? Bazı durumlarda biçimlendirme taslağı (**template**) dışarıdaki bir dosyadan veya veritabanından gelebilir. Örneğin: "Merhaba 0, bugün hava 1." gibi bir taslağınız varsa, çalışma anında (**run-time**) bu boşlukları doldurmak için `string.Format()` kullanmak zorundasınızdır. Çünkü enterpolasyon derleme anında (**compile-time**) çözülür.

Para birimi biçimlendiricisi olarak **C[n]** biçim belirleyicisi (**format specifier**) kullanılır. Bir sayıyı para birimi miktarını temsil eden bir metne (**string**) dönüştürür;

```
using System.Globalization;
double para = 2510.752525;
//Sistem tarafından ön tanımlı olarak bozukluk iki hane yazılır.
//Programın çalıştığı bilgisayar türkçe ayarlarına sahip:
Console.WriteLine(string.Format("{0:c}", para)); // $2.510,75
Console.WriteLine($"{{para:C}}"); // $2.510,75
//Bozukluk hane sayısı aşağıdaki şekild belirtilir:
Console.WriteLine(string.Format("{0:c1}", para)); // $2.510,8
Console.WriteLine(string.Format("{0:c2}", para)); // $2.510,75
Console.WriteLine(string.Format("{0:c3}", para)); // $2.510,753
Console.WriteLine(string.Format("{0:c4}", para)); // $2.510,7525
Console.WriteLine();

NumberFormatInfo nfi = new CultureInfo("en-US", false).NumberFormat;
nfi.CurrencySymbol = "€";
nfi.NumberGroupSeparator = ",";
nfi.NumberDecimalSeparator = ".";
nfi.CurrencyPositivePattern = 0;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); //€2,510.75
nfi.CurrencyPositivePattern = 1;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); //2,510.75€
nfi.CurrencyPositivePattern = 2;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); //€ 2,510.75
nfi.CurrencyPositivePattern = 3;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); //2,510.75 €
```

Ayrıca bölgesel ayarları `NumberFormatInfo` kullanılarak hem tam sayı, hem de gerçek sayıları özel ayraçlarla biçimlendirmek için kullanılabilir.

```
using System.Globalization;
double number = 1234567.89;
var invariantResult = string.Format(CultureInfo.InvariantCulture, "{0:#,###,##}", number);
Console.WriteLine(invariantResult); // 1,234,568
var customProvider = new NumberFormatInfo {
    NumberDecimalSeparator = "x",
    NumberGroupSeparator = "?",
};
var customResult = string.Format(customProvider, "{0:#,###.##}", number);
Console.WriteLine(customResult); // 1?234?567x89
```

§5.5.2 Zaman Biçimlendirme

Para birimi biçimlendiricisi olarak `d`, `M`, `y`, `H`, `m` ve `s` **biçim belirleyicileri** (**format specifier**) kullanılır. Aşağıda hem enterpolasyonlu, hem de enterpolasyon olmadan kullanım örneği verilmiştir;

```
using System.Globalization;
//Yıl-Ay-Gün-Saat-Dakika-Saniye Şeklinde Zamanı oluşturuyoruz.
DateTime date = new DateTime(1971, 09, 01, 11, 53, 24);

Console.WriteLine(String.Format("{0:dd}", date)); //Yalnızca Günü Yazar:01
//Bunun Kısa Hali:
Console.WriteLine($"{{date:dd}}");
Console.WriteLine("{0:dd}", date);

Console.WriteLine(String.Format("{0:dd/MM/yyyy}", date)); //Günü Ayı ve Yılı Yazar: 01.09.1971
//Bunun Kısa Hali:
Console.WriteLine($"{{date:dd/MM/yyyy}}");
Console.WriteLine("{0:dd/MM/yyyy}", date);

Console.WriteLine(String.Format("{0:dd/MM/yyyy HH:mm:ss}", date)); //Günü Ayı Yılı Saat Dakika ve
→ Saniyeyi Yazar: 01.09.1971 11:53:24
//Bunun Kısa Hali:
Console.WriteLine($"{{date:dd/MM/yyyy HH:mm:ss}}");
Console.WriteLine("{0:dd/MM/yyyy HH:mm:ss}", date);
```

```
//Günü İsim Olarak Almanca Yazmak için:
//Gün İsmi Almanca Yazar: Mittwoch
Console.WriteLine(String.Format(new System.Globalization.CultureInfo("de-DE"), "{0:dddd}",
    → date));
```

Aşağıda zaman biçimlendirmek için enterpolasyon işlemine bir örnek verilmiştir;

```
DateTime date = new DateTime(1971, 09, 01, 11, 53, 24);
Console.WriteLine($"{date:d}"); //Günü Yazar: 1.09.1971
Console.WriteLine($"{date:dd}"); //Günü Yazar: 01
Console.WriteLine($"{date:ddd}"); //Kısa Günü Yazar: Çar
Console.WriteLine($"{date:dddd}"); //Uzun Günü Yazar: Çarşamba
Console.WriteLine($"{date:D}"); //Uzun Tarih Yazar: 1 Eylül 1971 Çarşamba
Console.WriteLine($"{date:f}"); //Uzun Tarih ve Kısa Saati Yazar: 1 Eylül 1971 Çarşamba 11:53
Console.WriteLine($"{date:F}"); /* Uzun Tarih ve Uzun Saati Yazar:
1 Eylül 1971 Çarşamba 11:53:24 */
Console.WriteLine($"{date:g}"); //Kısa Tarih ve Kısa Saati Yazar: 1.09.1971 11:53
Console.WriteLine($"{date:hh}"); //Saatin 12 Saatlik Formatı: 11
Console.WriteLine($"{date:mm}"); //Dakika Yazar: 53
Console.WriteLine($"{date:MM}"); //Ayı Sayısal Olarak Yazar: 09
Console.WriteLine($"{date:MMM}"); //Ayı Kısa Olarak Yazar: Eyl
Console.WriteLine($"{date:MMMM}"); //Ayı Uzun Olarak Yazar: Eylül
Console.WriteLine($"{date:ss}"); //Saniye Yazar: 24
Console.WriteLine($"{date:r}"); //RFC1123 Formatında Yazar: Wed, 01 Sep 1971 11:53:24 GMT
Console.WriteLine($"{date:t}"); //Kısa Saati Yazar: 11:53
Console.WriteLine($"{date:T}"); //Uzun Saati Yazar: 11:53:24
Console.WriteLine($"{date:tt}"); //AM/PM Formatında Yazar: ÖÖ
Console.WriteLine($"{date:y}"); //Ay ve Yılı Yazar: Eylül 1971
Console.WriteLine($"{date:yy}"); //Yılı Kısa Olarak Yazar: 71
Console.WriteLine($"{date:yyyy}"); //Yılı Uzun Olarak Yazar: 1971
```

5.6 Düşün ve Çöz

- ♦ **Düşün:** `Console.Write()` ile `Console.WriteLine()` arasındaki farkı yalnızca teknik açıdan değil, kullanıcı deneyimi açısından da değerlendiriniz. Bir CLI aracı tasarlarken çıktı biçimlendirmesinin önemi nedir?
- ♦ **Düşün:** C#'ta metin enterpolasyonu (`$"degisken"`), `string.Format()` ve `+` ile birleştirmenin performans farkları nelerdir? Büyük döngülerde hangisi tercih edilmeli ve neden?
- ♦ **Düşün:** Kullanıcıdan konsol üzerinden sayısal veri okurken `int.Parse()`, `int.TryParse()` ve `Convert.ToInt32()` arasındaki fark nedir? Hangisi daha 'profesyonel' bir yaklaşımdır ve neden?
- ♦ **Düşün:** Kaçış dizileri (`escape sequences`) neden gereklidir? Dosya yollarını `string` olarak yazarken `'\'` yerine `'@'` (`verbatim string`) kullanmanın avantajı nedir?
- ♦ **Düşün:** Para birimi, tarih ve ondalık sayıların biçimlendirilmesinde kültür (`CultureInfo`) ayarlarının önemi nedir? Türkiye'de çalışan bir uygulamanın para gösterimi Almanya'ya taşındığında ne gibi sorunlar çıkabilir?
- ♦ **Düşün:** `Console.ReadLine()` her zaman bir `string` döner. Kullanıcının yanlış tip girmesi durumunu ele almak için hangi stratejiyi benimsersiniz? Sonsuz bir döngüde doğru giriş beklemek ne zaman mantıklıdır?
- ♦ **Düşün:** Metin enterpolasyonunda format belirleyicileri (örneğin `deger:C2` veya `deger:0.00`) kullanmak, `string.Format()` kullanımına kıyasla okunabilirliği nasıl etkiler?
- ♦ **Çöz:** Kullanıcıdan ad, soyad, doğum yılı ve aylık maaşını alan; yaşını hesaplayan ve maaşı hem TL hem de USD (sabit kur kullanarak) olarak gösteren bir program yazınız. Çıktıyı düzgün biçimlendiriniz.
- ♦ **Çöz:** Kullanıcıdan ondalıklı bir sayı girişi isteyen ve hatalı (sayısal olmayan) girişlerde hata mesajı gösterip tekrar soran bir program yazınız. `double.TryParse()` kullanınız.
- ♦ **Çöz:** Tarih ve saat bilgisini `DD/MM/YYYY HH:mm:ss` biçiminde konsola yazan, ardından bu tarihin haftanın hangi gününe denk geldiğini Türkçe olarak gösteren bir program yazınız.
- ♦ **Çöz:** Bir not ortalaması hesap makinesi yazınız: kullanıcı kaç ders girileceğini belirlesin, her dersin adını ve notunu sırasıyla yazsın, program en yüksek, en düşük notu ve ortalamayı düzgün biçimlendirilmiş tablo olarak gösterebilir.
- ♦ **Çöz:** C#'ta `string` sınıfının `Length`, `ToUpper()`, `ToLower()`, `Contains()`, `Replace()` ve `Substring()` yöntemlerini kullanan; kullanıcının girdiği cümleyi çeşitli biçimlerde işleyen bir program yazınız. Soru: "Modüler programlamada 'çağırma yükü' (`calling overhead`) nedir? Modern derleyiciler bu yükü nasıl optimize eder?"

- ◇ **Düşün** : "Bir tarih verisini `Console.WriteLine($"tarih:MMMM")` şeklinde yazdığınızda, ay isminin Türkçe mi yoksa İngilizce mi çıkacağını belirleyen faktör nedir?"

6. Bölüm

Kontrol İşlemleri

6.1 Ardışık İşlem ve Kontrol İşlemleri

Nesne yönelimli programlamada OOP (**object-oriented programming**) yöntemler, yapısal programlamadaki fonksiyonlar gibi kodlanır. Yani ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır. Şu ana kadar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş çıkış işlemleri talimatlar içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte programın icrasını değiştirmeyen bu tür talimatlara **ardışık işlem (sequential operation)** adı verilir. Ardışık işlemler aşağıdaki üç tür talimattan (**statement**) oluşur.

1. Değişkenlerin kimliklendirildiği değişken tanımlamaları (**identifier definition**):
 - (a) `int yariCap=3;`
 - (b) `const float PI=3.14F;`
 - (c) `float daireninAlani,daireninCevresi;`
2. İfadelerden (**expression**): İfade, sabit, değişken ve işlec (**operator**) içeren söz dizimleridir. C#'ta her ifade, mutlaka bir değer döndürür!
 - (a) `daireninAlani=PI*yariCap*yariCap;`
 - (b) `float daireninCevresi=2.0*PI*yariCap;`
3. Klavyeden veri okuma ve konsola veri yazma gibi giriş-çıkış işlemleri (**input-output operation**):
 - (a) `Console.WriteLine("Karekökü Alınacak Sayıyı Giriniz");`
 - (b) `string okunan = Console.ReadLine();`

Kontrol işlemleri (control operation) ise programın icra sırasını değiştiren kontrol yapıları olup bu talimatlar (**statement**) olup üç türü vardır;

1. Duruma göre seçimler (**conditional choice**): `if`, `if-else` ve `switch` talimatları.
2. İlişkisel döngü (**relational loop**): `while`, `do-while` ve `for` talimatları.
3. Dallanmalar (**jump**): `continue`, `break`, `goto` ve `return` talimatları.

Kontrol işlemleri programda yazılan talimatların icra sırasını değiştirir. Bu nedenle icra akışındaki bu değişikliğe **kontrol akışı (control flow)** adı verilir. Kısaca kodun **çalışma sırasıdır**. Kontrol akışı, yazdığınız talimatların hangi koşullarda, hangi döngülerle veya hangi istisnai durumlarda icra edileceğini belirler.

Kontrol işlemlerinde; **kodlanan mantıksal satırlar (logical sequence)** ile fiziksel olarak icra edilen satırlar (**physical sequence**) birbirinden farklıdır.

6.2 Akış Diyagramları

Yazılan programdaki talimatların icra sırasına, **kontrol akışı** demiştik. Kontrol işlemi içeren programların **kontrol akışını anlamak için** akış diyagramları çizilir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.

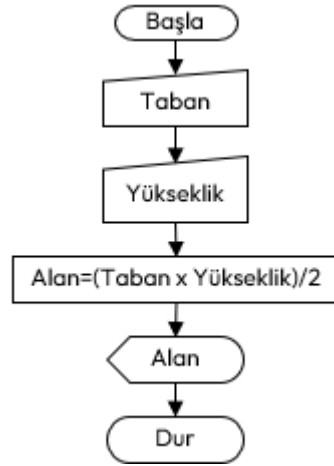


Şekil 6.1: Akış Diyagramları Genel Şekilleri

Akış diyagramları (flowchart), adımların grafik gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde

kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir.

Aşağıda taban ve yüksekliği klavyeden girilen bir dik üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.



Şekil 6.2: Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- ♦ Akış diyagramlarında tek bir başlangıç simgesi olmalıdır
- ♦ Bitiş simgesi birden çok olabilir.
- ♦ Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- ♦ Bağlaç simgesi sayfanın dolmasından ötürü parçalanmış akış şemasının öğelerini birleştirmede kullanılır.
- ♦ Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- ♦ Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

6.3 Duruma Göre Seçimler

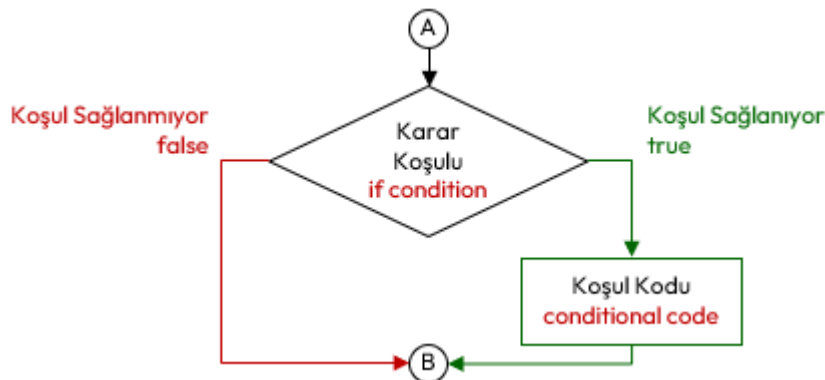
Duruma göre seçimler (**conditional choice**) programın akışını bir koşula göre değiştiren talimatlardır.

§6.3.1 If Talimatı

If talimatı, karar vermeye ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatının sözde kodu aşağıda verilmiştir;

```
if (karar-kosulu)
    koşul-kodu;
```

Burada ilişkisel işlemler içeren **karar koşulu** (**condition**) ifadesinin (**expression**) sonucu **true** ise **koşul kodu** (**conditional code**) icra edilir, değilse icra edilmez. Bu talimatın akış diyagramı da aşağıdaki gibidir;



Şekil 6.3: If Talimatı İcra Akışı

Aşağıda, yeni nesil konsol uygulamasındaki **Program.cs** içeriğini inceleyelim;

```
1 int yas = 18;
```

```

2 char cinsiyet = 'K';
3 if (yas < 30)
4     Console.WriteLine("Genç");
5 if (cinsiyet == 'E')
6     Console.WriteLine("Erkek");

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. ve 2. Satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
3. Satırda bir `if` talimatı bulunmaktadır. **Karar koşulu** olan `yas<30` ifadesinin sonucu `true` olduğundan
4. Satırdaki **koşul kodu** olan `Console.WriteLine("Genç");` talimatı ile konsola "Genç" yazılır.
- Sırada olan 5. Satırdaki kod icra edilir. Bu `if` talimatının karar koşulu olan `cinsiyet=='E'` ifadesinin sonucu `false` olduğundan 6. Satırdaki **koşul kodu** talimatı **icra edilmez**.

If talimatında **koşul kodu**, her zaman yukarıdaki gibi **ardışık işlem** (*sequential operation*) olmaz. Bazen `if` gibi bir başka **kontrol işlemi** (*control operation*) de olabilir. Aşağıda **kademeli if** (*compound if/cascade if*) kullanımına ilişkin kod örneğinde ikinci `if`, birinci `if` talimatının **koşul kodudur**;

```

if (yas<30)
    if (yas<7) // Bu if, üsteki if talimatının koşul kodudur.
        Console.WriteLine("Çocuk");

```

Koşul kodu birden fazla talimattan oluşacak ise kod bloğu içine alınır. Aşağıda verilen kod örneğinde ilk `if` talimatında `yas<30` ile test edilen koşul doğrulandığında kod bloğunun içindeki tüm talimatlar icra edilir.

```

if (yas<30) { //koşul-kodu bloğu başlangıcı
    if (yas<7)
        Console.WriteLine("Çocuk");
    if (yas<18)
        Console.WriteLine("Genç");
    if (yas>60)
        Console.WriteLine("Yaşlı");
} //koşul-kodu bloğu bitişi

```

Bazı durumlarda bir `if`, başka bir `if` bloğu içinde yani **iç içe** (*nested if*) olabilir. Buna ilişkin kod örneği de aşağıda verilmiştir.

```

if (cinsiyet=='E') { // dış if bloğu başlangıcı
    if (yas<7) { // iç if bloğu başlangıcı
        Console.WriteLine("Erkek Çocuk");
        Console.WriteLine("Yada Erkek Bebek");
    } // iç if bloğu başlangıcı
    if (yas<18)
        Console.WriteLine("Genç Delikanlı");
    if (yas>60)
        Console.WriteLine("Yaşlı Adam");
} // dış if bloğu bitişi

```

If talimatında **koşul kodu**, her zaman tek bir test ifadesinden oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```

if ((cinsiyet=='E') && (yas<18)) // hem Erkek, hem de genç
    Console.WriteLine("Genç Delikanlı");
if ((cinsiyet=='K') && (yas>60)) // hem yaşlı, hem de kadın
    Console.WriteLine("Yaşlı Kadın");

```

Ayrıca bazen programcı tarafından aşağıdaki gibi `if` talimatları yazabilir.

```

if (true)
    Console.WriteLine("Bu metin konsola her zaman yazılır.");
if (false)
    Console.WriteLine("Bu metin konsola hiçbir zaman yazılmaz!");

```

Örnek olarak klavyeden girilen bir sayının işaretini bulan bir programa ilişkin akış diyagramı Şekil 6.4'de verilmiştir. Şimdi programı yazalım;

```

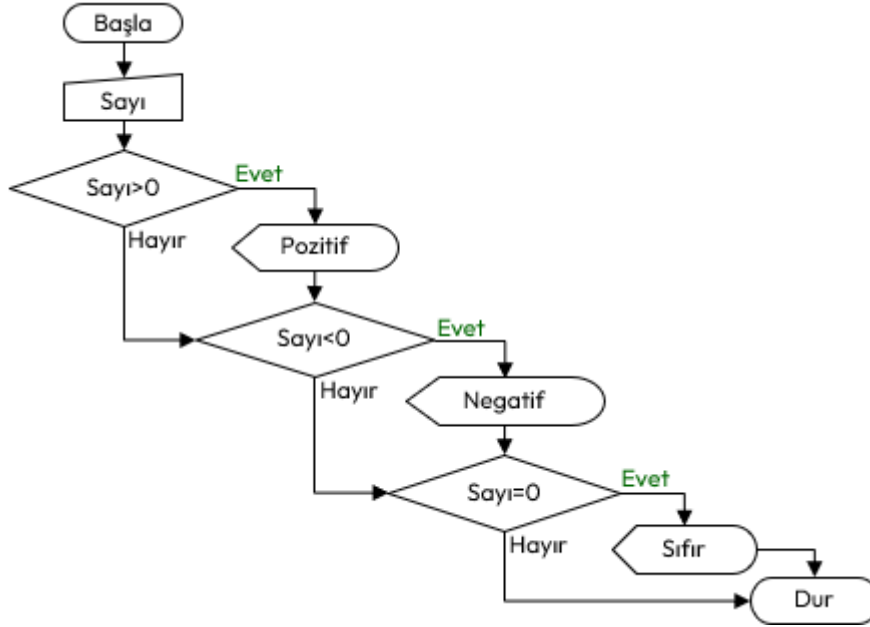
// See https://aka.ms/new-console-template for more information
int sayi;
Console.WriteLine("Sayı Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());

```

```

if (sayi > 0) // sayının pozitif olup olmadığı test ediliyor.
    Console.WriteLine("Pozitif"); // Burada sayının pozitif olduğu biliniyor.
if (sayi < 0) // sayının negatif olup olmadığı test ediliyor.
    Console.WriteLine("Negatif"); // Burada sayının negatif olduğu biliniyor.
if (sayi == 0) // sayının sıfır olup olmadığı test ediliyor.
    Console.WriteLine("Sıfır"); // Burada sayının sıfır olduğu biliniyor.

```



Şekil 6.4: Bir Sayının İşaretini Bulan Akış Diyagramı

§6.3.2 If-Else Talimatı

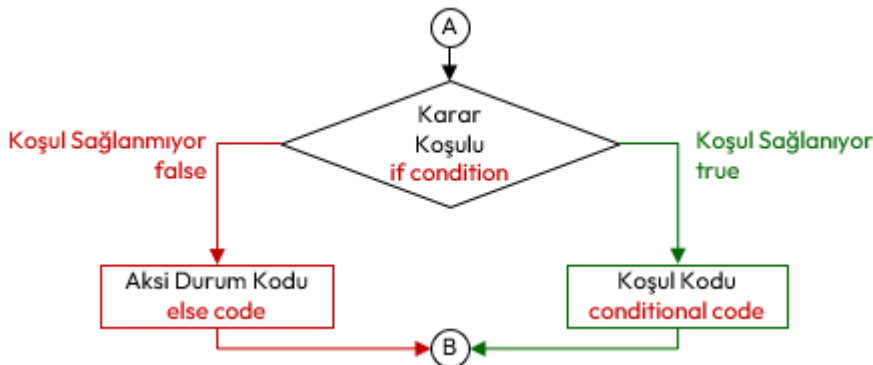
If-Else Talimatı, bir başka karar verme talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. Sözde kodu aşağıda verilmiştir;

```

if (karar-kşulu)
    kşul-kodu;
else
    aksi-durum-kodu;

```

If-Else talimatında, **karar kşulu** (condition) ifadesi **true** ise **kşul kodu** (conditional code) icra edilir, karar kşulu **false** ise **aksi durum kodu** (else code) icra edilir.



Şekil 6.5: If-Else Talimatı İcra Akışı

Aşağıda verilen örnek programı inceleyelim;

```

1 int yas = 65;
2 char cinsiyet = 'K';
3 if (yas < 50)
4     Console.WriteLine("Genç");
5 else
6     Console.WriteLine("Yaşlı");

```

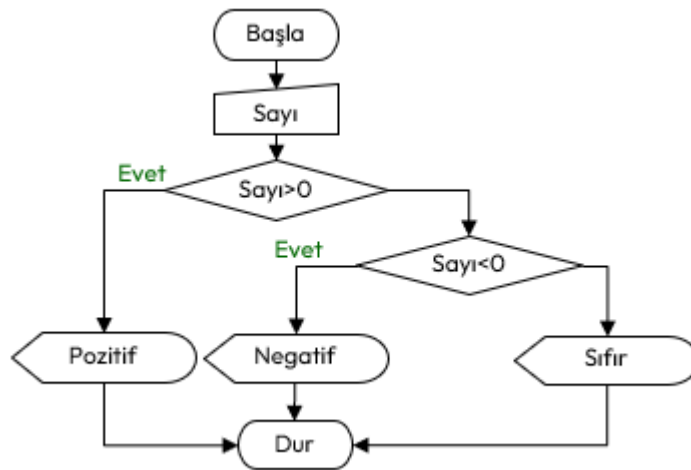
Örnek kodumuzu şu şekilde açıklayabiliriz;

1. ve 2. Satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
3. Satırda bir `if` talimatı bulunmaktadır. **Koşul kodu** olan `yas<50` ifadesinin sonucu `false` olduğundan
4. Satırdaki **koşul kodu** talimatı **icra edilmez**. Bunun yerine 5. satırdan başlayan `else` kodu yani 6. Satırdaki **aksi durum kodu** icra edilir. Yani `Console.WriteLine("Yaşlı");` talimatı ile konsola "Yaşlı" yazılır.

If talimatında olduğu gibi bu talimatta da **koşul kodu** ya da **aksi durum kodu** birden fazla talimattan oluşacak ise kod bloğu içine alınır. Buna ilişkin kod örneği aşağıda verilmiştir.

```
if (cinsiyet=='E') {
    if (yas<7)
        Console.WriteLine("Erkek Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Delikanlı");
    if (yas>60)
        Console.WriteLine("Yaşlı Adam");
} else {
    if (yas<3)
        Console.WriteLine("Kız Bebek");
    if (yas<7)
        Console.WriteLine("Kız Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Kız");
    if (yas>60)
        Console.WriteLine("Yaşlı Kadın");
}
```

Bir sayının işaretini bulan programının icra akışı olan Şekil 6.4 incelendiğinde ilk `if` talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmadığı anlaşılabacaktır. Benzer şekilde ilk iki test geçilirse üçüncü if talimatındaki teste gerek yoktur. Bu durumu Şekil 6.6'de verilen akış diyagramında daha net görebiliriz. Bu durumda aynı program If-Else talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;



Şekil 6.6: Bir Sayının İşaretini Bulan If-Else Akış Diyagramı

```
1 // See https://aka.ms/new-console-template for more information
2 int sayi;
3 Console.WriteLine("Sayı Giriniz:");
4 sayi=Convert.ToInt32(Console.ReadLine());
5 if (sayi > 0)
6 { // sayının pozitif olup olmadığı test ediliyor.
7     Console.WriteLine("Pozitif"); // Burada sayının pozitif olduğu biliniyor.
8 }
9 else
10 { // Burada sayının pozitif olmadığı biliniyor.
11     if (sayi < 0)
12     { // sayının negatif olup olmadığı test ediliyor.
13         Console.WriteLine("Negatif"); // Burada sayının negatif olduğu biliniyor.
```

```

14     }
15     else
16     { // Burada sayının pozitif ve negatif olmadığı test biliniyor.
17         Console.WriteLine("Sıfır"); // Burada sayının sıfır olduğu biliniyor.
18     }
19 }

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- ◊ Eğer `sayı` pozitif girilirse;
 - 5. Satırdaki `if` karar koşulu olan `sayı>0` ifadesinin sonucu `true` çıkacak ve sadece 7. Satırdaki koşul kodu olan `Console.WriteLine("Pozitif");` talimatı icra edilecektir. 9. Satırdan başlayan ve 19. satırda biten aksi durum kodu icra edilmeyecektir.
- ◊ Eğer `sayı` negatif girilirse;
 - 5. Satırdaki `if` karar koşulu olan `sayı>0` ifadesinin sonucu `false` çıkacak ve 9. Satırdan başlayan ve 19. Satırda biten aksi durum kodu icra edilecektir. Bu blok içinde
 - 11. Satırdaki `if` karar koşulu olan `sayı<0` ifadesinin `true` çıkacak ve 13. Satırdaki koşul kodu olan `Console.WriteLine("Negatif");` talimatı icra edilecektir.
- ◊ Eğer `sayı` sıfır girilirse;
 - 5. Satırdaki `if` karar koşulu olan `sayı>0` ifadesinin sonucu `false` çıkacak ve 9. Satırdan başlayan ve 19. Satırda biten aksi durum kodu icra edilecektir. Bu blok içinde;
 - 11. Satırdaki `if` karar koşulu olan `sayı<0` ifadesinin sonucu `false` çıkacak ve 13. Satırdaki koşul kodu talimatı icra edilemeyecektir. Onun yerine on 15. Satırdan başlayan aksi durum kodu icra edilecektir. Yani 17. Satırdaki `Console.WriteLine("Sıfır");` icra edilecektir.

Ayrıca kod incelendiğinde her bir `if` ve `else` sonrasında tek talimat bulunmaktadır. Dolayısıyla blok kullanmaya gerek de yoktur. Bu durumda kodun nihai hali aşağıda verilmiştir.

```

int sayi;
Console.WriteLine("Sayi Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());
if (sayi>0)
    Console.WriteLine("Pozitif");
else if (sayi<0)
    Console.WriteLine("Negatif");
else
    Console.WriteLine("Sıfır");

```

§6.3.3 Sarkan Else

Aşağıdaki program örneği incelendiğinde `else`, hangi `if` talimatına aittir? Böyle bir kod programcı tarafından yazılabilir ve derleyici buna hiçbir sıkıntı çıkarmaz. Bu problem **sarkan else** (**dangling else**) problemi olarak bilinir.

```
if (cinsiyet=='E') if (yas<18) Console.WriteLine("Delikanlı"); else Console.WriteLine("Adam");
```

Bu durumda kodu aşağıdaki gibi okunaklı hale getirdiğimizde ilk `if` talimatının **koşul kodunun** ikinci `if-else` talimatı olduğu görülmektedir.

```

if (cinsiyet=='E')
    if (yas<18)
        Console.WriteLine("Delikanlı");
    else
        Console.WriteLine("Adam");

```

Kod bloğu kullanılmadan yazılan bu tip kodlarda `else` her zaman kendinden bir önceki `if` talimatına aittir.

Sarkan else durumundaki mantıksal hataların önüne geçmek için
acemi kullanıcılar her zaman blok ve girinti kullanmalıdır.

```

if (cinsiyet=='E') {
    if (yas<18) {
        Console.WriteLine("Delikanlı");
    } else {
        Console.WriteLine("Adam");
    }
}

```

```

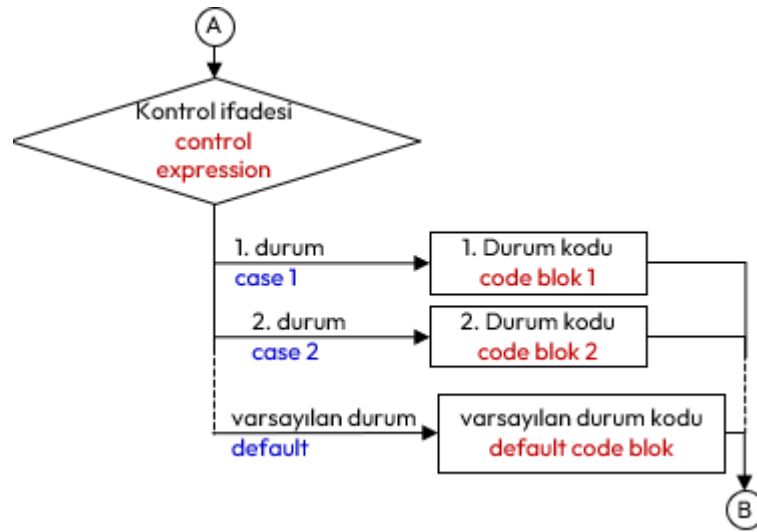
}
```

§6.3.4 Switch Talimatı ve İfadesi

Switch talimatı, bir kontrol ifadesi (**control expression**) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Sözde kodu aşağıda verilmiştir;

```

switch (kontrolifadesi) { //switch bloğu başlangıcı
    case alternatif-1:
        alternatif-1-kodu;
        break; //break talimatı zorunlu değildir
    case alternatif-2:
        alternatif-2-kodu;
        break; //break talimatı zorunlu değildir.
        // ...
    default: // Bu etiket zorunlu değildir!
        varsayılan-alternatif-kodu;
} //switch bloğu bitişi
```



Şekil 6.7: Switch Talimatı İcra Akışı

Switch talimatına ilişkin kurallar;

1. **Kontrol ifadesi**, bir kez değerlendirilir. Kontrol ifadesi temel sayı tipleri olabileceği gibi **string** de olabilir.
2. **Kod bloğu olmayan bir switch talimatı yazılamaz!**
3. **Her bir alternatif, kontrol ifadesinin sonuçlarından biri olup izleyen ve iki nokta ile biten bir etiket (label) olarak tanımlanır.**
4. **Alternatif, hiçbir zaman değişken veya ifade (expression) olamaz!**
5. **switch bloğu icra edilirken**, karşılaştığı ilk **break;** talimatında, **switch** bloğu dışına çıkılır. Bu talimat zorunlu değildir.
6. Tüm alternatifleri içerecek şekilde etiketler yazılmayabilir. Blok sonunda yer alacak şekilde **üzerinde yer alan alternatifler dışında kalan tüm alternatifler için default:** etiketli alternatif yazılabilir. Bu etiket de zorunlu değildir.

Aşağıda klavyeden girilen bir sayının 4 rakamına bölünmesinde kalan üzerinden işlem yapan **switch** talimatı bir başka örnek program verilmiştir;

```

1 // See https://aka.ms/new-console-template for more information
2 int sayi;
3 Console.WriteLine("Bir Sayı Giriniz:");
4 sayi=Convert.ToInt32(Console.ReadLine());
5 switch (sayi % 4)
6 { //kontrol ifadesi bir işlem içerir
7 case 3:
8     Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 3 dir.");
9     break;
```



```

10 case 2:
11 case 1:
12     Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 2 veya 1 dir.");
13     break;
14 default: // Başka alternatif yoktur.
15     Console.WriteLine("Sayı 4 Rakamına TAM Bölünür");
16     break;
17 }

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- ◊ Eğer `menu, 1,5,9,...` girilirse;
 - 5. Satırdaki `switch` talimatında kontrol ifadesinin sonucu 1 olacaktır.
 - Bu durumda 11. Satırdaki `case 1:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 12. Satırdaki `Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 2 veya 1 dir.");` icra edilir ve bir sonraki talimata geçilir.
 - Bir sonraki talimat 13. Satırdaki `break;` talimatı bizi `switch` bloğunun dışına çıkarır.
- ◊ Eğer `menu, 2,6,10,...` girilirse;
 - 5. Satırdaki `switch` talimatında kontrol ifadesinin sonucu 2 olacaktır.
 - Bu durumda 10. Satırdaki `case 2:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - Bulunan ilk talimat 12. satırdaki talimattır.
 - İlk önce 12. Satırdaki `Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 2 veya 1 dir.");` icra edilir ve bir sonraki talimata geçilir.
 - Bir sonraki talimat 13. Satırdaki `break;` talimatı bizi `switch` bloğunun dışına çıkarır.
- ◊ Eğer `menu, 3,7,11,...` girilirse;
 - 5. Satırdaki `switch` talimatında kontrol ifadesinin sonucu 3 olacaktır.
 - Bu durumda 7. Satırdaki `case 3:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 8. Satırdaki `Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 3 dir.");` icra edilir ve bir sonraki talimata geçilir.
 - Bir sonraki talimat 9. Satırdaki `break;` talimatı bizi `switch` bloğunun dışına çıkarır.
- ◊ Eğer `menu, 4,8,12,...` girilirse;
 - 5. Satırdaki `switch` talimatında kontrol ifadesinin sonucu 0 olacaktır.
 - Bu durumda diğer etiketlerde olmayan bir durum olan 14. Satırdaki `default:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 15. Satırdaki `Console.WriteLine("Sayı 4 Rakamına TAM Bölünür");` icra edilir ve bir sonraki talimata geçilir.
 - Daha sonra 16. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.

Şu ana kadar anlatılan Switch talimatı, **sabit desen (constant pattern)** kullanan talimattır. En klasik `switch` kullanımıdır. Bir değişkenin değerinin, önceden belirlenmiş bir **değişmez (literal)** değere eşit olup olmadığını kontrol eder. Aşağıda `string` ile bu testleri yapan bir program örneği verilmiştir;

```

string veri = "C#";
switch (veri) {
case "Java":
    Console.WriteLine("Dil: Java");
    break;
case "C#": // Sabit desen: Değer tam olarak "C#" mı?
    Console.WriteLine("Dil: C#");
    break;
case null: // Sabit desen: Veri null mı?
    Console.WriteLine("Veri yok.");
    break;
default:
    Console.WriteLine("Bilinmeyen dil.");
    break;
}

```

Yukarıdaki kod bloğunda da aslında büyük bir sorun diyebileceğimiz bir şey yoktur fakat, `case` sayısı arttıkça, bu blok uzayacak ve aslında kodda kalabalık yapacak `case` ve `break` anahtar kelimeleri de artacaktır. Sadece string atama yaptığımız bir `switch` bloğunu aşağıdaki gibi daha sade bir hale **switch ifadeleri (switch**

expression) ile getirebiliriz.

C#9+ ile gelen **switch ifadelerindeki ilişkisel desen (relational pattern)** özelliği ile, **case** etiketleri ifade (**expression**) olarak işlem görmeye başlamıştır. **Case ifadeleri** içinde doğrudan büyüklük/küçüklük (<, >, <=, >=) işleçlerini kullanmanıza olanak tanır.

```
int sicaklik = 25;
string durum = sicaklik switch
{
    < 0      => "Donuyor",          // İlişkisel desen
    >= 0 and <= 15 => "Soğuk",      // Mantıksal ve ilişkisel desen bir arada
    > 15 and <= 25 => "Ilık",
    > 25      => "Sıcak",
    _        => "Geçersiz veri"    // Discard (Geri kalan her şey)
};
Console.WriteLine($"Hava durumu: {durum}");
```

Ayrıca switch ifadelerinde bir de **mantıksal desenler (logical latterns)** kullanılmaya başlamıştır. Yukarıda bahsedilen desenleri birleştirirken **and**, **or** ve **not** anahtar kelimelerini kullanabiliriz. Bu, okunabilirliği zirveye taşır.

```
object girdi = 45;
if (girdi is int sayi and > 0) // Hem tip kontrolü hem ilişkisel desen
{
    Console.WriteLine("Pozitif bir tam sayı girildi.");
}
// switch içinde kullanımı:
string sonuc = girdi switch
{
    not int => "Tam sayı değil",
    < 0 or > 100 => "Sınır dışında",
    >= 0 and <= 100 => "Geçerli puan",
    _ => "Bilinmiyor"
};
```

§6.3.5 Üçlü Koşul İşleci

Daha önce 4.10.3 başlığı altında işlenen işleçlere ek olarak, **ardışık işlemlerde (sequential operation)** if talimatlarına gerek kalmadan bir koşula göre **ifadeler (expression)** işlenecek ise **üçlü koşul işleci (conditional tenary operator)** kullanılır. Aşağıda üçlü işlecin iki söz dizimi verilmiştir;

koşul ? doğruifadesi : yanlışifadesi;
değişken = koşul ? doğruifadesi : yanlışifadesi;

Aşağıda oy kullanma durumuna ilişkin örnek işlenmiştir;

```
int yas;
Console.WriteLine("Yaşınız?: ");
yas=Convert.ToInt32(Console.ReadLine());
string metin=(yas >= 18) ? "Oy Kullanabilirsin." : "Oy Kullanamazsın!";
Console.WriteLine(metin);
```

Aşağıda başka kullanımlarına örnek verilmiştir;

```
int sayı;
bool tek, çift;
Console.WriteLine("Bir Sayı Giriniz: ");
sayı=Convert.ToInt32(Console.ReadLine());
tek = (sayı % 2==0) ? true : false;
çift =!tek;
int sonuç1 = tek ? sayı + 30 : sayı + 40;
int sonuç2 = çift ? sayı * 30 : sayı * 40;
Console.WriteLine($"Sonuç1: {sonuç1}, Sonuç2: {sonuç2}");
```

6.4 İlişkisel Döngüler

İşlemciler iyi bir sayıcılardır. CPU içindeki kaydediciler (**registers**) sayma işlevini ve birçok matematiksel işlemi yapmak için de kullanılır. **Birçok problemlerin çözümüne, belli adımların tekrarlanması ile gidilir.** Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız.

Konsola 10 defa isminizi yazdıran bir program örneğini ele alalım.

```
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
```

Program incelendiğinde aynı `WriteLine` talimatının tekrar tekrar yazıldığını görürüz. **Kod tekrarı istenmeyen bir durumdur!**

§6.4.1 Sayaç Kontrollü Döngüler

Belirlenen bir sayaç değişkeninin istenilen bir değere ulaşp ulaşmadığı kontrol ederek kodun tekrar tekrar icra edilmesi sağlanır. Yapısal programlama öncesinde tekrar edilecek kodun başına dönüş `goto` talimatıyla sağlanır. **Bu talimat icra sırasını etkileyen bir talimattır.** Kodlama sırasında tekrar edilecek kodun başına iki



Şekil 6.8: Sayaç Kontrollü Döngü İcra Akışı

nokta ile biten bir **etiket (label)** konulur. Etiketlere kimlik verilirken yine 4.2.5 başlığındaki değişken kimliklendirme (**identifier definition**) kuralları uygulanır. Tekrar edilecek kodun sonunda ise `goto` talimatı etiketle birlikte kullanılır.

```
1 // See https://aka.ms/new-console-template for more information
2 int sayaç=0;
3 Etiket:
4 Console.WriteLine("İLHAN");
5 sayaç++;
6 if(sayaç<10)
7     goto Etiket;
```

Örneği şu şekilde açıklayabiliriz;

- ◇ 2.Satırda tekrar edilecek koda girmeden sayacıma ile değer verdik.
- ◇ 3.Satıra geri dönülecek kodun başını işaretleyen **Etiket:** kimlikli bir etiket konulmuştur.
- ◇ 4.Satırda tekrarlanacak talimat yazılmıştır. Birden fazla talimat da yazabiliriz.
- ◇ 5.Satırda sayacıma güncelledik.
- ◇ 6.Satırda sayacın istenilen değere ulaşp ulaşmadığını kontrol ettik. Eğer ulaşmamış ise 7.Satırdaki `goto Etiket;` talimatıyla 3.Satıra döndük.

Sonuç olarak 3. ile 7. Satır arasında dört talimat içeren **mantıksal satır (logical sequence)** olmasına karşın 40

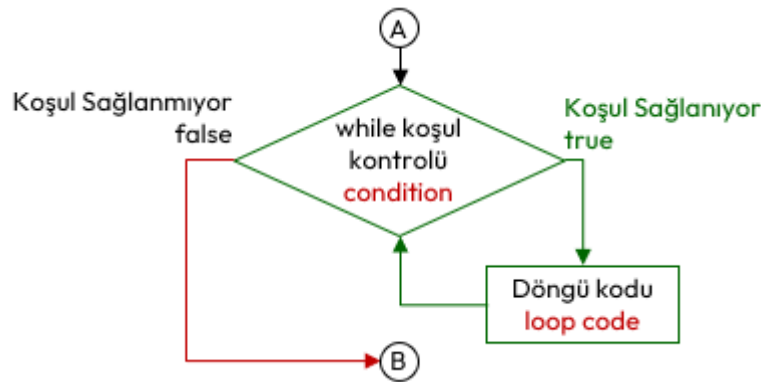
adet **fiziksel satır** (**physical sequence**) icra edilmiştir. C# programlama dili tarafından **goto** talimatı desteklenir. Ancak 1968 Yılında *Edsger W. Dijkstra* tarafından **goto** ifadelerinin zararlı olduğunu ilan ettiği unutulmamalıdır.

Yukarıda verilen döngü kodu örneğindeki gibi arapsaçı kodlamadaki **goto** talimatı yerine, **yapısal programlamada** (**structural programming**) ilişkisel döngü (**relational loop**) talimatları olan **while**, **do-while** ve **for** talimatları geliştirilmiştir.
goto talimatının kullanımı zorunlu olmadıkça önerilmez!

§6.4.2 While Talimatı

While döngüsünde **koşul** (**condition**), **true** olduğu sürece **döngü kodu** (**loop code**) icra edilir. Tekrarlanacak kod, tek bir talimat olabileceği gibi bir kod bloğu da olabilir. While döngüsünün sözde kodu aşağıda verilmiştir;

```
while (koşul)
    döngü-kodu;
```



Şekil 6.9: While Talimatı İcra Akışı

While talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir;

```
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
while (sayaç<10) // while koşul kontrolü
{ // döngü bloğu
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
    sayaç=sayaç+1; //Sayaç Güncelleme
}
```

Sayaç güncelleme ifadesi, while **koşul** ifadesine eklenerek program daha da kısaltılabilir. Her koşul kontrolü sonrası sayaç artırılacaktır;

```
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
while (sayaç++<10) // while koşul kontrolü
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
```

Yukarıdaki örnek incelendiğinde döngü bloğu bir adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 10 adet fiziksel satır (**physical sequence**) icra edilmiştir.

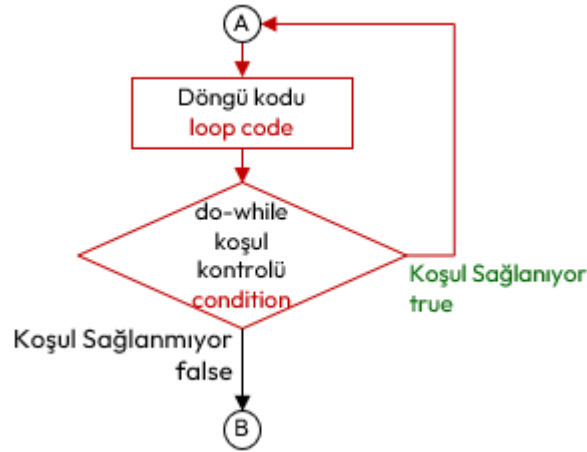
§6.4.3 Do-While Talimatı

Do-While döngüsünde, **do** ile **while** saklı kelimeleri arasındaki **döngü kodu** (**loop code**), **koşul** (**condition**) **true** olduğu sürece tekrarlanarak icra edilir. Tekrarlanacak kod tek bir talimat olabileceği gibi bir kod bloğu da olabilir. Do-While talimatının sözde kodu aşağıda verilmiştir;

```
do // "do" kelimesi goto talimatındaki etiket gibi davranır.
    döngü-kodu;
while (koşul);
```

Do-While talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir;

```
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
do
{ // döngü bloğu
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
    sayaç=sayaç+1; //Sayaç Güncelleme
}
```



Şekil 6.10: Do-While Talimatı İcra Akışı

```

} while (sayaç<10); // Do-While koşul kontrolü

```

Sayaç güncelleme ifadesi, **koşul** ifadesine eklenerek program aşağıdaki şekilde daha da kısaltılabilir.

```

int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
do
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
while (sayaç++<10); // Do-While koşul kontrolü

```

Yukarıdaki son örnek incelendiğinde ibir adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 10 adet fiziksel satır (**physical sequence**) icra edilmiştir.

While döngüsünde koşul kontrolü en başta yapılır!
Do-While döngüsünde ise döngü bloğu en az bir kez çalıştırıldıktan sonra koşul kontrolü yapılır!

§6.4.4 For Talimatı

For döngüsü özellikle tekrar edilen işlemlerin sayısı belli olduğunda kullanılan bir döngü yapısıdır. Sayaç kontrollü döngüler için en iyi seçimdir. Sözde kodu aşağıda verilmiştir.

```

for (ilk-değer-verme; koşul-kontrolü; sayaç-güncelleme)
    döngü-kodu;

```

For talimatı aşağıdaki şekilde icra edilir;

1. **İlk değer verme ifadesi bir kez icra edilir!**
2. **Koşul kontrolü yapılır!** Eğer test sonucu **true** ise döngü kodu icrasına geçilir. Aksi halde döngüden çıkarılır.
3. **Döngü kodu** icra edilir.
4. Döngü kodu icrası bitince **sayaç güncellemeleri yapılır.**
5. Tekrar ikinci adımdaki **koşul kontrolüne** dönülür.

For talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir.

```

for (int sayaç=0; sayaç<10; sayaç++)
    Console.WriteLine("İLHAN");

```

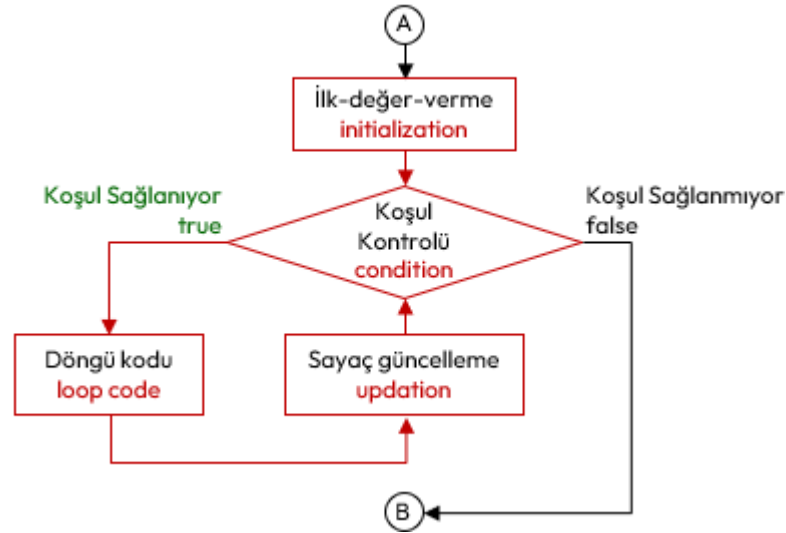
Buradaki döngü mantıksal satır (**logical sequence**) olarak iki talimattan oluşmaktadır. Ancak fiziksel satır (**physical sequence**) olarak 10 satır icra edilmiştir.

Aşağıda konsola yazdığımız her bir satırın başına satır numarası koyan bir program örneği gösterilmektedir.

```

1 // See https://aka.ms/new-console-template for more information
2 for (int sayaç=0; sayaç<10; sayaç++)
3     Console.WriteLine($"{sayaç}-İLHAN");
4 /*Program Çıktısı:
5 0-İLHAN
6 1-İLHAN
7 2-İLHAN

```



Şekil 6.11: For Talimatı İcra Akışı

```

8  3-İLHAN
9  4-İLHAN
10 5-İLHAN
11 6-İLHAN
12 7-İLHAN
13 8-İLHAN
14 9-İLHAN
15 */

```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu 2. Satırda belirtilen bir adet talimat içermektedir. Mantıksal olarak 1 satır içermesine rağmen 10 adet fiziksel satır icra edilmiştir.

For döngüsü, bir sayaç ile çalışılabileceği gibi **birden fazla sayaç ile de çalışılabilir**. Hem ilk değer verme hem de sayaç güncellemede birden fazla sayaca ilişkin işlem yapılabilir. Bunun için **virgül işleci (comma operator)** kullanılır.

```

for (int artanSayaç=0, azalanSayaç=10; artanSayaç<10; artanSayaç++, azalanSayaç--)
    Console.WriteLine($"{artanSayaç}-İLHAN-{azalanSayaç}");
/*Program Çıktısı:
0-İLHAN-10
1-İLHAN-9
2-İLHAN-8
3-İLHAN-7
4-İLHAN-6
5-İLHAN-5
6-İLHAN-4
7-İLHAN-3
8-İLHAN-2
9-İLHAN-1
*/

```

Aşağıda klavyeden girilen iki sayı aralığında tek ve çift sayıların toplamını bulan ve ekrana yazan programı verilmiştir.

```

int sayaç, say1,say2;
int tekToplam=0,çiftToplam=0;
Console.WriteLine("Birinci Sayıyı Giriniz: ");
say1=Convert.ToInt32(Console.ReadLine());
Console.WriteLine("İkinci Sayıyı Giriniz: ");
say2=Convert.ToInt32(Console.ReadLine());
if (say12<say1) { //say12, say1 den büyük olmalı.
    int temp=say1; //küçük ise yer değiştiriyoruz.
    say1=say2;
    say2=temp;
}

```



```

for (sayac=say1; sayac<=say12; sayac=sayac+1) {
    if (sayac%2==1) //sayac tek mi?
        tekToplam+=sayac;
    else
        çiftToplam+=sayac;
}
Console.WriteLine($"Tek Toplam:{tekToplam} Çift Toplam:{çiftToplam}");
/* Program Çıktısı:
Birinci Sayıyı Giriniz:
11
İkinci Sayıyı Giriniz:
1
Tek Toplam:36 Çift Toplam:30
*/

```

Bir ve kendisinden başka tam böleni olmayan sayıya **asal sayı** denir. Klavyeden girilen pozitif bir tam sayının asal olup olmadığını ekrana yazdıran program aşağıdaki şekilde kodlanabilir;

```

int sayı;
bool asalmı = true; // C#'ta mantıksal durumlar için bool kullanımı tercih edilir.
Console.Write("Bir Sayı Giriniz: ");
// Kullanıcı girişini okuyup tam sayıya dönüştürüyoruz.
if (int.TryParse(Console.ReadLine(), out sayı)) {
    if (sayı <= 1)
        asalmı = false;
    else {
        // Döngü: Sayıdan bir eksik değerden başlayıp 2'ye kadar iner.
        // Eğer asalmı false olursa (veya operatörü), döngü erken sonlanır.
        for (int sayac = sayı - 1; (sayac > 1) && (asalmı == true); sayac--) {
            if (sayı % sayac == 0)
                asalmı = false; // Sayı bölündüğü an asal değildir.
        }
    }
    // Sonucu ekrana yazdırıyoruz.
    if (asalmı)
        Console.WriteLine($"Girilen {sayı} sayısı asaldır.");
    else
        Console.WriteLine($"Girilen {sayı} sayısı asal değildir.");
} else
    Console.WriteLine("Geçersiz bir sayı girdiniz.");

```

Klavyeden girilen pozitif bir tam sayının **asal çarpanlarını** ekrana yazdıran program aşağıdaki şekilde kodlanabilir;

```

int sayı;
Console.Write("Bir Sayı Giriniz: ");
// Konsoldan okunan metni sayıya dönüştürüyoruz
if (int.TryParse(Console.ReadLine(), out sayı)) {
    // Döngü: Girilen sayıdan başlayıp 1'e kadar iner
    for (int sayac = sayı; sayac > 0; sayac--) {
        // Sayı sayaca tam bölünüyor mu?
        if (sayı % sayac == 0)
            Console.WriteLine($"Bölen: {sayac}");
    }
} else
    Console.WriteLine("Lütfen geçerli bir tam sayı giriniz.");
/* Program Çıktısı:
Bir Sayı Giriniz:12
Asal çarpan:12
Asal çarpan:6
Asal çarpan:4
Asal çarpan:3
Asal çarpan:2
Asal çarpan:1
*/

```

```
*/
```

§6.4.5 İç İç Döngü Kodu

Döngüler içinde yer alan döngü kodu bir başka döngüyü içerebilir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde **iç içe döngü** (**nested loop**) kullanırız. Aşağıda buna ilişkin bir örnek verilmiştir. İlk **for** döngüsünün tekrarlanan kodu ikinci bir **for** döngüsüdür.

```
int satır,sütun;
for (satır=0; satır<5; satır++)
{
    for (sütun=0; sütun<4; sütun++)
        Console.WriteLine("*");
}
```

```
/* Program Çıktısı:
```

```
****
****
****
****
****
*/
```

Bir başka örnek olarak elemanları, satır ve sütun çarpımları olan 4x5 boyutlarındaki matrisi ekrana yazdıran program aşağıda verilmiştir;

```
int satır, sütun;
// 1. Başlığı Yazdıran Kısım
Console.WriteLine(" Sütun: ");
for (sütun = 0; sütun < 3; sütun++)
// {0,3} ifadesi: 0. indeksteki veriyi 3 karakter genişliğinde sağa yasla demektir.
Console.WriteLine($"{(sütun + 1),3} ");
Console.WriteLine();
// 2. Matrisi Yazdıran Kısım
for (satır = 0; satır < 5; satır++) {
    // D2: Onluk sistemde 2 hane olacak şekilde yaz (01, 02 gibi).
    // PadLeft(2, '0') alternatifi de kullanılabilir.
    Console.WriteLine($"{(satır + 1):D2}.satır: ");
    for (sütun = 0; sütun < 3; sütun++)
        // Her sütun değerini 3 karakter genişliğinde ve başına 0 ekleyerek yaz.
        Console.WriteLine($"{sütun:D3} ");
    // Satır bitince imleci alt satıra geçir.
    Console.WriteLine();
}
```

```
/* Program Çıktısı:
```

```
Sütun: 1 2 3
01.satır: 000 001 002
02.satır: 000 001 002
03.satır: 000 001 002
04.satır: 000 001 002
05.satır: 000 001 002
*/
```

§6.4.6 Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (**sentinel value**) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden 0 girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
int sayı; /* okunan tamsayı */
int toplam = 0; /* girilen sayıların toplamı. başlangıçta 0*/
do
{
    Console.WriteLine("Bir sayı giriniz (Bitirmek için 0): ");
    sayı=Convert.ToInt32(Console.ReadLine());
```

```

    toplam += sayı; // sayı 0 girilse bile toplam etkilenmez
} while (sayı != 0); //sentinel value 0
Console.WriteLine($"Girilen Sayıların Toplamı: {toplam}");
/*Program Çıkışı:
Bir sayı giriniz (Bitirmek için 0):
10
Bir sayı giriniz (Bitirmek için 0):
20
Bir sayı giriniz (Bitirmek için 0):
30
Bir sayı giriniz (Bitirmek için 0):
5
Bir sayı giriniz (Bitirmek için 0):
0
Girilen Sayıların Toplamı: 65
*/

```

Aşağıda pozitif sayı girilmesini sağlayan bir başka kod örneği verilmiştir.

```

int sayı;
int pozitifSayı = 0;
do {
    Console.Write("Lütfen pozitif bir tam sayı giriniz: ");
    bool geçerliMi = int.TryParse(Console.ReadLine(), out sayı);
    // 2. İyileştirme: Hem sayısal geçerliliği hem de pozitiflik şartını kontrol ediyoruz.
    if (!geçerliMi || sayı <= 0)
        Console.WriteLine("HATA: Geçersiz giriş! Lütfen pozitif bir tam sayı giriniz.");
} while (sayı <= 0);
pozitifSayı = sayı;
Console.WriteLine($"Girilen pozitif tam sayı: {pozitifSayı}");

```

Aynı örnek bir başka şekilde aşağıdaki gibi kodlanabilir. Ancak burada 4. ve 5. satırlar ile 10. ve 11. satırların aynı olduğuna ve kod tekrarı yapıldığına dikkat ediniz!

```

1 int sayı = 0;
2 int pozitifSayı;
3 // Kullanıcıdan ilk girişi alıyoruz (Döngüye girebilmek için şartın kontrolü lazım)
4 Console.Write("Lütfen pozitif bir tam sayı giriniz: ");
5 bool basarili = int.TryParse(Console.ReadLine(), out sayı);
6 // Şart: Sayı geçersizse (harf vb.) VEYA sayı <= 0 ise döngüye gir
7 while (!basarili || sayı <= 0)
8 {
9     Console.WriteLine("HATA: Pozitif bir tam sayı girmediniz!");
10    Console.Write("Lütfen pozitif bir tam sayı giriniz: ");
11    basarili = int.TryParse(Console.ReadLine(), out sayı);
12 }
13 pozitifSayı = sayı;
14 Console.WriteLine($"Girilen pozitif tam sayı: {pozitifSayı}");

```

§6.4.7 BigInteger Veri Tipi

Fibonacci dizisi, her sayının kendisinden önceki iki sayının toplamı olduğu bir dizidir. Bu basit algoritma, en az 1000 ondalık basamağa ulaşana kadar Fibonacci sayılarını yineler, sonra da yazdırır. Bu değer, bir `ulong` veri tipinin tutabileceğinden bile önemli ölçüde daha büyüktür. Teorik olarak, `BigInteger` sınıfındaki tek sınır, uygulamanızın tüketebileceği RAM miktarıdır;

```

using System.Numerics;
BigInteger l1 = 1;
BigInteger l2 = 1;
BigInteger current = l1 + l2;
while (current.ToString().Length < 1000)
{
    l2 = l1;
    l1 = current;
    current = l1 + l2;
}

```

```
Console.WriteLine(current);
```

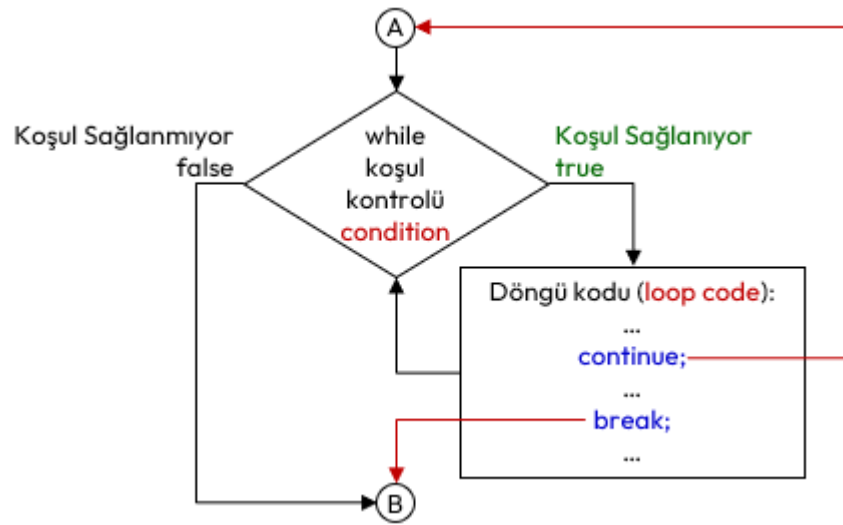
6.5 Dallanmalar

Dallanmalar (**jump**), bir döngünün yada fonksiyonun icrasını kesen ve bir başka talimata yönlendiren talimatlardır.

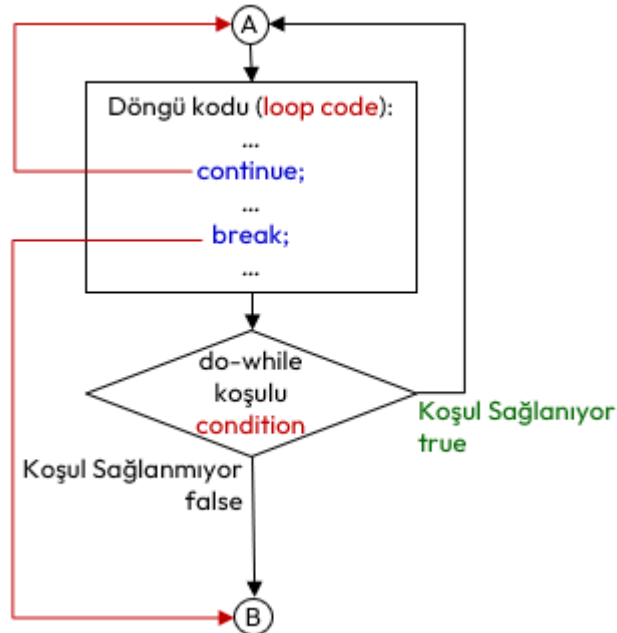
Ş6.5.1 Continue ve Break Talimatları

Continue talimatı, yalnızca geçerli yinelemeyi (**iteration**) sonlandırır ve sonraki yinelemelerle devam eder. Yani bu talimat, bir sonraki yinelemeyi yapmak için kullanılır. Dolayısıyla sadece **while**, **do-while** ve **for** döngü kodları (**loop code**) içinde kullanılabilir.

Break talimatı, döngüyü sonlandırır ve program icrası döngü sonrası ilk talimattan devam eder. Bunu daha önce **switch** talimatında görmüştük. Bu talimat, aynı şekilde **while**, **do-while** ve **for** döngüleri ile kullanılabilir. Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı Şekil 6.12’de verilmiştir.



(a) While



(b) Do-While

Şekil 6.12: While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı

Bu talimatlar döngü kodu içinde amacımıza uygun olarak birden çok kullanabiliriz. Aşağıda 0 ile 15 arasındaki sayıların beşe bölünenleri ve 10 dışındaki sayılar ekrana yazdıran bir program örneği verilmiştir.

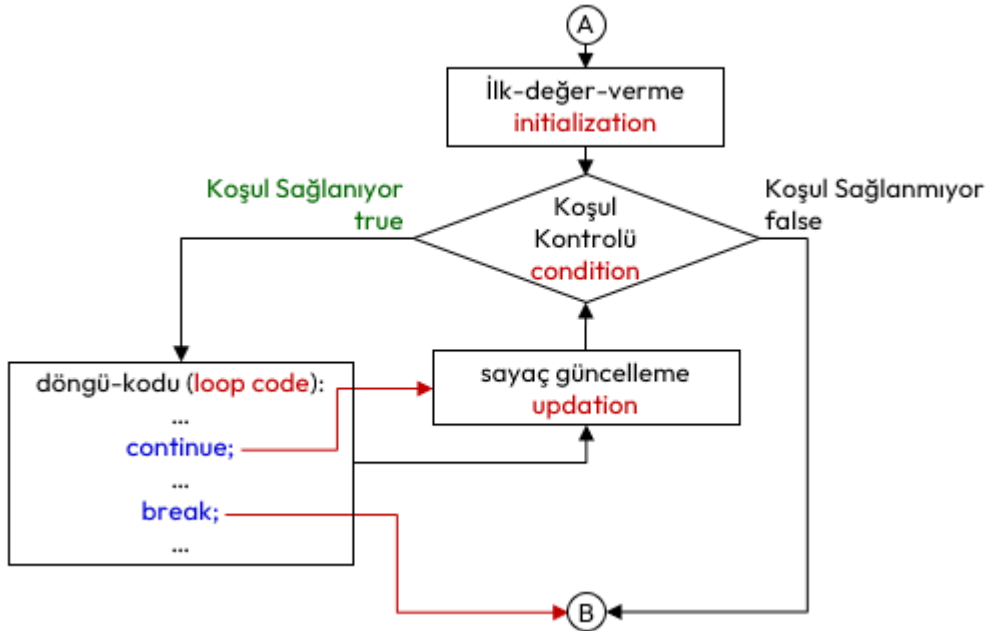
```
for (int sayaç = 0; sayaç <= 15; sayaç = sayaç + 1) {
    if (sayaç < 7) //sayaç'ın değeri 7 den küçük ise
```

```

    continue; //bir sonraki yinelemeye geç
if (sayac == 10) //sayac değeri 10 ise
    continue; //bir sonraki yinelemeye geç
if (sayac % 5 == 0) // sayac değeri 5 e bölünüyorsa
    continue; //bir sonraki yinelemeye geç
Console.WriteLine($"{sayac}");
}
/*Program Çıktısı:
7
8
9
11
12
13
14
*/

```

Yukarıdaki örnek incelendiğinde döngü bloğu üç adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 42 adet fiziksel satır (physical sequence) icra edilmiştir. Bazı `continue;` talimatları bazı satırların icrasını engellemiştir. Aşağıda pozitif sayı girilmesini zorlayan sonsuz döngü (infinite loop) içeren



Şekil 6.13: For Döngüsünde Break ve Continue Talimatları İcra Akışı

program verilmiştir. Programda döngü, pozitif sayı girildiğinde `break;` talimatı ile kırılmaktadır.

```

int sayı;
int pozitifSayı = 0;
while (true) // Sonsuz döngü başlatılıyor
{
    Console.Write("Lütfen pozitif bir tam sayı giriniz: ");
    // Güvenlik: Harf girilirse programın çökmemesi için TryParse kullanıyoruz.
    bool geçerliMi = int.TryParse(Console.ReadLine(), out sayı);
    if (geçerliMi && sayı > 0)
        break; // Şart sağlandığında döngüden zorla çıkılır.
    // Şart sağlanmazsa bu kısım çalışır ve döngü başa döner.
    Console.WriteLine("HATA: Pozitif bir tam sayı girmediniz!");
}
pozitifSayı = sayı;
Console.WriteLine($"Girilen pozitif tam sayı: {pozitifSayı}");

```

Bu talimatları dikkatli kullanmalıyız. Bazı durumlarda sonsuz döngüye girme durumu tüm döngü talimatlarında yaşanabilir. For döngüsünde bu durum farklıdır. `continue;` talimatı sonrasında bir sonraki yineleme için önce sayaç güncelleme ifadeleri icra edilir ve sonra koşul testi yapılır.

§6.5.2 Return Talimatı

Yöntemlerde çokça kullanacağımız return talimatı yapısal programlamadaki bir fonksiyonun gibidir. Yöntemler de fonksiyonlar gibi kodlanır. Üreteceği değerleri `return` talimatı ile geri döndürür.

Aşağıda üç defa negatif sayı girilmesi halinde programı sonlandıran bir klasik bir projedeki `Program.cs` örneği verilmiştir;

```
int sayı;
int pozitifSayı = 0;
int sayac = 0; // Kaç defa hatalı giriş yapıldığını tutan değişken.
do {
    Console.WriteLine("Lütfen pozitif bir sayı giriniz: ");
    // Kullanıcı girişi okunuyor ve sayıya çevriliyor.
    bool basarili = int.TryParse(Console.ReadLine(), out sayı);
    // Giriş başarısızsa veya sayı pozitif değilse hata bloğuna girer.
    if (!basarili || sayı <= 0) {
        Console.WriteLine("HATA: Pozitif Sayı GİRMEDİNİZ!");
        sayac++;
        if (sayac == 3) {
            Console.WriteLine("Üç defa hatalı veya negatif sayı girdiniz.");
            Console.WriteLine("Programdan Çıkılıyor...");

            // İşletim sistemine hata kodu (1) göndererek programı sonlandırır.
            return 1;
        }
    }
} while (sayı <= 0);
pozitifSayı = sayı;
Console.WriteLine($"Girilen pozitif tam sayı: {pozitifSayı}");
return 0; // Program başarıyla tamamlandı.

/*Program Çıktısı:
Lütfen pozitif sayı giriniz: -1
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -2
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -3
HATA: Pozitif Sayı GİRMEDİNİZ!
Üç defa negatif sayı girdiniz. Programdan Çıkılıyor.
*/
```

§6.5.3 Goto Talimatı

Tanımlı bir etikete program akışını yönlendiren `goto` talimatıdır. 6.4.1 başlığında örneği verilmiştir.

6.6 Düşün ve Çöz

- ◇ **Düşün:** Sarkan else (**dangling else**) problemi nedir? Bu belirsizliği önlemek için neden her zaman süslü parantez kullanılması önerilir? Gerçek bir projede bu hatayı bulmak neden zor olabilir?
- ◇ **Düşün:** While ve Do-while döngüleri arasındaki farkı, ATM’de PIN kodu doğrulama senaryosu üzerinden karşılaştırınız. Hangi senaryoda do-while daha doğal bir tercih olur?
- ◇ **Düşün:** Switch ifadesinde `break`; kullanılmaması durumundaki ‘fall-through’ davranışını hem bir hata hem de kasıtlı bir özellik olarak kullanmak mümkün müdür? C#8 ile gelen **switch ifadesi (expression)** bu sorunu nasıl ele alır?
- ◇ **Düşün:** `for` döngüsünde döngü değişkeninin döngü dışında değiştirilmesi neden tehlikelidir? Sonsuz döngü yazmanın kasıtlı ve kasıtsız yolları nelerdir? Sonsuz döngülerin gerçekte kullanıldığı senaryolara örnek veriniz.
- ◇ **Düşün:** `goto` talimatı neden ‘zararlı’ kabul edilir? Bu talimatın gerçekten faydalı olduğu (ya da kaçınılmaz görüldüğü) bir durum düşünebilir misiniz? C#’ta `goto` kullanımı hangi durumda mantıklı olabilir?
- ◇ **Düşün:** Döngü içinde `continue` ve `break` talimatlarının yanlış kullanımı ne tür mantık hatalarına yol açabilir? İç içe döngülerde `break`’in yalnızca içteki döngüden çıkması bazen istenmedik bir davranışa neden olur: bunu nasıl çözürsünüz?
- ◇ **Düşün:** Koşul ifadelerinde kısa devre değerlendirmesi (**short-circuit evaluation**) nedir? `&&` ve `&` opera-

törleri arasındaki farkı bir `null` kontrolü senaryosuyla açıklayınız.

- ◇ **Çöz:** Kullanıcıdan sürekli tamsayı girişi alan; 0 girildiğinde durarak, o ana kadar girilen sayıların en büyüğünü, en küçüğünü, toplamını ve ortalamasını ekrana yazan bir gözcü kontrollü döngü tasarlayınız.
- ◇ **Çöz:** 1'den 100'e kadar olan sayıları ekrana yazan, ancak 3'ün katları için sayı yerine 'Fizz', 5'in katları için 'Buzz', her ikisinin de katları için 'FizzBuzz' yazan programı yazınız (klasik FizzBuzz).
- ◇ **Çöz:** Kullanıcıdan bir tam sayı alarak o sayının asal olup olmadığını belirleyen programı yazınız. Ardından 2 ile 100 arasındaki tüm asal sayıları listeleyen (Eratosthenes Kalburu) versiyonunu geliştiriniz.
- ◇ **Çöz:** İç içe döngülerle 1'den N'e kadar olan sayıların çarpım tablosunu (N kullanıcıdan alınacak) düzgün sütun hizalamasıyla ekrana basan programı yazınız.
- ◇ **Çöz:** Bir ATM şifre ekranı simüle edin: kullanıcı doğru 4 haneli PIN'i (sabit olarak tanımlanmış) girinceye kadar en fazla 3 deneme hakkı verilsin; 3 denemede de yanlış girilirse kart bloke mesajı gösterilsin.
- ◇ **Düşün :** Bir `if` bloğu içinde `(x > 0 | y++ > 0)` yazmak ile `(x > 0 || y++ > 0)` yazmak arasında, `y` değişkeninin son değeri açısından nasıl bir fark oluşur?
- ◇ **Düşün :** Yüzlerce koşulun olduğu bir senaryoda `if-else` if yapısı yerine neden bir `Dictionary` veya `switch` yapısı kullanmayı tercih etmeliyiz?

7. Bölüm

Diziler

7.1 Dizi Nedir

Şu ana kadar en basit **veri yapıları** (**data structure**) olan değişkenlerle karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (**array size**) denir. Doğal olarak eleman sayısı tam sayıdır (**integer**). Elemanların sırasını tam sayı olan **indis** (**index**) belirtir. Kümelerin (**set**) aksine, aynı elemanlar dizide farklı konumlarda bulunabilir. Elemanlara terim adı da verilir. Örnekler;

- ◇ Örnek olarak elemanları 1'den 10'a kadar tam sayıların karesinden oluşan bir dizi, {1,4,9,...,100} olarak gösterilir.
- ◇ Bir başka örnek olarak {K,İ,T,A,P} dizisi verilebilir. Bu dizi ile {P,A,T,İ,K} dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları yani indisleri farklıdır.
- ◇ {1,3,5,1,2,1,7} dizisinde birden farklı sırada yani indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.

Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler kullanılır. Dizinin eleman sayısı, dizinin **uzunluğudur** (**size**) ve dizinin uzunluğu dizi tanımlanırken belirlenir!

7.2 Dizi Tanımlama

C# dilinde **tek boyutlu** (**one-dimensional**), **çok boyutlu** (**multidimensional**) ve **pürüzlü dizi** (**jagged array**) olmak üzere üç farklı dizi tanımlanabilir;

```
veri-tipi[] dizi-kimliği; // Tek boyutlu dizi
veri-tipi[,] dizi-kimliği; // Çok boyutlu dizi, Her bir boyut virgül ile ayrılır
veri-tipi[][] dizi-kimliği; // Pürüzlü dizi
```

C# dilinde bir dizi tanımlanırken, dizinin tutacağı veri tipi ve eleman sayısı mutlaka belirtilmelidir. Diziler **referans tipli** (**reference type**) nesnelerdir. Tüm referans tiplerde olduğu gibi diziler de ve **new** anahtar kelimesiyle, eleman sayısı belirtilerek, öbek (**heap**) bellekte oluşturulurlar. **Tahsis edilen bellek bölgesinin adresini referans tipli dizi değişkeni tutar.**

Dizilerin;

- ◇ İçinde, **aynı veri tipindeki öğeleri barındırır** ve bu öğeler **bitişik bellek bölgesini paylaşırlar**.
- ◇ Elemanları, temel veri tipleri (**byte**, **short**, **int**, **long**, **float**, **double**, ...) veya daha sonra göreceğimiz kullanıcı tanımlı tip olan yapı **struct** veya gösterici (**pointer**) olabilir.
- ◇ İçinde aynı değer birkaç farklı yerde bulunabilir.
- ◇ Veri tipi, **dizideki elemanlarının veri tipidir**. Bir başka deyişle **elemanların veri tipi dizinin veri tipiyle aynı olmalıdır**.
- ◇ Elemanlarının indisi (**index**) sıra belirttiğinden **her zaman tam sayıdır**.
- ◇ İlk elemanın indisi **0**, son elemanın indisi dizi uzunluğundan (**size**) bir küçük olan tam sayıdır. Bunların dışında bir indis girildiğinde hata alınır.

Aşağıda değişik dizi tanımlama örnekleri verilmiştir;

```
//1. Elşeman Sayısı Bilinen TEK Boyutlu Dizi;
int[] ilkdeğeriolmayandizi = new int[5]; // 5 Elemanlı Tek Boyutlu Dizi
ilkdeğeriolmayandizi = null; // üst satırda ayrılan bellek serbest bırakılır.

//2. Eleman sayısı ve Eleman Değerleri Bilinen Dizi;
int[] dizi1 = new int[6] { 1, 2, 3, 4, 5, 6 }; // 6 Elemanlı Dizi
int[] dizi2 = { 1, 2, 3, 4, 5, 6 }; // Elemanları Listesi Verilertek Dizi Başlatma

//3. Biz Dizi Referansı Bildirme:
int[] dizi3; //dizi tanımlandıktan sonra elemanlara ilk değer verilemez!
```

```
// dizi3 = { 24, 2, 13, 47, 45 }; //derleme hatası olur!

//4. Üstü Kapalı Olarak Dizi Tanımlama;
var arr1 = new [] { 1, 2, 3 }; // new int[] { 1, 2, 3 }; ile aynıdır
var arr2 = new [] { "one", "two", "three" }; // string[] ile aynıdır
var arr3 = new [] { 1.0, 2.0, 3.0 }; // double[] ile aynıdır
```

7.3 Dizi Elemanlarını Değiştirme

Bir dizi tanımlandıktan sonra tam sayı indislerle elemanlara erişilebilir ve erişilen eleman değeri değiştirilebilir;

```
int[] dizi = new int[] { 0, 10, 20, 30};
Console.WriteLine(arr[2]); // dizinin 3. Elemanı yazdırılıyor: 20
arr[2] = 100; //dizinin 3.elemanına 100 değeri atandı.
Console.WriteLine(arr[2]); // güncellenmiş elemana yazdırılıyor: 100
```

7.4 Dizi Üzerinde Yinelemeler

Dizi üzerindeki elemanlarının tümüne yineleme (**iteration**) ile erişilebilir. Yineleme indis değişkeni içeren **for** döngüleriyle yapılır.

Tek boyutlu dizilere örnek olarak klavyeden girilen 5 adet tam sayıyı, giriş sırasının tersinden ekrana yazan programını verebiliriz;

```
int[] dizi = new int[5]; // 5 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++) {
    Console.Write($"{indis}.Sayıyı Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
for (int indis = dizi.Length-1; indis >= 0; indis--)
    Console.WriteLine($"{indis}.Sayı:{dizi[indis]}");
/*
0.Sayıyı Girin:10
1.Sayıyı Girin:11
2.Sayıyı Girin:13
3.Sayıyı Girin:45
4.Sayıyı Girin:55
4.Sayı:55
3.Sayı:45
2.Sayı:13
1.Sayı:11
0.Sayı:10
*/
```

Bir başka örnek olarak konsoldan girilen 10 adet sınav notundan, ortalamanın üstünde olanları ekrana yazan programı;

```
int[] dizi = new int[10]; // 10 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++) {
    Console.Write($"{indis}.Notu Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
Console.WriteLine("{0} üzerindeki notlar:", dizi.Average());
for (int indis = 0; indis < dizi.Length; indis++)
    if ((double)dizi[indis] > dizi.Average())
        Console.WriteLine($"{indis}.Not:{dizi[indis]}");
/*Program Çıktısı:
0.Notu Girin:10
1.Notu Girin:15
2.Notu Girin:25
3.Notu Girin:35
4.Notu Girin:40
5.Notu Girin:60
6.Notu Girin:15
7.Notu Girin:20
```

```

8.Notu Girin:12
9.Notu Girin:16
24,8 üzerindeki notlar:
2.Not:25
3.Not:35
4.Not:40
5.Not:60
*/

```

Bir başka örnek olarak değeri belli 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani **sapma** (**deviation**) ve karesi alınmış sapmalar yani **varyans** (**variance**) ile standart sapmayı yazdıran program verilebilir.

```

// 10 elemanlı tek boyutlu bir tamsayı dizisi:
decimal[] satislar = { 100.0M, 850.0M, 500.0M, 600.0M, 750.0M, 650.0M, 450.0M, 800.0M, 900.0M,
    ↪ 110.0M };
decimal ortalama = satislar.Average();
Console.WriteLine($"Ortalama:{ortalama}");
double varyansToplam = 0;
for (int indis = 0; indis < satislar.Length; indis++) {
    decimal sapma = satislar[indis] - ortalama;
    decimal varyans = sapma * sapma;
    Console.WriteLine($"Sapma:{sapma}, varyans:{varyans}");
    varyansToplam += (double) varyans;
}
double standartSapma = Math.Sqrt(varyansToplam / satislar.Length);
Console.WriteLine($"Standat Sapma:{standartSapma}");
/*
Ortalama:571,0
Sapma:-471,0, varyans:221841,00
Sapma:279,0, varyans:77841,00
Sapma:-71,0, varyans:5041,00
Sapma:29,0, varyans:841,00
Sapma:179,0, varyans:32041,00
Sapma:79,0, varyans:6241,00
Sapma:-121,0, varyans:14641,00
Sapma:229,0, varyans:52441,00
Sapma:329,0, varyans:108241,00
Sapma:-461,0, varyans:212521,00
Standat Sapma:270,49768945408755
*/

```

Ortalamaya olan uzaklıklar yani sapma verilerin ortalamaya ne kadar yakın olduğunu gösteren dağılımdır. Sapmaların toplamı sıfır olabileceğinden karelerinin toplamı yani varyans hesaplanır. Varyansın eleman sayısına bağlı karekökü de standart sapmayı verir. Standart sapmanın büyük olması verilerin ortalamadan daha uzak yayıldıklarını; küçük bir standart sapma ise verilerin ortalama etrafında daha çok yakın gruplaştıklarını gösterir.

Bir başka örnek olarak 5 elemanlı diziye girilen elemanlardan **tekil** (**unique**) olanlarını bulan program verilebilir.

```

int[] dizi = new int[5]; // 5 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++) {
    Console.Write($"{indis}.Sayıyı Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
Console.WriteLine("Dizi İçindeki Tekil Elemanlar:");
for (int indis = 0; indis < dizi.Length; indis++) {
    int indis2, sayac = 0; // Her elemanda sayacı sıfırla
    for (indis2 = 0; indis2 < dizi.Length; indis2++)
        if (indis != indis2) //elemanın kendisini kontrol etmiyoruz
            if (dizi[indis] == dizi[indis2])
                sayac++;
    if (sayac == 0)
        Console.WriteLine($"dizi[{indis}]:{dizi[indis]} tekil olarak dizide bulundu.");
}

```

```

/*
0.Sayıyı Girin:10
1.Sayıyı Girin:15
2.Sayıyı Girin:12
3.Sayıyı Girin:10
4.Sayıyı Girin:12
Dizi İçindeki Tekil Elemanlar:
dizi[1]:15 tekil olarak dizide bulundu.
*/

```

Bir başka örnek olarak 5 elemanlı belli olan dizide en büyük elemana en küçük elemanını ekleyen program verilebilir;

```

int[] dizi = { 10, 12, 3, 4, 6, 12, 4, 7, 8, 9 };
Console.WriteLine("Dizinin İLK Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
Console.WriteLine();
int enKucuk = dizi[0], enBuyuk = dizi[0];
/* İlk elemanlar hem en küçük hem de en büyük olsun. */
int enKucukIndex = 0, enBuyukIndex = 0;
/* En küçük ve en büyük elemanların indisi 0 olsun. */
for (int indis = 0; indis < dizi.Length; indis++) {
    /* En küçük ve en büyük eleman ile indislerini bulan kısım. */
    if (dizi[indis] < enKucuk) {
        enKucuk = dizi[indis];
        enKucukIndex = indis;
    }
    if (dizi[indis] > enBuyuk) {
        enBuyuk = dizi[indis];
        enBuyukIndex = indis;
    }
}
Console.WriteLine($"En Büyük Eleman İndisi: {enBuyukIndex} ve Eleman {enBuyuk}");
Console.WriteLine($"En Küçük Eleman İndisi: {enKucukIndex} ve Eleman {enKucuk}");
//En büyük elemana en küçüğünü ekleme
dizi[enBuyukIndex] += dizi[enKucukIndex];
Console.WriteLine("Dizinin SON Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
/*
Dizinin İLK Hali:
10 12 3 4 6 12 4 7 8 9
En Büyük Eleman İndisi: 1 ve Eleman 12
En Küçük Eleman İndisi: 2 ve Eleman 3
Dizinin SON Hali:
10 15 3 4 6 12 4 7 8 9
*/

```

Aynı program dizi nesnesinin yöntemleriyle aşağıdaki gibi yazılabilir;

```

int[] dizi = { 10, 12, 3, 4, 6, 12, 4, 7, 8, 9 };
Console.WriteLine("Dizinin İLK Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
Console.WriteLine();
int enKucuk = dizi.Min();
int enBuyuk = dizi.Max();
/* İlk elemanlar hem en küçük hem de en büyük olsun. */
int enKucukIndex = dizi.Single(enKucuk);
int enBuyukIndex = dizi.Single(enBuyuk);
Console.WriteLine($"En Büyük Eleman İndisi: {enBuyukIndex} ve Eleman {enBuyuk}");
Console.WriteLine($"En Küçük Eleman İndisi: {enKucukIndex} ve Eleman {enKucuk}");
//En büyük elemana en küçüğünü ekleme

```

```
dizi[enBuyukIndex] += dizi[enKucukIndex];
Console.WriteLine("Dizinin SON Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.WriteLine($"{dizi[indis],3}");
```

7.5 Dizileri Kopyalama

Dizinin bir kısmı bir başka diziye `Copy()` yöntemiyle kopyalanabilir;

```
var kaynak1 = new int[] { 11, 12, 3, 5, 2, 9, 28, 17 };
var hedef1 = new int[3];
Array.Copy(kaynak1, hedef1, 3); // 11, 12, 3
```

Kaynak dizinin bir kısmı hedef dizinin bir bölümüne `CopyTo()` yöntemiyle kopyalanabilir;

```
var kaynak2 = new int[] { 11, 12, 7 };
var hedef2 = new int[6];
kaynak2.CopyTo(hedef2, 2); // 0, 0, 11, 12, 7, 0
```

Bir dizinin özdeşi `Clone()` yöntemiyle yapılabilir;

```
var kaynak3 = new int[] { 11, 12, 7 };
var hedef3 = kaynak3.Clone(); // 11,12,17
```

7.6 Dizileri Karşılaştırma

Diziler, dizi nesnesinin `SequenceEqual()` yöntemi ile karşılıklı elemanlar karşılaştırılabilir.

```
int[] dizi1 = { 3, 5, 7 };
int[] dizi2 = { 3, 5, 7 };
bool sonuc = dizi1.SequenceEqual(dizi2);
Console.WriteLine("İki Dizi Eşit Mi? {0}", sonuc); //true
```

7.7 Array Sınıfı Yöntemleri

Yukarıda birkaç yöntemi açıklanan `Array` sınıfının yöntemleri aşağıda verilmiştir;

Yöntem	Açıklama	Örnek Kullanım
<code>Sort()</code>	Dizi elemanlarını küçükten büyüğe (veya alfabetik) sıralar.	<code>Array.Sort(sayilar);</code>
<code>Reverse()</code>	Dizinin mevcut eleman sıralamasını tam tersine çevirir.	<code>Array.Reverse(sayilar);</code>
<code>Clear()</code>	Belirlenen aralıktaki elemanları varsayılan değerine (0, null vb.) çeker.	<code>Array.Clear(sayilar, 0, 2);</code>
<code>IndexOf()</code>	Belirli bir değer dizideki ilk konumunu (indisini) döndürür.	<code>int sira = Array.IndexOf(dizi, "C#");</code>
<code>LastIndexOf()</code>	Belirli bir değer dizideki son konumunu döndürür.	<code>Array.LastIndexOf(dizi, 5);</code>
<code>Copy()</code>	Bir dizinin bir kısmını veya tamamını başka bir diziye kopyalar.	<code>Array.Copy(kaynak, hedef, 3);</code>
<code>Resize()</code>	Dizinin boyutunu yeniden ayarlar (Arka planda yeni bir dizi oluşturur).	<code>Array.Resize(ref dizi, 10);</code>
<code>BinarySearch()</code>	Sıralanmış bir dizide ikili arama yaparak değer indisini bulur.	<code>int konum = Array.BinarySearch(dizi, 42);</code>
<code>Find()</code>	Belirli bir koşula uyan ilk elemanı döndürür.	<code>var sonuc = Array.Find(dizi, x => x > 5);</code>
<code>Exists()</code>	Dizide belirli bir koşulu sağlayan eleman olup olmadığını kontrol eder.	<code>bool varMi = Array.Exists(dizi, x => x < 0);</code>

Tablo 7.1: Array Sınıfı Yöntemleri

7.8 Çok Boyutlu Diziler

Su ana kadar gördüğümüz tek boyutlu dizilerdi. **İki boyutlu diziler** (**two-dimentional array**) matematikten de bildiğimiz üzere matris (**matrix**) olarak adlandırılır. İki boyutlu dediğimizde en-boy ve satır-sütun gibi kavramlar akla gelmelidir. Matris işlemleri gibi bazı problemlerde; bir dizinin her bir elemanının da dizi olması istenir. Bu tür iki boyutlu dizilerde en içteki dizinin boyutu kimliklendirmede sağda yer alır.

```
// 2 satır ve 3 sürundan oluşan iki boyutlu tam sayı dizisi aşağıdaki gibi tanımlanır;
int[,] çokboyutludizi = new int[2, 3];
// Eleman değerleri bilinen bir iki boyutlu dizinin başlatılması aşağıdaki gibi yapılır;
int[,] çokboyutludizi1 = new int[2, 3] { { 1, 2, 3 }, { 4, 5, 6 } };
int[,] çokboyutludizi2 = { { 1, 2, 3 }, { 4, 5, 6 } };
```

Örnek olarak 3x4'lük matris elemanlarını klavyeden girip, tablo halinde ekrana yazdıran programı verebiliriz;

```
int[,] matris = new int[3, 4]; // 3 satır 4 sütun
int satır, sütun; // indis değişkenleri
/*
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[0,sütun] = Convert.ToInt32(Console.ReadLine());
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[1,sütun] = Convert.ToInt32(Console.ReadLine());
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[2,sütun] = Convert.ToInt32(Console.ReadLine());
//Bunun yerine iç içe iki for tanımlanır;
*/
for (satır = 0; satır < matris.GetLength(0); satır++)
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
        matris[satır, sütun] = Convert.ToInt32(Console.ReadLine());
Console.WriteLine("Okunan Matris:");
for (satır = 0; satır < matris.GetLength(0); satır++) {
    for (sütun = 0; sütun < matris.GetLength(1); sütun++) //Birinci Boyut İçin
        Console.Write($"{matris[satır, sütun],4}");
    Console.WriteLine();
}
/*
11
12
13
14
21
22
23
24
31
32
33
34
Okunan Matris:
11 12 13 14
21 22 23 24
31 32 33 34
*/
```

Bir başka örnek olarak elemanları belirli 5x4'lük matrisin satır ve sütun toplamalarını ekrana yazan programı verilebilir;

```
int[,] matris = { {10,20,30},
                  {40,50,60},
                  {70,80,90},
                  {15,25,35},
                  { 5,15,25} };
int satır, sütun;
Console.WriteLine("Matris:");
```

```

for (satır = 0; satır < matris.GetLength(0); satır++) {
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
        Console.WriteLine($"{matris[satır, sütun],4}");
    Console.WriteLine();
}
Console.WriteLine("Satır Toplamları:");
for (satır = 0; satır < matris.GetLength(0); satır++) {
    int satırToplam = 0;
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
        satırToplam += matris[satır, sütun];
    Console.WriteLine($" {satırToplam,4}");
}
Console.WriteLine();
Console.WriteLine("Sütun Toplamları:");
for (sütun = 0; sütun < matris.GetLength(1); sütun++) {
    int sütunToplam = 0;
    for (satır = 0; satır < matris.GetLength(0); satır++)
        sütunToplam += matris[satır, sütun];
    Console.WriteLine($" {sütunToplam,4}");
}
/*
Matris:
10 20 30
40 50 60
70 80 90
15 25 35
5 15 25
Satır Toplamları: 60 150 240 75 45
Sütun Toplamları: 140 190 240
*/

```

Çok boyutlu dizi (multi-dimentional array) örneği olarak şöyle bir örnek verilebilir; Üç farklı dersten (1.boyut) dört değişik not (2.boyut) alan 10 öğrencili (3.boyut) bir sınıf vardır. Notların ağırlıkları %10, %30, %20 ve %40'tır. Notlar rastgele 0 ile 100 arasında verilecektir. Her bir sınavın ortalaması ile her bir öğrencinin ağırlıklı not ortalamalarını bulan programı aşağıda verilmiştir;

```

int[, ,] notlar = new int[3, 4, 10]; //DERSSAYISI=3, SINAVSAYISI=4, ÖĞRENCİSAYISI=10
int i, j, k; //indis değişkenleri sırasıyla i:ders, j:sınav, k:öğrenci
int[] agirlik = { 10, 30, 20, 40 }; //Odevler:%10, Vize:%30, Hızlı Sınav:%20, Final:%40
Random rastgete = new Random();
//Console.WriteLine("Notlar:");
for (i = 0; i < notlar.GetLength(0); i++) { //derslerdeki (3. boyut)
    //Console.WriteLine($"{i}. Ders: ");
    for (j = 0; j < notlar.GetLength(1); j++) { //sınavlara giren (2. boyut)
        //Console.WriteLine($" {j}. Sınav:");
        for (k = 0; k < notlar.GetLength(2); k++) { //öğrencilerin (1. boyut)
            int rastgeleNot = rastgete.Next(0, 100); //Notlarını Rastgele Not Veriliyor!
            notlar[i, j, k] = rastgeleNot;
            //Console.WriteLine($"{rastgeleNot} ");
        }
        //Console.WriteLine();
    }
}
Console.WriteLine("Sınav Ortalamaları:");
for (i = 0; i < notlar.GetLength(0); i++) { //derslerdeki (3. boyut)
    Console.WriteLine($"{i}.Ders:");
    for (j = 0; j < notlar.GetLength(1); j++) { //sınavlara giren (2. boyut)
        Console.WriteLine($" {j}.Sınav: ");
        int notToplam=0;
        for (k = 0; k < notlar.GetLength(2); k++) //öğrencilerin Notları (1. boyut)
            notToplam += notlar[i, j, k];
        float ortalama=notToplam / notlar.GetLength(2);
        Console.WriteLine($"{ortalama:F2} ");
    }
}

```



```

    }
    Console.WriteLine();
}
Console.WriteLine("Her bir Öğrencinin Ağırlıklı Ortalaması:");
for (k = 0; k < notlar.GetLength(2); k++) { //öğrencilerin (1. boyut)
    Console.Write($"{k}.Öğrenci: ");
    for (i = 0; i < notlar.GetLength(0); i++) { //derslerdeki (3. boyut)
        Console.Write($"{i}.Ders: ");
        float agirlikliNot = 0.0f;
        for (j = 0; j < notlar.GetLength(1); j++) { //sınavları (2. boyut)
            agirlikliNot += notlar[i, j, k] * agirlik[j] / 100.0f;
            Console.Write($"{agirlikliNot:F1} ");
        }
    }
    Console.WriteLine();
}
}
/*Program Çıktısı:
Sınav Ortalamaları:
0.Ders: 0.Sınav: 34.00 1.Sınav: 54.00 2.Sınav: 56.00 3.Sınav: 45.00
1.Ders: 0.Sınav: 50.00 1.Sınav: 49.00 2.Sınav: 46.00 3.Sınav: 46.00
2.Ders: 0.Sınav: 56.00 1.Sınav: 36.00 2.Sınav: 43.00 3.Sınav: 66.00
Her bir Öğrencinin Ağırlıklı Ortalaması:
0.Öğrenci: 0.Ders: 2.5 18.1 29.1 39.9 1.Ders: 5.6 17.3 30.7 47.5 2.Ders: 2.0 18.8 30.4 41.2
1.Öğrenci: 0.Ders: 5.2 32.5 46.3 79.9 1.Ders: 8.5 11.5 28.1 32.5 2.Ders: 7.8 14.7 22.3 59.9
2.Öğrenci: 0.Ders: 1.4 21.2 32.2 51.0 1.Ders: 2.2 12.7 17.1 38.7 2.Ders: 8.4 24.6 31.4 51.0
3.Öğrenci: 0.Ders: 8.6 12.2 27.6 46.0 1.Ders: 6.0 19.5 20.7 41.1 2.Ders: 1.1 21.5 40.5 78.1
4.Öğrenci: 0.Ders: 2.8 28.9 37.1 37.9 1.Ders: 5.9 34.4 44.0 77.6 2.Ders: 2.9 5.3 12.7 29.9
5.Öğrenci: 0.Ders: 0.5 12.8 28.2 53.0 1.Ders: 2.0 11.9 16.7 25.5 2.Ders: 7.8 12.9 13.9 36.7
6.Öğrenci: 0.Ders: 1.2 24.6 34.6 63.0 1.Ders: 7.4 32.9 38.5 38.9 2.Ders: 9.3 14.1 25.3 60.1
7.Öğrenci: 0.Ders: 1.4 17.3 30.5 46.5 1.Ders: 2.8 11.2 26.8 48.0 2.Ders: 6.0 17.7 24.1 59.7
8.Öğrenci: 0.Ders: 7.8 12.9 22.7 31.9 1.Ders: 3.2 12.5 17.5 55.9 2.Ders: 1.6 6.7 11.3 41.3
9.Öğrenci: 0.Ders: 3.5 16.7 22.7 42.7 1.Ders: 7.2 35.4 51.8 73.4 2.Ders: 9.1 29.8 40.6 59.0
*/

```

7.9 Pürüzlü Diziler

Pürüzlü diziler (jagged array) dizilerin dizisi olarak adlandırılabilir. Buradaki her dizi farklı boyutta olabilir.

```

// Pürüzlü (jagged) dizi
int[][] pürüzlüDizi = new int[5][]; // 5 adet dizinin dizisi
pürüzlüDizi[0] = new int[3] { 1, 2, 3 }; //birinci dizi 3
pürüzlüDizi[1] = new int[] { }; // ikinci dizi 0 elemanlı
pürüzlüDizi[2] = new int[2] { 4, 5 }; // üçüncü dizi 2 elemanlı
pürüzlüDizi[3] = new int[1] { 6 }; // dördüncü dizi 1 elemanlı
pürüzlüDizi[4] = new int[5] { 7, 8, 9, 10, 11 }; // beşinci dizi 5 elemanlı
Console.WriteLine("Pürüzlü Dizi:");
for (int i = 0; i < pürüzlüDizi.Length; i++) {
    for (int j=0; j < pürüzlüDizi[i].Length; j++)
        Console.Write("{0,4}",pürüzlüDizi[i][j]);
    Console.WriteLine();
}

```

7.10 Foreach Talimatı

Yalnızca bir koleksiyondaki öğeler arasında döngü oluşturmak için kullanılan **foreach döngüsü (foreach loop)**, **aralık tabanlı döngü (range based loop)** olarak da bilinir. Bu döngü ileride göreceğimiz bütün koleksiyonlarda kullanılır. Aşağıdaki gibi yazılır;

```
foreach (veritipi dizi-elemanını-temsil-eden-değişken in dizi-kimliği)
    döngü-kodu;
```

foreach, bir koleksiyonun (dizi, liste vb.) her bir öğesi için bir kez çalışan, yönetimi tamamen C# derleyicisine bırakılmış bir döngü yapısıdır. Buna örnek olarak aşağıda bir dizi programı örneği verilebilir;

```
int[] dizi = { 1, 2, 3, 4, 5 };
float toplam = 0.0f;
foreach (int eleman in dizi)
    toplam += eleman;
float ortalama = toplam / dizi.Length;
Console.WriteLine("Dizi Ortalaması:{0}", ortalama);
```

§7.10.1 Foreach Neden Tercih Edilir?

- ♦ **Güvenlik:** `for` döngüsünde dizinin sınırlarını aşma (`IndexOutOfRangeException`) riski varken, `foreach` ile bu risk yoktur. C# dizinin sonuna geldiğini kendisi bilir.
- ♦ **Okunabilirlik:** İndis değişkenleri (`i`, `j`) ile uğraşmazsınız. Kod, "Koleksiyondaki her bir öge için şunu yap" şeklinde bir cümle gibi okunur.
- ♦ **Soyutlama:** Sadece dizilerle değil, ileride göreceğimiz `List`, `Dictionary`, `Stack` gibi tüm `IEnumerable` arayüzünü uygulayan yapılarla aynı şekilde çalışır.

`foreach` döngüsü içerisinde, o an üzerinde gezindiğiniz değişkenin değerini değiştiremezsiniz!

```
int[] sayilar = { 1, 2, 3 };
foreach (int s in sayilar)
{
    // s = s * 2; // HATA! Döngü değişkenine atama yapılamaz.
}
```

§7.10.2 IEnumerable ve IEnumerator

Derleyici `foreach` gördüğünde onu bir `while` döngüsüne çevirir. Aslında arka planda şu işlemler döner:

- ♦ Koleksiyondan bir `GetEnumerator()` istenir.
- ♦ `MoveNext()` metodu ile bir sonraki öge var mı bakılır.
- ♦ `Current` özelliği ile o anki değer okunur.

Bunun detayı için ileride anlatacağımız Yineleyici desenini inceleyebilirsiniz.

`foreach` döngüsü ile bir listenin içinde gezerken, listeden bir öğeyi silemeyiz (`liste.Remove(öge)`). C# çalışma zamanı (`run-time`), bir koleksiyon `foreach` ile taranırken eleman ekleme ve çıkarma gibi koleksiyonun yapısının değiştirilmesine izin verilmez ve `InvalidOperationException` hatası fırlatılır.

7.11 Dizilerde Dilimleme

Geleneksel programlamada bir dizinin son elemanına erişmek veya belirli bir parçasını kopyalamak için `array.Length-1` gibi matematiksel hesaplamalar gerekirdi. C#8.0+ ile gelen `Index` ve `Range` yapıları, bu işlemleri doğal bir söz dizimi ile gerçekleştirmenize olanak tanır.

§7.11.1 Index Yapısı

`System.Index` yapısı, bir dizinin başından veya sonundan bir konumu temsil eder. Buradaki en kritik araç, şapka işleci (hat operator) olarak da bilinen (^) işaretidir;

- ♦ `^1`: Dizinin son elemanını (`Length-1`) temsil eder.
- ♦ `^0`: Dizinin uzunluğunu temsil eder (doğrudan indisleme için kullanıldığında hata verir, ancak aralıklarda bitiş belirtmek için kullanılır).

```
string[] diller = { "C", "C++", "C#", "Java", "Python" };
// Eski yöntem
string sonDilEski = diller[diller.Length - 1];
// Modern yöntem
string sonDilYeni = diller[^1]; // Python
string sondanIkincisi = diller[^2]; // Java
```

§7.11.2 Range Yapısı

`System.Range` yapısı, bir dizinin başlangıç ve bitiş indislerini belirterek bir **dilim** (`slice`) almanızı sağlar. **Aralık işleci** (`range operator`) olarak bilinen `..` işleci ile işlem yapılır.

```
int[] sayilar = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
// 1. indisten başla, 4. indise kadar git (4 dahil değil)
int[] altDizi = sayilar[1..4]; // { 1, 2, 3 }
// İlk 3 elemanı al
```

```
int[] ilkUc = sayilar[..3]; // { 0, 1, 2 }
// 7. indisten sona kadar al
int[] sonUc = sayilar[7..]; // { 7, 8, 9 }
// Tüm diziyi kopyala
int[] kopya = sayilar[..];
```

7.12 Düşün ve Çöz

- ◊ **Düşün:** C#'ta dizi ile `List<T>` arasındaki temel farklar nelerdir? Sabit boyutlu bir dizi yerine liste kullanmak her zaman daha iyi midir? Hangi senaryolarda dizi tercih edilmelidir?
- ◊ **Düşün:** Çok boyutlu dizi (**multidimensional array**) ile pürüzlü dizi (**jagged array**) arasındaki farkı bellek kullanımı açısından açıklayınız. Bir matris hesaplama için hangisi daha uygundur?
- ◊ **Düşün:** `foreach` döngüsü, `for` döngüsüne kıyasla bir dizi üzerinde yinelemede hangi avantajları sağlar? `foreach` içinde dizinin eleman değerini değiştirmeye çalışırsanız ne olur ve neden?
- ◊ **Düşün:** Bir dizinin sıralanması (**sorting**) için kabarcık sıralama (**bubble sort**) ile `Array.Sort()` karşılaştırıldığında performans ve kullanım kolaylığı açısından hangisi tercih edilmelidir? Kendi sıralama algoritmanızı yazmayı ne zaman düşünürsünüz?
- ◊ **Düşün:** Dizi sınırı dışı erişim (**IndexOutOfRangeException**) C#'ta en sık karşılaşılan hatalardan biridir. Bu hatayı önlemenin hem tasarım hem de kod yazım aşamasındaki yolları nelerdir?
- ◊ **Düşün:** `IEnumerable<T>` arayüzü diziler ve koleksiyonlar için neden temel bir soyutlama olarak kabul edilir? `foreach` döngüsünün arka planda bu arayüzü nasıl kullandığını açıklayınız.
- ◊ **Düşün:** Dizileri kopyalarken yüzeysel kopya (**shallow copy**) ile derin kopya (**deep copy**) arasındaki farkı referans tipler içeren bir dizi üzerinden açıklayınız. `Array.Copy()` hangi türde kopya yapar?
- ◊ **Çöz:** 10 elemanlı bir `int` dizisi tanımlayın, elemanları rastgele 1-100 arasında doldurun ve kabarcık sıralama (**bubble sort**) algoritmasıyla küçükten büyüğe sıralayın. Her adımı konsola yazdırın.
- ◊ **Çöz:** Kullanıcıdan N adet not alan (N kullanıcı tarafından girilecek) ve bu notların ortalamasını, standart sapmasını, en yüksek ve en düşük notu hesaplayıp ekrana yazan programı yazınız.
- ◊ **Çöz:** Bir 3x3 matris (çok boyutlu dizi) tanımlayıp elemanlarını kullanıcıdan alın. Matrisin transpozunu hesaplayıp ekrana yazdırın.
- ◊ **Çöz:** Bir `string` dizisi içinde aynı ismin kaç kez geçtiğini bulan; 'İlhan', 'Ahmet', 'Mehmet', 'İlhan', 'Ahmet', 'İlhan' gibi bir dizide her ismin tekrar sayısını gösteren programı yazınız.
- ◊ **Çöz:** İki adet sıralı `int` dizisini tek bir sıralı dizide birleştiren (**merge**) algoritmasını `Array.Sort()` kullanmadan yazınız. Zaman karmaşıklığını belirtiniz.
- ◊ **Düşün:** Bir dizinin boyutunu program çalışırken neden `sayilar.Length = 10;` diyerek değiştiremiyoruz? Eğer daha büyük bir diziye ihtiyacımız olursa ne yapmalıyız?
- ◊ **Düşün:** `foreach` döngüsüyle bir dizi üzerinde dönerken dizi elemanlarını değiştiremiyoruz; ancak `for` döngüsüyle değiştirebiliyoruz. Bu kısıtlamanın temel sebebi (`IEnumerable` mantığıyla) nedir?
- ◊ **Düşün:** Elinizde bir sınıfın öğrenci isimlerini tutan bir dizi var. Bu diziyi alfabetik sıraya dizmek için C#'ın hangi hazır metodunu kullanırsınız?

8. Bölüm

Nesne Yönelimli Programlama

8.1 Nesne Yönelimli Programlama Paradigması

Yapısal programlamada talimatlar (**statement**) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya **emreden programlama** (**imperative programming**) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.

Emreden programlamanın bir örneği olan yapısal programlamada talimatlar yine art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Kodlar büyüdükçe sorunlar artmış ve 2.8 başlığı altında anlatılan problemler ortaya çıkmıştır. Bu yıllarda Simula ve Smalltalk yaklaşımı yazılımcıların imdadına yetişmiştir. Bu dillerde **nesneler** (**object**) programın temel yapı taşlarıdır. Nesneler; **durum** (**state**) ve **davranışlara** (**behavior**) sahiptir. Programlama, nesnelerin birbirlerine **ileti göndermesiyle** (**message-passing**) yapılır.

Nesne yönelimli programlamada **kontrol akışı** (**kontrol flow**), **nesnelerin birbirlerine yöntem çağrıları yaparak etkileşime girmesiyle** yönetilir.

1979 senesinde bir Danimarkalı bilgisayar bilim adamı olan *Bjarne Stroustrup*, sonradan C++ olarak bilinecek olan sınıflarla C (**C with classes**) üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı. C++ dili uzun yıllar donanımına yakınlığı nedeniyle ve yüksek performans isteyen uygulamalarda hala tercih edilmektedir. Ancak saf nesnelerden oluşan bir programlama dili değildir. Bir de kodlar her platform (işletim sistemi ve işlemci için) için yeniden derlenmelidir.

C# programlama dili saf nesne yönelimli programlama dili olup, Microsoft tarafından geliştirilmiştir. 2000 Yılında kapalı kaynak olarak geliştirilen bu dil, 2016 yılından sonra **açık kaynak** olarak yayınlanmıştır. C++ ve C dilinden daha fazla kütüphaneye sahiptir. .NET platformu için derlendiğinden geliştirilen uygulamalar, derlenmeden platformlar arası taşınarak çalıştırılabilmektedir.

C# dilinde, altyapının bize sunduğu kütüphanelerdeki hazır nesneleri kullanarak programları geliştiremeyiz. Mutlaka programcı tarafından ihtiyaçlara göre veri içeren ve davranış gösteren nesnelere ihtiyaç duyarız. İşte bu nesneler, **kullanıcı tanımlı sınıflar** (**user defined class**) ile tanımlanır.

Emreden paradigmanın aksine nesnelerin birbirlerine ileti göndermesi (yöntem çağrıları yaparak etkileşime girmesi) bakış açısıyla yapılan programlamaya **nesne yönelimli programlama** OOP (**object-oriented programming**) paradigması adı verilir.

8.2 Sınıf ve Nesne Nedir? Nasıl Tanımlanır?

Gerçek dünyamızda nesneleri sınıflamaya tabi tutarız. Örneğin bir birine benzeyen at, eşek ve zebra gibi hayvanları daha çocukluktan itibaren ayırt edebilir. İşte kafamızda oluşan bu mekanizmaya **sınıflayıcı** (**classifier**) adı verilir. Bu mekanizma ile sınıflanmış her bir türün gerçek dünyadaki her birine ise **örnek** (**instance**) adı verilir.

Nasıl ki dünyamızda evler bir mimari plan (**blueprint**) üzerinden inşa ediliyor, yazılımda kullanacağımız **nesneler** (**object**) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu **durumların** (**state**) ve nesnelerin göstereceği **davranışlarının** (**behavior**) tanımlandığı bu plana **sınıf** (**class**) adı verilir. Nesneler (**object**) yukarıda anlatılan gerçek dünya **örneklerine** (**instance**) karşılık gelir. Sınıflar ise **sınıflayıcıya** (**classifier**) karşılık gelir.

Bir mimari plandan birçok ev yapılabileceği gibi, **bir sınıftan da birçok nesne imal edilebilir.**

Gerçek dünyadaki nesneleri modelleyen **kullanıcı tanımlı sınıflar** (**user defined class**), aşağıdaki şekilde tanımlanır;

```
[erişim-değiştiricisi] class sınıf-kimliği
```

```
{
    [erişim-değiştiricisi] veri-tipi alanKimliği; // Alan değişkeni (field variable)
    [erişim-değiştiricisi] veri-tipi yöntemKimliği() // Davranış/Yöntem
    {
        // Yöntemler, yapısal programlamadaki fonksiyonlar gibi kodlanır.
    };
}
```

İmal edilecek nesnelerin durumları (**state**), sınıftaki alanlarla (**field**) tanımlanır. Nesnelerin göstereceği davranışlar da sınıf içinde fonksiyon gibi tanımlanır. **Aynı davranış, farklı yöntemlerle yerine getirilebileceğinden**, davranışları tanımlayan bu fonksiyonlara **yöntem (method)** adı verilir. İşin doğası gereği davranışlar durumlardan etkilenirler.

Özet olarak sınıflar, kodumuzda **imal edilecek nesnelere ilişkin ortak davranış ve durumları tasnif eden bir taslak bileşenidir.**

Kullanıcı Tanımlı sınıflar referans tip olduğundan bu sınıflardan, **new** işleci (**new operator**) ile nesne imal edilir. Bu husus 4.2.2 başlığında anlatılmıştı. Bir referans tipine ilişkin bellekte yer tahsisi, çalışma zamanında (**run-time**), öbek (**heap**) bellek üzerinde yapılır.

Şimdi ortak davranış ve özelliklerin tanımlandığı bir **Öğrenci** sınıfı ve bu sınıftan nesne imal eden ana programı kodlayalım;

```
1 //ANA PROGRAM:
2 Öğrenci ali = new Öğrenci();
3 ali.yaş = 40;
4 ali.YaşınıSöyle();
5 ali.CinsiyetiniSöyle();
6
7 //SINIF:
8 class Öğrenci // "Öğrenci" Kimlikli Sınıf
9 {
10     // Alanlar (Fields) isimleri için Deve Yazımı tercih edilir.
11     public uint yaş;
12     public char cinsiyet;
13
14     // Yöntem (Method) isimleri PascalCase olmalıdır.
15     public void YaşınıSöyle()
16     {
17         Console.WriteLine($"Yaşım:{yaş}");
18     }
19     public void CinsiyetiniSöyle()
20     {
21         switch (char.ToUpper(cinsiyet)) // Küçük/büyük harf karmaşasını tek satırda çözer.
22         {
23             case 'E':
24                 Console.WriteLine("Cinsiyetim: ERKEK");
25                 break;
26             case 'K':
27                 Console.WriteLine("Cinsiyetim: KADIN");
28                 break;
29             default:
30                 Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
31                 break;
32         }
33     }
34 }
```

Programı şöyle açıklayabiliriz;

- ◇ 3. Satırda Kullanıcı Tanımlı **Öğrenci** sınıfından **ali** kimlikli bir **nesne imal edilmiştir (instantiation)**. **ali** değişkeni referans tip olup kendisi yığın (**stack**) bellekte yer alır. Referansını tuttuğu nesneye öbek (**heap**) bellekte yer ayrılmıştır.

- ◇ 4. Satırda `ali` nesnesinin `yaş` durumu yani verisi `40` olarak değiştirildi. Bu durum değişimi `ali` nesnesinin davranışlarını etkiler!
- ◇ 5. Satırda `ali` nesnesine `YaşınıSöyle()` mesajı verildi (**message-passing**). Böylece `ali` nesnesi durumundan veri olarak davranışını konsola `Yaşım:40` yazarak sergiliyor.
- ◇ 6. Satırda `ali` nesnesine `CinsiyetiniSöyle()` mesajı verildi. Böylece `ali` nesnesi durumundan veri olarak davranışını konsola `Cinsiyetimi Söylemek İstemiyorum!` yazarak sergiliyor. Çünkü `ali` nesnesi imal edilirken cinsiyetine varsayılan olarak bir değer atanmamıştır!
- ◇ 8. ve 24. Satırlar arasında **kullanıcı tanımlı Öğrenci** sınıfı tanımlanmıştır.
 - 8. Satırda kullanıcı tanımlı **sınıf kimliği Öğrenci** olarak belirlenmiştir. Sınıflar kod bloğu olmadan tanımlanamaz. 9. Satırda kod bloğu açılmış, 30. Satırda kod bloğu kapatılmıştır. Sınıf üyeleri bu blok içinde tanımlanmıştır.
 - 11. Satırda sınıftan imal edilecek nesnelerin yaşına ait durumları tutacak **yaş** kimlikli **alan değişkeni** (**field variable**) tanımlanmıştır. Başındaki **public** sıfatı, imal edilecek nesnelerin bu verisinin umuma açık olacağını belirtir.
 - Benzer şekilde 12. Satırda sınıftan imal edilecek nesnelerin cinsiyetine ait durumları tutacak **cinsiyet** alan değişkeni tanımlanmıştır.
 - 14. ve 18. Satırlar arasında imal edilecek nesnelerin `YaşınıSöyle()` kimlikli **yöntem** (**method**) tanımlanmıştır. Başındaki **public** sıfatı, imal edilecek nesnelerin bu verisinin umuma açık olacağını belirtir.
 - Benzer şekilde 19. ve 28. Satırlar arasında imal edilecek nesnelerin `CinsiyetiniSöyle()` kimlikli yöntem tanımlanmıştır.

- ◇ Sınıftan nesne imal etme işlemine **örnekleme** (**instantiation**) adı verilir.
- ◇ Her bir nesne, sınıfın bir örneğidir (**instance**).
- ◇ Örnek üyelerine programın 4., 5. ve 6. satırlarında üzere **nokta işleci** (**dot operator**) (`.`) ile erişilir.

8.3 Erişim Değiştiricileri

Sınıf üyelerinin (alan değişkeni veya yöntem) başka nesne ve sınıfların nasıl erişeceğini **erişim değiştiricileri** (**access modifier**) belirler. **Görünürlük** (**visibility**) sıfatları olarak da adlandırılan bu değiştiriciler, sınıf üyelerine sınıf içinden ve dışından erişimi belirleyen sıfatlardır.

- ◇ **public**: Sınıf üyesine, sınıf dışından her zaman erişilebilir. **Umuma açık** (**public**) davranış ve özellikleri için bu değiştirici kullanılır.
- ◇ **private**: Sınıf üyesine, sınıf dışından erişime hiçbir zaman izin verilmez. **Mahrem/şahsi/kişisel** davranış ve özellikleri için bu değiştirici kullanılır. Varsayılan (**default**) erişim değiştiricidir. Bir sınıfta hiçbir erişim değiştirici yoksa tanımlanan durum ve davranışlar **mahrem** (**private**) olarak kabul edilir.
- ◇ **protected**: Dış dünyaya kapalı (**private**), ama mirasçılara açık sınıf üyeleri için **korumalı** (**protected**) bu değiştirici kullanılır. Miras alma başlığında anlatılacaktır.
- ◇ **internal**: Sınıf üyesine, içinde bulunduğu proje ile kısıtlı olacak şekilde sınıf dışından her zaman erişilebilir.
- ◇ **file**: Sınıf üyesine, içinde bulunduğu kaynak dosyası ile kısıtlı olacak şekilde sınıf dışından her zaman erişilebilir.

Şimdiye kadarki örnekler de **tek bir konsol projesi** (**project**) üzerinden örnekler verilmiştir. Bazı durumlarda geliştireceğimiz uygulama **birden fazla projenin olduğu bir çözüm** (**solution**) olabilir. Bu durumda **protected internal**, **private protected** gibi erişim belirleyiciler de ihtiyaca göre kullanılabilir. Aşağıda tüm erişim değiştiricilerin bir tablosu verilmiştir. Yeri geldikçe bunlara örnek verilecektir.

Aşağıda erişim belirleyicilerin uygulandığı basit bir **Kişi** sınıfı örnek verilmiştir. Bu sınıftan imal edilen nesnelerin `yaş` durumu diğer tüm nesneler tarafından erişilip değiştirilebilir, ancak `sır` durumuna ise hiçbir nesne tarafından erişilemez. `maas` durumuna ise **Kişi** sınıfı ve bu sınıftan türemiş sınıflardan imal edilen nesneler tarafından bu özelliğe erişilebilir. İleride göreceğiz.

```
//ANA PROGRAM:
Kişi ahmet = new Kişi();
ahmet.adı = "Ahmet";
ahmet.AdınıSöyle();
//ali._sır="Benim Sırrım!"; /* Alan değiştirilemez! Çünkü mahrem (private) olarak tanımlanmış. */
ahmet.SırDeğiştir(); //Nesne, kendi içinde sırrını değiştirebilir.
```


Erişim Değiştiricisi	Sınıf İçi	Aynı Proje	Miras Alan	Dış Proje
public	✓	✓	✓	✓
private	✓	×	×	×
internal	✓	✓	✓*	×
protected	✓	×	✓	✓*
protected internal	✓	✓	✓	✓*
private protected	✓	✓*	✓*	×

* Not: Miras alma durumunda, belirleyicinin proje kısıtlamaları geçerliliğini korur.

Tablo 8.1: Erişim Belirleyicileri ve Erişim Sınırları

```
//SINIF:
class Kişi // "Kişi" Kimlikli Sınıf
{
    // Alanlar (Fields)
    public string adı;
    private string _sır; //mahrem(private) alanlar önünde altçizgi karakteri olacak şekilde küçük
    ↪ harfle kimliklendirilir
    // Yöntemler (Methods)
    public void SırDeğiştir()
    {
        _sır = "Gizli bir şey öğrendim!";
    }
    public void AdınıSöyle()
    {
        Console.WriteLine($"Adım:{adı}");
    }
}
```

8.4 Yöntemler

Yapısal Programlama mantığında çözülecek problem genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı 2.7 başlığında anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız.

Nesne yönelimli programlamada ise birbiriyle ilgili nesnelere ilişkin sınıflar bir araya toplanarak bir sınıf kütüphanesinde tutulur. Bu durum 5.1 başlığı altında anlatılmıştı.

Bir bilgisayar programı, genel olarak, tek başına tüm görevleri yerine getiren tek bir fonksiyonundan oluşmaz. Bu durumda ana problemin büyüklüğüne göre bu fonksiyon bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani **böl ve yönet (divide and conquer)** tekniği uygulanır. Çünkü **büyük bir problemin toptan çözülmesi**, problemin **parçalar halinde çözülmesinden her zaman daha yorucu ve zordur**. Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna **yukarıdan aşağıya (top down)** yaklaşım adı verilir.

Nesnelerin davranışlarını yöntemlerle tanımlarız. Yöntemler de yapısal programlamadaki fonksiyonlara karşılık gelir;

- Yöntemler, nesnenin belli bir davranışını gerçekleştiren bir kod bloğudur. Bu nedenle, **sınıf (class)** ve ileride göreceğimiz **yapı (struct)** ve **arayüz (interface)** içinde tanımlanır.
- Yöntemler parametrelili alabilir, girdilerle veya nesnenin durumlarıyla parametresiz olarak bir şeyler yapar ve ardından cevabını verebilir. Nesnede olmayan bilgiyi yöntemlere argümanlar ile aktarırız.
- Sınıf içinde her yöntemin bir kimliği vardır. Kimliklendirme **PascalCase** olarak (sözcüklerin baş harfler büyük olacak şekilde bitişik yazılarak) yapılır. Kimliklendirme kuralları için 4.2.5 başlığındaki değişken kimliklendirme kurallarına uyulur.
- Bir yöntem, çağıran koda bir değer döndürebilir.
- Eğer birden fazla davranış içeriyorsa hepsi ayrı tanımlanmalıdır.
- Bir fonksiyon, bir başka fonksiyondan çağrılabilir. Bir başka deyişle yöntemler tekrar kullanılabilir kodlardır.

§8.4.1 Yöntem Tanımı Nasıl Yapılır

Yöntem tanımı (method definition), yöntemin işleyip **döndüreceği veri tipini**, **kimliği**, işleyeceği bağımsız değişkenleri temsil eden **parametre listesini**, içerecek şekilde **başlığının** ve **gövdesinin** kodlanmasından oluşur.

```
class Sınıf {
    Erişim-Değiştirici Seçimlik-Erişim-Değiştirici Dönüş-tipi YöntemKimliği ( //Parametre Listesi:
        [veritipi parametre1]
        [,veritipi parametre2]
        [,veritipi parametre3]...
    )
    { //Yöntem Blok Başlangıcı
        /* Yöntem Gövdesi ... */
        [return dönüş-tipinde-değer;]
        /* Yöntem Gövdesi ... */
    } //Yöntem Bloğu Bitişi
}
```

Sözde kodu şu şekilde açıklayabiliriz;

- ◊ Alan değişkenlerinde olduğu gibi yöntemlerinde başında erişim değiştiricileri bulunur.
- ◊ Seçimlik erişim değiştiricileri olan **static**, **override**, **virtual**, **abstract**, **new** ve **sealed** olabilir. İleride anlatılacaktır.
- ◊ **Yöntemin kimliği benzersizdir**. Aynı zamanda **yöntem adı (method name)** olarak adlandırılır. Yöntem kimliklendirmede, 4.2.5 başlığındaki değişken kimliklendirme kuralları geçerlidir.
- ◊ Yöntem kimliğinden sonra **parametreleri barındıracak açıp kapatılan parantez** olmalıdır. Bu durum değişken tanımından çağrılabilir yöntem tanımlarını ayırmak içindir.
- ◊ Bağımsız değişkenler, parametre olarak adlandırılır;
 - **Parametre listesi**, bir fonksiyonun parametrelerinin veri tipini, sırasını ve sayısını belirtir.
 - Parametreler isteğe bağlıdır, yani bir yöntem hiçbir parametre içermeyebilir.
 - **Her bir parametreye kimlik verilmelidir**. Çünkü yöntem gövdesinde bu değişkenler işlenir.
 - Parametreler birbirinden **virgül işleci (comma operator) (,)** ile ayrılır.
- ◊ Parametreler yöntem **gövdesinde** yapılacak işlere temel oluşturur;
 - Gövde, yöntemin ne yaptığını tanımlayan ve süslü parantezler arasında yer alan talimatları içerir.
 - Gövdede, **yapısal programlama yapıldığından**, ilk önce olacaksa yerel değişkenler tanımlanmalı daha sonra parametrelerle birlikte bu değişkenleri işleyecek talimatlar yazılmalıdır.
 - **Yerel değişkenler (local variable)**, yöntem gövdesi içerisinde tanımlanan değişkenlerdir. Yöntemden çıkılınca kaybolurlar.
 - Gövdede, yöntemde istenilen sonuca ulaşmak için kullanacağı adımlara karar verilir yani **algoritması belirlenir**.
- ◊ İşlenen parametrelerden, alan değişkenlerinden ve gövde içerisinde tanımlanan değişkenlerden bir **geri dönüş değeri** hesaplanabilir. Değer hesaplanmış ise **return geri-dönüş-değeri**; talimatıyla yöntemin icrası sonlandırılır. `textttreturn` Talimatı çoğunlukla fonksiyonundaki son talimat olarak görünür ama yöntemin gövdesinde herhangi bir yerden geri dönülebilir.
- ◊ **return** Talimatıyla geri döndürülen değer tipi, **yöntemin geri dönüş tipidir**.
- ◊ C# programlama dilinde **hiçbir değeri olmayan ve bellekte yer kaplamayan void** anahtar kelimesi vardır. **void**, bir yöntemin değer döndürmediğini belirtir. Yani yöntemin **işini yaptığını** ancak **çağırılan yere herhangi bir veri teslim etmediğini** ifade eder. Bu durumlarda fonksiyon, geri değer döndürmeyebilir;
 - Bu durumda yöntemin **geri dönüş tipi yoktur** ve bu nedenle geri dönüş tipi **void** olarak belirtilir.
 - Geri dönüş tipi **void** olan yöntemin icrası **return**; talimatıyla sonlandırılır.
 - Eğer **return**; talimatı hiç yazılmaz ise süslü parantez kapatılıncaya kadar fonksiyon icra edilir ve fonksiyondan geri dönülür.

Bir de yöntemin çağrıldığı yerde, **yöntemin döndürdüğü değer**in **tipi ile eşleşen bir değişkene dönüş değeri atanabilir**. Zorunlu değildir.

Ayrıca bir yöntem gövdesinde bir başka yöntem tanımlanamaz!

C# dilinde **void** kelimesinin kullanımı, C++ dilindeki **void** mantığıyla tamamen aynıdır. Ancak C#'ta **void** kullanımı daha güvenlidir (**managed code**).

C# dilinde yöntemler, yapısal programlama dilindeki fonksiyonlar gibi tanımlanır. 2.7 başlığında anlatılan yapısal programlamada veri yapıları (**data structure**) ve kontrol yapıları (**control structure**) birbirinden ayrıdır. Yapısal programlamadaki gibi yöntemler kodlanırken, önce veriyi taşıyan veri yapıları tanımlanmalı daha sonra veriyi işleyecek kontrol yapıları kullanılmalıdır.

§8.4.2 Nesneye Mesaj Verme

Bir nesnenin davranış göstermesini istiyorsak ona mesaj göndeririz (**message-passing**). Bunun yapısal programlamadan gelen adı **çağrıdır** (**call/invoke**).

Aşağıdaki örnekte, **Hesap** sınıfının **Topla()** ve **ÖrnekHesap()** yöntemlerini inceleyebilirsiniz. Bu sınıftan imal edilen **hesap** nesnesine davranış göstermesi için çeşitli mesajlar verilmiştir;

```
//ANA PROGRAM:
Hesap hesap = new Hesap();
Console.WriteLine(hesap.Topla(20, 30)); /* hesap nesnesine Topla mesajı verildi. Topla yönteminin
→ geri döndürdüğü değer konsola yazdırıldı. */
hesap.ÖrnekHesap(); /* hesap nesnesine ÖrnekHesap mesajı verildi. ÖrnekHesap yöntemi geri değer
→ döndürmüyor! */
Console.WriteLine(hesap.KatsayıylaÇarp(11)); /* hesap nesnesine KatsayıylaÇarp mesajı verildi.
→ KatsayıylaÇarp yönteminin geri döndürdüğü değer konsola yazdırıldı. */

//SINIF
class Hesap // Hesap Sınıfı
{
    private int _çarpan;
    public Hesap()
    {
        this._çarpan=10;
    }
    public int Topla(int p1, int p2)
    {
        return p1 + p2;
    }
    public void ÖrnekHesap()
    {
        Topla(1, 2); /* Topla yöntemi, 1 ve 2 argümanlarıyla çağrılıyor. Ama geri döndürdüğü
        → değer kullanılmıyor */

        Console.WriteLine(Topla(3, 4)); /* 3 ve 4 argümanlarıyla çağrılan Topla yönteminin geri
        → döndürdüğü değer konsola yazdırıldı */

        int toplam; //toplam yerel değişkeni tanımlandı
        toplam = Topla(5, 6); /* 5 ve 6 argümanlarıyla çağrılan Topla yönteminin geri döndürdüğü
        → değer toplam değişkenine artandı */
        Console.WriteLine(toplam);

        Console.WriteLine(Topla(7, 8) + Topla(9, 10)); /* Topla yönteminin geri döndürdüğü değer
        → ile bir başka Topla yönteminin geri döndürdüğü değer toplanarak konsola yazdırıldı */

        Console.WriteLine(Topla(10, Topla(11, 12))); /* Topla yönteminin argüman olarak bir başka
        → Topla yönteminin geri döndürdüğü değer geçirildi ve konsola yazdırıldı */
    }
    public int KatsayıylaÇarp(int p1)
    {
        return this._çarpan*p1; // alan değişkeni yöntemde kullanılıyor.
    }
}
```

§8.4.3 Yöntemlere Değerle Parametre Geçirme

Bir yönteme değer tip parametre olarak geçirildiğinde çağrı sırasında bellekteki değişim aşağıdaki programdan anlaşılabilir;

```
//Ana program değişkenleri:
```

```
int a = 10;
int b = 20;
int toplam = 0;
Hesap hesap = new Hesap();
```

```
//ÇAĞRI ORTAMI:Ana program nesnenin yöntemlerini çağırarak.
```

```
Console.WriteLine($"Çağrı öncesi a:{a}, b:{b}"); //çağrı öncesi a:10, b:20
```

```
toplam = hesap.Topla(a, b); //Değer tiplerin yöntemle geçirilmesi
```

```
Console.WriteLine($"Çağrı sonrası a:{a}, b:{b}, toplam:{toplam}"); //çağrı sonrası a:10, b:20,
→ toplam:30
```

```
//SINIF:
```

```
public class Hesap
```

```
{
```

```
    public int Topla(int x, int y)
```

```
    {
```

```
        int temp = x + y;
```

```
        x = 100; y = 200; // Parametre değişkenleri değiştiriliyor!
```

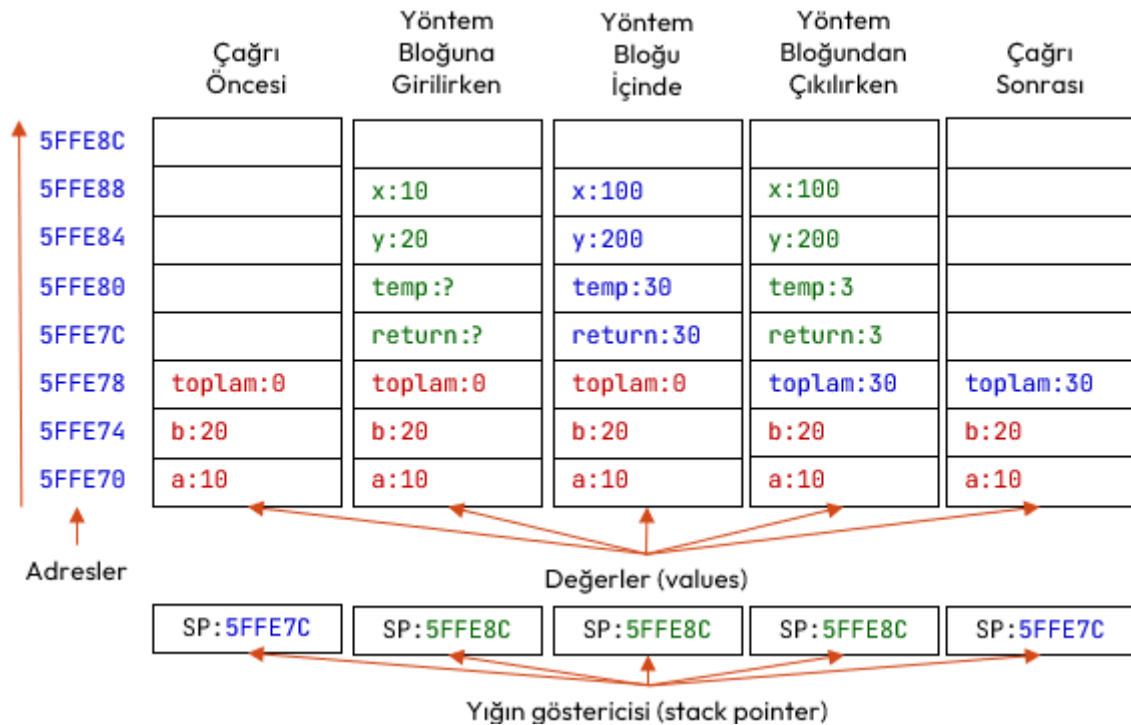
```
        Console.WriteLine($"Çağrı içinde x:{x}, y:{y}"); //çağrı içinde x:100, y:200
```

```
        return temp;
```

```
    }
```

```
}
```

Yukarıda örneği verilen programda `hesap` nesnesinin `Topla()` yönteminin çağırma sürecindeki yığın bellek bölgesindeki (stack segment) değişimi Şekil 8.1’de gösterilmektedir.



Şekil 8.1: Yöntem Çağrısı Sırasında Yığın Bellek

Yukarıdaki kod şu şekilde açıklanabilir;

- İcra sırası `hesap.Topla(a,b)` talimatına geldiğinde yöntem çağrılmadan önce ana programda yerel değişkenler `a`, `b` ve `toplam` yığın belleğe (stack segment) itilmiştir (push).
- Sonra `hesap.Topla(a,b)` yöntemi çağrıldığı anda `a` ve `b` değişkenlerinin değerleri, `x` ve `y` parametrelerine kopyalanarak fonksiyona geçirilir. Değerler parametrelere kopyalandığından buna **değerle çağırma** (call by value) adı verilir.
- `hesap.Topla(a,b)` yöntemi icra edilirken, yereldeki `temp` ve parametre değişkenleri `x`, ve `y` istenildiği gibi değiştirilebilir. Geri dönüş değeri de belirlenir.
- `hesap.Topla(a,b)` yöntemi içine bir başka yöntem de çağrılabilir. Böyle bir durumda `x`, `y` ve `temp` değerleri yığın belleğe itilir ve çağrılan yöntem icra edilir. Böylece iç içe yapılan her çağrıda yığın

belleğin büyüyeceği görülmektedir. Program derlendiğinde yığın bellek (**stack segment**) için sınırlı bir bellek bölgesi ayrılır. İç içe yapılan çağrılarda bu bellek sınırı aşılabılır. Bu durumda program, **yığın taşması (stack overflow)** hatası verecektir.

- ◊ `hesap.Toplam=topla(a,b)` fonksiyon bloğundan çıkılırken geri dönüş değeri çağrı ortamındaki `toplama` değişkenine kopyalanır ve ana programda yığın belleğe itilmiş değerler geri çekilir (**pop**).

§8.4.4 Yöntemlere Referansla Parametre Geçirme

Referans değişkenler, değerin kendisine değil bellek konumuna bir referanstır. Referans değişkenlerin işaret ettiği adresler yığın (stack) bellekte tutulur. Bir yonteme referans veri tipleri ya da bir **referansı parametre olarak geçirdiğinizde (pass by reference)**, referansların işaret ettiği adreslerin kopyaları parametre olarak kopyalanır. Parametrelerin değerleri için yeni bir depolama konumu oluşturulmaz. Böylece orijinal değişkenleri değiştirme imkânımız olur. Ayrıca hafızada çok yer kaplayan parametreler için çok kullanışlıdır.

Değer tipindeki parametreleri yöntem içerisinde değiştirmek istiyorsak önünde **ref** sıfatını (**ref keyword**) kullanmalıyız.

```
Hesap işlemci = new Hesap();
int a = 10;
int b = -25;
Console.WriteLine($"Çağrı öncesi a:{a}, b:{b}"); // a:10 b:-25
işlemci.DeğerDeğiştir(ref a, ref b);
Console.WriteLine($"Çağrı sonrası a:{a}, b:{b}"); // a:-25 b:10
```

```
class Hesap
{
    public void DeğerDeğiştir(ref int x, ref int y)
    {
        int temp= x;
        x = y;
        y = temp;
    }
}
```

§8.4.5 Yöntem Parametrelerini Çıktı Olarak Kullanma

Bir yöntemin içinden bir değeri dışarı çıkarmak için **return** talimatı kullanılır. Ancak bu talimat bir yöntemden yalnızca bir değer döndürmek için kullanılabilir.

Bunun yanında **out** parametrelerini kullanarak bir fonksiyondan birden fazla değer döndürebilirsiniz. **out** Parametreleri, referans parametrelerine benzerdir, ancak veriyi yonteme aktarmak yerine yöntemden dışarı aktarırlar.

```
Hesap hesap = new Hesap();
int okunan, okunanınKaresi;
okunan = hesap.birSayıOku(out okunanınKaresi);
Console.WriteLine($"Okunan: {okunan}, karesi:{okunanınKaresi}");
```

```
class Hesap
{
    public int birSayıOku(out int kare)
    {
        int temp;
        Console.WriteLine("Bir sayı giriniz: ");
        temp = Convert.ToInt32(Console.ReadLine());
        kare = temp * temp;
        return temp;
    }
}

/*Program Çıktısı:
Bir sayı giriniz:
12
Okunan: 12, karesi:144
*/
```

§8.4.6 Varsayılan Argüman

Argümanlar, yöntemin çağrıldığı anda yönteme geçirilen gerçek parametrelerdir. Aşağıda verilen **Topla** yönteminin iki farklı **int** parametresi aldığı görülmektedir. C#4.0+ ile gelen özellikle istenirse bu parametrelere varsayılan bir değer atanabilir. Yani yöntemler **varsayılan argümanlar** (**default argument**) ile tanımlanabilir.

```
Hesap işlem = new Hesap();
int sonuç= işlem.Topla(10); // 10 + 1 = 11
Console.WriteLine($"Sonuç: {sonuç}"); //11

class Hesap
{
    public int Topla(int p1, int p2 = 1) //İkinci argüman varsayılanı 1
    {
        return p1 + p2;
    }
}
```

§8.4.7 Özyinelemeli Yöntemler

Özyineleme (**recursion**), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, yineleme (**iteration**), döngüler (**loop**) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. Özyinelemeli algoritmalarda, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır;

1. Cevabı kesin olarak bildiğimiz temel durum (**base case**)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n sayısının faktöriyeli, $n!$ ile gösterilir ve n sayısına eşit ve küçük olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 3 \times 2 \times 1$$

Sıfır sayısı da pozitif bir sayıdır ama sonucun sıfır çıkmaması için **sıfır sayısının faktöriyeli 1 kabul edilir**. Bu aslında faktöriyel problemi için **temel durumdur** (**base case**) ve **boş ürün** (**empty product**) kuralı olarak bilinir. Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
Fakyöriyel f = new Fakyöriyel();
Console.WriteLine(f.FaktöriyelHesapla(5)); // 120
```

```
class Fakyöriyel
{
    public uint FaktöriyelHesapla(uint n)
    {
        uint sonuc = 1, sayaç;
        for (sayaç = n; sayaç > 0; sayaç--)
            sonuc = sonuc * sayaç;
        return sonuc;
    }
}
```

Aslında negatif olmayan bir tam sayı olan sayının faktöriyeli, kendisi ile kendisinden bir küçük olan sayının faktöriyeli ile çarpımı şeklinde yazılabilir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \times (n - 1)!$$

Bu durumda faktöriyel fonksiyonu aşağıdaki şekilde **özyinelemeli** (**recursive**) tanımlanabilir;

$$n! = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot (n - 1)! & \text{eğer } n > 0 \end{cases}$$

Aynı faktöriyel fonksiyonu **parametrelili** olarak aşağıdaki şekilde yine özyinelemeli olarak tanımlanabilir;

$$f(n) = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot f(n - 1) & \text{eğer } n > 0 \end{cases}$$

Buna örnek olarak aşağıdaki hesabı verebiliriz;

$$5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! = 5 \times 4 \times 3 \times 2 \times 1 \times 0! = 120$$

Özyinelemeli (**recursive**) olarak ise faktöriyel yöntemi aşağıdaki şekilde kodlanabilir;

```
public uint Faktöriyel(uint n)
{
    return n == 0 ? 1 : n * Faktöriyel(n - 1);
}
```

Burada iç içe çağrılan her yöntemde **yığın (stack)** belleğin şiştiği unutulmamalıdır!

Bir başka örnek olarak, özyinelemeli Fibonacci sayısı hesaplama örneği verilebilir;

```
public int fibonacci(int i)
{
    if(i <= 2) return 1;
    return fibonacci(i - 2) + fib(i - 1);
}
```

Yine bir başka örnek, özyinelemeli olarak bir tam sayının kuvveti aşağıdaki gibi hesaplanabilir;

```
public int kuvvet(int taban, int üs)
{
    if (taban == 0) return 0;
    if (üs == 0) return 1;
    return taban * kuvvet(taban, üs - 1);
}
```

§8.4.8 Yöntem Parametresi Olarak Diziler

Yöntemlere parametre olarak gönderilen dizinin boyutu, köşeli parantez içindeki **virgül işlecinden (comma operator)** anlaşılır.

Aşağıda bir diziyi okuyan, ekrana yazan ve eleman ortalamalarını hesaplayan fonksiyonlara sahip bir program verilmiştir;

```
int[] dizi = { 1, 2, 3, 4, 5 };
int[,] matris = new int[4, 4]{ { 1, 2, 3, 4},
                               { 5, 6, 7, 8},
                               { 9, 10, 11, 12},
                               {13, 14, 15, 16} };

Diziİşlemleri işlemler = new Diziİşlemleri();
işlemler.diziyaz(dizi);
/*
Dizi:
1  2  3  4  5
*/
Console.WriteLine("Ortalama: {0}", işlemler.ortalama(dizi)); // Ortalama: 3
işlemler.matrisyaz(matris);
/*
Matris:
1  2  3  4
5  6  7  8
9  10 11 12
13 14 15 16
*/
işlemler.transpose(matris);
işlemler.matrisyaz(matris);
/*
Matris:
1  5  9 13
2  6 10 14
3  7 11 15
4  8 12 16
*/

class Diziİşlemleri
```

```

{
    public float ortalama(int[] dizi)
    {
        return (float)dizi.Average();
    }
    public void diziyaz(int[] dizi)
    {
        Console.WriteLine("Dizi: ");
        for (int i = 0; i < dizi.Length; i++)
            Console.Write($"{dizi[i],4}");
        Console.WriteLine();
    }
    public void transpose(int[,] dizi)
    {
        for (int i = 0; i < dizi.GetLength(0); i++) {
            for (int j = i; j < dizi.GetLength(1); j++) {
                int temp = dizi[i, j];
                dizi[i, j] = dizi[j, i];
                dizi[j, i] = temp;
            }
        }
    }
    public void matrisyaz(int[,] dizi)
    {
        Console.WriteLine("Matris:");
        for (int i = 0; i < dizi.GetLength(0); i++){
            for (int j = 0; j < dizi.GetLength(1); j++){
                Console.Write($"{dizi[i, j],4}");
            }
            Console.WriteLine();
        }
        Console.WriteLine();
    }
}

```

§8.4.9 Yöntemlerin Aşırı Yüklenmesi

Yöntemlerin aşırı yüklenmesi (method overloading), aynı isimdeki bir metodun farklı parametre listeleriyle birden fazla kez tanımlanabilmesidir. C# derleyicisi, hangi yöntemin çağrılacağına karar verirken **yöntem imzasına (method signature)** bakar. Bu özellik, programcının benzer işleri yapan metotlar için farklı isimler türetme zahmetini ortadan kaldırır. Bir **yöntemin imzası** şunlardan oluşur:

1. Metodun ismi.
2. Parametrelerin sayısı.
3. Parametrelerin veri tipleri.
4. Parametrelerin sırası.
5. Metodun geri dönüş tipi (`int`, `void` vb.) ve erişim belirleyicileri (`public`, `private`) metodun imzasının bir parçası değildir. Sadece geri dönüş tipi farklı olan iki metot aşırı yüklenmiş sayılmaz ve derleme hatası verir!

Aşağıda `Topla()` yönteminin aşırı yüklenmesine ilişkin örnek verilmiştir;

```

public class Hesaplayici
{
    // 1. İki tamsayıyı toplar
    public int Topla(int sayi1, int sayi2)
    {
        return sayi1 + sayi2;
    }
    // 2. Üç tamsayıyı toplar (Parametre sayısı farklı)
    public int Topla(int sayi1, int sayi2, int sayi3)
    {
        return sayi1 + sayi2 + sayi3;
    }
    // 3. İki ondalıklı sayıyı toplar (Parametre tipi farklı)
    public double Topla(double sayi1, double sayi2)

```



```

{
    return sayi1 + sayi2;
}
}

```

Yukarıdaki kod incelendiğinde;

- Okunabilirlik: `ToplaInt`, `ToplaDouble`, `ToplaUcSayi` gibi karmaşık isimler yerine tek bir `Topla` ismiyle temiz bir kod yapısı sağlar.
- Esneklik: Kullanıcıya aynı işlemi farklı veri tipleriyle yapma özgürlüğü tanır.
- Kod Tamamlama Desteği: Visual Studio gibi editörlerde metod ismini yazdığınızda kod tamamlama (*intellisense*) aracı, aşırı yüklenmiş tüm versiyonlar "1 of 3" şeklinde listelenerek yazılımcıya rehberlik eder.

Aşırı yükleme yaparken `ref` ve `out` anahtar kelimeleri de imzanın bir parçası kabul edilir. Ancak bir metodu hem normal parametrelili hem de aynı tipte `ref` parametrelili olarak tanımlamak belirsizliğe yol açacağı için izin verilmez.

```

// Bu iki kullanım birlikte aşırı yükleme OLABİLİR:
public void Islem(int x) { }
public void Islem(string x) { }

// Ancak bu ikisi AYNI ANDA bulunamaz:
// public void Islem(int x) { }
// public void Islem(ref int x) { } // Hata!

```

§8.4.10 Çağrı Kuralı

Çağrı kuralı (*calling convention*), bir fonksiyon çağrısıyla karşılaşıldığında argümanların fonksiyona hangi sıraya göre iletileceğini belirtir;

- Birincisinde argümanlar soldan başlanarak sağa doğru fonksiyona geçirilir. C# dilinde bu kullanılır.

```

Sayiİşlemleri işlem = new Sayiİşlemleri();
int sonuç = işlem.topla(10,20,30); //p1:10,p2:20,p3:30
Console.WriteLine($"Sonuç: {sonuç}"); // Sonuç: 60
int a = 1;
int toplam = işlem.topla(a, ++a, a++);
/* Beklenen Sonuç: 1+2+2=5 Parametreler: p1:1,p2:2,p3:2 */
Console.WriteLine($"Sonuç: {toplam}"); // Sonuç: 5
Console.WriteLine($"a'nın son değeri: {a}"); // a'nın son değeri: 3

```

```

class Sayiİşlemleri
{
    public int topla(int p1, int p2, int p3)
    {
        Console.WriteLine($"p1:{p1},p2:{p2},p3:{p3}");
        return p1 + p2+ p3;
    }
}

```

Yukarıdaki örnek incelendiğinde `sonuç1` ve `sonuc2` argümanların hangi sırada fonksiyona geçirildiğinin bir önemi yoktur.

- İkincisinde ise C/C++ dilinde kullanılır ve argümanlar sağdan başlanarak sola doğru fonksiyona geçirilir.

```

#include <iostream>
using namespace std;
int topla(int, int, int);
int main () {
    int a = 1, toplam;
    toplam = topla(a, ++a, a++);
    cout << "toplam:" << toplam << endl;
}
int topla(int x, int y, int z) {
    cout << "x:"<< x << " y:" << y << " z:" << z<< endl;
    return x+y+z;
}
/* Program çalıştırıldığında:

```

```
x:3 y:3 z:1
toplam:7
*/
```

Bu kodun çıktısı 1 2 3 olarak beklenir ancak çağrı kuralından ötürü çıktı 3 3 1 olur. Bunun nedeni C++ dilinin çağrı kuralının sağdan sola olmasıdır. Bunu şöyle açıklayabiliriz;

- ◊ Fonksiyona sağdan ilk argüman olarak **a** değişkeni değeri 1 geçirilir.
- ◊ Ardından **a++** ifadesi ile **a** değişkeni 2'ye yükseltilir.
- ◊ Sonra **++a** ifadesi ile **a** değişkeni 3'e yükseltilir.
- ◊ Fonksiyona ikinci argüman olarak 3 geçirilir.
- ◊ Fonksiyona son olarak **a**'nın son değeri olan 3 geçirilir.

Çağrı kuralı, C++ dillerinde yazılmış bir kütüphanedeki fonksiyonları projemizde kullanmak istediğimizde önem kazanır¹.

8.5 Nesne Yapıcısı

Bir sınıftan birçok nesne de imal edilebileceği anlatılmıştı. İmal edilen her nesne (**object**) sınıfın bir örneğidir (**instance**) ve bu nesnelerin varsayılan (**default**) olarak farklı verilerle yani durumlarla (**state**) imal edilmesi gerekiyorsa **nesne yapıcısı** (**constructor**) kullanılır. Nesnelere ait veriler, alanlarda (**field**) tutulurlar. Nesnenin durumları (**state**), nesnenin davranışını (**behavior**) etkiler! Bu nedenle daha nesneler imal edilirken durumlarının belirlenen değerde olması istenir.

§8.5.1 Yapıcı Tanımlama

Yapıcılar tanımlanırken aşağıdaki kurallar uygulanır;

- ◊ **Yapıcıların kimliği, sınıf kimliğiyle aynıdır.**
- ◊ Yapıcıların görevi sınıftan nesneleri, varsayılan durumlara sahip olarak imal etmektir. Kısaca görevi, alan değişkenlerine ilk değer vermektir.
- ◊ Yapıcıların parametre listesi olabilir. Alan değişkenlerini atayan kod blokları vardır.
- ◊ Yapıcılar, yöntemlerin aksine **void** dahil **geri değer döndürmezler.**
- ◊ Bu nedenle bir yapıcı içerisinde **return** talimatı da kullanılmaz.

Aşağıda verilen örnekte **Öğrenci** sınıfından imal edilecek her nesnenin, yaşı 18 ve cinsiyeti 'E' olacaktır. **Hiçbir parametresi olmayan** bu yapıcıya **varsayılan yapıcı** (**default constructor**) adı verilir. Yapıcılar, **new** işleci ile nesne imal edilirken, nesnenin verilerine/durumlarına ilk değer vermek için çağrılır. Yani nesneler **başlatılırken** (**initialization**) çağrılırlar.

```
Öğrenci ilhan = new Öğrenci(); // Öğrenci() yapıcısı kullanılarak nesne imal edildi!
ilhan.YaşınıSöyle();           // Çıktı: Yaşım: 18
ilhan.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: ERKEK
```

```
class Öğrenci
{
    // Alanlar
    public uint yaş;
    public char cinsiyet;
    // 1. Varsayılan (Default) Yapıcı
    public Öğrenci()
    {
        yaş = 18;
        cinsiyet = 'E';
    }
    // Yöntemler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım: {yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(cinsiyet))
        {
```

¹<https://learn.microsoft.com/en-us/dotnet/standard/native-interop/calling-conventions>

```

        case 'E':
            Console.WriteLine("Cinsiyetim: ERKEK");
            break;
        case 'K':
            Console.WriteLine("Cinsiyetim: KADIN");
            break;
        default:
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
            break;
    }
}
}

```

Eğer bir sınıfa hiç yapıcı yazmazsanız, C# derleyicisi **arka planda parametresiz ve boş bir yapıcı oluşturur**. Bunu 8.3 başlığındaki **Kişi** sınıfında görebiliriz. Ancak sınıfta herhangi bir yapıcı yazdığınız anda bu otomatik yapı kaybolur. Bunu da yukarıdaki örnekte görebilirsiniz. İki örnekte de nesne imalatı kısmını inceleyiniz.

§8.5.2 Yapıcı Aşırı Yükleme

Yöntemlerin aşırı yüklenmesinde, aynı kimlikle farklı imzaya (**signature**) sahip birden fazla yöntem tanımlaması yapılabilir. Benzer şekilde bir sınıfta **farklı parametre listelerine sahip** birden fazla yapıcı bulunabilir. Buna **yapıcı aşırı yükleme** (**constructor overloading**) adı verilir. Bu da, nesnenin farklı şekillerde başlatılmasına olanak tanır.

```

Öğrenci ilhan = new Öğrenci(); // Varsayılan Yapıcı İle Nesne İmalatı
ilhan.YaşınıSöyle(); // Çıktı: Yaşım: 18
ilhan.CinsiyetiniSöyle(); // Çıktı: Cinsiyetim: ERKEK
Öğrenci damla = new Öğrenci(20, 'K'); // Parametrelili Yapıcı İle Nesne İmalatı
damla.YaşınıSöyle(); // Çıktı: Yaşım: 20
damla.CinsiyetiniSöyle(); // Çıktı: Cinsiyetim: KADIN

```

```

class Öğrenci
{
    // Alanlar (Fields)
    public uint yaş;
    public char cinsiyet;

    // Yapıcılar (constructors)
    public Öğrenci() // 1. Varsayılan (Default) Yapıcı
    {
        yaş = 18;
        cinsiyet = 'E';
    }
    public Öğrenci(uint pYaş, char pCinsiyet) // 2. Parametrelili Yapıcı
    {
        yaş = pYaş;
        cinsiyet = pCinsiyet;
    }
    public Öğrenci(Öğrenci p) // 3. Parametrelili Kopya Yapıcı (copy constructor)
    {
        yaş = p.yaş;
        cinsiyet = p.cinsiyet;
    }

    // Yöntemler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım: {yaş}");
    }
    public void CinsiyetiniSöyle()
    {

```

```

        switch (char.ToUpper(cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
}

```

Yukarıda parametresi olmayan **varsayılan yapıcıya (default constructor)** ek olarak bir başka **parametrelili yapıcı (parameterized constructor)** tanımlanmıştır. Parametre listesi farklı olduğunda, **yapıcı aşırı yüklenmiştir (overloaded constructor)**.

§8.5.3 this Anahtar Kelimesi

this anahtar kelimesi (**this keyword**), nesne yönelimli programlamanın en temel **öz-referans** aracıdır. En basit tanımıyla **this**, o anda üzerinde çalışılan nesnenin yani örneğin kendisini temsil eder.

İsim karışıklığını giderme (**ambiguity**), en yaygın kullanım alanıdır. Yapıcılarda (**constructor**) veya yöntemlerde, parametre kimlikleri veya yerel değişkenleri sınıfın alan (field) değişkenleri ile aynı olduğunda **this** kelimesi kullanarak hangisinin sınıfa ait olduğunu belirtiriz.

```

Öğrenci damla = new Öğrenci(20, 'K'); // Parametrelili Yapıcı İle Nesne İmalatı
damla.YaşınıSöyle(); // Çıktı: Yaşım: 20
damla.CinsiyetiniSöyle(); // Çıktı: Cinsiyetim: KADIN

```

```

class Öğrenci
{
    // Alanlar (Fields)
    public uint yaş;
    public char cinsiyet;

    // Yapıcılar (constructors)
    public Öğrenci(uint yaş, char cinsiyet)
    {
        this.yaş = yaş;
        this.cinsiyet = cinsiyet;
    }

    // Yöntemler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım: {yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
}

```

}
Bazen bir yöntemin içindeyken, o anki nesneyi başka bir yöntem veya sınıfa parametre olarak göndermeniz gerekir. İşte burada "ben buradayım" demek için `this` kullanılır.

```
public class Veritabani
{
    public void Kaydet(Ogrenci ogr) { /* Kayıt işlemleri */ }
}
public class Ogrenci
{
    public void SistemeEkle()
    {
        Veritabani db = new Veritabani();
        // Bu sınıftan imal edilen nesnenin kendisini gönderiyoruz.
        db.Kaydet(this);
    }
}
```

Bu anahtar kelime sadece bu durumlarda kullanılmaz. Yöntemlerin içinde de kullanılır.

§8.5.4 Yapıcı Zincirleme

Bir yapıcının, aynı sınıf içindeki (veya üst sınıftaki) başka bir yapıcıyı kendi işini yapması için çağırması işlemine **yapıcı zincirleme** (**constructor chaining**) denir.

Bir yapıcı, başka bir yapıcıya **delege edilecekse**, yapıcı gövdesinden önce iki nokta üstüste işleci (**colon operator**) (`:`) ve `this` anahtar kelimesi kullanılır.

```
Ogrenci ilhan = new Ogrenci(); // Varsayılan Yapıcı İle Nesne İmalatı
ilhan.YaşınıSöyle();           // Çıktı: Yaşım: 18
ilhan.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: ERKEK
Ogrenci damla = new Ogrenci(20, 'K'); // Parametrelili Yapıcı İle Nesne İmalatı
damla.YaşınıSöyle();           // Çıktı: Yaşım: 20
damla.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: KADIN
Ogrenci reyhan = new Ogrenci('K'); // Delege Eden Yapıcı İle Nesne İmalatı
reyhan.YaşınıSöyle();           // Çıktı: Yaşım: 30
reyhan.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: KADIN
Ogrenci osman = new Ogrenci(55); // Delege Eden Yapıcı İle Nesne İmalatı
osman.YaşınıSöyle();           // Çıktı: Yaşım: 55
osman.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: ERKEK
```

```
class Ogrenci
{
    // Alanlar (Fields)
    public uint yaş;
    public char cinsiyet;

    // Yapıcılar (constructors)
    public Ogrenci()// 1. Varsayılan (Default) Yapıcı
    {
        yaş = 18;
        cinsiyet = 'E';
    }
    public Ogrenci(uint pYaş, char pCinsiyet)// 2. Parametrelili Yapıcı
    {
        yaş = pYaş;
        cinsiyet = pCinsiyet;
    }
    public Ogrenci(char pCinsiyet) : this(30,pCinsiyet)// 3. Delege Eden Yapıcı
    {
        // Delege Eden Yapıcı: Sadece cinsiyeti alanını ayarlar, yaşı varsayılan olarak 30 kabul
        // eder.
        // İş i Ogrenci(uint pYaş, char pCinsiyet) yapıcısına devreder (delegate).
    }
    public Ogrenci(uint pYaş) : this(pYaş, 'E')// 4. Delege Eden Bir Başka Yapıcı
```

```

{
    // Delege Eden Yapıcı: Sadece yaş alanını ayarlar, cinsiyeti varsayılan olarak 'E' kabul
    // eder.
    // İşi Öğrenci(uint pYaş, char pCinsiyet) yapıcısına devreder (delegate).
}
// Yöntemler
public void YaşınıSöyle()
{
    Console.WriteLine($"Yaşım: {yaş}");
}
public void CinsiyetiniSöyle()
{
    switch (char.ToUpper(cinsiyet))
    {
        case 'E':
            Console.WriteLine("Cinsiyetim: ERKEK");
            break;
        case 'K':
            Console.WriteLine("Cinsiyetim: KADIN");
            break;
        default:
            Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
            break;
    }
}
}

```

§8.5.5 Yapıcılarda Varsayılan Argümanlar

Yöntemlerde olduğu gibi aşırı yüklenmiş yapıcılarda da varsayılan argümanlar kullanılabilir. Bu durumda aşırı yüklemde parametre listesinin farklı olması kuralına uyulmalıdır.

```

Öğrenci ilhan = new Öğrenci(); // pYaş=18, pCinsiyet='E' Varsayılan Argümanı ile Nesne İmalatı
ilhan.YaşınıSöyle();           // Çıktı: Yaşım: 18
ilhan.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: ERKEK
Öğrenci abdullah = new Öğrenci(35); // Cinsiyet='E' Varsayılan Argümanı ile Nesne İmalatı
abdullah.YaşınıSöyle();        // Çıktı: Yaşım: 35
abdullah.CinsiyetiniSöyle();   // Çıktı: Cinsiyetim: ERKEK
Öğrenci damla = new Öğrenci(20, 'K'); // Başka Argümanlarla Nesne İmalatı
damla.YaşınıSöyle();           // Çıktı: Yaşım: 20
damla.CinsiyetiniSöyle();      // Çıktı: Cinsiyetim: KADIN

```

```

class Öğrenci
{
    // Alanlar (Fields)
    public uint yaş;
    public char cinsiyet;

    // Yapıcı (constructor)
    public Öğrenci(uint pYaş=18, char pCinsiyet='E') // Varsayılan Argümanlarla Parametrelili Yapıcı
    {
        yaş = pYaş;
        cinsiyet = pCinsiyet;
    }

    // Yöntemler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım: {yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(cinsiyet))

```

```

{
    case 'E':
        Console.WriteLine("Cinsiyetim: ERKEK");
        break;
    case 'K':
        Console.WriteLine("Cinsiyetim: KADIN");
        break;
    default:
        Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
        break;
}
}
}

```

§8.5.6 Birincil Yapıcı

C#12+ ile gelen **birincil yapıcı** (**primary constructor**), yapıcı parametrelerinin sınıf gövdesinin her yerinde doğrudan kullanılabilmesini sağlar. Bu özellik, özellikle bağımlılık enjeksiyonu (**dependency injection**) kullanılan sınıflarda kod karmaşasını ciddi oranda azaltır.

Geleneksel C# sınıflarında, dışarıdan gelen bir değeri sınıf içindeki özel bir alana (**field**) atamak için hem alanı tanımlamanız hem de yapıcı (**constructor**) içinde bu atamayı yapmanız gerekir. Birincil yapıcı, bu süreci doğrudan sınıf isminin yanına taşıyarak **bürokrasi** (**boilerplate**) oluşturan kod ihtiyacını ortadan kaldırır:

- ◊ **Doğrudan Kapsam:** Parametreler, sınıfın veya yapının tüm gövdesi boyunca doğrudan kullanılabilir.
- ◊ **Bürokrasi Azaltma:** Özel alan tanımlama ve yapıcı içinde `this.x = x` gibi tekrarlayan atama satırlarını yok eder.
- ◊ **Esneklik:** Yalnızca **record** yapılarında değil, artık standart **class** ve **struct** yapılarında da kullanılabilir.

```

// Geleneksel Yöntem
public class KitapServisiOld
{
    private readonly string _baglantiCumlesi;
    public KitapServisiOld(string baglantiCumlesi)
    {
        _baglantiCumlesi = baglantiCumlesi;
    }
}

// Modern Yöntem (Primary Constructor)
public class KitapServisi(string baglantiCumlesi)
{
    public void Baglan()
    {
        Console.WriteLine($"{baglantiCumlesi} kullanılarak bağlanıldı.");
    }
}

```

§8.5.7 required Anahtar Kelimesi

C#11+ ile hayatımıza giren **required** anahtar kelimesi, nesne yönelimli tasarımda "nesne bütünlüğünü" korumak için geliştirilmiş çok kritik bir araçtır. Bu özellik, bir sınıfın veya kaydın (**record**) belirli özelliklerinin nesne oluşturulurken mutlaka atanması gerektiğini derleyici düzeyinde garanti altına alır.

Modern C# yazımında nesne başlatıcıları (**object initializers**), yapıcılara (**constructor**) göre daha okunabilir bir alternatif sunar. Ancak nesne başlatıcılarının en büyük zayıflığı, bazı özelliklerin atanmasının unutulabilmesidir. **required** belirleyicisi (**required specifier**), bu boşluğu doldurarak hem esnekliği hem de veri güvenliğini aynı anda sağlar.

Eskiden bir özelliğin mutlaka doldurulmasını istiyorsak, onu sınıfa bir yapıcı parametresi olarak eklemek zorundaydık. Bu durum, çok fazla özelliği olan sınıflarda devasa ve yönetilmesi zor yapıcılara yol açıyordu. **required**, yapıcı yazma zorunluluğunu ortadan kaldırırken, derleyicinin o özelliğin boş geçilmesine izin vermemesini sağlar.

Bu lanlar internet formlarındaki kırmızı yıldızlı alanlar gibidir. Bilgileri istediğiniz sırayla girebilirsiniz ancak zorunlu bir alanı boş bıraktığınızda sistem (derleyici) işlemi tamamlamanıza izin vermez.

```
public class Personel
```



```
{
    public required int SicilNo { get; init; }
    public required string Müdürlük { get; set; }
}

// Derleme hatası verir: SicilNo eksik
// var p = new Personel { Müdürlük = "AR-GE" };
```

8.6 Soyut, Somut ve Bilgi Gizleme

Soyut (**abstract**) kavramı kelime itibarıyla detay içermeyen, özet anlamına gelir. Soyutlama (**abstraction**), bir nesnenin sadece "ne yaptığını" ortaya koyup, "nasıl yaptığını" gizleme sürecidir. Kullanıcıya sadece gerekli olan temel özellikleri sunar, karmaşık detayları arka planda tutar. Örnek olarak bir **Araba** sınıfı düşünün. Kullanıcı bu sınıftan imal edilmiş bir nesneden **gazaBas()** davranışını göstermesi için ileti gönderebilir. Motorun içindeki yanma odalarında neler olduğu (detay), kullanıcının bilmesi gereken bir şey değildir (soyutlama).

Soyut sınıflar (**abstract class**) da tanımlanabilir ancak bu sınıflardan nesne imal edilemez. Yani bu sınıfın bir **örneği** (**instance**) **olamaz!** Daha çok **kavramsal olarak kullanılır!** Soyut bir sınıfın en az bir soyut yöntemi (**abstract method**) olur. Soyut sınıflar hangi davranışı göstereceğini söyler ama nasıl davranış göstereceğini anlatmaz. Bu nedenle **abstract** kelimesiyle tanımlanan **soyut yöntem** (**abstract method**) için gövde tanımlanmaz. Aşağıda soyut bir sınıf tanımı verilmiştir;

```
//Soyut soyutnesne = new Soyut(); //HATA: Soyut sınıftan nesne imal edilemez!
```

```
abstract class Soyut
{
    public abstract void Yöntem(); //Soyut Yöntemler için gövde tanımlanmaz!
}
```

Somut (**concrete**) ise en ince detayına kadar bilinen, elle tutulur anlamına gelir. **Somutlama** (**concretization**), soyut olarak tanımlanmış bir yapının gerçek hayattaki karşılığının oluşturulmasıdır. **Soyut olmayan** geri kalan sınıflar **somut sınıftır** (**concrete class**) be bu sınıflardan nesne imal edilebilir. Yani bu sınıfların **örneği** (**instance**) vardır. İleride göreceğimiz bir arayüzün (**interface**) içinin doldurulması da somutlamadır. Yani kodlayarak uygulamanın hayata geçirilmesidir (**implementation**).

Bilgi gizleme (**information hiding**), bir nesnenin iç durumuna (verilerine) dışarıdan doğrudan erişilmesini engellemek ve bu verileri koruma altına almaktır. Genellikle ileride göreceğimiz kapsülleme (**encapsulation**) ile yapılır.

Bir sınıfta **soyutlama** (**abstraction**) iki türlü yapılır; Birincisi **veri soyutlaması** (**data abstraction**), nesnenin durumları üzerinde yapılan soyutlamadır. İkincisi ise **kontrol soyutlaması** (**control abstraction**), nesnenin davranışları üzerinde yapılan soyutlamadır.

8.7 Kapsülleme

Yukarıda verilen **Öğrenci** örneğinden devam edecek olursak imal edilecek nesnenin **yaş** durumuna bir başka nesne veya program, **-1** gibi bir değer atayabilir. Bu durumda **YaşınıSöyle()** davranışında istenmeyen bir çıktı verilmesine sebep olur. Bunu gidermek için **veri soyutlaması** (**data abstraction**) yapılması gerekir.

Veri soyutlaması bir alan değişkeni üzerinde şöyle yapılır;

- ◊ Öncelikle alan değişkenini mahrem (**private**) olarak tanımlayıp dışarıya erişime kapatmalıyız. Örnek: **private uint yaş;**
- ◊ Daha sonra alan değişkeniyle aynı olacak şekilde Büyük harfle başlayan bir umuma açık (**public**) özellik (**property**) tanımlanır. Örnek: **public uint Yaş { }**
- ◊ Sonra özelliğe alan değişkenine kontrollü değer atayacak **set** yöntemi ve alan değişkeninden değeri dışarıya aktaran **get** yöntemleri eklenir.

Öğrenci örneğine ilişkin sınıfımızın Yaş ve Cinsiyet özelliği eklenmiş hali aşağıda kodlanmıştır;

```
Öğrenci ilhan = new Öğrenci(30, 'E');
ilhan.Yaş = 50; // "ilhan" nesnesinin "yas" özelliği değiştirildi.
ilhan.YaşınıSöyle(); // Yaşım:50
ilhan.CinsiyetiniSöyle(); // Cinsiyetim:ERKEK
ilhan.Yaş = 5; /* ilhan nesnesinin Yaş özelliği değişmez: Öğrenci 6 Yaşından Küçük Olamaz.*/
ilhan.Cinsiyet = 'H'; /* ilhan nesnesinin Cinsiyet özelliği değişmez: Cinsiyet E,e,K,k,B,b
→ dışında bir değer olamaz. */
```

```

class Öğrenci
{
    //Alanlar
    private char _cinsiyet; //private alanlar altçizgi karakteriyle başlar
    private uint _yaş; //private alanlar altçizgi karakteriyle başlar
    //Yapıcı
    public Öğrenci(uint yaş,char cinsiyet)
    {
        this._yaş = yaş;
        this._cinsiyet = cinsiyet;
    }
    //Özellikler
    public uint Yaş //Yaş Özelliği: Özelliğin get yada set yöntemi olur.
    {
        get { return _yaş; }
        set {
            if (value < 6)
                Console.WriteLine("Öğrenci 6 Yaşından Küçük Olamaz.");
            else
                _yaş = value;
        }
    }
    public char Cinsiyet //Cinsiyet Özelliği: Özelliğin get yada set yöntemi olur.
    {
        get { return _cinsiyet; }
        set {
            string sonuc = value switch
            {
                'e' or 'E' => "ERKEK",
                'k' or 'K' => "KADIN",
                'b' or 'B' => "BELİRTMİYOR",
                _ => "X"
            };
            if (sonuc == "X")
                Console.WriteLine("Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz.");
            else
                _cinsiyet = value;
        }
    }
    //Yönetimler
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{_yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        string sonuç = _cinsiyet switch
        {
            'e' or 'E' => "ERKEK",
            'k' or 'K' => "KADIN",
            _ => "SÖYLEMEK İSTEMİYORUM"
        };
        Console.WriteLine($"Cinsiyetim:{sonuç}");
    }
}

```

Nesneye ilişkin durum (**state**) verileri, alan değişkenlerinde saklanır. Bir durumun veri soyutlaması yapılmış haline **özellik** adı veririz. Çünkü artık bu duruma dışarıdan doğrudan **müdahale edilemez!** Nesnenin **durum üzerinde kontrol hakları vardır.**

Benzer şekilde davranışlar (**behavior**) da nesne dışına kontrollü olarak açılabilir. Bu durumda da **kontrol soyutlaması (control abstraction)** yapılır. Aşağıda faktöriyel hesaplayan bir hesap makinesinin ekranı

yenileme mahrem davranışı olan `EkraniYenile()` buna örnek verilebilir;

```
HesapMakinesi hesaplayici = new HesapMakinesi(); // Hesaplanan: 0
hesaplayici.Hesaplanan = 3;
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 6
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 720 (6! = 720)
hesaplayici.MakineyiSıfırla();
hesaplayici.Hesaplanan = 8;
hesaplayici.FaktöriyelHesapla(); // Hesaplanan: 40320 (8! = 40320)
```

```
class HesapMakinesi
{
    //Alan
    private int _hesaplanan;
    //Yapıcı
    public HesapMakinesi()
    {
        _hesaplanan = 0;
        EkraniYenile();
    }
    //Özellik
    public int Hesaplanan
    {
        set
        {
            _hesaplanan = value;
            EkraniYenile();
        }
        get
        {
            return _hesaplanan;
        }
    }
    //Yöntemler
    public void MakineyiSıfırla()
    {
        _hesaplanan = 0;
        EkraniYenile();
    }
    public void FaktöriyelHesapla()
    {
        int temp = 1;
        for (int sayac = _hesaplanan; sayac > 0; sayac--)
            temp = temp * sayac;
        _hesaplanan = temp;
        EkraniYenile();
    }
    private void EkraniYenile() //Mahrem Davranış!
    {
        Console.WriteLine($"Hesaplanan: {_hesaplanan}");
    }
}
```

Bir sınıf üzerinde veri soyutlaması (**data abstraction**) yapılarak veriler üzerine bir kılıf (**capsule**) çekilir, bir de kontrol soyutlaması (**control abstraction**) ile davranışların üzerine bir kılıf (**capsule**) daha çekilir. **Bir sınıf üzerinde her iki soyutlamanın yapılmasına kapsülleme (**encapsulation**) adı verilir.**

Soyutlamaların gerekli olup olmadığına programcı karar verir. Kapsülleme işlemi örneklerde görüldüğü üzere erişim değiştiricilerle (**access modifier**) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış, istenmeyen detaylar dış dünyadan gizlenir (**information hiding**). Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden soyut (**abstract**) olarak kullanmak mümkün olmaktadır. Programcı, tasarladığı sınıfın soyut olarak kullanılacağını düşünerek sınıfları tasarlamalıdır.

Kapsülleme (**encapsulation**) tekniğini uygulamak, **kullanımı kolay** sınıf (**class**), bileşen (**component**), kütüphane (**library**) ve çerçeve yapılar (**framework**) elde etmemizi sağlar;

- ◊ n adet durum/veri (**state/data**) ve n adet davranış (**behavior**) kapsülleme işlemine tabi tutulursa sınıf,
- ◊ n adet sınıf kapsülleme işlemine tabi tutulursa bileşen,
- ◊ n adet bileşen kapsülleme işlemine tabi tutulursa kütüphane,
- ◊ n adet kütüphane kapsülleme işlemine tabi tutulursa çerçeve yapılar oluşur.

Sonuç olarak kapsülleme, projemizde proje süresini ve maliyetini kısaltma üstünlüğünü verir.

8.8 Nesne Başlatıcıları

Başlatma kavramı değişken tanımlama ile ilgili 4.2.3 başlığında anlatılmıştı. **Nesne başlatıcıları** (**object initializer**), bir nesnenin özelliklerine (**properties**) veya alanlarına (**fields**) nesne oluşturulma anında, açıkça bir yapıcı (**constructor**) çağırma gereke kalmadan değer atamamızı sağlayan bir söz dizimidir;

- ◊ **Okunabilirlik**: Yapıcı metotlara kıyasla daha okunabilir bir alternatif sunar. Parametrelerin neyi temsil ettiğini parantez içindeki sırasından değil, doğrudan isimlerinden anlarız.
- ◊ **Bürokrasi (Boilerplate) Azaltma**: Sadece veri taşımak için oluşturulan sınıflarda onlarca farklı yapıcı (**constructor overload**) yazma zorunluluğunu ortadan kaldırır.
- ◊ **required ile Tam Uyum**: C# 11 ile gelen **required** üyeler, nesne başlatıcıları aracılığıyla zorunlu hale getirilebilir.

Geleneksel yapıcılar, fabrikadan her şeyi monte edilmiş halde gelen bir masaya benzer. Masanın her detayı üretim bandında (**constructor**) belirlenmiştir ve sonradan değiştirmek zordur.

```
public class Kitap
{
    public string Ad { get; set; } = string.Empty;
    public string Yazar { get; set; } = string.Empty;
    public int SayfaSayisi { get; set; }
}

public class Program
{
    public static void Main()
    {
        // Geleneksel Yöntem (Constructor gerektirir)
        // var kitap1 = new Kitap("Suç ve Ceza", "Dostoyevski", 600);

        // Object Initializer Yöntemi
        var kitap2 = new Kitap
        {
            Ad = "Nutuk",
            Yazar = "Mustafa Kemal Atatürk",
            SayfaSayisi = 543
        };
    }
}
```

8.9 init Erişim Belirleyicisi ile Esnek Değişmezlik

C#9.0 ile tanıtılan **init** anahtar kelimesi, özelliklerin (**properties**) yalnızca nesne oluşturulma aşamasında atanabilmesini sağlayan bir erişim belirleyicidir. Bu yapı, nesne yönelimli tasarımda "değişmezlik" (**immutability**) ilkesini korurken, nesne başlatıcılarının (**object initializers**) sunduğu esneklikten ödün vermememizi sağlar.

Geleneksel **get; set;** yapısında özellikler nesne ömrü boyunca her an değiştirilebilir. Eğer salt okunur bir yapı istenirse **get; private set;** veya sadece **get;** kullanılabilir; ancak bu durumda değerlerin mutlaka bir yapıcı (**constructor**) üzerinden geçilmesi gerekir. **init** ise bu iki yaklaşımın ortasında durur: Nesne oluşturulurken (**new** anahtar kelimesi ile) değer atanmasına izin verir, ancak atama işlemi bittiği anda özelliği (**property**) salt okunur hale getirir.

Bir kargo kutusunu hazırladığınızı düşünelim. Kutunun içine eşyayı koyma ve üzerine adres etiketini yapıştırma yetkiniz vardır; ancak bu yetki sadece kutu kargoya verilmek üzere "hazırlanırken" geçerlidir. Kutu bir kez mühürlenip yola çıktıktan (nesne oluşturulduktan) sonra içindeki eşyayı değiştiremez veya etiketi

sökemezsiniz. `init`, o mühür vurulmadan önceki son atama yetkisidir.

```
public class Bilgisayar
{
    // Sadece başlangıçta atanabilir
    public string Islemci { get; init; }
    public int Ram { get; init; }

    // Her zaman güncellenebilir
    public decimal Fiyat { get; set; }
}

public class Program
{
    public static void Main()
    {
        // Nesne başlatıcı kullanarak atama yapmak mümkün
        var pc = new Bilgisayar
        {
            Islemci = "i9",
            Ram = 32,
            Fiyat = 45000m
        };

        // HATA: 'Islemci' init-only olduğu için sonradan değiştirilemez
        // pc.Islemci = "i7";

        // 'Fiyat' set olduğu için değiştirilebilir
        pc.Fiyat = 42000m;
    }
}
```

Bir başka örnek olarak dağıtıma çıkan bir kargo paketi örnek verilebilir. Bir paket hazırlanırken içeriği belirlenebilir ancak paket mühürlenip dağıtıma çıktığında içerik artık değiştirilemez. `init`, nesnenin mühürlenme anına kadar olan atama yetkisidir.

```
public class SensorVerisi
{
    public DateTime KayitZamani { get; init; }
    public double Deger { get; init; }
}

// Geçerli kullanım
var veri = new SensorVerisi { KayitZamani = DateTime.Now, Deger = 24.5 };
// Derleme zamanı hatası:
// veri.Deger = 25.0;
```

8.10 Miras Alma

Öğrenci, öğretmen ve müdürün olduğu bir okul sistemini örnek alarak ele alalım. Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir;

```
//ANA PROGRAM:
Öğrenci ali = new Öğrenci(20, 'E', 12000);
ali.YaşınıSöyle(); //Yaşım:20
ali.CinsiyetiniSöyle(); //Cinsiyetim: ERKEK
ali.DersÇalış(); //Dersimi Çalışıyorum...

Öğretmen ilhan = new Öğretmen(50, 'E', 101);
ilhan.YaşınıSöyle(); //Yaşım:50
ilhan.CinsiyetiniSöyle(); //Cinsiyetim: ERKEK
ilhan.DersVer(); //101 Dersini Veriyorum...

Müdür damla = new Müdür(30, 'K', 5000);
damla.YaşınıSöyle(); // Yaşım:30
damla.CinsiyetiniSöyle(); //Cinsiyetim: KADIN
```

damla.DersProgramıHazırla(); //5000 Bölümünde Ders Programı Hazırlıyorum...

//SINIFLAR:

class Öğrenci

```
{
    //yapıcı:
    public Öğrenci(uint pYaş, char pCinsiyet, uint pOkulNumarası) {
        Yaş=pYaş; Cinsiyet=pCinsiyet; OkulNumarası=pOkulNumarası;
    }
    //Özellikler:
    public uint Yaş { get; set; } //sanki "yaş" alan değişkeni varmış gibi kısa özellik tanımı!
    public char Cinsiyet { get; set; }
    public uint OkulNumarası { get; set; }
    // Yöntemler:
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(Cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
    public void DersÇalış()
    {
        Console.WriteLine("Dersimi Çalışıyorum...");
    }
}
```

class Öğretmen

```
{
    //Yapıcı:
    public Öğretmen(uint pYaş, char pCinsiyet, uint pVerdiğiDersKodu) {
        Yaş=pYaş; Cinsiyet=pCinsiyet; VerdiğiDersKodu=pVerdiğiDersKodu;
    }
    //Özellikler:
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public uint VerdiğiDersKodu { get; set; }
    // Yöntemler:
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(Cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
        }
    }
}
```

```

        break;
    default:
        Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
        break;
    }
}
public void DersVer()
{
    Console.WriteLine($"{VerdiğiDersKodu} Dersini Veriyorum...");
}
}
class Müdür
{
    //Yapıcı:
    public Müdür(uint pYaş, char pCinsiyet, uint pMüdürOlduğuBölümKodu) {
        Yaş=pYaş; Cinsiyet=pCinsiyet; MüdürOlduğuBölümKodu=pMüdürOlduğuBölümKodu;
    }
    //Özellikler:
    public uint Yaş { get; set; }
    public char Cinsiyet { get; set; }
    public uint BölümKodu { get; set; }
    // Yöntemler:
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(Cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
}
public void DersProgramıHazırla()
{
    Console.WriteLine($"{BölümKodu} Bölümünde Ders Programı Hazırlıyorum...");
}
}

```

Yukarıdaki örnek programdaki sınıfların her birinde **Yaş** ve **Cinsiyet** gibi ortak özellikler (**property**) veya her sınıfta aynı kodu içerecek şekilde **YaşınıSöyle()** ve **CinsiyetiniSöyle()** gibi ortak davranışlar (**behavior**) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanmıştır.

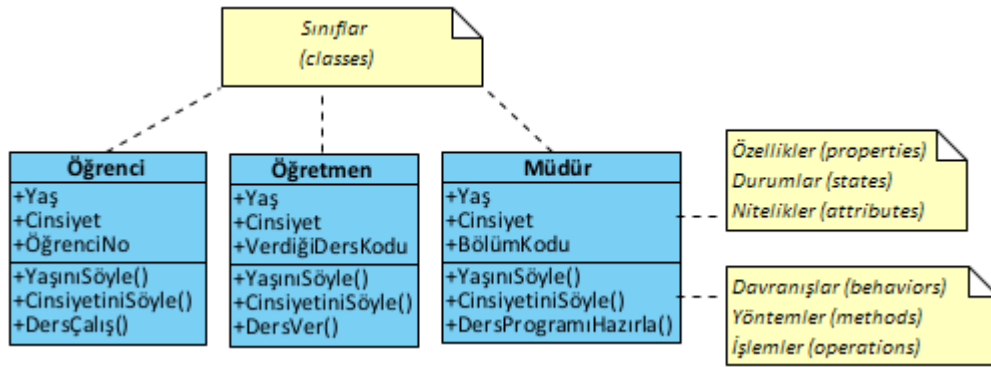
Yazılım analizi yapılırken, takım içerisinde iletişimi kolaylaştırmak için aşağıdaki **sınıf diyagramları** (**class diagram**) kullanılır. Bu diyagramlar UML (**Unified Modeling Language**) dilinin bir parçasıdır. Sonraki bölümde anlatılacaktır. Mevcut okul projesinin UML diyagramı Şekil 8.2’de verilmiştir.

Nesne yönelimli programlamada, örnekteki gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanarak bu sınıflar ortak olmayan özellik ve davranışları barındıracak şekilde yeniden tanımlanabilir. Buna **miras alma** (**inheritance**) veya kısaca **kalıtım** adı verilir. Böylece proje zamanı kısalar ve daha yönetilebilir bir kod elde edilir. Bu durumda miras almış halleriyle okul projesi aşağıdaki gibi kodlanabilir;

```

//ANA PROGRAM:
Öğrenci ali = new Öğrenci(20, 'E', 12000);
ali.YaşınıSöyle(); //Yaşım:20
ali.CinsiyetiniSöyle(); //Cinsiyetim: ERKEK

```

Şekil 8.2: Okul Projesi Sınıfları UML Diyagramı

```
ali.DersÇalış();//Dersimi Çalışıyorum...
```

```
Öğretmen ilhan = new Öğretmen(50, 'E', 101);
ilhan.YaşınıSöyle(); //Yaşım:50
ilhan.CinsiyetiniSöyle(); //Cinsiyetim: ERKEK
ilhan.DersVer(); //101 Dersini Veriyorum...
```

```
Müdür damla = new Müdür(30, 'K', 5000);
damla.YaşınıSöyle(); // Yaşım:30
damla.CinsiyetiniSöyle(); //Cinsiyetim: KADIN
damla.DersProgramıHazırla(); //5000 Bölümünde Ders Programı Hazırlıyorum...
```

```
//SINIFLAR:
```

```
class Kişi //Taban(base)/Ebeveyn(parent)/Genel(general) Sınıf
{
    //Yapıcı
    public Kişi (uint pYaş, char pCinsiyet)
    {
        Yaş=pYaş; Cinsiyet=pCinsiyet;
    }
    //Ortak Özellikler
    public uint Yaş { get; set; } //sanki "yaş" alan değişkeni varmış gibi kısa özellik tanımı!
    public char Cinsiyet { get; set; }

    // Ortak Davranışlar
    public void YaşınıSöyle()
    {
        Console.WriteLine($"Yaşım:{Yaş}");
    }
    public void CinsiyetiniSöyle()
    {
        switch (char.ToUpper(Cinsiyet))
        {
            case 'E':
                Console.WriteLine("Cinsiyetim: ERKEK");
                break;
            case 'K':
                Console.WriteLine("Cinsiyetim: KADIN");
                break;
            default:
                Console.WriteLine("Cinsiyetimi Söylemek İstemiyorum!");
                break;
        }
    }
}

class Öğrenci: Kişi //Türemiş(derived)/Çocuk(child)/Özel(special) Sınıf
{
}
```

```

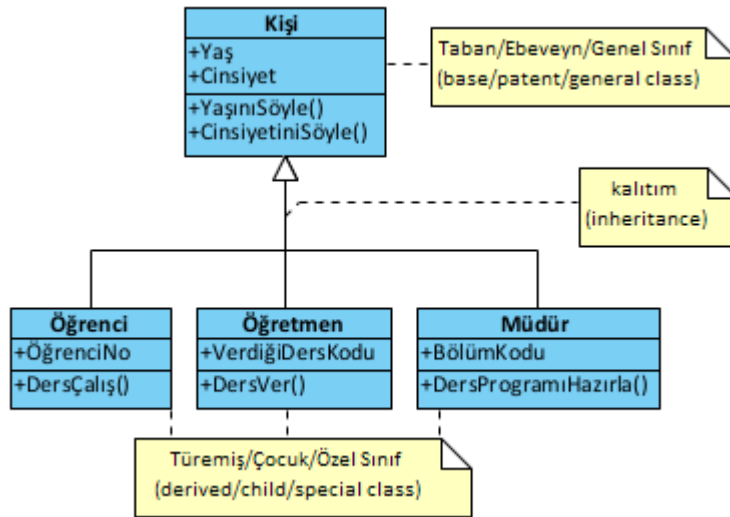
//Yapıcı
public Öğrenci(uint pYaş, char pCinsiyet, uint pOkulNumarası): base(pYaş,pCinsiyet) //base
→ ile taban sınıfın yapıcısı çağrılır
{
    OkulNumarası=pOkulNumarası;
}
//Özellikler
public uint OkulNumarası { get; set; }
//Davranışlar
public void DersÇalış()
{
    Console.WriteLine("Dersimi Çalışıyorum...");
}
}
class Öğretmen: Kişi //Türemiş(derived)/Çocuk(child)/Özel(special) Sınıf
{
    public Öğretmen(uint pYaş, char pCinsiyet, uint pVerdiğiDersKodu): base(pYaş,pCinsiyet)
    → //base ile taban sınıfın yapıcısı çağrılır
    {
        VerdiğiDersKodu=pVerdiğiDersKodu;
    }
    //Özellikler
    public uint VerdiğiDersKodu { get; set; }
    // Yöntemler
    public void DersVer()
    {
        Console.WriteLine($"{VerdiğiDersKodu} Dersini Veriyorum...");
    }
}
class Müdür:Kişi //Türemiş(derived)/Çocuk(child)/Özel(special) Sınıf
{
    public Müdür(uint pYaş, char pCinsiyet, uint pBölümKodu): base(pYaş,pCinsiyet) //base ile
    → taban sınıfın yapıcısı çağrılır
    {
        BölümKodu=pBölümKodu;
    }
    //Özellikler
    public uint BölümKodu { get; set; }
    // Yöntemler
    public void DersProgramıHazırla()
    {
        Console.WriteLine($"{BölümKodu} Bölümünde Ders Programı Hazırlıyorum...");
    }
}

```

Yukarıdaki kod incelendiğinde şunları söyleyebiliriz;

- ◇ Nesne oluşturan ve kullanan ana programda hiçbir değişiklik olmamıştır.
- ◇ Ortak özellik ve davranışları barındıran **Kişi** sınıfı tanımlanmıştır.
- ◇ **Öğrenci**, **Öğretmen** ve **Müdür** sınıflarında ortak özellik ve davranışlar kaldırılmıştır. Bunun yerine her bir sınıf kimliğinin arkasına **:Kişi** kodu eklenerek **Kişi** sınıfından ortak özellik ve davranışlar **miras alınmıştır**.
- ◇ Her bir sınıfın yapıcısına, miras alınan sınıfın özelliklerine ilk değer vermek amacıyla **base** anahtar kelimesi ile yapıcı argümanları geçirilmiştir.
- ◇ **Kişi** olarak tanımlanan ortak özellik ve davranışlar sınıfı, **genel sınıf (general class)**, **taban sınıf (base class)** veya **ebeveyn sınıf (parent class)** olarak adlandırılır.
- ◇ **Öğrenci**, **Öğretmen** ve **Müdür** sınıfları ise **Kişi** taban sınıfından **türemiştir (derived)**. Bu sınıflar genel sınıf olan **Kişi** sınıfının yanında kendine has özel özellik ve davranışlara da sahiptir. Bu nedenle **özel sınıf (private class)**, **türemiş sınıf (derived class)** veya **çocuk sınıf (child class)** olarak adlandırılır. Bu sınıflar mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

Kalıtım uygulanmış haliyle okul projesinin UML diyagramı Şekil 8.3’de verilmiştir;



Şekil 8.3: Kalıtım Uygulanmış Okul Projesi Sınıfları UML Diyagramı

Örneğimizden gidecek olursak, **Kişi** sınıfında yapılacak bir özellik veya davranış eklemesi anında bu sınıftan türemiş sınıflara da yansır. Türemiş olan **Öğrenci**, **Öğretmen** ve **Müdür** sınıflarında bununla ilgili değişiklik gerekmez.

Değişim yönetimini kolayca yapma üstünlüğünü veren miras almanın (**inheritance**) iki önemli özelliği vardır;

1. Paylaşılan bir ortak kod (**shared code**) vardır.
2. Paylaşılan kodda yapılan değişiklik anında alt sınıflara yansır (**snap change**).

§8.10.1 Miras Alınamayanlar

Taban sınıftan **teknik olarak miras alınamayan** aşağıdaki yapılar vardır;

- ◊ **Yapıcılar (constructor)**: Taban sınıfın yapıcıları türetilmiş sınıfa geçmez. Ancak, türetilmiş sınıfın yapıcısı çalıştığında taban sınıfın yapıcısını (**base** ile) gizli veya açık bir şekilde çağırır.
- ◊ **Sonlandırıcılar (finalizer)**: C# dilinde sonlandırıcılar, miras alınmaz. Her sınıfın kendi yıkım süreci vardır. sonlandırıcılar miras alınmaz ancak hiyerarşik olarak tetiklenirler. Bir nesne yok edilirken önce en alt sınıfın (**derived**), sonra sırasıyla üst sınıfların (**base**) sonlandırıcıları çalışır.

Finalizer Order: DerivedClass → BaseClass → System.Object

Modern C# geliştirmede, sonlandırıcılar tek başına yetersiz görülür çünkü zamanlaması belirsizdir. Bu yüzden **IDisposable** arayüzü ve **Dispose()** metodu ile birlikte kullanılırlar. 8.17 başlığında anlatılacaktır.

- ◊ **Kopya Yapıcılar**: C# dilinde, C++ dilindeki gibi yerleşik bir kopya yapıcı (**copy constructor**) kavramı yoktur. Eğer kendiniz yazarsanız, bu da bir yapıcı olduğu için miras alınmaz.

§8.10.2 protected Erişim Değiştiricisi

Miras alma (Inheritance) konusunu anlatırken karşımıza çıkan en kritik konulardan biri de **protected** erişim değiştiricisinin (**access modifier**) kullanılmasıdır. Bu değiştirici kısaca şöyle tanımlayabiliriz: "Dış dünyaya kapalı (**private**), ama mirasçılara açık."

Bir üyeyi **protected** olarak tanımladığınızda:

- ◊ Sınıfın dışından (programın geri kalanından) bu üyeye erişilemez.
- ◊ Bu sınıftan türetilen (miras alan) tüm sınıflar bu üyeyi sanki kendi içindeymiş gibi kullanabilir.

Bunu anlamak için aşağıdaki örnek verilebilir;

```

Yoneticici mudur = new Yoneticici();
mudur.MaasArtir();
// mudur.Maas = 50000; // HATA: 'Maas' protected olduğu için Main metodundan (dışarıdan)
→ erişilemez.
  
```

```

class Personel
{
  
```

```

private string _tcKimlikNo = "12345678901"; // private: Sadece bu sınıfa özel. Kimse göremez.
protected double Maas = 20000; // protected: Miras alan sınıflar (Yönetici gibi) görebilir.
public void BilgiVer()
{
    Console.WriteLine("Ben bir personelim.");
}
}
class Yoneticici : Personel
{
    public void MaasArtir()
    {
        Maas += 5000; // ERIŞEBİLİR: Maas, üst sınıfta 'protected' olduğu için miras alındı.
        Console.WriteLine($"Yeni Maaş: {Maas}");
        // Console.WriteLine(_tcKimlikNo); // HATA: taban sınıfta _tcKimlikNo 'private' olduğu
        // için türetilmiş sınıftan dahi erişilemez.
    }
}

```

Neden `protected` Kullanınız?

- ♦ **Güvenli kapsülleme (encapsulation):** Veriyi tamamen dışarıya açmak (`public`) yerine, sadece o veriyi işleme yetkisi olan "aile üyelerine" (mirasçılara) açmış olursunuz.
- ♦ **Kod Tekrarını Önleme:** Ortak özellikleri üst sınıfta `protected` tutarak, alt sınıfların bu özelliklere doğrudan erişmesini ve yeniden tanımlama yapmamasını sağlarsınız.
- ♦ **Hiyerarşik Kontrol:** Sınıf hiyerarşisi içinde bir veri akışı kurmanıza olanak tanır.

Ayrıca; Eğer alt sınıfların bir özelliğin değerini değiştirmesini (`set`) istiyor ama dış dünyadan bu özelliğin sadece okunmasını (`get`) istiyorsanız, erişim değiştiricilerini hibrit kullanabiliriz. Buna örnek olarak şu özellik tanımını verebiliriz: `public string Ad { get; protected set; }`

§8.10.3 Gölgeleme

Miras aldığınız bir sınıfın içinde, üst sınıfta (base class) zaten var olan bir metodun veya alan değişkeninin ismini tekrar tanımlamak istediğinizde C# derleyicisi bir uyarı verir. "Ben bu kimliği tekrar kullanıyorum ve üst sınıftakini gizlemek istiyorum" demek için `new` belirleyicisini (`new specifier`) kullanırsınız. Bu işleme **üye gizleme (member hiding)** veya gölgeleme adı verilir.

```

Taban taban = new Taban(10);
Türemiş türemiş = new Türemiş(30);
taban.Yöntem(); // Taban Sınıfta 10
türemiş.Yöntem(); /* Türemiş Sınıfta 0: Çünkü; türemiş sınıfta tanımlanan alan değişkeni
→ başlangıç değeri sıfırdır ve taban sınıftaki x gizlenmiştir. */

```

```

public class Taban
{
    public Taban(int x)
    {
        this.x = x;
    }
    public int x;
    public void Yöntem()
    {
        Console.WriteLine($"Taban Sınıfta {x}");
    }
}
public class Türemiş : Taban
{
    public Türemiş(int x) : base(x) { }
    new public int x; //taban sınıftaki kimlikle yeni x değişkeni
    new public void Yöntem() //taban sınıftaki kimlikle yeni Yöntem
    {
        Console.WriteLine($"Türemiş Sınıfta {x}");
    }
}

```

Türemiş sınıfta kullanılan **new** belirleyicisi üyelerin taban sınıfta olan bağı kopartılmıştır. Program çalışırken neye bakacağını referans tipine göre karar verir.

8.11 Geçersiz Kılma

Geçersiz kılma (overriding), nesne yönelimli programlamanın, taban sınıfta önceden tanımlanmış bir yöntemi türemiş bir sınıfta yeniden tanımlamasına olanak tanıyan bir kavramdır. Yani taban sınıfın davranışının türemiş sınıfta değiştirilmesidir.

Geçersiz kılınacak yöntem, taban sınıfta ya **sanal (virtual)** ya da **soyut (abstract)** olarak tanımlanmalıdır. Soyut kavramı 8.6 başlığında verilmişti. Soyut yöntemler **abstract** sıfatıyla tanımlanır ve türetilmiş sınıfta **mutlaka geçersiz kılınmalıdır (override)**. Sanal yöntemler ise **virtual** sıfatıyla tanımlanır ve geçersiz kılınma zorunluluğu yoktur. Geçersiz kılınarak yeniden tanımlanan yöntem **override** sıfatı ile tanımlanır. Her iki sınıf ta da yöntemin kimliği aynıdır. Taban sınıftan imal edilen nesne taban sınıfın yöntemini, türemiş sınıftan imal edilen nesne ise türemiş sınıfın yöntemini icra eder.

```
Baba baba = new Baba();
Çocuk çocuk = new Çocuk();
baba.SoyutYap(); //Baba soyutu somutlaştırdı.
baba.Yap(); //Baba bu şekilde yapar ...
çocuk.SoyutYap(); //Çocuk babanın somutlaştırdığını değiştirdi.
çocuk.Yap(); //Çocuk babanı yaptığını başka şekilde yapar...
```

```
//Sınıflar:
abstract class Kişi //Soyut Sınıf
{
    public abstract void SoyutYap(); //Soyut Bir Davranış Tanımı:
}
class Baba: Kişi // Somut Sınıf
{
    public override void SoyutYap() //Soyut Yöntem Mutlaka Geçersiz Kılınmalıdır!
    {
        Console.WriteLine("Baba soyutu somutlaştırdı.");
    }
    public virtual void Yap() // Sanal bir davranış Tanımı:
    {
        Console.WriteLine("Baba bu şekilde yapar ...");
    }
}
class Çocuk : Baba
{
    public override void SoyutYap() //Babanın somutlaştırdığı yöntem de geçersiz kılınabilir
    {
        Console.WriteLine("Çocuk babanın somutlaştırdığını değiştirdi.");
    }
    public override void Yap() // Babanın sanal Davranışı geçersiz kılınıyor. İstenirse
    ↪ yapılmayabilir!
    {
        Console.WriteLine("Çocuk babanı yaptığını başka şekilde yapar...");
    }
}
```

En çok karıştırılan nokta, gölgeleme (**new**) ile geçersiz kılma (**override**) arasındaki farktır. Tablo 8.2 ile konuyu netleştirebilirsiniz:

8.12 Statik Üyeler ve Statik Sınıf

Programda çalışmaya başladığı anda bazı değişkenler ve yöntemlere henüz bellekte tahsis edilmemiş veya istenilen nesneler henüz imal edilmemiştir. Program daha çalışmadan bazı değişken, yöntem ve nesnelerin bellekte hazır olması istenir. Uygulamamızın daha belleğe yüklenirken ve yaşamı boyunca bellekte bulunacak değişken, yöntem ve nesneler **statik (static)** olarak tanımlanır.

Bir sınıfın bir veya birkaç üyesi statik olabilir. Statik olan değişken veya yöntemler yani statik üyeler

Özellik	new (Gölgeleme)	override (Geçersiz Kılma)
Bağlantı Referans Tipi	Üst sınıf ile bağı koparır. Referans tipine göre yöntem çağrılır.	Üst sınıfın davranışını günceller. Nesne tipine göre yöntem çağrılır.
Gereksinim	Üst sınıftaki yöntemin virtual olmasına gerek yoktur.	Üst sınıf yöntem mutlaka virtual veya abstract olmalıdır.

Tablo 8.2: new ile override Karşılaştırması

(**static member**), örnek üyeleri (**instance member**) gibi nesne ile imal edilip nesne ile ölmezler. Bu üyeler program yüklenirken ver bellekte (**data segment**) hazır olurlar. Bu nesnelere statik üyeler, örneğe ait değil sınıfa ait olarak nitelendirilir. Yani program bellekte kaldığı sürece statik sınıf ve üyeleri hayattadır.

Programlarda sıkça kodladığımız programın başladığı **static void Main(string[] args)** yöntemi de bu nedenle **static anahtar kelimesiyle** (**static keyword**) tanımlanır.

Sadece statik üyeleri barındıran **statik sınıflar** (**static class**) da tanımlayabiliriz. Bu sınıflara **yardımcı sınıf** (**utility class**) adı da verilir;

- ◇ Doğası gereği nesne yapıcısı barındırmazlar!
- ◇ Bu sınıflar türetilemez!
- ◇ Sınıfın tüm üyeleri statik olmak zorundadır!

```
string bağlantı = Yardımcı.GetConnection();
Yardımcı.GenelSayaç++;
ulong f = Yardımcı.Faktöriyel(5);
```

```
static class Yardımcı /* Statik sınıf */
{
    public static ulong GenelSayaç = 0;
    public static string GetConnection()
    {
        return "Data Source=(LocalDB)\\MSSQLLocalDB; Integrated Security=True;Connect
        ⇐ Timeout=30;Encrypt=True";
    }
    public static ulong Faktöriyel(uint n)
    {
        if (n > 1) return n * Faktöriyel(n - 1);
        else return 1;
    }
}
```

8.13 İşleçlerin Aşırı Yüklenmesi

İşleçleri aşırı yüklenmesi (**operator overloading**), dilin yerleşik operatörlerine (+, -, *, == gibi) kendi oluşturduğunuz sınıflar için özel anlamlar yüklemenizi sağlar. Aşırı yüklenecek işleçlere ilişkin yöntemler;

- ◇ **Statik Olma Zorunluluğu:** Tüm operatör metotları **public static** olmak zorundadır.
- ◇ **Parametre Sayısı:** Tek işlenenle işlem yapan işleçler (**unary operator**) bir, iki işlenen ile işlem yapan ikili işleçler (**binary operator**) iki parametre alır.
- ◇ **En Az Bir Nesne:** Parametrelerden en az biri, işlecin tanımlandığı sınıf tipinde olmalıdır.

```
public class Koordinat
{
    public int X { get; set; }
    public int Y { get; set; }

    // + Operatörünü Vektor sınıfı için tanımlıyoruz:
    public static Koordinat operator +(Koordinat v1, Koordinat v2)
    {
        return new Vektor { X = v1.X + v2.X, Y = v1.Y + v2.Y };
    }
}
```

C# dilinde her işleci aşırı yükleyemezsiniz. Aşağıda aşırı yüklenebilecek işleç listesi tablo olarak verilmiştir;

Kategori	İşleçler
Tekli (Unary)	<code>+, -, !, ~, ++, --, true, false</code> (<i>true ve false çift olarak yüklenmelidir</i>)
İkili (Binary)	<code>+, -, *, /, %, &, , ^, <<, >></code>
Karşılaştırma	<code>==, !=, <, >, <=, >=</code> (<i>çiftler olarak yüklenmelidir</i>)
Yüklenemez	<code>&&, , =, ?:, +=, -=, *=, /=, nokta (.), new</code>

Tablo 8.3: Aşırı Yüklenebilecek İşleçler

İşleçleri ön yüklerken aşağıdakilere dikkat etmek gerekir;

- ◊ **Eşli Yükleme Zorunluluğu:** Eğer `==` işlecini yüklüyorsanız, mutlaka `!=` işlecini de yüklemek zorundasınız. Aynı kural `<` ve `>`, `<=` ve `>=`, `true` ve `false` çiftleri için de geçerlidir.
- ◊ **Mantıksal İşleçler:** `&&` ve `||` işleçler doğrudan yüklenemez; ancak `&`, `|`, `true` ve `false` işleçlerini yükleyerek bu davranış dolaylı yoldan kontrol edilebilir.
- ◊ **Atama İşleçleri:** `+=` işleci doğrudan yüklenemez. Ancak siz `+` işleci yüklerseniz, C# otomatik olarak `+=` işlemini bu yeni kurala göre gerçekleştirir.

8.14 Sabit Alan Değişkenleri ve Yalnız Okunabilir Nesneler

Sınıflardaki alan değişkenlerinin (**field**) nesnelere ilişkin durumları (state) yani verileri tuttuğu anlatılmıştı. Sabitler, derleme zamanında bilinen ve ömrü boyunca değişmeyen değişmez değerlerdir. Sabit alan değişkenleri ve yerel değişkenler `const` sıfatıyla tanımlanırlar.

Metinler (**string**) dışındaki referans tiplerine ilişkin sabitlere yalnızca `null` olarak ilk değer verilebilir! Sınıflar (**class**), yapılar (**struct**) ve diziler (**array**) dahil olmak üzere kullanıcı tanımlı referans tipler `const` sıfatıyla tanımlanamaz! Çalışma zamanında yapıcı ile bir kez imal edilen ve bundan sonra değiştirilemeyen bir nesne, yapı veya dizi oluşturmak için `readonly` sıfatı kullanılır.

```
const float PI = 3.1415f;
Takvim t = new Takvim();
t.Yaz();
/*
Ay Sayısı:12
Saat Sayısı:8786 */
Console.WriteLine(Takvim.TakvimAdı);
/*
const alanlar static olarak algılanır ve Sınıf İsmi ile erişilir.
*/
Console.WriteLine(t.günler[0]); //Pazartesi
/*
readonly alanlar, static olarak algılanmaz ve nesne üzerinden erişilir.
*/

class Takvim
{
    public const int AySayısı = 12;
    public const int GünSayısı = 365;
    public const int HaftaSayısı = 52;
    public const int SaatSayısı = GünSayısı * 24 + 6;
    public const string TakvimAdı = "Takvim";
    public readonly string[] günler = { "Pazartesi", "Salı", "Çarşamba", "Perşembe", "Cuma",
        ↪ "Cumartesi", "Pazar" };
    public void Yaz()
    {
        Console.WriteLine($"Ay Sayısı:{AySayısı}");
        Console.WriteLine($"Saat Sayısı:{SaatSayısı}");
    }
}
```

`readonly` anahtar kelimesi (**readonly keyword**) aşağıdaki bağlamlarda kullanılabilen bir sıfattır.

1. **Salt okunur bir alan değişkeni tanımlanmasında:** Alana (**field**) atama yalnızca bildirimde veya aynı sınıftaki bir yapıcıda gerçekleşir.

```
class Takvim
{
```


- ```

 public readonly int AySayısı=12;
 public readonly int HaftaSayısı;
 public Takvim()
 {
 HaftaSayısı = 365 / 7;
 /* Sınıf içinde bile olsa Başka yerde bu alanlar değiştirilemez. */
 }
}

```
2. Salt okunur bir yapı tanımında; Yapı (**struct**) tipinin değiştirilemez olduğunu belirtir.
- ```

var p1 = new Koordinat(0, 0);
Console.WriteLine(p1); // (0, 0)
var p2 = p1 with { X = 3 };
Console.WriteLine(p2); // (3, 0)
var p3 = p1 with { X = 1, Y = 4 };
Console.WriteLine(p3); // (1, 4)

```
- ```

public readonly struct Koordinat
{
 public Koordinat(double x, double y)
 {
 X = x;
 Y = y;
 }
 public double X { get; init; }
 public double Y { get; init; }
 public override string ToString()
 {
 return $"({X}, {Y})";
 }
}

```
3. Örnekleme yapılarak: Nesne imal edilerek (**instantiation**) salt okunur bir üye nesne tanımlanıyorsa; bu örnek üyesinin durumunu değiştirmedigini belirtir.
- ```

Kişi ali = new Kişi("Ali", "YILMAZ");
ali.Yaz();

```
- ```

class Kitap
{
 public string Adı { get; set; }
 public Kitap(string adı)
 {
 this.Adı = adı;
 }
}
class Kişi
{
 public string Adı { get; set; }
 public string Soyadı { get; set; }
 public readonly Kitap matematikKitabı = new Kitap("Calculus I");
 public readonly Kitap kitap;
 public Kişi(string adı, string soyadı)
 {
 this.Adı = adı;
 this.Soyadı = soyadı;
 kitap = new Kitap("C# Programlama");
 }
 public void Yaz()
 {
 Console.WriteLine($"Adı:{Adı}, Soyadı:{Soyadı}");
 Console.WriteLine($"Matematik: {matematikKitabı.Adı}");
 Console.WriteLine($"Kitap: {kitap.Adı}");
 }
}

```

4. Bir `ref readonly` yöntemi dönüşünde: `readonly` kelimesi yöntemin bir referans döndürdüğünü ve bu referansa yazmalara izin verilmediğini belirtir.

## 8.15 Değiştirilemez Nesneler

Nesne imal edildikten sonra, **durumları değiştirilebilir** (**mutable**) nesnelerdir. Bir nesnenin durumlarının değiştirilebilir olması, dahili verilerinin değiştirilebileceği ve bu da nesnenin bütünlüğünün korunmasında olası yan etkilere ve zorluklara yol açabileceği anlamına gelir. Değiştirilebilir sınıfların yaygın örnekleri arasında, elemanlarının eklenebildiği, kaldırılabilirdiği veya değiştirilebildiği birçok koleksiyon türü bulunur.

Değiştirilebilir nesneler yerine değiştirilemez nesneler kullanmak istediğimiz durumlar vardır. Değiştirilemez nesneler **iş parçacığı güvenlidir** (**thread-safe**), ancak diğer yandan birçok durumda performans sorunlarına neden olabilirler. Her geliştiricinin günlük olarak kullandığı en popüler değiştirilemez nesne metin değişkenleridir (**string**). Metinlerle yapılan her işlem sonrasında yeni bir metin ortaya çıkar. Değiştirilemez bir nesne elde etmek için sınıflardaki alanlar `readonly` tanımlanır. Bu sınıfın yapıcısında ilk değerler verilir ve nesne imalatı olduktan sonra bu alanlar değiştirilemez.

```
Takvim t = new Takvim();
t.Yaz();
Console.WriteLine($"Saat Sayısı:{t.SaatSayısı}");
Console.WriteLine(t.TakvimAdı.ToUpper());
/* TakvimAdı immutable string sınıfındandır. Burada mevcut değeri "Takvim" aynen kalır ve büyük
→ harfe çevrilmiş olan yeni bir string imal edilir. */
```

```
class Takvim
{
 public readonly int AySayısı;
 public readonly int GünSayısı;
 public readonly int HaftaSayısı;
 public readonly int SaatSayısı;
 public readonly string TakvimAdı;
 public Takvim()
 {
 AySayısı = 12;
 GünSayısı = 365;
 HaftaSayısı = GünSayısı / 7;
 SaatSayısı = GünSayısı * 24 + 6;
 TakvimAdı = "Takvim";
 }
 public void Yaz()
 {
 Console.WriteLine($"Hafta Sayısı:{HaftaSayısı}");
 Console.WriteLine($"Gün Sayısı:{GünSayısı}");
 }
}
```

## 8.16 Arayüzler

**Arayüzler** (**interface**) nesnelerin sahip oldukları **rollerdir** (**role**). Ancak saf nesne yönelimli dillerde arayüzler **interface** saklı kelimesiyle tanımlanırlar.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav veya değerlendirme yapma, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

Arayüzler de **referans tiplerdir**. Bir sınıfa, bir veya **birden fazla arayüz uygulanabilir** (**interface implementation**). Bir sınıfın bir arayüzü miras almasına **arayüz gerçekleştirme** (**interface realization**) adı da verilir. Arayüzler içinde yöntem imzaları (**signature**) yazılır. Yani sadece yöntem başlığı yazılır. Gövdesi yazılmaz. Ayrıca yöntemler için **erişim belirleyici** (**access modifier**) kullanılmaz.

```
Kişi ilhan = new Kişi("İlhan", "ÖZKAN"); //imal edilen nesne tüm arayüzdeki davranışları
→ gösterebilir.
ilhan.AdınıSöyle(); // İlhan ÖZKAN
ilhan.Öğret(); // Öğretiyorum...
```

```
ilhan.GitarÇal(); // Gitar Çalıyorum...
ilhan.HanımaYardımEt(); // Hanıma Yardım Ediyorum...

IBaba baba = ilhan; //imal edilen nesne sadece bir roldeki davranışları da gösterebilir:
baba.ÇocuğaBak(); // Çocuğa Bakıyorum...
baba.HanımaYardımEt(); // Hanıma Yardım Ediyorum...
İMüzişyen müzişyen = ilhan;
müzişyen.GitarÇal(); // Gitar Çalıyorum...
müzişyen.PiyanoÇal(); // Piyano Çalıyorum...
İÖğretmen öğretmen = ilhan;
öğretmen.DersNotuHazırla(); // Ders notu hazırlıyorum...
öğretmen.Öğret(); // Öğretiyorum...

interface IBaba //Baba Rolündeki Davranışlar:
{
 void ÇocuğaBak(); // Aarayüzde yöntemin gövdesi olmaz!
 void HanımaYardımEt();
}
interface İMüzişyen //Müzişyen Rolündeki Davranışlar:
{
 void GitarÇal();
 void PiyanoÇal();
}
interface İÖğretmen//Öğretmen Davranışları:
{
 void Öğret();
 void DersNotuHazırla();
}
class Kişi : IBaba, İMüzişyen, İÖğretmen //Arayüz Gerçekleştirme!
{
 /*
 Arayüz Gerçekleştirme/Uygulama(interface realization/implementation):
 Gerçekleştirilen IBaba, İMuzisişyen ve İOğretmen arayüzlerinde bulunan yöntemler
 Kişi sınıfında hepsi gövdeleriyle tanımlanmalıdır!
 */
 public string Adı { get; set; }
 public string Soyadı { get; set; }
 public Kişi(string adı, string soyadı)
 {
 this.Adı = adı;
 this.Soyadı = soyadı;
 }
 public void AdınıSöyle()
 {
 Console.WriteLine($"{Adı} {Soyadı}");
 }
 public void ÇocuğaBak()
 {
 Console.WriteLine("Çocuğa Bakıyorum..");
 }
 public void HanımaYardımEt()
 {
 Console.WriteLine("Hanıma Yardım Ediyorum..");
 }
 public void PiyanoÇal()
 {
 Console.WriteLine("Piyano Çalıyorum..");
 }
 public void GitarÇal()
 {
 Console.WriteLine("Gitar Çalıyorum..");
 }
}
```

```

}
public void Öğret()
{
 Console.WriteLine("Öğretiyorum..");
}
public void DersNotuHazırla()
{
 Console.WriteLine("Desr Notu Hazırlıyorum..");
}
}

```

## 8.17 Sonlandırıcılar

**Sonlandırıcılar** (**finalizers**), bir nesnenin yaşam döngüsünün "gün batımı" aşamasını temsil eder. C++ dilindeki yıkıcılara (**destructors**) benzer bir söz dizimine sahip olsalar da, C# dilinin bellek yönetim sistemi olan **çöp toplama** (GC) nedeniyle çalışma mantıkları oldukça farklıdır.

Sonlandırıcı, bir sınıfın örneği (**instance**) **bellekten temizlenmeden hemen önce** çöp toplayıcı tarafından otomatik olarak çağrılan özel bir yöntemdir. Temel amacı, nesne yok edilmeden önce yönetilmeyen kaynakları (**unmanaged resources**) serbest bırakmaktır.

Yazım Kuralları:

- ◊ Sınıf kimliğinin başına eğik çizgi (~) karakteri konularak tanımlanır.
- ◊ **Parametre almazlar ve geri dönüş tipleri yoktur.**
- ◊ Erişim belirleyicisi (**public**, **private** vb.) **sıfatları olmaz.**
- ◊ Bir sınıfta **sadece bir tane** sonlandırıcı bulunabilir.

```

class DosyaYonetici
{
 ~DosyaYonetici() // Sonlandırıcı (Finalizer)
 {
 // Nesne bellekten silinirken yapılacak işlemler
 Console.WriteLine("Nesne imha ediliyor, kaynaklar serbest bırakıldı.");
 }
}

```

C# öğrenenlerin en çok düştüğü hata, sonlandırıcının nesne kapsam dışına çıktığı an çalışacağını sanmaktır;

- ◊ **Belirsiz Zamanlama** (**non-deterministic**): C++ dilinin aksine, C# dilinde sonlandırıcının ne zaman çalışacağını bilemezsiniz. **Çöp toplayıcı** (GC), ne zaman uygun görürse o zaman çalışır.
- ◊ **Yönetilmeyen Kaynaklar**: Sonlandırıcılar, açık dosyalar, ağ bağlantıları veya donanım portları gibi .NET dışı kaynakları kapatmak için kullanılır. Normal bir değişken (**int**, **string** vb.) için sonlandırıcı yazmaya gerek yoktur; bunları çöp toplayıcı zaten temizler.

Modern C# geliştirmede, sonlandırıcılar tek başına yetersiz görülür çünkü zamanlaması belirsizdir. Bu yüzden **IDisposable** arayüzü ve **Dispose()** yöntemi ile birlikte kullanılırlar. Buna **kullan-at deseni** (**disposable pattern**) adı verilir.

```

Console.WriteLine("--- İşlem Başlıyor ---");
using (KaynakYonetici yonetici = new KaynakYonetici()) // using bloğu dışına çıkıldığı an
 → Dispose() otomatik tetiklenir.
{
 Console.WriteLine("Kaynaklar kullanılıyor...");
 // Burada dosyalarla veya ağ ile ilgili işlemler yapılır.
}
Console.WriteLine("--- İşlem Bitti ve Kaynaklar Serbest Bırakıldı ---");

```

```

class KaynakYonetici : IDisposable
{
 private bool _disposed = false;
 // Kullanıcının manuel çağırabileceği temizlik metodu
 public void Dispose()
 {
 Console.WriteLine("Dispose() Yöntemi Çalıştı.");
 }
}

```

```

 Temizle(true);
 GC.SuppressFinalize(this); // Çöp toplayıcı (GC)'ye "sonlandırıcıyı çalıştırmaya gerek
 ↳ kalmadı" der.
 }
 ~KaynakYonetici() {
 Temizle(false);
 }
 protected virtual void Temizle(bool manuelTemizlik)
 {
 if (_disposed) return;
 if (manuelTemizlik)
 {
 /* Yönetilen kaynakları temizle */
 }
 // Yönetilmeyen kaynakları her durumda temizle
 _disposed = true;
 }
}

```

## 8.18 Genişleme Yöntemleri

**Genişleme yöntemleri** (**extension method**), C#3.0+ ile hayatımıza girmiştir. Genişletme yöntemleri, yeni bir türetilmiş tip oluşturmadan, yeniden derlemeden veya orijinal türü başka şekilde değiştirmeden mevcut türlere davranışı genişletir ve ekler. Özellikle geliştirmek istediğiniz bir tipin kaynağını değiştiremediğinizde faydalıdırlar. Genişletme yöntemleri **int**, **char**, **string** gibi sistem tipleri, üçüncü taraflarca kütüphanelerde tanımlanan tipler ve **class** gibi kendiniz tanımladığınız tipler için oluşturulabilir. Genişletme yöntemi orijinal tipin bir üye yöntemiymiş gibi çağrılabilir.

Bir genişletme yöntemi statik olmalı ve ilk parametresi her zaman **this** anahtar sözcüğünden önce gelmeli, ardından genişlettiğiniz nesnenin "geçerli" örneğine başvurmak için tanımlayıcı gelmelidir;

```

class Program
{
 static void Main()
 {
 // aşağıdaki atama String.ToUpper() yöntemini çağırır.
 var myString = "Hello World!".ToUpper();
 /*
 Aşağıdaki yöntem Extensions sınıfının StringExtensions.Shorten() yöntemini çağırır
 */
 var newString = myString.Shorten(5);
 // Aşağıda ise genişletilmiş yöntem çağırılmıştır.
 var newString2 = StringExtensions.Shorten(myString, 5);
 }
}

static class StringExtensions
{
 public static string Shorten(this string text, int length)
 {
 return text.Substring(0, length);
 }
}

```

Bir sınıfın kaynak kodu mevcut değilse veya mevcut sınıfta değişiklik yapma izniniz yoksa, genişleme yöntemlerinin mevcut sınıfa yeni yöntemler ekleyerek bir sınıfın işlevselliğini genişletmek için bir yaklaşım olarak kullanılabileceğini söyleyebiliriz. Hatırlamanız gereken en önemli nokta, genişleme yöntemlerinin statik bir sınıfın statik yöntemleri olduğu, ancak genişletilmiş tipteki örnek yöntemlermiş gibi çağrılacaklarıdır.

## 8.19 Sınıf Çeşitleri

### §8.19.1 Soyut Sınıf

**Soyut sınıflar** (**abstract class**) da tanımlanabilir ancak bu sınıflardan nesne imal edilemez. Yani bu sınıfın bir **örneği** (**instance**) **olamaz!** Daha çok **kavramsal olarak kullanılır!** Soyut bir sınıfın en az bir soyut yöntemi (**abstract method**) olur. Soyut sınıflar hangi davranışı göstereceğini söyler ama nasıl davranış göstereceğini

anlatmaz. Bu nedenle **abstract** kelimesiyle tanımlanan **soyut yöntem** (**abstract method**) için gövde tanımlanmaz. Aşağıda soyut bir sınıf tanımı verilmiştir;

```
//Soyut soyutnesne = new Soyut(); //HATA: Soyut sınıftan nesne imal edilemez!

abstract class Soyut
{
 public abstract void Yöntem(); //Soyut Yöntemler için gövde tanımlanmaz!
}
```

### §8.19.2 Somut Sınıf

**Somut sınıflardan** (**concrete class**) nesne imal edilebilir yani örneği (**instance**) olabilir. Bu sınıflarda davranış ve durumlar eksiksiz tanımlıdır. Bir sınıf tanımlarken soyut bir sınıfa genel davranış ve durumları toplamak ve ondan miras alarak somut sınıfları tanımlamak her zaman iyi bir seçimdir.

```
public class Somut : Soyut
/* Somut (concrete) sınıf, her zaman soyut sınıftan miras almak zorunda değildir. */
{
 public override void Yöntem()
 /* soyut yöntem, soyut olan ve miras alınan Yöntem" metodunu * geçersiz kılar (override)
 Ve mutlaka ete kemiğe bürünen bir davranışı tanımlamak zorundadır. */
 {
 /* soyut olarak gösterilecek davranış burada kodlanır. */
 }
}
```

### §8.19.3 Taban Sınıf

**Taban sınıf** (**base class**), miras alınan sınıf veya genel özellik ve davranışların toplandığı yani genelleştirmenin yapıldığı genel sınıf (**general class**) veya ebeveyn (**parent**) sınıftır.

```
public class TabanSınıf
{
 public virtual void Yöntem()
 {
 }
}
```

### §8.19.4 Türemiş Sınıf

**Türemiş sınıf** (**derived class**), miras alan sınıf veya davranış ve durumlar hakkında uzmanlaşmış yani uzman (**special class**) veya çocuk (**child class**) sınıftır. Türemiş ya da miras almış sınıf.

```
public class TüreyenSınıf : TabanSınıf
{
 public override void Yöntem()
 {
 }
}
```

### §8.19.5 Kök Sınıf

**Kök sınıf** (**root class**), babası olmayan ya da yetim sınıf olup aynı zamanda baba sınıftır.

### §8.19.6 Yaprak Sınıf

**Yaprak sınıf** (**leaf class**), kendisinden henüz miras alan bir sınıf olmayan bir sınıftır. Kısaca türememiş sınıftır.

### §8.19.7 Kısır Sınıf

**Kısır sınıf** (**sealed class**) kendisinden türeyen bir sınıf tanımlayamayan bir sınıftır. **Kısır yöntem** (**sealed method**) ise türeyen sınıfta geçersiz kılınamayan (**override**) yöntemdir.

```
public class Sınıf
{
 public virtual sealed void Yöntem()
 { // Kısır bir Yöntem tanımlanıyor
 }
}
```

```

public class Çocuk : Sınıf
{
 public override void Yöntem() /* Derleme Sırasında Hata: Kısır yöntem geçersiz kılınamaz
 → (override edilemez) */
 {
 }
}
public sealed class Kısır //Kısır (sealad) sınıf: miras bırakmaz.
{
 public Kısır() { }
 public void Yöntem()
 {
 }
}
public class Türemiş : Kısır /* Derleme Sırasında Hata: Kısır sınıftan miras alınamaz. */
{
}

```

### §8.19.8 Tekil Sınıf

**Tekil sınıftan (singleton class)** yalnızca bir nesne imal edilebilir. Yani yalnızca bir örneği vardır. Bunun olabilmesi için mahrem (**private**) yapıcısı olması gerekir. Tekil Nesne başlığında incelenecektir.

### §8.19.9 Yardımcı Sınıf

**Yardımcı sınıf (utility class)**, üyeleri statik olan sınıflardır. Ayrıca evrensel yapılandırma ayarlarını (**configuration settings**) veya değişmezleri (**literal**) depolamak için kullanılabilir. Bir kaynak havuzunu (önbellek, veri tabanı bağlantı havuzu, vb.) yönetmek ve örnekler arasında paylaşılan bir günlük sistemi uygulamak için yararlıdır. Bunların dışında, izleme yöntemi (**trace method**) çağrılarını için de kullanılabilir. Yardımcı sınıflar statik (**static**) sınıflardır ve bu sınıflardan nesne imal edilemez.

### §8.19.10 Kısmi Sınıf

**Kısmi sınıf (partial class)**, bir sınıfın farklı yerlerde parçalar halinde tanımlanmasıdır. Kısmi sınıf bir dosya içinde tanımlanabileceği gibi kod tabanında herhangi bir yerde tanımlanabilir.

```

Kişi kişi = new Kişi();
kişi.AdınıSöyle();
kişi.SoyadınıSöyle();
kişi.AdınıTamSöyle();

```

```

partial class Kişi
{
 string Adı { get; set; }
 public void AdınıSöyle()
 {
 Console.WriteLine($"Adım:{Adı}");
 }
}

```

//Aynı dosyanın başka bölümünde ya da projenin bir başka kaynak dosyasında olabilir

```

partial class Kişi
{
 public Kişi()
 {
 Adı = "İlhan";
 Soyadı = "Özkan";
 }
 string Soyadı { get; set; }
 public void SoyadınıSöyle()
 {
 Console.WriteLine($"Soyadı:{Soyadı}");
 }
 public void AdınıTamSöyle()

```



```

{
 Console.WriteLine($"Adı ve Soyadı:{Adı} {Soyadı}");
}
}

```

## 8.20 Nesnelerle İlgili İşleçler

### §8.20.1 Tip Kontrol İşleci

**is** işleci (**is operator**) bir nesnenin hangi veri tipinde (bir sınıfa ait olup olmadığını) kontrol eder. Nesnenin kendisini değiştirmez, sadece "Bu nesne bu veri tipinde mi?" sorusunu sorar.

```

object obj = "Hello, World!";
if (obj is string)
{
 Console.WriteLine("obj nesnesinin veri tipi stringdir."); // Bunu konsola yazar
}

```

Bu işleç, C#7.0+ sonrasında desen seçimi (**pattern matching**) mantığıyla çalışır. Modern C# sürümlerinde **is** operatörü ile hem kontrol yapıp hem de aynı satırda değişken oluşturabilirsiniz.

```

if (nesne is string metin) // hem tip kontrolü hem de metin değişkeni oluşturuldu
{
 // Eğer nesne string ise, 'metin' isminde bir değişken oluşturulur
 Console.WriteLine($"Metnin uzunluğu: {metin.Length}");
}

```

Sınıf hiyerarşisinde bu işlecin nasıl kullanıldığını anlatan örnek aşağıda verilmiştir;

```

public class Sekil { }
public class Kare : Sekil { public void KareCiz() { } }

public void SekilIsle(Sekil s)
{
 if (s is Kare k) // s bir Kare mi? Öyleyse 'k' değişkeniyle kullan.
 {
 k.KareCiz();
 }
}

```

### §8.20.2 Güvenli Tip Dönüşüm İşleci

**Güvenli tip dönüşüm işleci** olan **as** işleci (**as operator**), bir nesneyi belirli veri tipine dönüştürmeye çalışır. Eğer dönüşüm başarısız olursa program hata fırlatmaz, bunun yerine geriye **null** döndürür. Dönüşüm, sadece referans tipleri ve boş olabilir (**nullable**) tipler ile çalışır. **try-catch** bloğuna gerek kalmadan dönüşüm denemesi yapmanızı sağlar.

```

object veri = 123;
string metin = veri as string; // int, string'e dönüşemez.
if (metin == null)
{
 Console.WriteLine("Dönüşüm başarısız, veri bir string değil.");
}

```

Neden **int x = nesne as int;** şeklinde bir kod derleme hatası verir? Çünkü **int** bir değer tipidir (value type) ve asla **null** olamaz. **as** işleci ise başarısızlık durumunda **null** döndürmek üzere tasarlanmıştır. Değer tiplerinde güvenli dönüşüm yapmak istiyorsanız **is** operatörünü kullanmalı veya **Nullable<int>** yani **int?** tercih etmelisiniz.

### §8.20.3 Eşitlik İşleci

C# dilinde iki farklı eşitlik türü vardır. Birincisi referans eşitliği, ikincisi ise değer eşitliğidir. Değer eşitliği, eşitliğin yaygın olarak anlaşılan anlamıdır. İki nesnenin aynı değerleri içermesi anlamına gelir.

```

object a = new object();
object b = a;
System.Object.ReferenceEquals(a, b); //true

```

```

Console.WriteLine((2 + 2) == 4); // değer eşitliği: true
object s = 1;
object t = 1;
Console.WriteLine(s == t); //referans eşitliği, farklı nesneler: false

string a = "hello"; //const
string b = String.Copy(a); //instance
string c = "hello"; //const
Console.WriteLine(a == b); // metin değerlerini karşılaştırma: true

Console.WriteLine((object)a == (object)b); //metin referanslarını karşılaştırma: false
Console.WriteLine((object)a == (object)c); //metinlerin ikisi de sabit: true

```

## 8.21 İsim Uzayları

**İsim uzayları** (**namespaces**), projeyi mantıksal bölümlere ayırmak ve **isim çakışmalarını** (**naming collisions**) önlemek için kullanılan hayati bir yapıdır. Büyük bir kütüphaneyi binlerce kitabın olduğu bir oda yerine, kategorilere ayrılmış raflar olarak düşünün; isim uzayları işte bu raflardır.

### §8.21.1 İsim Uzayı Nedir?

Bir projede aynı isimde iki farklı sınıf (Müşteri, Dosya, vb.) tanımlamanız gerekebilir. Örneğin, hem "Muhasebe" modülünde hem de "Satış" modülünde bir Fatura sınıfı olabilir. İsim uzayları, bu sınıfları birbirinden ayırmanızı sağlar:

```

Şirketim.Muhasebe.Fatura
Şirketim.Satış.Fatura

```

**İsim Uzayı Tanımlama ve Kullanımı** Bir sınıfın hangi isim uzayına ait olduğunu **namespace** anahtar kelimesi ile belirtirsiniz.

```

namespace VeriYönetimi.Modeller
{
 public class Kullanıcı
 {
 public string Ad { get; set; }
 }
}

```

Bu sınıfa başka bir yerden erişmek için iki yolunuz vardır:

1. Tam Adını Kullanmak (Fully Qualified Name)  
VeriYönetimi.Modeller.Kullanıcı k = new VeriYönetimi.Modeller.Kullanıcı();
2. **using** Direktifi ile Dahil Etmek (Önerilen)  
**using VeriYönetimi.Modeller;**  
// Artık doğrudan sınıf adını yazabilirsiniz  
Kullanıcı k = new Kullanıcı();

### §8.21.2 Hiyerarşik İsim Uzayları

İsim uzayları, nokta işleci (**dot operator**) (.) ile hiyerarşik olarak iç içe dallandırılabilir. Bu, projenin büyüklüğüne göre derinleşebilir:

```

İsimUzay1.KullanıcıArayüzü.Kullanıcı.Kullanıcı kullanıcı1 = new
→ İsimUzay1.KullanıcıArayüzü.Kullanıcı.Kullanıcı
{
 Ad = "Ahmet",
 Soyad = "Yılmaz",
 Tip = İsimUzay1.KullanıcıArayüzü.Kullanıcı.KullanıcıTipi.ADMIN
};
kullanıcı1.Ad = "Ali"; // Değişiklik yapıldı

namespace İsimUzay1
{
 namespace KullanıcıArayüzü
 {

```

```

namespace Kullanıcı {
 enum KullanıcıTipi { ADMIN, RAPORTÖR, İZLEYİCİ }
 class Kullanıcı
 {
 public string Ad { get; set; }
 public string Soyad { get; set; }
 public KullanıcıTipi Tip { get; set; }
 }
}

```

### §8.21.3 Dosya Kapsamlı İsim Uzayları

C#10.0+ ile birlikte, dosyanın tamamını kapsayan isim uzayları (**file-scoped namespaces**) geldi. Bu sayede gereksiz süslü parantezlerden `{ }` ve girintilerden kurtulabilirsiniz:

```

// Eski yöntem (Block-scoped)
namespace Projem {
 class A
 {
 }
}

// Yeni yöntem (file-scoped) - Daha temiz kod!
namespace Projem;
class A
{
}

```

### §8.21.4 İsim Uzaylarında Takma İsim

**Alias Kullanımı:** İsimler çok uzunsa veya çakışma varsa **takma isim** (**alias**) verilir.

```

using System.Text;

var sb = new StringBuilder(); //var sb = new System.Text.StringBuilder(); yerine
 using static: Bir sınıfın statik yöntemler sınıf adını yazmadan erişmenizi sağlar.
using static System.Math; //-> Artık Math.PI yerine sadece PI yazabilirsiniz.

```

### §8.21.5 Standart .NET İsim Uzayları

Şu ana kadar üstü örtük olarak birçok isim uzayını kullandık. Bunlar;

- ◊ **System:** Temel veri türleri, konsol işlemleri.
- ◊ **System.Collections.Generic:** Listeler, sözlükler.
- ◊ **System.Linq:** Veri sorgulama araçları.
- ◊ **System.IO:** Dosya okuma ve yazma.

Bir sınıfı herhangi bir **namespace** bloğu içine almazsak o sınıf **küresel isim uzayı** (**global namespace**) içinde yer alır. Küçük projelerde sorun olmasa da, büyük projelerde bu durum "isimlendirme kirliliğine" yol açar ve profesyonel yazılım geliştirmede asla önerilmez.

## 8.22 Düşün ve Çöz

- ◊ **Düşün:** Kapsülleme (**encapsulation**) neden yalnızca alanları **private** yapmaktan ibaret değildir? Özel-lik (**property**) kullanımının **getter/setter** yöntemlerine göre C#'a özgü avantajları nelerdir?
- ◊ **Düşün:** Miras alma (**inheritance**) ile kompozisyon (**composition**) 'is-a' ve 'has-a' ilişkileri üzerinden karşılaştırınız. Tasarım Desenleri literatüründeki 'Kalıtım yerine kompozisyonu tercih et' ilkesini somut bir örnekle açıklayınız.
- ◊ **Düşün:** C#'ta çok biçimlilik (**polymorphism**) **virtual/override** mekanizmasıyla nasıl sağlanır? Erken bağlama (**early binding**) ile geç bağlama (**late binding**) arasındaki farkı bir hayvan sınıfı hiyerarşisi örneğiyle gösteriniz.

- ◇ **Düşün:** Arayüz (**interface**) ile soyut sınıf (**abstract class**) arasındaki fark nedir? C#8 ile arayüzlere varsayılan yöntem gövdesi eklenebilmesi bu ikisi arasındaki sınırı nasıl değiştirmiştir?
- ◇ **Düşün:** Statik üyeler (**static members**) nesne yönelimli tasarımın hangi ilkesiyle çelişebilir? Utility sınıflarını tamamen statik yapmak ne zaman mantıklı, ne zaman sorunludur?
- ◇ **Düşün:** C#'ta sonlandırıcı (**finalizer/destructor**) kullanımı neden önerilmez? **IDisposable** arayüzü ve **using** bloğu, sonlandırıcı yerine neden tercih edilmelidir?
- ◇ **Düşün:** **new** anahtar kelimesi gölgeleme (**shadowing**) amacıyla kullanıldığında çok biçimliliği nasıl kırar? **override** ile **new** arasındaki farkı bir test senaryosunda gözlemleyebileceğiniz bir örnek kurunuz.
- ◇ **Çöz:** Bir kütüphane sistemi modelleyin: Materyal (soyut), Kitap, Dergi ve DVD sınıflarını oluşturun. Her birinin ödünç alınabilme süresi farklı olsun. Ödünç işlemini çok biçimli bir yöntemle gerçekleştirin.
- ◇ **Çöz:** Bir banka hesabı hiyerarşisi oluşturun: **Hesap** (taban), **ÇekingHesabı** ve **TasarrufHesabı** (türemiş). Her birinde para yatırma, çekme ve bakiye sorgulama işlemlerini uygulayın. Tasarruf hesabında minimum bakiye kısıtlamasını ekleyin.
- ◇ **Çöz:** **IYazılabilir** ve **IYazdırılabilir** adlı iki arayüz tanımlayın. Belge, Fatura ve Rapor sınıflarını bu arayüzleri farklı kombinasyonlarda uygulayarak oluşturun.
- ◇ **Çöz:** Bir ürün katalog sistemi tasarlayın: Ürün sınıfı kapsülleme kurallarına tam uygun olsun (**private** alanlar, **public** özellikler, doğrulama mantığı). Fiyat sıfır veya negatif girildiğinde istisna fırlatılsın.
- ◇ **Çöz:** Genişleme yöntemi (**extension method**) kullanarak **string** sınıfına 'Tersine çevir', 'Sözcük sayısını bul' ve 'İlk harfi büyüt' yöntemlerini ekleyiniz.
- ◇ **Düşün:** Bir televizyonu nesne olarak düşünürsek; televizyonun 'kanal numarası' bir durum (**state**) mudur yoksa bir davranış (**behavior**) mıdır? Peki 'kanal değiştirme' işlemi hangisidir?
- ◇ **Düşün:** Neden tüm kodlarımızı tek bir **namespace** içinde yazmıyoruz? İsim uzaylarının (**namespaces**) büyük ölçekli projelerdeki en büyük faydası nedir?
- ◇ **Düşün:** Bir sınıfın içinde hem **static** hem de **non-static** metotlar olabilir mi? Hangisi nesne üretilmeden çağrılabilir?

## 9. Bölüm

# Temsilciler, Lambda İfadeleri ve Olaylar

### 9.1 Temsilciler

**Temsilci** (**delegate**), yöntemleri dolaylı olarak çağırmanın bir yoludur. Saf nesne yönelimli programlama dillerinde (Java ve C#), yarattığı karmaşadan ötürü gösterici (**pointer**) kullanımı tercih edilmez. Bu sebeple C++ dilindeki fonksiyon göstericisi (**function pointer**) yerine, ondan daha yetenekli olan temsilciler kullanılır.

Temsilciler, C#'taki olay tabanlı programlamanın (**event-driven programming**) ve fonksiyonel programlama (**functional programming**) yeteneklerinin temelidir.

Temsilciler, yöntemlerin (**method**) bellek adreslerini tutan tip güvenli (**type-safe**) yapılardır. Temsilciye bağlanacak yöntemin imzası (**signature**) temsilci tanımında verilir. Yöntem imzası kavramı 8.4.9 başlığında verilmiştir. Bir temsilciye ilk yöntem atama (=) işleciyle bağlanır. Geri kalan yöntemler += işleci ile bağlanır. **Bir temsilciyi çağırdığınızda, ona bağlı olan tüm yöntemler eklendiği sırayla çalıştırılır FIFO (first in first out)!**

```
// 1. Temsilci Tanımı
public delegate void Yap(string pİş); //(İmza: Geridönüş tipi: void, parametre: string olacak!)
```

```
//Temsilciler Sınıflar Olmadan Çalışmaz!. Bu nedenle "Main" Fonksiyonu içeren bir sınıf mutlaka
→ olmalı!
```

```
public class Program
{
 public static void Main()
 {
 YöntemliSınıf nesne = new YöntemliSınıf();
 // 2. Temsilci Örneği Oluşturma ve İlk Yöntemi Atama
 Yap pİşyap = nesne.MusluğuAç;
 // 3. Çoklu Gönderim (Multicast): Yeni yöntemler ekleme
 pİşyap += nesne.MusluğuKapat;
 pİşyap += YöntemliSınıf.BulaşıkYık; // Statik yöntem ekleme
 // 4. Temsilciyi Çağırma (Invocation)
 Console.WriteLine("--- Temsilci Çalıştırılıyor ---");
 pİşyap("İlhan Özkan");
 // 5. Yöntem Çıkarma (İstenirse bir yöntem listeden çıkarılabilir)
 pİşyap -= nesne.MusluğuAç;
 }
}

/* Program Çıktısı:
--- Temsilci Çalıştırılıyor ---
İlhan Özkan Musluğu Açtı.
İlhan Özkan Musluğu Kapattı.
İlhan Özkan Bulaşık Yıkadı.
*/
```

```
public class YöntemliSınıf
{
 // Örnek (Instance) Yöntemi
 public void MusluğuAç(string pİsim)
 {
 Console.WriteLine($"{pİsim} Musluğu Açtı.");
 }

 public void MusluğuKapat(string pİsim)
 {
 Console.WriteLine($"{pİsim} Musluğu Kapattı.");
 }
}
```

```

 }

 // Statik Yöntem
 public static void BulaşıkYıkama(string pİsim)
 {
 Console.WriteLine($"{pİsim} Bulaşık Yıkadı.");
 }
}

```

## 9.2 Lambda İfadeleri

### §9.2.1 Lambda İfadesi Nedir?

Bir **lambda ifadesi** (**lambda expression**), basit fonksiyon nesneleri oluşturmak için özlü bir yol sağlar. **lambda** adı, *Alonzo Church* tarafından 1930'larda mantık ve hesaplanabilme hakkındaki soruları araştırmak için icat edilen matematiksel bir form olan lambda hesabından gelir. lambda hesabı, fonksiyonel bir programlama dili olan LISP'in temelini oluşturmuştur.

C# dilindeki lambda ifadeleri, kodunuzu daha kısa, daha okunabilir ve daha fonksiyonel bir hale getirir. lambda ifadeleri, aslında **isimsiz yöntemler** (**anonymous method**) yazmanın en kestirme yoludur. lambda ifadelerinde girdi parametrelerinin veri tipini belirtmenize gerek yoktur. Bu da kullanımı daha esnek hale getirir.

### §9.2.2 Lambda Nasıl Tanımlanır?

lambda ifadesi iki bölüme ayrılır, sol taraf girdidir, sonrasında **lambda işleci** (**lambda operator**) (**=>**) bulunur. Sağ taraf ise ifadedir (**expression**).

**Talimatlar** (**statement**), yapılması gerekeni kısa ve net olarak belirten mantıksal cümlelerdir (**logical sequences**). Talimatlar anahtar kelimeler içerirler ve noktalı virgül karakteri ile biter. **İfadelerin** (**expression**) ise yalnızca sabit, değişken ve işleç (**operator**) içeren talimatları olduğu daha önce anlatılmıştı.

Lambda ifadeleri, temsilcilere veya ileride göreceğimiz LINQ sorgularına atanan anonim fonksiyonlardır. Söz diziminde kullanılan lambda işleci, "şuna gider" veya "için şunu yap" anlamı taşır. Genel formül aşağıda verilmiştir;

$$(parametreler) \Rightarrow \{yapılacak\ işlemler\}$$

Lambda ifadelerinin söz dizimi iki türlüdür:

1. **İfade lambda'ları** (**expression lambdas**): Sadece tek bir işlem yapan ve sonucu otomatik dönen kısa yapılardır.  
(x) => x \* x; // x parametresini alır ve karesini döner.
2. **Talimat lambda'ları** (**statement lambdas**): Süslü parantez { } kullanarak birden fazla işlem satırı içeren yapılardır.  
(x, y) => {  
    **int** toplam = x + y;  
    Console.WriteLine(toplam);  
    **return** toplam;  
};

### §9.2.3 Temsilcilerle Lambda Kullanımı

Daha önce örneğini verdiğimiz **Yap** temsilcisini hatırlayalım. Onu lambda ile aşağıdaki gibi daha kısa yazabiliriz. Örnekte başka temsilci örneği de bulunmaktadır;

```

public delegate void Yap(string pİş); // Temsilci tanımı

public delegate int intGirdiAlanveİntDönenBirTemsilci(int input);

class Program {
 static void Main() {
 // Eski yöntem (Anonim yöntem - C#2.0)
 Yap işlem1 = delegate(string mesaj) { Console.WriteLine(mesaj); };
 // Yeni yöntem (Lambda ifadesi - C#3.0+)
 Yap işlem2 = (mesaj) => Console.WriteLine(mesaj);
 işlem2("İlhan Özkan işe başladı.");

 intGirdiAlanveİntDönenBirTemsilci ikiİleÇarp = x => x * 2;
 }
}

```

```

intGirdiAlanveIntDönerBirTemsilci karesiniAl = x => x * x;
intGirdiAlanveIntDönerBirTemsilci küpünüAl = x => x * x * x;
int sayi = 10;
Console.WriteLine("10*2 = " + ikiİleÇarp(10)); // 20
Console.WriteLine("sayi^2 = " + karesiniAl(sayi)); //100
Console.WriteLine("sayi^3 = " + küpünüAl(sayi)); //1000
}
}

```

Anonim yöntemler (**anonymous method**) bir kod bloğunu temsilci (**delegate**) parametresi olarak geçirmek için bir teknik sağlar. Bunlar bir gövdeye sahip ancak adı olmayan yöntemlerdir ve lambda ifadeleri ile tanımlanırlar.

```

delegate int IntOp(int lhs, int rhs);

class Program
{
 static void Main(string[] args)
 {
 // C#2.0 Tanımlaması
 IntOp add = delegate(int lhs, int rhs)
 {
 return lhs + rhs;
 };
 // C#3.0 Uzun Tanımlama
 IntOp mul = (lhs, rhs) =>
 {
 return lhs * rhs;
 };
 // C#3.0+ Kısa Tanımlama
 IntOp sub = (lhs, rhs) => lhs - rhs;

 // Her bir anonim yöntemi çağıralım:
 Console.WriteLine("2 + 3 = " + add(2, 3));
 Console.WriteLine("2 * 3 = " + mul(2, 3));
 Console.WriteLine("2 - 3 = " + sub(2, 3));
 }
}

```

### §9.2.4 Yerleşik Temsilciler

Her seferinde yeni bir **delegate** tanımlamak yerine, .NET'in hazır yapılarını kullanabiliriz.

- ◊ **Action<T>**: Geriye değer döndürmeyen (**void**) yöntemler için kullanılır.
- ◊ **Func<T, TResult>**: Geriye mutlaka bir değer döndüren yöntemler için kullanılır.

```

// Parametre olarak int alır, geriye string döner
Func<int, string> notaCevir = (puan) => puan >= 50 ? "Geçti" : "Kaldı";
string sonuc = notaCevir(75); // "Geçti"

```

```

Action line = () => Console.WriteLine("-----"); //Parametresiz
line(); //Action delegate'leri, geriye değer döndürmezler!

```

```

Func<double, double> cube = x => x * x * x; //Tek Parametrelili
Console.WriteLine(cube(3));
//Func delegate'leri, geriye değer döndürürler!
/* Yukarıdaki kodun açılımı:
double cube(double x) {
 return x * x * x;
}
*/

```

```

Func<int, double, double> kat = (x,y) => x * y; //İki Parametrelili
Console.WriteLine(kat(3,6.0));

```



```

/* Yukarıdaki kodun açılımı
double kat(int x, double y) {
 return x * y;
}
*/

//Veri tipi içeren parametrelili
Func<int, string, bool> MetinKarakterSayısıVerilendenBüyükMü = (int x, string s)
=> s.Length > x;
Console.WriteLine(MetinKarakterSayısıVerilendenBüyükMü(3, "Merhaba"));

//Parametreler Göz Ardı Edildi
Func<int, int, int> constant = (_, _) => 61;
Console.WriteLine(constant(3, 5)); // Output: 61

```

### §9.2.5 Lambda İfadelerinin Gücü

Lambda ifadelerinin asıl parladığı yer LINQ sorgularıdır. Bu konuyu ileride detaylı göreceğiz ama kısa bir örnek verilebilir. Bir diziyi filtrelemek hiç bu kadar kolay olmamıştı:

```

int[] sayılar = { 1, 5, 8, 12, 15, 20 };
var çiftSayılar = sayılar.Where(s => s % 2 == 0); // LINQ ve lambda kullanımı: "s öyle ki s'in
→ 2'ye bölümünden kalan 0 olsun"
foreach (var sayı in çiftSayılar)
 Console.WriteLine($"{sayı} "); // 8 12 20

```

Bir lambda ifadesi, tanımlandığı yöntemin içindeki yerel bir değişkene erişebilir mi? Eğer erişirse ve o değişkenin değeri metodun ilerleyen kısımlarında değişirse ne olur? Evet, buna **kapatma** (**closure**) denir. lambda, dışarıdaki değişkeni "yakalar" (**capture**). Ancak dikkat edilmelidir ki; lambda değişkenin o anki değerini değil, referansını yakalar. Bu yüzden dışarıdaki değişken değişirse lambdanın sonucu da değişecektir.

### §9.2.6 Anonim Değişkenlerle Lambda İfadeleri

Bir lambda ifadesini tanımlarken var kelimesi ile temsilci tipi otomatik olarak çıkarılır. Aslında genel olarak bir ifadenin sonucunda ortaya çıkan veri tipinin çıkarımı otomatik olması isteniyorsa anonim değişkenler kullanılır.

```

var IncrementBy = (int source, int increment = 1) => source + increment;
Console.WriteLine(IncrementBy(5)); // 6
Console.WriteLine(IncrementBy(5, 2)); // 7

```

Ayrıca, açıkça yazılmış bir parametre listesindeki son parametre olarak, parametre dizileri veya koleksiyonları olan lambda ifadelerini de tanımlanabilir;

```

var sum = (params IEnumerable<int> values) =>
{
 int sum = 0;
 foreach (var value in values)
 sum += value;
 return sum;
};
var empty = sum();
Console.WriteLine(empty); // 0
var sequence = new[] { 1, 2, 3, 4, 5 };
var total = sum(sequence);
Console.WriteLine(total); // 15

```

## 9.3 Olaylar

Nesneler, olaylardan (**event**) etkilenirler ve olaylar karşısında tepki verir ya da davranış (**behavior**) gösterirler. Olayı yayan (**publish**) nesneye diğer tüm nesneler tepki vermezler. Bu nedenle tepki verecek nesneler, davranış göstereceği olaya önceden abone (**subscribe**) olurlar. Abone olunan olay gerçekleştiği zaman tepki (**handler**) verirler.

### §9.3.1 Olay Tanımlama ve Olay Tetiklendiğinde Tepki Verme

```
//ANA PROGRAM
// 1. Temsilci (Delegate) Tanımı. Olaya abone olmak için kullanılır!
public delegate void OlayOluncaAboneOlunacakYöntem(Kişi s);
internal class Program
{
 static void Main(string[] args)
 {
 Kişi kişi = new Kişi("İlhan", 50);
 Dinleyici dinleyici = new Dinleyici();
 dinleyici.AboneOl(kişi); // Abone olma işlemi
 // Değişiklikler olayı tetikleyecek
 kişi.Yaş = 51;
 kişi.Yaş = 52;
 }
}

//SINIFLAR
public class Kişi
{
 public event OlayOluncaAboneOlunacakYöntem? YaşDeğiştirdi; //Başlangıçta null olmalı!
 /*
 OlayOluncaAboneOlunacakYöntem? olarak nullable tanımlamak istemiyorsak aşağıdaki kodları
 ↪ yazabiliriz:
 1. Boş bir lambda ile başlatıldığı için asla null olmaz:
 public event OlayOluncaAboneOlunacakYöntem YaşDeğiştirdi = delegate { };
 2. Aynı şeyin modern C3.0+ yazımıyla:
 public event OlayOluncaAboneOlunacakYöntem YaşDeğiştirdi = _ => { };
 */
 private int _yaş;
 public Kişi(string pİsim, int pYaş)
 {
 this.İsim = pİsim;
 /*
 DİKKAT: Burada doğrudan '_yaş' mahrem alanına atama yapıyoruz.
 Eğer 'this.Yaş = pYaş' denip Özellik değiştirilseydi, henüz kimse abone (+=)
 olmadan olay tetiklenmeye çalışılır ve hata alabilirsiniz.
 */
 this._yaş = pYaş;
 }
 public string İsim {get; set;}
 public int Yaş
 {
 get => _yaş;
 set
 {
 if (_yaş != value)
 {
 _yaş = value;
 YaşDeğiştirdi?.Invoke(this); // 'Null-conditional' operatörü (?.) kullanarak daha
 ↪ güvenli tetikleme:
 }
 }
 }
}

public class Dinleyici
{
 public void AboneOl(Kişi k)
 {
 k.YaşDeğiştirdi += Kişinin_yaşıDeğişinceNeYapılacak; // += ile olaya kayıt olunur
 }
}
```

```

private void Kişinin_yaşıDeğişinceNeYapılacak(Kişi kişi)
{
 Console.WriteLine($"{kişi.İsim} adlı kişinin yeni Yaşı: {kişi.Yaş}");
}

```

### §9.3.2 Yerleşik Olaylar

C# dilinde yerleşik olarak olaylar parametre alacak şekilde aşağıdaki gibi tanımlanır;

```
public event EventHandler<EventArgs> EventName;
```

<EventArgs> için jenerik başlığını inceleyebilirsiniz. Olaylara tepki verecek yöntemler aşağıdaki gibi tanımlanır;

```

public void HandlerName(object sender, EventArgs args)
{
 /* Handler logic */
}

```

Olaya abonelik de aşağıdaki gibi yapılır;

```
EventName += HandlerName; // Olaya Abone Olma
```

Eğer bir **Windows Form** projesinde yazıyorsanız, Form üzerinde sağ tıklayıp özelliklerine giderseniz form ile ilgili olay listesini görebilirsiniz. Herhangi birine çift tıklayarak ona ilişkin tepki yöntemi otomatik olarak kodlanacak şekilde oluşacaktır. Aşağıda örneği verilmiştir;

```

//Form1.cs
namespace WinFormsApp1
{
 public partial class Form1 : Form
 {
 public Form1()
 {
 InitializeComponent();
 }
 private void Form1_Load(object sender, EventArgs e)
 {

 }
 private void Form1_Click(object sender, EventArgs e)
 {

 }
 private void Form1_DoubleClick(object sender, EventArgs e)
 {

 }
 }
}

```

Yukarıda verilen **Kişi** sınıfı örneğini yerleşik <EventArgs> ile aşağıdaki gibi yeniden yazabiliriz;

```

//ANA PROGRAM:
internal class Program
{
 static void Main(string[] args)
 {
 Kişi kişi = new Kişi("İlhan", 50);
 Dinleyici dinleyici = new Dinleyici();

 dinleyici.AboneOl(kişi);

 kişi.Yaş = 51; // Çıktı: İlhan adlı kişinin yaşı değişti! 50 -> 51
 kişi.Yaş = 52; // Çıktı: İlhan adlı kişinin yaşı değişti! 51 -> 52
 }
}

```

```
// 1. Standart Temsilci Tanımı
// object sender: Olayı tetikleyen nesne
// YaşDeğişmeOlayıArgs args: Olay ile ilgili veriler
public delegate void YaşDeğişinceAboneOlunacakYöntem(object sender, YaşDeğişmeOlayıArgs args);

//SINIFLAR:
// 2. Veri Taşıyıcı Sınıf (EventArgs)
public class YaşDeğişmeOlayıArgs : EventArgs
{
 public int MevcutYaş { get; }
 public int YeniYaş { get; }

 public YaşDeğişmeOlayıArgs(int mevcutYaş, int yeniYaş)
 {
 MevcutYaş = mevcutYaş;
 YeniYaş = yeniYaş;
 }
}

// 3. Yayıncı Sınıf (Publisher)
public class Kişi
{
 public event YaşDeğişinceAboneOlunacakYöntem? YaşDeğişti;

 private int _yaş;
 public string İsim { get; set; }

 public Kişi(string pİsim, int pYaş)
 {
 İsim = pİsim;
 _yaş = pYaş; // Doğrudan field'a atama (Olay tetiklenmez)
 }

 public int Yaş
 {
 get => _yaş;
 set
 {
 if (_yaş != value)
 {
 int eskiYaş = _yaş; // Önce eski değeri saklıyoruz
 _yaş = value; // Sonra yeni değeri atıyoruz

 // Olayı argümanlarla birlikte tetikliyoruz
 YaşDeğişti?.Invoke(this, new YaşDeğişmeOlayıArgs(eskiYaş, value));
 }
 }
 }
}

// 4. Abone Sınıf (Subscriber)
public class Dinleyici
{
 public void AboneOl(Kişi k)
 {
 k.YaşDeğişti += KişininYaşıDeğişinceYapılacak;
 }

 private void KişininYaşıDeğişinceYapılacak(object sender, YaşDeğişmeOlayıArgs e)
 {
 // sender nesnesini Kişi tipine güvenli bir şekilde dönüştürebiliriz
 if (sender is Kişi k)
 {

```

```

 Console.WriteLine($"{k.İsim} adlı kişinin yaşı değişti! Eski: {e.MevcutYaş}, Yeni:
 ↪ {e.YeniYaş}");
 }
}

```

### §9.3.3 Olaylara Tepki Vermek İçin Lambda İfadeleri

Lambda ifadeleri olaylara tepki vermek için kullanılabilir. Bu durumda tepki gösterilecek olaya abone (**subscribe**) olduktan sonra abonelikten ayrılma söz konusu olmaz. Ana programı inceleyiniz!

**internal class Program**

```

{
 static void Main(string[] args)
 {
 Kişi kişi = new Kişi();
 Dinleyici dinleyici = new Dinleyici();
 dinleyici.AboneOl(kişi);
 kişi.Yaş = 20;
 kişi.Yaş = 30;
 Console.WriteLine("----- ");
 YaşDeğişinceAboneOlunacakYöntem tepki = (sender, args)
 => Console.WriteLine("Kişinin Yaşı Değişemez!!");
 kişi.yaşDeğişti += tepki; //kişi.yaşDeğişti -= tepki; talimatı yazılarak abonelikten
 ↪ çıkılabilir.
 kişi.yaşDeğişti += (sender, args) => Console.WriteLine("Kişinin Yaşı Savcılık Emriyle
 ↪ Değişebilir!"); // Bu tepki için aynı şey yapılamaz!
 kişi.Yaş = 40;
 }
}
public class YaşDeğişmeOlayıArgs : EventArgs
{
 public YaşDeğişmeOlayıArgs(int mevcutyaş, int yeniyaş)
 {
 this.MevcutYaş = mevcutyaş;
 this.YeniYaş = yeniyaş;
 }
 public int MevcutYaş { get; private set; }
 public int YeniYaş { get; private set; }
}
public delegate void YaşDeğişinceAboneOlunacakYöntem(object sender, YaşDeğişmeOlayıArgs args);
public class Kişi
{
 public Kişi()
 {
 yaş = 0;
 İsim = "İsimsiz";
 yaşDeğişti = null;
 }
 public event YaşDeğişinceAboneOlunacakYöntem yaşDeğişti;
 public string İsim { get; set; }
 private int yaş;
 public int Yaş
 {
 get { return yaş; }
 set
 {
 if (yaş != value)
 {
 if (yaşDeğişti != null)
 {
 YaşDeğişmeOlayıArgs args = new YaşDeğişmeOlayıArgs(yaş, value);
 yaşDeğişti(this, args);
 }
 }
 }
 }
}

```

```

 }
 yaş = value;
 }
}
}
}
public class Dinleyici
{
 /* "Dinleyici" sınıfından imal edilen nesneler,
 * "Kişi" sınıfından imal edilen nesnelere abone olabilir. */
 public void AboneOl(Kişi k)
 {
 k.yaşDeğiştirdi += this.kişininYaşDeğişinceYap;
 }
 public void kişininYaşDeğişinceYap(object sender, YaşDeğişmeOlayıArgs args)
 {
 Console.WriteLine($"{args.MevcutYaş} olan Kişinin Yaşı " +
 $"{args.YeniYaş} olarak Değişti!");
 }
}
}

```

Olaylara += ile yapılan aboneliklerin, nesneyle iş bitince -= ile sonlandırılmaması durumunda oluşabilecek bellek sızıntılarına yol açar!

### §9.3.4 Modern ve Standart Olay Kullanımı

C# dilinde her olay için yeni bir `delegate` tanımlamanıza gerek yoktur.

Bunun için .NET içindeki hazır `EventHandler<TEventArgs>` yapısını kullanarak kodunuzu daha da sadeleştirir bilirsiniz:

```

internal class Program
{
 static void Main()
 {
 Kişi ilhan = new Kişi("İlhan", 50);
 Dinleyici takipçi = new Dinleyici();
 takipçi.AboneOl(ilhan);
 // Değişiklikler otomatik olarak bildirilecek
 ilhan.Yaş = 55;
 ilhan.Yaş = 60;
 }
}
// 1. Olay Veri Sınıfı: EventArgs'tan türetilmelidir.
public class YaşDeğişmeOlayıArgs : EventArgs
{
 public int EskiYaş { get; }
 public int YeniYaş { get; }
 public YaşDeğişmeOlayıArgs(int eskiYaş, int yeniYaş)
 {
 EskiYaş = eskiYaş;
 YeniYaş = yeniYaş;
 }
}
// 2. Yayıncı Sınıf (Publisher)
public class Kişi
{
 // Standart EventHandler kullanımı: Ayrı bir delegate tanımına gerek duymaz.
 public event EventHandler<YaşDeğişmeOlayıArgs>? YaşDeğiştirdi;
 private int _yaş;
 public string İsim { get; set; }
 public Kişi(string isim, int yaş)
 {

```

```

 İsim = isim;
 _yaş = yaş;
 }
 public int Yaş
 {
 get => _yaş;
 set
 {
 if (_yaş != value)
 {
 int eski = _yaş;
 _yaş = value;
 // Olayı tetiklerken 'this' gönderen nesneyi,
 // yeni YaşDeğişmeOlayıArgs ise veriyi temsil eder.
 YaşDeğiştirdi?.Invoke(this, new YaşDeğişmeOlayıArgs(eski, value));
 }
 }
 }
}
// 3. Abone Sınıf (Subscriber)
public class Dinleyici
{
 public void AboneOl(Kişi k)
 {
 k.YaşDeğiştirdi += KişininYaşıDeğiştirdiğinde; // Olay aboneliği
 }
 // Standart imza: (object sender, TEventArgs e)
 private void KişininYaşıDeğiştirdiğinde(object? sender, YaşDeğişmeOlayıArgs e)
 {
 if (sender is Kişi k)
 {
 Console.WriteLine($"[BİLGİ] {k.İsim} adlı kişinin yaşı güncellendi.");
 Console.WriteLine($" => Eski Yaş: {e.EskiYaş}");
 Console.WriteLine($" => Yeni Yaş: {e.YeniYaş}\n");
 }
 }
}

```

## 9.4 Düşün ve Çöz

- ◇ **Düşün:** Bir temsilciye (**delegate**) üç farklı metot bağladınız. Eğer bu metotlardan ikincisi çalışma anında bir istisna (**exception**) fırlatırsa, üçüncü metot çalışmaya devam eder mi? Neden?
- ◇ **Düşün:** Neden olayları (**event**) doğrudan bir temsilci değişkeni (**delegate variable**) gibi **public** olarak tanımlayıp dışarıdan erişime açmıyoruz da **event** anahtar kelimesini kullanıyoruz? (Kısıtlama farkları nelerdir?)
- ◇ **Düşün:** **Action** ve **Func** arasındaki temel fark nedir? Hangisi geriye değer döndüren metotlar için kullanılır?
- ◇ **Düşün:** Temsilci (delegate) nedir ve C#’ta fonksiyon işaretçisi (function pointer) ihtiyacını nasıl karşılar? Nesne yönelimli bir dilde fonksiyon işaretçisine sahip olmanın faydası ne olabilir?
- ◇ **Düşün:** Çok noktaya yayın temsilcisi (**multicast delegate**) nasıl çalışır? Birden fazla yöntemi zincirlemenin hangi gerçek dünya senaryolarında faydalı olduğunu açıklayınız. Zincirden bir yöntem çıkarmak nasıl mümkündür?
- ◇ **Düşün:** Lambda ifadesi ile anonim yöntem (**anonymous method**) arasındaki söz dizimi farkı nedir? Lambda ifadelerinin okunabilirlik açısından üstünlüğünü bir örnek üzerinde gösterin.
- ◇ **Düşün:** **Func<T>**, **Action<T>** ve **Predicate<T>** yerleşik temsilcileri arasındaki farklar nelerdir? Hangisinin dönüş değeri vardır, hangisi yoktur? Gerçek bir senaryo üzerinden her birinin kullanım durumunu belirtiniz.
- ◇ **Düşün:** Olay ile temsilci arasındaki temel fark nedir? ‘event’ anahtar kelimesi olmadan bir temsilci alanı kullanmanın güvenlik riski nedir? Neden doğrudan temsilci yerine olay tercih edilir?
- ◇ **Düşün:** Olayların abonelik (**subscribe**) ve abonelik iptali (**unsubscribe**) yönetimi neden önemlidir? Olay aboneliğinin iptal edilmemesi bellek sızıntısına nasıl yol açar? Bu sorunu önlemenin yolları nelerdir?
- ◇ **Düşün:** **EventArgs** ve özel olay argümanı sınıflarının (**custom EventArgs**) rolü nedir?



Standart `EventHandler<TEventArgs>` imzasını kullanmanın avantajları nelerdir?

- ◇ **Düşün:** Lambda ifadelerinde dışarıdaki değişkenlerin yakalanması (**closure**) nasıl çalışır? Bir döngü içinde lambda ifadesi kullanıldığında ne tür tuzaklarla karşılaşılabilir? Bu tuzaktan nasıl kaçınılır?
- ◇ **Çöz:** Sıralama stratejisi dinamik olarak değiştirilebilen bir ürün listesi tasarlayınız: kullanıcı 'ada göre', 'fiyata göre' veya 'stoka göre' sıralama seçeneği sunulsun. Her sıralama kriterini bir `Func<Ürün, object>` temsilcisiyle ifade edin.
- ◇ **Çöz:** Fare tıklaması gibi davranan bir `Button` sınıfı oluşturunuz. `Clicked` adında bir olay tanımlayın; birden fazla dinleyici abone olabilsin ve olay tetiklendiğinde hepsi haberdar edilsin.
- ◇ **Çöz:** Bir sayı listesi üzerinde filtreleme ve dönüştürme işlemlerini lambda ifadesiyle yapan program yazınız: çift sayıları filtreleyin, karesini alın ve toplamı hesaplayın.  
Önce `foreach` ile, sonra `Func/Predicate` ile yazın; okunabilirlik farkını karşılaştırın.
- ◇ **Çöz:** `EventArgs`'tan türetilen `SicaklikDegistiEventArgs` sınıfını oluşturunuz. Bir Termometre sınıfı tanımlayın; sıcaklık belirli bir eşiği aştığında olay fırlatsın ve bir alarm sistemi bu olaya abone olsun.
- ◇ **Çöz:** `Action<string>` temsilcisini kullanarak birden fazla günlük (**log**) kaydedici (konsol, dosya, veritabanı simülasyonu) arasında seçim yapılabilen esnek bir günlükleme sistemi tasarlayınız.

## 10. Bölüm

# Sınıflar Arası İlişkiler ve SOLID İlkeleri

## 10.1 Unified Modeling Language

UML(**Unified Modeling Language**), nesne yönelimli yazılım geliştirme (OOP) ile birlikte gelişen bir modelleme aracıdır. Bu nedenle 8. bölümünde açıklanan nesne yönelimli birçok kavram, izleyen sayfalarda daha da pekişecektir. Adından da anlaşılacağı üzere nesneye yönelimli problem çözmenin kalbi, model oluşturmaktır.

Model, gerçek dünyadaki karmaşık problemin esaslarını soyut olarak önümüze koyar. Buradan hareketle UML dilini, basit bir dil, gösterim ya da söz dizimi gibi algılamalıyız. UML dilinin, yazılımı nasıl geliştireceğimizi kesin olarak belirtmediği göz önüne alınmalıdır.

UML, bir grafik modelleme dili olup yazılım sistemlerini oluşturan ana elemanları (**artifact**) çeşitli şekillerden oluşacak şekilde tanımlamak için oluşturulmuş bir kurallar bütünüdür. Örnek verecek olursak, bir mimari projeye ilişkin maket ne ise yazılım için UML de aynı şeyi ifade eder. Yani UML, söz konusu yazılımın neyi yapacağını anlatan çeşitli şekillerin anlamlı bir şekilde bir araya getirilmesini sağlayan bir standarttır.

Yazılım mühendisliğiyle (**software engineering**) birlikte 1989 ve 1994 yılları arasında elliden fazla modelleme dili kullanılmaya başlanmıştır. Bu nedenle bu dönem yöntem savaşlarının yaşandığı bir dönem olarak adlandırılır. Bu yöntemlerin çoğu kendi söz dizimini oluşturmuş olmakla birlikte kullandıkları dil elemanları açısından birbirleriyle benzerlik göstermektedirler.

Doksanlı yılların ortalarında üç yöntem, daha güçlü ve yaygın hale gelmiştir. Bu yöntemlerin her biri, diğerlerinin içerdiği elemanları da içeriyordu ancak her birinin diğerine karşı çeşitli güçlü özellikleri bulunuyordu. Bu yöntemler:

- ♦ Booch yönteminin tasarım (**design**) ve kodlama (**implementation**) yönleri oldukça iyi idi. *Grady Booch*, Ada diliyle oldukça çalışmıştı ve nesne yönelimli tekniklerin Ada diline uygulanması konusunda yaptığı çalışmalarla bu konuda önemli bir oyuncu olmuştur. Booch yöntemi güçlü olmasına rağmen model, birçok bulut şeklinden oluşmasından dolayı gösterim şekli oldukça sevimsizdi.
- ♦ OMT(**Object Modeling Technique**) tekniği, *Jim Rumbaugh* tarafından geliştirilmiş olup, analiz ve veri merkezli bilgi sistemlerinin modellenmesinde oldukça iyi bir yöntemdir.
- ♦ OOSE (**Object Oriented Software Engineering**) modeli **use-case** olarak bilinen bir model olup *Ivar Jacobson* tarafından geliştirilmiştir. Use-case modeli, yazılımı yapılacak sistemin davranışlarını anlamaya yönelik gelişmiş güçlü bir tekniktir.

1994 yılında *Jim Rumbaugh* ve General Electric firmasından ayrılan *Grady Booch* birlikte Rational şirketini kurarak bir araya geldiler. Bu birlikteliğin amacı, kendi yöntemlerini bir araya getirecek yeni ve tek bir yöntem olan “Unified Method” oluşturmak idi.

1995 yılında *Ivar Jacobson* da Rational şirketine katıldı. *Ivar Jacobson*’ın use-case’ler konusundaki fikirleriyle birlikte Rational şirketi, bugün UML olarak adlandırılan, yeni bir “Unified Method” geliştirdi. Üç kişiden oluşan bu grup “Three Amigos” olarak bilinir.

Yöntem savaşlarının ardından güçlü yazılım üreticileri bir araya gelerek OMG (**Object Management Group**) adında bir şirketler birliği kurdular<sup>1</sup>. Bu şirketler birliği 1997 yılında UML’e sahip çıkarak herkes tarafından kullanılan bir dil olmasını ve bir standart haline gelmesini sağlamıştır.

UML sadece, gerçek dünyadaki süreçlerin modelini ortaya koyan bir gösterim (**notation**) sistemidir. Süreçlerin standartlaştırılmasına yönelik bir dil değildir. UML ile ortaya çıkan modeller, süreçleri soyut (**abstract**) olarak tanımlayacaktır, ancak süreçlerin doğruluğu hakkında bir bilgi vermeyecektir. Yani, A şirketi için çalışan bir süreç, B şirketine uygulandığında felaketle sonuçlanabilir. Model, yukarıda da anlatıldığı üzere çözülecek probleme ilişkin bir soyutlamadır (**abstraction**). Etki alanı (**domain**) ise problemin yaşandığı gerçek dünyayı ifade eder.

Model, birbirlerine ileti (**message**) gönderen nesnelerden (**object**) oluşur. Bu modelde nesneler canlı (**alive**) olarak düşünülmelidir. Nesneler bir şeyler bilir (**know**) ve bildikleriyle bir şeyler yaparlar (**do**). Modelde nesneler, nesne tanımında verilen özellik (**property**) ve alanları (**field**) gösteren niteliklere (**attribute**) sahiptirler.

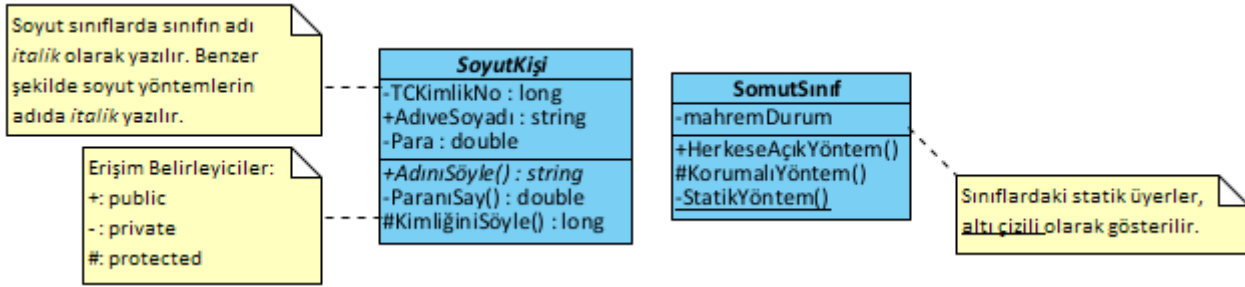
<sup>1</sup>OMG (Object Management Group), [www.omg.org](http://www.omg.org)

Nesnelerin gösterdikleri davranış (**behavior**) ya da geliştirdiği yöntemler (**method**) ise işlem (**operation**) olarak adlandırılır. Nesnenin özelliklerine ait o anki değerler ise nesnenin durumunu (**state**) belirler. UML, sekiz tür diyagrama sahiptir ve bu kitapta **sınıf diyagramlarına** (**class diagram**) yer verilecektir<sup>2</sup>.

## 10.2 Sınıf Diyagramları

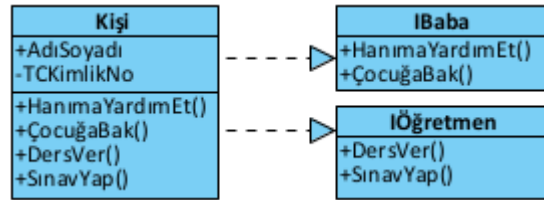
**Sınıf diyagramları** (**class diagram**), yazılımı geliştirilecek olan sisteme ait sınıfları (**class**) ve bu sınıflar arasındaki ilişkiyi (**relationship**) gösterir. Sınıf diyagramları durağandır (**static**). Yani sadece sınıfların bir-biriyle etkileşimini (**what interacts**) gösterir, bu etkileşimler sonucunda ne olduğunu (**what happens**) göstermez.

Sınıf diyagramlarında sınıflar, bir dikdörtgen ile temsil edilir. Bu dikdörtgen üç parçaya ayrılır. En üstteki birincisine sınıf ismi yazılır. Ortadaki ikincisinde nitelikler (**attribute**), en alttaki ve sonuncusunda ise ve işlemler (**operation**) yer alır.



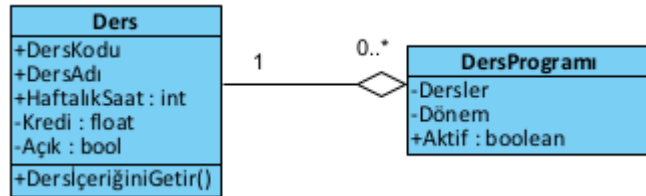
Şekil 10.1: Örnek Bir Sınıf Diyagramı

Sınıf isminin italik yazılması, sınıfın soyut sınıf (**abstract class**) olduğunu gösterir. Benzer şekilde işlemlerin yer aldığı kısımda, yöntemlerin italik yazılması halinde, ilgili yöntemin soyut yöntem (**abstract method**) olduğunu gösterir. Sınıflardaki erişim belirleyicilerin (**access modifier**), özel karakterlerle birbirinden ayrılır. Burada (-) karakteri mahrem (**private**), (+) karakteri umuma açık (**public**), (#) karakteri ise korunmalı (**protected**) olduğunu anlatmaktadır. Statik (**static**) üyeler altı çizili olarak gösterilir. **Ara yüz gerçekleştirme** (**interface realization**) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (**interface**) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar **lolipop diyagramı** olarak da adlandırılır. Çokluk (**multiplicity**), ilişkide bir sınıfın örnek (**instance**) sayısını gösterir. İlişkiyi gösteren



Şekil 10.2: Sınıf Diyagramlarında Arayüz Gerçekleştirme

**terface realization**) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (**interface**) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar **lolipop diyagramı** olarak da adlandırılır. Çokluk (**multiplicity**), ilişkide bir sınıfın örnek (**instance**) sayısını gösterir. İlişkiyi gösteren



Şekil 10.3: Sınıf Diyagramlarında Sınırlama ve Çokluk

çizginin sonunda bir numara olarak gösterilir. Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk bir sonraki başlıkta verilen, **iş birliği** (**association**), **bileşim** (**composition**) ve **bütünleşmeye** (**aggregation**) uygulanır. Çokluk örnekleri:

- ◊ 0..1: Sıfır ya da 1 örnek.
- ◊ 0.. veya \*: Örnek sayısı sınırsızdır.
- ◊ 1: Yalnızca 1 örnek
- ◊ 1..: En az bir örnek

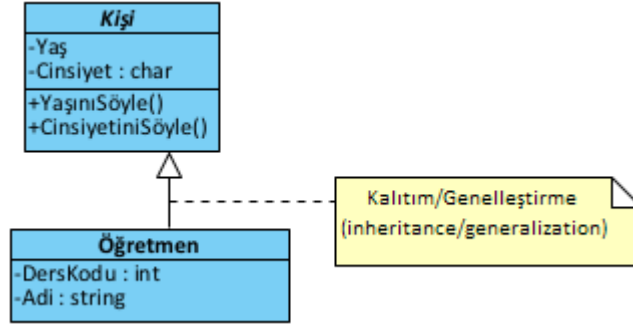
<sup>2</sup>[www.togethersoftware.com/services/practical\\_guides/umlonlinecourse/index.html](http://www.togethersoftware.com/services/practical_guides/umlonlinecourse/index.html)

## 10.3 Sınıflar Arası İlişkiler

Nesneleri imal ettiğimiz sınıflar arasında ya ilişki yoktur ya da aşağıda sıralanan altı çeşit ilişki vardır.

### §10.3.1 Genelleştirme İlişkisi

**Genelleştirme** (**generalization**), taban sınıfla (**base class**) ile türemiş sınıf (**derived class**) arasındaki **kalıtım** (**inheritance**) ilişkisidir. Örneği 8.10 başlığında verilmiştir. Aşağıda **Kişi** sınıfı ile bu sınıftan miras alan **Öğretmen** sınıfının UML diyagramı verilmiştir.

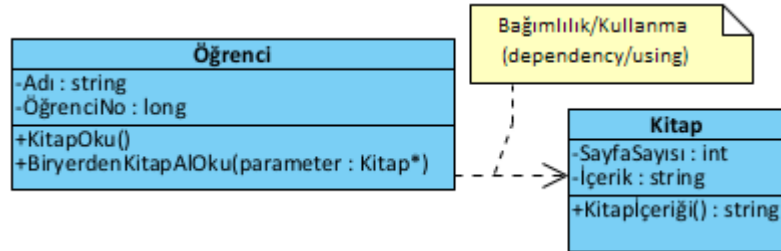


Şekil 10.4: Genelleştirme İlişkisi Örneği

### §10.3.2 Bağımlılık İlişkisi

**Bağımlılık** (**dependency**) ya da bir başka deyişle **kullanma** (**using**), bir sınıf ile diğer arasındaki geçici ilişkidir. Burada kullanan sınıf istemci (**client**) diğer sınıf ise sağlayıcı (**supplier**) olarak adlandırılır. İki türlü bağımlılık vardır;

1. İstemci sınıf, sağlayıcıyı sınıfı **parametre olarak** kullanır (**dependency as a parameter**).
2. İstemci sınıf, sağlayıcı sınıftan **örnekleme** yapar (**dependency as an instantiation**).



Şekil 10.5: Bağımlılık İlişkisi Örneği

Aşağıda **Öğrenci-Kitap** ilişkisini gösteren bir bağımlılık örneği verilmiştir.

```

//ANA PROGRAM:
class Program
{
 static void Main(string[] args)
 {
 Öğrenci ogrenci = new Öğrenci("Ali", 12000);
 ogrenci.KitapOku(); //Kitap imal edilip kullanılıyor...
 Kitap kitap = new Kitap(350, "Kurumsal Yazılım Geliştirme");
 ogrenci.BirYerdenKitapAlOku(kitap); //Kitap imal edilip kullanılıyor...
 }
}

//SINIFLAR
public class Kitap
{
 private string _icerik;
 private int _sayfaSayisi;
 public Kitap(int sayfaSayisi, string icerik)
 {
 _sayfaSayisi = sayfaSayisi;
 _icerik = icerik;
 }
}

```

```

public string KitapIcerigi()
{
 return _icerik;
}
}
public class Ogrenci
{
 private string _adi;
 private int _ogrenciNo;
 public Ogrenci(string adi, int no)
 {
 _adi = adi;
 _ogrenciNo = no;
 }
 // Dependency as Instantiation (Bağımlılığın yöntem içinde oluşturulması)
 public void KitapOku()
 {
 Kitap kitap = new Kitap(150, "C# ile Nesne Yönelimli Programlama Notları");
 Console.WriteLine("Kitap imal edilip kullanılıyor...");
 string notlar = kitap.KitapIcerigi();
 }
 // Dependency Injection (Bağımlılığın parametre olarak gelmesi)
 public void BirYerdenKitapAlOku(Kitap pKitap)
 {
 if (pKitap != null)
 {
 Console.WriteLine("Kitap dışarıdan alınıp kullanılıyor...");
 string okunacak = pKitap.KitapIcerigi();
 }
 }
}

```

Bu program, nesne yönelimli programlamada sınıflar arasındaki **bağımlılık** (**dependency**) ilişkisini ve bu bağımlılığın iki farklı şekilde nasıl kurulabileceğini gösteren çok temiz bir örnektir. Programda **Öğrenci** sınıfı, işini yapabilmek (kitap okumak) için **Kitap** sınıfına ihtiyaç duyar. Örnek Program, bir nesnenin başka bir nesneye olan bağımlılığını iki ana senaryoda ele almıştır;

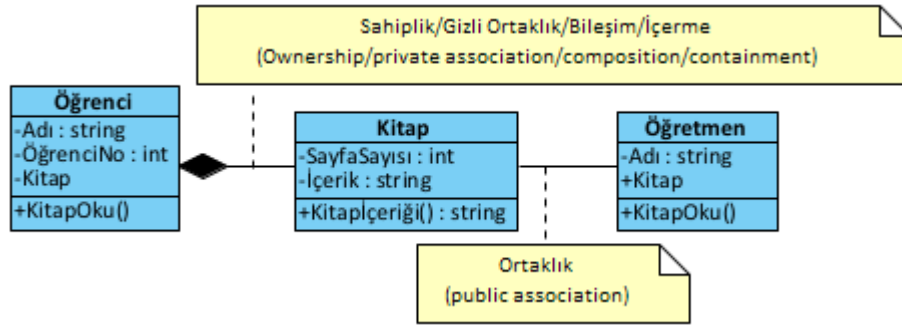
1. **Örnekleme Olarak Bağımlılık (Dependency as Instantiation):** **KitapOku()** yöntemi içinde, **Kitap** nesnesi metodun içinde **new** ile oluşturuluyor.
  - ◊ **Özellik:** **Öğrenci** nesnesi, hangi kitabın okunacağına kendisi karar verir ve kitabın yaşam döngüsünü (oluşturma ve silme) kendisi yönetir.
  - ◊ **Risk:** Bu durum sıkı **bağlılık** (**tight coupling**) yaratır. Eğer **Kitap** sınıfının yapısı (yapıcı parametreleri gibi) değişirse, **Öğrenci** sınıfının içindeki bu yöntemi de değiştirmek zorunda kalırız.
2. **Parametre Olarak Bağımlılık (Dependency as a Parameter):** **BirYerdenKitapAlOku(Kitap pKitap)** yönteminde, kitap nesnesi dışarıdan bir parametre olarak geliyor.
  - ◊ **Özellik:** Bu, modern yazılım mimarilerinde **bağımlılık enjeksiyonu** (**dependency injection**) kavramının temelidir. **Öğrenci** nesnesi, kendisine hangi kitap verilirse onu okur.
  - ◊ **Avantaj:** Daha esnek. Öğrenciye farklı kitap nesneleri (roman, ders kitabı vb.) dışarıdan kolayca gönderilebilir.

### §10.3.3 Sahiplik ve Ortaklık İlişkisi

**Sahiplik** (**ownership**) ve **ortaklık** (**public association**) ilişkileri, iki farklı sınıf arasındaki sürekli olan bütün-parça (**whole-part**) ilişkileridir. Bunlar;

1. Sahiplik ilişkisinde, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. İlişkinin sürekli olabilmesi için kullanılan sınıftan bir alan değişkeni (**field**) mahrem (**private**) olarak tanımlanır. Bu nedenle **içerme** (**containment**), **gizli ortaklık** (**private association**) veya **bileşim** (**composition**) olarak da adlandırılır.
2. Ortaklık ilişkisinde, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. Ancak bu nesnenin dışarıdan değiştirilmesine izin verir. İlişkinin geçici olabilmesi için kullanılan sınıftan bir alan değişkeni (**field**) umuma açık (**public**) olarak tanımlanır.

Aşağıda **Kitap**, **Öğrenci** ve **Öğretmen** nesnelerinin olduğu bir ortaklık örneği verilmiştir. **Öğrenci**, **Kitap** nesnesine sahip iken **Öğretmen** nesnesi **Kitap** ile ortaklık ilişkisindedir;



Şekil 10.6: Sahiplik ve Ortaklık İlişkisi Örneği

```

//ANA PROGRAM
class Program
{
 static void Main()
 {
 // 1. Öğrenci Testi
 Öğrenci öğrenci = new Öğrenci("Ali", 12000);
 öğrenci.KitapOku();
 // 2. Öğretmen Testi
 Öğretmen öğretmen = new Öğretmen("Ilhan");
 öğretmen.KitapOku();
 // Öğretmenin kitabına mülkiyet (Property) üzerinden ulaşılabilir
 Kitap öğretmeninOkuduguKitap = öğretmen.AktifKitap;
 Console.WriteLine("Öğretmenin Okuduğu Kitaba Main İçinden Ulaşılabilir;");
 Console.WriteLine($"Okunan Kitap: {öğretmeninOkuduguKitap.Kitapİçeriği()}");
 // Kitap değiştirme işlemi (Dışarıdan enjekte etme)
 Kitap baskaKitap = new Kitap(250, "Kurumsal Yazılım Geliştirme");
 öğretmen.AktifKitap = baskaKitap;
 öğretmen.KitapOku();
 }
}

//SINIFLAR
public class Kitap
{
 private string _icerik;
 private int _sayfaSayisi;
 public Kitap(int sayfaSayisi, string icerik)
 {
 _sayfaSayisi = sayfaSayisi;
 _icerik = icerik;
 }
 public string Kitapİçeriği()
 {
 return _icerik;
 }
}

public class Öğrenci
{
 private string _adı;
 private int _öğrenciNo;
 private Kitap _kitap;
 public Öğrenci(string adı, int no)
 {
 _adı = adı;
 _öğrenciNo = no;
 // SAHİPLİK İLİŞKİSİ (Composition): Kitap, Öğrenci ile doğar.
 _kitap = new Kitap(150, "C ile Yapısal Programlama");
 }
 public void KitapOku()

```



```

{
 Console.WriteLine("Öğrenci ile birlikte imal edilen kitap kullanılıyor...");
 Console.WriteLine($"Okunan Kitap: {_kitap.Kitapİçeriği()}");
}
}
public class Öğretmen
{
 private string _adı;
 // ORTAKLIK İLİŞKİSİ (Public Association): Özellik (Property) olarak dışa açılıyor.
 public Kitap AktifKitap { get; set; }
 public Öğretmen(string adı)
 {
 _adı = adı;
 // İlk üretimde bir kitap atanıyor
 AktifKitap = new Kitap(200, "C++ ile Nesne Yönelimli Programlama");
 }
 public void KitapOku()
 {
 Console.WriteLine("Öğretmen ile birlikte imal edilen kitap kullanılıyor...");
 Console.WriteLine($"Okunan Kitap: {AktifKitap.Kitapİçeriği()}");
 }
}

```

Örnek program, nesne yönelimli programlamada nesneler arasındaki **sahiplik** (**ownership**) ve birliktelik (**association**) ilişkilerini, özellikle de mahrem (**private**) ve umuma açık (**public**) erişim düzeylerinin bu ilişkileri nasıl kısıtladığını veya esnettiğini gösterir. Programdaki temel kavramları ve teknik detayları;

1. **Sahiplik** (**ownership** / **composition** / **private association**): **Öğrenci-Kitap** Sınıfları
  - ◊ **Durum**: **Öğrenci** nesnesi yaratıldığında **Kitap** nesnesi de onun içinde otomatik olarak **new** ile oluşturuluyor.
  - ◊ **Gizlilik**: **\_kitap** üyesi **private** olduğu için, ana program içinden öğrencinin kitabına erişilemez veya bu kitap değiştirilemez.
  - ◊ **Yaşam Döngüsü**: Kitap, öğrenciye bağlıdır.
2. **Ortaklık** (**public association**): **Öğretmen-Kitap** Sınıfları
  - ◊ **Durum**: Öğretmen de bir kitapla doğar ve bu kitap dış dünyaya açılmıştır.
  - ◊ **Esneklik**: Ana programda öğretmenin okuduğu kitaba ulaşabildik ve hatta kitabını başka bir kitap nesnesiyle değiştirebildik.
  - ◊ **Bağımsızlık**: Öğretmen nesnesi hala hayattayken, ona dışarıdan bambaşka bir kitap nesnesi enjekte edilebilir.

### §10.3.4 Kalıtım Yerine Kompozisyon İlkesi

Kalıtım Yerine Kompozisyon (**composition over inheritance**), nesne yönelimli programlama (OOP) dünyasında kodun daha esnek, bakımı kolay ve değişime dirençli olması için geliştirilmiş temel bir tasarım ilkesidir. Kompozisyon yukarıda anlatılan sahiplik ilişkisidir. Bu ilke, sınıflar arasındaki ilişkiyi kurarken **"is-a"** ilişkisi yerine **"has-a"** ilişkisini tercih etmemiz gerektiğini savunur.

| Özellik            | Kalıtım                                         | Kompozisyon                                         |
|--------------------|-------------------------------------------------|-----------------------------------------------------|
| <b>Bağımlılık</b>  | Sıkı bağıllık ( <b>tight coupling</b> ).        | Gevşek bağıllık ( <b>loose coupling</b> ).          |
| <b>Değişim</b>     | Üst sınıf değişirse her şey etkilenir.          | Parçalar birbirinden bağımsız değişebilir.          |
| <b>Esneklik</b>    | Tasarım zamanında ( <b>static</b> ) belirlenir. | Çalışma anında ( <b>dynamic</b> ) değiştirilebilir. |
| <b>Karmaşıklık</b> | Çok derin hiyerarşiler kafa karıştırıcıdır.     | Küçük, odaklanmış ve yönetilebilir sınıflar oluşur. |

Tablo 10.1: Niçin Kompozisyon Kullanılmalı?

Kalıtım (**inheritance**), bir sınıfın başka bir sınıftan tüm özellik ve davranışları miras almasıdır. Genellikle katı bir hiyerarşi oluşturur. Bi bir "is-a" ilişkisidir. Örnek: "Bir Kamyon, Araç'tır". Kalıtım, "beyaz kutu" (**white-box**) yeniden kullanımıdır. Üst sınıftaki (**parent**) bir değişiklik, alt sınıfların (**children**) tamamını bozabilir. Hiyerarşi derinleştikçe sistem kırılabilir.

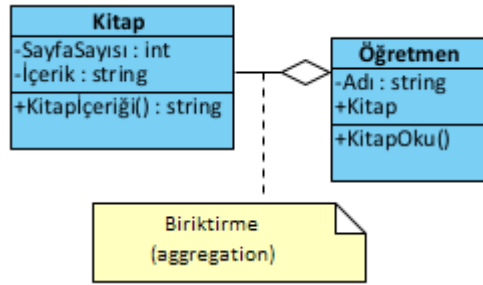
Kompozisyon (**composition**), bir sınıfın işlevselliğini başka bir sınıfın nesnesini kendi içinde barındırarak (ona sahip olarak) elde etmesidir. Kompozisyon bir "has-a" ilişkisidir. Örnek: "Bir Araba, Motor'a sahiptir". Kompozisyon, "siyah kutu" (**black-box**) yeniden kullanımıdır. İçerideki nesnenin nasıl çalıştığını bilmenize gerek yoktur, sadece arayüzünü kullanırsınız. Nesneler çalışma anında (**run-time**) kolayca değiştirilebilir.



Kalıtım, eğer bir sınıf, diğerinin gerçekten özelleşmiş bir versiyonuysa ve bu ilişki zamanla hiç değişmeyecekse (Örn: Daire bir Sekildir) kullanılabilir. Kompozisyon ise davranışları birleştirmek, modüler yapılar kurmak ve sistemin gelecekteki değişimlere uyum sağlamasını istiyorsanız (Örn: OdemeSistemi bir KrediKartina sahiptir) tercih edilmelidir.

### §10.3.5 Biriktirme İlişkisi

**Biriktirme** (**aggregation**) ilişkisi, iki farklı sınıf arasındaki sürekli olan bütün-parça (**whole-part**) ilişkisidir. Ancak **sahiplik** (**ownership**) ve **ortaklık** (**public association**) ilişkilerinden farklı olarak burada nesnelerin yaşam döngüleri birbirinden ayrılmıştır.



Şekil 10.7: Biriktirme İlişkisi Örneği

```
//ANA PROGRAM
class Program
{
 static void Main()
 {
 // 1. Önce bağımsız parçalar (kitaplar) oluşturulur.
 Kitap kitap1 = new Kitap(150, "C# ile Nesne Yönelimli Programlama");
 Kitap kitap2 = new Kitap(200, "Veri Yapıları ve Algoritmalar");
 // 2. Bütün (Öğretmen) mevcut bir kitap referansı ile oluşturulur.
 Öğretmen ilhan = new Öğretmen("İlhan", kitap1);
 ilhan.KitapOku();
 // 3. Çalışma zamanında parça (kitap) kolayca değiştirilir.
 ilhan.AktifKitap = kitap2;
 Console.WriteLine("Öğretmen kitabını değiştirdi.");
 ilhan.KitapOku();
 }
}

//SINIFLAR
public class Kitap
{
 private string _icerik;
 private int _sayfaSayisi;
 public Kitap(int sayfaSayisi, string icerik)
 {
 _sayfaSayisi = sayfaSayisi;
 _icerik = icerik;
 }
 public string Kitapİçeriği() => _icerik;
}

public class Öğretmen
{
 private string _adı;

 // Biriktirme (Aggregation) ilişkisi: Kitap dışarıdan gelir.
 // Özellik (Property) kullanarak C++'taki get/set metodunu C# tarzına çeviriyoruz.
 public Kitap AktifKitap { get; set; }

 // Yapıcı Metot: Kitap olmadan öğretmen nesnesi oluşturulamaz.
 public Öğretmen(string adı, Kitap kitap)
 {

```

```

 _adı = adı;
 AktifKitap = kitap;
 }

 public void KitapOku()
 {
 if (AktifKitap != null)
 {
 Console.WriteLine($"{_adı} şu kitabı okuyor: {AktifKitap.Kitapİçeriği()}");
 }
 }
}

```

Programdaki biriktirme (**aggregation**) ilişkisi **Öğretmen** ve **Kitap** arasında zayıf bir sahiplik ilişkisidir;

- ◊ **Bağımsız Yaşam Döngüsü:** **Kitap** nesnesi, **Öğretmen** nesnesinden önce ana program içinde oluşturulur. Burada **Öğretmen** emekli olsa (nesne silinse) bile kitap nesnesi ana program içinde var olmaya devam eder.
- ◊ **Zorunlu Varlık:** **Öğretmen** yapıcısı olan `public Öğretmen(string adı, Kitap kitap)` bir kitap adresi beklediği için, sistemde bir kitap olmadan öğretmen nesnesi imal edilemez.
- ◊ **Esneklik:** **AktifKitap** özelliği sayesinde **Öğretmen** elindeki kitabı bırakıp başka bir kitabı alabilir.

## 10.4 Sınıf Tasarlama İlkeleri

Nesne yönelimli programlama (**object oriented programming**) kodumuzda değişim yönetimi (**change management**), gösterici kullanımı yerine referansları kullanarak güvenli kod (**safe code**) aynı kodları tekrar yazmak (**duplicate code**) yerine kodların yeniden kullanımı (**reusing**) sağlayan aşağıdaki özellikleri kullanırız;

- ◊ Kod küçüldüğü için kalıtım (**inheritance**),
- ◊ Veri ve kontrol soyutlaması (**abstraction**) yaparak daha güvenli bileşen tasarladığımız için sarmalama (**encapsulation**),
- ◊ Yazılım mimarisinin daha esnek ve genişleyebilir olması için çok biçimlilik (**polymorphism**).

Ayrıca yazılım geliştirmede;

- ◊ Zayıf bağlaşımın (**loose couple**) sağlanması için yazılımın bir noktasında olabilecek değişikliğin, diğer kısımlarda değişiklik gerektirmeyecek şekilde tasarlanması gerekir. Yani yapılan değişiklik, çorap söküğü gibi yazılımın diğer kısımlarını değiştirmemelidir.
- ◊ Aynı çağrışım kümesine ilişkin bütün bileşenler bir arada geliştirilmeli diğer kümelerle karıştırılmamalıdır (**seperation of concern**). Yani bileşenler arasında yüksek uyum (**high cohesion**) olmalıdır.

İşte yeniden kullanım, zayıf bağlaşım ve yüksek uyum sağlamak için programcı, nesne yönelimli programlama özellikleriyle birlikte temel ilkelere (**principle**) uyar. Programcı kodu tasarlariken, yazarken ve yeniden düzenlerken bu ilkeleri aklında tutarsa; Kod çok daha temiz, genişletilebilir ve test edilebilir olur. İlkelerin İngilizce baş harflerinin alınarak **SOLID ilkeleri** (**SOLID principles**) olarak isimlendirilir.

### §10.4.1 Tek Sorumluluk İlkesi

Bu ilke ile SRP (**Single Responsibility Principle**) sınıflar, tasarlanırken birden fazla işten sorumlu tutulmamalıdır. *Robert C. Martin*'in vurguladığı gibi "**Bir sınıfın değişmek için tek bir nedeni (aktörü) olmalıdır**". Bir sınıfın tek bir şey yapması gerektiğini ve dolayısıyla değişmesi için tek bir nedeninin olması gerektiğini belirtir.

// 1. Veri Sorumluluğu: Sadece çalışanın temel bilgilerini tutar.

```

public class Çalışan
{
 public int TCKimlikNo { get; set; }
 public string Adı { get; set; }
 public decimal Maaş { get; set; }
}

```

// 2. Hesaplama Sorumluluğu: Maaş algoritmalarından sorumludur.

```

public class MuhasebeServisi
{
 public void ÖdemeHesapla(Çalışan işçi)
 {
 // Hesaplama mantığı buraya gelir.
 Console.WriteLine($"{işçi.Adı} için maaş hesaplandı.");
 }
}

```

```

 }
}
// 3. Veri Tabanı Sorumluluğu: Kayıt ve okuma işlemlerinden sorumludur (Repository Pattern).
public class ÇalışanDepolama
{
 public void Sakla(Çalışan işçi)
 {
 // Veritabanı kayıt mantığı.
 Console.WriteLine($"{işçi.Adı} veritabanına kaydedildi.");
 }
}
// 4. Raporlama Sorumluluğu: Çıktı formatlarından (PDF, Excel vb.) sorumludur.
public class RaporlamaServisi
{
 public void RaporÜret(Çalışan işçi)
 {
 // Rapor oluşturma mantığı.
 Console.WriteLine($"{işçi.Adı} için rapor oluşturuldu.");
 }
}

```

### §10.4.2 Açık-Kapalı İlkesi

Bu ilke ile OCP (**Open Closed Principle**) sınıflar, **değişikliklere kapalı** (**closed for modification**) **eklen-tilere açık** (**open for extension**) şekilde tasarlanmalıdır. Bu şekilde değişikliklere açık genişleyebilir modüllere sahip oluruz. Değişikliklere kapalı olmak, mevcut bir sınıfın kodunu değiştirmemek anlamına gelirken, eklen-tilere açık olma yeni işlevler eklemek anlamına gelir.

```

using System;
using System.Collections.Generic;
// 1. GENİŞLEMeye AÇIK YAPI: Soyut Sınıf veya Ara yüz (Interface) olarak tanımlanabilir
// Yeni şekiller eklenebilir (Gelişime Açık)
public abstract class Şekil
{
 public abstract double AlanHesapla();
}
// 2. ÖZEL ŞEKİLLER: Mevcut yapıyı bozmadan eklenirler
public class Daire : Şekil
{
 public double Yarıçap { get; set; }
 public override double AlanHesapla() => Math.PI * Math.Pow(Yarıçap, 2);
}
public class Dikdörtgen : Şekil
{
 public double Genişlik { get; set; }
 public double Yükseklik { get; set; }
 public override double AlanHesapla() => Genişlik * Yükseklik;
}
// 3. DEĞİŞİME KAPALI SINIF: AlanHesaplayıcı
// Yeni bir şekil eklendiğinde bu sınıfın koduna dokunmamız gerekmez!
public class AlanHesaplayıcı
{
 public double ToplamAlan(IEnumerable<Şekil> Şekiller) //IEnumerable ileride anlatılacaktır.
 {
 double toplam = 0;
 foreach (var Şekil in Şekiller)
 {
 // Polimorfizm sayesinde her şekil kendi alanını hesaplar
 toplam += Şekil.AlanHesapla();
 }
 return toplam;
 }
}

```

### §10.4.3 Liskov İkame İlkesi

Bu ilkede LSP (**Liskov Substitution Principle**) alt sınıflar türediği sınıfların yerine konulabilmelidir. Bu prensipten ilk olarak *Barbar Liskov* tarafından bahsedilmiştir. Alt sınıflar, türediği sınıf davranışlarıyla kullanılabilir. Burada amaç, alt sınıfların daha çok değişebileceği göz önüne alınarak değişimlerden an az şekilde etkilenmektir. 10.3 başlığındaki ilişkiler bu ilkenin temelini oluşturur.

Bu ilke der ki: "Bir programdaki nesneler, programın doğruluğunu bozmadan alt tipleriyle (**child classes**) yer değiştirebilmelidir." Yani, üst sınıf için yazılmış bir kod bloğu, o sınıftan türeyen herhangi bir alt sınıf geldiğinde de patlamadan çalışmaya devam etmelidir.

```
// 1. TEMEL SINIF: Tüm kuşların ortak özelliği (Beslenme)
public abstract class Kuş
{
 public virtual void Ye() => Console.WriteLine("Kuş yem yedi.");
}
// 2. DAVRANIŞSAL AYRIM: Sadece uçabilen kuşlar için bir sınıf veya arayüz
public abstract class UçanKuş : Kuş
{
 public abstract void Uc();
}
// 3. UYGULAMALAR: LSP'yi ihlal etmeyen doğru hiyerarşi
public class Serçe : UçanKuş
{
 public override void Uc() => Console.WriteLine("Serçe kanat çırparak uçuyor.");
}
public class Penguen : Kuş
{
 // Penguen sadece Kuş'tan türer.
 // fly() metodunu override edip hata fırlatmak (NotImplementedException) zorunda kalmaz.
}
```

Türemiş sınıfından bir nesneyi, Taban sınıfından bir nesne bekleyen herhangi bir yöntem parametre olarak geçirdiğimizde yöntemin herhangi bir tuhaf çıktı vermemesi gerektiği anlamına gelir. Bu beklenen davranıştır, çünkü kalıtım kullandığımızda alt sınıfın, üst sınıfın sahip olduğu her şeyi miras aldığını varsayabiliriz. Alt sınıf davranışı genişletir ancak asla daraltmaz. Dolayısıyla bir sınıf bu ilkeye uymadığında, tespit edilmesi zor bazı kötü hatalara yol açar.

### §10.4.4 Ara Yüzleri Ayırma İlkesi

Bu ilke ile ISP (**Interface Segregation Principle**) genel amaçlı ara yüzler yerine istemciye bağlı ara yüzler tasarlanmalıdır. Yani roller, birbirinden ayrı olacak şekilde tanımlanmalıdır. Birden fazla rol tek bir rol altında olacak şekilde düşünülmemelidir. Bu nedenle buna rollerin ayrılması prensibi RSP (**Role Segregation Principle**) olarak da adlandırılır. Bu ilke der ki: "Hiçbir istemci (**client**), kullanmadığı yöntemlere bağımlı olmaya zorlanmamalıdır." Devasa ve her işi yapan "şişman" (**fat**) arayüzler yerine, daha spesifik ve küçük arayüzler oluşturmayı savunur. Eğer bir arayüz çok fazla yetenek barındırıyorsa, bu arayüzü uygulayan sınıflar ihtiyaç duymadıkları metodları da `throw new NotImplementedException();` diyerek boşu boşuna taşımak zorunda kalırlar.

```
// 1. KÜÇÜK VE ÖZEL ARAYÜZLER (Specific Interfaces)
public interface IYazdırabilir
{
 void Yazdır();
}
public interface ITarayabilir
{
 void Tara();
}
public interface IFaksLayabilir
{
 void FaksGönder();
}
// 2. SADECE YAZDIRMA YETENEĞİ OLAN CİHAZ
// Sadece ihtiyacı olan arayüzü uygular.
public class BasitYazıcı : IYazdırabilir
{
}
```

```

 public void Yazdır() => Console.WriteLine("Basit yazıcı çıktı veriyor...");
}
// 3. ÇOK FONKSİYONLU CİHAZ (All-in-One)
// İhtiyaç duyduğu tüm yetenekleri "biriktirir".
public class CokFonksiyonluYazici : IYazdirabilir, ITarayabilir, IFakslayabilir
{
 public void Yazdır() => Console.WriteLine("Yazdırılıyor...");
 public void Tara() => Console.WriteLine("Taranıyor...");
 public void FaksGönder() => Console.WriteLine("Fakslanıyor...");
}

```

#### §10.4.5 Bağımlılıkları Ters Çevirme İlkesi

Bu ilke DIP (**Dependency Inversion Principle**), sınıflarımızın somut sınıflara ve fonksiyonlara değil, ara yüzlere veya soyut sınıflara bağlı olması gerektiğini belirtir. Nesne yönelimli programlamanın en önemli ilkesidir. Aslında bu ilke ile açık-kapalı ilkesi doğrudan birbiriyle ilişkilidir. Bu ilke özetle şunu söyler: "Yüksek seviyeli modüller, düşük seviyeli modüllere bağımlı olmamalıdır. Her ikisi de soyutlamalara (arayüzler veya soyut sınıflar) bağımlı olmalıdır."

```

// 1. SOYUTLAMA (Interface): Her iki taraf da buna bakar
public interface IDatabase
{
 void Save(string data);
}
// 2. DÜŞÜK SEVİYELİ MODÜLLER (Implementations)
public class MySqlConnection : IDatabase
{
 public void Save(string data) => Console.WriteLine($"Veri MySQL'e kaydedildi: {data}");
}
public class PostgreSQLDatabase : IDatabase
{
 public void Save(string data) => Console.WriteLine($"Veri PostgreSQL'e kaydedildi: {data}");
}
// 3. YÜKSEK SEVİYELİ MODÜL (Business Logic)
// UserService, hangi veritabanının kullanıldığını bilmez, sadece IDatabase olduğunu bilir.
public class UserService
{
 private readonly IDatabase _database;
 // Constructor Injection: Bağımlılık dışarıdan enjekte ediliyor.
 public UserService(IDatabase database)
 {
 _database = database;
 }
 public void SaveUser(string user)
 {
 _database.Save(user);
 }
}

```

## 10.5 Düşün ve Çöz

- ♦ **Düşün:** UML sınıf diyagramında birliktelik (**composition**) ile biriktirme (**aggregation**) arasındaki farkı somut bir örneklerle gösterin. 'Araba-Motor' ilişkisi ile 'Üniversite-Öğrenci' ilişkisi bu iki kavramdan hangisine karşılık gelir ve neden?
- ♦ **Düşün:** Tek Sorumluluk İlkesi'nin (SRP) ihlal edildiğini nasıl anlarsınız? Bir sınıfın birden fazla 'değişim nedeni' olduğunu tespit etmenin pratik yöntemi nedir?
- ♦ **Düşün:** Açık-Kapalı İlkesi'ni (OCP) uygulamak için kalıtım mı, arayüz mi yoksa strateji deseni mi kullanmak daha uygundur? Bir vergi hesaplama sistemi örneği üzerinden tartışınız.
- ♦ **Düşün:** Liskov İkame İlkesi'ni (LSP) ihlal eden 'Kare-Dikdörtgen' problemi neden sezgisel olmayan bir ihlaldir? Bu sorunun kök nedeni tasarım mı yoksa dil sınırlaması mı?
- ♦ **Düşün:** Arayüzleri Ayırma İlkesi (ISP) neden 'şişman arayüzler'in (**fat interfaces**) sorunlu olduğunu savunur? Çok yöntemli bir arayüzü küçük arayüzlere bölmenin olumsuz bir etkisi var mı?
- ♦ **Düşün:** Bağımlılıkları Ters Çevirme İlkesi (DIP) ile bağımlılık enjeksiyonu (**dependency injection**) arasın-

daki ilişki nedir?

- ◇ **Düşün:** Sıkı bağlaşım (**tight coupling**) ile gevşek bağlaşım (**loose coupling**) arasındaki farkı bir HTTP servisi çağrısı örneğiyle açıklayınız. Arayüzler bu bağı nasıl gevşetir?
- ◇ **Çöz:** SOLID ilkelerinden Tek Sorumluluk İlkesi'ni ihlal eden bir Personel sınıfı yazınız (maaş hesaplama, veritabanı kaydetme ve e-posta gönderme aynı sınıfta). Ardından her sorumluluğu ayrı sınıflara taşıyın.
- ◇ **Çöz:** Liskov İkame İlkesi'ni ihlal eden 'Kare-Dikdörtgen' örneğini kodlayın; ihlalin neden oluştuğunu test koduyla gösterin ve tasarımı düzeltin.
- ◇ **Çöz:** Bağımlılık enjeksiyonu kullanarak bir bildirim sistemi tasarlayın: **INotificationService** arayüzü, **EmailNotification** ve **SmsNotification** uygulamaları. Müşteri sınıfı hangi bildirim sisteminin kullanılacağını bilmeden çalışabilsin.
- ◇ **Çöz:** Bir ödeme sistemi için Açık-Kapalı İlkesine uygun bir tasarım yapın: **KrediKarti**, **BankaTranferi** ve Kripto ödeme yöntemleri mevcut kodu değiştirmeden eklenebilsin.
- ◇ **Çöz:** Yazıcı, tarayıcı ve faks işlevleri olan büyük bir **IMultifunctionDevice** arayüzünü, Arayüzleri Ayırma İlkesine uygun olarak küçük arayüzlere bölün. Sadece yazdırma yapabilen bir sınıfı bu yapıya uygun olarak yazın.

## 11. Bölüm

# Yapılar, Birlikler, Öznitelikler ve Yansıma

### 11.1 Yapılar

Yapılar, belleğin yönetimi ve verimlilik açısından hayati önem taşır. Sınıflar (**class**) ile olan benzerliği yeni başlayanları şaşırtsa da, arka plandaki çalışma mantıkları taban tabana zıttır. Yapılar aslında **değer tipi nesnelerdir**. Sınıflar "referans tipi" olup bellek tahsisi öbek (**heap**) bölgesinden yapılır. Yapılar "değer tipi" olup doğrudan yığın (**stack**) bölgesinde saklanırlar.

Yapıların (**struct**) sınıflardan en büyük farklarından biri kalıttır (**inheritance**). **Yapılar, başka bir yapı veya sınıftan türetilemez ve taban sınıf olamaz!** Yapılara sadece arayüzler (**interface**) uygulayabilir.

#### §11.1.1 Yapı Tanımlama ve Temel Yapı

Bir yapı (**struct**) tanımlamak, sınıf (**class**) tanımlamaya çok benzer ancak kullanım amacı genellikle küçük, hafif ve kısa ömürlü veri gruplarını temsil etmektir. Farklı tiplerdeki **değer tipi (value type)** elemanlarının bir arada gruplandırılan özel bir veri tipi tanımlamak için yapı tanımlarız. Yapılar, türetilmiş (**derived**) veya kullanıcı tanımlı (**user defined**) bir veri olup tanımlamak için **struct** anahtar kelimesini kullanırız.

```
public struct Nokta
{
 public int X { get; set; }
 public int Y { get; set; }
 public Nokta(int x, int y)
 {
 X = x;
 Y = y;
 }
 public void KomuYazdır() => Console.WriteLine($"Konum: ({X}, {Y})");
}
```

#### §11.1.2 Yapı Öğelerine Erişim

Tanımlanan yapı kullanılarak değişkenler aşağıdaki gibi tanımlanabilir;

```
yapı-kimliği değişken-kimliği;
//yada
yapı-kimliği değişken-kimliği=new yapı-kimliği();
```

Tanımlanan yapı değişkenleri üzerinden her bir üyeye **nokta işlecisi (dot operator)** ile erişilir. Bu işleç de parantez işlecisi ile aynı önceliktedir. Ayrıca **atama işlecisi (=)** bir yapıyı doğrudan kopyalamak için kullanılabilir. Bunun yanında, bir yapının üyesinin değerini başka birine atamak için yine atama işlecini de kullanabiliriz. Yukarıda örneği verilen yapılardan birçok değişken kimliklendirilebilir;

```
Öğrenci ilhan;
ilhan.yas = 20;
ilhan.cinsiyet = 'E';
ilhan.kilo = 80;
ilhan.boy = 180;
```

```
Öğrenci mehmet=new Öğrenci();
mehmet.yas = 25;
mehmet.cinsiyet = 'E';
mehmet.kilo = 75;
mehmet.boy = 175;
```

```
Koordinat nokta1;
nokta1.X = 10.5;
```



```
nokta1.Y = 20.5;

Koordinat nokta2= new Koordinat();
nokta2.X = 30.5;
nokta2.Y = 40.5;

nokta2 = nokta1;
```

### §11.1.3 Yapılarda Yapıcılar

Sınıflardan ön tanımlı nesneler imal edilebildiği gibi yapılardan da varsayılan değerlere sahip yapılar oluşturulabilir. Bunun için **yapıcılar** (**constructor**) kullanılır. Yapı alanları (**field**) parametrelili bir yapıcı aracılığıyla imal edilir.

```
Koordinat nokta1;
nokta1.X = 10.5;
nokta1.Y = 20.5;
Console.WriteLine($"Nokta 1: ({nokta1.X}, {nokta1.Y})");
Koordinat nokta2 = new Koordinat();
nokta2.X = 30.5;
nokta2.Y = 40.5;
Console.WriteLine($"Nokta 2: ({nokta2.X}, {nokta2.Y})");
nokta2 = nokta1;
Console.WriteLine("Nokta 2: {0}, {1}", nokta2.X, nokta2.Y);
```

```
public struct Koordinat
{
 public double X;
 public double Y;
 public Koordinat()
 {
 X = 0;
 Y = 0;
 }
 public Koordinat(double x, double y)
 {
 X = x;
 Y = y;
 }
}
```

**Mahrem** (**private**) üyelere yalnızca yapıcı içinde ilk değer verilebilir yani başlatılabilir (**initialize**). Diğer değer tipleri gibi yapılar da **null** olamaz. Ancak değişken olarak **null** olabilir olarak tanımlanabilir;

```
Koordinat nokta1 = null; //HATA!: Değer tiplere null atanamaz!
Koordinat? v2 = null; //OK
Nullable<Koordinat> v3 = null // OK
```

### §11.1.4 Yapılarda Ara Yüz Gerçekleştirme

Tanımlanan her yapı (**struct**), **System.ValueType** tipinden dolayı olarak miras alan kısır (**sealed**) bir veri tipidir. Yapılar başka herhangi bir tipten miras alamaz, ancak ara yüzleri gerçekleştirebilirler (**interface realization**);

```
Dikdörtgen dikdörtgen = new Dikdörtgen(5, 10);
Console.WriteLine($"Dikdörtgenin alanı: {dikdörtgen.Alan()}");
Daire daire = new Daire(7);
Console.WriteLine($"Dairenin alanı: {daire.Alan()}");
public interface IŞekil // Bir arayüz
{
 double Alan();
}
public struct Daire : IŞekil // Daire yapısı IŞekil arayüzünü gerçekleştiriyor
{
 public double yarıcap;
 public Daire(double yarıcap)
```

```

{
 this.yaricap = yaricap;
}
public double Alan()
{
 return Math.PI * yaricap * yaricap;
}
}
public struct Dikdörtgen : IŞekil // Dikdörtgen yapısı IŞekil ayayüzünü uyguluyor
{
 public double uzunKenar;
 public double kısaKenar;
 public Dikdörtgen(double uzunKenar, double kısaKenar)
 {
 this.uzunKenar = uzunKenar;
 this.kısaKenar = kısaKenar;
 }
 public readonly double Alan()
 {
 return uzunKenar * kısaKenar;
 }
}

```

Sınıflarda olduğu gibi bir yapıya üye yöntem tanımlanabilir. Eğer üye yöntem bir yapı alanını değiştirmeyecek ise yukarıdaki gibi `readonly` tanımlanabilir.

### §11.1.5 Yapı ve Sınıf Arasındaki Farklar

Üç temel fark aşağıdaki tabloda verilmiştir;

| Özellik        | Struct (Yapı)                     | Class (Sınıf)                           |
|----------------|-----------------------------------|-----------------------------------------|
| Bellek Bölgesi | Stack (Hızlı erişim)              | Heap (Çöp toplayıcı yönetimi)           |
| Tip Kategorisi | Değer Tipi ( <b>value type</b> )  | Referans Tipi ( <b>reference type</b> ) |
| Kalıtım        | Kalıtımı desteklemez (Mühürlüdür) | Kalıtımı destekler                      |

Tablo 11.1: Yapı ve Sınıf Karşılaştırması

### §11.1.6 Değer Tipi Davranışı

Bir `struct` değişkenini başka birine eşitlediğinizde, verinin kendisi kopyalanır. Birinde yapılan değişiklik diğerini etkilemez.

```

Nokta n1 = new Nokta(10, 20);
Nokta n2 = n1; // n1'in kopyası oluşturuldu
n2.X = 100;
Console.WriteLine(n1.X); // Çıktı: 10 (Etkilenmedi!)
Console.WriteLine(n2.X); // Çıktı: 100

```

### §11.1.7 Ne zaman Yapı Kullanmalı?

Aşağıda bu durumlar özetlenmiştir;

- ◊ **Boyut:** Eğer verinin boyutu 16 byte'tan küçükse genellikle `struct` uygundur.
- ◊ **Ömür:** Nesne çok kısa süre kullanılacak ve hemen yok edilecekse `struct` daha performanslıdır.
- ◊ **Değişmezlik (immutability):** Yapıların `readonly` olarak tasarlanması en iyi uygulamadır.
- ◊ **Kalıtım Yokluğu:** Bir `struct` başka bir sınıftan veya yapıdan türeyemez. Ancak `interface` gerçekleştirebilir.

### §11.1.8 Yalnız Okunabilir Modern Yapı

C#7.2+ ve sonrası için daha güvenli olan `readonly struct` yapısı kullanılır. Değiştirilemez yapılar, performans optimizasyonu sağlar.

```

public readonly struct Renk
{
 public int R { get; init; }
 public int G { get; init; }
 public int B { get; init; }
}

```

```
public Renk(int r, int g, int b) => (R, G, B) = (r, g, b);
}
```

Bir `struct` (değer tipi) nesnesini, `object` (referans tipi) bekleyen bir metoda gönderirsek bellekte ne olur? Bu işleme kutulama (**boxing**) denir. Yığındaki (**stack**) veri, geçici olarak öbek (**heap**) bölgesinde bir kutuya alınır. Bu işlem performans açısından maliyetlidir. Bu yüzden döngüler içerisinde yapıları gereksiz yere `object` tipine dönüştürmekten kaçınmalıyız.

## 11.2 Birlikler

C# dünyasında, C++'taki `union` anahtar kelimesine doğrudan karşılık gelen bir yapı bulunmamaktadır. Ancak C#, bellek yönetimi üzerinde tam kontrol sağlamak istediğimiz durumlar için bu davranışı taklit etmemiz için `System.Runtime.InteropServices` kullanılır.

C++'ta `union`, içindeki tüm değişkenlerin aynı bellek adresini paylaşmasını sağlar. C#'ta bunu başarmak için `struct` kullanırız ve derleyiciye her bir değişkenin bellekte hangi "başlangıç noktası" (**offset**) değerinden başlayacağını el yordamıyla söyleriz.

Aşağıdaki örnekte, 4 adet 8-bitlik (**byte**) değer ile 1 adet 32-bitlik (**int**) değer aynı bellek bölgesini nasıl paylaştığını görebilirsiniz;

```
using System.Runtime.InteropServices;
// FieldOffset(0) ifadesi, tüm değişkenlerin belleğin 0. noktasından başladığını belirtir (Union
// → davranışı).
[StructLayout(LayoutKind.Explicit)]
public struct VeriPaketi
{
 [FieldOffset(0)]
 public int TamSayı; // 4 byte

 [FieldOffset(0)]
 public byte Bayt1; // TamSayı'nın 1. byte'ı
 [FieldOffset(1)]
 public byte Bayt2; // TamSayı'nın 2. byte'ı
 [FieldOffset(2)]
 public byte Bayt3; // TamSayı'nın 3. byte'ı
 [FieldOffset(3)]
 public byte Bayt4; // TamSayı'nın 4. byte'ı
}
```

Bu program için aşağıdakiler söylenebilir;

- ◊ **Aynı Adres, Farklı Bakış Açısı:** Yukarıdaki örnekte `TamSayı` değişkenine bir değer atadığınızda, aslında `Bayt1`'den `Bayt4`'e kadar olan kısımları da değiştirmiş olursunuz. Çünkü hepsi aynı fiziksel bellek hücrelerine bakar.
- ◊ **Kullanım Alanları:** Genellikle düşük seviyeli (**low-level**) ağ protokolleri, dosya formatları veya donanım sürücülerini çalışırken veriyi bit-bit parçalamak için kullanılır.
- ◊ **Güvenlik (safety):** Bu yapı C#'ın "güvenli" (**safe**) dünyasından biraz uzaklaştığı için dikkatli kullanılmalıdır. Yanlış offset değerleri vermek veri bozulmalarına yol açabilir.

Bir başka örnek olarak bir IP adresini düşünebiliriz. Dahili olarak, bir IP adresi bir tam sayı (**int**) olarak tutulur. Ancak bazen `Bayt1.Bayt2.Bayt3.Bayt4` şeklinde farklı bayt öğelerine erişmek isteriz.

```
using System.Runtime.InteropServices;
var ip = new IPAdresi(new Random().Next());
Console.WriteLine($"{ip} = {ip.ipAdresi}"); //33.62.229.21 = 367345185
ip.Bayt1 = 100;
Console.WriteLine($"{ip} = {ip.ipAdresi}"); //100.62.229.21 = 367345252

[StructLayout(LayoutKind.Explicit)]
struct IPAdresi
{
 // "FieldOffset" ipAdresi tamsayısının adresidir.
 // sizeof(int) 4, sizeof(byte) = 1
 [FieldOffset(0)] public int ipAdresi;
```

```
[FieldOffset(0)] public byte Bayt1; // Yapılın 1. byte'ı ofset 0
[FieldOffset(1)] public byte Bayt2; // Yapılın 2. byte'ı ofset 1
[FieldOffset(2)] public byte Bayt3; // Yapılın 2. byte'ı ofset 2
[FieldOffset(3)] public byte Bayt4; // Yapılın 2. byte'ı ofset 3
public IPAdresi(int address) : this()
{
 ipAdresi = address;
}
public override string ToString() => $"{Bayt1}.{Bayt2}.{Bayt3}.{Bayt4}";
}
```

**Referans tiplerini** (sınıflar, metinler vb.) **FieldOffset** kullanarak **aynı adrese çakıştıramazsınız!** Derleyici, bellek güvenliği nedeniyle buna izin vermez. Bu yapı sadece değer tipler (**value type**) olan **int**, **float**, **byte**, **struct** gibi tipler için çalışır.

## 11.3 record Anahtar Kelimesi

C# 9.0 ile hayatımıza giren **record** yapısı, nesne yönelimli programlamada "veriyi temsil etme" görevini sınıflardan devralır. Referans tipi olmasına rağmen değer bazlı karşılaştırma yeteneği sunar;

- Varsayılan olarak **init** property'leri ile gelir. Nesne bir kez oluşturulduktan sonra durumu değiştirilemez. Bu, çok kanallı (**multi-threaded**) ortamlarda güvenliği sağlar.
- ki farklı sınıf nesnesi içindeki veriler aynı olsa bile **==** operatörü ile kıyaslandığında false döner (referans farkından dolayı). İki **record** nesnesinin verileri aynıysa **true** döner
- Tek bir satırda koca bir veri modelini (Constructor, Finalizer/Deconstructor, Equality yöntemleri dahil) tanımlayabilirsiniz.

Bunu bir gerçek hayat örneği ile detaylandıralım. Banka dekontu üzerindeki bilgiler basıldıktan sonra değiştirilemez (**immutability**). Bir hata durumunda mevcut dekontu düzenlemek yerine, hatalı alanın düzeltildiği yeni bir kopyası oluşturulur (**with** ifadesi).

```
public record Kitap(string Isin, string Baslik);

var b1 = new Kitap("123", "C# Kitabı");
var b2 = new Kitap("123", "C# Kitabı");

// Referanslar farklı olsa da içerik aynı olduğu için True döner.
bool esit Mi = (b1 == b2);
```

## 11.4 Öznitelikler

C#'ta **öznitelikler** (**attributes**), bir programı oluşturan sınıf, yöntem, özellik gibi öğelere **bildirim detayını** içeren bir bilgi eklemek için kullanılır. Köşeli parantez [ ] içinde belirtilirler ve kodun çalışma mantığını değiştirmekten ziyade, o kodun nasıl ele alınması gerektiğini tanımlarlar.

### §11.4.1 Hazır Öznitelikler

.NET kütüphanesi ile gelen ve sıkça kullanılan bazı **hazır öznitelikler** (**built-in attributes**) vardır:

- [Obsolete]**: Bir yöntemin artık eskidiğini ve kullanılmaması gerektiğini belirtir. Derleyici bunu gördüğünde uyarı (**warning**) verir.
- [Serializable]**: Bir sınıfın dosyaya veya ağa veya uzak bir ağa aktarılabilir bir biçime dönüştürülebileceğini belirtir.
- [Required]**: Özellikle ileride göreceğimiz ASP.NET ve Varlık çerçevesinde (**Entity Framework**) bir alanın (**field**) boş geçilemeyeceğini tanımlar.

```
public class EskiSistem
{
 [Obsolete("Bu yöntem verimsizdir, lütfen YeniSistem.YeniYöntem() kullanın.")]
 public void EskiYöntem() { /* ... */ }
}
```

### §11.4.2 Kendi Özniteliğimizi Tanımlama

Kendi özniteliklerinizi (**custom attributes**) oluşturmak için **System.Attribute** sınıfından türetilen bir sınıf yazmanız yeterlidir.

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)] // Bu özneliğin nerelerde
→ kullanılabileceğini kısıtlıyoruz.
public class BilgiAttribute : Attribute
{
 public string Yazar { get; }
 public string Versiyon { get; set; }

 public BilgiAttribute(string yazar)
 {
 Yazar = yazar;
 }
}

// Kullanımı:
[Bilgi("İlhan", Versiyon = "2.0")]
public class VeriIsleyici { }
```

### §11.4.3 Çalışma Zamanında Özneliklere Erişme

Öznelikler (**attribute**) tek başına bir iş yapmaz. Onların içindeki bilgiyi okumak için yansıma (**reflection**) kütüphanesini kullanırız.

```
var tip = typeof(VeriIsleyici);
var oznelikler = tip.GetCustomAttributes(typeof(BilgiAttribute), false);

foreach (BilgiAttribute attr in oznelikler)
{
 Console.WriteLine($"Kod Yazarı: {attr.Yazar}, Versiyon: {attr.Versiyon}");
}
```

### §11.4.4 Öznelikler Nerede Kullanılır?

- ♦ **Veritabanı Eşleme:** İleride göreceğimiz varlık çerçevesinde, sınıftaki bir özelliğin veritabanındaki hangi tablo sütununa karşılık geleceğini belirtmek için `[Column("MusteriAdi")]` gibi nesne eşlemesinde ORM (**Object-Relational Mapping**) kullanılır.
- ♦ **Birim Testleri (unit test):** Bir yöntemin test yöntemi olduğunu işaretlemek için `[Test]` şeklinde kullanılır.
- ♦ **JSON İşlemleri:** Sınıftaki bir özelliğin, JSON (**Javascript Object Notation**) içindeki ismini değiştirmek için `[JsonPropertyName("user_id")]` şeklinde kullanılır.

Bir sınıfa ek bilgi eklemek için neden kalıtım (**inheritance**) yerine öznelik kullanmayı tercih ederiz? Kalıtım, bir "biridir" (**is-a**) ilişkisi kurar ve sınıfın yapısını değiştirir. Oysa öznelikler, sınıfa dışarıdan "etiket" yapıştırmak gibidir. Sınıfın hiyerarşisini bozmadan ona çalışma zamanında keşfedilebilecek özellikler kazandırır. Bu da "gevşek bağlılık" (**loose coupling**) sağlar.

## 11.5 Yansıma

**Yansıma** (**reflection**), `System.Reflection` isim uzayı altında bulunur. Derlenmiş olan sınıf kütüphaneleri `*.dll` veya `.NET` uygulaması `*.exe` gibi montaj (**assembly**) dosyalarınızın içindeki tip bilgilerine ulaşmanızı sağlar.

### §11.5.1 Temel Giriş Noktası Type Sınıfı

Yansıma dünyasında her şey `Type` sınıfı ile başlar. Bir sınıfın kimlik kartı gibidir. Bir tipin bilgilerine üç yolla ulaşabilirsiniz:

```
// 1. Tipin adından (Derleme zamanında biliniyorsa)
Type t1 = typeof(Kullanici);
// 2. Örnek bir nesneden
Kullanici k = new Kullanici();
Type t2 = k.GetType();
// 3. String bir isimden (Dinamik yükleme için ideal)
Type t3 = Type.GetType("Sistem.Modeller.Kullanici");
```

### §11.5.2 Çalışma Zamanında Sınıfı Keşfetmek

Yansıma kullanarak bir sınıfın içindeki tüm detayları (**private** olanlar dahil!) dökümleriz. Ancak **vaktinden önce derleme** AOT (**Ahead of Time Compilation**) gibi durumlarda yansıma işlemleri kısıtlanabilir. Vaktinden önce derleme 3.4 başlığında anlatılmıştı.

```
Type t = typeof(Kullanici);
Console.WriteLine($"Sınıf Adı: {t.Name}");
// Yöntemleri (method) listele
foreach (var method in t.GetMethods())
{
 Console.WriteLine($"Metot: {method.Name} - Dönüş Tipi: {method.ReturnType}");
}
// Özellikleri (property) listele
foreach (var prop in t.GetProperties())
{
 Console.WriteLine($"Özellik: {prop.Name}");
}
```

### §11.5.3 Dinamik Nesne Oluşturma ve Yöntem Çağırma

Yansımanın asıl gücü, derleme zamanında (**compile-time**) adını bile bilmediğiniz bir sınıftan nesne üretip onun yöntemini çalıştırmaktır. Ama böyle bir sınıfı, kodunuzun erişebileceği bir noktada (aynı proje veya referans verilmiş bir **\*.dll**) olarak tanımlamalısınız.

```
Type t = typeof(HesapMakinesi);

// 1. Dinamik olarak nesne örneği oluştur (Instantiate)
object instance = Activator.CreateInstance(t);

// 2. "Topla" metodunu bul
MethodInfo method = t.GetMethod("Topla"); //Yöntem bulamaz ve geriye "null" döndürür. Aramada
→ küçük büyük harf ayrımı yapıldığı unutulmamalıdır. Bu yüzden Yansıma kodlarında her zaman
→ null kontrolü yapmak profesyonel bir yaklaşımdır.

// 3. Metodu çalıştır (Invoke)
object[] parametreler = { 5, 10 };
object sonuc = method.Invoke(instance, parametreler);

Console.WriteLine($"Sonuç: {sonuc}"); // Çıktı: 15
```

### §11.5.4 Mahrem Alanlara Müdahale

Yansıma, normal şartlarda erişemeyeceğiniz **private** alanlara bile erişebilir. Böylece kapsüllemeyi (**encapsulation**) delmiş oluruz. Bu, genellikle test araçları veya özel çerçeve (**framework**) yazımlarında kullanılır.

```
FieldInfo field = t.GetField("_gizliSifre", BindingFlags.NonPublic | BindingFlags.Instance);
field.SetValue(instance, "123456"); // Private alanı dışarıdan değiştirdik!
```

### §11.5.5 Yansıma Nerede Kullanılır?

- ♦ **ORM Araçları:** Sınıf isimlerinizden tablo isimlerini, özellik isimlerinizden sütun isimlerini yansıma (**reflection**) kullanılarak eşleştirilir.
- ♦ **Bağımlılık Enjeksiyonu:** Yapıcılar (**constructor**) parametrelerini analiz ederek hangi sınıftan nesne üretmesi gerektiğini yansıma ile anlar.
- ♦ **JSON Serileştiriciler:** **Newtonsoft.Json**, nesnenin içindeki alanları tarayıp onları metin formatına yansıma ile dönüştürür.
- ♦ **Eklenti Sistemleri:** Programınız çalışırken dışarıdan bir **\*.dll** dosyasını tarayıp içindeki uygun sınıfları yüklemek için yansıma kullanılır. Bu, eklenti (**plugin**) için uygun bir yöntemdir

Yansıma (**reflection**) neden maliyetli bir işlemdir? Normal bir yöntem çağırısı derleme zamanında adreslenir ve çok hızlıdır. Ancak yansımada; **string** üzerinden yöntemi arar, güvenlik kontrollerini yapar, parametre tiplerini kontrol eder ve ardından çalıştırır. Bu da normal bir çağrıdan binlerce kat daha yavaş olabilir.

Yansımayı sık çalışan döngülerin içinde kullanmaktan kaçınmalıyız veya sonuçları "önbelleğe" (**cache**) almalıyız.

2026 yılı itibarıyla modern .NET dünyasında yansımanın (**reflection**) en büyük alternatifi **kaynak üreticileri** (**source generators**) teknolojisidir. Yansımanın, çalışma zamanındaki (**run-time**) maliyetine karşın, kaynak üreticilerinin derleme zamanında (**compile-time**) kod üreterek "sıfır maliyetli" bir alternatif sunarlar.

## 11.6 Düşün ve Çöz

- ◇ **Düşün:** Yapı ile sınıf arasındaki bellek yönetimi farkı nedir? Bir yapı stack'te, bir sınıf heap'te bulunur. Bu ayrımın performans açısından ne anlamı vardır? Hangi durumlarda sınıf yerine yapı tercih edilmelidir?
- ◇ **Düşün:** Değer kopyalamanın (**value copy**) yapılar için yarattığı tuzaklar nelerdir? Bir yapı içindeki referans tipi alanlar kopyalandığında ne olur? Bu durum 'savunmacı kopyalama' (**defensive copy**) gerekliliğini nasıl doğurur?
- ◇ **Düşün:** C#10 ile gelen kayıt yapıları (**record struct**) ne zaman düz **struct**'a tercih edilir? Değiştirilemezlik (**immutability**) ile **record struct**'ın nasıl bir ilişkisi vardır?
- ◇ **Düşün:** Öznitelik (**attribute**) mekanizması çalışma zamanında ve derleme zamanında nasıl farklı kullanılır? **[Obsolete]**, **[Serializable]** ve özel özniteliklerin bir framework veya araç için nasıl anlam ifade ettiğini açıklayınız.
- ◇ **Düşün:** Yansıma (**reflection**) nedir ve çalışma zamanında tip bilgisine erişmenin pratik kullanım alanları nelerdir? Yansımanın performans maliyeti var mıdır ve bu maliyet hangi senaryolarda kabul edilemez hale gelir?
- ◇ **Düşün:** Birlik (**union**) benzeri yapılar C#'ta nasıl oluşturulabilir? **[StructLayout(LayoutKind.Explicit)]** özniteliği ne işe yarar? Bu yapının birlikte çalışabilirlik (**interop**) senaryolarındaki önemi nedir?
- ◇ **Düşün:** **readonly struct** ve **in** parametresi anahtar kelimeleri performans optimizasyonunda nasıl kullanılır? Bu özelliklerin olmadığı durumda yapı kopyalamanın yarattığı maliyet nedir?
- ◇ **Çöz:** 2D koordinatı temsil eden Nokta adında bir **struct** tanımlayınız. İki nokta arasındaki uzaklığı hesaplayan, toplamı döndüren ve eşitliği kontrol eden yöntemler ekleyiniz. Aynı işlevi bir class ile uygulayın ve bellek davranışını tartışın.
- ◇ **Çöz:** Özel bir **[DogrulamaKurali]** özniteliği tasarlayın: **string** özellikler üzerinde minimum/maksimum uzunluk kuralları belirlenebilsin. Yansıma kullanarak bu kuralları çalışma zamanında denetleyen bir **Doğrulayici** sınıfı yazınız.
- ◇ **Çöz:** Yansıma kullanarak çalışma zamanında bir sınıfın tüm özelliklerini, yöntemlerini ve alanlarını listeleyen bir 'nesne inceleyici' programı yazınız. Çıktıyı okunabilir biçimde formatlayın.
- ◇ **Çöz:** Bir nesneyi anahtar-değer çiftlerine (sözlük) ve geri çeviren (sözlükten nesneye) dönüştüren; yansımayı kullanan genel bir yardımcı sınıf yazınız.
- ◇ **Çöz:** **[JsonIgnore]** benzeri özel bir **[GizliAlan]** özniteliği oluşturunuz. Yansıma kullanarak bu öznitelikle işaretlenmiş alanları atlayan basit bir nesne serileştirici yazınız.



## 12. Bölüm

# İstisna Yönetimi

### 12.1 Yapısal Programlamada Hata Yönetimi

Yapısal programlamada sorun olan, hata ayıklama kodu ile işlevsel kodun iç içe olması durumudur. Nesne yönelimli programlamanın getirdiği yenilikle bu sorun tarihe karışmıştır. Yapısal programlamada hatalı bir duruma yol açmamak ya da hatalı bir durumla karşılaşıldığında programdan çıkılır. Aşağıda bir C program örneği verilmiştir;

```
#include <stdio.h>
int faktoriyel(int sayi) {
 if (sayi<0) return -1;
 if(sayi <= 1) return 1;
 return sayi * faktoriyel(sayi - 1);
}
int main() {
 int okunan,deger,hata;
 do {
 printf("Bir Sayı Giriniz:");
 scanf("%d",&okunan);
 deger=faktoriyel(okunan);
 printf("%d nin faktöriyeli: %d\n", okunan, deger);
 if (deger==-1) {
 hata=100;
 break; // exit(100);
 }
 } while (okunan!=0);
 return hata;
}
/* Programın Çıktısı:
Bir Sayı Giriniz:-1
-1 nin faktöriyeli: -1
*/
```

Bu program örneğinde `faktoriyel` fonksiyonu hata durumunda `-1` geri döndürmektedir. Ancak gerçek hayatta bu fonksiyonun böyle bir değer döndürmesi mümkün değildir.

### 12.2 İstisna Yönetimi

Nesne yönelimli programlamada (OOP) ise **istisna** (**exception**), bir programın yürütülmesi sırasında ortaya çıkan beklenmeyen bir sorundur. İstisna, çalışma sırasında (**run-time**) oluşur. Bu tür beklenmeyen **özel durumlarda** imal edilen istisna nesneleri (**exception object**) vardır. İstisna iki türlü ortaya çıkar;

1. **Eşzamanlı** (**synchronous**): Giriş verilerindeki bir hata nedeniyle bir şeylerin ters gitmesi veya programın çalıştığı mevcut veri türünü işleme durumunda oluşan istisnalar (örneğin bir sayıyı sıfıra bölmek).
2. **Zaman Uyumsuz** (**asynchronous**): Disk arızası, klavye kesintileri vb. gibi yazılan programın kontrolü dışındaki istisnalar.

İstisnai durumları yönetmek için `throw`, `try` ve `catch` anahtar kelimeleri kullanılır. İstisnaları işlemek için sözde kod aşağıda verilmiştir;

```
try {
 /*
 ...
 İstisnai Durumun Ortaya Çıkabileceği Kod...
 ...
 */
 throw SomeExceptionType("İstisnai Durum Açıklaması");
 /* throw an exception: istisnai bir duruma düşme */
}
```

```

} catch(ExceptionName1 e1) {
 /* catch bloğu try bloğundaki istisnayı yakalamak ve gerekli işlemi yapmak için kullanılır. */
}

```

Bu sözde kod şu şekilde açıklanabilir;

- ◊ **try talimatı:** İçerisinde istisnai bir duruma düşülebileceğini tahmin ettiğimiz bir kod bloğunu temsil eder. Bu bloğu bir veya daha fazla **catch** bloğu takip eder.
- ◊ **catch talimatı:** **try** bloğunda bir istisnai duruma düşüldüğünde bu istisnayı (özel durumu) işlemek için yazılacak kod, **catch** bloğunun içine yazılır.
- ◊ **İstisnai duruma düşme:** Bir istisnai duruma **throw** anahtar sözcüğü kullanılarak da düşülebilir. İstisna nesneleri imal edilince bir havuza atılıyorymuş gibi **istisna nesneleri fırlatılır (throw an exception)**.
- ◊ **İstisna sonrası süreç:** Bir program bir **throw** ifadesiyle karşılaştığında hemen içinde bulunduğu **try** bloğu dışına çıkılır ve ilgili istisnayı işlemek için eşleşen bir **catch** bloğu bulmaya başlar. Bulduğunda istisna havuzdan **bir istisna nesnesi yakalanmış (catch an exception)** olur.
- ◊ Bir **try** bloğu içinde birden fazla **throw** ifadesi bulunabilir.

Bu yeni bakış açısıyla 12.1 başlığındaki yapısal programlama faktöriyel örneği aşağıdaki gibi yazabiliriz;

```

using System;
namespace HataYonetimi
{
 class Program
 {
 // Faktöriyel fonksiyonu
 static int Faktoriyel(int sayi)
 {
 if (sayi < 0)
 {
 throw new ArgumentException("Negatif sayıların faktöriyeli hesaplanamaz!");
 }

 if (sayi <= 1) return 1;
 return sayi * Faktoriyel(sayi - 1);
 }
 //Ana Program
 static void Main()
 {
 int okunan;
 while (true)
 {
 Console.WriteLine("Bir Sayı Giriniz (Çıkış için 0): ");
 if (!int.TryParse(Console.ReadLine(), out okunan)) continue;
 if (okunan == 0) break;
 try
 {
 // Hata riski olan kod bloğu
 int sonuc = Faktoriyel(okunan);
 Console.WriteLine($"{okunan} sayısının faktöriyeli: {sonuc}");
 }
 catch (ArgumentException e) // Özel bir hata türünü yakalıyoruz
 {
 // e.Message fırlatılan hata metnini içerir.
 Console.WriteLine($"HATA: {e.Message}");
 // return -1; // İstenirse program sonlandırılabilir
 }
 catch (Exception e) // Beklenmedik tüm hataları yakalamak için (Genel havuz)
 {
 Console.WriteLine($"Beklenmedik bir hata oluştu: {e.Message}");
 }
 }
 }
 }
}

```

Özel durumları (istisnaları) işlemeye niçin ihtiyaç duyarız?

- ◊ Hata işleme kodunun işlevsel koddan (iş mantığını içeren koddan) ayrılması için.
- ◊ Yöntemler yalnızca seçtikleri istisnaları işleyebilir. Bir yöntemde birden çok istisnai duruma düşülebilir. Ancak bunlardan bazılarını catch bloğunda işlemeyi seçebilir. Geri kalanlarını yöntemi çağıran yere bırakabilir.
- ◊ Hata Türlerinin Gruplanması: C# dilinde yeni bir istisna sınıfı tanımlanabileceği gibi mevcut istisna sınıflarından da istisna nesnesi imal edilebilir.

Aşağıda dizi sınırları ile ilgili bir program örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
int[] array = {10, 20, 30 };
try
{
 array[4]=40;
}
catch (Exception e) //tüm istisna nesneleri yakalanabilir!
{
 Console.WriteLine("Dizi indisi sınırların dışında!");
 Console.WriteLine($"Hata Mesajı:{e.Message}");
 Console.WriteLine($"Hata Verisi:{e.Data}");
 Console.WriteLine($"Hataya Sebep Olan İç Hata:{e.InnerException}");
}
/* Programın Çıktısı:
Dizi indisi sınırların dışında!
Hata Mesajı:Index was outside the bounds of the array.
Hata Verisi:System.Collections.ListDictionaryInternal
Hataya Sebep Olan İç Hata:

...Program finished with exit code 0
*/
```

## 12.3 Standart İstisna Tipleri

Aşağıda sıfıra bölme hatasını işleyen bir program örneği verilmiştir. Örnekte standart istisna tiplerinden biri olan `DivideByZeroException` istisnası imal edilmiştir.

```
using System;
namespace HataYonetimi
{
 class Program
 {
 static void Main()
 {
 Console.WriteLine("Burası her zaman icra edilir.");
 try
 {
 // try BLOĞU: İstisna oluşma ihtimali olan yer
 int bolunen = 10;
 int bolen = 0;
 if (bolen == 0)
 {
 // Manuel olarak hazır bir hata sınıfı fırlatıyoruz
 throw new DivideByZeroException("Sıfıra Bölmeye İzin Verilmez!");
 // throw satırından sonraki kodlar asla çalışmaz
 Console.WriteLine("Burası hiçbir zaman icra edilmez.");
 }
 int sonuc = bolunen / bolen;
 Console.WriteLine($"Bölme Sonucu: {sonuc}");
 }
 catch (DivideByZeroException e) // catch BLOĞU: Sadece sıfıra bölme hatası
 ↪ yakalandığında çalışır
 {
 Console.WriteLine($"İstisna Yakalandı: {e.Message}");
 }
 }
 }
}
```

```

 }
 catch (Exception e) //Varsayılan Catch
 {
 // C++'taki catch (...) bloğunun karşılığıdır.
 // Yukarıdaki özel tiplere uymayan tüm hatalar burada yakalanır.
 Console.WriteLine($"Genel bir hata oluştu: {e.Message}");
 }
 Console.WriteLine("Burası da her zaman icra edilir.");
}
}
}

```

Yukarıdaki örnekte `try` bloğunda izin verilmeyen bölme işleminde istisnai bir durum fark ediliyor ve istisna nesnesi imal ediliyor ve `throw` ile fırlatılıyor (`throw an exception`). `catch` bloğunda ise (istisna nesneleri havuzundan) **istisna nesnesi yakalanarak** (`catch an exception`) işleniyor. Örnekte standart tipte istisna nesnesi (`DivideByZeroException`) kullanılmıştır. Bunun gibi kullanılacak istisna nesnelerinin listesi Tablo 12.1'de verilmiştir;

| İstisna Tipi                             | Açıklama                                                                                                                                         |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ArithmeticException</code>         | Aritmetik işlemler sırasında oluşan hatalar için temel sınıftır ( <code>DivideByZeroException</code> ve <code>OverflowException</code> gibi).    |
| <code>ArrayTypeMismatchException</code>  | Bir diziyeye, dizinin türüyle uyumsuz bir öğe yerleştirilmeye çalışıldığında fırlatılır.                                                         |
| <code>DivideByZeroException</code>       | Bir tamsayı değerini sıfıra bölme girişimi gerçekleştiğinde fırlatılır.                                                                          |
| <code>IndexOutOfRangeException</code>    | Bir dizinin sınırları dışındaki (negatif veya boyuttan büyük) bir indekse erişilmeye çalışıldığında fırlatılır.                                  |
| <code>InvalidCastException</code>        | Zorlama ( <code>casting</code> ) sırasında geçersiz bir tip dönüşümü işlemi çalışma zamanında başarısız olduğunda fırlatılır.                    |
| <code>NullReferenceException</code>      | Değeri <code>null</code> olan bir nesne referansı üzerinden bir üyeye (yöntem, özellik vb.) erişilmeye çalışıldığında fırlatılır.                |
| <code>OutOfMemoryException</code>        | Yeni bir nesne oluşturmak ( <code>new</code> ) için yeterli sistem belleği kalmadığında fırlatılır.                                              |
| <code>OverflowException</code>           | <code>checked</code> blokları içinde yapılan aritmetik işlemlerin sonucunun, veri tipinin sınırlarını aşması durumunda fırlatılır.               |
| <code>StackOverflowException</code>      | Yürütme yığını ( <code>stack</code> ), genellikle sonsuz özyineleme ( <code>recursion</code> ) nedeniyle tükendiğinde fırlatılır.                |
| <code>TypeInitializationException</code> | Statik bir yapıcı ( <code>static constructor</code> ) hata verdiğinde veya statik bir alan ( <code>field</code> ) başlatılamadığında fırlatılır. |

Tablo 12.1: Yaygın Sistem İstisnaları

Bir `try` bloğunda (istisna havuzuna) birden fazla istisna nesnesi fırlatılabilir. Ama tüm `catch` talimatlarıyla bu istisnalar yakalanamayabilir. Ayrıca tüm istisna nesnelerin tipleri de bilinmeyebilir. Bu durumda `catch (Exception e)` şeklinde kodlanan varsayılan `catch` kullanılır.

Sistem tarafından istisnaların yakalanması yerine istisnaları biz `throw` anahtar kelimesiyle yeni bir istisna nesnesi imal edip yönetebiliriz. Anı örnek bu durumda aşağıdaki gibi yazılabilir;

```

int[] array = {10, 20, 30 };
try
{
 int indis = 4;
 if (indis >= array.Length) // İstisnai durum
 {
 //İstisna nesnesi oluşturulup (istisna havuzuna) fırlatılıyor
 throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
 }
 array[indis]=40;
}
catch (Exception e)
{
 Console.WriteLine($"Hata Mesajı:{e.Message}");
 Console.WriteLine($"Hata Verisi:{e.Data}");
}

```

```

 Console.WriteLine($"Hataya Sebep Olan İç Hata:{e.InnerException}");
}

```

Bir `try` bloğunda birden fazla istisna nesnesi (istisna havuzuna) fırlatılabilir; Aşağıdaki örnekte `try` bloğunda birden fazla istisnai nesnesi fırlatılmıştır.

```

int[] array = {10, 20, 30 };
try
{
 int indis = 2;
 if (indis >= array.Length) //koşul sağlanmaz!
 {
 throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
 }
 array[2]=40;
 int bolen=0, bolunen=100;
 if (bolen == 0)
 {
 throw new DivideByZeroException("Bölme işlemi sıfıra bölünemez!");
 }
}
catch (IndexOutOfRangeException e)
{
 Console.WriteLine($"Hata Mesajı:{e.Message}");
 Console.WriteLine("İndis Hatası Olunca Neler Yapılmalı? Burada Yazılır.");
}
catch (DivideByZeroException e)
{
 Console.WriteLine($"Hata Mesajı:{e.Message}");
 Console.WriteLine("Sıfıra Bölme Olunca Neler Yapılmalı? Burada Yazılır.");
}
/*Program Çıktısı:
Hata Mesajı:Bölme işlemi sıfıra bölünemez!
Sıfıra Bölme Olunca Neler Yapılmalı? Burada Yazılır.
*/

```

Yukarıdaki programda ilk karşılaşılan özel durum sıfıra bölme durumudur ve `throw` ile (istisna havuzuna) `DivideByZeroException` nesnesi fırlatılır. Bu durumda `catch` bloklarına sırasıyla bakılır ilgili özel durum nesnesini işleyecek olan `catch` bloğu icra edilir.

Aşağıda `Kişi` sınıfta `Yaş` özelliğine ilişkin istisna içeren bir kod örneği verilmiştir;

```

//ANA PROGRAM
Kişi ali = new Kişi();
try
{
 // 1. Geçerli bir değer atama
 ali.Yaş = 12;
 Console.WriteLine($"Güncel Yaş: {ali.Yaş}");
 // 2. İstisna yaratacak değer (Hata fırlatıldığı an try bloğu terk edilir)
 ali.Yaş = -1;
 // --- BU SATIRDAN SONRASI İCRA EDİLMEZ ---
 Console.WriteLine($"Bu satıra ulaşamayacak: {ali.Yaş}");
}
catch (ArgumentOutOfRangeException ex)
{
 Console.WriteLine("İstisnai Bir Durum Yakalandı!");
 // ex.ParamName hataya sebep olan parametreyi, ex.Message ise açıklamayı verir.
 Console.WriteLine($"Hata Mesajı: {ex.Message}");
}
catch (Exception ex)
{
 Console.WriteLine($"Beklenmedik Hata: {ex.Message}");
}

```

```
//SINIF
public class Kişi
{
 private int _yaş; // Field (Alan)
 public Kişi()
 {
 _yaş = 15; // Varsayılan değer
 }
 public int Yaş
 {
 get { return _yaş; }
 set
 {
 if (value < 0)
 throw new ArgumentOutOfRangeException(nameof(Yaş), "Geçersiz Yaş Değeri: < 0!");
 else if (value > 120)
 throw new ArgumentOutOfRangeException(nameof(Yaş), "Çok Büyük Yaş Değeri: > 120!");
 _yaş = value;
 }
 }
}
```

## 12.4 İstisna Filtreleri

Bazı durumlarda birden fazla istisna tipini tek bir `catch` ile yakalamak isteriz. Bu durumda `when` süzgeci kullanılabilir;

```
int[] array = { 10, 20, 30 };
try
{
 int indis = 2;
 if (indis >= array.Length) //koşul sağlanmaz!
 {
 throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
 }
 array[2] = 40;
 int bolen = 0, bolunen = 100;
 if (bolen == 0)
 {
 throw new DivideByZeroException("Bölme işlemi sıfıra bölünemez!");
 }
}
catch (Exception e) when (e is IndexOutOfRangeException || e is DivideByZeroException)
{
 Console.WriteLine($"Hata Mesajı:{e.Message}");
 Console.WriteLine("Hata Olunca Neler Yapılmalı? Burada Yazılır.");
}
```

## 12.5 Kullanıcı Tanımlı İstisnalar

Kendi istisna tipimizi oluşturmak için `Exception` sınıfından türemiş bir sınıf yazarız. Aşağıda `Kişi` sınıfta yaş özelliğine ilişkin istisna içeren bir kod örneği verilmiştir;

```
using System.Diagnostics; // StackTrace sınıfı için gerekli
int[] array = { 10, 20, 30 };
try
{
 int indis = 4;
 if (indis >= array.Length)
 {
 //throw new İndisİstisnası(); //Bu satır yada aşağıdaki
 string currentFile = new StackTrace(true).GetFrame(0).GetFileName();
 int currentLine = new StackTrace(true).GetFrame(0).GetFileLineNumber();
 }
}
```

```

 throw new Dosyaİsimliİndisİstisnası(currentFile, currentLine);
 }
 array[indis] = 40;
}
catch (İndisİstisnası e)
{
 Console.WriteLine($"Hata Mesajı:{e.Message}");
 Console.WriteLine("İndis Hatası Olunca Neler Yapılmalı? Burada Yazılır.");
}
catch (Dosyaİsimliİndisİstisnası e)
{
 Console.WriteLine($"Hata Mesajı:{e.Message}");
 // Hata mesajı: D:\CSharp\Samples\Program.cs Dosyası 12 Satırında İndis Hatası!
 Console.WriteLine($"Hata Mesajı:{e.FileName}");
 // Hata mesajı: D:\CSharp\Samples\Program.cs
 Console.WriteLine($"Hata Mesajı:{e.LineNumber}");
 // Hata mesajı: 12
}

public class İndisİstisnası : Exception
{
 public İndisİstisnası() : base("Dizide olmayan bir elemana indis belirlendi!")
 { }
}

public class Dosyaİsimliİndisİstisnası : Exception
{
 public Dosyaİsimliİndisİstisnası(string fileName, int lineNumber) :
 base($"{fileName} Dosyası {lineNumber} Satırında İndis Hatası!")
 {
 FileName = fileName;
 LineNumber = lineNumber;
 }
 public string FileName { get; private set; }
 public int LineNumber { get; private set; }
}

```

## 12.6 finally Bloğu

**finally** bloğu, bir **try** bloğunda gerçekleştirilen eylemleri temizlemenizi sağlar. Varsa, **finally** bloğu en son bloktur ve eşleşen **catch** bloklardan sonra icra edilir. Özel bir durum/istisna oluşturma veya özel durum/istisna tipiyle eşleşen bir **catch** blok bulunması fark etmeksizin **finally** bloğu her zaman çalışır. Ayrıca **finally** bloğu, **try** içinde **return** olsa dahi mutlaka çalıştırılır. Sözde kod aşağıda verilmiştir;

```

try
{
 /* istisna olabilecek kod burada yazılır. */
}
catch (Exception e)
{
 /* istisnayı yönetme kodu buraya yazılır. */
}
finally
{
 /* Bir istisna atılmış/yakalanmış olsun veya olmasın, yürütülecek kod buraya yazılır. */
}

```

Burada **catch** bloğu programcı tarafından kodlanmayabilir. Dosyalarla ilgili bir örnek aşağıda verilmiştir;

```

FileStream f = null;
try
{
 f = File.OpenRead("file.txt");
}
finally

```



```
{
 f?.Close(); /* f null olabilir, bu nedenle null koşullu işlecini kullanın.
 Çünkü dosya bir şekilde açılmamış olabilir. */
}
```

## 12.7 İstisnayı Tekrar Fırlatma

Bazı durumlarda istisna nesnesinin yakalanıp `catch` bloğunda işlenmesi sırasında mevcut istisna nesnesini `throw` ile (istisna havuzuna) **tekrar fırlatmak** (**re-throw**) gerekir.

```
try
{
 //...
}
catch (Exception ex)
{
 //...
 throw;
 //yada throw ex;
 //derleyiciye icra akışının asla try bloğundan ayrılmadığını bildirmek için yapılır.
}
```

`throw ex;` yığın izini sıfırlarken, `throw;` orijinal hatanın nerede oluştuğu bilgisini korur. Bu, kurumsal yazılımlarda hata ayıklama için hayati önemdedir. Katmanlı mimaride bir katmandan diğerine istisnaları iletmek için bu yöntem kullanılır. Genellikle iç katmanlarda hatanın gösterileceği bir kullanıcı ara yüzü bulunmaz.

## 12.8 Yığın Geri Sarma

Program akışı sırasında bir istisna (**exception**) fırlatıldığında, .NET çalışma zamanı (CLR) mevcut yöntemden başlayarak yukarıya doğru (çağrı hiyerarşisinde) uygun bir `catch` bloğu aramaya başlar. Bu arama sürecine ve bu sırada yerel değişkenlerin bellekten temizlenmesine **yığın geri sarma** (**stack unwinding**) denir.

Bir hata fırlatıldığı anda şu adımlar izlenir:

- ◊ **Arama:** CLR, hatanın oluştuğu metotta bir `try-catch` bloğu olup olmadığına bakar.
- ◊ **Geri Sarma:** Eğer bulamazsa, mevcut metodu sonlandırır ve onu çağıran bir üst metoda geçer. Bu sırada o metodun yığındaki (**stack**) yerel değişkenleri temizlenir.
- ◊ **Sonlandırma:** Her bir adımda, eğer varsa `finally` blokları icra edilir.
- ◊ **Sonuç:** Uygun bir `catch` bloğu bulunana kadar bu işlem "yığını geri sararak" devam eder. Eğer hiçbir yerde yakalanmazsa uygulama çöker (**unhandled exception**).

Aşağıdaki örnekte, hatanın en derinden en yukarıya nasıl tırmandığını görebilirsiniz:

```
using System;
class Program
{
 static void Main()
 {
 try {
 AYöntemi();
 }
 catch (Exception ex) {
 Console.WriteLine($"Hata Main'de Yakalandı: {ex.Message}");
 // StackTrace sayesinde hatanın hangi yollardan geçtiğini görebiliriz.
 Console.WriteLine(ex.StackTrace);
 }
 }
 static void AYöntemi() => BYöntemi();
 static void BYöntemi()
 {
 Console.WriteLine("BYöntemi çalışıyor...");
 throw new InvalidOperationException("Derinlerde bir hata oluştu!");
 // Buradan sonra Stack Unwinding başlar:
 // 1. BYönteminden çıkılır.
 // 2. AYönteminde catch yok, oradan da çıkılır.
 }
}
```

```

 // 3. Main'deki catch bloğuna ulaşılır.
 }
}

```

## 12.9 Özel Durum İşleme Kuralları

### Kontrol Akışını İstisna Kodları ile Gerçekleştirmeyin!

İstisnaları (özel durumları) işleyen kod ile iş mantığını içeren kod nesne yönelimli paradigmada her zaman ayrı olmalıdır! Bunun için;

- ♦ İş mantığını içeren kontrol akışı (**control flow**) istisna kodlarıyla yapılmamalıdır. Bunun yerine kontrol işlemleri (**control operations**) ile iş mantığı gerçekleştirilmelidir.
- ♦ Ayrıca Bir kontrol akışı, **if-else** ifadeleriyle açıkça yapılabiliriyorsa, okunabilirliği ve performansı azalttığı için istisnalar kullanılmaz.

Aşağıdaki örneği göz önüne alalım;

```

object myObject;
void NesneYaz()
{
 Console.WriteLine(myObject.ToString()); //Bunu YAPMA!
}

```

Bu örnekle `Console.WriteLine` satırı icra edilmeye çalışıldığında `NullReferenceException` istisnası oluşur. Bunu yönetmek için aşağıdaki gibi bir kod yazmamalıyız;

```

// Bunu YAPMA!
object myObject;
void NesneYaz()
try
{
 Console.WriteLine(myObject.ToString());
}
catch(NullReferenceException ex)
{
 // eğer object'in yeni bir örneğini oluşturup myObject'e atarsam sorunu çözebilirim.
 myObject = new object();
 // artık myObject ile çalışmaya devam edebilirim
 NesneYaz();
}

```

Doğru olan kodlama aşağıdaki gibi olmalıdır;

```

// Bunu YAP!
object myObject;
void NesneYaz()
{
 if(myObject == null)
 myObject = new object();
 // Programın icrası bu noktaya ulaştığında, myObject'in null olmadığından emin oluruz
 NesneYaz();
}

```

### Zorunlu Olmadıkça İstisnayı Tekrar İmal Etmeyin!

İstisnaları **tekrar fırlatmak** (**re-throw**) pahalıdır. Performansı olumsuz etkiler! Rutin olarak başarısız olan kodlar için, performans sorunlarını en aza indirmek için **tasarım desenlerini** (**design pattern**) kullanabilirsiniz. Microsoft sitesinde yer alan İstisnalar ve Performans konusu istisnaların performansı önemli ölçüde etkileyebileceği zamanlarda faydalı olan iki tasarım desenini açıklamaktadır.

### İz Kaydı Olmadan İstisnaları Görmezden Gelmeyin!

Aşağıdaki gibi bir istisna işleme kötü bir uygulamadır;

```

try
{
 //...
}

```

```
catch(Exception ex) { } // Boş catch bloğu kötü uygulamadır.
```

İstisnaları görmezden gelmek o anı kurtaracaktır ancak daha sonra sürdürülebilirlik için bir kaos yaratacaktır. İstisnaların iz kaydını (log) günlüğe kaydederken, istisna örneğini her zaman kaydetmelisiniz. Böylece tüm yığın izi günlüğe kaydedilir ve yalnızca istisna mesajı kaydedilmez! Aşağıdaki gibi bir kod daha doğru olabilir;

```
try
{
 //...
}
catch(NullReferenceException ex)
LogManager.Log(ex.ToString());
```

### İşleyemeyeceğiniz İstisnaları Yakalamayın!

İşleyemeyeceğiniz istisnaları yakalamayın. Sadece yakalamayın. Yalnızca o konumda işleyebildiğiniz bir istisnayı yakalamalısınız. Bunun yerine, iyi bir çözümünüz olana veya ana fonksiyona ulaşana kadar bunların çağrı ağacında yukarı doğru ilerlemesine izin verin, bu durumda hata bilgilerinin doğru şekilde yönlendirildiğinden emin olabilirsiniz.

## 12.10 Düşün ve Çöz

- ◊ **Düşün:** İstisna yönetimi (exception handling) ile geleneksel hata kodu döndürme yöntemi arasındaki farkları karşılaştırınız. Hangi durumlarda istisna fırlatmak, hangi durumlarda null veya bool döndürmek daha uygundur?
- ◊ **Düşün:** catch(Exception e) bloğu ile tüm istisnaları yakalamak neden kötü bir pratik olarak değerlendirilir? 'Pokemon exception handling' ifadesi neyi eleştirir?
- ◊ **Düşün:** finally bloğu ne zaman çalışır? Bir veritabanı bağlantısını finally yerine using bloğuyla kapatmanın neden daha iyi bir tasarım olduğunu açıklayınız.
- ◊ **Düşün:** İstisnayı yeniden fırlatmak (rethrow) için throw; ile throw ex; arasındaki fark nedir? Bu farkın hata ayıklama (debugging) sürecindeki önemi nedir?
- ◊ **Düşün:** Kullanıcı tanımlı istisna (custom exception) ne zaman yazılmalıdır? Exception sınıfından mı, ApplicationException'dan mı türetilmelidir? Bir stok yönetim sisteminde hangi özel istisnalar tanımlamak mantıklı olur?
- ◊ **Düşün:** İstisna hiyerarşisi (Exception → SystemException → IOException gibi) catch bloklarını nasıl etkiler? Daha spesifik istisnaları daha üstteki catch bloklarına yazmak neden derleme hatasına yol açar?
- ◊ **Düşün:** try-catch bloklarının performans maliyeti var mıdır? İstisnaların normal akış kontrolü için (örneğin int.Parse yerine try-catch) kullanılması neden önerilmez?
- ◊ **Çöz:** Kullanıcıdan iki sayı ve bir işlem (+, -, \*, /) alarak sonucu hesaplayan; sıfıra bölme, geçersiz işlem ve sayısal olmayan giriş durumlarını ayrı catch bloklarıyla ele alan bir hesap makinesi programı yazınız.
- ◊ **Çöz:** YetersizBakiyeException ve HesapKapaliException adında iki özel istisna sınıfı oluşturun. Bunları bir banka hesabı sınıfında uygun yerlerde fırlatın ve test edin.
- ◊ **Çöz:** Bir dosyadan satır satır sayı okuyan ve toplayan program yazınız. Dosya bulunamazsa, satır sayısal değilse veya beklenmedik bir hata oluşursa her durum için ayrı hata mesajı gösterilsin. finally bloğuyla dosya kapatma işlemini güvence altına alın.
- ◊ **Çöz:** AggregateException kullanarak birden fazla hatayı tek bir istisna nesnesinde toplayan bir senaryo kurunuz. Bu yapının paralel programlamada neden kullanıldığını açıklayınız.
- ◊ **Çöz:** İstisna filtreleri (exception filters, when koşulu) kullanarak HTTP durum koduna göre farklı davranış sergileyen bir örnek yazınız.

## 13. Bölüm

# Tip Dönüşümleri ve Göstericiler

### 13.1 Tip Dönüşümleri

C# dilinde derleme öncesi bir değişken bildirildikten sonra, bu tip örtük olarak bir başka değişkenin tipine dönüştürülebilir olmadığı sürece tekrar bildirilemez veya başka bir tipte bir değer atanamaz. Örneğin, `string` örtük olarak `int` veri tipine dönüştürülemez. Bu nedenle, bir değişken `int` olarak bildirdikten sonra, aşağıdaki koddaki gibi, ona "11" metnini atayamazsınız;

```
int i;
i = "11"; // error CS0029: can't implicitly convert type 'string' to 'int'
```

Ancak bazen bir değeri başka bir tipin değişkenine veya yöntem parametresine kopyalamanız gerekebilir. Örneğin, `double` parametresi bekleyen bir yönteme geçirmeniz gereken bir tam sayı değişkeniniz olabilir. Bir başka örnek olarak bir sınıf örneğini (`instance`) bir arayüz (`interface`) tipinin değişkenine atamanız gerekebilir. Bu tür işlemlere tip dönüşümleri denir;

- ♦ **Örtük dönüşümler (implicit conversion):** Dönüştürme her zaman başarılı olduğundan ve veri kaybı olmadığından özel bir söz dizimi gerekmez. Örnek olarak küçükten daha büyük tam sayı türlerine dönüştürmeler ve türetilmiş sınıflardan temel sınıflara dönüştürmeler verilebilir.
- ♦ **Açık dönüşümler (explicit conversion)** veya zorlama (casting): Açık dönüşümler bir atama ifadesi gerektirir. Dönüşümde bilgi kaybolabileceği veya dönüştürmenin başka nedenlerle başarılı olmaya-bileceği durumlarda atama gereklidir. Buna örnek olarak da daha az duyarlığa veya daha küçük bir belleğe sahip bir tipe sayısal dönüştürme ve temel sınıf örneğinin türetilmiş bir sınıfa dönüştürülmesi verilebilir.
- ♦ **Kullanıcı tanımlı dönüşümler:** Kullanıcı tanımlı dönüştürmeler, temel sınıf türetilmiş sınıf ilişkisi olmayan özel türler arasında açık ve örtük dönüştürmeleri etkinleştirmek için tanımlayabileceğiniz özel yöntemler kullanır.
- ♦ **Yardımcı sınıflarla dönüşümler:** Uyumlu olmayan tipler arasında dönüştürme yapmak için yardımcı sınıflar kullanılır. `System.Convert`, `Parse`, `Int32.Parse` ve `System.BitConverter` bu yardımcı sınıf ve yöntemleridir.

### 13.2 Örtük Dönüşümler

Derleyicinin herhangi bir özel müdahale gerektirmeden, verinin güvenli bir şekilde taşınabileceğini garanti ettiği durumlarda **örtük dönüşüm (implicit conversion)** uygulanır. Genellikle küçük (bellekte az yer kaplayan) bir veri türü, kendisinden daha geniş bir türe aktarılırken kullanılır. Veri kaybı ihtimali yoktur. Hedef veri tipin kapasitesi, kaynak veri tipinden büyük olmalıdır.

```
int tamSayi = 123;
double ondalikliSayi = tamSayi; // Implicit: int, double'ın içine rahatça sığar.
```

```
char karakter = 'A';
int asciiKarsiligi = karakter; // Implicit: char 16-bit, int 32-bit'tir.
```

Referans tipler için, örtük dönüştürme her zaman bir sınıftan doğrudan veya dolaylı temel sınıflarından veya arayüzlerinden herhangi birine var olur. Türetilmiş bir sınıf her zaman bir temel sınıfın tüm üyelerini içerdiğinden özel söz dizimi gerekmez.

```
Derived d = new Derived();
Base b = d; // Türetilmiş sınıf, her zaman taban sınıfa örtük olarak dönüştürülür.
```

### 13.3 Açık Dönüşümler

Veri kaybı riski olan veya dönüşümün her zaman başarılı olamayacağı durumlarda **açık dönüşümler (explicit conversion)** yapılır. Derleyiciye "Ne yaptığımı biliyorum, sorumluluk bende" demek için parantez içinde hedef tür belirtilerek dönüşüme zorlanır (`casting`). Veri kaybı veya taşma yaşanabilir. Bundan dolayı risk yüksektir. Büyük (bellekte çok yer kaplayan) veri tipinden küçük veri tipine geçerken veya uyumsuz türler arasında kullanılır. Bu dönüşümde bir atama gerçekleştirmek için, dönüştürülecek değerin veya değişkenin

önündeki parantez içinde atama yaptığınız veri tipi belirtilerek **zorlama işleci** (**cast operator**) kullanılır;

```
double pi = 3.14;
int tamPi = (int)pi; // Explicit: Ondalık kısım (.14) atılır. Veri kaybı!

long büyükSayi = 9876543210;
int küçükSayi = (int)büyükSayi; // Explicit: Veri taşması yaşanabilir! int veri tipine zorlandı
→ (cast)
```

Bu program, **zorlama** (**cast**) olmadan derlenmez. Referans tipler için, bir taban tipten türetilmiş tipe dönüştürmeniz gerekiyorsa zorlama yine gereklidir;

```
Derived d = new Derived(); // Türetilmiş tipten bir nesne imal ediliyor.
Base b = d; // Örtük (implicit) olarak taban tipe dönüştürülebilir.
/*
Ancak bu taban tipteki b nesnesi türetilmiş tipe dönüştürülecek ise açık olarak dönüştürülmesi
→ gerekir.
*/
Derived d2 = (Derived)b;
```

Referans tipleri arasındaki dönüştürme işlemi, temel alınan nesnenin çalışma zamanı (**run-time**) tipini değiştirmez. Yalnızca bu nesneye referans olarak kullanılan değerin tipini değiştirir. Yani çok biçimlilik (**polymorphism**) burada uygulanır. Bazı referans tipi dönüştürmelerinde, derleyici bir zorlamanın geçerli olup olmadığını belirleyemez. Doğru derlenen bir açık dönüşüm işleminin çalışma zamanında başarısız olması mümkündür;

```
Canlı canlı = new Hayvan();
Bitki bitki = (Bitki)canlı; // Derlenir ama çalıştırma anında hata olur:InvalidCastException
```

```
class Canlı
{
 public virtual string ToString() => "Ben Bir Canlıyım.";
}
class Hayvan : Canlı {
 public void HareketEt() => System.Console.WriteLine("Hareket Ediyorum..");
}
class Bitki : Canlı {
 public void FotosentezYap() => System.Console.WriteLine("Fotosentez Yapıyorum..");
}
```

Örtük ve açık dönüşümler aşağıdaki tabloda özetlenmiştir;

| Özellik          | Örtük                           | Açık                                                        |
|------------------|---------------------------------|-------------------------------------------------------------|
| <b>Güvenlik</b>  | Tamamen Güvenli                 | Riskli (Veri kaybı/Taşma)                                   |
| <b>Sözdizimi</b> | Doğrudan atama ( <b>a = b</b> ) | Zorlama işleci ( <b>cast operator</b> ) ( <b>((int)b)</b> ) |
| <b>Yön</b>       | Genişleten ( <b>widening</b> )  | Daraltan ( <b>narrowing</b> )                               |
| <b>Derleyici</b> | Otomatik yapar                  | Müdahale bekler                                             |

Tablo 13.1: Örtük ve Açık Dönüşüm Karşılaştırması

Açık dönüşüm sırasında bir **long** değeri **int** kapasitesini aşarsa ne olur? C# derleyicisi bizi uyarır mı? Varsayılan olarak C# taşmaları görmezden gelir (**silent overflow**) ve yanlış bir sonuç üretir. Ancak bu dönüşümü bir **checked** bloğu içine alırsanız, taşma yaşandığında program **OverflowException** fırlatır. Özel olarak bu kontrol yapılmasını istiyorsanız **unchecked** bloğu yazarsınız.

```
long uzunTamsayı=0x11FFFFFFFFFFFFFFF;
int sonuç;
sonuç = (int)uzunTamsayı; // Limit aşılrırsa hata fırlatmaz!
Console.WriteLine($"{sonuç:X}"); //FFFFFFF

unchecked {
 sonuç = (int)uzunTamsayı; // Limit aşılrırsa hata fırlatmaz!
 Console.WriteLine($"{sonuç:X}"); //FFFFFFF
}

checked {
 sonuç = (int)uzunTamsayı; /*Limit aşılrırsa hata fırlatır! Unhandled exception.
→ System.OverflowException: Arithmetic operation resulted in an overflow. */
```

```
Console.WriteLine($"{sonuç:X}");
}
```

### 13.4 Kullanıcı Tanımlı Dönüşümler

Kullanıcı tanımlı bir veri tipi, aynı iki tip arasında standart bir dönüşüm olmadığı sürece, başka bir tipe özel olarak **örtük** (**implicit**) veya **açık** (**explicit**) dönüşüm tanımlayabilir. Örtük dönüşümler, çağrılması için özel bir söz dizimi gerektirmez. Bu dönüşümler atamalarda ve yöntem çağrılarında meydana gelebilir. Önceden tanımlanmış örtük dönüşümleri her zaman başarılı olur ve asla bir istisna oluşturmaz. Kullanıcı tanımlı örtük dönüşümler de bu şekilde davranmalıdır.

Özel bir dönüşüm bir istisna oluşturabilir veya bilgi kaybedebilirse, bunu açık bir dönüşüm olarak tanımlamamız gerekir. Nesnelerle ilgili olan 8.20 başlığındaki **is** ve **as** işleçleri kullanıcı tanımlı dönüşümleri dikkate almaz. Kullanıcı tanımlı açık bir dönüşümü çağırarak için bir dönüştürme ifadesi kullanılması gerekir.

Sırasıyla örtük veya açık bir dönüşümü tanımlamak için **implicit operator** veya **explicit operator** anahtar sözcükleri kullanılır. İşleçlerin **static** tanımlandığı daha önce anlatılmıştı. Bir dönüşümü tanımlayan tip, o dönüşümün kaynak tipi veya hedef tipi olmalıdır. İki kullanıcı tanımlı tip arasındaki bir dönüşüm, iki tipten herhangi birinde tanımlanabilir.

```
// Implicit Kullanımı: Parantez içinde veri tipi belirtmeye gerek yok.
double doubleHız = 300.0;
Hız hızNesnesi = doubleHız;

int tamSayıHız = (int)hızNesnesi; // Explicit Kullanımı: (int) yazmak zorunludur.

Console.WriteLine($"Hız Nesnesi: {hızNesnesi.MetreSaniye} m/s"); //Hız Nesnesi: 300 m/s
Console.WriteLine($"Tam Sayı Hız: {tamSayıHız}"); //Tam Sayı Hız: 300

public struct Hız
{
 public double MetreSaniye { get; set; }

 // Implicit: double'dan Hız'a geçerken casting gerekmesin
 public static implicit operator Hız(double d) => new Hız { MetreSaniye = d };

 // Explicit: Hız'dan int'e geçerken hassasiyet kaybolacağı için zorunlu kılalım
 public static explicit operator int(Hız h) => (int)h.MetreSaniye;
}
```

### 13.5 Yardımcı Sınıflarla Dönüşümler

Çoğu programımızda klavyeden girilen metni bir metin (**string**) veri tipinde değişkenlerle işleriz. Bazı durumlarda da bu dizgileri değer tiplere dönüştürürüz. Bunun için **ToString()**, **System.Convert()** ve **TryParse()** yöntemlerini kullanırız.

```
using System.Globalization;

DateTime zaman = new(1971,9,1,11,53,0);
Console.WriteLine(zaman.ToString("yyyy-MM-dd HH:mm")); // 1971-09-01 11:53
Console.WriteLine(zaman.ToString(CultureInfo.InvariantCulture)); // 09/01/1971 11:53:00
Console.WriteLine(zaman.ToString(new CultureInfo("en-us"))); // 9/1/1971 11:53:00 AM
Console.WriteLine(zaman.ToString(new CultureInfo("tr-TR"))); // 1.09.1971 11:53:00
Console.WriteLine(zaman.ToString(new CultureInfo("ja-JP"))); // 1971/09/ 01 11:53:00

string doubleMetin = "12,53";
double sayi = Convert.ToDouble(doubleMetin); //YARDIMCI SINIF ile Dönüşüm
Console.WriteLine(sayi); // 12,53
string intMetin = "6161";
int num = int.Parse(intMetin);
Console.WriteLine(num); // 6161
if (int.TryParse(intMetin, out int result)) //YARDIMCI SINIF ile Dönüşüm
{
 Console.WriteLine(result); // 6161
}
```



```
else
{
 Console.WriteLine("Dönüştürme Başarısız Oldu.");
}
```

## 13.6 Kutulama ve Kutudan Çıkarma

### §13.6.1 Kutulama

Kutulama (**boxing**), bir değer tipinin (**int**, **struct**, **double**, ...) bir referans tipine (**object** veya arayüz) dönüştürülmesi işlemidir. Bu işlem sırasında:

- ◊ Öbek (**heap**) bölgesinde yeni bir nesne tahsis edilir.
- ◊ Yığın(**stack**) bölgesindeki değer, Öbek (**heap**) bölgesindeki bu yeni nesnenin içine kopyalanır.
- ◊ Yığın (**stack**) bölgesinde bu nesnenin adresini tutan bir referans oluşturulur.

```
int sayi = 123; // Değer tipi (Stack)
object kutu = sayi; // BOXING (Heap'e taşındı)
```

### §13.6.2 Kutudan Çıkarma

Kutudan çıkarma (**unboxing**), öbek(**heap**) bölgesinde kutulanmış olan değer tekrar kendi orijinal değer tipine dönüştürülmesidir. Bu işlem açıkça (**explicit**) bir tür dönüşümü olan zorlama (**cast**) gerektirir.

```
object kutu = 123; // Boxing
int sayi = (int)kutu; // UNBOXING (Heap'ten Stack'e geri kopyalandı)
```

Kutudan çıkarma (**unboxing**) yaparken tipin birebir uyuşması gerekir. Eğer **int** olarak kutulanan bir değeri **double** olarak çıkarmaya çalışırsanız **InvalidCastException** hatası alırsınız. Ayrıca performans etkisi de göz önüne alınmalıdır;

- ◊ **Zaman Maliyeti:** Kutulama (**boxing**) işlemi, yeni bir nesne tahsis etme ve veri kopyalama gerektirdiği için doğrudan atamalardan çok daha yavaştır.
- ◊ **Bellek Maliyeti:** Öbek (**heap**) bölgesinde sürekli yeni nesneler oluşmasına neden olur. Bu da **çöp toplayıcı** (GC) üzerindeki baskıyı artırır ve uygulamanın çöp toplayıcıdan dolayı ara ara "donmasına" yol açabilir.

Örnek olarak aşağıdaki kodu inceleyelim.

```
int yaş = 25;
Console.WriteLine("Yaşınız: " + yaş);
```

Yukarıda yazılan kodda kaç tane kutulama yapılmıştır? Burada gizli bir kutulama (**boxing**) vardır! Konsola bir şey yazmak için kullanılan **Console.WriteLine** veya **string.Concat** gibi yöntemler, sayısal bir değeri **string** ile birleştirirken onu **object** olarak ele alır. Bu yüzden yoğun döngülerde **yaş.ToString()** kullanmak, kutulama işlemini engelleyeceği için daha performanslıdır.

## 13.7 Göstericiler

C# dili, göstericileri sınırlı bir ölçüde destekler. Gösterici (**pointer**), bir bellek adresini tutan bir değişkenden başka bir şey değildir. C# göstericisi yalnızca değer tiplerin ve dizilerin bellek adresini tutmak için bildirilebilir (**declaration**).

1990 yıllarının sonlarına kadar, geliştirme çalışmalarımın çoğu C programlama dilini kullanarak yapıldı. C veya C++ dillerini kullanan programcılar göstericileri kullanmaktan kaçınmazdı. C# ve Java gibi saf nesne yönelimli dilleri kullanmaya başladığımızdan beri göstericiler çok nadir kullanılmaya başlandı ve genellikle buna gerek duyulmamaktadır. Çünkü göstericilerin çokça kullanıldığı dinamik veri yapılarını sağlayan kütüphaneler doğrudan bize bu yeni altyapılar tarafından sunulmuştur. Ancak göstericiler, bazı durumlarda düşük düzey iş ve işlemler için gerekli olabilir. Buna örnek olarak **P/Invoke** verilebilir. **P/Invoke**, yönetilen kodunuzdan (**managed code**) yönetilmeyen kütüphanelerdeki (**unmanaged code**) yapılara, geri aramalara (**callback**) ve fonksiyonlara erişmenizi sağlayan bir teknolojidir.

Referans tiplerin aksine, gösterici tipleri varsayılan çöp toplama (GC) mekanizması tarafından izlenmez. Aynı nedenden dolayı göstericilerin bir referans tipe veya hatta bir referans tipi içeren bir yapı (**struct**) türüne işaret etmesine izin verilmez. Göstericilerin yalnızca yönetilmeyen (**unmanaged**) tipleri, yani tüm temel veri tiplerini (**sbyte**, **byte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**, **char**, **float**, **double**, **decimal**, **bool**), **enum** tiplerini, diğer gösterici tiplerini ve yalnızca yönetilmeyen tipleri içeren yapıları (**struct**) işaret edebileceğini söyleyebiliriz.



Göstericiler başlığındaki verilen örneklerin çalışabilmesi için proje dosyalarında (.csproj) `<AllowUnsafeBlocks>true</AllowUnsafeBlocks>` ayarı yapılmalıdır!

### §13.7.1 Gösterici Tipi Bildirme

Bir gösterici tipi bildiriminin genel biçimi aşağıda gösterildiği gibidir:

```
veritipi *gösterici-kimliği;
```

Yıldız (\*) karakteri, **referanstan çıkarma işleci** (**de-reference operator**) olarak bilinir. Bir `int` tipinin adresini tutabilen bir gösterici değişkeni `ptrToInt` aşağıdaki gibi bildirir;

```
int* ptrToInt;
```

**Referans işleci** (&) bir değişkenin bellek adresini almak için kullanılabilir. Bir göstericiye ilk değer vermek için kullanılabilir. Aşağıdaki örnekte gösterici veri tipiyle, değişkenin veri tipinin aynı olduğuna dikkat edilmelidir.

```
int x = 100;
int* ptrToInt = & x;
Console.WriteLine((int) ptrToInt) // Konsola ptrToInt göstericisinin tuttuğu adres yazılır
Console.WriteLine(*ptr) // Konsola ptrToInt göstericisinin gösterdiği adresteki değer yazılır.
```

C# ifadeleri güvenli (**safe**) veya güvenli olmayan (**unsafe**) bir bağlamda yürütülebilir. **unsafe** anahtar sözcüğü kullanılarak güvenli olmayan olarak işaretlenen ifadeler çöp toplayıcısının (GC) denetimi dışında çalışır. C# dilinde göstericiler içeren herhangi bir kodun güvenli olmayan bir bağlam gerektirdiğini unutulmamalıdır. **Main** dahil tüm yöntemleri güvenli olmayan olarak işaretleyebilir kullanabiliriz.

```
unsafe
{
 int x = 1;
 int y = 2;
 int* ptr1 = &x;
 int* ptr2 = &y;
 Console.WriteLine((int)ptr1); //871885156
 Console.WriteLine((int)ptr2); //871885152
 Console.WriteLine(*ptr1); //1
 Console.WriteLine(*ptr2); //2
}
Sınıf nesne = new Sınıf();
nesne.GüvenliOmlayanYöntem();
class Sınıf
{
 public unsafe void GüvenliOmlayanYöntem()
 {
 int x = 3;
 int y = 4;
 int* ptr1 = &x;
 int* ptr2 = &y;
 Console.WriteLine((int)ptr1); //871885052
 Console.WriteLine((int)ptr2); //871885048
 Console.WriteLine(*ptr1); //3
 Console.WriteLine(*ptr2); //4
 }
}
```

C# çöp toplayıcısı (GC), çöp toplama işlemi sırasında nesneleri istediği gibi bellekte taşıyabilir. Çöp toplayıcısına bir nesneyi taşımasını söylemek için özel bir **fixed** anahtar sözcüğü kullanılır. Böylece işaret edilen değer türlerinin konumunu bellekte sabitletir. Bu durum **sabitleme** (**pinning**) olarak bilinir. Aşağıda C dili göstericileri gibi bir dizi elemanlarına erişim ve nesne durumuna erişim örneği verilmiştir;

```
unsafe
{
 var buffer = new int[1024];
 fixed (int* p = &buffer[0])
 {
```

```

 for (var i = 0; i < buffer.Length; i++)
 {
 *(p + i) = i;
 }
 }

 Sınıf nesne = new Sınıf();
 nesne.alan = 32;
 nesne.KonsolaAlanıYaz();
 fixed (int* alan = &nesne.alan)
 {
 *alan += 3;
 }
 nesne.KonsolaAlanıYaz();
}

```

```

class Sınıf
{
 public Sınıf() { }
 public int alan; //field
 public void KonsolaAlanıYaz()
 {
 Console.WriteLine("alan = {0}", alan);
 }
}

```

C#'ta `stackalloc` işleci, belleği yığın (`stack`) üzerinde tahsis eder ve bu işlemin sonucunda size o bellek bloğunun başlangıç adresini döndürür. Bu adresi yönetmek için en temel yol bir gösterici kullanmaktır. Normal bir dizi (`new int[]`) kullandığınızda, dizi nesnesi öbek (`heap`) üzerinde oluşur ve **çöp toplayıcı** (GC) tarafından takip edilir. Ancak `stackalloc` ile bir gösterici kullandığınızda bellek tahsisi ve serbest bırakılması neredeyse maliyetsizdir. Bellek yığında olduğu için çöp toplayıcının haberi bile olmaz, bu da **"stop-the-world"** duraksamalarını engeller. İşlemci, yığın belleğine çok daha hızlı erişir.

Göstericilerle çalışmak "güvensiz" kabul edildiği için bu kod bloğunun `unsafe` anahtar kelimesi ile işaretlenmesi gerekir.

// Proje ayarlarından "Allow unsafe code" seçeneği işaretlenmelidir.

```

unsafe
{
 // 1. Yığında 5 adet tam sayılık (20 byte) yer aç ve adresini 'p' göstericisine ata
 int* p = stackalloc int[5];

 // 2. Gösterici aritmetiği veya indisleme ile değer atama
 for (int i = 0; i < 5; i++)
 {
 // p[i] kullanımı aslında *(p + i) ile aynıdır
 p[i] = (i + 1) * 10;
 }

 // 3. Veriye erişim
 Console.WriteLine($"İlk eleman (p[0]): {*p}"); // Dereferencing
 Console.WriteLine($"Üçüncü eleman (p[2]): {*(p + 2)}"); // Adres aritmetiği

 // Bellek, bu 'unsafe' bloğun veya metodun dışına çıkıldığında otomatik geri alınır.
}

```

`stackalloc` ile ayrılan ve gösterici ile tutulan bellek, sadece tanımlandığı yöntemin yaşam süresi boyunca geçerlidir. Bu göstericiyi bir sınıfın özelliği (`property`) olarak saklamaya veya metod dışına döndürmeye çalışmak hatalı/tehlikelidir. Göstericilerde sınır kontrolü yoktur. Eğer 5 birimlik yer ayırıp `p[10]` dersanız, komşu bellek alanlarını bozabilir (`buffer overflow`) veya uygulamanın çökmesine neden olabilirsiniz. Eğer mutlak bir performans zorunluluğu yoksa, C#7.2+ ile gelen `Span<T>` yapısı, `stackalloc` kullanımını `unsafe` bloğuna ihtiyaç duymadan ve sınır kontrolleri (`bounds check`) yaparak daha güvenli hale getirir.

`stackalloc` Ne Zaman Kullanılmalı?

- ◇ **Küçük Veri Grupları:** Yığın boyutu sınırlıdır (genellikle 1 MB). Çok büyük dizileri yığında oluşturmaya çalışmak `StackOverflowException` hatasına neden olur.
- ◇ **Kısa Ömürlü Diziler:** Eğer dizi sadece o yöntemin içinde kullanılıp atılacaksa `stackalloc` idealdir.
- ◇ **Performans Kritik İşler:** Saniyede binlerce kez çalışan döngüler içinde sürekli yeni dizi oluşturmanız gerekiyorsa, çöp toplayıcı (GC) baskısını önlemek için kullanılır.

### §13.7.2 Gösterici Aritmetiği

Göstericilerde toplama ve çıkarma tam sayılardan farklı şekilde çalışır. Bir gösterici artırıldığında veya azaltıldığında, işaret ettiği adres, referans tipinin bellek boyutu kadar artırılır veya azaltılır. Örneğin, `int` tipi 4 bayt boyutundadır. Bir `int`, 0 adresinde saklanıyorsa, bir sonraki `int`, 4 adresinde saklanır;

**unsafe**

```
{
 int* ptr = (int*)IntPtr.Zero;
 Console.WriteLine(new IntPtr(ptr)); // 0
 ptr++;
 Console.WriteLine(new IntPtr(ptr)); // 4
 ptr++;
 Console.WriteLine(new IntPtr(ptr)); // 8
}
```

Benzer şekilde, `long` tipi 8 bayt boyutundadır. Eğer bir `long`, 0 adresinde saklanıyorsa, bir sonraki `long` 8 adresinde saklanır;

**unsafe**

```
{
 long* ptr = (long*)IntPtr.Zero;
 Console.WriteLine(new IntPtr(ptr)); // 0
 ptr++; // bellekteki bir sonraki long adresini alır
 Console.WriteLine(new IntPtr(ptr)); // 8
 ptr+=4; // bellekteki 4 sonraki long adresini alır
 Console.WriteLine(new IntPtr(ptr)); // 40
}
```

Hiçbir değeri olmayan veri tipi anlamındaki `void` özeldir ve `void` göstericileri de özeldir. Veri tipi bilinmediğinde veya önemli olmadığında her şeyi ifade eden `void` göstericileri kullanılırlar. C ve C++ dilinin aksine boyuttan bağımsız yapıları nedeniyle, `void*` göstericileri artırılmaz veya azaltılmaz:

**unsafe**

```
{
 void* ptr = (void*)IntPtr.Zero;
 Console.WriteLine(new IntPtr(ptr));
 ptr++; // Hata: The operation in question is undefined on void pointers
 Console.WriteLine(new IntPtr(ptr));
 ptr++; // Hata: The operation in question is undefined on void pointers
 Console.WriteLine(new IntPtr(ptr));
}
```

Herhangi bir gösterici tipi, örtük (**implicit**) bir dönüşüm kullanılarak `void*` tipine atanabilir;

```
int* p1 = (int*)IntPtr.Zero;
void* ptr = p1; //örtük olarak void* tipine atama.
Bunun tersi durum ancak açık (explicit) olarak mümkündür;
int* p1 = (int*)IntPtr.Zero;
void* ptr = p1; //örtük olarak void* tipine atama.
int* p2 = (int*)ptr; //void* tipi int* tipine örtük olarak atanamaz!
```

### §13.7.3 Referanstan Çıkarma İşleci

C ve C++ dillerinde, bir gösterici değişkeninin bildirimindeki yıldız karakteri, bildirilen ifadenin bir parçasıdır. Ama C# dilindeki bildirimdeki yıldız karakteri referanstan çıkarma işleci (**de-reference operator**) (\*) olup veri tipinin bir parçasıdır.

Aşağıdaki örnekte verilen kod C# dilinde iki `int` göstericisi bildirmesine rağmen C ve C++ dillerinde ilki gösterici, ikincisi değişken olarak bildirilir.

```
int* a,b;
/*
```

```
C ve C++ olarak eşdeğeri: int* a; int b;
C# eşdeğeri: int* a; int* b;
*/
int *bir, *iki; //Hata:C# dilinde derlenmez.
/*
Ancak C ve C++ dillerinde geçerli iki farklı göstericidir.
*/
```

### §13.7.4 Üyelere Erişim

C ve C++ gibi C# dilinde de bir yapı ya da sınıf örneğinin üyelerine bir gösterici aracılığıyla erişme, **ok işleci (arrow operator) ->** ile yapılır.

```
unsafe
{
 Koordinat v;
 v.X = 5;
 v.Y = 10;
 Koordinat* ptr = &v;
 int x = ptr->X;
 int y = ptr->Y;
 string s = ptr->ToString();
 Console.WriteLine(x); // 5
 Console.WriteLine(y); // 10
 Console.WriteLine(s); // Koordinat
}
struct Koordinat
{
 public int X;
 public int Y;
}
```

## 13.8 Düşün ve Çöz

- ◊ **Düşün:** Örtük (**implicit**) ve açık (**explicit**) tip dönüşümü arasındaki fark nedir? Örtük dönüşümün güvenli kabul edilmesinin nedeni nedir? Veri kaybına yol açabilecek bir dönüşümün açık yapılmasını zorunlu kılan tasarım kararı nedir?
- ◊ **Düşün:** is ve as işlemleri arasındaki fark nedir? Bir taban sınıf referansının türemiş sınıf tipine dönüşürülmesinde **InvalidCastException** riskini nasıl önlersiniz? C#7 ile gelen desen eşleştirme (**pattern matching**) bu süreçte nasıl kolaylaştırır?
- ◊ **Düşün:** Kutulama (**boxing**) ve kutudan çıkarma (**unboxing**) işlemleri neden performans maliyeti yaratır? Sıkça tekrarlanan işlemlerde kutulama kaçınılmaz hale geldiğinde neler yapılabilir? Jenerik yapılar bu sorunu nasıl çözer?
- ◊ **Düşün:** C#'ta unsafe kod bloğu ve göstericiler (**pointer**) ne zaman kullanılır? **Managed** ve **unmanaged** bellek arasındaki fark nedir? .NET'te bellek güvenliğini sağlayan mekanizmalar nelerdir?
- ◊ **Düşün:** **Span<T>** ve **Memory<T>** tipleri göstericilere alternatif olarak sunulmuştur. Bu tiplerin güvenli bellekle çalışma avantajı nedir? Büyük dizilerin bir bölümü üzerinde kopyasız (**zero-copy**) işlem yapmanın önemi nedir?
- ◊ **Düşün:** Sabitlenmiş ifade (**fixed statement**) neden gereklidir? Çöp toplayıcının belleği taşıması (**GC compaction**) gösterici aritmetiğini nasıl tehlikeli hale getirir? **fixed** bu riski nasıl ortadan kaldırır?
- ◊ **Düşün:** Tip dönüşümü operatörleri (**implicit/explicit operator**) kendi sınıflarınıza nasıl eklenir? Bu mekanizmanın aşırı kullanımı kodun okunabilirliğini nasıl olumsuz etkiler?
- ◊ **Çöz:** Bir Sıcaklık sınıfı tanımlayınız: **implicit operator** ile **double**'a, **explicit operator** ile **int**'e dönüşüm sağlasın. Celsius ve Fahrenheit arası çevirimi implicit operatörle uygulayın ve test edin.
- ◊ **Çöz:** Hayvan taban sınıfından türeyen Köpek ve Kedi sınıflarınız olsun. is, as ve C#7 desen eşleştirme söz dizimini kullanarak farklı hayvan tiplerini ele alan bir yöntemi üç farklı şekilde yazınız ve karşılaştırınız.
- ◊ **Çöz:** Kutulama maliyetini ölçmek için: 1 milyon eleman üzerinde **int\object** kutulaması yapan ve jenerik **List<int>** kullanan iki döngünün çalışma süresini Stopwatch ile ölçüp karşılaştıran bir program yazınız.
- ◊ **Çöz:** **Span<T>** kullanarak bir büyük dizinin ortasındaki 100 elemanlık bölümü kopyalamadan işleyen (toplamı bulan) bir program yazınız. Aynı işlemi normal dizi kopyalamayla yapan versiyonla karşılaştırın.

- ◇ **Çöz:** `unsafe` bloğu içinde gösterici aritmetiği kullanarak bir `byte` dizisini tersine çeviren (`reverse`) bir yöntem yazınız. Ardından aynı işlemi `Span<T>.Reverse()` ile yapın ve sonuçları karşılaştırın.

## 14. Bölüm

# Jenerikler, Koleksiyonlar ve İndisleyiciler

Şu ana kadar en basit veri yapıları (**data structure**) olan değişkenlerle karşılaştık. Programlamada en çok kullanılan bir başka veri yapısı da dizilerdir (**array**). Diziler de koleksiyonlardan (**collection**) biridir ve daha önce anlatılmıştır. Koleksiyonlar çok sayıda nesneyi barındıran dinamik veri yapıları olup **new** anahtar kelimesiyle çalışma anında (**run-time**) bellekte oluşturulan yapılardır.

### 14.1 ArrayList

Dizilerin çalışma anında boyutu belirlenir ve daha sonra boyutu değiştirilmez. Aksine boyutu dinamik olarak artırılan bir dizi kullanmak istediğimizde **ArrayList** kullanırız. **ArrayList**, .NET için standart olarak tanımlı **ICollection** arayüzünü (**interface**) uygulayan bir koleksiyondur.

```
using System.Collections;
```

```
ArrayList liste = new ArrayList();
liste.Add("Renault");
liste.Add("Fiat");
liste.Add("Toyota");
Console.WriteLine("Araçlar");
Console.WriteLine(" Adet: {0}", liste.Count); //3
Console.WriteLine(" Kapasite: {0}", liste.Capacity); //4
Console.WriteLine(" Değerler:");
foreach (Object obj in liste)
Console.WriteLine(" {0}", obj); // Renault Fiat Toyota
```

**ArrayList** içindeki verileri **object** olarak sakladığı için değer tipleriyle (int, double vb.) çalışırken kutulama (**boxing/unboxing**) maliyetine yol açar. Bu nedenle modern uygulamalarda ileride anlatılan **List<T>** kullanılır.

Nesneleri bir listede saklamak için eskiden **ArrayList** kullanırdık. **ArrayList** Her şeyi **object** olarak tutar. İçine **int** eklediğinizde her defasında kutulama (**boxing**) yapar. Okurken de kutudan çıkarma (**unboxing**) yapar. Bu yöntem **çok yavaştır!** Artık bu işlemi aşağıda anlatılacak **List<int>** ile yapıyoruz. **List<int>** tip güvenlidir (**type-safe**) . Verileri doğrudan kendi tipinde tutar. Kutulama (**boxing**) yapmaz ve hızlıdır.

### 14.2 Jenerik Programlama

Jenerik programlamada (**generic programming**) jenerik yapılar, kodun tipten bağımsız olarak yazılmasını, ancak kullanım anında tip güvenliğinin (**type safety**) korunmasını sağlar. Bu sayede aynı mantığı **int**, **string** veya kullanıcı tanımlı sınıflar için tekrar tekrar yazmak zorunda kalmazsınız.

Jenerik yapılar gelmeden önce (**List<T>** yerine **ArrayList** varken), her şey **object** olarak tutulurdu. Bu da her işlemde kutulama (**boxing**) ve kutuyu açma (**unboxing**) maliyetine ve tip hatalarına yol açardı. Jenerik yapılar bu sorunu kökten çözer.

C# dili farklı veri tipleriyle çalışan yöntemler, sınıflar, algoritmaları tekrar tekrar yazmamak için bir programlama türü olan jenerik programlamayı (**generic programming**) destekler. **Dizi** ve **ArrayList** dışındaki tüm koleksiyonlar jenerik programa ile çalışır.

**Jenerikler** (**generic**), veri tiplerine bağlı olmadan algoritma içeren bir yöntem veya sınıf oluşturmamızı sağlar! Jenerik, aynı mantığı yeniden yazmadan bir yöntemin farklı veri tipleri üzerinde çalışmasını sağlayan bir sınıf, yöntem ve algoritma planı tanımlar. Jenerikler, kodunuzda sınıfı veya yöntemi kullanana kadar bir veya daha fazla tip parametresinin belirtilmesini erteleyen sınıflar ve yöntemler tasarlamayı mümkün kılar.

Genel bir veri tipini temsil eden **T** veya **V** parametresi kullanarak, çalışma zamanı dönüştürmelerinin veya kutulama (**boxing**) işlemlerinin maliyetine veya riskine katlanmadan diğer istemci kodunun kullanabileceği

tek bir sınıf yazabilirsiniz;

```
GenericList<int> list1 = new(); //int veri tipinde elemanlardan oluşan liste
list1.Add(1);
```

```
GenericList<string> list2 = new(); //string veri tipinde elemanlardan oluşan liste
list2.Add("");
```

```
GenericList<AClass> list3 = new(); //AClass veri tipinde elemanlardan oluşan liste
list3.Add(new AClass());
```

```
public class GenericList<T>
{
 public void Add(T item) { }
}
```

```
public class AClass { } // Örnek Bir Sınıf
```

Yukarıdaki örnek programda tanımladığımız jenerik olan `GenericList<T>`, hem `int` veri tipiyle, hem `string` veri tipiyle hem de kullanıcı tanımlı `AClass` veri tipiyle çalışmaktadır.

### §14.2.1 Jenerik Sınıf Tanımlama

C++'taki `template <typename T>` ifadesinin yerini C# dilinde `<T>` alır.

```
public class Depo<T>
{
 private T _icerik;

 public void Ekle(T veri) => _icerik = veri;
 public T Getir() => _icerik;
}
```

// Kullanımı:

```
Depo<int> sayiDeposu = new Depo<int>();
sayiDeposu.Ekle(100);
Depo<string> metinDeposu = new Depo<string>();
metinDeposu.Ekle("C# Harika!");
```

### §14.2.2 Jenerik Kısıtlamalar

C++ şablonlarında bir tipin belirli özellikleri karşılamadığını ancak derleme hatası aldığınızda anlarsınız. C# dilinde ise `where` anahtar kelimesiyle en baştan kurallar koyabilirsiniz. Bu **kısıtlamalar** (**constraints**) için `where` şartı yazılır. Bu, jenerik programlamanın en güçlü yanlarından biridir.

```
public class MatematikIslem<T> where T : struct // T mutlaka bir değer tipi (int, double vb.)
 → olmalı
{
 // ...
}

public class VeriKaydedici<T> where T : class, new() // T bir sınıf olmalı ve parametresiz yapıcı
 → metodu olmalı
{
 public T YeniNesneOlustur() => new T();
}
```

Sık kullanılan kısıtlamalar:

- ◇ `where T : struct` → Değer tipi zorunluluğu.
- ◇ `where T : class` → Referans tipi zorunluluğu.
- ◇ `where T : new()` → T sınıfından varsayılan yapıcıyla bir örnek olmalı.
- ◇ `where T : NotNull` → `null` olamaz.
- ◇ `where T : ASınıfı` → Belirli bir sınıftan türemiş olmalı.
- ◇ `where T : IArayüzü` → Belirli bir arayüzü uygulamış olmalı.



### §14.2.3 Jenerik Yöntemler

Sadece sınıflar değil, yöntemler da jenerik olabilir. Tip çıkarımı (**type inference**) sayesinde çoğu zaman tipi belirtmenize bile gerek kalmaz.

```
public void Takas<T>(ref T a, ref T b)
{
 T gecici = a;
 a = b;
 b = gecici;
}
```

```
// Kullanımı:
int x = 5, y = 10;
Takas(ref x, ref y); // Derleyici T'nin int olduğunu anlar.
```

### §14.2.4 C++ Şablonları ve C# Jenerikleri

Karşılaştırmalı tablo aşağıda verilmiştir;

| Özellik        | C++ Templates                                                           | C# Jenerikleri                                     |
|----------------|-------------------------------------------------------------------------|----------------------------------------------------|
| Derleme        | Her tip için ayrı kod üretilir ( <b>code bloat</b> ).                   | Tek bir kod üretilir, çalışma zamanında özelleşir. |
| Tip Kontrolü   | Gevşek (Kullanılana kadar hata vermez).                                 | Sıkı (Kısıtlamalarla önceden belirlenir).          |
| Statik Alanlar | Her tip örneği için ayrı statik alan oluşur.                            | Her tip örneği için ayrı statik alan oluşur.       |
| Uzmanlaşma     | <code>template&lt;&gt;</code> ile belirli tiplere özel kod yazılabilir. | Doğrudan desteklenmez (Arayüzlerle çözülür).       |

Tablo 14.1: C++ Şablonu ve C# Jenerik Karşılaştırması

### §14.2.5 Numaralandırıcılar ve Koleksiyonlar

C# dilinde bir nesnenin **foreach** döngüsü ile taranabilmesi için o nesnenin "sayılabilir" olması gerekir. Bu yetenek, .NET ekosisteminde iki temel arayüz ile sağlanır. Bunlar **IEnumerable** ve **IEnumerator** arayüzleridir.

1. **IEnumerable<T>** (Kumanda Edilebilir): Bir nesnenin "üzerinde gezilebilir" olduğunu beyan eder. Tek bir metodu vardır: **GetEnumerator()**. Bu, size bir "numaralandırıcı" (kumanda) verir.
2. **IEnumerator<T>** (Kumanda): Koleksiyon üzerinde adım adım ilerlemeyi sağlayan asıl mekanizmadır. O an hangi elemandayız? Bir sonraki eleman var mı? Bu soruların cevabını verir.

#### *IEnumerator<T> Arayüzünün Anatomisi*

Bir **IEnumerator<T>** arayüzü şu üç temel üyeye sahiptir:

1. **Current** Özelliği: İmlecin o an üzerinde bulunduğu elemanı döndürür.
2. **MoveNext()** Yöntemi: İmleci bir sonraki elemana kaydırır. Eğer bir sonraki eleman varsa **true**, listenin sonuna gelindiye **false** döner.
3. **Reset()** Yöntemi: İmleci başlangıç noktasına (ilk elemanın öncesine) geri getirir.

Aşağıda bu arayüzün kodu verilmiştir;

```
public interface IEnumerator<out T> : IDisposable, IEnumerator
{
 T Current { get; }
 bool MoveNext();
 void Reset();
}
```

#### *foreach Döngüsünün Arka Planı*

Programcı **foreach** yazdığında, C# derleyicisi bunu arka planda bir **while** döngüsüne ve **IEnumerator** kullanımına dönüştürür.

**Programcının Yazdığı:**

```
foreach (var item in liste) { Console.WriteLine(item); }
```

**Sadeleştirilmiş Şekilde Derleyicinin Çalıştırdığı:**

```
IEnumerator<int> enumerator = liste.GetEnumerator();
try {
 while (enumerator.MoveNext()) {
```

```

 int item = enumerator.Current;
 Console.WriteLine(item);
 }
}
finally {
 enumerator.Dispose();
}

```

### Neden Doğrudan IEnumerator Kullanmıyoruz?

`IEnumerable`, listenin durumunu (*state*) korumaz. Her `GetEnumerator()` çağrıldığında yeni bir imleç oluşturulur. Bu sayede aynı listeyi aynı anda iki farklı `foreach` döngüsü ile (birbirinden bağımsız yerlerde) tarayabilirsiniz. Eğer sadece `IEnumerator` olsaydı, bir döngü ilerlediğinde diğeri de kaldığı yerden devam etmek zorunda kalırdı.

Bir `foreach` döngüsü ile bir listeyi dönerken, döngü içinde `liste.Add()` veya `liste.Remove()` yaparsanız ne olur? `InvalidOperationException` hatası alırsınız. `IEnumerator`, oluşturulduğu andaki koleksiyonun bir "versiyon" numarasını tutar. Koleksiyonda herhangi bir değişiklik yapıldığında bu versiyon numarası güncellenir. `MoveNext()` her çağrıldığında versiyon kontrolü yapılır; eğer numaralar eşleşmiyorsa koleksiyonun değiştiği anlaşılır ve hata fırlatılır.

### §14.2.6 Yineleyiciler

Bu başlık sonrasında işleyeceğimiz koleksiyonlarda çok kullanılan yineleyiciler (*iterator*), `IEnumerable` ve `IEnumerator<T>` arayüzlerini kullanır.

Aşağıda bu arayüzleri uygulayan sınıflara örnek verilmiştir;

```

using System.Collections;
foreach (var araç in new AraçKoleksiyonu())
{
 Console.WriteLine($"{araç} "); //sedan heçbek siteyşın limuzin
}

public class AraçTipiNumaralandırma : IEnumerator<string> //IEnumerator Interface Realization
{
 string[] araçTipleri = new string[4] { "sedan", "heçbek", "siteyşın", "limuzin" };
 int currentIndex = -1;
 public string Current
 {
 get
 {
 if (currentIndex < 0 || currentIndex >= araçTipleri.Length)
 throw new InvalidOperationException();
 return araçTipleri[currentIndex];
 }
 }
 object IEnumerator.Current // Explicit implementation of IEnumerator.Current
 {
 get
 {
 return Current;
 }
 }
 public void Dispose()
 {
 }
 public bool MoveNext()
 {
 currentIndex++;
 if (currentIndex < araçTipleri.Length)
 {
 return true;
 }
 return false;
 }
}

```

```

 public void Reset()
 {
 currentIndex = 0;
 }
}
public class AraçKoleksiyonu : IEnumerable<string> //IEnumerable Interface Realization
{
 private AraçTipiNumaralandırma enumerator;
 public AraçKoleksiyonu()
 {
 enumerator = new AraçTipiNumaralandırma();
 }
 public IEnumerator<string> GetEnumerator()
 {
 return enumerator;
 }
 IEnumerator IEnumerable.GetEnumerator() // Explicit implementation of
 ↪ IEnumerable.GetEnumerator
 {
 return GetEnumerator();
 }
}

```

### §14.2.7 yield Anahtar Kelimesi

Normal bir yöntemin aksine, içinde `yield` bulunan bir yöntem çalıştırıldığında tüm kod bloğu bir kerede icra edilmez. Bunun yerine yöntem, bir "durum makinesi" (*state machine*) gibi davranır ve her `yield return` satırına geldiğinde o anki değeri dışarıya fırlatıp, kaldığı yeri hatırlar.

#### Adım Adım Üretim

"Sonsuz Veri Akışları" veya "Büyük Veri Filtreleme" senaryosuyla geleneksel `List<T>` yönteminin neden yetersiz kaldığını ve `yield` anahtar kelimesinin belleği nasıl kurtardığını anlatacağız. Örnek senaryo olarak belleği yormadan milyonlarca iz kaydını (*log*) işlemeyi ele alalım. Elinizde milyonlarca satırlık bir iz kayıtları dosyası veya veri akışı olduğunu hayal edin. Amacınız sadece "HATA" (*error*) içeren kayıtları bulup ekrana basmak.

Geleneksel yöntemde, tüm hatalı kayıtları önce bir listeye doldurup sonra döndürürüz. Eğer 1 milyon hata varsa, 1 milyon nesne belleğe (RAM) hücum eder.

```

public List<string> HatalariGetir(List<string> tumLoglar)
{
 List<string> hatalar = new List<string>();
 foreach (var log in tumLoglar)
 {
 if (log.Contains("HATA"))
 hatalar.Add(log); // 1 milyon hata varsa RAM burada şişer!
 }
 return hatalar;
}

```

`yield return` yönteminde, bir liste oluşturulmaz. Program bir hata bulduğu anda onu dışarı fırlatır, işlenmesini bekler ve sonra kaldığı yerden aramaya devam eder. RAM'de sadece o anki tek bir satır tutulur. Bellek dostu olan bu yöntem **tembel değerlendirme** (*lazy evaluation*) olarak adlandırılır.

```

public IEnumerable<string> HatalariGetirYiseld(IEnumerable<string> tumLoglar)
{
 foreach (var log in tumLoglar)
 {
 if (log.Contains("HATA"))
 {
 // Buldum! Bunu al, işle, bitince geri gel.
 yield return log;
 }
 }
}

```

Bu örneği şu adımlarla açıklayabiliriz:

- ◊ Talep Anı: `foreach (var hata in HatalariGetirYield(loglar))` satırı çalıştığında yöntemin içine girilir.
- ◊ Bul ve Dur: İlk "HATA" satırı bulunduğunda `yield return` çalışır. Yöntem burada "donar" (durumunu kaydeder).
- ◊ Tüketim: Kontrol ana programa geçer, hata ekrana yazdırılır.
- ◊ Geri Dönüş: Döngü bir sonraki elemanı istediğinde, metot kaldığı yerden (dondugu satırdan) devam eder; sanki hiç ayrılmamış gibi bir sonraki iz kaydını (`log`) kontrol eder.

`yield return` bir koleksiyondaki tüm elemanları bir kerede belleğe yüklemek yerine, her seferinde bir tanesini işlemek için kullanılır.

Bir başka örnek sonsuz sayı üreticini ele alabiliriz. C++'ta bunu bir `vector` ile yapmaya çalışmak saniyeler içinde `OutOfMemory` hatası verecektir.

```
public IEnumerable<int> SonsuzSayac()
{
 int i = 0;
 while (true) // Korkutucu görünüyor ama güvenli!
 {
 yield return i++;
 }
}

// Kullanımı:
foreach (var sayi in SonsuzSayac())
{
 Console.WriteLine(sayi);
 if (sayi > 100) break; // İstedığımız an dururuz, bellek tertemiz.
}
```

`yield return` kullanan bir yöntemin içinde `try-catch` bloğu kullanabilir misiniz? Peki `yield return` satırı bir `try` bloğu içindeyse `catch` ne zaman çalışır? `yield return` içeren bir yöntem içinde `try-catch` kullanabilirsiniz ancak bir kısıtlama vardır. `yield return` ifadesi içeren bir blokta `catch` bloğu bulunamaz (fakat `finally` bulunabilir). Çünkü yöntem donduğunda, dışarıdaki kodun ne zaman hata fırlatacağı belirsizdir. Hata yönetimi genellikle bu metodun çağrıldığı (tüketildiği) yerde yapılır.

## Yinelemeyi Erken Sonlandırmak

Adım adım üretim (`yield return`) konusunu "üretim hattı" olarak anlattıktan sonra, `yield break` ifadesini bu hattın "Acil Durdurma Butonu" veya "Mantıksal Bitiş Noktası" olarak tanımlayabiliriz.

C++'ta bir fonksiyondan erken çıkmak için kullandığınız `return;` ifadesi neyse, bir yineleyici (`iterator`) metodu içinde de `yield break` odur. Ancak aralarında çok kritik bir fark vardır.

Bir `IEnumerable` metodu içinde döngü devam ediyor olsa bile, belirli bir koşul gerçekleştiğinde veri üretimi tamamen durdurmak ve koleksiyonun "sonuna gelindi" sinyalini vermek için `yield break` kullanılır.

- ◊ `yield break` satırı çalıştırıldığında: Yöntemin icrası o anda durur.
- ◊ Çağırana taraftaki `foreach` döngüsüne "artık veri kalmadı" (`MoveNext = false`) bilgisi gönderilir.
- ◊ Yöntem içindeki yerel değişkenler temizlenir ve kontrol çağrı yapan koda geri döner.

```
public IEnumerable<string> SansürlüMetinGetir(IEnumerable<string> kelimeler)
{
 foreach (var kelime in kelimeler)
 {
 if (kelime == "YASAKLI_İÇERİK")
 {
 // Bu kelimeyi gördüğün an yayını kes, sonrasını okuma bile!
 yield break;
 }

 yield return kelime;
 }
}
```

```
// Normal akış biterse burası çalışır ama yield break çalışırsa buraya ulaşamaz.
}
```

Bir `IEnumerable` metodu içinde sadece `return`; yazamazsınız; derleyici buna izin vermez. `return`; bildiğimiz yöntemlerde yöntemin icrasını sonlandırır ve varsa bir değer döndürür. Bu talimatın `yield` içeren yineleyici yöntemlerde doğrudan kullanımı yasaktır.

Bir veritabanından veri çekerken zaman aşımına uğradığınızda üretimi durdurmak için harikadır. Buna ilişkin kod örneği aşağıda verilmiştir;

```
public IEnumerable<Data> VeriSorgula(DateTime bitisZamani)
{
 while (true)
 {
 var veri = VeriTabanındanOku();

 if (DateTime.Now > bitisZamani)
 {
 Console.WriteLine("Zaman doldu, üretim durduruluyor...");
 yield break; // Döngü sonsuz olsa da burada kesilir.
 }

 yield return veri;
 }
}
```

Eğer `yield break` bir `try-finally` bloğu içindeyse, üretim durdurulmadan önce `finally` bloğu mutlaka çalıştırılır. Bu, C++'taki nesne yıkıcılarının (`destructor`) temizlik yapmasına benzer bir güvenlik sağlar.

içerik...

### Neden yield Kullanmalıyız?

Geleneksel yöntem (`List` kullanmak) ile tüm liste belleğe yüklenir. Listenin tamamı dolana kadar bekler. Sonsuz döngü durumunda bellek dolar.

`yield` yönteminde (`IEnumerable`) sadece o anki eleman bellektedir (`lazy`). İlk eleman üretildiği an işlem başlar. Sonsuz sayı dizisi üretilebilir.

C# derleyicisi arka planda sizin için gizli bir sınıf oluşturur ve bu sınıf `IEnumerator` arayüzünü uygulayarak "geçerli eleman" (`Current`) ve "sonraki adıma geç" (`MoveNext`) mantığını yönetir. C++'ta bunu yapmak için manuel olarak yineleyici (`iterator`) sınıfları yazmanız gerekirdi.

Aşağıdaki kodu inceleyelim;

```
using System;
using System.Collections.Generic;
public class Program
{
 public static void Main(string[] args)
 {
 Console.WriteLine("1. Adım: Yöntem çağrılıyor...");
 // DİKKAT: Yöntem çağrılır ama içindeki kodlar HENÜZ çalışmaz!
 var sayiAkisi = TekSayilariGetir(5);

 Console.WriteLine("2. Adım: Yöntem çağrıldı, döngüye girmek üzereyiz.");
 Console.WriteLine("-----");

 // Döngü başladığı an Yöntem içindeki kodlar "talep üzerine" çalışmaya başlar.
 foreach (int sayi in sayiAkisi)
 {
 Console.WriteLine($"--> Ana Program: Yöntemtan {sayi} değeri geldi ve işleniyor.");
 }
 Console.WriteLine("-----");
 Console.WriteLine("3. Adım: Tüm elemanlar işlendi, program bitti.");
 }
 public static IEnumerable<int> TekSayilariGetir(int limit)
```

```

{
 Console.WriteLine(" [Yöntem İçi]: Veri üretimi BAŞLADI.");
 for (int i = 1; i <= limit; i++)
 {
 if (i % 2 != 0)
 {
 Console.WriteLine($" [Yöntem İçi]: {i} değeri bulundu, yield return ile
 ↪ gönderiliyor...");
 yield return i;
 Console.WriteLine(" [Yöntem İçi]: Ana program işini bitirdi, Yöntem kaldığı
 ↪ yerden devam ediyor.");
 }
 }
 Console.WriteLine(" [Yöntem İçi]: Döngü bitti, üretim TAMAMLANDI.");
}
}
/*Program Çıktısı:
1. Adım: Yöntem çağrılıyor...
2. Adım: Yöntem çağrıldı, döngüye girmek üzereyiz.

[Yöntem İçi]: Veri üretimi BAŞLADI.
[Yöntem İçi]: 1 değeri bulundu, yield return ile gönderiliyor...
--> Ana Program: Yöntemtan 1 değeri geldi ve işleniyor.
[Yöntem İçi]: Ana program işini bitirdi, Yöntem kaldığı yerden devam ediyor.
[Yöntem İçi]: 3 değeri bulundu, yield return ile gönderiliyor...
--> Ana Program: Yöntemtan 3 değeri geldi ve işleniyor.
[Yöntem İçi]: Ana program işini bitirdi, Yöntem kaldığı yerden devam ediyor.
[Yöntem İçi]: 5 değeri bulundu, yield return ile gönderiliyor...
--> Ana Program: Yöntemtan 5 değeri geldi ve işleniyor.
[Yöntem İçi]: Ana program işini bitirdi, Yöntem kaldığı yerden devam ediyor.
[Yöntem İçi]: Döngü bitti, üretim TAMAMLANDI.

3. Adım: Tüm elemanlar işlendi, program bitti.
*/

```

## 14.3 Koleksiyonlar ve Veri Yapıları

Koleksiyonlar (collections), verileri bellekte nasıl organize ettiğimizi belirler. C++ tarafındaki `std::vector`, `std::map` veya `std::list` yapılarına çok benzerler ancak C# dilinde bu yapılar `System.Collections.Generic` isim alanı altında çok daha optimize ve güvenli (type-safe) bir şekilde sunulur.

Doğru koleksiyonu seçmek, uygulamanızın hızı ve bellek kullanımı için hayatidir.

### §14.3.1 Dinamik Dizi List<T>

C++'taki `std::vector` yapısının tam karşılığıdır. Elemanlara indis ([0]) ile erişmek çok hızlıdır.

- ♦ **Özellik:** Boyutu dinamik olarak büyür. Sıralı bir yapıdır.
- ♦ **Performans:** Sona eleman eklemek  $O(1)$ , araya eleman eklemek veya silmek (kaydırma yaptığı için)  $O(n)$  maliyetlidir.
- ♦ **Ne zaman kullanılır?** Eleman sayısı belirsiz olduğunda ve verilere indeksle erişmek gerektiğinde.

```

var liste = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
liste.Add(6);
Console.WriteLine(liste.Count); // 6
liste.RemoveAt(3);
Console.WriteLine(liste.Count); // 5
Console.WriteLine(liste[3]); // 5

```

### §14.3.2 Sözlük Dictionary<TKey, TValue>

C++'taki `std::unordered_map` yapısına benzer. Anahtar-değer çiftleri tutar.

- ♦ **Özellik:** Anahtarlar (key) benzersiz olmalıdır. Arka planda karma tablosu (hash table) kullanır.
- ♦ **Performans:** Bir anahtar üzerinden değere ulaşmak, eklemek ve silmek  $O(1)$  maliyetlidir (inanılmaz hızlıdır).
- ♦ **Ne zaman kullanılır?** Bir kimlik (identifier) üzerinden veriye anında ulaşmak istendiğinde.

```

var kişiler = new Dictionary<int, string> { { 10, "İlhan" },
 { 20, "Mehmet" },
 { 30, "Abdullah" } };

Console.WriteLine(kişiler[10]); // "İlhan"
//Console.WriteLine(kişiler[50]); // HATA!:throws KeyNotFoundException
string? adı; //null olabilen string
if (kişiler.TryGetValue(20, out adı))
{
 Console.WriteLine(adı); // "Mehmet"
}
kişiler[40]="Abdullah";
kişiler.Add(50, "Damla");
//kişiler.Add(40, "Damla"); // HATA!:Throws ArgumentException since 40 already exists
foreach (KeyValuePair<int, string> kişi in kişiler)
{
 Console.WriteLine("No={0}, Adı={1}", kişi.Key, kişi.Value);
}
foreach (int no in kişiler.Keys)
{
 Console.WriteLine("No={0}", no);
}
foreach (string adı in kişiler.Values)
{
 Console.WriteLine("Adı={0}", adı);
}

```

### §14.3.3 Benzersiz Küme HashSet<T>

Sadece benzersiz değerleri tutan, matematiksel küme mantığıyla çalışan bir koleksiyondur.

- ◊ **Özellik:** Aynı elemandan iki tane barındırmaz. Sıralama garantisi vermez.
- ◊ **Performans:** Eleman var mı yok mu kontrolü (**contains**)  $O(1)$  maliyetlidir.
- ◊ **Ne zaman kullanılır?** Bir listede mükerrer kayıtları önlemek ve hızlı varlık kontrolü yapmak için.

```

HashSet<int> aidatVerenDaireler = new HashSet<int>();
aidatVerenDaireler.Add(4);
aidatVerenDaireler.Add(6);
aidatVerenDaireler.Add(9);
aidatVerenDaireler.Add(10);
Console.WriteLine($"Aidat Veren Daire Sayısı:{aidatVerenDaireler.Count}");
//Karma Küme Başlatma (İlk Değer Verme)
HashSet<int> aidatVermeyenDaireler = new HashSet<int>() { 1, 2, 3, 5, 8 };
bool vermemişmi = aidatVermeyenDaireler.Contains(8);
Console.WriteLine($"8 Numaralı Daire Aidat Vermemiş Mi:{vermemişmi}");

```

### §14.3.4 Sıralı Küme SortedSet<T>

C++'taki **std::set** yapısına benzer. Elemanları her zaman sıralı tutar.

- ◊ **Özellik:** Kendi içinde elemanları otomatik sıralar.
- ◊ **Performans:** Ekleme ve arama işlemleri ikili arama ağacı (**binary search tree**) kullandığı için  $O(\log n)$  maliyetlidir.
- ◊ **Ne zaman kullanılır?** Hem benzersizlik hem de sürekli sıralı bir liste gerektiğinde.

```

var sıralıListe = new SortedSet<int>();
sıralıListe.Add(4);
sıralıListe.Add(3);
sıralıListe.Add(3); // Kümede var eklenmez!
sıralıListe.Add(2);
sıralıListe.Add(1);
foreach (var item in sıralıListe)
{
 Console.Write(item+" ");
}
/*Program Çıktısı:
1 2 3 4

```



\*/

### §14.3.5 Yığın Stack<T>

"Son gelen, ilk çıkar" LIFO (**last in first out**) prensibiyle çalışır. Bir kutuya kitapları üst üste dizmek gibidir.

- ◊ **Özellik:** Sadece en üstteki elemana erişebilirsiniz (**peek** veya **pop**).
- ◊ **Ne zaman kullanılır?** Geriye dönük işlemler (**undo/redo**), fonksiyon çağrıları (**stack unwinding**) gibi senaryolarda.

```
var yığın = new Stack<int>();
yığın.Push(10);
yığın.Push(3);
yığın.Push(5);
yığın.Push(8);
Console.WriteLine(yığın.Peek()); // 8
Console.WriteLine(yığın.Pop()); // 8
Console.WriteLine(yığın.Pop()); // 5
Console.WriteLine(yığın.Pop()); // 3
```

### §14.3.6 Kuyruk Queue<T>

"İlk gelen, ilk çıkar" FIFO (**first in first out**) prensibiyle çalışır. Ekmek sırasına girmek gibidir.

- ◊ **Özellik:** Elemanlar arkadan eklenir (**enqueue**), önden çıkarılır (**dequeue**).
- ◊ **Ne zaman kullanılır?** İşlem sırasına alınmış görevler, yazıcı kuyrukları veya mesaj sistemlerinde.

```
var kuyruk = new Queue<int>();
kuyruk.Enqueue(10);
kuyruk.Enqueue(6);
kuyruk.Enqueue(4);
kuyruk.Enqueue(9);
Console.WriteLine(kuyruk.Peek()); // 10
Console.WriteLine(kuyruk.Dequeue()); // 10
Console.WriteLine(kuyruk.Dequeue()); // 6
Console.WriteLine(kuyruk.Dequeue()); // 4
```

### §14.3.7 Bağlı Liste LinkedList<T>

C++'taki `std::list` yapısıdır. Elemanlar bellekte ardışık değil, birbirlerine referanslarla (göstericilerle) bağlıdır.

- ◊ **Özellik:** Çift yönlüdür (**doubly linked list**).
- ◊ **Performans:** Aradan eleman silmek veya eklemek (sadece referans güncellendiği için) çok hızlıdır. Ancak indeksle erişim yoktur, eleman bulmak için listeyi taramak gerekir.
- ◊ **Ne zaman kullanılır?** Listenin ortasına çok sık ekleme-çıkarma yapılıyorsa.

```
LinkedList<int> bağlıListe = new LinkedList<int>();
bağlıListe.AddLast(3);
bağlıListe.AddLast(5);
bağlıListe.AddLast(8);
bağlıListe.AddFirst(10);
bağlıListe.AddFirst(2);
bağlıListe.RemoveFirst();
bağlıListe.RemoveLast();
foreach (int i in bağlıListe)
Console.Write(i+" ");
/* Program Çıktısı:
10 3 5
*/
```

### §14.3.8 Özet Tablo

## 14.4 Performans ve Bellek Optimizasyon Jenerikleri

Bu yapılar, "bir nesneye gerçekten ihtiyaç duyulana kadar onu bellekte oluşturma" prensibine dayanır.

| Koleksiyon | Erişim Hızı      | Ekleme/Silme | Sıralama       | Benzersizlik |
|------------|------------------|--------------|----------------|--------------|
| List       | $O(1)$ (İndeks)  | $O(n)$       | Eklendiği sıra | Hayır        |
| Dictionary | $O(1)$ (Key)     | $O(1)$       | Belirsiz       | Key: Evet    |
| HashSet    | $O(1)$ (Var mı?) | $O(1)$       | Belirsiz       | Evet         |
| SortedSet  | $O(\log n)$      | $O(\log n)$  | Sıralı         | Evet         |
| LinkedList | $O(n)$           | $O(1)$       | Eklendiği sıra | Hayır        |

Tablo 14.2: Koleksiyonlar Özet Tablosu

### §14.4.1 Lazy<T>

En yaygın kullanılan yapıdır. **Tembel başlatma** (**lazy initialization**), nesnenin oluşturulma maliyeti yüksekse ve her senaryoda kullanılmayacaksa tercih edilir. İlk erişimde (**Value** özelliği çağrıldığında) nesneyi oluşturur ve sonraki çağrılarda aynı örneği döner. Varsayılan olarak iş parçacığı güvenlidir (**thread-safe**); yani birden fazla iş parçacığı (**thread**) aynı anda erişse bile nesne sadece bir kez güvenli bir şekilde oluşturulur.

Düşünelim ki çok ağır bir veritabanı yükleme işleminiz var. Bu nesneyi program açılır açılmaz oluşturmak yerine, ilk defa bir veri çekilmek istendiğinde oluşturulmasını sağlayalım.

```
using System;
public class AgirVeritabaniBaglantisi
{
 public AgirVeritabaniBaglantisi()
 {
 Console.WriteLine(">>> Çok ağır veritabanı bağlantısı kuruluyor... (5 saniye sürdü)");
 // Burada ağır bir yükleme işlemi olduğunu varsayalım.
 }
 public void VeriGetir() => Console.WriteLine("Veriler başarıyla getirildi.");
}
class Program
{
 static void Main()
 {
 // 1. Tanımlama: Nesne henüz oluşturulmadı.
 // Sadece nasıl oluşturulacağını (factory method) söyledik.
 Lazy<AgirVeritabaniBaglantisi> lazyBaglanti = new Lazy<AgirVeritabaniBaglantisi>(() =>
 ↪ new AgirVeritabaniBaglantisi());
 Console.WriteLine("Program başladı. Nesne henüz bellekte yok.");
 Console.WriteLine("-----");
 // 2. İlk Erişim: Value özelliği çağrıldığı anda constructor çalışır.
 Console.WriteLine("Veritabanına şimdi ihtiyaç duyuluyor...");
 lazyBaglanti.Value.VerGetir();
 // 3. İkinci Erişim: Nesne zaten olduğu için constructor tekrar çalışmaz.
 Console.WriteLine("\nİkinci defa veri isteniyor...");
 lazyBaglanti.Value.VerGetir();
 // Nesnenin oluşup oluşmadığını kontrol edebiliriz:
 Console.WriteLine($"Nesne oluştu mu? : {lazyBaglanti.IsValueCreated}");
 }
}
```

Varsayılan olarak **Lazy<T>** nesnesi, birden fazla iş parçacığı (**thread**) aynı anda **Value** özelliğine erişse bile nesnenin sadece bir kez oluşturulacağını garanti eder (**race condition**'ı önler).

### §14.4.2 LazyInitializer

**Lazy<T>** sınıfının sağladığı konforu, ek bir nesne sarmalayıcı (**wrapper**) oluşturmadan, doğrudan mevcut değişkenler üzerinden sağlar. Bellek kullanımının çok kritik olduğu, **Lazy<T>** nesnesinin kendisinin yaratacağı küçük ek yükün bile istenmediği durumlarda (örneğin binlerce nesne içeren listeler) kullanılır.

**EnsureInitialized** yöntemi ile çalışır.

**Lazy<T>** kullandığınızda, asıl nesnenizden önce bir de **Lazy** nesnesi oluşturursunuz. Eğer sisteminizde milyonlarca "belki oluşturulacak" nesne varsa, **Lazy<T>**'nin kendisi ciddi bir bellek yükü oluşturabilir.

**LazyInitializer** ise hiç ek nesne oluşturmadan, doğrudan **ref** anahtar kelimesiyle mevcut değişkeniniz üzerinde çalışır.

```
using System;
```

```

using System.Threading;
public class AgirRapor
{
 public AgirRapor() => Console.WriteLine(">>> Agir Rapor Nesnesi Oluşturuldu.");
 public void Yazdir() => Console.WriteLine("Rapor içeriği basılıyor...");
}
class Program
{
 // 1. Ham değişkenimizi tanımlıyoruz (Henüz null)
 private static AgirRapor _rapor;
 static void Main()
 {
 Console.WriteLine("Program Başladı...");
 // 2. EnsureInitialized: Değişken null ise oluşturur, değilse mevcut olanı döner.
 // Thread-safe (iş parçacığı güvenli) bir şekilde çalışır.
 LazyInitializer.EnsureInitialized(ref _rapor, () =>
 {
 return new AgirRapor();
 });
 _rapor.Yazdir();
 // Tekrar çağrıldığında constructor çalışmaz, mevcut olan kullanılır.
 LazyInitializer.EnsureInitialized(ref _rapor);
 _rapor.Yazdir();
 }
}

```

### §14.4.3 İş Parçacığı Yönetimi ve İzole Veri Yapıları

Modern yazılım geliştirme süreçlerinde, uygulamaların aynı anda birden fazla işi yapabilmesi (**concurrency**) vazgeçilmez bir gereksinimdir. C#, bu süreci yönetmek için düşük seviyeli **Thread** sınıflarından, yüksek seviyeli **Task** tabanlı asenkron yapılara kadar geniş bir yelpaze sunar.

**Thread**, işletim sistemi seviyesindeki en küçük yürütme birimidir. Gerçek bir işçidir. Bir **Thread** başlattığınızda, işletim sisteminden kaynak yani bellek, yığın (**stack**) alanı talep edersiniz. Her **Thread** yaklaşık 1 MB yığın alanı kaplar. Binlerce iş parçacığı oluşturmak sistemi kilitleyebilir.

**async** bir anahtar kelimedir ve bir yöntemin "beklenebilir" (**awaitable**) olduğunu, yani bu metodun içinde zaman alan bir işlem yapılabileceğini belirtir. Bir yöntemin imzasını değiştirir. Bu yöntemin mutlaka bir **Task** veya **Task<T>** döndüreceğini taahhüt eder. **async** tek başına bir işi arka planda çalıştırmaz; sadece o yöntemin içinde **await** kullanılmasına olanak tanır.

**await**, asenkron bir işlem bitene kadar metodun yürütülmesini askıya alır (**suspend**) ancak bu sırada ana iş parçacığını bloklamaz. "Bu işlem bitene kadar buradaki işi durdur, ama iş parçacığı boşa çıksın ve başka işlere baksın. İşlem bittiğinde kaldığın yerden devam et" demektir.

| Özellik  | Thread                                         | Task (async/await)                            |
|----------|------------------------------------------------|-----------------------------------------------|
| Seviye   | Düşük seviyeli işletim sistemi birimi.         | Yüksek seviyeli soyutlama ( <b>promise</b> ). |
| Kaynak   | Ağır (Her biri 1MB Stack).                     | Hafif (Sadece bir nesne referansı).           |
| Kullanım | CPU yoğunluklu işler (Matematiksel hesaplama). | I/O yoğunluklu işler (Dosya, Ağ, Veritabanı). |
| Bloklama | Thread'i durdurur.                             | Thread'i serbest bırakır.                     |

Tablo 14.3: Thread ve Task Karşılaştırması

Bir restoranda olduğunuzu hayal edin:

- ◇ **Thread** (Senkron): Garson siparişi aşçıya verir ve yemek pişene kadar mutfağın önünde bekler. Başka hiçbir masaya bakamaz. (Bloklama).

```

public void VeriIsle() //Bloklayan Yaklaşım!
{
 Thread.Sleep(5000); // Thread burada 5 saniye boyunca donar. UI cevap vermez.
 Console.WriteLine("İşlem bitti");
}

```

- ◇ **Çoklu İş Parçacığı (multi-threading)**: Her masa için yeni bir garson işe alırsınız. Yemekler pişerken

hepsi bekler. Çok fazla masa gelirse dükkan garsonla dolar, batarsınız. (Kaynak tüketimi).

- ◊ **async/await**: Garson siparişi verir, mutfağa "Hazır olunca zile bas" der ve gidip diğer masaların siparişini alır. Zil çalınca (işlem bitince) geri döner ve yemeği servis eder. (Verimlilik).

```
public async Task VeriIsleAsync() //Verimli Yaklaşım.
{
 await Task.Delay(5000); // Thread boşa çıkar, başka işler yapar. // 5 saniye sonra
 ↪ sistem kaldığı yerden devam eder.
 Console.WriteLine("İşlem bitti");
}
```

## İş Parçacığı İzolasyonunun Önemi

Çok iş parçacıklı (**multi-threaded**) ortamlarda en büyük zorluk, paylaşılan verilerin (**shared state**) güvenli bir şekilde yönetilmesidir. Kilitlenme (**locking**) mekanizmaları veri güvenliğini sağlasa da, performans darboğazlarına neden olabilir. Bu noktada, veriyi iş parçacığına veya program akışına özel hale getiren izolasyon yapıları devreye girer.

### ThreadLocal<T>

Verinin her iş parçacığı (**thread**) için özel bir kopyasını tutar. Bir değişkenin değeri "Thread A" için farklı, "Thread B" için farklıdır. İş parçacıkları arası kilitlenmeleri (**locking**) önlediği için paralel programlamada performansı artırır.

Diyelim ki bir döngü içinde birden fazla iş parçacığı çalıştırıyoruz ve her iş parçacığının kendine has bir "sayaç" değerine sahip olmasını istiyoruz.

```
using System;
using System.Threading;
using System.Threading.Tasks;
class Program
{
 // Her thread için özel bir tam sayı deposu oluşturuyoruz. Başlangıç değeri olarak her
 ↪ thread'e rastgele bir ID veriyoruz.
 private static ThreadLocal<int> _threadOzelDeger = new ThreadLocal<int>(() =>
 {
 return Thread.CurrentThread.ManagedThreadId * 100;
 });
 static void Main()
 {
 // 3 farklı işi paralel olarak başlatalım
 Parallel.For(0, 3, i =>
 {
 // Her thread kendi özel kopyasını artırır
 _threadOzelDeger.Value++;
 Console.WriteLine($"Thread ID:
 ↪ {Thread.CurrentThread.ManagedThreadId}, Özel Değeri:
 ↪ {_threadOzelDeger.Value}");
 });
 // Main thread'in kendi kopyası bunlardan etkilenmez
 Console.WriteLine($"Main Thread Değeri: {_threadOzelDeger.Value}");
 // Kaynakları temizle
 _threadOzelDeger.Dispose();
 }
}
```

### AsyncLocal<T>

Modern asenkron yöntemlerde (**async/await**), bir işlem başladıktan sonra **await** satırında farklı bir iş parçacığına (**thread**) geçebilir. **AsyncLocal<T>**, verinin iş parçacığından bağımsız olarak mantıksal işlem akışını (**logical control flow**) takip etmesini sağlar.

**ThreadLocal<T>**'den farklı olarak, verinin asenkron yöntemlerde (**async/await**), (**await**) arasında kaybolmadan taşınmasını sağlar. Örneğin, bir kullanıcının "Kültür/Dil" bilgisini tüm asenkron çağrılar boyunca taşımak için kullanılır.

Düşünelim ki bir web isteği aldınız ve bu isteğe özel bir `KullanıcıID` değerini, metotlara parametre olarak tek tek geçmeden en alt katmanlara kadar ulaştırmak istiyorsunuz.

```
using System;
using System.Threading;
using System.Threading.Tasks;
class Program
{
 // Akış boyunca taşınacak asenkron yerel değişken
 private static AsyncLocal<string> _kullaniciKodu = new AsyncLocal<string>();
 static async Task Main()
 {
 // 1. Akış: Ahmet için başlatılıyor
 await İşlemBaşlat("Ahmet");
 // 2. Akış: Mehmet için başlatılıyor
 await İşlemBaşlat("Mehmet");
 }
 static async Task İşlemBaşlat(string isim)
 {
 _kullaniciKodu.Value = isim;
 Console.WriteLine($"[Başlangıç] Akis sahibi: {_kullaniciKodu.Value}, Thread ID:
 ↳ {Thread.CurrentThread.ManagedThreadId}");
 await AltYöntem();
 }
 static async Task AltYöntem()
 {
 // Bir asenkron bekleme yapıyoruz. Bu noktadan sonra farklı bir thread'e geçebiliriz.
 await Task.Delay(100);
 // AsyncLocal sayesinde veri hala burada!
 Console.WriteLine($"[Alt Yöntem] Akis sahibi: {_kullaniciKodu.Value}, Thread ID:
 ↳ {Thread.CurrentThread.ManagedThreadId}");
 await EnAltKatman();
 }
 static async Task EnAltKatman()
 {
 await Task.Delay(50);
 Console.WriteLine($"[En Alt] Akis sahibi: {_kullaniciKodu.Value}, Thread ID:
 ↳ {Thread.CurrentThread.ManagedThreadId}");
 }
}
```

### Veri İzolasyon Yapıları Karşılaştırması

Aşağıdaki tablo, C# içerisinde verinin nasıl izole edilebileceğini ve asenkron akışlarla olan uyumunu özetlemektedir:

| Yapı                              | Kapsam / İzolasyon Seviyesi                   | async/await Desteği                        |
|-----------------------------------|-----------------------------------------------|--------------------------------------------|
| <code>static</code>               | Uygulama Geneli (Paylaşılan bellek)           | İzolasyon Yok                              |
| <code>ThreadLocal&lt;T&gt;</code> | Fiziksel İş Parçacığı ( <b>thread-bound</b> ) | Yok ( <code>await</code> sonrası kaybolur) |
| <code>AsyncLocal&lt;T&gt;</code>  | Mantıksal Akış ( <b>execution context</b> )   | Tam Desteğe (Akışı takip eder)             |

Tablo 14.4: Veri Saklama ve Taşıma Yapıları Karşılaştırması

### Asenkron Hata Ayıklama

Asenkron programlamada hata yönetimi, senkron koddan farklı olarak istisnanın (**exception**) fırlatıldığı iş parçacığı ile yakalandığı iş parçacığının aynı olmaması nedeniyle özel bir dikkat gerektirir.

**async void** Tehlikesi : Asenkron hata yönetiminde en büyük düşman `async void` kullanımudur. `async void` ile işaretlenmiş bir yöntem hata fırlattığında, çağırıcı (**caller**) bu hatayı `try-catch` ile yakalayamaz. Hata doğrudan uygulamanın işlem (**process**) seviyesine çıkar ve genellikle uygulamanın çökmesine neden olur. Olay işleyiciler (**event handlers**) dışında her zaman `async Task` kullanın. `Task` döndüren yöntemlerde istisna, dönen `Task` nesnesinin içine paketlenir.

**try-catch** Kullanımı: Asenkron metotlarda `try-catch` blokları senkron koda çok benzer şekilde çalışır,

ancak arka planda derleyici (**state machine**) hatayı sizin için doğru bağlama taşır.

```
public async Task VeriIsleAsync()
{
 try
 {
 await ServisCagrisi();
 }
 catch (HttpRequestException ex)
 {
 // Hata burada yakalanır, await sayesinde doğru context'e (bağlama) geri dönmüştür.
 Console.WriteLine($"Hata oluştu: {ex.Message}");
 }
}
```

Birden Fazla Hata (**AggregateException**): Aynı anda birden fazla asenkron işi başlattığınızda (**Task.WhenAll**), birden fazla hata oluşabilir. **await** Davranışı: **await Task.WhenAll(...)** kullandığınızda, yakalanan istisna sadece ilk oluşan hatadır. Tüm Hataları Yakalamak: Eğer tüm hataları görmek istiyorsanız, **Task** nesnesinin **Exception** özelliğini incelemeniz gerekir.

```
Task tumIsler = Task.WhenAll(task1, task2, task3);
try
{
 await tumIsler;
}
catch (Exception)
{
 // Sadece ilk hata burada yakalanır. Tüm hatalara erişmek için:
 foreach (var ex in tumIsler.Exception.InnerExceptions)
 {
 Console.WriteLine(ex.Message);
 }
}
```

**ConfigureAwait(false)** ve Hatalar: Kütüphane (**library**) geliştirirken sıkça kullanılan **ConfigureAwait(false)**, hatanın yakalandığı iş parçacığını değiştirebilir. Eğer hatayı yakaladıktan sonra bir UI bileşenine (örneğin **WinForms Label**) erişecekseniz, bu durum "**Cross-thread operation not valid**" hatasına yol açar.

| Senaryo                                         | Önerilen Yöntem                                              | Risk / Dikkat                                                                                                              |
|-------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>async Task</b>                               | Standart <b>try-catch</b> .                                  | İstisna Task içine paketlenir.                                                                                             |
| <b>async void</b>                               | Sadece Event Handler'larda kullanın.                         | Hata yakalanamaz, uygulama çökebilir.                                                                                      |
| <b>Task.WhenAll</b><br><b>.Result / .Wait()</b> | <b>AggregateException</b> kontrolü.<br>Kullanmaktan kaçınin. | <b>await</b> sadece ilk hatayı döner.<br><b>AggregateException</b> fırlatır ve kilitlenme ( <b>deadlock</b> ) riski taşır. |

Tablo 14.5: Asenkron Hata Yönetimi Yöntemleri

#### §14.4.4 Performans Notları

Eğer bellek kullanımı kritikse **LazyInitializer** statik sınıfı tercih edilmelidir. **Lazy<T>** nesnesinin oluşturacağı ek yükten kaçınılmak isteniyorsa da bu yapı kullanılır. Bu yapı, ham göstericiler (**raw pointers**) üzerinde **atomic** operasyonlar yürüterek düşük seviyeli ve yüksek performanslı bir başlatma sağlar.

- ◊ **Lazy<T>**: Nesne tabanlı, kolay kullanım, iş parçacığı güvenlidir (**thread-safe**).
- ◊ **LazyInitializer**: Statik yardımcı, düşük bellek tahsisi, yüksek performans.

İş parçacığı yönetiminde uygulamanın zaman uyumsuz (**asynchron**) olup olmadığına dikkat edilmelidir. Eğer **Task** ve **await** yapıları yoğun kullanılıyorsa **AsyncLocal<T>**, düşük seviyeli iş parçacığı havuzları yönetiliyorsa **ThreadLocal<T>** doğru tercihtir.

## 14.5 İleri Seviye Jenerik Mimariler

Bu yapılar, kodun tip güvenliğini (**type safety**) sağlarken performansı maksimize eder.



| Yapı                              | Temel Odak            | Kritik Avantaj                                                          |
|-----------------------------------|-----------------------|-------------------------------------------------------------------------|
| <code>Lazy&lt;T&gt;</code>        | Nesne Ömrü Yönetimi   | Kolay kullanım, otomatik iş parçacığı-güvenli ( <b>thread-safety</b> ). |
| <code>LazyInitializer</code>      | Düşük Bellek Tüketimi | Ekstra nesne yaratmadan başlatma yapar.                                 |
| <code>ThreadLocal&lt;T&gt;</code> | Paralel Programlama   | İş parçacıkları arası izole veri saklar.                                |
| <code>AsyncLocal&lt;T&gt;</code>  | Asenkron Akış         | <b>async/await</b> akışında veriyi taşır.                               |

Tablo 14.6: Ertelenmiş Başlatma ve Veri Saklama Yapıları

### §14.5.1 Nullable<T>

Sadece değer tipleri için kullanılır. Aslında `int?` veya `bool?` olarak kullandığımız yapıların arkasındaki jenerik `struct` yapısıdır. Yığında olan değer tiplerinin (**stack-based**) `null` değer alabilmesini sağlar. Veritabanı işlemlerinde (opsiyonel alanlar) hayatı öneme sahiptir. `HasValue` ve `Value` özellikleriyle kontrol edilir. 4.7 başlığında anlatılmıştı.

### §14.5.2 ValueTuple<T1, T2, ...>

C#7.0+ ile gelmiştir. Yöntemlerden birden fazla değer döndürmek için kullanılan hafif (**lightweight**) bir yapıdır. `Tuple<T>` bir sınıftır ve öbek bellekte (**heap**) bulunur, `ValueTuple` bir yapıdır ve yığın bellekte (**stack**) bulunur. Bu da onu çok daha performanslı kılar. İsimlendirilebilir alanlar (**field**) sunar.

Aşağıdaki örnekte, bir dizideki hem en küçük hem de en büyük değeri aynı anda döndüren bir metod tasarlayalım. Eskiden bunun için `out` parametreleri veya özel bir `class` kullanmamız gerekirdi.

```
using System;
using System.Linq;
public class IstatistikAraci
{
 // Yöntemin dönüş tipi (int Min, int Max) olan bir ValueTuple'dır.
 public (int Min, int Max) Hesapla(int[] sayilar)
 {
 if (sayilar == null || sayilar.Length == 0)
 return (0, 0);
 int min = sayilar.Min();
 int max = sayilar.Max();
 // Geriye isimlendirilmiş bir ValueTuple döndürüyoruz.
 return (min, max);
 }
}
class Program
{
 static void Main()
 {
 var arac = new IstatistikAraci();
 int[] veriler = { 45, 12, 89, 3, 67 };
 // 1. Kullanım: Doğrudan isimlendirilmiş alanlara erişim
 var sonuc = arac.Hesapla(veriler);
 Console.WriteLine($"En Küçük: {sonuc.Min}, En Büyük: {sonuc.Max}");
 // 2. Kullanım: Deconstruction (Nesneyi parçalarına ayırma)
 var (enKucuk, enBuyuk) = arac.Hesapla(veriler);
 Console.WriteLine($"Parçalanmış Veri: {enKucuk} ve {enBuyuk}");
 }
}
```

| Özellik    | Tuple (Eski)                                            | ValueTuple (Yeni)                                  |
|------------|---------------------------------------------------------|----------------------------------------------------|
| Tip        | Reference Type ( <code>class</code> )                   | Value Type ( <code>struct</code> )                 |
| Bellek     | Öbek ( <b>heap</b> ) bellekte yer alır.                 | Yığın ( <b>stack</b> ) üzerinde tutulur.           |
| Sorumluluk | <code>Item1</code> , <code>Item2</code> (Sabit isimler) | <code>Min</code> , <code>Max</code> (Özel isimler) |
| Performans | Daha yavaştır (çöp toplayıcı baskısı).                  | Çok hızlıdır.                                      |

Tablo 14.7: ValueTuple Teknik Karşılaştırması

`ValueTuple` kullanarak;

- ◆ Dönüş tipindeki alanlara `Isim`, `Yaş` gibi anlamlı isimler verebilirsiniz. `Tuple` kullanarak eskiden sadece



`Item1, Item2` diyebiliyordunuz.

- ◊ Bir `struct` olduğu için öbek (**heap**) yerine yığın (**stack**) belleği kullanır, bu da çöp toplayıcı (GC) üzerindeki yükü azaltır.
- ◊ `var (x, y) = GetPoint();` şeklinde tek satırda parçalarına ayırabilirsiniz.

### §14.5.3 EqualityComparer<T>

Jenerik koleksiyonlarda (`Dictionary` veya `HashSet` gibi) nesnelerin birbirine eşit olup olmadığını kontrol etmek için kullanılan standart yapıdır. `T` tipinin `Equals()` yöntemini her seferinde çağırmak yerine, tipin en uygun karşılaştırma yöntemini (`IEquatable<T>` gibi) otomatik olarak seçer. Kutulama (**boxing/unboxing**) işlemlerini önleyerek büyük veri setlerinde hız kazandırır.

Diyelim ki elimizde `Urun` nesneleri var ve biz bu nesneleri bir `HashSet` içinde saklamak istiyoruz. Varsayılan olarak `C#`, nesnelerin bellek adreslerine bakar. Ancak biz, `Ürün Kodu` aynı olan ürünlerin "aynı" sayılmasını istiyorsak kendi karşılaştırıcımızı yazmalıyız.

```
using System;
using System.Collections.Generic;
public class Urun
{
 public string UrunKodu { get; set; }
 public string Ad { get; set; }
}
// 1. Karşılaştırıcı Sınıf: EqualityComparer<T>'den türetilir.
public class UrunKarsilastirici : EqualityComparer<Urun>
{
 public override bool Equals(Urun x, Urun y)
 {
 // Null kontrolleri
 if (ReferenceEquals(x, y)) return true;
 if (x == null || y == null) return false;
 // Ürün koduna göre eşitlik kontrolü
 return x.UrunKodu == y.UrunKodu;
 }
 public override int GetHashCode(Urun obj)
 {
 if (obj == null) return 0;
 // Hash kodu olarak ürün kodunun hash'ini döndürürüz
 return obj.UrunKodu.GetHashCode();
 }
}
class Program
{
 static void Main()
 {
 // Karşılaştırıcıyı koleksiyona enjekte ediyoruz
 HashSet<Urun> urunler = new HashSet<Urun>(new UrunKarsilastirici());
 Urun u1 = new Urun { UrunKodu = "ELMA1", Ad = "Amasya Elması" };
 Urun u2 = new Urun { UrunKodu = "ELMA1", Ad = "Yeşil Elma" }; // Kod aynı!
 urunler.Add(u1);
 urunler.Add(u2); // Bu eklenmeyecektir çünkü kodlar aynı sayılır.
 Console.WriteLine($"Listedeki Ürün Sayısı: {urunler.Count}"); // Çıktı: 1
 }
}
```

### §14.5.4 IProgress<T>

Zaman uyumsuz (**asynchron**) işlemlerde (özellikle uzun süren döngülerde) ilerleme bilgisini UI katmanına veya çağırıcı metoda güvenli bir şekilde iletmek için kullanılır. `Task.Run` içinde çalışırken `Report(T value)` yöntemi ile ilerleme yüzdesi veya durumu gönderilir. `C#`'in `async/await` mimarisiyle tam uyumludur.

`IProgress<T>`, özellikle uzun süren bir işlem (dosya indirme, veritabanı işleme vb.) sırasında kullanıcı arayüzüne veya kullanıcıya "işlemin % kaç bitti" bilgisini asenkron bir şekilde iletmek için kullanılır. Diyelim ki 1'den 10'a kadar sayan ve her adımda ilerlemeyi raporlayan asenkron bir yöntemimiz var.

```
using System;
```

```

using System.Threading.Tasks;
public class İşlemci
{
 // IProgress<int> parametresi, ilerleme bilgisini taşıyan kanaldır.
 public async Task UzunSürenİşlem(IProgress<int> progress)
 {
 for (int i = 1; i <= 10; i++)
 {
 await Task.Delay(500); // Gerçek bir iş yapılmıyormuş gibi bekleme
 progress?.Report(i * 10); // İlerlemeyi raporla (% cinsinden veya adım sayısı)
 }
 }
}
class Program
{
 static async Task Main()
 {
 İşlemci işlemci = new İşlemci();
 // 1. Progress nesnesi oluşturulur ve rapor geldiğinde ne yapılacağı tanımlanır.
 var progressKanalı = new Progress<int>(yuzde =>
 {
 Console.WriteLine($"İlerleme: %{yuzde}%");
 });
 Console.WriteLine("İşlem başlatılıyor...");
 // 2. Yöntem çağrılırken bu kanal parametre olarak geçer.
 await işlemci.UzunSürenİşlem(progressKanalı);
 Console.WriteLine("İşlem tamamlandı!");
 }
}

```

### §14.5.5 Comparer<T> ve IComparable<T>

Sıralama (sorting) algoritmalarının temelini oluşturan jenerik yapılardır. İki nesne arasındaki büyük-lük/küçüklük ilişkisini belirler. Default özelliği ile o tip için tanımlanmış varsayılan sıralama kuralına erişilir.

IComparable<T>, bir sınıfın kendi içindeki varsayılan sıralama kuralıdır. Örneğin bir Öğrenci listesini her zaman "Not" değerine göre sıralamak istiyorsanız bu arayüzü kullanırsınız.

```

using System;
using System.Collections.Generic;
public class Öğrenci : IComparable<Öğrenci>
{
 public string Ad { get; set; }
 public int Not { get; set; }
 // Kendimi (this) diğeriyle (other) kıyaslıyorum
 public int CompareTo(Öğrenci other)
 {
 if (other == null) return 1;
 // Not değerine göre küçükten büyüğe sıralama
 return this.Not.CompareTo(other.Not);
 }
}

```

Comparer<T>, eğer varsayılan kuralın dışında (örneğin bazen isme göre, bazen nota göre) sıralama yapmak istiyorsanız, dışarıdan bir karşılaştırıcı enjekte edersiniz.

```

public class ÖğrenciAdınaGoreSırala : Comparer<Öğrenci>
{
 public override int Compare(Öğrenci x, Öğrenci y)
 {
 // Null kontrolleri
 if (x == null && y == null) return 0;
 if (x == null) return -1;
 if (y == null) return 1;
 // İsme göre alfabetik sıralama
 }
}

```

```

 return string.Compare(x.Ad, y.Ad);
 }
}

```

### §14.5.6 Özet

| Yapı             | Veri Tipi | Bellek        | Temel Kullanım Senaryosu                      |
|------------------|-----------|---------------|-----------------------------------------------|
| Nullable<T>      | struct    | Yığın (stack) | Opsiyonel sayısal/mantıksal veriler.          |
| ValueTuple       | struct    | Yığın (stack) | Hızlı ve geçici çoklu veri döndürme.          |
| Lazy<T>          | class     | Öbek (heap)   | Maliyetli nesnelerin ertelenmiş başlatılması. |
| EqualityComparer | class     | Öbek (heap)   | Koleksiyonlarda nesne eşleşme mantığı.        |

Tablo 14.8: Kritik Jenerik Yapıların Teknik Analizi

## 14.6 İndisleyiciler

**İndisleyiciler** (**indexer**), nesneleri bir dizi gibi kullanmayı sağlarlar. İndisleyiciler, köşeli parantez içinde parametresi olan **this** kelimesiyle tanımlanırlar. Özellikler (**property**) gibi indisleyiciler de **get** ve **set** yöntemleri kullanılır ve yalnız okunur (**read only**), yalnız yazılır (**write only**) olabilirler. Bunların dışında indisleyiciler sanal olarak da tanımlanabilir ve parametresi farklı olmak kaydıyla indisleyicilere aşırı yükleme (**overload**) yapılabilir.

```

İndisliListe<string> names = new İndisliListe<string>();
names.Add("Mehmet");
names.Add("Abdullah");
names.Add("İlhan");
for (int i = 0; i < names.ElemanSayısı; i++)
{
 string s = names[i]; //indis kullanımı:
 names[i] = s.ToUpper();
 Console.WriteLine(names[i]);
}

public class İndisliListe<T>
{
 //Sabitler:
 const int varsayılanElemanKapasitesi = 4;
 //Alanlar:
 T[] elemanlar;
 int elemanSayısı;
 //Yapıcılar:
 public İndisliListe(int kapasite)
 {
 elemanlar = new T[kapasite];
 }
 public İndisliListe() : this(varsayılanElemanKapasitesi) { }

 //Özellikler:
 public int ElemanSayısı
 {
 get { return elemanSayısı; }
 }
 public int Kapasite
 {
 get { return elemanlar.Length; }
 set
 {
 if (value < elemanSayısı) value = elemanSayısı;
 if (value != elemanlar.Length)
 {
 T[] yeniElemanlar = new T[value];
 Array.Copy(elemanlar, 0, yeniElemanlar, 0, elemanSayısı);
 elemanlar = yeniElemanlar;
 }
 }
 }
}

```

```

 }
}
//İndisleyici Tanımı:
public T this[int indis]
{
 get { return elemanlar[indis]; }
 set
 {
 if (indis < 0 || indis >= elemanSayısı)
 throw new ArgumentOutOfRangeException("İndis Hatası");
 elemanlar[indis] = value;
 }
}
//Yöntemler:
public void Add(T eleman)
{
 if (elemanSayısı == Kapasite) Kapasite = elemanSayısı * 2;
 elemanlar[elemanSayısı] = eleman;
 elemanSayısı++;
}
}

```

## 14.7 Düşün ve Çöz

- ♦ **Düşün:** Jenerik programlama neden tip güvenliği ve performans açısından jenerik olmayan (object tabanlı) koleksiyonlara üstündür? Eski `ArrayList` ile `List<T>` arasındaki farkı kutulama/kutudan çıkarma üzerinden açıklayınız.
- ♦ **Düşün:** Jenerik kısıtlamalar (constraints: `where T : class`, `where T : new()`, `where T : IComparable`) ne amaçla kullanılır? Kısıtlamalar olmadan jenerik bir algoritmanın hangi işlemleri gerçekleştiremeyeceğini örnekleyiniz.
- ♦ **Düşün:** `List<T>`, `Dictionary<K,V>`, `HashSet<T>`, `Stack<T>` ve `Queue<T>` koleksiyonlarının hangi senaryolarda tercih edildiğini karşılaştırınız. Arama (lookup) hızı açısından neden `Dictionary` ve `HashSet` öne çıkar?
- ♦ **Düşün:** `ConcurrentDictionary<K,V>` ve diğer thread-safe koleksiyonlar ne zaman gereklidir? Normal `Dictionary` üzerinde lock kullanmakla kıyaslandığında performans ve kullanım kolaylığı açısından farkı nedir?
- ♦ **Düşün:** İndisleyici (indexer) özelliği nedir? Sınıfınızı dizi gibi kullanılabilir kılmanın avantajı ne olabilir? Çok boyutlu indisleyici tanımlamak mümkün müdür?
- ♦ **Düşün:** `IEnumerable<T>` ile `ICollection<T>` ve  `IList<T>` arasındaki hiyerarşi nedir? Her arayüzün sunduğu ek yetenekler nelerdir? Yalnızca okuma gereken bir senaryoda `IEnumerable<T>` döndürmenin neden daha iyi bir API tasarımı olduğunu açıklayınız.
- ♦ **Düşün:** Değiştirilemez koleksiyonlar (`ImmutableList<T>`, `ImmutableDictionary<K,V>`) ne zaman ve neden tercih edilir? Çok iş parçacıklı (multi-threaded) bir ortamda değiştirilemez koleksiyonların lock ihtiyacını nasıl ortadan kaldırdığını açıklayınız.
- ♦ **Çöz:** Jenerik bir `MinYığılı` (MinHeap) sınıfı tasarlayınız: ekleme, minimum değeri çekme (extract-min) ve minimum değeri gözetleme (peek) yöntemlerini uygulayın. `IComparable<T>` kısıtlamasını kullanın.
- ♦ **Çöz:** Bir metin belgesi içindeki sözcük frekanslarını tutan bir program yazınız (`Dictionary<string, int>`). En çok geçen 10 sözcüğü sıralayarak listeleyin. Büyük/küçük harf duyarsızlığını ekleyin.
- ♦ **Çöz:** Jenerik bir önbellek (cache) sınıfı tasarlayınız: maksimum eleman sayısı aşıldığında en eski eklenen eleman silinsin (FIFO). `Dictionary<K,V>` ve `Queue<K>` birlikte kullanınız.
- ♦ **Çöz:** İndisleyici kullanarak matris gibi çalışan iki boyutlu bir jenerik `Izgara<T>` sınıfı tasarlayınız: hem `[satır, sütun]` hem de `[indis]` erişimini desteklesin.
- ♦ **Çöz:** Thread-safe bir mesaj kuyruğu (message queue) sınıfı oluşturun: üretici (producer) iş parçacıkları mesaj eklesin, tüketici (consumer) iş parçacıkları mesaj çeksın. `ConcurrentQueue<T>` veya `BlockingCollection<T>` kullanın.

## 15. Bölüm

# LINQ Sorguları

### 15.1 LINQ Nedir?

Dile Entegre Sorgu LINQ(Language INtegrated Query) anlamına gelir. Çeşitli veri kaynakları ve biçimleri arasında verilerle çalışmak için tutarlı bir model sunarak bir sorgu dilini programlama diline entegre eden bir kavramdır; XML belgelerinde, SQL veri tabanlarında, ADO.NET Veri Kümelerinde, .NET koleksiyonlarında ve LINQ sağlayıcısının sunmuş olduğu biçimlerde verileri sorgulamak ve dönüştürmek için aynı temel kodlama kalıplarını kullanırsınız.

LINQ, çeşitli veri kaynaklarından gelen verileri tek tip bir şekilde sorgulamamıza olanak tanıyan ortak bir söz dizimi sağlar. Bu, tek bir LINQ sorgusu kullanarak SQL Server veri tabanı, XML belgeleri, ADO.NET Veri Kümeleri ve Koleksiyonlar, Jenerik gibi çeşitli veri kaynaklarından veri alabileceğimiz anlamına gelir.

```
List<İl> iller = new List<İl> {
 new İl { ilTrafikKodu = 1, Adı = "Adana" },
 new İl { ilTrafikKodu = 34, Adı = "İstanbul" },
 new İl { ilTrafikKodu = 5, Adı = "Amasya" },
 new İl { ilTrafikKodu = 6, Adı = "Ankara" },
 new İl { ilTrafikKodu = 7, Adı = "Antalya" },
 new İl { ilTrafikKodu = 16, Adı = "Bursa" },
 new İl { ilTrafikKodu = 10, Adı = "Balıkesir" },
 new İl { ilTrafikKodu = 61, Adı = "Trabzon" }
};
```

// Örneklerdeki Sorgular Buraya Gelecek

```
struct İl
{
 public int ilTrafikKodu;
 public string Adı;
}
```

Yukarıdaki gibi bir veri kaynağını göz önüne aldığımızda LINQ sorguları iki şekilde yazılabilir. Başlıklar halinde verilmiştir.

#### §15.1.1 Sorgu Şeklinde

SQL'e benzer ve genellikle SQL'e aşina olanlar için daha okunabilir. Bir `from` ifadesiyle başlar, ardından bir aralık değişkeni gelir ve `where`, `select`, `group`, `join` vb. gibi standart sorgu işlemlerini içerir.

```
from object in datasource
where condition
select object
```

Yukarıdaki söz dizimine sahip sorguya örnek aşağıdaki kod verilebilir;

```
var query = from i in iller
 where i.Adı == "Ankara"
 select i.Adı;
```

#### §15.1.2 Akıcı (fluent) Şekilde

Genişleme yöntemleri (`extension method`) ve lambda ifadeleri kullanır. Daha öz olabilir ve karmaşık sorgular yazarken tercih edilir çünkü okunması ve oluşturulması daha kolay olabilir.

```
datasource.ConditionMethod().SelectionMethod()
```

Yukarıdaki söz dizimine sahip sorguya örnek aşağıdaki kod verilebilir;

```
var query = iller.Where(i => i.Adı == "Ankara").Select(i => i.Adı);
IEnumerable ve IQueryable, veri işleme ve sorgulama için sıklıkla kullanılan iki ara yüzdür (interface),
```

ancak farklı amaçlara hizmet ederler ve farklı davranışlara sahiptirler. Aşağıdaki örnekte `query` değişkenleri `IEnumerable<string>?` veri tipine sahiptir.

```
IEnumerable<int> queryIlKodları = iller.Select(i => i.ilTrafikKodu);
IEnumerable<string> queryIlListesi = iller.Where(i => i.Adı.Contains("an")).Select(i => i.Adı);
```

LINQ sorgularının tanımlandığı anda değil, üzerinde `foreach` ile döndüğünde veya `.ToList()` gibi yöntemler çağrıldığında çalıştırıldığına veri döndürür. Bu, LINQ'in en temel mantığıdır!

## 15.2 IEnumerable ve IEnumerator Arayüzleri

LINQ fonksiyonları hem bir `IEnumerable<TSource>` üzerinde çalışır ve bir `IEnumerable<TResult>` döndürür. Bu arayüzler daha önce 14.2.5 başlığında anlatılmıştır. Bu arayüzler .NET Framework'un önemli bir parçasıdır ve bir nesne koleksiyonu üzerinde yineleme (`iteration`) yapmak için kullanılır. Yukarıdaki örnekte yapılan sorgulamalar aşağıdaki şekilde yinelenabilir;

```
foreach (var item in queryIlKodları)
 Console.WriteLine($"{item} ");
// 1 34 5 6 7 16 10 61
foreach (var item in queryIlListesi)
 Console.WriteLine($"{item} ");
// Adana İstanbul
```

## 15.3 IQueryable Arayüzü

`IQueryable`, bir veri kaynağından veri sorgulamak için kullanılan bir ara yüzüdür. `System.Linq` isim uzayının bir parçasıdır ve önemli bir bileşendir. Bellek içi koleksiyonlar üzerinde yineleme yapmak için kullanılan `IEnumerable` ara yüzünün aksine, `IQueryable`, nesne numaralandırılana kadar sorgunun yürütülmediği veri kaynaklarını sorgulamak için tasarlanmıştır. Bu, özellikle veri tabanları gibi uzak veri kaynakları için yararlıdır ve sorgunun sonucu tarafında yürütülmesine izin vererek verimli sorgulamayı mümkün kılar.

```
IQueryable<İl> bazıİller = iller.AsQueryable().Where(i => i.ilTrafikKodu < 30);
foreach (var item in bazıİller) //Sorgu sonucu İl nesneleri döner
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*Program Çıktısı:
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
*/
```

`IEnumerable` istemci belleğinde yürütülürken, `IQueryable` veri kaynağında yürütülür. `IEnumerable` nesneler ve bellek içi verilerle çalışmak için uygundur. `IQueryable`, SQL veri tabanları ile etkileşim kurması için uygundur. `IQueryable`, veri tabanının verileri optimize etmesine ve filtrelemesine izin verdiği için büyük veri kümeleri için daha iyi performans gösterebilir.

## 15.4 LINQ Genişleme Yöntemleri

Genişleme yöntemleri (`extension method`), yeni bir türetilmiş tip oluşturmadan, yeniden derlemeden veya orijinal türü başka şekilde değiştirmeden mevcut türlere davranışı genişletir ve ekler. Genişleme yöntemleri konusu 8.18 başlığında incelenmişti.

### §15.4.1 Where

```
var filteredResult = iller.Where(item => item.Adı == "Ankara");
foreach (var item in filteredResult)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 6, Adı: Ankara
*/
```

### §15.4.2 Select

```
var projectedResult = iller.Select(item => new { ilTrafikKodu = 61, Adı = "Trabzon" });
```

```

foreach (var item in projectedResult)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/* iller koleksiyonunda ne kadar eleman varsa o kadar yeni il ekler
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
*/
foreach (var item in iller)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
*/

```

### §15.4.3 OrderBy ve OrderByDescending

```

var sortedResult = iller.OrderBy(item => item.ilTrafikKodu);
foreach (var item in sortedResult)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 61, Adı: Trabzon
*/
var sortedDescendingResult = iller.OrderByDescending(item => item.Adı);
foreach (var item in sortedDescendingResult)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 1, Adı: Adana
*/

```

### §15.4.4 Group By

```

List<İl> iller = new List<İl> {
 new İl { ilTrafikKodu = 1, Adı = "Adana", Bölge="Akdeniz" },
 new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
 new İl { ilTrafikKodu = 5, Adı = "Amasya", Bölge = "Karadeniz" },
 new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
 new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
 new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
 new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },

```



```

 new İl { İlTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" }
};
IEnumerable<IGrouping<string, İl>> groupedResult = iller.GroupBy(item => item.Bölge);
foreach (var item in groupedResult)
{
 Console.WriteLine($"{item.Key} "); //Akdeniz Marmara Karadeniz İç Anadolu
}
struct İl
{
 public int İlTrafikKodu;
 public string Adı;
 public string Bölge;
}

```

### §15.4.5 First ve Last

Bir dizinin ilk ögesini veya sağlanan öngörüye uyan ilk veya son elemanı döndürür. Dizi hiçbir öge içermiyorsa, şu iletiyle bir `InvalidOperationException` istisnası oluşur.

```

new[] { "a" }.First(); // "a"
new[] { "a", "b" }.First(); // "a"
new[] { 1, 2 }.First(); // 1
new[] { "a", "b" }.First(x => x.Equals("b")); // "b"
new[] { 1, 2 }.First(x => x > 1); // 2
new[] { "ba", "be" }.First(x => x.Contains("b")); // "ba"
//new[] { "ca", "ce" }.First(x => x.Contains("b")); // Hata: InvalidOperationException
new[] { "a" }.Last(); // "a"
new[] { "a", "b" }.Last(); // "b"
Console.WriteLine(new[] { 1, 2 }.Last()); // 2
new[] { "a", "b" }.Last(x => x.Equals("a")); // "a"
Console.WriteLine(new[] { 1, 2 }.Last(x => x < 2)); // 1
new[] { "ba", "be" }.Last(x => x.Contains("b")); // "be"
//new[] { "ca", "ce" }.Last(x => x.Contains("b")); // Hata: InvalidOperationException

```

### §15.4.6 FirstOrDefault ve LastOrDefault

Bir dizinin ilk ögesini veya sağlanan süzgece uyan ilk elemanı veya son elemanı döndürür. Dizi hiçbir öge içermiyorsa veya sağlanan fonksiyona uyan hiçbir öge yoksa, `default(T)` kullanılarak dizi veri tipinin varsayılan değeri döndürülür.

```

new[] { "a" }.FirstOrDefault(); // "a"
new[] { "a", "b" }.FirstOrDefault(); // "a"
new[] { 1, 2 }.FirstOrDefault(); // 1
new[] { "a", "b" }.FirstOrDefault(x => x.Equals("b")); // "b"
new[] { 1, 2 }.FirstOrDefault(x => x > 1); // 2
new[] { "ba", "be" }.FirstOrDefault(x => x.Contains("b")); // "ba"
new[] { 1, 2 }.FirstOrDefault(x => x > 2); // 0
new[] { "ca", "ce" }.FirstOrDefault(x => x.Contains("b")); // null
new[] { "a" }.LastOrDefault(); // "a"
new[] { "a", "b" }.LastOrDefault(); // "b"
new[] { 1, 2 }.LastOrDefault(); // 2
new[] { "a", "b" }.LastOrDefault(x => x.Equals("a")); // "a"
new[] { 1, 2 }.LastOrDefault(x => x < 2); // 1
new[] { "ba", "be" }.LastOrDefault(x => x.Contains("b")); // "b3"
new[] { 1, 2 }.LastOrDefault(x => x > 2); // 0
new[] { "ca", "ce" }.LastOrDefault(x => x.Contains("b")); // null

```

### §15.4.7 Single

Bir dizinin tek elemanı varsa veya sağlanan süzgece uyan tek elemanı döndürür. Dizi hiçbir öge içermiyorsa, şu iletiyle bir `InvalidOperationException` istisnası oluşur.

```

new[] { "a" }.Single(); // "a"
//new[] { "a", "b" }.Single(); // InvalidOperationException
new[] { 1 }.Single(); // 1
new[] { "a", "b" }.Single(x => x.Equals("b")); // "b"
new[] { 1, 2 }.Single(x => x < 2); // 1

```

```
//new[] { "a", "b" }.Single(x => x.Equals("c")); // InvalidOperationException
//new[] { 1, 2 }.Single(x => x > 2); //InvalidOperationException
//new string[0].Single(); // InvalidOperationException
```

### §15.4.8 SingleOrDefault

Bir dizinin son öğesini veya sağlanan süzgece uyan son öğeyi döndürür. Dizi hiçbir öğe içermiyorsa veya sağlanan fonksiyona uyan hiçbir öğe yoksa, `default(T)` kullanılarak dizi veri tipinin varsayılan değeri döndürülür.

```
new[] { "a" }.SingleOrDefault(); // "a"
//new[] { "a", "b" }.SingleOrDefault(); // InvalidOperationException
new[] { 1 }.SingleOrDefault(); // 1
new[] { "a", "b" }.SingleOrDefault(x => x.Equals("b")); // "b"
new[] { 1, 2 }.SingleOrDefault(x => x > 1); // 2
new[] { "a", "b" }.SingleOrDefault(x => x.Equals("c")); // null
new[] { 1, 2 }.SingleOrDefault(x => x > 2); // 0
```

### §15.4.9 Sum, Min, Max ve Average

```
var total = iller.Sum(item => item.ilTrafikKodu);
Console.WriteLine($"Toplam İl Trafik Kodu: {total}");
var average = iller.Average(item => item.ilTrafikKodu);
Console.WriteLine($"Ortalama İl Trafik Kodu: {average}");
var min = iller.Min(item => item.ilTrafikKodu);
Console.WriteLine($"En Küçük İl Trafik Kodu: {min}");
var max = iller.Max(item => item.ilTrafikKodu);
Console.WriteLine($"En Büyük İl Trafik Kodu: {max}");
```

### §15.4.10 Any ve All

Herhangi bir eleman veya tüm dizi elemanlarını bir koşulu karşılayıp karşılamadığını kontrol eder.

```
bool hasItems = iller.Any();
Console.WriteLine(hasItems); // true
bool allMatchCondition = iller.All(item => item.ilTrafikKodu < 70);
Console.WriteLine(allMatchCondition); // true
bool anotherMatchCondition = iller.All(item => item.ilTrafikKodu > 70);
Console.WriteLine(anotherMatchCondition); // false
```

### §15.4.11 Count

Bir dizideki öğeleri sayar, isteğe bağlı olarak koşulla verilebilir.

```
int count = iller.Count();
Console.WriteLine($"Toplam İl Sayısı: {count}"); // 8
int conditionalCount = iller.Count(item => item.Bölge == "Karadeniz");
Console.WriteLine($"Karadeniz Bölgesindeki İl Sayısı: {conditionalCount}"); // 2
```

### §15.4.12 FindAll

Verilen ifadeye uygun bir dizideki öğeleri döndürür.

```
List<int> girdiler = new List<int>() { 10, 20, 30, 40, 50, 60 };
Console.WriteLine("Girdiler: ");
foreach (var değer in girdiler)
 Console.WriteLine("{0} ", değer);
// Girdiler: 10 20 30 40 50 60
Console.WriteLine();
List<int> üçeBölünenler = girdiler.FindAll(x => (x % 3) == 0);
Console.WriteLine("Üçe Bölünen Girdiler: ");
foreach (var değer in üçeBölünenler)
 Console.WriteLine("{0} ", değer);
// Üçe Bölünen Girdiler: 30 60
```

### §15.4.13 Distinct

Bir diziden tekil olan veya diğerlerinden farklı elemanları döndürür.

```
List<İl> iller = new List<İl> {
 new İl { ilTrafikKodu = 1, Adı = "Adana" , Bölge="Akdeniz" },
```

```

new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
new İl { ilTrafikKodu = 1, Adı = "Adana", Bölge="Akdeniz" },
new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
new İl { ilTrafikKodu = 1, Adı = "Adana", Bölge="Akdeniz" },
new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
};
Console.WriteLine("Listenin Tekil Olan Elemanları:");
var tekilliste = iller.Distinct();
foreach (var item in tekilliste)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
Listenin Tekil Olan Elemanları:
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 6, Adı: Ankara
*/
struct İl
{
 public int ilTrafikKodu;
 public string Adı;
 public string Bölge;
}

```

#### §15.4.14 Intersect, Except ve Union

Diziler üzerinde kesişim, fark ve birleşim gibi küme işlemlerini gerçekleştirir.

```

List<İl> iller1 = new List<İl> {
 new İl { ilTrafikKodu = 1, Adı = "Adana", Bölge="Akdeniz" },
 new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
 new İl { ilTrafikKodu = 5, Adı = "Amasya", Bölge = "Karadeniz" },
 new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
 new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
 new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
 new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },
 new İl { ilTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" }
};
List<İl> iller2 = new List<İl> {
 new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
 new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
 new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
 new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
 new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },
 new İl { ilTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" },
 new İl { ilTrafikKodu = 35, Adı = "İzmir", Bölge = "Ege" },
 new İl { ilTrafikKodu = 45, Adı = "Manisa", Bölge = "Ege" },
 new İl { ilTrafikKodu = 67, Adı = "Zonguldak", Bölge = "Karadeniz" }
};
Console.WriteLine("İki Listenin Kesişim Kümesi:");
var kesişimKümesi = iller1.Intersect(iller2);
foreach (var item in kesişimKümesi)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
*/

```

```

Console.WriteLine("İki Listenin Birleşim Kümesi:");
var birleşimKümesi = iller1.Union(iller2);
foreach (var item in birleşimKümesi)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 35, Adı: İzmir
İl Trafik Kodu: 45, Adı: Manisa
İl Trafik Kodu: 67, Adı: Zonguldak
*/
Console.WriteLine("İki Listenin Fark Kümesi:");
var farkKümesi = iller1.Except(iller2);
foreach (var item in farkKümesi)
 Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
*/
struct İl
{
 public int ilTrafikKodu;
 public string Adı;
 public string Bölge;
}

```

### §15.4.15 OfType

Elamanları değişik veri tiplerinde olan koleksiyonlarda filtreleme yapar.

```

System.Collections.ArrayList meyveler = new()
{
 "Mango",
 "Portakal",
 null,
 "İğde",
 3.0,
 "Armut",
 10,
 "Çağla"
};
IEnumerable<string> query1 = meyveler.OfType<string>();
Console.WriteLine("'string' veri tipli elemanlar:");
foreach (string meyve in query1)
 Console.WriteLine(meyve);
/*
'string' veri tipli elemanlar:
Mango
Portakal
İğde
Armut
Çağla
*/
IEnumerable<string> query2 = meyveler.OfType<string>().Where(fruit => fruit.Contains('ğ',
 ↳ StringComparison.CurrentCultureIgnoreCase));
Console.WriteLine("\n'ğ' harfli meyveler:");
foreach (string meyve in query2)

```

```

 Console.WriteLine(meyve);
 /*
 'ğ' harfli meyveler:
 İğde
 Çağla
 */

```

### §15.4.16 SelectMany

Bu yöntem bir `IEnumerable<IEnumerable<T>` öğesini bir `IEnumerable<T>` öğesi olarak değiştirir. Kaynak `IEnumerable`'da bulunan `IEnumerable` örnekleri içindeki tüm `T` öğeleri tek bir `IEnumerable`'da birleştirilir.

```

var kelimeler = new[] { "ali,ahmet,cemal", "damla,efe", "fatma" };
var virgilleayrilmiskelimeler = kelimeler.SelectMany(x => x.Split(','));
foreach (var item in virgilleayrilmiskelimeler)
 Console.Write($"{item} ");
//ali ahmet cemal damla efe fatma
Console.WriteLine();
Yemek[] yemekler = new[] {
 new Yemek() {
 Meyveler = new [] { new Meyve { Adı="Elma"}, new Meyve { Adı = "Armut" } },
 },
 new Yemek() {
 Meyveler = new [] { new Meyve { Adı = "Çilek"}, new Meyve { Adı = "Ahududu" } }
 }
};
var tümYemekler = yemekler.SelectMany(y => y.Meyveler.Select(m => m.Adı));
foreach (var item in tümYemekler)
 Console.Write($"{item} ");
//Elma Armut Çilek Ahududu
class Meyve
{
 public required string Adı { get; set; }
}
class Yemek
{
 public required Meyve[] Meyveler { get; set; }
}

```

### §15.4.17 Join

Birleştirmeler (**join**), ortak bir anahtar aracılığıyla veri tutan farklı listeleri veya tabloları birleştirmek için kullanılır. SQL'de olan türde birleştirmeler desteklenir: İç (**inner**), Sol (**left**), Sağ (**right**), Çapraz (**cross**) ve Tam Dış (**full outer**) birleştirmeler. Aşağıdaki iki tabloyu göz önüne alalım.

```

var sol = new List<string>() { "armut", "muz", "elma" }; // Sol (Left) tablo
var sağ = new List<string>() { "armut", "mango", "elma", "çilek" }; // Sağ (Right)

```

#### İç Birleştirme (Inner Join):

```

var innerjoin = from l in sol
 join s in sağ on l equals s
 select new { İLK = l, İKİNCİ = s };
var innerjoinfluent = sol.Join(sağ, l => l, s => s, (l, s) => new { İLK = l, İKİNCİ = s });
foreach (var item in innerjoinfluent)
 Console.Write($"{item.İLK}-{item.İKİNCİ} ");
//armut-armut elma-elma

```

#### Sol Bileştirme (Left Outer Join):

```

var leftOuterJoin = from l in sol
 join s in sağ on l equals s into tmp
 from t in tmp.DefaultIfEmpty()
 select new { İLK = l, İKİNCİ = t };
var leftOuterJoin2 = from l in sol
 from s in sağ.Where(x => x == l).DefaultIfEmpty()
 select new { İLK = l, İKİNCİ = s };

```

```
var leftOuterJoinFluent = sol.GroupJoin(sağ, l => l, s => s, (l, s) => new { İLK = l, İKİNCİ= s
 → }).SelectMany(temp => temp.İKİNCİ.DefaultIfEmpty(),(l, s) => new { İLK = l.İLK, İKİNCİ = s });
foreach (var item in leftOuterJoinFluent)
 Console.WriteLine($"{item.İLK}-{item.İKİNCİ} ");
//armut-armut muz-(null) elma-elma
```

### Sağ Bileştirme (Right Outer Join):

```
var rightOuterJoin = from s in sağ
 join l in sol on s equals l into temp
 from t in temp.DefaultIfEmpty()
 select new { İLK = t, İKİNCİ = s };
foreach (var item in rightOuterJoin)
 Console.WriteLine($"{item.İLK}-{item.İKİNCİ} ");
//armut-armut (null)-mango elma-elma (null)-çilek
```

### Çapraz Birleştirme (Cross Join):

```
var CrossJoin = from f in sol
 from s in sağ
 select new { İLK=f, İKİNCİ=s };
foreach (var item in CrossJoin)
 Console.WriteLine($"{item.İLK}-{item.İKİNCİ}");
/*
armut-armut
armut-mango
armut-elma
armut-çilek
muz-armut
muz-mango
muz-elma
muz-çilek
elma-armut
elma-mango
elma-elma
elma-çilek
*/
```

### Tam Dış Birleşme (Full Outer Join):

```
var leftOuterJoin = from l in sol
 join s in sağ on l equals s into tmp
 from t in tmp.DefaultIfEmpty()
 select new { İLK = l, İKİNCİ = t };
var rightOuterJoin = from s in sağ
 join l in sol on s equals l into temp
 from t in temp.DefaultIfEmpty()
 select new { İLK = t, İKİNCİ = s };
var fullOuterjoin = leftOuterJoin.Union(rightOuterJoin);
foreach (var item in fullOuterjoin)
 Console.WriteLine($"{item.İLK}-{item.İKİNCİ}");
/*
armut-armut
muz-(null)
elma-elma
(null)-mango
(null)-çilek
*/
```

## §15.4.18 Skip ve Take

**Skip** yöntemi, kaynak koleksiyonun başlangıcından itibaren belirli sayıda elemanı hariç tutan bir koleksiyon döndürür. Hariç tutulan eleman sayısı, argüman olarak verilen sayıdır. Koleksiyonda argümanda belirtilenden daha az öğe varsa, boş bir koleksiyon döndürülür.

**Take** yöntemi, kaynak koleksiyonun başlangıcından itibaren bir dizi eleman içeren bir koleksiyon döndü-

rür. Dahil edilen eleman sayısı, argüman olarak verilen sayıdır. Koleksiyonda argümanda belirtilenden daha az eleman varsa, döndürülen koleksiyon kaynak koleksiyonuyla aynı elemanları içerecektir.

```
var values = new[] { 5, 4, 3, 2, 1 };
var skipTwo = values.Skip(2); // { 3, 2, 1 }
var takeThree = values.Take(3); // { 5, 4, 3 }
var skipOneTakeTwo = values.Skip(1).Take(2); // { 4, 3 }
var takeZero = values.Take(0); // IEnumerable<int> fakat elemanı yok
```

### §15.4.19 Range ve Repeat

`Enumerable`'daki `Range` ve `Repeat` statik yöntemleri basit diziler oluşturmak için kullanılabilir.

```
var range = Enumerable.Range(1,100); // [1, 2, 3, ..., 98, 99, 100]
/* elemanları 1 den başlayıp 100'e kadar devam eden bir tamsayı dizisi oluşturur. */
var repeatedValues = Enumerable.Repeat("a", 3); // ["a","a","a"]
/* aynı elemandan birden fazla içeren dizi oluşturmak için kullanılır*/
```

### §15.4.20 Contains

Belirtilen bir `IEqualityComparer<T>` kullanarak bir dizinin belirtilen bir elemanı içerip içermediğini belirler.

```
List<int> sayılar = new List<int> { 1, 2, 3, 4, 5 };
var sonuç1 = sayılar.Contains(4); // true
var sonuç2 = sayılar.Contains(8); // false
List<int> başkaSayılar = new List<int> { 4, 5, 6, 7 };
var sonuç3 = başkaSayılar.Where(item => sayılar.Contains(item)); //true (4,5)
```

### §15.4.21 Zip

`Zip` yöntemi iki koleksiyon üzerinde etki eder. İki serideki her bir elemanı indise (konuma) göre eşleştirir. İki koleksiyondan gelen elemanları çiftler halinde işlemek için kullanırız. Seriler boyut olarak farklıysa, daha büyük serinin ekstra elemanları yok sayılır.

```
int[] sayılar = { 3, 4, 5, 7 };
string[] sözcükler = { "üç", "dört", "beş", "yedi", "göz ardı edilecek" };
IEnumerable<string> zip = sayılar.Zip(sözcükler, (n, s) => n + "=" + s);
foreach (var item in zip)
 Console.WriteLine(item);

/*
3=üç
4=dört
5=beş
7=yedi
*/
```

### §15.4.22 Aggregate

Bir dizi üzerinde bir biriktirici fonksiyonu uygular.

```
int[] sayıListesi = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
int toplam = sayıListesi.Aggregate((aratoplama, şimdiki) => aratoplama + şimdiki);
Console.WriteLine(toplam); // 55
string[] stringList = { "Merhaba", "C#", "!" };
string birleşikmetin = stringList.Aggregate((prev, current) => prev + " " + current);
Console.WriteLine(birleşikmetin); // Merhaba C# !
```

## 15.5 Let Anahtar Kelimesi

Bir LINQ ifadesinin içinde bir değişken tanımlamak için `let` anahtar sözcüğünü kullanabilirsiniz. Bu genellikle ara alt sorguların sonuçlarını depolamak için yapılır.

```
int[] sayılar = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
var ortalamaÜstüKareler = from sayı in sayılar
 let ortalama = sayılar.Average()
 let kare = Math.Pow(sayı, 2)
 where kare > ortalama
 select sayı;
Console.WriteLine("Ortalama: {0}", sayılar.Average());
```



```
foreach (int n in ortalamaÜstüKareler)
 Console.WriteLine("{0} sayısının karesi {1} Ortalamayı Geçti!", n, Math.Pow(n,2));
/*
Ortalama: 4,5
3 sayısının karesi 9 Ortalamayı Geçti!
4 sayısının karesi 16 Ortalamayı Geçti!
5 sayısının karesi 25 Ortalamayı Geçti!
6 sayısının karesi 36 Ortalamayı Geçti!
7 sayısının karesi 49 Ortalamayı Geçti!
8 sayısının karesi 64 Ortalamayı Geçti!
9 sayısının karesi 81 Ortalamayı Geçti!
*/
```

## 15.6 Düşün ve Çöz

- ◊ **Düşün:** LINQ'in 'ertelenmiş yürütme' (**deferred execution**) özelliği ne anlama gelir? Bir sorgu tanımlandığı anda değil, üzerinde foreach döngüsü kurulduğunda veya `.ToList()` çağrıldığında neden çalışır? Bu özelliğin hem avantajı hem de tuzağı nedir?
- ◊ **Düşün:** Sorgu söz dizimi (**query syntax**) ile akıcı söz dizimi (**method/fluent syntax**) arasındaki fark nedir? Hangi durumlarda biri diğerine üstün gelir? `GroupJoin` gibi karmaşık işlemler hangi söz diziminde daha okunabilir?
- ◊ **Düşün:** `IEnumerable<T>` üzerinde çalışan LINQ ile `IQueryable<T>` üzerinde çalışan LINQ arasındaki fark nedir? Veritabanı sorgulamada `IQueryable`'ın 'sorgu sunucu tarafında derlenir' avantajı performansı nasıl etkiler?
- ◊ **Düşün:** `Select` ile `SelectMany` arasındaki fark nedir? İç içe koleksiyonları düzleştirme (**flatten**) gereken bir senaryoda `SelectMany` nasıl kullanılır?
- ◊ **Düşün:** LINQ'in `GroupBy` yöntemi hangi senaryolarda kullanılır? Gruplandırılmış veriler üzerinde ekleme (**aggregation**) yapmanın yolları nelerdir? SQL'deki `GROUP BY` ile karşılaştırınız.
- ◊ **Düşün:** LINQ sorguları döngülerle (**imperative**) yazılmış koda kıyasla performans açısından nasıldır? Her zaman LINQ kullanmak doğru mudur? LINQ'in okunabilirlik kazancını performans maliyetiyle dengelemenin kriterleri nelerdir?
- ◊ **Düşün:** `let` anahtar kelimesi LINQ içinde ara değişken tanımlamak için kullanılır. Bu özellik olmadan yazılması güçleşen sorgu türlerine örnek veriniz. `let`'in derleyici tarafından nasıl dönüştürüldüğünü düşününüz.
- ◊ **Çöz:** Bir öğrenci listesi üzerinde şu LINQ sorgularını yazınız: a) Ortalaması 70'in üzerinde olanlar, b) Sınıfa göre gruplanmış öğrenci sayıları, c) Her sınıftaki en yüksek not, d) Adı 'A' ile başlayanlar alfabetik sıralı.
- ◊ **Çöz:** İki liste arasında LINQ Join, GroupJoin ve Zip kullanarak veriyi birleştirin: Ürünler ve Kategoriler listesi verilsin; her ürünün kategori adını içeren birleşik liste oluşturun.
- ◊ **Çöz:** Akıcı söz dizimi kullanarak: bir metin içindeki sözcükleri uzunluklarına göre gruplandırın, her grubun en uzun sözcüğünü bulun ve sonuçları uzunluğa göre azalan sırayla listeleyin.
- ◊ **Çöz:** LINQ'in `Any`, `All`, `Count`, `Sum`, `Min`, `Max` ve `Average` yöntemlerinin her birini kullanan; bir sınav sonuçları listesi üzerinde istatistiksel bir rapor üreten program yazınız.
- ◊ **Çöz:** İç içe listeler içeren bir veri yapısı (örneğin her öğretmenin birden fazla dersi) üzerinde `SelectMany` kullanarak tüm dersleri düzleştiren ve LINQ ile filtreleyen bir sorgu yazınız.

## 16. Bölüm

# Dosyalar, Akışlar ve Serileştirme

### 16.1 Akış Nedir?

C# dilinde akış (**stream**), verilerin bir kaynaktan (klavye, dosya, ağ) bir hedefe (ekran, dosya, bellek) doğru aktığı **soyut bir "kanal" veya "boru hattı"** olarak düşünülür. Akışlar, verinin fiziksel olarak **nerede saklandığıyla ilgilenmez**; sadece **verinin sıralı bir şekilde transfer edilmesini sağlar**. Bu nedenle **akış (stream)**, verilerin bir kaynaktan bir hedefe doğru **sıralı ve sürekli bir şekilde hareket etmesini** temsil eden soyut bir kavramdır.

Bir de kontrol akışı (**flow**) kavramı vardır. **Akış Kontrolü (control flow)**, kontrol işlemlerinin anlatıldığı 6.1 başlığında işlenmiştir. Kodun icra sırası anlamındadır.

### 16.2 Dosyalar ve Klasörler

Kısaca dosya (**file**), verilerin bir araya geldiği (**collection of data**) ikincil saklama ortamıdır (**secondary storage**). Bu ikincil ortam; Bir **bilgisayar belleği (memory)** olabileceği gibi, verilerin elektrikler kesildiğinde kaybolmayacağı **sabit disk (hard disk)**, ya da bir başka ortama/bilgisayara veri gönderen (**send**) veya alan (**receive**) modem ve benzeri bir **cihaz (device)** olabilir. **Akışları (stream)** temsil eden boru hattının iki ucuna bağlanabilen yapılardır.

Dosyalar söz konusu olduğunda bazı kavramları bilmemiz gerekir;

- ♦ **Yol (path)**: Dosyanın dosya sistemindeki konumu. Örneğin; `C:\Users\ILHANOZKAN\Okubeni.txt`. Burada yolun `C:\Users\ILHANOZKAN\` kısmı dosyanın içinde bulunduğu klasörü (**directory**) gösterir. Geri kalan `Okubeni.txt` ise dosyanın adı ve uzantısını temsil eder. **Tam metin (verbatim string)** kullanarak dosya yolundaki ters bölü karakterini iki kez yazmaktan kurtulabiliriz.
- ♦ **Sonuna Ekleme (append)**: Dosya mevcutsa, dosyanın sonuna veri ekler değilse yeni dosya üzerine yazar.
- ♦ **Metin (text)**: Yazılacak veya saklanacak verinin metin şeklinde, yani gözle okunabilir olduğunu ifade eder. Yani okunup yazılan veri, basit metin editörleriyle görülebilir.
- ♦ **İçerik (contents)**: Yazılacak metinlerin bir koleksiyonudur.
- ♦ **Kaynak (source)**: Kaynak dosyanın dosya sistemindeki konumu.
- ♦ **Hedef (destination)**: Hedef dosyanın dosya sistemindeki konumu

Veriler dosyalara aynı nehirde birbirinin peşi sıra giden tekneler gibi bayt-bayt gönderilir veya alınırlar. Bu nedenle dosyalara veri taşıyan ve veri getiren nesnelere **akış (stream)** adı verilir. Benzer şekilde nehirde hangi sırada veri gelirse o sırada verileri nehirde alırız. Burada nehir akış tanımındaki boru hattıdır. Bu boru hattına konulacak nesneler, art arda gönderilecek belli sırada baytlara çevrilir. Bu işleme **serileştirme (serialization)** adı verilir. Boru hattının diğer ucunda bu nesneyi oluşturacak bu baytları bir araya getirme işlemine ise **seriden çıkarma (deserialization)** adı verilir.

### 16.3 Veri Akışları

C# dilinde akış işlemleri `System.IO.Stream` kütüphanesinde altında toplanırlar. En temel anlamda akış, bir kaynaktaki (dosya, ağ bağlantısı, bellek) baytları okumak veya yazmak için kullanılan mekanizmadır. Verinin tamamını bir anda belleğe yüklemek yerine, küçük parçalar halinde işlenmesine olanak tanır.

1. **MemoryStream**: Verileri bellek (RAM) üzerinde bir akış gibi yönetmek için.
2. **FileStream**: Dosyalardan veri okumak/yazmak için.
3. **NetworkStream**: İnternet veya yerel ağ üzerinden veri transferi için.

Bir ağ akışı (**NetworkStream**) üzerinden veri okurken internet kablosu çekilirse ne olur? Akış "bozulur" ve program `IOException` fırlatır. Akışlar genellikle "bağlantı temelli" (**connection-oriented**) oldukları için, kaynak ile hedef arasındaki yolun açık kalması gerekir. Bu yüzden tüm akışlar, kullanıldıktan sonra mutlaka **Close()** veya **Dispose()** ile kapatılmalıdır. Ayrıca akışları açık tutmak maliyetlidir. Bu nedenle işimiz bitince hemen kapatılmalıdır.

## 16.4 Dosya İşlemleri

C#, `System.IO` isim alanı altında çok daha zengin ve yönetimi kolay bir araç seti sunar. Dosya işlemleri temelde ikiye ayrılır:

- ◊ Dosyanın kendisiyle ilgili işlemler (silme, taşıma, varlık kontrolü) ve
- ◊ Dosyanın içeriğiyle ilgili işlemler (okuma, yazma).

### §16.4.1 Dosya ve Klasör Bilgisi

C#, statik yöntemler yardımıyla dosya sistemine hızlıca müdahale etmenizi sağlar.

1. `File.Exists(yol)`: Dosya var mı?
2. `File.Delete(yol)`: Dosyayı sil.
3. `File.Create(yol)`: Dosya Oluştur.
4. `File.Move(yol)`: Dosyayı Taşı.
5. `Directory.CreateDirectory(yol)`: Yeni klasör oluştur.
6. `Directory.GetFiles(yol, filtre, aramaseçenekleri)`: Klasördeki dosyaları öğrenmek için kullanılır

#### Bir Klasördeki Dosyaları Öğrenme

Klasördeki tüm dosyaları öğrenmek için aşağıdaki kodu yazarız;

```
var FileSearchRes = Directory.GetFiles(Path, "*.*", SearchOption.AllDirectories);
```

Belirtilen dizindeki tüm dosyaları temsil eden bir `FileInfo` dizisi döndürür. `*.*` ifadesindeki yıldız karakteri **joker (wildcard)** olarak adlandırılır. Bu yüzden dosya adı ve uzantısında **yıldız karakteri kullanılmaz**. Adı ve uzantısı ne olursa olsun anlamında kullanılmıştı. Pdf uzantılı dosyaları seçmek için joker `*.pdf` şeklinde yazılır. Arama seçenekleri aşağıdaki tabloda verilmiştir;

| Seçenek                       | Kapsam             | Açıklama                                                                                                         |
|-------------------------------|--------------------|------------------------------------------------------------------------------------------------------------------|
| <code>TopDirectoryOnly</code> | Sadece Hedef Dizin | Arama işlemini sadece belirtilen klasörle sınırlar. Alt klasörlere bakmaz. (Varsayılan mod)                      |
| <code>AllDirectories</code>   | Tüm Alt Dizinler   | Belirtilen klasörden başlayarak hiyerarşideki tüm alt klasörleri özyinelemeli ( <b>recursive</b> ) olarak tarar. |

Tablo 16.1: SearchOption Seçenekleri ve Karşılaştırması

```
using System;
using System.IO;

string yol = @"C:\Projelerim";

// 1. Sadece ana klasördeki .jpg dosyaları
string[] ustDosyalar = Directory.GetFiles(yol, "*.jpg", SearchOption.TopDirectoryOnly);

// 2. Ana klasör VE tüm alt klasörlerdeki .jpg dosyaları
string[] tumDosyalar = Directory.GetFiles(yol, "*.jpg", SearchOption.AllDirectories);

Console.WriteLine($"Ana Klasörde: {ustDosyalar.Length} dosya bulundu.");
Console.WriteLine($"Tüm Alt Klasörlerde Toplam: {tumDosyalar.Length} dosya bulundu.");

//3. Ana klasör VE tüm alt klasörlerdeki .pdf dosyaları
string[] FileSearchRes = Directory.GetFiles(yol, "*.pdf", SearchOption.AllDirectories);
foreach (var item in FileSearchRes)
 Console.WriteLine(item);
```

`SearchOption.AllDirectories` seçeneğini kullanırken, tarama yapılan alt klasörlerden herhangi birine erişim izniniz yoksa (örneğin sistem klasörleri), yöntem bir `UnauthorizedAccessException` fırlatır ve tüm arama yarıda kesilir. C#8.0+ ve sonrasında, bu sorunu aşmak için `EnumerationOptions` sınıfı kullanılabilir. Bu sınıf, erişilemeyen klasörleri "atla" (`IgnoreInaccessible`) demenize olanak tanır.

```
string yol = @"C:\Projelerim";
var options = new EnumerationOptions { IgnoreInaccessible = true, RecurseSubdirectories = true };
var dosyalar = Directory.GetFiles(yol, "*.*", options);
```

#### Mevcut Bir Dosyayı Taşıma

`File` statik sınıfının `Move()` yöntemini kullanarak dosyaları taşıyabiliriz. Yalnızca dosyanın klasörünü değiştirir. Bu işlem dosya boyutuna göre zaman almaz. Ayrıca aynı aldı dosya hedefte var ise kopyalama

olmaz.

```
//File.Move("kaynakYol", @"hedefYol");
File.Move(@"C:\Users\ILHANOZKAN\YeniDosya2.txt", @"D:\YeniDosya.kopya.txt");
```

### Mevcut Bir Dosyayı Silme

Yol belirterek dosyayı `Delete()` yöntemiyle silebiliriz. Bu yöntem, dosya mevcut değilse istisna oluşturmaz, ancak belirtilen yol geçersizse veya çağıran gerekli izinlere sahip değilse istisna oluşturur. `Delete()` çağrılarını her zaman `try-catch` bloğunun içine koymalı ve tüm beklenen istisnaları işlemelisiniz. Birden fazla program aynı dosyayı silmeye kalkışabilir ve bu durumda yarış durumu (`race condition`) ortaya çıkar. Bunu önlemek için silme mantığını lock ifadesinin içine koymalıyız.

```
string path = @"D:\YeniDosya.kopya.txt";
File.Delete(path);
```

### §16.4.2 Yol Yönetimi

Farklı işletim sistemlerinde farklı klasör ayracı; Windows işletim sisteminde ters bölü (\), Unix ve Linux işletim sisteminde (/) kullanır. Yolların bozulmaması için `Path` sınıfının `Combine()` yöntemini kullanmak profesyonel bir zorunluluktur.

```
string klasor = "Veriler";
string dosya = "ayarlar.json";
// İşletim sistemine göre en doğru yolu birleştirir:
string tamYol = Path.Combine(klasor, dosya); //Linux için "Veriler/ayarlar.json" üretirken
→ Windows için "Veriler\ayarlar.json" üretir.
```

### §16.4.3 Dosya Oluşturma

`File` statik sınıfının `Create()` yöntemini kullanarak dosyalar oluşturabiliriz. Bu yöntem, verilen yolda dosyayı oluşturur, aynı zamanda dosyayı açar ve bize dosyanın `FileStream` nesnesini verir. İşimiz bittikten sonra dosyayı `Close()` yöntemi ile kapattığımızdan emin olmalıyız.

```
FileStream akış = File.Create("Metin.txt"); // Programın çalıştığı klasörde dosya oluşturur.
// Dosya İşlemleri...
akış.Close(); //Dosya Create ile açıldığından mutlaka kapatılmalıdır.
```

Buna eşdeğer olarak `Open()` yöntemini kullanabiliriz.

```
FileStream akış = File.Open("C:\\Users\\ILHANOZKAN\\Metin.txt", FileMode.Create);
// Programın çalıştığı dizine dosya oluşturur.
akış.Close();
```

Bu yöntemde ikinci parametre olarak dosya açma modunu belirten `FileMode` kullanabilir.

| Mod                                      | Dosya Varsa           | Dosya Yoksa                | Açıklama                                                                                           |
|------------------------------------------|-----------------------|----------------------------|----------------------------------------------------------------------------------------------------|
| <code>CreateNew</code>                   | Hata Fırlatır         | Oluşturur                  | Sadece yeni bir dosya oluşturmak için kullanılır. Dosya zaten varsa risk almaz.                    |
| <code>Create</code><br><code>Open</code> | Üzerine Yazar<br>Açar | Oluşturur<br>Hata Fırlatır | Dosya varsa içeriğini siler ve sıfırdan başlar. Var olan bir dosyayı okumak veya yazmak için açar. |
| <code>OpenOrCreate</code>                | Açar                  | Oluşturur                  | En güvenli yoldur. Varsa açar, yoksa yeni bir tane oluşturur.                                      |
| <code>Truncate</code>                    | İçini Boşaltır        | Hata Fırlatır              | Dosyayı açar ama içindeki tüm veriyi anında siler (0 byte yapar).                                  |
| <code>Append</code>                      | Sonuna Ekler          | Oluşturur                  | Mevcut veriye dokunmaz, yazmaya dosyanın en sonundan başlar.                                       |

Tablo 16.2: FileMode Seçenekleri ve Davranışları

```
string yol = "log.txt";
// Senaryo A: Her seferinde eskiyi sil, yeni başla
using (FileStream fs = new FileStream(yol, FileMode.Create)) {
 // ...
}
// Senaryo B: Eski verilere dokunma, altına ekle (Loglama için ideal)
using (FileStream fs = new FileStream(yol, FileMode.Append)) {
 // ...
}
```

```
}
```

`using` anahtar kelimesi, **açılan bir kaynağın kapatılmasını otomatik olarak yapar**. Böylece `Close()` yazmayı unutmaktan korkmayız.

```
using (var akış=File.Create(@"C:\Users\ILHANOZKAN\Metin.txt"))
{
 //akış kullanılmıyorsa blok dışında otomatik olarak kapatılır.
}
```

Dosya oluşturmanın bir diğer yolu da `FileStream` sınıfını kullanmaktır.

```
FileStream akış = new FileStream("Metin.txt", FileMode.OpenOrCreate, FileAccess.ReadWrite,
 ↪ FileShare.None);
//...
fs.Close();
```

Bu yöntemin fazladan `FileAccess` ve `FileShare` adlı iki parametresi daha vardır. `FileMode` genellikle bu iki iki yardımcı `enum` ile birlikte kullanılır.

1. `FileAccess`: Sizin dosyaya ne yapacağınız (`Read`, `Write`, `ReadWrite`).
2. `FileShare`: Siz dosyayı kullanırken başkalarının (başka programların) ne yapabileceği (`None`, `Read`, `Write`).

Örnek olarak dosyayı sadece okumak için aç ve ben okurken kimse değiştiremesin istiyorsak aşağıdaki gibi dosyayı açarız;

```
FileStream fs = new FileStream("data.bin", FileMode.Open, FileAccess.Read, FileShare.Read);
//...
fs.Close();
```

## 16.5 Metin Dosyalarını Toptan Okuma ve Yazma

Bir dosyanın tüm içeriğini bir metin (`string`) değişkenine koymak için `System.IO.File.ReadAllText()` ve bir metin değişkeni yazmak için `System.IO.File.WriteAllText()` yöntemi kullanılabilir;

```
System.IO.File.WriteAllText(@"Metin.txt", "Bu bir deneme metnidir.\nBu metin ikinci satırdır.");
string text = System.IO.File.ReadAllText(@"Metin.txt");
string[] lines = text.Split(new[] { '\n' }, StringSplitOptions.None);
int satırSayısı = lines.Length;
Console.WriteLine($"Toplam {satırSayısı} satır var.");
```

Tüm satırları bir metin dizisine (`string[]`) okumak için `System.IO.File.ReadAllLines()` ve yazmak için `System.IO.File.WriteAllLines()` yöntemi kullanılabilir;

```
string[] lines = System.IO.File.ReadAllLines(@"\Metin.txt");
foreach (string line in lines)
{
 Console.WriteLine(line);
}
```

### §16.5.1 Büyük Metin Dosyaları

Belleği çok zorlayacak büyük metin dosyalarıyla çalışırken, bir dosyadaki tüm satırları `IEnumerable<string>` jenerik ara yüzüne okumak için `System.IO.File.ReadLines()` yöntemi kullanabiliriz;

```
using System.Text; //Encoding için gerekli
IEnumerable<string> AllLines = File.ReadLines(@"\Metin.txt", Encoding.Default);
foreach (string line in AllLines)
{
 Console.WriteLine(line);
}
```

Bu yöntemle farklı yerel kodlamaya (`encoding`) sahip metin dosyasının okuma imkanına da sahip oluruz. Ayrıca `File.WriteLine()` yöntemi yazmak için de kullanılabilir.

## 16.6 Akışlar ile Dosya Yazma ve Okuma

Dosya içeriğiyle uğraşırken en güvenli ve yaygın yol `StreamWriter` ve `StreamReader` sınıflarını kullanmaktır. Bu sınıflar, karmaşık bayt işlemlerini bizim yerimize yöneterek doğrudan **metinlerle** (`string`) çalışmamıza

olarak tanır.

Akışlar verinin nerede saklandığıyla ilgilenmez. Bu teorik olarak doğru olsa da, `FileStream` alt sınıfının doğrudan saklama ortamına bağlı olduğunu, ancak `Stream` ebeveyn sınıfı bu soyutlamayı sağlar.

### §16.6.1 Dosyaya Yazma

```
string yol = "metin.txt";
using (StreamWriter sw = new StreamWriter(yol)) // 'using' bloğu, işlem bitince dosyayı otomatik
 ↪ kapatır.
{
 sw.WriteLine("C# Programlama Kitabı");
 sw.WriteLine("Bölüm: Dosya İşlemleri");
 sw.Write("Tarih: ");
 sw.WriteLine(DateTime.Now);
} // sw.Dispose() burada otomatik çağrılır.
```

### §16.6.2 Dosyadan Okuma

```
string yol = "metin.txt";
if (File.Exists(yol))
{
 using (StreamReader sr = new StreamReader(yol))
 {
 string satır;
 while ((satır = sr.ReadLine()) != null) // akıştan bir satır oku
 {
 Console.WriteLine($"Okunan: {satır}");
 }
 }
}
```

### §16.6.3 İkili Dosyalar

Metin dosyalarından farklı olarak, verileri bellekte tutulduğu gibi ikili olarak dosyalara yazabilir ve okuyabiliriz. Bu şekilde oluşan dosyalar gözle okunamazlar. Çünkü işlemcinin doğal olarak anladığı ikili sayı sisteminde veriler dosyalara yazılır ve okunur.

```
using System.Text;
string yol = "ikili.bin";

//İKİLİ OLARAK DOSYAYA YAZMA:
using (var stream = File.Open(yol, FileMode.Create))
{
 using (var writer = new BinaryWriter(stream, Encoding.UTF8, false))
 {
 writer.Write(1.250F);
 writer.Write(@"c:\Temp");
 writer.Write("İlhan Özkan");
 writer.Write(10);
 writer.Write(true);
 }
}

//İKİLİ OLARAK DOSYADAN OKUMA:
float oran;
string geçiciKlasör;
string geçiciKullanıcıAdı;
int adet;
bool doluMu;
if (File.Exists(yol))
{
 using (var stream = File.Open(yol, FileMode.Open))
 {
 using (var reader = new BinaryReader(stream, Encoding.UTF8, false))
 {
```



```

 oran = reader.ReadSingle();
 geçiciKlasör = reader.ReadString();
 geçiciKullanıcıAdı = reader.ReadString();
 adet = reader.ReadInt32();
 doluMu = reader.ReadBoolean();
 }
}
//Okunanları Konsola Yazma
Console.WriteLine($"Oran: {oran}");
Console.WriteLine($"Geçici Klasör: {geçiciKlasör}");
Console.WriteLine($"Kullanıcı Adı: {geçiciKullanıcıAdı}");
Console.WriteLine($"Adet: {adet}");
Console.WriteLine($"Dolu Mu?: {doluMu}");
}

```

Oluşan **ikili.bin** dosyasına;

1. İlk önce kayan noktalı sayı (**float**) değeri,
2. Sonra iki adet metin (**string**)
3. Ardından bir tamsayı (**int**) ve
4. Son olarak **bool** değer ikili olarak yazılmıştır.

Bellekte saklandığı şekliyle dosyaya kaydedildiğinden içeriği gözle okunamaz. İkili dosyaları açan görüntüleyiciler ile açılıp görülebilir. Bir ikili dosyayı onaltılık rakamlara çevirip metin olarak görüntülemek için Windows ortamında `certutil -encodehex ikili.bin ikili.txt` komutu verilebilir. Benzer işlem Linux ortamında `hexdump -C ikili.bin | less >> ikili.txt` komutuyla yapılabilir. Böylece onaltılık tabanda sayılar ile dosyanın her bir baytı **ikili.txt** dosyasına yazılacaktır. Artık dönüştürülen dosyayı istediğimiz metin düzenleyicisinde açıp okuyabiliriz.

```

0000 00 00 a0 3f 07 63 3a 5c 54 65 6d 70 0d c4 b0 6c ...?.c:\Temp...l
0010 68 61 6e 20 c3 96 7a 6b 61 6e 0a 00 00 00 01 han ..zkan....

```

## 16.7 Bellek Akışları

Dosya işlemleri veriyi diskte saklarken, **MemoryStream** veriyi RAM (Bellek) üzerinde bir akış (**stream**) olarak yönetmenizi sağlar. **MemoryStream**, verileri bayt dizileri (**byte[]**) olarak bellekte tutan bir **Stream** türevidir. Fiziksel bir dosya ile çalışıyormuş gibi kod yazmanıza rağmen, tüm işlemler RAM üzerinde gerçekleştiği için inanılmaz hızlıdır.

Akışlar verinin nerede saklandığıyla ilgilenmez. Bu teorik olarak doğru olsa da, **FileStream** gibi **MemoryStream** alt sınıflarının doğrudan saklama ortamına bağlı olduğunu, ancak **Stream** ebeveyn sınıfı bu soyutlamayı sağlar.

Neden MemoryStream Kullanılır?

- ♦ **Geçici Depolama:** Veriyi diske kaydedip silmekle uğraşmadan bellekte manipüle etmek için.
- ♦ **Veri Dönüşümleri:** Bir resmi veya nesneyi **serileştirme** (**serialization**) yapıp ağ üzerinden göndermeden önce paketlemek için.
- ♦ **Şifreleme:** Veriyi şifrelerken veya çözerken ara katman olarak.

Aşağıda bir metin (**string**) ifadesinin nasıl belleğe yazıldığını ve oradan nasıl geri okunduğunu gösterir:

```

using System.Text; //Encoding sınıfı için

string mesaj = "C# Kitabı MemoryStream Örneği";
byte[] veriPaketi = Encoding.UTF8.GetBytes(mesaj);

// Bellekte bir akış oluşturuyoruz
using (MemoryStream ms = new MemoryStream())
{
 // 1. Veriyi belleğe yaz
 ms.Write(veriPaketi, 0, veriPaketi.Length);

 // DİKKAT: Yazma işleminden sonra imleç sona gider.
 // Okumak için imleci başa sarmalıyız!
 ms.Position = 0;

 // 2. Bellekten oku
 byte[] okunanVeri = new byte[ms.Length];
}

```



```
ms.Read(okunanVeri, 0, okunanVeri.Length);

string sonuc = Encoding.UTF8.GetString(okunanVeri);
Console.WriteLine("Bellekten Okunan: " + sonuc);
}
```

## 16.8 Serileştirme ve Seriden Çıkarma

**Serileştirme (serialization)**, bir nesnenin durumunu (verilerini) saklamak veya bir ağ üzerinden iletmek amacıyla bayt dizisine, JSON'a veya XML formatına dönüştürülmesi işlemidir. **Seriden çıkarma (deserialization)** ise bu serileştirilmiş verinin tekrar canlı bir nesneye dönüştürülmesidir. Serileştirilen veri, bir akış (**stream**) üzerinden gönderilir veya alınır.

### §16.8.1 JSON Serileştirme

JSON (**Javascript Object Notation**), günümüzde en çok kullanılan yöntemdir. Modern .NET projelerinde `System.Text.Json` kütüphanesi standart olarak gelir.

```
using System;
using System.Text.Json;

public class Kitap
{
 public string Ad { get; set; }
 public string Yazar { get; set; }
 public int Yil { get; set; }
}

public class Program
{
 public static void Main()
 {
 var kitap = new Kitap { Ad = "C# Rehberi", Yazar = "İlhan", Yil = 2026 };
 // 1. Serileştirme (Nesne -> JSON String)
 string jsonString = JsonSerializer.Serialize(kitap);
 Console.WriteLine($"JSON Çıktısı: {jsonString}"); //JSON Çıktısı: {"Ad":"C#
 ↪ Rehberi","Yazar":"\u0130lhan","Yil":2026}
 // 2. Ters Serileştirme (JSON String -> Nesne)
 Kitap cozulenKitap = JsonSerializer.Deserialize<Kitap>(jsonString);
 Console.WriteLine($"Çözülen Kitap: {cozulenKitap.Ad} by {cozulenKitap.Yazar}"); //Çözülen
 ↪ Kitap: C# Rehberi by İlhan
 }
}
```

`System.Text.Json` kütüphanesinin varsayılan olarak büyük-küçük harf duyarlı (**case-sensitive**) olduğunu, bu nedenle serileştirme seçeneklerinde (`JsonSerializerOptions`) `PropertyNameCaseInsensitive = true` ayarının sıklıkla yapılması gerekir.

### §16.8.2 XML Serileştirme

XML (**eXtensible Markup Language**), daha çok konfigürasyon dosyalarında ve eski sistemlerle entegrasyonda tercih edilir.

```
using System.Xml.Serialization;
using System.IO;

public class Kitap
{
 public string Ad { get; set; }
 public string Yazar { get; set; }
 public int Yil { get; set; }
}

public class Program
{
 public static void Main()
```

```

{
 var kitap = new Kitap { Ad = "C# Rehberi", Yazar = "İlhan", Yil = 2026 };
 // XML serileştirme için boş bir yapıcı (constructor) gereklidir.
 XmlSerializer serializer = new XmlSerializer(typeof(Kitap));
 using (StringWriter writer = new StringWriter())
 {
 serializer.Serialize(writer, kitap);
 string xmlOutput = writer.ToString();
 Console.WriteLine(xmlOutput);
 }
}
}
/*Program Çıktısı:
<?xml version="1.0" encoding="utf-16"?>
<Kitap xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:xsd="http://www.w3.org/2001/XMLSchema">
 <Ad>C# Rehberi</Ad>
 <Yazar>İlhan</Yazar>
 <Yil>2026</Yil>
</Kitap>
*/

```

### §16.8.3 Binary (İkili) Serileştirme

Nesneyi doğrudan makine diline yakın bir bayt dizisine dönüştürür. Hızlıdır ve az yer kaplar ancak okunabilir değildir. C#11+ sürümlerinde güvenlik nedeniyle **BinaryFormatter** yerine **Protocol Buffers** (kısaca **protobuf**) veya **MessagePack** gibi kütüphaneler önerilir.

### §16.8.4 Serileştirme Kontrol Mekanizmaları

Serileştirme süreci aşağıdaki özniteliklerle (**attributes**) manipüle edebilir;

- ◊ **[JsonPropertyName("book\_name")]**: JSON içindeki anahtar ismini değiştirir.
- ◊ **[JsonIgnore]**: Belirli bir alanın serileştirilmesini engeller (örneğin şifreler).
- ◊ **[Serializable]**: Bir sınıfın ikili (**binary**) serileştirmeye uygun olduğunu işaretler.

### §16.8.5 Özet

| Format   | Okunabilirlik | Boyut     | Kullanım Alanı                      |
|----------|---------------|-----------|-------------------------------------|
| JSON     | Çok Yüksek    | Orta      | Web API, NoSQL, Mobil.              |
| XML      | Yüksek        | Büyük     | Konfigürasyon, SOAP servisler.      |
| Binary   | Yok           | Çok Küçük | Yüksek performanslı veri transferi. |
| Protobuf | Yok           | En Küçük  | Mikroservisler (gRPC).              |

Tablo 16.3: Serileştirme Formatlarının Teknik Analizi

## 16.9 Düşün ve Çöz

- ◊ **Düşün:** Akış (**stream**) soyutlamasının temel değeri nedir? Ağ üzerinden veri okumak ile dosyadan veri okumak aynı API ile yapılabiliyorsa programcıya ne kazandırır? Bu soyutlama olmadan aynı işi yapmak nasıl daha karmaşık olurdu?
- ◊ **Düşün:** **FileStream**, **StreamReader** ve **File.ReadAllText()** arasındaki fark nedir? Büyük dosyalarda satır satır okuma (**StreamReader**) yerine tüm dosyayı belleğe yüklemenin (**File.ReadAllText**) riskleri nelerdir?
- ◊ **Düşün:** Akışların mutlaka kapatılması gerektiğini biliyoruz. **using** bloğu ile **try-finally** bloğunun davranışı hangi senaryoda farklılaşır? C#8 ile gelen **using** bildirimi (**using declaration**) bu sorunu nasıl sadeleştirir?
- ◊ **Düşün:** Serileştirme (**serialization**) ne anlama gelir? JSON, XML ve ikili (**binary**) serileştirmenin avantaj ve dezavantajlarını karşılaştırınız. Hangi senaryoda hangisi tercih edilir?
- ◊ **Düşün:** **System.Text.Json** ve **Newtonsoft.Json** arasındaki temel farklar nelerdir? .NET 5+ ile birlikte **System.Text.Json**'ın varsayılan kütüphane haline gelmesinin nedenleri nelerdir?
- ◊ **Düşün:** Dosya yollarını (**path**) hardcode yazmak neden kötü bir pratiktir? **Path.Combine()**, **Environment.GetFolderPath()** ve **AppDomain.CurrentDomain.BaseDirectory** yöntemlerinin doğru kullanımı çapraz platform uyumluluğunu nasıl sağlar?
- ◊ **Düşün:** Dosya izleme (**FileSystemWatcher**) ne gibi kullanım senaryolarına hizmet eder? Bir dizini

izlerken ortaya çıkabilecek yarış koşulları (**race condition**) nasıl ele alınır?

- ◇ **Çöz:** Bir öğrenci listesini JSON formatında dosyaya kaydeden ve dosyadan geri okuyan program yazınız. `System.Text.Json` kullanınız; kayıt bulunamazsa boş liste dönsün, varsa listeye yeni kayıt eklensin.
- ◇ **Çöz:** Büyük bir metin dosyasını satır satır okuyarak her satırdaki sözcük sayısını, toplam satır sayısını ve en uzun satırı bulan; sonuçları ayrı bir rapor dosyasına yazan program yazınız. `StreamReader` kullanınız.
- ◇ **Çöz:** Bir dizin altındaki tüm `.txt` dosyalarını özyinelemeli olarak tarayan, her dosyanın adını, boyutunu ve son değiştirilme tarihini listeleyen; sonuçları büyükten küçüğe boyuta göre sıralayan program yazınız.
- ◇ **Çöz:** Bir yapılandırma (**config**) sistemi tasarlayınız: anahtar-değer çiftlerini JSON dosyasına kaydeden, okuyan ve güncelleyen; dosya yoksa varsayılan değerlerle oluşturan bir `ConfigManager` sınıfı yazınız.
- ◇ **Çöz:** `FileSystemWatcher` kullanarak belirli bir klasörü izleyen; yeni dosya eklendiğinde, silindiğinde veya değiştirildiğinde konsola zaman damgalı bildirim yazan bir program tasarlayınız.

## 17. Bölüm

# Veri Tabanı İşlemleri

### 17.1 Veri Tabanı Nedir?

**Veri tabanları** (**database**), tuttukları veriler hakkında bilgi sahibi olan programlardır. Veri tabanları, yazdığımız uygulamadan bağımsız olarak verilerimizi sistematik olarak saklar, erişilebilir kılar ve yönetmemizi sağlar. Veri tabanı kullanmak bize veri tutarlılığı, veri paylaşımı, veri güvenliği ve veri analitiği için bir altyapı sağlar. Veri tabanı;

- ◊ Kullanıcılardan gelen sorgu taleplerini analiz eder,
- ◊ Analiz sonrasında varsa mevcut veri üzerinde değişiklik yapar veya istenen veriyi belirler
- ◊ Sorgu sonucunu kullanıcının erişebileceği formda sunar.

Veri tabanında sorgular, **SQL** (**Structured Query Language**) dili kullanan **ilişkisel veri tabanında** veya **NoSQL** yaklaşımlarıyla **ilişkisel olmayan veri tabanlarında** yapılmaktadır. NoSQL bir dilden ziyade bir veri modelleme yaklaşımıdır (Document, Key-Value, Graph benzeri). Veri tabanları, **veri tabanı yönetim sistemleri** (VTYS) aracılığıyla güvenli ve ölçeklenebilir şekilde yönetilebilmektedir. Veri tabanı yönetim sistemi verinin fiziksel veya mantıksal alanda bulunduğunu belirler ve gelen sorguları bu kapsamda ayrıştırmaktadır. Sorguları hızlandırmak için indeksler oluşturulup veriye hızlıca erişilmesi sağlanmaktadır.

Bir uygulama geliştirmeye başladığınızda yapacağınız ilk tercihlerden biri **SQL** veya **NoSQL** veritabanı kullanıp kullanmayacağınızdır. Verileri düzenlemek çok zor bir iştir ve düzenlediğimizde verilerimizi çeşitli türlere göre kategorize ederiz;

1. **PostgreSQL**, **MySQL**, **Microsoft SQL Server**, **Oracle Database** veya **SQLite** gibi geleneksel bir veri tabanları şema (**schema**) kullanır ve bu veri tabanlarında **saklanmış verileri düzenlemek** daha kolaydır.
2. **MongoDB**, **Redis**, **CouchDB**, **Cassandra**, **Elasticsearch**, **Couhbase** ve **LiteDB** gibi NoSQL veri tabanları SQL veri tabanları kadar **veri tutarlılığı sağlamaz ve saklanmış verileri düzenlemek çok daha zordur**. Daha çok iz kayıtları (**log**), belge ve resim saklamak için kullanılırlar.

SQL ve NoSQL arasındaki temel fark, verilerin nasıl depolandığı ve nasıl dışarıya aktarıldığıdır.

Bir **veri tabanı projesini yazabilmeniz için ilk önce** ilgili veri tabanına karar verip projenizin çalıştığı bilgisayara ya da projenizle aynı ağda ve **erişilebilir olan bir bilgisayara kurmanız gerekir**.

**Erişilebilir bir veri tabanınız varsa o veri tabanına ilişkin bileşenleri projemize dahil ederek** veri tabanına bağlanıp gerekli veri işlemlerini gerçekleştirebiliriz.

Visual Studio ile **3.11.2** başlıındaki gibi oluşturulmuş projelere veritabanlarını kullanacak bileşenlerini dahil etmek için **Tools→NuGet Package Manager→Package Manager Console** üzerinden komut vererek kullanacağımız veritabanına ilişkin bileşenleri projemize dahil ederiz;

```
PM>Install-Package Microsoft.Data.Sqlite
PM>Install-Package Microsoft.Data.SqlClient
PM>Install-Package LiteDb
```

.NET Core CLI ile **3.11.1** başlıındaki gibi oluşturulmuş projelere veritabanlarını kullanacak bileşenlerini dahil etmek için proje klasöründe aşağıdaki komutlar verilir;

```
$dotnet add package Microsoft.Data.Sqlite
$dotnet add package Microsoft.Data.SqlClient
$dotnet add package LiteDb
```

Veri tabanını kullanacak her projeye **ilgili paket dahil edilmelidir!**

### 17.2 Bağlantı Metni

Projemizde kullanacağımız veri tabanları uygulamamızdan ayrı bir programdır. Bu veri tabanlarına bağlanmamızı sağlayan metinleri (**string**) değişkenleri olarak tanımlarız. Bu değişkenlere **bağlantı metni** (**connection string**) adı veririz. Bu değişken genellikle veri tabanı yönetim sistemine iletilen kullanıcı adı, şifre ve

şema bilgileri içerir.

```
string sqliteConnectionString="Data Source=hello.db"; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde oluşur.
var sqliteConnection = new SqliteConnection(sqliteConnectionString);
string nosqlConnectionString=@"MyData.db"; //Çalıştırılabilir .NET uygulamasının olduğu klasörde
→ oluşur.
var nosqlConnection = new LiteDatabase(nosqlConnectionString);
```

Küçük ve yerel bilgisayarınızda çalışabilecek veri tabanlarının bağlantı metinleri kısa ve öz olur. Ancak kurumsal ve ölçeklenebilir Veri tabanlarının bağlantı dizgisi daha karmaşık olabilir. Bu gibi bağlantı metinlerini oluşturmak için `DbConnectionStringBuilder`, `SqlConnectionStringBuilder` gibi yardımcı sınıflar kullanılır;

```
DbConnectionStringBuilder builder1 = new DbConnectionStringBuilder();
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "MyDb.mdb");//Access DataBase
builder1.ConnectionString = $"Data Source={db}";
builder1.Add("Provider", "Microsoft.Jet.Oledb.4.0");
builder1.Add("Jet OLEDB:Database Locking Mode", 1);
OleDbConnection oledbConnect = new OleDbConnection(builder1.ConnectionString);

var builder2 = new SqlConnectionStringBuilder {
 DataSource = "(local)",
 UserID = "sa",
 Password = "123456*",
 InitialCatalog = "CSharpDB"
};
var SqlConnection = new SqlConnection(builder2.ConnectionString);
var SqlConnection2 = new SqlConnection("server=(local);initial catalog=CSharpDB");
```

Veri kaynakları genellikle hem mali hem de tükettikleri kaynak açısından pahalı sistemlerdir. **Pahalı olan bu kaynaklar başka yazılımlar ile ortak kullanılırlar. Bu nedenle veri kaynaklarına olan bağlantılarımızı sınırlı tutmalıyız.** Bu tür kaynakları etkin kullanmak için `using` saklı kelimesi kullanılır. `using` kelimesi veri kaynağı gibi değerli kaynaklara ilişkin işlem yapılırken bir hata meydana geldiğinde, bağlantıya ilişkin tüm nesneleri `Dispose()` yöntemi ile bellekten siler yani `Dispose()` yönteminin çağrılmasını garanti eder.

## 17.3 İlişkisel Veri Tabanı İşlemleri

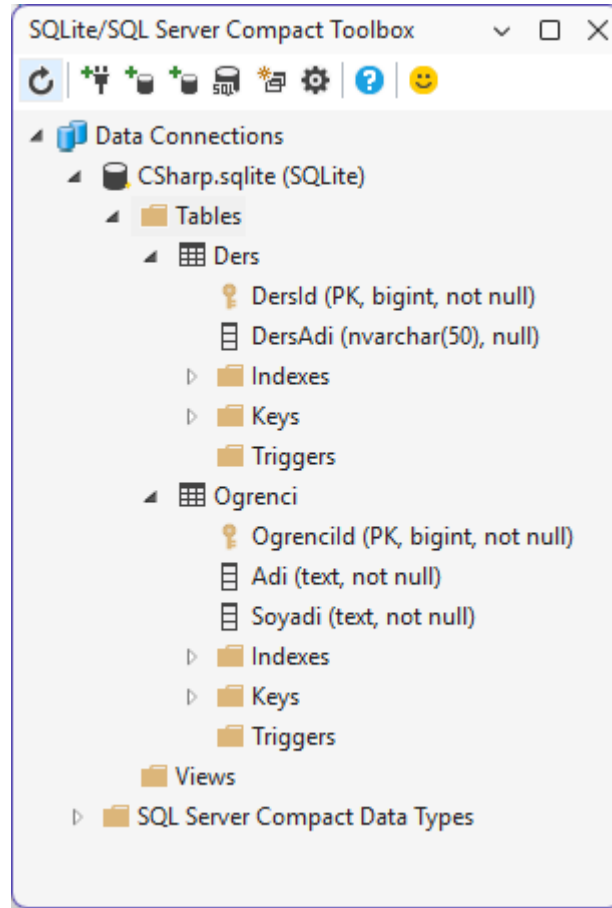
Bir veri tabanını kullanabilmek için ilk önce bir bağlantı (**connection**) kurmak gerekir. Sonrasında veri tabanı üzerinde işlem yapacak SQL komutu (**command**) nesnesi hazırlanır ve hazırlanan komut icra (**execute**) edilir. SQL komutuna ilişkin `CommandText` özelliğine noktalı virgül karakteriyle ayırarak veritabanı üzerinde birden fazla sorgu çalıştırabiliriz. Bu bölümde verilen örneklerde SQL sorgusu bildiğiniz kabul edilmiştir.

### §17.3.1 Veri Tabanında Tablolar Oluşturma

Aşağıdaki örnek kod ile SQLite veri tabanında **Oğrenci** ve **Ders** adında iki tablo oluşturulmuştur. Oluşturulan tablolara 6 adet öğrenci ve 2 adet ders kaydı atılmıştır.

```
using Microsoft.Data.Sqlite; //Nuget Paketini Kur!

var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "CSharp.sqlite");
var connectionString = $"Data Source={db}";
using (var connection = new SqliteConnection(connectionString))
{
 connection.Open();
 var command = connection.CreateCommand();
 command.CommandText = @"
 DROP TABLE IF EXISTS Oğrenci;
 CREATE TABLE Oğrenci (
 OğrenciId INTEGER PRIMARY KEY AUTOINCREMENT,
```



Şekil 17.1

```

 Adi text NOT NULL,
 Soyadi text NOT NULL
);
 DROP TABLE IF EXISTS Ders;
 CREATE TABLE Ders(
 DersId INTEGER PRIMARY KEY AUTOINCREMENT,
 DersAdi NVARCHAR(50)
);
";
int rowsAffected = command.ExecuteNonQuery(); //Veri getirmeyecek bir sorgu!
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi."); //0 kadar kayıt etkilendi.
command.CommandText =@"
 INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ali', 'Veli');
 INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ayşe', 'Fatma');
 INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Mehmet', 'Can');
 INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Zeynep', 'Yılmaz');
 INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ahmet', 'Demir');
 INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Fatma', 'Kaya');
 INSERT INTO Ders (DersAdi) VALUES ('Matematik');
 INSERT INTO Ders (DersAdi) VALUES ('Fizik');
";
rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi."); //8 kadar kayıt etkilendi.
connection.Close();
}

```

Visual Studio açık iken **Extensions** → **Manage Extensions...** tıklanarak arama ekranında *SQLite/SQL Compact Toolbox* eklentisini arayıp kurabilirsiniz. Kurulum sonrasında SQLite veri tabanını Şekil 17.1'deki gibi açıp sorgulayabiliriz <sup>1</sup>.

<sup>1</sup><https://marketplace.visualstudio.com/items?itemName=ErikEJ.SQLServerCompactSQLiteToolbox>

### §17.3.2 Veri Tabanından Veri Okuma

Veri tabanına eklenmiş kayıtları da sorgulayıp öğrenebiliriz;

```
using Microsoft.Data.Sqlite; //Nuget Paketini Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "CSharp.sqlite");
var connectionString = $"Data Source={db}";
using (var connection = new SqliteConnection(connectionString))
{
 connection.Open();
 var command = connection.CreateCommand();
 command.CommandText = @"SELECT OgrenciId, Adi, Soyadi FROM Ogrenci;";
 using (SqliteDataReader reader = command.ExecuteReader()) //Veri getirecek bir sorgu!
 {
 while (reader.Read())
 {
 var öğrenciId = reader.GetString(0);
 var adı = reader.GetString(1);
 var soyadı = reader.GetString(2);
 Console.WriteLine($"{öğrenciId} {adı} {soyadı}");
 }
 }
 connection.Close();
}
/*Program Çıktısı:
(1) Ali Veli
(2) Ayşe Fatma
(3) Mehmet Can
(4) Zeynep Yılmaz
(5) Ahmet Demir
(6) Fatma Kaya
*/
```

### §17.3.3 Sorgu Parametreleri

Veri tabanı için yazılacak sorgulara parametreler metin olarak yazılma yerine kapsamdaki bir değişkene bağlanabilir;

```
using Microsoft.Data.Sqlite; //Nuget Paketini Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "CSharp.sqlite");
var connectionString = $"Data Source={db}";
using (var connection = new SqliteConnection(connectionString))
{
 connection.Open();
 var command = connection.CreateCommand();
 command.CommandText = @" INSERT INTO Ogrenci (Adi, Soyadi) VALUES ($Ad, $Soyad);";
 string ad = "İlhan";
 string soyad = "Özkan";
 command.Parameters.AddWithValue("$Ad", ad);
 command.Parameters.AddWithValue("$Soyad", soyad);
 int rowsAffected = command.ExecuteNonQuery();
 Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
 connection.Close();
}
```

### §17.3.4 Veritabanı İşlemleri

Veritabanı işlemlerinde (**transaction**), bazen birden fazla sorgunun birbirine bağlı olarak çalışması gerekir. Örneğin; bir banka hesabından para çekilip başka bir hesaba yatırılırken, para çekme işlemi başarılı olup



yatırma işlemi başarısız olursa veri tutarsızlığı oluşur. `SqlTransaction` bu gibi durumlarda tüm işlemleri geri almamızı (**rollback**) sağlar. Veri tabanında aynı anda çok sayıda değişiklik yapıldığı durumlarda faydalıdır. İşlem akışı;

1. `BeginTransaction()`: Bağlantı üzerinden işlemi başlatır.
2. **Transaction Ataması**: Oluşturulan işlem nesnesi, ilgili komutun (`SqlCommand.Transaction`) özelliğine atanır.
3. `Commit()`: Her şey yolundaysa değişiklikleri veritabanına kalıcı olarak kaydeder.
4. `Rollback()`: Bir hata oluşursa, işlem başladığından beri yapılan tüm değişiklikleri iptal eder.

Aşağıda veri tabanında güvenli veri güncelleme örneği verilmiştir;

```
using (SqlConnection baglanti = new SqlConnection(baglantiCumlesi))
{
 baglanti.Open();
 // 1. İşlemi (Transaction) başlatıyoruz.
 SqlTransaction islem = baglanti.BeginTransaction();
 SqlCommand komut = baglanti.CreateCommand();
 komut.Transaction = islem; // Komutun bu işleme ait olduğunu belirttik.
 try
 {
 // Birinci sorgu
 komut.CommandText = "UPDATE Hesaplar SET Bakiye = Bakiye - 100 WHERE ID = 1";
 komut.ExecuteNonQuery();
 // İkinci sorgu
 komut.CommandText = "UPDATE Hesaplar SET Bakiye = Bakiye + 100 WHERE ID = 2";
 komut.ExecuteNonQuery();
 // 2. Her şey başarılıysa onayla.
 islem.Commit();
 Console.WriteLine("İşlem başarıyla tamamlandı.");
 }
 catch (Exception ex)
 {
 // 3. Hata varsa tüm adımları geri al!
 islem.Rollback();
 Console.WriteLine("Hata oluştu, değişiklikler geri alındı: " + ex.Message);
 }
}
```

Kurumsal veri tabanlarının aksine SQLite veri tabanında, aynı anda yalnızca bir işlemin beklemede olan değişikliklere sahip olmasına izin verilir. Bu durum **eş zamanlılık (concurrency)** olarak adlandırılır. Bu nedenle, `SQLiteCommand` ile başlanan işte `BeginTransaction` ve `Execute` yöntemlerine yapılan çağrılar, başka bir işlemin tamamlanması çok uzun sürerse zaman aşımına uğrayabilir<sup>2</sup>. Bunu bizim SQLite veri tabanımıza Burslar adlı tabloyu ekleyerek bir öğrencinin burs miktarını değiştirme senaryosu üzerinde yapalım;

```
using Microsoft.Data.Sqlite; //Nuget Paketini Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "CSharp.sqlite");
var connectionString = $"Data Source={db}";
using (var connection = new SQLiteConnection(connectionString))
{
 connection.Open();
 // 1. Tabloyu Hazırlama ve Veri Ekleme
 using (var command = connection.CreateCommand())
 {
 command.CommandText = @"
 DROP TABLE IF EXISTS Burslar;
 CREATE TABLE Burslar (
 OgrenciId INTEGER NOT NULL,
 BursMiktari REAL NOT NULL
);
```

<sup>2</sup><https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/transactions>

```

INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (1,1000.0);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (2,2000.0);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (3,3000.0);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (4,4000.0);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (5,3000.0);
";
command.ExecuteNonQuery();
}
// 2. Transaction İşlemi: SQLite'da 'deferred' varsayılan moddur; yazma anına kadar
→ kilitlemeyi erteler.
using (var transaction = connection.BeginTransaction())
{
 try
 {
 using (var command = connection.CreateCommand())
 {
 // Komutu başlattığımız işlem (transaction) bağlıyoruz.
 command.Transaction = transaction;
 // Örnek Senaryo: Öğrenci 1'in bursuna 500 TL ekleme yapıyoruz.
 long ogrenciId = 1;
 decimal ekBurs = 500.0m;
 command.CommandText = "UPDATE Burslar SET BursMiktari = BursMiktari + @Miktar
 → WHERE OgrenciId = @Id";
 // Parametre kullanımı güvenlik için şarttır!
 command.Parameters.AddWithValue("@Miktar", ekBurs);
 command.Parameters.AddWithValue("@Id", ogrenciId);
 int etkilenenSatir = command.ExecuteNonQuery();
 if (etkilenenSatir == 0)
 {
 // Eğer güncellenecek öğrenci bulunamadıysa işlemi iptal edebiliriz.
 throw new Exception("Güncellenecek öğrenci kaydı bulunamadı!");
 }
 // 2. Her şey başarılıysa değişiklikleri kalıcı hale getir.
 transaction.Commit();
 Console.WriteLine($"{ogrenciId} ID'li öğrencinin bursu başarıyla güncellendi.");
 }
 }
 catch (Exception ex)
 {
 // 3. Hata varsa (bağlantı kopması, SQL hatası vb.) tüm adımları GERİ AL!
 transaction.Rollback();
 Console.WriteLine("İşlem başarısız, veritabanı eski haline döndürüldü: " +
 → ex.Message);
 }
}
}

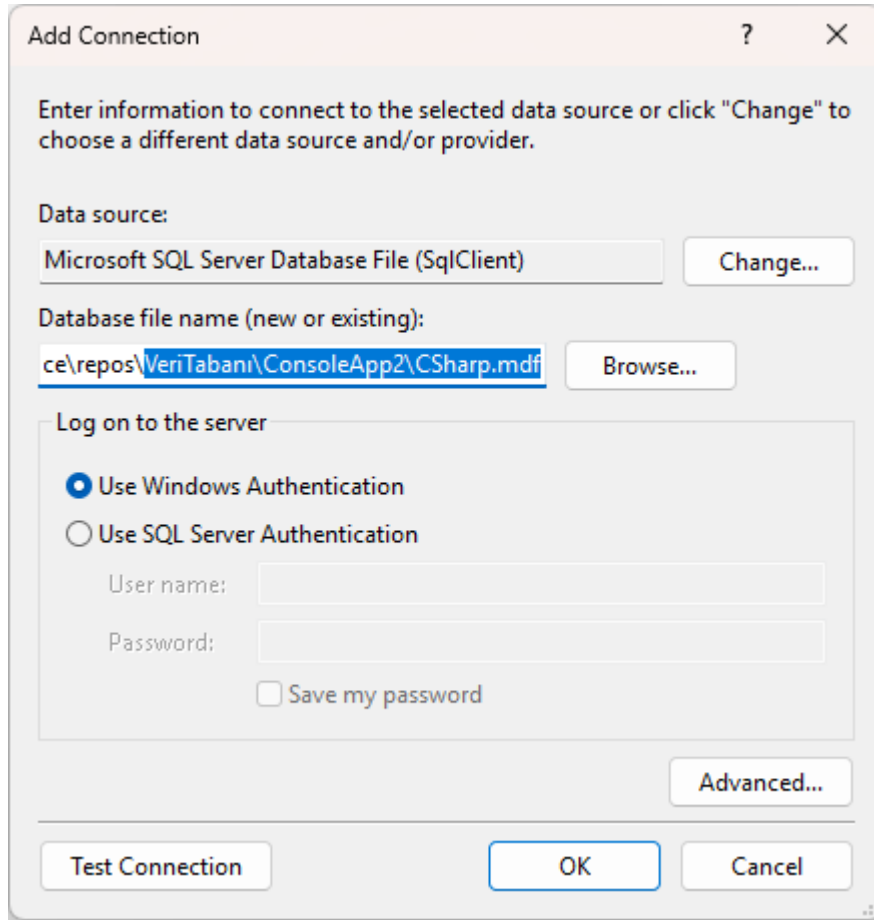
```

## 17.4 Veri Kaynaklarına Erişim Yöntemleri

.NET Framework ile veri kaynaklarına iki stratejiyle erişim yapılır;

1. Bağlı (**connected**): Bu yöntemde veri üzerindeki değişiklikler doğrudan veri kaynağı üzerinde yapılır. Veri kaynakları pahalı yatırımlarla kurulduğundan, kaynağı en az meşgul edecek işlemler için veya zorunlu durumlarda kullanılır. .NET teknolojisinde bunu **yukarıdaki örneklerde verildiği gibi DataReader** nesnesi ile yaparız.
2. Kopuk (**disconnected**): Bu yöntemde veriler bellekte geçici bir yere aktarılırlar. Veri seti olan ve **DataSet** olarak adlandırılan bu yer, sanki bir veri kaynağıymış gibi kullanılır. Veri üzerinden işlemler tamamlandığında, bellekteki değişiklikler veri kaynağına kaydedilir. Kısaca veri setleri, **hafızada oluşturulan bir veri tabanlarıdır**.

Veri Seti için *Microsoft SQL Server LocalDB* sürümünü kullanacağız. Bu veritabanının kurulumu, **Visual Studio** kurulumunda veri araçları seçilerek yapılabilir. Kurulum yapıldıktan sonra, yeni bir proje oluşturup **Tools → Connect To Database...** seçilir. Açılan pencerede yeni bir veri tabanı **proje klasöründe** oluşturulur;



Şekil 17.2: Microsoft SQL Server LocalDB Üzerinde Veri Tabanı Oluşturma

Oluşan veri tabanına **View→Server Explorer** menüsünden ulaşılabilir. Eğer **.NET Core CLI** ile proje oluşturmuş iseniz veri tabanını yönetmek için **SQL Server Management Studio** yazılımını kullanabilirsiniz<sup>3</sup>.

```
using Microsoft.Data.SqlClient; //Nuget Paketini Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
var db = Path.Combine(projeKlasörü, "CSharp.mdf");
string connectionString = $"Server=(LocalDB)\MSSQLLocalDB; AttachDbFilename={db}; Integrated
→ Security=True;Connect Timeout=30;";
using (var connection = new SqlConnection(connectionString))
{
 connection.Open();
 var command = connection.CreateCommand();
 command.CommandText = @"
 DROP TABLE IF EXISTS Ogrenci;
 CREATE TABLE Ogrenci (
 OgrenciId INT PRIMARY KEY IDENTITY (1, 1),
 Adi NVARCHAR (50) NOT NULL,
 Soyadi NVARCHAR (50) NOT NULL
);
 DROP TABLE IF EXISTS Ders;
 CREATE TABLE Ders(
 DersId INT PRIMARY KEY IDENTITY (1, 1),
 DersAdi NVARCHAR(50) NOT NULL
);
";
 int rowsAffected = command.ExecuteNonQuery();
 Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
 command.CommandText = @"
```

<sup>3</sup><https://learn.microsoft.com/en-us/ssms/install/install>

```

INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ali', 'Veli');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ayşe', 'Fatma');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Mehmet', 'Can');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Zeynep', 'Yılmaz');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ahmet', 'Demir');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Fatma', 'Kaya');
INSERT INTO Ders (DersAdi) VALUES ('Matematik');
INSERT INTO Ders (DersAdi) VALUES ('Fizik');
";
rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");

DataSet dataSet = new DataSet();
dataSet.Tables.Add("Ogrenci");
dataSet.Tables.Add("Ders");

SqlDataAdapter dataAdapterOgrenci = new SqlDataAdapter("Select * From Ogrenci", connection);
dataAdapterOgrenci.Fill(dataSet.Tables[0]); /* veriler veritabanından okunup verisetine
→ yüklendi. veritabanına bağlatı (connection) sadece bu an için yapıldı. */

SqlDataAdapter dataAdapterDers = new SqlDataAdapter("Select * From Ders", connection);
dataAdapterDers.Fill(dataSet.Tables[1]); /* veriler veritabanından okunup verisetine
→ yüklendi. veritabanına bağlatı (connection) sadece bu an için yapıldı. */

connection.Close(); //Bağlantı kapatıldıktan sonra veri setleriyle işlemler yapılabilir!

Console.WriteLine("OgrenciId,Adı,Soyadı");
foreach (DataRow row in dataSet.Tables[0].Rows)
Console.WriteLine("{0},{1},{2}", row[0], row[1], row[2]);
Console.WriteLine("DersId,DersAdi");
foreach (DataRow row2 in dataSet.Tables[1].Rows)
Console.WriteLine("{0},{1}", row2[0], row2[1]);
//Bağlantı kapatılmadan burada çeşitli tablo satırlarını (row) güncelleyebiliriz.
}

```

## 17.5 NoSQL Veri Tabanı

NoSQL veri tabanları ilişkisel veri tabanları kadar veri tutarlılığı sağlamaz ve verileri düzenlemek çok daha zordur. Bu nedenle genellikle çok sayıda veri yazmak ve çok sayıda veriyi okumak için kullanılır. Aşağıda bir okuldaki öğrenciler ve diplomalarının saklandığı bir NoSQL örneği verilmiştir. Diploma örneği olarak [CSharp.R3.pdf](#) dosyasını proje klasörüne koymalıyız.

```

using LiteDB; //Nuget Paketini (by Mauricio David) Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\\"));
var veritabanı = Path.Combine(projeKlasörü, "CSharp.litedb");
using (var db = new LiteDatabase(veritabanı))
{
 var öğrenciler = db.GetCollection<Öğrenci>("Öğrenciler");
 var öğrenci = new Öğrenci
 {
 Adı = "İlhan Özkan",
 EPosta = "ilhanozkan@outlook.com",
 CepTelefonu = "03121234567"
 };
 // Yeni Öğrenci koleksiyona eklenir (Id otomatik artar)
 öğrenciler.Insert(öğrenci);
 var result = öğrenciler.Find(x => x.EPosta == "ilhanozkan@outlook.com").FirstOrDefault();
 Console.WriteLine($"{result.Id}-{result.Adı}");
 öğrenci.CepTelefonu = "04621234567";
 öğrenciler.Update(öğrenci);
 var diplomalar = db.GetCollection<Diploma>("Diplomalar");
}

```

```

byte[] fileBytes = File.ReadAllBytes(Path.Combine(projeKlasörü, "CSharp.R3.pdf")); //PDF
↳ dosyayı veritabanına
var diploma = new Diploma
{
 OgrenciId = öğrenci.Id,
 PdfDocument = fileBytes
};
diplomalar.Insert(diploma);
}
public class Diploma
{
 public int Id { get; set; }
 public int OgrenciId { get; set; }
 public byte[] PdfDocument { get; set; }
}
public class Öğrenci
{
 public int Id { get; set; }
 public string Adı { get; set; }
 public string EPosta { get; set; }
 public string CepTelefonu { get; set; }
}

```

Veri tabanına yazdırılan nesneleri okumak için aşağıdaki kod kullanılabilir;

```

using LiteDB; //Nuget Paketini (by Mauricio David) Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
var veritabani = Path.Combine(projeKlasörü, "CSharp.Litedb");

using (var db = new LiteDatabase(veritabani))
{
 var öğrenciler = db.GetCollection<Öğrenci>("Öğrenciler");
 var liste = öğrenciler.FindAll();
 foreach (var o in liste)
 Console.WriteLine($"{o.Id}-{o.Adı}-{o.EPosta}-{o.CepTelefonu}");
 var diplomalar = db.GetCollection<Diploma>("Diplomalar");
 var result = diplomalar.Find(x => x.OgrenciId == 1).FirstOrDefault();
 File.WriteAllBytes(Path.Combine(projeKlasörü, "CSharp.R3.Veritabanından.pdf"),
 ↳ result.PdfDocument);
 //veritabanından dosya sistemine dosyayı yazdık.
}
public class Diploma
{
 public int Id { get; set; }
 public int OgrenciId { get; set; }
 public byte[] PdfDocument { get; set; }
}
public class Öğrenci
{
 public int Id { get; set; }
 public string? Adı { get; set; }
 public string? EPosta { get; set; }
 public string? CepTelefonu { get; set; }
}

```

## 17.6 Düşün ve Çöz

- ◊ **Düşün:** İlişkisel (SQL) ve ilişkisel olmayan (NoSQL) veri tabanları arasındaki temel tasarım felsefesi farkı nedir? ACID (Atomiklik, Tutarlılık, İzolasyon, Dayanıklılık) özellikleri neden ilişkisel veri tabanlarında kritik bir öneme sahiptir?
- ◊ **Düşün:** SQL enjeksiyonu (**SQL injection**) saldırısı nedir? Kullanıcı girdisini **string** birleştirme ile sorguya eklemek neden tehlikeli olduğunu somut bir örnekle gösterin. Parametrelili sorgular bu saldırıyı

nasıl engeller?

- ◇ **Düşün:** Bağlantı havuzu (**connection pooling**) neden gereklidir? Her işlemde yeni bir veri tabanı bağlantısı açmanın maliyeti nedir? **using** bloğuyla bağlantı yönetiminin bağlantı havuzuyla etkileşimi nasıldır?
- ◇ **Düşün:** İşlem (**transaction**) yönetimi neden gereklidir? Banka havalesi senaryosunda gönderen hesaptan para düşüldükten sonra alıcı hesaba para eklenemezse ne olur? **Rollback** mekanizması bu durumu nasıl çözer?
- ◇ **Düşün:** ADO.NET ile doğrudan SQL yazmak ile bir ORM kullanmak arasındaki ödünleşimler (**trade-offs**) nelerdir? Performans kritik bir raporlama ekranında neden doğrudan SQL tercih edilebilir?
- ◇ **Düşün:** SQLite, MSSQL ve PostgreSQL arasındaki seçimi etkileyen faktörler nelerdir? Gömülü uygulama, web API ve kurumsal analitik için hangisi neden tercih edilir?
- ◇ **Düşün:** N+1 sorgu problemi nedir? Bir yazarın tüm kitaplarını çeken uygulamada bu problemin nasıl oluştuğunu ve tek bir **JOIN** sorgusuyla nasıl çözüldüğünü açıklayınız.
- ◇ **Çöz:** SQLite kullanarak bir kütüphane veri tabanı oluşturun: Kitaplar ve Yazarlar tabloları arasında ilişki olsun. CRUD işlemlerinin (ekleme, listeleme, güncelleme, silme) her biri için parametrelili sorgu yazınız.
- ◇ **Çöz:** İşlem (**transaction**) kullanarak iki hesap arasında para transferi gerçekleştiren program yazınız: gönderen ve alıcı hesap bilgileri kullanıcıdan alınsın; yetersiz bakiye veya hesap bulunamama durumlarında geri alma (**rollback**) yapılsın.
- ◇ **Çöz:** SQL enjeksiyonu açığı içeren bir sorgu yazın (bilerek), ardından parametrelili sorguyla güvenli versiyonunu yazın. Her iki versiyonu **OR 1=1** girdisiyle test edin ve farkı gözlemleyin.
- ◇ **Çöz:** Öğrenci ve Ders tablolarını içeren bir veri tabanı oluşturun. LINQ benzeri bir arayüzle (yöntem zincirleme) SQL sorgusu oluşturan basit bir sorgu oluşturucu (**query builder**) sınıfı tasarlayın.
- ◇ **Çöz:** ADO.NET ile bir ürün envanteri uygulaması yazın: ürün arama (ada veya kategoriye göre), stok güncelleme ve düşük stoklu ürünleri listeyen raporlama yöntemleri içersin.

## 18. Bölüm

# Tasarım Desenleri

### 18.1 Desen Nedir?

Nesne yönelimli programlamanın ortaya çıkışıyla beraber, daha önceden yazılmış hazır bileşenlerin (**component**) yeniden kullanımı (**reusing**) konusunda oldukça önemli ilerlemeler sağlamıştır. Bunun paralelinde tecrübelerin (**experience**) yeniden kullanımı konusunda da çeşitli çalışmalar yapılmış ve **desen** (**pattern**) kavramı ortaya çıkmıştır. Desenlerin aksine **anti-desen** (**anti-pattern**) ise **başlangıçta çekici görünen** ancak uygulama sürecinde **verimsiz olduğu kanıtlanmış yaygın hatalı çözümlerdir**.

Yazılım geliştirme öncesi, geliştirme aşması ve sonrasında daha önce yaşanmış çeşitli problemlere karşı birçok kimse tarafından çeşitli çözümler getirilmiştir. **Desenler, daha önce geliştirme yapan programcıların yanlış yaparak doğru yapmayı öğrendikleri başarılı tecrübelerin anlatılması için bir araçtır**. Gerçekleştirilen bu çözümlerin, daha sonradan yaşanacak benzer nitelikli problemlerde kullanılması amacıyla desenler kullanılır. Desen kavramının temelleri Mimar *Christopher Alexander*'ın 1970 sonlarında başlattığı çalışmalara dayanmaktadır<sup>1</sup>.

1987 yılında uluslararası “*Nesne Yönelimli Programlama, Sistemler, Diller ve Uygulamalar*” konferansına kadar desenlerle ilgili bir çalışma ortaya çıkmamıştır<sup>2</sup>. Bu tarihten sonra ise başta *Grady Booch*, *Richard Helm*, *Erich Gamma* ve *Kent Beck* olmak üzere pek çok bilim adamı desenlerle ilgili makale ve sunumlar yayınlamışlardır. 1994 yılında *Erich Gamma*, *Richard Helm*, *Ralph Johnson* ve *John Vlissides* tarafından yayınlanan “*Tasarım Desenleri: Tekrar kullanılabilir Nesne Yönelimli Yazılımın Temelleri*” kitabı, tasarım desenlerin yazılımda kullanılmasında dönüm noktası olmuştur<sup>3</sup>. Yazılımlarda kullanılan desenler yedi ana başlıkta toplanırlar. Bunlar;

1. Mimari desenler (**architectural pattern**)
2. Analiz desenleri (**analysis pattern**)
3. Tasarım desenleri (**design pattern**)
4. Kodlama desenleri (**implementation pattern**)
5. Test desenleri (**test pattern**)
6. Çözüm desenleri (**solution pattern**)
7. Veri desenleri (**data pattern**)

Bu desenlerin en önemlisi ve ilk gelişenlerden birisi, **tasarım desenleridir** (**design pattern**). Tasarım desenleri aslında, Nesne Yönelimli Tasarım Prensiplerini, yazılımlara uygulamanın bir başka ifadesidir. Bu nedenle çok önemlidir. Tasarım desenleri ise üç ana başlıkta toplanmaktadır.

1. Nesne imali ile ilgili desenler (**creational patterns**)
2. Yapısal desenler (**structural patterns**)
3. Davranışlarla ilgili desenler (**behavioral patterns**)

Desenlerin çok sık kullanıldığı programlama yöntemine günümüzde **desen yönelimli programlama** (**pattern oriented programming**) adı verilmektedir<sup>4</sup>. İzleyen sayfalarda verilen desen UML diyagramlarının birçoğu *Erich Gamma*'nın **Design Pattern CD** yayınından alınmıştır<sup>5</sup>. Bölümün devamında verilecek tasarım desenlerinin UML diyagramları visual-paradigm adresinden alınmıştır<sup>6</sup>.

### 18.2 Nesne İmalatı ile İlgili Desenler

**Nesne imalatı** ile ilgili desenler (**creational patterns**), nesne (**object**) örnekleme ile ilgili ortaya çıkabilecek çeşitli problemlere çözüm getirmektedirler. Bu desenlerde nesneler dinamik olarak imal edilirler, çünkü değişim yönetiminde statik nesneler kullanılmazlar.

<sup>1</sup><http://gee.cs.oswego.edu/dl/ca/ca/ca.html>

<sup>2</sup>OOPSLA, Object Oriented Programming, Systems, Languages, and Applications

<sup>3</sup>Design Patterns: Elements of Reusable Object-Oriented Software, ISBN 0-201-63361-2

<sup>4</sup>[www.mathcs.sjsu.edu/faculty/pearce/patterns.html](http://www.mathcs.sjsu.edu/faculty/pearce/patterns.html)

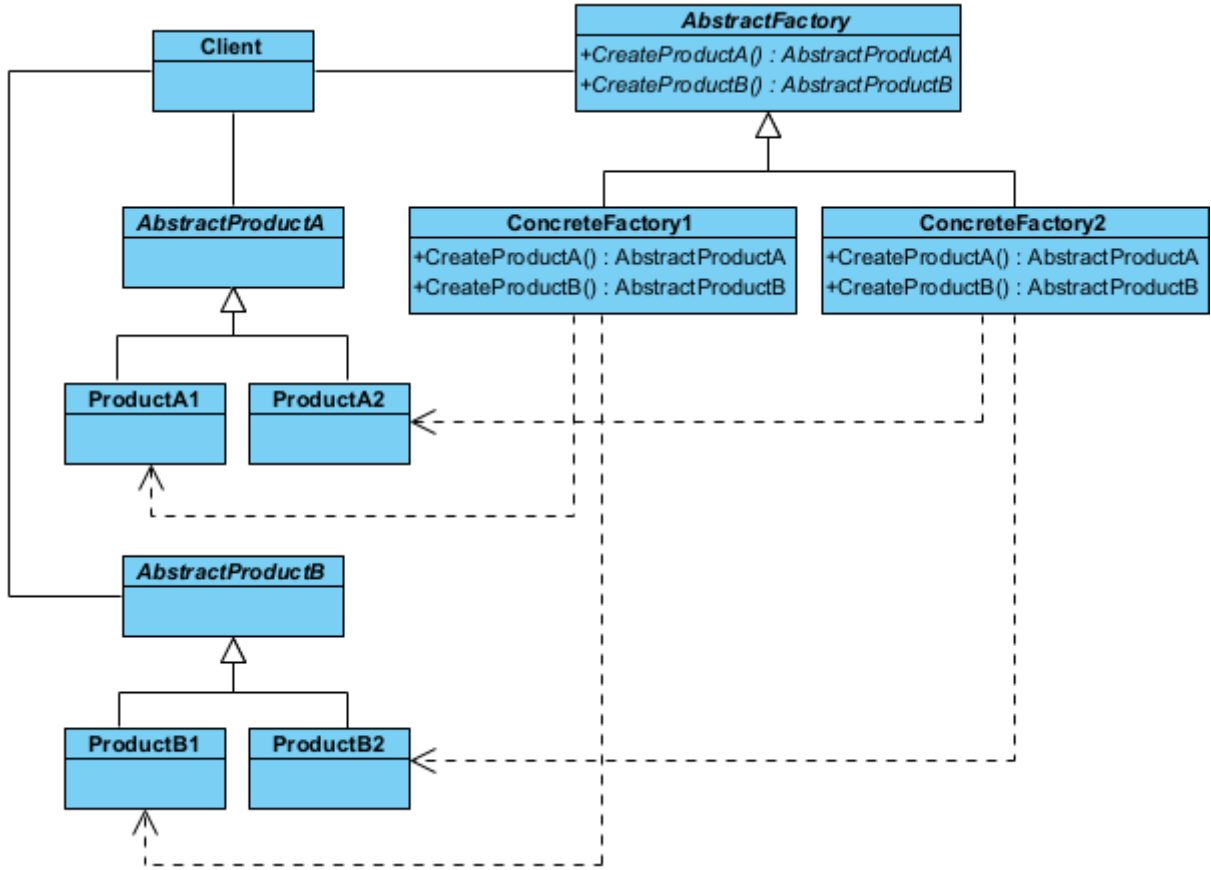
<sup>5</sup>Design Pattern CD, Addison-Wesley, 1998

<sup>6</sup><https://circle.visual-paradigm.com/category/uml-diagrams/gof-design/>



### §18.2.1 Soyut Fabrika

**Soyut fabrika deseninde** (**abstract factory pattern**) amaç, nesnelerin imal edilmesi ve kullanılması ile ilgili olan kısımlarının birbirinden ayrılmasıdır. Bu amaca, birbiriyle ilgili ya da birbirine bağımlı olabilecek nesnelerin imal edilmesinde, nesnelerin somut (**concrete**) sınıflarını değil soyut (**abstract**) sınıfları kullanılarak ulaşılır. Somut sınıflar istemciden izole edilir ve programcıya somut sınıfları daha kolay değiştirme için olanak sağlar.



Şekil 18.1: Soyut Fabrika Deseni UML Diyagramı

Bu tasarım deseninde, birbirleriyle ilişkili veya bağımlı nesne ailelerini, somut sınıflarını belirtmeden oluşturmak için kullanılan imalatla ilgili (**creational**) bir desendir. "Fabrikaların fabrikası" olarak da adlandırılır. Eğer sisteminizin farklı ürün aileleriyle (örneğin; Windows uyumlu buton/pencere ailesi ve Mac uyumlu buton/pencere ailesi) çalışması gerekiyorsa ve bu ailelerin birbirine karışmasını istemiyorsanız bu deseni kullanırsınız.

- ◊ Soyut Fabrika Arayüzü (**AbstractFactory**), ilgili tüm ürünleri oluşturacak yöntemlerin imzalarını (şablonunu) tanımlar. "Bir buton oluştur" ve "Bir pencere oluştur" gibi soyut görevleri belirler.
- ◊ Somut Fabrikalar (**ConcreteFactory**), belirli bir ürün ailesine ait nesneleri fiilen oluşturur. Örneğin **ModernMobilyaFabrikasi** sadece modern tarzdaki sandalyeleri ve masaları üretir.
- ◊ Soyut Ürünler (**AbstractProduct**), ürün ailesindeki her bir temel ürün türü için bir arayüz tanımlar. Tüm sandalyelerin sahip olması gereken ortak özellikleri (Örnek olarak **Otur()**) belirler.
- ◊ Somut Ürünler (**ConcreteProduct**), soyut ürünlerin farklı varyasyonlarını (ailelerini) temsil eden gerçek sınıflardır. **ModernSandalye** veya **AntikaSandalye** gibi sınıflar burada yer alır.

Bu deseni kodlayalım;

```

// 1. Products (Ürünler)
// --- Abstract Products ---
public abstract class AbstractProductA
{
 public string ProductName { get; }
 protected AbstractProductA(string pAdi) => ProductName = pAdi;
 public virtual string GetUrunAdi() => ProductName;
}
public class ConcreteProductA1 : AbstractProductA

```

```

{
 public ConcreteProductA1(string pAdi) : base(pAdi) { }
}
public class ConcreteProductA2 : AbstractProductA
{
 public ConcreteProductA2(string pAdi) : base(pAdi) { }
}
// --- Abstract Product B ---
public abstract class AbstractProductB
{
 public string ProductName { get; }
 protected AbstractProductB(string pAdi) => ProductName = pAdi;
 public virtual string GetUrunAdi() => ProductName;

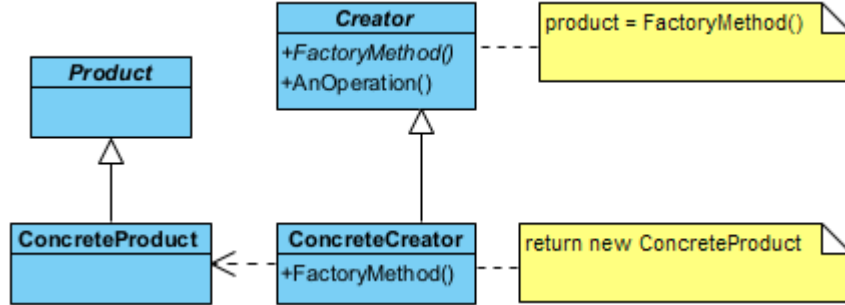
 public void Interact(AbstractProductA a)
 {
 Console.WriteLine($"{this.ProductName} ürünü {a.GetUrunAdi()} ile etkileşim halinde...");
 }
}
public class ConcreteProductB1 : AbstractProductB
{
 public ConcreteProductB1(string pAdi) : base(pAdi) { }
}
public class ConcreteProductB2 : AbstractProductB
{
 public ConcreteProductB2(string pAdi) : base(pAdi) { }
}
// 2. Factory (Fabrika)
// C#'ta soyut fabrikalar için genellikle interface (arayüz) tercih edilir. Çünkü özellik ve alan
→ tanımlaması içermez.
public interface IAbstractFactory
{
 AbstractProductA CreateProductA();
 AbstractProductB CreateProductB();
}
public class ConcreteFactory1 : IAbstractFactory
{
 public AbstractProductA CreateProductA() => new ConcreteProductA1("A1 ürünü");
 public AbstractProductB CreateProductB() => new ConcreteProductB1("B1 Ürünü");
}
public class ConcreteFactory2 : IAbstractFactory
{
 public AbstractProductA CreateProductA() => new ConcreteProductA2("A2 ürünü");
 public AbstractProductB CreateProductB() => new ConcreteProductB2("B2 Ürünü");
}
// 3. Client (İstemci)
class Program
{
 static void Main(string[] args)
 {
 IAbstractFactory factory1 = new ConcreteFactory1();
 IAbstractFactory factory2 = new ConcreteFactory2();
 var productA = factory1.CreateProductA();
 Console.WriteLine($"Fabrika 1 ürünü: {productA.GetUrunAdi()}");
 var productB = factory2.CreateProductB();
 Console.WriteLine($"Fabrika 2 ürünü: {productB.GetUrunAdi()}");
 productB.Interact(productA);
 }
}
/*Program Çıktısı:
Fabrika 1 ürünü: A1 ürünü
Fabrika 2 ürünü: B2 Ürünü

```

B2 Ürünü ürünü A1 ürünü ile etkileşim halinde...  
\*/

### §18.2.2 Fabrika Yöntemi

Fabrika yöntemi deseninde (**factory method pattern**) amaç, imal edilecek nesnenin sınıfını kesin olarak belirtmeden nesne yaratma işleminin gerçekleştirilmesidir. Bu desenin görevi, nesne üretim işini alt sınıflara



Şekil 18.2: Fabrika Yöntemi Deseni UML Diyagramı

bırakır. Hangi nesnenin üretileceğine çalışma zamanında karar verilir.

- ◊ Üretici (**Creator**), fabrika yöntemini tanımlayan soyut sınıftır.
- ◊ Ürün (**Product**), üretilecek nesnelerin ortak arayüzüdür.
- ◊ Somut Üretici (**ConcreteCreator**), ihtiyaca göre "A Ürünü" veya "B Ürünü" üreten gerçek fabrikadır.

Bu deseni kodlayalım;

```
// 1. Products (Ürünler)
// --- Ürün Sınıfları ---
public abstract class Product
{
 public string ProductName { get; }
 protected Product(string pAdi)
 {
 ProductName = pAdi;
 }
 public virtual string GetUrunAdi() => ProductName;
}
public class ConcreteProduct : Product
{
 public ConcreteProduct(string pAdi) : base(pAdi) { }
}
// 2. Creators (İmalatçılar)
// --- Creator (İmalatçı) Sınıfları ---
public abstract class Creator
{
 // Fabrika Yöntemi: Alt sınıflar tarafından geçersiz kılınacak olan metot.
 public abstract Product FactoryMethod();
 public abstract void AnOperation();
}
public class ConcreteCreator : Creator
{
 public override Product FactoryMethod()
 {
 return new ConcreteProduct("Fabrika Yöntemi Ürünü");
 }
 public override void AnOperation()
 {
 Console.WriteLine("Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir...");
 }
}
// 3. Client (İstemci)
class Program
{

```

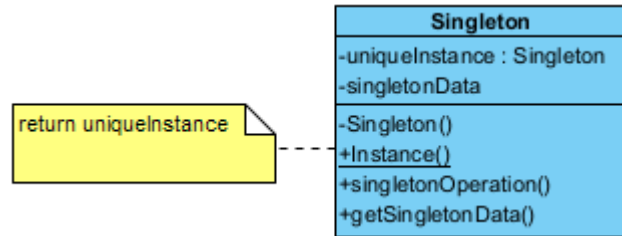
```

static void Main(string[] args)
{
 // 1. Creator (İmalatçı) nesnesini oluşturuyoruz.
 Creator imalatci = new ConcreteCreator();
 // 2. Ürünü oluşturuyoruz. Bellek yönetimi tamamen .NET tarafından yapılır.
 Product urun = imalatci.FactoryMethod();
 Console.WriteLine($"İmalatçı tarafından imal edilen ürün: {urun.GetUrunAdi()}");
 imalatci.AnOperation();
}
}
/*Program Çıktısı:
İmalatçı tarafından imal edilen ürün: Fabrika Yöntemi Ürünü
Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir...
*/

```

### §18.2.3 Tekil Nesne

Tekil Nesne deseninde (singleton pattern) amaç, bir sınıftan yalnızca bir örnek (instance) imal etmektir. Bir sınıfın yalnızca bir nesnesinin olmasına izin verilir, birden fazla olmasına izin verilmez.



Şekil 18.3: Tekil Deseni UML Diyagramı

Şimdi Singleton sınıfını kodlayalım;

```

//1. Singleton sınıfı: Öyle bir sınıftır ki yalnızca bir örneği(instance) vardır.
class Singleton
{
 // Sınıftan yaratılan örneklerle değil, sınıfa ait üye değişken (class scope member variable)
 // tanımlanıyor.
 private static Singleton? _uniqueInstance=null;
 private int _singletonData;
 // Birden fazla örneğe(instance) izin vermemek için yapıcı(constructor) işlevin
 // görünürlüğü(visibility) özel(private) olmalıdır.
 private Singleton()
 {
 _singletonData=0;
 }
 //Tekil Nesneyi verecek Yöntem:
 public static Singleton Instance()
 {
 if (_uniqueInstance == null)
 _uniqueInstance = new Singleton();
 return _uniqueInstance;
 }
 //Tekil nesnenin yöntem ve özellikleri:
 public void SingletonOperation()
 {
 Console.WriteLine("Singleton İşlevi Çağrıldı.");
 }
 public int SingletonData {
 get
 {
 return _singletonData;
 }
 set{
 _singletonData=value;
 }
 }
}

```

```

 }
}
//2. İstemci
class Client
{
 static void Main(string[] args)
 {
 Singleton object1 = Singleton.Instance();
 Singleton object2 = Singleton.Instance();
 if (object1 == object2)
 Console.WriteLine
 ("object1 ve object2 AYNI nesnedir.");
 else
 Console.WriteLine
 ("object1 ve object2 FARKLI nesnedir.");
 object1.SingletonOperation();
 }
}
/* Program Çıktısı:
object1 ve object2 AYNI nesnedir.
Singleton İşlevi Çağrıldı.
*/

```

Bu desende `if (instance == null)` kontrolüyle nesne oluşturulduğu görülüyor. Bu yöntem çoklu iş parçacığı (**multi-threading**) ortamlarında aynı anda iki nesnenin oluşmasına neden olabilir. Bunun için çift kilitleme (**double-check locking**) iş parçacığı güvenli tekil nesne (**thread-safe singleton**) desnei veya .NET'in **Lazy<T>** sınıfını kullanmalıyız;

```

//1. Singleton Sınıfı
public sealed class VeritabaniBaglantisisi
{
 // Lazy<T> kullanımı, nesnenin sadece .Value özelliğine erişildiğinde
 // ve thread-safe bir şekilde oluşturulacağını garanti eder.
 private static readonly Lazy<VeritabaniBaglantisisi> _instance = new
 ↳ Lazy<VeritabaniBaglantisisi>(() => new VeritabaniBaglantisisi());

 // Dışarıdan erişim için tek nokta
 public static VeritabaniBaglantisisi Instance => _instance.Value;

 // Yapıcı metodun private olması, dışarıdan new ile nesne oluşturulmasını engeller.
 private VeritabaniBaglantisisi()
 {
 // Ağır yük getiren başlangıç işlemleri burada yapılabilir.
 Console.WriteLine("Veritabanı bağlantı nesnesi oluşturuldu.");
 }

 public void Baglan()
 {
 Console.WriteLine("Veritabanına bağlanıldı.");
 }
}

//2. İstemci
class Client
{
 static void Main(string[] args)
 {
 // Programın herhangi bir yerinde:
 // Bu satır çağrılana kadar nesne bellekte oluşturulmaz.
 VeritabaniBaglantisisi.Instance.Baglan();
 }
}

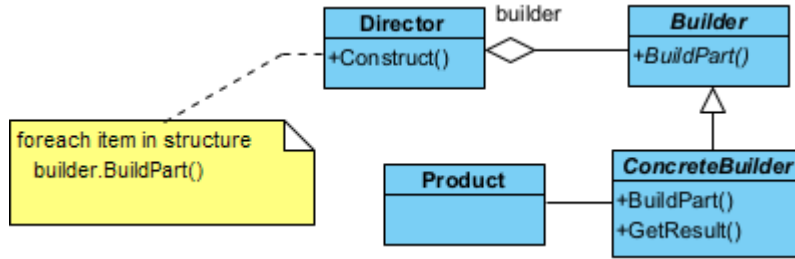
```

Neden `Lazy<T>` Kullanmalıyız?

- ◊ **İş Parçacığı-Güvenli (thread-safe) Olması:** `Lazy<T>` varsayılan olarak **thread-safe** çalışır. `lock` anahtar kelimesini manuel olarak kullanmanıza (**double-check locking**) gerek kalmaz.
- ◊ **Performans:** Nesne uygulama başladığında değil, gerçekten `Instance` özelliğine ilk kez erişildiğinde oluşturulur.
- ◊ **Okunabilirlik:** Karmaşık kilit mekanizmalarına kıyasla çok daha temiz ve anlaşılır bir söz dizimi sunar.
- ◊ **Hata Yönetimi:** Eğer nesne oluşturulurken bir hata (**exception**) oluşursa, `Lazy<T>` bu durumu yönetebilir.

## §18.2.4 Kurucu

**Kurucu deseninde (builder pattern)** amaç, çok karmaşık bir nesnenin imal edilmesiyle ilgili işlemleri bir başka sınıfa bırakmaktır. Böylece nesnenin gösterimi ve kullanılması ilişkin kodlar ile imalatına ilişkin kodlar birbirinden ayrılmış olur.



Şekil 18.4: Kurucu Deseni UML Diyagramı

Bu deseni bir fast-food restoranındaki "Menü Hazırlama" sürecine benzetebilirsin. Hamburgerin içine turşu konulup konulmayacağı, içeceğin boyutu veya yan ürünün ne olacağı gibi adımlar tek tek belirlenir, ancak en sonunda ortaya tek bir "Menü" nesnesi çıkar.

- ◊ **Ürün (Product)**, inşa edilmeye çalışılan karmaşık nesnedir. Genellikle çok sayıda parçadan veya konfigürasyondan oluşur.
- ◊ **Soyut Kurucu (Builder)**, ürünün parçalarını oluşturmak için gerekli olan tüm adımları (yöntemleri) arayüz olarak tanımlar. "Ekmek koy", "Köfte ekle", "Sos dök" gibi soyut inşa adımlarını belirler.
- ◊ **Somut Kurucu (ConcreteBuilder)**, soyut kurucudaki adımları fiilen gerçekleştirir ve ürünün bir örneğini saklar. "Vejetaryen Burger" veya "Double Burger" gibi farklı ürün varyasyonlarını nasıl inşa edeceğini bilir.
- ◊ **Yönetici (Director)**, inşa adımlarının hangi sırayla çağrılacağını kontrol eder. İstemciden (**Client**) aldığı kurucu nesnesini kullanarak, doğru sırayla (Örn: Önce temel, sonra yan ürünler) inşa sürecini yürütür.

```

using System;
using System.Collections.Generic; // List<T> için gerekli
// 1. Product (Ürün)
class Product
{
 private readonly List<string> _parts = new List<string>();

 public void Add(string part)
 {
 _parts.Add(part);
 }

 public void Show()
 {
 Console.WriteLine("\nÜrün Parçaları:");
 foreach (string part in _parts)
 {
 Console.WriteLine($"- {part}");
 }
 }
}
// 2. Builder (Soyut Kurucu)
abstract class Builder
{

```

```

 public abstract void BuildPartA();
 public abstract void BuildPartB();
 public abstract Product GetResult();
}
// 3. Concrete Builders (Somut Kurucular)
class ConcreteBuilder1 : Builder
{
 private Product _product = new Product();
 public override void BuildPartA() => _product.Add("A Parçası");
 public override void BuildPartB() => _product.Add("B Parçası");
 public override Product GetResult()
 {
 Product finishedProduct = _product;
 _product = new Product(); // Yeni bir inşa için builder'ı sıfırlıyoruz.
 return finishedProduct;
 }
}
class ConcreteBuilder2 : Builder
{
 private Product _product = new Product();
 public override void BuildPartA() => _product.Add("X Parçası");
 public override void BuildPartB() => _product.Add("Y Parçası");

 public override Product GetResult()
 {
 Product finishedProduct = _product;
 _product = new Product();
 return finishedProduct;
 }
}
// 4. Director (Yönetici)
class Director
{
 // İnşa sürecinin algoritmasını Director belirler.
 public void Construct(Builder builder)
 {
 builder.BuildPartA();
 builder.BuildPartB();
 }
}
// 5. Client (İstemci)
class Program
{
 static void Main(string[] args)
 {
 Director director = new Director();
 // Builder 1 ile inşa
 Builder b1 = new ConcreteBuilder1();
 director.Construct(b1);
 Product p1 = b1.GetResult();
 p1.Show();
 // Builder 2 ile inşa
 Builder b2 = new ConcreteBuilder2();
 director.Construct(b2);
 Product p2 = b2.GetResult();
 p2.Show();
 }
}
/*Program Çıktısı:
Ürün Parçaları:
- A Parçası
- B Parçası

```



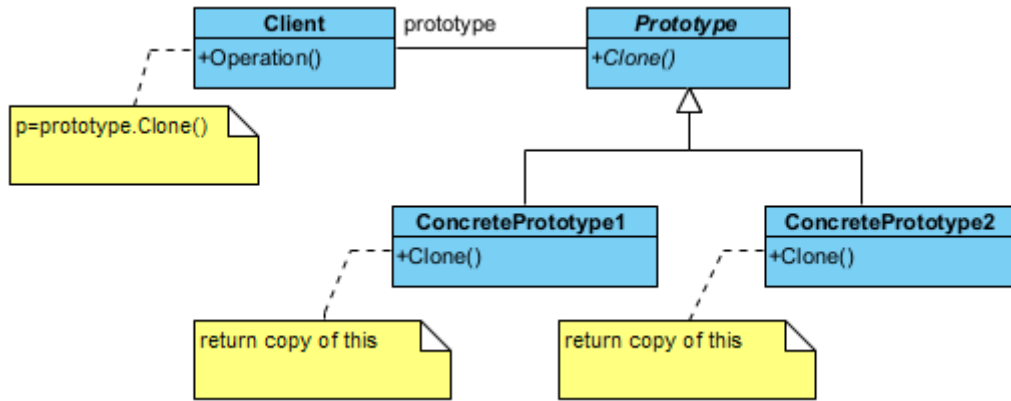
Ürün Parçaları:

- X Parçası
  - Y Parçası
- \*/

### §18.2.5 Prototip

**Prototip deseni** (**prototype pattern**), mevcut bir nesnenin kopyasını (klonunu) oluşturmak için kullanılan imalatla ilgili bir desendir. Bu desenin temel amacı, bir nesneyi oluşturmanın maliyeti (zaman, bellek veya işlem gücü açısından) çok yüksek olduğunda, sıfırdan **new** işleci ile üretmek yerine mevcut bir örneği klonlayarak yeni nesneler elde etmektir.

- ◊ Soyut Prototip (**Prototype**), nesnenin kendisini kopyalayabilmesi için gerekli olan arayüzü tanımlar. Genellikle sadece bir **clone()** yönteminin imzasını barındırır.
- ◊ Somut Prototip (**ConcretePrototype**), soyut prototip arayüzünü uygular ve asıl kopyalama işlemini gerçekleştirir. Kendi verilerini yeni bir nesneye aktarır.
- ◊ İstemci (**Client**), yeni bir nesneye ihtiyaç duyduğunda, sınıfa gidip "yeni bir tane üret" demek yerine, elindeki prototip nesnesine "kendini klonla" der.



Şekil 18.5: Prototip Deseni UML Diyagramı

Şimdi bu deseni kodlayalım;

```
// 1. Prototip Arayüzü
public abstract class Prototype
{
 protected string _data;
 protected Prototype(string data) => _data = data;
 public abstract Prototype Clone();
 public void ShowData()
 {
 Console.WriteLine($"Nesne Verisi: {_data}");
 }
}

// 2. Somut Prototip (Concrete Prototype)
public class ConcretePrototype : Prototype
{
 public ConcretePrototype(string data) : base(data) { }
 // Bu metot "Shallow Copy" (Sığ Kopyalama) yapar.
 public override Prototype Clone()
 {
 // MemberwiseClone, nesnenin tüm alanlarını yeni bir nesneye kopyalar.
 return (Prototype)this.MemberwiseClone();
 }
}

// 3. İstemci (Client)
class Program
{
 static void Main(string[] args)
 {

```

```
// 1. Orijinal nesneleri oluşturalım
Prototype object1 = new ConcretePrototype("40.30");
object1.ShowData();
// 2. Klonlama işlemi
Prototype clone1 = object1.Clone();
clone1.ShowData();
Prototype object2 = new ConcretePrototype("Mehmet");
object2.ShowData();
Prototype clone2 = object2.Clone();
clone2.ShowData();
// Nesnelerin aynı olup olmadığını kontrol edelim (Adres kontrolü)
Console.WriteLine($"Nesneler aynı mı?: {ReferenceEquals(object1, clone1)}");
}
}
/*Program Çıktısı:
Nesne Verisi: 40.30
Nesne Verisi: 40.30
Nesne Verisi: Mehmet
Nesne Verisi: Mehmet
Nesneler aynı mı?: False
*/
```

Nesneleri kopyalama söz konusu olduğunda, genel olarak üç tür kavramdan bahsedilir;

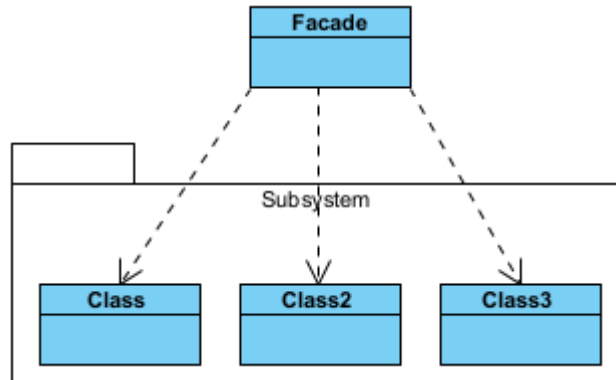
1. **Üstünkörü kopyalamada (shallow copy)** yeni nesne eskisini referans gösterir. Bu durumda birinci nesnenin bir alanı (**field**) değiştiğinde ikincisinin ki de değişmiş olur. Yani birinde olan değişiklik diğerini etkiler. Bu kopyalamanın diğer adı da bit düzeyi kopyalamadır (**bitwise copy**).
2. **Derinlemesine kopyalamada (deep copy)** ise birinci nesnenin aynısı bir başka bellek bölgesinde oluşturulur ve ikinci nesne yeni bellek bölgesini referans gösterir. Bu kopyalamanın diğer adı da üye düzeyi kopyalamadır (**memberwise copy**). Kopyalama sonrasında bir nesne üzerinde olan değişiklik yeni nesneyi etkilemez.
3. Üçüncü tip bir kopyalama ise **tembel kopyalamadır (lazy copy)**. Yukarıda bahsi geçen iki kopyalamanın iç içe girmiş şeklidir

## 18.3 Yapısal Desenler

**Yapısal desenler (structural pattern)**, sınıflar ve nesnelerin çok olduğu daha karmaşık programlarda getirilen yapısal çözüm yöntemleridir.

### §18.3.1 Önyüz

**Önyüz deseninde (facade pattern)** karmaşık bir alt sistemdeki (**subsystem**) çok sayıda sınıfı ve arayüzü, kullanımı kolay tek bir üst düzey arayüz arkasında gizlemek için kullanılan yapısal bir desendir. Bu deseni bir restoranın "Garsonu" gibi düşünebilirsin. Sen sadece sipariş verirsın; garson ise mutfak, bulaşıkhanenin ve kasanın karmaşıklığıyla senin adına ilgilenir.



Şekil 18.6: Önyüz Deseni UML Diyagramı

- ◇ Önyüz (**Facade**), karmaşık alt sistemleri bilen ve onları yöneten "ana giriş kapısı"dır. İstemciden (**Client**) gelen basit isteği, alt sistemdeki doğru nesnelere yönlendirir ve onları uygun sırayla çalıştırır.
- ◇ Alt Sistem Sınıfları (**SubSystem**), işin asıl detaylarını yapan çok sayıda sınıftır (Örn: SesSistemi, Işık-Denetleyici, Projeksiyon). **Facade** nesnesinden tamamen habersizdirler. Kendi işlerini yaparlar. **Facade**

sadece bunları koordine eder.

- ◊ İstemci (**Client**), sistemi kullanan son kullanıcı veya başka bir kod parçasıdır. Alt sistemin karmaşık detaylarıyla (hangi sınıf hangi sırayla çağrılacak gibi) uğraşmaz; sadece **Facade**'in sunduğu basit yöntemleri çağırır.

Şimdi desenimizi kodlayalım;

```
// 1. Alt Sistemler (Subsystems)
// Bu sınıflar karmaşık iş mantığını barındırır.
public class SubSystem1
{
 public void Operation11() => Console.WriteLine("Subsystem1: Yöntem 1 çalıştırıldı.");
}
public class SubSystem2
{
 public void Operation22() => Console.WriteLine("Subsystem2: Yöntem 2 çalıştırıldı.");
}
public class SubSystem3
{
 public void Operation31() => Console.WriteLine("Subsystem3: Yöntem 1 çalıştırıldı.");
 public void Operation32() => Console.WriteLine("Subsystem3: Yöntem 2 çalıştırıldı.");
 public void Operation33() => Console.WriteLine("Subsystem3: Yöntem 3 çalıştırıldı.");
}
// 2. Facade (Önyüz) Sınıfı
// İstemciye (Client) basit bir arayüz sunar, karmaşayı yönetir.
public class Facade
{
 // C#'ta nesneler referans tipli olduğu için 'new' ile doğrudan oluşturulur.
 // Bellek yönetimi Garbage Collector tarafından yapılır.
 private readonly SubSystem1 _subsystem1;
 private readonly SubSystem2 _subsystem2;
 private readonly SubSystem3 _subsystem3;
 public Facade()
 {
 _subsystem1 = new SubSystem1();
 _subsystem2 = new SubSystem2();
 _subsystem3 = new SubSystem3();
 }
 public void ExecuteComplexOperation()
 {
 Console.WriteLine("--- Facade Karmaşık İşlemi Başlatıyor ---");
 _subsystem1.Operation11();
 _subsystem2.Operation22();
 _subsystem3.Operation31();
 _subsystem3.Operation33();
 _subsystem3.Operation32();
 Console.WriteLine("--- İşlem Tamamlandı ---");
 }
}
// 3. Client (İstemci)
class Program
{
 static void Main(string[] args)
 {
 // İstemci alt sistemlerin detaylarını bilmek zorunda değil.
 // Sadece Facade ile konuşur.
 Facade facade = new Facade();
 facade.ExecuteComplexOperation();
 }
}
/*Program Çıktısı:
--- Facade Karmaşık İşlemi Başlatıyor ---
Subsystem1: Yöntem 1 çalıştırıldı.
```

```
Subsystem2: Yöntem 2 çalıştırıldı.
Subsystem3: Yöntem 1 çalıştırıldı.
Subsystem3: Yöntem 3 çalıştırıldı.
Subsystem3: Yöntem 2 çalıştırıldı.
--- İşlem Tamamlandı ---
*/
```

Aşağıda bu desene gerçek dünya örneği olarak alt sistemleri olan bir bilgisayarın başlatılması modellenmiştir;

```
// --- Sabit Tanımlamalar ---
public static class BootConstants
{
 public const long Address = 0x7C00;
 public const int Sector = 0;
 public const int SectorSize = 512;
}
// 1. --- Alt Sistemler (Subsystems) ---
public class Cpu
{
 public void Freeze() => Console.WriteLine("CPU: Durduruldu (Freeze).");
 public void Jump(long position) => Console.WriteLine($"CPU: Adrese atlandı:
 ↪ 0x{position:X}");
 public void Execute() => Console.WriteLine("CPU: Komutlar çalıştırılıyor...");
}
public class Memory
{
 public void Load(long position, string data) => Console.WriteLine($"Memory: 0x{position:X}
 ↪ adresine veri yüklendi: [{data}]");
}
public class HardDrive
{
 public string Read(long lba, int size)
 {
 Console.WriteLine($"HardDrive: Sector {lba} üzerinden {size} byte veri okundu.");
 return "OS_KERNEL_DATA";
 }
}
// 2. --- Facade (Önyüz) ---
public class ComputerFacade
{
 private readonly Cpu _cpu;
 private readonly Memory _memory;
 private readonly HardDrive _hardDrive;
 public ComputerFacade()
 {
 _cpu = new Cpu();
 _memory = new Memory();
 _hardDrive = new HardDrive();
 }
 public void Start()
 {
 Console.WriteLine("Bilgisayar başlatılıyor...");
 Console.WriteLine(new string('-', 35));
 // Facade karmaşık boot algoritmasını yönetir:
 _cpu.Freeze();
 string data = _hardDrive.Read(BootConstants.Sector, BootConstants.SectorSize);
 _memory.Load(BootConstants.Address, data);
 _cpu.Jump(BootConstants.Address);
 _cpu.Execute();
 Console.WriteLine(new string('-', 35));
 Console.WriteLine("Sistem hazır!");
 }
}
```

```

}
// 3. --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 // İstemci alt sistemlerle (CPU, RAM, HDD) hiç ilgilenmez.
 var computer = new ComputerFacade();
 computer.Start();
 }
}
/*Program Çıktısı:
Bilgisayar başlatılıyor...

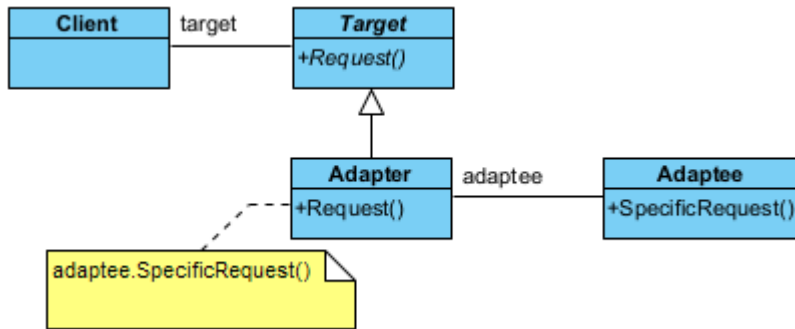
CPU: Durduruldu (Freeze).
HardDrive: Sector 0 üzerinden 512 byte veri okundu.
Memory: 0x7C00 adresine veri yüklendi: [OS_KERNEL_DATA]
CPU: Adrese atlandı: 0x7C00
CPU: Komutlar çalıştırılıyor...

Sistem hazır!
*/

```

### Ş18.3.2 Adaptör

**Adaptör deseni (adapter pattern)**, yabancı bir ara yüzü istemcinin anlayacağı bir ara yüze dönüştürür. Yani farklı ara yüzlere sahip sınıfların, iletişim ve etkileşim kurabilecekleri ortak bir nesne oluşturarak birlikte çalışmalarına izin verir. Bu desenin amacı, uyumsuz iki arayüzün (**interface**) birlikte çalışmasını sağlar. Burada



Şekil 18.7: Adaptör Deseni UML Diyagramı

yabancı ara yüze yaptırılacak işin bir şekilde yapıldığı ve zaten yapılan bu işlemin istemcinin (**Client**) isteğine uygun hale getirildiği düşünülmelidir. Bir nevi elektrik prizi dönüştürücüsü görevi görür.

- ◊ İstemci (**Client**), mevcut sistemle çalışan ana kod.
- ◊ Uyumsuz Nesne (**Adaptee**), sisteme dahil edilmek istenen ama arayüzü farklı olan nesne.
- ◊ Hedef (**Target**), istemcinin (**Client**) beklediği arayüz.
- ◊ Uyarlayıcı (**Adapter**), uyumsuz nesneyi sarmalayarak, istemcinin beklediği biçime çeviren aracı sınıf.

Şimdi de desenimizi kodlayalım;

```

// 1. Adaptee (Uyarlanan): Mevcut olan ama arayüzü uyumsuz olan sınıf.
// Bu sınıfın koduna dokunamadığımızı varsayıyoruz (Örn: Bir DLL içindedir).
public class Adaptee
{
 public void SpecificRequest()
 {
 Console.WriteLine("Adaptee: Özel istek (SpecificRequest) yöntemi çalıştırıldı.");
 }
}

// 2. Target (Hedef): İstemcinin beklediği standart arayüz.
public interface ITarget
{

```

```

 void Request();
}
// 3. Adapter (Uyarlayıcı): ITarget arayüzünü Adaptee'ye bağlar.
public class Adapter : ITarget
{
 private readonly Adaptee _adaptee;
 public Adapter()
 {
 _adaptee = new Adaptee();
 }
 public void Request()
 {
 Console.WriteLine("Adapter: Standart istek alındı, Adaptee'ye yönlendiriliyor...");
 // Adaptörün asıl görevi: Standart çağrıyı özel çağrıya dönüştürmek.
 _adaptee.SpecificRequest();
 }
}
// 4. Client (İstemci)
class Program
{
 static void Main(string[] args)
 {
 // İstemci ITarget arayüzünü bilir, arka planda Adaptee olduğunu bilmez.
 ITarget target = new Adapter();
 target.Request();
 }
}
/*Program Çıktısı:
Adapter: Standart istek alındı, Adaptee'ye yönlendiriliyor...
Adaptee: Özel istek (SpecificRequest) yöntemi çalıştırıldı.
*/

```

Bu desen gerçek dünya örneği olarak aşağıdaki iz kaydı (log) örneği verilebilir;

```

// 1. --- Adaptee (Uyarlanan Sınıf) ---
// Değiştiremediğimiz veya eski (Legacy) bir kütüphaneden gelen sınıf
public class OldLogger
{
 public void LogMessage(string msg)
 {
 Console.WriteLine($"[Eski Kayıt Sistemi]: {msg}");
 }
}
// 2. --- Target Interface (Hedef Arayüz) ---
// Yeni sistemin beklediği modern ve standart arayüz
public interface ILogger
{
 void Log(string msg);
}
// 3. --- Adapter (Adaptör Sınıfı) ---
public class LoggerAdapter : ILogger
{
 private readonly OldLogger _oldLogger;
 // C#'ta bağımlılıkları yapıcı metot üzerinden (Dependency Injection) almak en iyi pratiktir.
 public LoggerAdapter(OldLogger oldLogger)
 {
 _oldLogger = oldLogger;
 }
 public void Log(string msg)
 {
 // Adaptörün asıl görevi: Metot ismini ve gerekiyorsa parametre yapısını dönüştürmek
 _oldLogger.LogMessage(msg);
 }
}

```

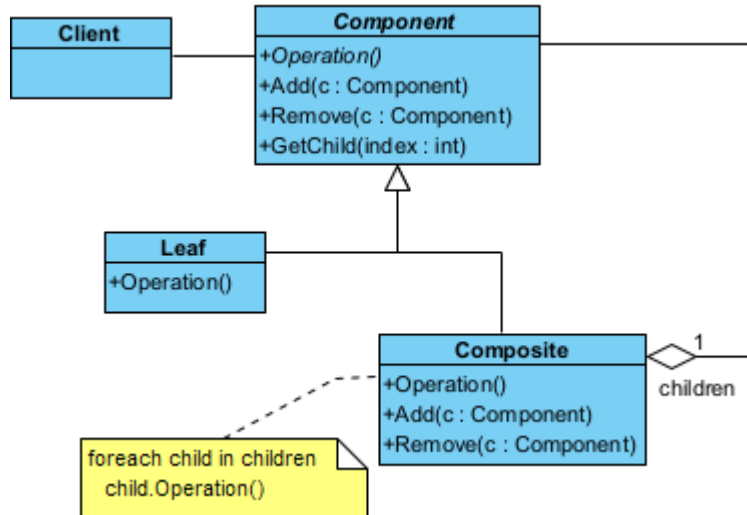
```

 }
}
// 4. --- Client (İstemci) ---
class Program
{
 // İstemci sadece ILogger arayüzünü tanır, arka plandaki implementasyondan bağımsızdır.
 static void AppLog(ILogger logger)
 {
 logger.Log("Sistem başarıyla çalıştı.");
 }
 static void Main()
 {
 Console.WriteLine("Modern uygulama log gönderiyor...");
 // 1. Eski sistemden bir nesne oluşturuyoruz.
 OldLogger oldSys = new OldLogger();
 // 2. Eski nesneyi adaptör içine "paketliyoruz".
 // C#'ta sahiplik devri (move) gerekmez, referans aktarılır.
 ILogger modernLogger = new LoggerAdapter(oldSys);
 // 3. Uygulama artık eski sistemi yeni arayüz üzerinden kullanabilir.
 AppLog(modernLogger);
 }
}
/*Program Çıktısı:
Modern uygulama log gönderiyor...
[Eski Kayıt Sistemi]: Sistem başarıyla çalıştı.
*/

```

### §18.3.3 Bileşik

**Bileşik deseni (composite pattern)**, özyinelemeli (**recursive**) ya da hiyerarşik yapıya sahip nesne oluşturmak gerektiğinde yapısal olarak nasıl bir kodlama yöntemi izleneceğini söyler. Bu desen, her nesnenin bağımsız olarak veya aynı ara yüz üzerinden iç içe geçmiş nesneler kümesi olarak ele alınabileceği nesne hiyerarşilerinin oluşturulmasını kolaylaştırır.



Şekil 18.8: Bileşik Deseni UML Diyagramı

Bu desenin görevi, nesneleri "parça-bütün" hiyerarşisinde (ağaç yapısı) düzenler. Tekil nesne ile nesne gruplarına aynı şekilde davranılmasını sağlar. Bir klasör örneği üzerinden ilerlersek;

- ◊ Bileşen (**Component**), hem dosyaların hem klasörlerin ortak özelliklerini tanımlayan arayüz.
- ◊ Yaprak (**Leaf**), En küçük birim (Örnek olarak Dosya).
- ◊ Bileşik (**Composite**), içinde başka birimler barındırabilen yapı (Örnek olarak Dosya içerebilen Klasör).

Şimdi de desenimizi kodlayalım;

```

using System.Collections.Generic;
using System.Linq;

```



```
// 1. --- Abstract Component ---
public abstract class Component
{
 public Component? Parent { get; set; }
 public virtual bool IsComposite() => false;
 // Varsayılan boş gövdeler (Yapraklarda hata vermemesi için)
 public virtual void AddChild(Component component) { }
 public virtual void RemoveChild(Component component) { }
 public abstract string DisplayOperation();
}

// 2. --- Leaf (Yaprak) ---
public class Leaf : Component
{
 public override string DisplayOperation() => "Yaprak";
}

// 3. --- Composite (Dal / Bileşen Grubu) ---
public class Composite : Component
{
 // Çocukların listesi. C#'ta bellek yönetimi otomatik olduğu için
 // pointer yönetimine gerek yoktur.
 private readonly List<Component> _children = new List<Component>();
 public override void AddChild(Component component)
 {
 component.Parent = this;
 _children.Add(component);
 }
 public override void RemoveChild(Component component)
 {
 _children.Remove(component);
 component.Parent = null;
 }
 public override bool IsComposite() => true;
 public override string DisplayOperation()
 {
 // LINQ kullanarak çocukların operasyonlarını birleştiriyoruz.
 var results = _children.Select(c => c.DisplayOperation());
 string combinedResult = string.Join("+", results);
 return $"Dal({combinedResult})";
 }
}

// 4. --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 // 1. Sadece yaprak örneği
 var yaprak = new Leaf();
 Console.WriteLine("Sadece Yapraktan Oluşan Nesne: " + yaprak.DisplayOperation());
 // 2. Karmaşık ağaç yapısı
 var kok = new Composite();
 var dal1 = new Composite();
 dal1.AddChild(new Leaf()); // Yaprak 1
 var yaprak2 = new Leaf(); // İleride silmek istersek referansını tutarız
 dal1.AddChild(yaprak2);
 kok.AddChild(dal1);
 Console.WriteLine("Yapraktan ve Dallardan Oluşan Nesne: " + kok.DisplayOperation());
 // 3. Bir dal daha ekleme
 var dal2 = new Composite();
 dal2.AddChild(new Leaf());
 kok.AddChild(dal2);
 Console.WriteLine("Bir dal daha eklenmiş Nesne: " + kok.DisplayOperation());
 }
}
```

```

 // C#'ta 'delete' yoktur; 'kok' referansı bittiğinde tüm ağaç GC tarafından temizlenir.
 }
}
/*Program Çıktısı:
Sadece Yapraktan Oluşan Nesne: Yaprak
Yapraktan ve Dallardan Oluşan Nesne: Dal(Dal(Yaprak+Yaprak))
Bir dal daha eklenmiş Nesne: Dal(Dal(Yaprak+Yaprak)+Dal(Yaprak))
*/

```

Bu desene gerçek dünya örneği olarak aşağıdaki dosya sistemini gösteren örnek verilebilir;

```

using System;
using System.Collections.Generic;
using System.Linq;

// 1. --- Component (Bileşen) ---
public abstract class FileSystemNode
{
 public abstract void ShowDetails(int indent = 0);
 public abstract long GetSize();
}

// 2. --- Leaf (Yaprak) ---
public class File : FileSystemNode
{
 private string _name;
 private long _size;
 public File(string name, long size)
 {
 _name = name;
 _size = size;
 }
 public override void ShowDetails(int indent)
 {
 Console.WriteLine($"{new string(' ', indent)}- Dosya: {_name} ({_size} KB)");
 }
 public override long GetSize() => _size;
}

// 3. --- Composite (Bileşik) ---
public class Directory : FileSystemNode
{
 private string _name;
 private List<FileSystemNode> _children = new List<FileSystemNode>();
 public Directory(string name)
 {
 _name = name;
 }
 public void Add(FileSystemNode node)
 {
 _children.Add(node);
 }
 public override void ShowDetails(int indent=0)
 {
 // LINQ kullanarak toplam boyutu o anki çocuklardan hesaplıyoruz
 Console.WriteLine($"{new string(' ', indent)}+ Klasör: {_name} [Toplam: {GetSize()}
 ↳ KB]");
 foreach (var child in _children)
 {
 // Özyinelemeli (Recursive) çağrı
 child.ShowDetails(indent + 4);
 }
 }
 public override long GetSize()
 {

```

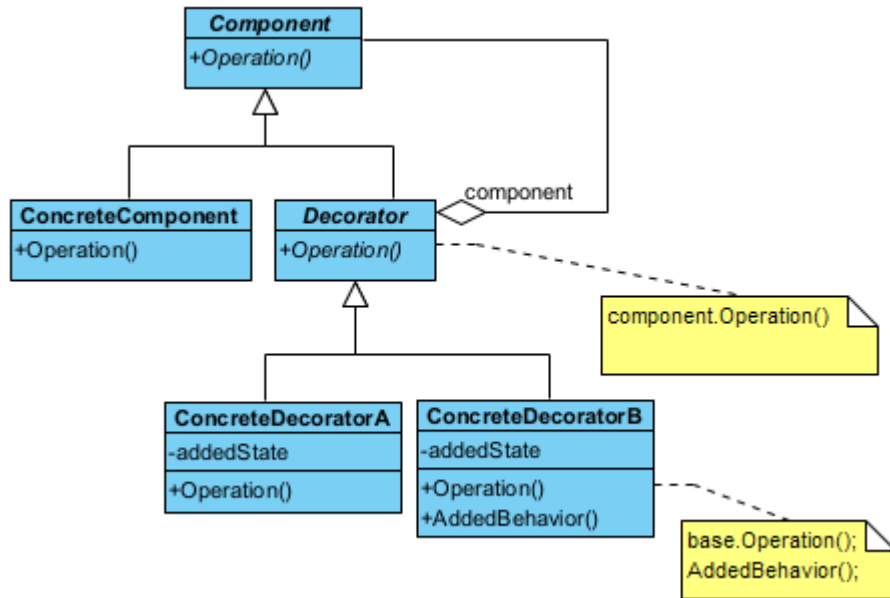
```

 // LINQ'nun Sum metodu ile çocukların boyutlarını topluyoruz
 return _children.Sum(child => child.GetSize());
 }
}
// 4. --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 // Klasörleri oluşturalım
 var root = new Directory("C_Surucusu");
 var docs = new Directory("Belgelerim");
 // Dosyaları ekleyelim
 docs.Add(new File("Notlar.txt", 10));
 root.Add(docs);
 root.Add(new File("Sistem.log", 500));
 root.ShowDetails();
 }
}
/*Program Çıktısı:
+ Klasör: C_Surucusu [Toplam: 510 KB]
+ Klasör: Belgelerim [Toplam: 10 KB]
- Dosya: Notlar.txt (10 KB)
- Dosya: Sistem.log (500 KB)
*/

```

### §18.3.4 Süsleyici

Süsleyici deseninde (**decorator pattern**), bir nesneye dinamik olarak yeni durum ve davranışları eklemek mümkündür. Yani çalıştırma anında, bir nesnenin sahip olduğu yeteneklere, yeni yeteneklerin eklenmesini sağlar.



Şekil 18.9: Süsleyici Deseni UML Diyagramı

Bu tasarım deseni, bir nesneye dinamik olarak (çalışma zamanında) yeni sorumluluklar veya özellikler eklemek için kullanılan yapısal bir desendir. Kalıtımın (**inheritance**) aksine, özellikleri sınıf hiyerarşisini karmaşıktırmadan, nesneleri birbirinin içine "sarmalayarak" (**wrapping**) ekler. Bunu bir Rus matruşka bebeği veya bir kahve siparişine eklenen ekstralar (süt, şeker, karamel) gibi düşünebilirsin.

- ◆ Bileşen (**Component**), hem temel nesnenin hem de süsleyicilerin uyması gereken ortak soyut arayüzü tanımlar. Temel işlevin (Örnek olarak `ucretHesapla()`) imzasını tanımlar.
- ◆ Somut Bileşen (**ConcreteComponent**), üzerine eklemeye yapılacak olan "temel" nesnedir. Fonksiyonun en yalın halini gerçekleştirir (Örnek olarak Sade Kahve).
- ◆ Soyut Süsleyici (**Decorator**), süsleyicilerin soyut temel yapısını kurar. İçinde bir **Component** referansı

tutar. Bu referans sayesinde hem temel nesneyi hem de başka bir süsleyiciyi sarmalayabilir.

- ◊ Somut Süsleyiciler (**ConcreteDecorator**), gerçek özellikleri ekleyen sınıflardır. Temel nesnenin metodunu çağırır ve üzerine kendi eklemesini yapar (Örnek olarak Kahve ücretine +5 TL süt bedeli eklemek).

Şimdi desenimizi kodlayalım;

```
using System;

// 1. Component (Bileşen): Ortak arayüz
public abstract class Component
{
 public abstract void Operation();
}

// 2. ConcreteComponent (Somut Bileşen): Temel nesne
public class ConcreteComponent : Component
{
 public override void Operation()
 {
 Console.WriteLine("Temel Ürün İşlemi");
 }
}

// 3. Decorator (Süsleyici): Sarmalayıcı temel sınıf
public abstract class Decorator : Component
{
 protected Component _component;
 protected Decorator(Component component)
 {
 _component = component;
 }
 public override void Operation()
 {
 // Eğer bir bileşen sarmalanmışsa onun işlemini çağır
 _component?.Operation();
 }
}

// 4. ConcreteDecorator1: Yeni Durum Ekleyen Süsleyici
public class ConcreteDecorator1 : Decorator
{
 private int _addedState;
 public ConcreteDecorator1(Component component, int state) : base(component)
 {
 _addedState = state * 100;
 }
 public override void Operation()
 {
 base.Operation(); // Önceki nesnenin (zincirin) davranışını çağır
 Console.WriteLine($"-> Decorator 1: Eklendi (State: {_addedState})");
 }
}

// 5. ConcreteDecorator2: Yeni Davranış Ekleyen Süsleyici
public class ConcreteDecorator2 : Decorator
{
 public ConcreteDecorator2(Component component) : base(component) { }
 private string AddedBehavior()
 {
 return "Bu metin Yeni Davranıştan Geliyor...";
 }
 public override void Operation()
 {
 base.Operation(); // Önceki nesnenin davranışını çağır
 Console.WriteLine($"-> Decorator 2: Yeni Davranış Tetiklendi: {AddedBehavior()}");
 }
}
```

```

}
// 6. Client (İstemci)
class Program
{
 static void Main()
 {
 Console.WriteLine("--- Dekoratör Zinciri Oluşturuluyor ---");
 // 1. Temel ürün oluştur
 Component myProduct = new ConcreteComponent();
 // 2. Ürünü Decorator 1 ile süsle
 // C#'ta 'new' kullanarak nesneleri iç içe geçirmek çok doğaldır.
 var decorated1 = new ConcreteDecorator1(myProduct, 2);
 // 3. Ortaya çıkan nesneyi Decorator 2 ile tekrar süsle
 var decorated2 = new ConcreteDecorator2(decorated1);
 // Tek bir çağrı tüm zinciri (içten dışa) tetikler
 decorated2.Operation();
 // C#'ta bellek yönetimi otomatik olduğu için manuel temizliğe gerek yoktur.
 }
}
/*Program Çıktısı:
--- Dekoratör Zinciri Oluşturuluyor ---
Temel Ürün İşlemi
-> Decorator 1: Eklendi (State: 200)
-> Decorator 2: Yeni Davranış Tetiklendi: Bu metin Yeni Davranıştan Geliyor...
*/

```

Aşağıda gerçek dünya örneği olarak sipariş edilen kahveyi süsleyen bir örnek verilmiştir;

```

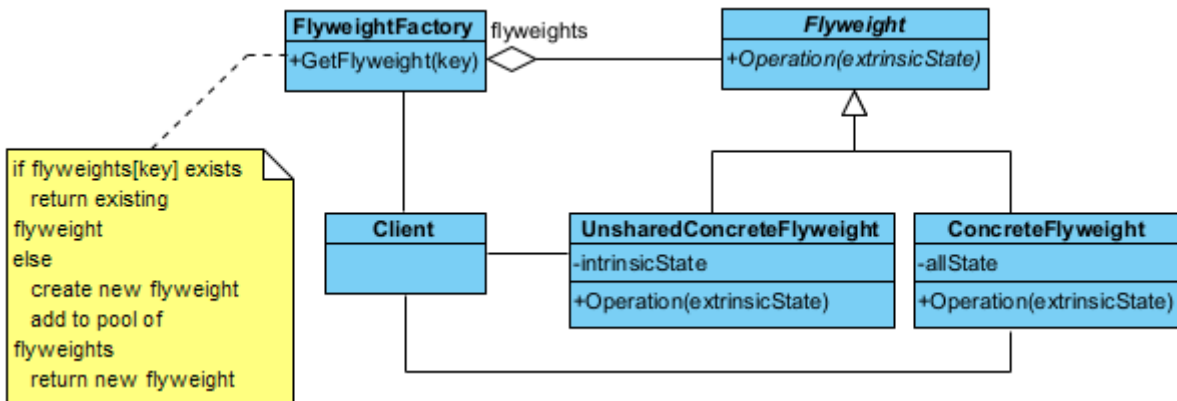
// 1. --- Base Component (Temel Bileşen) ---
public abstract class Coffee
{
 public abstract double GetCost();
 public abstract string GetDescription();
}
// 2. --- Concrete Component (Somut Bileşen) ---
public class SimpleCoffee : Coffee
{
 public override double GetCost() => 5.0;
 public override string GetDescription() => "Sade Kahve";
}
// 3. --- Base Decorator (Temel Süsleyici) ---
public abstract class CoffeeDecorator : Coffee
{
 protected Coffee _coffee;
 // C#'ta dekoratör sarmaladığı nesneyi referans olarak tutar
 protected CoffeeDecorator(Coffee coffee)
 {
 _coffee = coffee;
 }
}
// 4. --- Concrete Decorators (Somut Süsleyiciler) ---
public class MilkDecorator : CoffeeDecorator
{
 public MilkDecorator(Coffee coffee) : base(coffee) { }
 public override double GetCost() => _coffee.GetCost() + 1.5;
 public override string GetDescription() => _coffee.GetDescription() + ", süt";
}
public class SugarDecorator : CoffeeDecorator
{
 public SugarDecorator(Coffee coffee) : base(coffee) { }
 public override double GetCost() => _coffee.GetCost() + 0.5;
 public override string GetDescription() => _coffee.GetDescription() + ", şeker";
}

```

```
// 5. --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 // 1. Sade bir kahve oluşturuyoruz
 Coffee kahvem = new SimpleCoffee();
 // 2. Kahveyi süt ile sarmalıyoruz
 kahvem = new MilkDecorator(kahvem);
 // 3. Mevcut (sütlü) kahveyi şeker ile sarmalıyoruz
 kahvem = new SugarDecorator(kahvem);
 // Sonuçları yazdıralım
 Console.WriteLine($"Sipariş: {kahvem.GetDescription()}");
 Console.WriteLine($"Toplam Fiyat: {kahvem.GetCost()} TL");
 }
}
/*Program Çıktısı:
Sipariş: Sade Kahve, süt, şeker
Toplam Fiyat: 7 TL
*/
```

### §18.3.5 Sinek Sıklet

**Sinek Sıklet deseninde** (**flyweight pattern**), çok sayıda benzer nesnenin bulunduğu durumlarda bellek kullanımını minimize etmek için kullanılan yapısal (structural) bir desendir.



Şekil 18.10: Sinek Sıklet Deseni UML Diyagramı

Bu desenin temel stratejisi, nesnelerin ortak verilerini (**intrinsic state**) paylaşarak bellekte tek bir kopyasını tutmak, her nesneye özgü değişen verileri (**extrinsic state**) ise dışarıdan sağlamaktır.

- ◇ Soyut Sinek Sıklet (**Flyweight**), paylaşılan nesnelerin uyması gereken arayüzü tanımlar. Nesneye özgü verilerin (**extrinsic state**) parametre olarak geçildiği yöntemin imzasını barındırır.
- ◇ Somut Sinek Sıklet (**ConcreteFlyweight**), paylaşılan veriyi (**intrinsic state**) kendi içinde saklayan nesnedir. Bu nesneler "değişmez" (**immutable**) olmalıdır. Örneğin; bir oyun için "Ağaç" modelinin 3D verisi burada bir kez tutulur.
- ◇ Sinek Sıklet Fabrikası (**FlyweightFactory**), **Flyweight** nesnelerini yönetir ve paylaşılmasını sağlar. İstendiğinde, eğer nesne daha önce oluşturulmuşsa mevcut olanı verir; yoksa yenisini oluşturup havuza ekler.
- ◇ Bağlam (**Context**), her nesneye özgü, sürekli değişen verileri (konum, renk gibi) tutar. Somut **ConcreteFlyweight** nesnesini bu verilerle birlikte kullanarak asıl işlemi gerçekleştirir.

Şimdi desenimizi kodlayalım;

```
using System;
using System.Collections.Generic;
// 1. Flyweight Arayüzü
public abstract class Flyweight
{
 // İçsel Durum (Intrinsic State): Nesneler arasında ortak olan ve değişmeyen veri
 protected string IntrinsicState;
 protected Flyweight(string state)
```

```

 {
 IntrinsicState = state;
 }
 public abstract void Operation(string extrinsicState);
}
// 2. Somut Paylaşılan Flyweight (Concrete Flyweight)
public class ConcreteFlyweight : Flyweight
{
 public ConcreteFlyweight(string state) : base(state) { }
 public override void Operation(string extrinsicState)
 {
 // Dışsal Durum (Extrinsic State): İşlem anında dışarıdan gelen, değişken veri (konum,
 // ↪ renk vb.)
 Console.WriteLine($"[Paylaşılan] İç Durum: {IntrinsicState} | Dış Durum:
 // ↪ {extrinsicState}");
 }
}
// 3. Flyweight Factory (Fabrika)
public class FlyweightFactory
{
 private readonly Dictionary<string, Flyweight> _flyweights = new Dictionary<string,
 // ↪ Flyweight>();
 public Flyweight GetFlyweight(string key)
 {
 // Eğer bu anahtara sahip nesne zaten varsa onu döndür
 if (_flyweights.ContainsKey(key))
 {
 Console.WriteLine($"-> Fabrika: {key} zaten var, mevcut olan döndürülüyor.");
 return _flyweights[key];
 }
 // Yoksa yeni bir tane oluştur ve depola
 Console.WriteLine($"-> Fabrika: {key} bulunamadı, yeni oluşturuluyor.");
 var flyweight = new ConcreteFlyweight(key);
 _flyweights.Add(key, flyweight);
 return flyweight;
 }
 public void ListFlyweights()
 {
 Console.WriteLine($"\\nFabrikadaki toplam nesne sayısı: {_flyweights.Count}");
 foreach (var key in _flyweights.Keys)
 {
 Console.WriteLine($"- {key}");
 }
 }
}
// 4. Client (İstemci)
class Program
{
 static void Main(string[] args)
 {
 FlyweightFactory factory = new FlyweightFactory();
 // Aynı nesneleri tekrar tekrar isteyelim
 Flyweight v1 = factory.GetFlyweight("Binek Araç");
 Flyweight v2 = factory.GetFlyweight("Kamyon");
 Flyweight v3 = factory.GetFlyweight("Binek Araç"); // Fabrikadan mevcut olan gelecek
 // Dışsal durumları (konum gibi) işlem sırasında gönderiyoruz
 v1.Operation("Konum: 41.00, 28.97");
 v2.Operation("Konum: 39.93, 32.85");
 v3.Operation("Konum: 40.71, -74.00");
 factory.ListFlyweights();
 }
}

```



```

/* Program Çıktısı:
-> Fabrika: Binek Araç bulunamadı, yeni oluşturuluyor.
-> Fabrika: Kamyon bulunamadı, yeni oluşturuluyor.
-> Fabrika: Binek Araç zaten var, mevcut olan döndürülüyor.
[Paylaşılan] İç Durum: Binek Araç | Dış Durum: Konum: 41.00, 28.97
[Paylaşılan] İç Durum: Kamyon | Dış Durum: Konum: 39.93, 32.85
[Paylaşılan] İç Durum: Binek Araç | Dış Durum: Konum: 40.71, -74.00

```

Fabrikadaki toplam nesne sayısı: 2

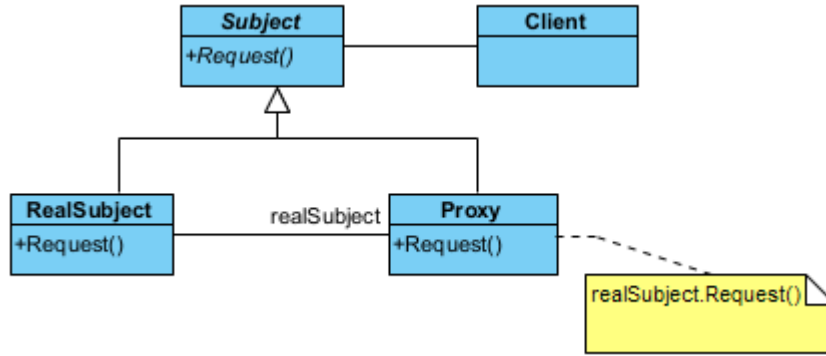
```

- Binek Araç
- Kamyon
*/

```

### §18.3.6 Vekil

**Vekil deseni (proxy pattern)**, kullanılacak ve erişilecek bir nesnenin yerine geçen bir başka nesneye ihtiyaç duyulduğunda kullanılır. Bu desen, başka bir nesneye erişimi kontrol etmek için kullanılan yapısal bir desendir. Bir nesnenin önüne "vekil" bir katman koyarak, asıl nesneye erişmeden önce güvenlik kontrolü yapılabilir, nesnenin yüklenmesini geciktirebilir (**lazy loading**) veya işlem sonuçlarını önbelleğe alabilirsiniz.



Şekil 18.11: Vekil Deseni UML Diyagramı

Bu deseni bir şirketteki "Yönetici Asistanı" gibi düşünebilirsin. Yöneticiyle doğrudan görüşemezsin; önce asistanla (**Proxy**) görüşürsün, asistan uygun görürse seni yöneticiye (**RealSubject**) yönlendirir.

- ◊ Bağlam (**Subject**), hem asıl nesnenin hem de vekilin (**Proxy**) uyması gereken ortak soyut arayüzü tanımlar. İstemcinin (**Client**) her iki nesneye de aynı şekilde davranabilmesini sağlar.
- ◊ Gerçek Bağlam (**RealSubject**), asıl işi yapan, asıl mantığı barındıran nesnedir. Genellikle oluşturulması maliyetli veya erişimi kısıtlı olan nesnedir.
- ◊ Vekil (**Proxy**), gerçek nesneye olan referansı kendi içinde saklar. İstekleri gerçek nesneye iletmeden önce araya girer. Erişim kontrolü yapabilir, gerçek nesne henüz oluşturulmamışsa onu oluşturabilir veya gelen isteği reddedebilir.
- ◊ İstemci (**Client**), nesnenin doğrudan kendisine değil, vekil (**Proxy**) nesnesine ulaşır ve bu nesneye isteklerini iletir.

Şimdi desenimizi kodlayalım;

```

using System;
// 1. Subject (Özne Arayüzü): Gerçek nesne ve vekilin ortak kontratı
public interface ISubject
{
 void Request();
}
// 2. RealSubject (Gerçek Özne): Asıl iş mantığını barındıran sınıf
public class RealSubject : ISubject
{
 private readonly int _state;
 public RealSubject(int state)
 {
 _state = state;
 }
 public void Request()
 {

```

```

 Console.WriteLine($"RealSubject: İstek işleniyor (Durum: {_state})");
 }
}
// 3. Proxy (Vekil): Gerçek nesneye erişimi denetleyen aracı
public class Proxy : ISubject
{
 private RealSubject? _realSubject;
 private readonly int _defaultState;
 // Senaryo 1: Tembel Yükleme (Lazy Initialization) için varsayılan yapıcı
 public Proxy()
 {
 _defaultState = 20;
 }
 // Senaryo 2: Mevcut bir nesne üzerinden vekillik yapmak
 public Proxy(RealSubject realSubject)
 {
 _realSubject = realSubject;
 }
 private bool CheckAccess()
 {
 Console.WriteLine("Proxy: Erişim yetkisi kontrol ediliyor...");
 return true;
 }
 private void LogAccess()
 {
 Console.WriteLine("Proxy: İşlem günlüğü (log) kaydedildi.");
 }
 public void Request()
 {
 if (CheckAccess())
 {
 // Nesne henüz oluşturulmamışsa (Lazy Initialization) burada oluşturulur
 if (_realSubject == null)
 {
 _realSubject = new RealSubject(_defaultState);
 }
 _realSubject.Request();
 LogAccess();
 }
 }
}
// 4. Client (İstemci)
class Program
{
 static void Main()
 {
 // 1. Doğrudan erişim
 Console.WriteLine("--- Doğrudan RealSubject Kullanımı ---");
 ISubject gercekOzne = new RealSubject(10);
 gercekOzne.Request();
 // 2. Kendi nesnesini yöneten vekil (Lazy Proxy)
 Console.WriteLine("\n--- Proxy (Yeni Nesne İle / Lazy) ---");
 ISubject vekil1 = new Proxy();
 vekil1.Request();
 // 3. Mevcut nesne üzerinden vekillik yapan vekil
 Console.WriteLine("\n--- Proxy (Mevcut Nesne Referansı İle) ---");
 ISubject vekil2 = new Proxy((RealSubject)gercekOzne);
 vekil2.Request();
 }
}
/* Program Çıktısı:

```

```

--- Doğrudan RealSubject Kullanımı ---
RealSubject: İstek işleniyor (Durum: 10)

--- Proxy (Yeni Nesne İle / Lazy) ---
Proxy: Erişim yetkisi kontrol ediliyor...
RealSubject: İstek işleniyor (Durum: 20)
Proxy: İşlem günlüğü (log) kaydedildi.

--- Proxy (Mevcut Nesne Referansı İle) ---
Proxy: Erişim yetkisi kontrol ediliyor...
RealSubject: İstek işleniyor (Durum: 10)
Proxy: İşlem günlüğü (log) kaydedildi.
*/

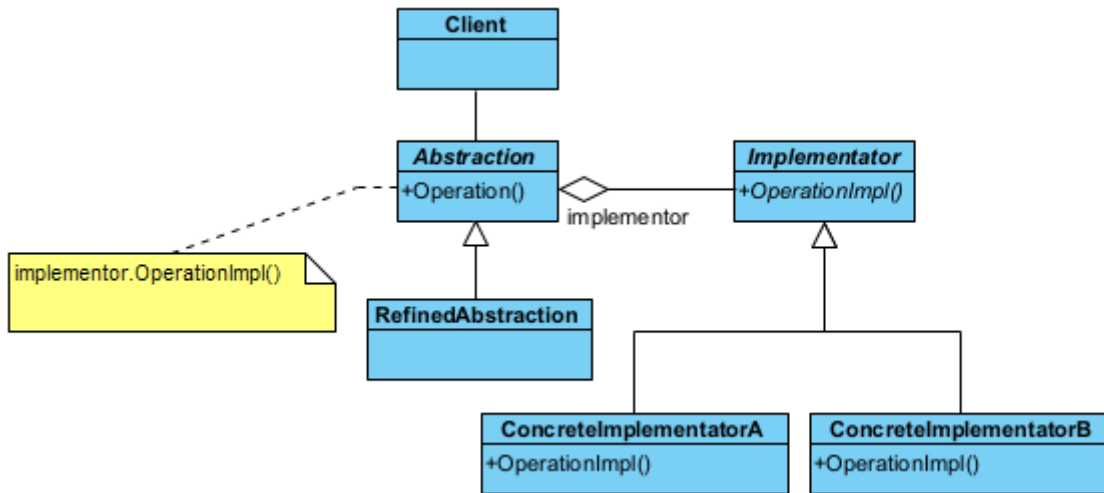
```

Bu desenle ilgili şu kavramlardan bahsetmek gerekir;

- ◊ Bir nesne, bulunduğu yerden uzaktaki bir nesneyi kullanacaksa uzak vekil (**remote proxy**) kullanılır. Gerçek nesne çok ağırsa (örneğin büyük bir resim), nesneyi sadece gerçekten ihtiyaç duyulduğunda (**lazy loading**) oluşturur.
- ◊ Bir nesneye başka nesneler tarafından erişim kısıtlanacaksa koruma vekili (**protection proxy**) olarak adlandırılır. Yetkilendirme (**authentication**) kontrolü yapar.
- ◊ Bir nesneye erişim sırasında başka işlemler yapılacaksa vekil, akıllı referans (**smart reference**) olarak adlandırılır. Böyle durumda vekil, nesneye bir erişim yapıldığında, diğer nesnelerin erişmemesi için kullanılacak nesneye kilit koyabilir. Akıllı referans, akıllı gösterici (**smart pointer**) olarak da adlandırılır. Nesne başka bir sunucudaysa ağ trafiğini yönetir.

### §18.3.7 Köprü

**Köprü deseni (bridge pattern)**, işi yapan alt yüklenici veya taşeron (**implementator**) nesne ile işi yaptıran nesne (**client**) arasındaki bağımlılığı soyutlama ile ortadan kaldırır. Yani birbiriyle oldukça iç içe olan kodlama ile soyutlamayı ortak ara yüz ile birbirinden uzaklaştırır. Sistemin genişlemesine izin verir. Bu desen, yazılımlarda oldukça çok kullanılan bir desendir. Çünkü taşeronlar değişse de yeni taşeron eklense de istemci tarafında bir şey değişmez.



Şekil 18.12: Köprü Deseni UML Diyagramı

Bu tasarım deseni, bir nesnenin soyutlamasını (**abstraction**), onun uygulamasından (**implementation**) ayırarak her ikisinin de birbirinden bağımsız olarak geliştirilmesini sağlayan yapısal bir desendir. Bu desen, özellikle **sınıf patlaması (class explosion)** dediğimiz, her yeni özellik için onlarca alt sınıf türetmek zorunda kaldığımız durumlarda imdadımıza yetişir. Kalıtım yerine bileşimi (**composition**) kullanır.

- ◊ Soyutlama (**Abstraction**), istemcinin (**Client**) kullandığı üst düzey kontrol arayüzüdür. İçinde bir **Implementor** referansı tutar. İstemciden gelen komutları bu referansa iletir.
- ◊ Geliştirilmiş Soyutlama (**RefinedAbstraction**),soyutlamayı genişleten alt sınıflardır. Temel arayüzü kullanarak daha spesifik kontrol mantıkları ekler (Örn: Standart Kumanda→Gelişmiş Akıllı Kumanda).
- ◊ Taşeron Arayüzü (**Implementor**), tüm somut uygulama sınıfları için ortak arayüzü tanımlar. Genellikle düşük seviyeli (ilkel) operasyonları barındırır (Örn: **cihazAc()**, **sesAyari()**).
- ◊ Somut Taşeronlar (**ConcreteImplementor**), **Implementor** arayüzünü gerçekleyen asıl cihazlardır. Plat-forma veya cihaza özgü gerçek kodları barındırır (Örn: Televizyon, Radyo).

Şimdi desenimizi kodlayalım;

```
using System;
// 1. --- Implementor (Uygulayıcı / Taşeron Arayüzü) ---
public interface IImplementor
{
 void OperationImpl();
}
// 2. --- Concrete Implementors ---
public class ConcreteImplementorA : IImplementor
{
 public void OperationImpl()
 {
 Console.WriteLine("A Taşeronu tarafından yapılan iş...");
 }
}
public class ConcreteImplementorB : IImplementor
{
 public void OperationImpl()
 {
 Console.WriteLine("B Taşeronu tarafından yapılan iş...");
 }
}
// 3. --- Abstraction (Soyut Yüklenici) ---
public abstract class Abstraction
{
 protected IImplementor _implementor;
 protected Abstraction(IImplementor implementor)
 {
 _implementor = implementor;
 }
 // Çalışma anında taşeronu değiştirme (Property veya Method ile yapılabilir)
 public void SetImplementor(IImplementor implementor)
 {
 _implementor = implementor;
 }
 public virtual void Operation()
 {
 if (_implementor != null)
 {
 Console.WriteLine("Yüklenici Taşerona iş yaptırıyor:");
 _implementor.OperationImpl();
 }
 }
}
// 4. --- Refined Abstraction (Geliştirilmiş Soyutlama) ---
public class RefinedAbstraction : Abstraction
{
 public RefinedAbstraction(IImplementor implementor) : base(implementor) { }
}
// 5. --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 // 1. Yükleniciyi ilk taşeronla (A) başlatıyoruz
 var yuklenici = new RefinedAbstraction(new ConcreteImplementorA());
 yuklenici.Operation();
 Console.WriteLine("--- Taşeron Değiştiriliyor ---");
 // 2. Çalışma anında yeni bir taşerona (B) geçiyoruz
 yuklenici.SetImplementor(new ConcreteImplementorB());
 yuklenici.Operation();
 }
}
```

```

}
/* Program Çıktısı:
Yüklenici Taşeronu iş yaptırıyor:
A Taşeronu tarafından yapılan iş...
--- Taşeron Değiştiriliyor ---
Yüklenici Taşeronu iş yaptırıyor:
B Taşeronu tarafından yapılan iş...
*/

```

Köprü deseni aşağıdaki durumlarda kullanılır;

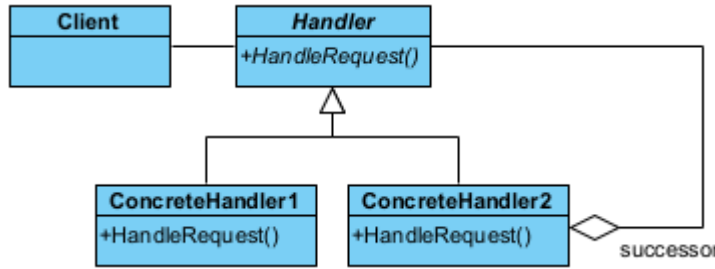
- ◊ Taşeron nesneye kalıcı bir bağımlılık istenmediği durumlar. Çalışma anında (**run-time**) taşeronlar arasında geçiş yapmayı sağlar.
- ◊ Birden çok taşeron kullanarak sistemin genişlemesine ihtiyaç olduğu durumlar.
- ◊ Taşeronlar üzerindeki değişiklikler, istemciyi etkilemez. İstemci tarafında derleme olmadan yazılımın değiştirilmesi sağlanır.
- ◊ Projede aynı anda farklı birden çok taşeron tasarımı yapılabildiğinden, proje parçalara ayrılarak kendi içinde hızla sınıf tasarımı yapılabilir. Bu durum *Rumbaugh* tarafından iç içe genelleştirme (**nested generalization**) olarak adlandırılmıştır.

## 18.4 Davranışla İlgili Desenler

**Davranışla ilgili desenler** (**behavioral patterns**) nesnelerin çeşitli olaylar karşısında gösterecekleri davranışlara çözüm getirmektedirler.

### §18.4.1 Sorumluluk Zinciri

Bazı durumlarda bir nesne, kendinin yapmayacağı bir işlemi halefine (**successor**) devredebilir. Ya da nesnenin kendi sorumluluğunu aşan durumlarda ilgili diğer nesneye işlem yaptırılır. İşte bu durumda **sorumluluk zinciri deseni** (**chain of responsibility pattern**) kullanılır.



Şekil 18.13: Sorumluluk Zinciri Deseni UML Diyagramı

Bu desenin temel amacı, isteği gönderen ile isteği işleyen arasındaki bağı gevşetmektir (**decoupling**). İstek, zincir boyunca ilerler ve her bir işleyici ya isteği çözer ya da bir sonraki işleyiciye paslar.

- ◊ İşleyici (**Handler**), İsteklerin nasıl işleneceğine dair ortak bir arayüz tanımlar. İşleyici, bir sonraki işleyiciyi tutan bir referans barındırır ve zincirin halkalarını birbirine bağlar.
- ◊ Somut İşleyiciler (**ConcreteHandler**), kendisine gelen isteği kontrol eder. Eğer istek kendi sorumluluk alanındaysa işi bitirir. Eğer değilse (veya işin bir kısmını yapıp devam etmesi gerekiyorsa), isteği zincirdeki bir sonraki halkaya iletir.
- ◊ İstemci (**Client**), zinciri oluşturur (halkaları birbirine bağlar) ve ilk isteği zincirin başına gönderir. İsteğin kim tarafından, nasıl çözüleceğiyle ilgilenmez; sadece "çözülmesini" bekler.

Şimdi desenimizi kodlayalım;

```

using System;
// 1. Soyut İşleyici (Handler)
public abstract class Handler
{
 // C#'ta nesne ömrü referans bazlıdır, unique_ptr yerine basit bir alan yeterlidir.
 protected Handler? Successor;
 public void SetSuccessor(Handler next)
 {
 Successor = next;
 }
 public abstract void HandleRequest(int request);
}

```

```
// 2. Somut İşleyici 1: Küçük Talepler (Örn: Memur)
public class Clerk : Handler
{
 public override void HandleRequest(int request)
 {
 if (request < 100)
 {
 Console.WriteLine($"Memur: {request} birimlik talebi onayladı.");
 }
 else if (Successor != null)
 {
 Console.WriteLine("Memur: Talep limitimi aşıyor, Şefe devrediliyor...");
 Successor.HandleRequest(request);
 }
 else
 {
 Console.WriteLine("Hata: Talebi karşılayacak kimse bulunamadı!");
 }
 }
}

// 3. Somut İşleyici 2: Büyük Talepler (Örn: Şef)
public class Manager : Handler
{
 public override void HandleRequest(int request)
 {
 if (request >= 100 && request < 500)
 {
 Console.WriteLine($"Şef: {request} birimlik talebi onayladı.");
 }
 else if (Successor != null)
 {
 Console.WriteLine("Şef: Talep limitimi aşıyor, Müdüre devrediliyor...");
 Successor.HandleRequest(request);
 }
 else
 {
 Console.WriteLine("Şef: Bu kadar büyük bir yetkim yok ve başka amir tanımlı değil!");
 }
 }
}

// 4. Client (İstemci)
class Program
{
 static void Main()
 {
 // Zinciri oluşturuyoruz
 var clerk = new Clerk();
 var manager = new Manager();
 // Memurun halefi Şef olsun
 clerk.SetSuccessor(manager);
 Console.WriteLine("--- Talep 1 (50) ---");
 clerk.HandleRequest(50);
 Console.WriteLine("--- Talep 2 (250) ---");
 clerk.HandleRequest(250);
 Console.WriteLine("--- Talep 3 (1000) ---");
 clerk.HandleRequest(1000);
 }
}

/* Program Çıktısı:
--- Talep 1 (50) ---
Memur: 50 birimlik talebi onayladı.
--- Talep 2 (250) ---
```

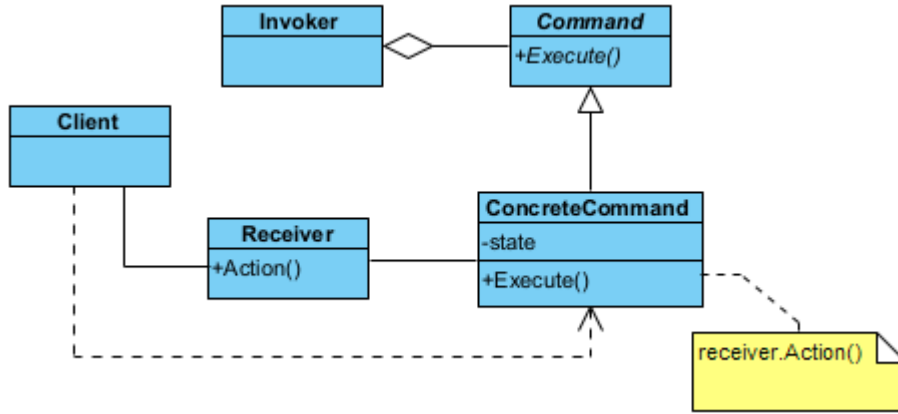
```

Memur: Talep limitimi aşıyor, Şefe devrediliyor...
Şef: 250 birimlik talebi onayladı.
--- Talep 3 (1000) ---
Memur: Talep limitimi aşıyor, Şefe devrediliyor...
Şef: Bu kadar büyük bir yetkim yok ve başka amir tanımlı değil!
*/

```

### §18.4.2 Komut

Bazı durumlarda bir nesne diğer bir nesneye ileti (**message**) verip bir işlem başlattığında, bu işlem bitmeden başka işlemler yapmak isteyebilir. İşte bu tür durumlarda **komut deseni** (**command pattern**) kullanılır.



Şekil 18.14: Komut Deseni UML Diyagramı

Bu tasarım deseni, bir isteği veya eylemi kendi başına bir nesneye dönüştüren davranışsal bir desendir. Bu dönüşüm sayesinde eylemleri parametre olarak geçebilir, kuyruğa alabilir (**queue**), iz kaydını tutabilir (**log**) veya geri alabilirsiniz (**undo**). Kısacası; "ne yapılacağı" bilgisini, "kimin yapacağı" bilgisinden tamamen ayırır.

- ◊ Komut (**Command**), tüm komutların uyması gereken soyut arayüzü tanımlar. Genellikle tek bir yöntem barındırır (Örnek olarak **execute()**). Bazı durumlarda geri alma işlemi için **undo()** yöntemini de içerir.
- ◊ Somut Komut (**ConcreteCommand**), soyut komutu gerçek bir eylemle birleştirir. Alıcı (**Receiver**) nesnesini tanıır ve **execute()** çağrıldığında alıcının ilgili metodunu tetikler.
- ◊ Alıcı(**Receiver**), işin asıl mutfağıdır. Gerçek mantığı ve operasyonları barındırır. Komut nesnesi ona "çalış" dediğinde, o işi nasıl yapacağını bilir (Örnek olarak Işığı açmak, dosyayı kaydetmek).
- ◊ Çağırıcı(**Invoker**), komutun ne zaman çalışacağına karar verir. Komutu saklar ve kullanıcı bir düğmeye bastığında veya bir olay gerçekleştiğinde komutun **execute()** metodunu çağırır. Alıcıyı doğrudan tanımaz.

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
// 1. Receiver (Alıcı): Asıl iş mantığını barındıran sınıf
public class Receiver
{
 public void Action(string param) => Console.WriteLine($"Receiver: [{param}] komutu için temel
 ↳ işlemler yapılıyor.");
 public void ActionA(string param) => Console.WriteLine($"Receiver: [{param}] için A
 ↳ işlemi.");
 public void ActionB(string param) => Console.WriteLine($"Receiver: [{param}] için B
 ↳ işlemi.");
}
// 2. Command (Komut Arayüzü)
public interface ICommand
{
 void Execute();
}
// 3. Concrete Commands (Somut Komutlar)
public class SimpleCommand : ICommand
{
 private readonly string _state;

```



```

 private readonly Receiver _receiver;
 public SimpleCommand(string state, Receiver receiver)
 {
 _state = state;
 _receiver = receiver;
 }
 public void Execute()
 {
 _receiver.Action(_state);
 }
}
public class ComplexCommand : ICommand
{
 private readonly string _state;
 private readonly Receiver _receiver;
 public ComplexCommand(string state, Receiver receiver)
 {
 _state = state;
 _receiver = receiver;
 }
 public void Execute()
 {
 _receiver.ActionA(_state);
 _receiver.ActionB(_state);
 }
}
// 4. Invoker (Çağırıcı): Komutları saklar ve tetikler
public class Invoker
{
 private readonly List<ICommand> _history = new List<ICommand>();
 public void StoreAndExecute(ICommand cmd)
 {
 cmd.Execute();
 _history.Add(cmd); // Komutu sakla (Geri alma işlemleri için kullanılabilir)
 }
}
// 5. Client (İstemci)
class Program
{
 static void Main()
 {
 // Alıcı ve Çağırıcı nesneler oluşturulur
 Receiver receiver = new Receiver();
 Invoker invoker = new Invoker();

 Console.WriteLine("--- Komutlar Tetikleniyor ---");

 // Komutlar oluşturulur ve Invoker aracılığıyla çalıştırılır
 invoker.StoreAndExecute(new SimpleCommand("Yazdır", receiver));
 invoker.StoreAndExecute(new ComplexCommand("Logla ve Gönder", receiver));
 }
}
/* Program Çıktısı:
--- Komutlar Tetikleniyor ---
Receiver: [Yazdır] komutu için temel işlemler yapılıyor.
Receiver: [Logla ve Gönder] için A işlemi.
Receiver: [Logla ve Gönder] için B işlemi.
*/

```

Aşağıda komut desenine uygun bir ışığın kapatılıp açılması örneği verilmiştir;

```

using System;
using System.Collections.Generic;

```

```
// --- Receiver (Alıcı) ---
public class Light
{
 public void On() => Console.WriteLine("Işık AÇILDI.");
 public void Off() => Console.WriteLine("Işık KAPATILDI.");
}
// --- Command (Soyut Komut) ---
public interface ICommand
{
 void Execute();
 void Undo();
}
// --- Concrete Commands (Somut Komutlar) ---
public class LightOnCommand : ICommand
{
 private readonly Light _light;
 public LightOnCommand(Light light)
 {
 _light = light;
 }
 public void Execute() => _light.On();
 public void Undo() => _light.Off();
}
public class LightOffCommand : ICommand
{
 private readonly Light _light;
 public LightOffCommand(Light light)
 {
 _light = light;
 }
 public void Execute() => _light.Off();
 public void Undo() => _light.On();
}
// --- Invoker (Çağırıcı) ---
public class RemoteControl
{
 private readonly Stack<ICommand> _history = new Stack<ICommand>();
 public void ExecuteCommand(ICommand cmd)
 {
 cmd.Execute();
 _history.Push(cmd);
 }
 public void UndoLast()
 {
 if (_history.Count > 0)
 {
 Console.Write("[UNDO] ");
 // Pop() metodu hem en üstteki elemanı döndürür hem de yığından çıkarır.
 ICommand lastCommand = _history.Pop();
 lastCommand.Undo();
 }
 else
 {
 Console.WriteLine("[UYARI] Geri alınacak işlem yok!");
 }
 }
}
// --- Client (İstemci) ---
class Program
{
 static void Main()
 {

```

```

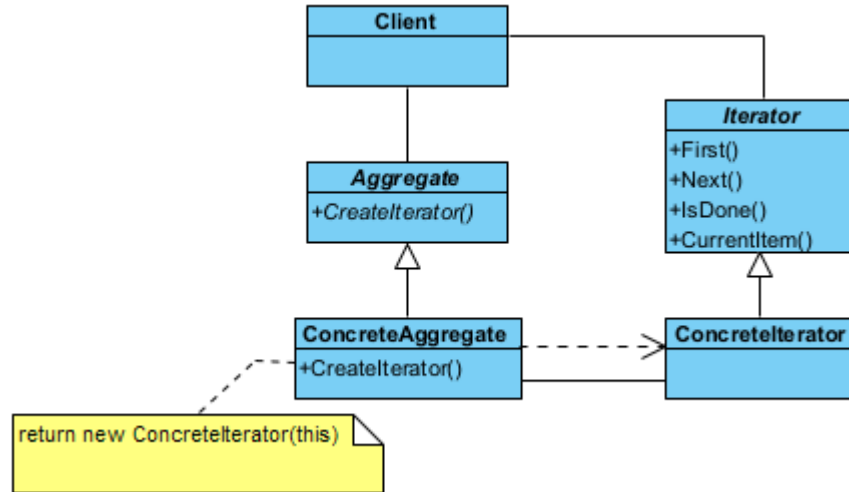
Light oturmaOdasiIsigi = new Light();
RemoteControl kumanda = new RemoteControl();
Console.WriteLine("--- İşlemler Başlıyor ---");
// Işığın açalım
kumanda.ExecuteCommand(new LightOnCommand(oturmaOdasiIsigi));
// Işığın kapatalım
kumanda.ExecuteCommand(new LightOffCommand(oturmaOdasiIsigi));
Console.WriteLine("\n--- Geri Alma (Undo) İşlemleri ---");
kumanda.UndoLast(); // Kapatmayı geri alır -> Işığın Açar
kumanda.UndoLast(); // Açmayı geri alır -> Işığın Kapatır
kumanda.UndoLast(); // Liste boş uyarısı verir
}
}
/* Program Çıktısı:
--- İşlemler Başlıyor ---
Işık AÇILDI.
Işık KAPATILDI.

--- Geri Alma (Undo) İşlemleri ---
[UNDO] Işık AÇILDI.
[UNDO] Işık KAPATILDI.
[UYARI] Geri alınacak işlem yok!
*/

```

### §18.4.3 Yineleyici

**Yineleyici deseninde (iterator pattern)**, birden fazla elemandan oluşan küme (aggregate) içindeki her bir elemana sırayla erişim (iteration) sağlar. Erişim işleminde, istemcinin kümenin nasıl yapılandırıldığı konusunda bilgi sahibi olması gerekmez. Bu desende, istemci tarafından veri kümesi soyut (abstract) olarak kullanılmış olur. Yani istemciye, veri yapısından bağımsız bir şekilde verilere erişim sağlayan bir ara yüz (interface) sunulur. Bu tasarım deseni, bir veri grubunun (liste, ağaç, yığın vb.) iç yapısını dışarıya açmadan, o



Şekil 18.15: Yineleyici Deseni UML Diyagramı

grubun elemanları üzerinde sırayla gezinebilmemizi sağlayan davranışsal bir desendir. Bu desen sayesinde, verilerin bellekte nasıl tutulduğunu (bir dizi mi yoksa birbirine bağlı halkalar mı?) bilmenize gerek kalmadan standart bir şekilde tüm elemanları dolaşabilirsiniz.

- ◇ Yineleyici (Iterator), elemanlar üzerinde gezinmek için gerekli olan standart arayüzü tanımlar. Genellikle `next()` (sonraki eleman), `hasNext()` (başka eleman var mı?) ve `current()` (mevcut eleman) gibi yöntemleri barındırır.
- ◇ Somut Yineleyici (ConcreteIterator), belirli bir veri yapısı (örneğin bir Vektör veya Liste) için gezinme mantığını uygular. O an hangi elemanda olduğunu (indis veya işaretçi bazlı) takip eder.
- ◇ Soyut Koleksiyon (Aggregate), koleksiyonun (container) kendisi için ortak bir arayüz sağlar. Kendi elemanlarını gezmek için kullanılacak olan uygun Iterator nesnesini oluşturan `createIterator()` adlı bir yöntemle sahiptir.
- ◇ Somut Koleksiyon (ConcreteAggregate), verileri asıl saklayan sınıftır. İstendiğinde, kendi iç yapısını en

verimli şekilde gezecek olan somut yineleyiciyi (`ConcreteIterator`) geri döndürür.

Şimdi desenimizi kodlayalım;

```
using System;
using System.Collections.Generic;
// 1. Soyut Iterator Arayüzü
public interface IIterator
{
 double First();
 double Next();
 bool HasNext();
 double CurrentItem();
}
// 2. Soyut Koleksiyon (Aggregate) Arayüzü
public interface IAggregate
{
 void AddItem(double item);
 int Count { get; }
 double GetItem(int index);
 IIterator CreateIterator();
}
// 3. Somut Iterator (Concrete Iterator)
public class ConcreteIterator : IIterator
{
 private readonly IAggregate _aggregate;
 private int _current = 0;
 public ConcreteIterator(IAggregate aggregate)
 {
 _aggregate = aggregate;
 }
 public double First()
 {
 _current = 0;
 return CurrentItem();
 }
 public double Next()
 {
 if (HasNext())
 {
 return _aggregate.GetItem(_current++);
 }
 return -1.0;
 }
 public bool HasNext() => _current < _aggregate.Count;
 public double CurrentItem() => _aggregate.GetItem(_current);
}
// 4. Somut Koleksiyon (Concrete Aggregate)
public class ConcreteAggregate : IAggregate
{
 private readonly List<double> _items = new List<double>();
 public void AddItem(double item) => _items.Add(item);
 public int Count => _items.Count;
 public double GetItem(int index) => _items[index];
 public IIterator CreateIterator()
 {
 return new ConcreteIterator(this);
 }
}
// 5. Client (İstemci)
class Program
{
```

```

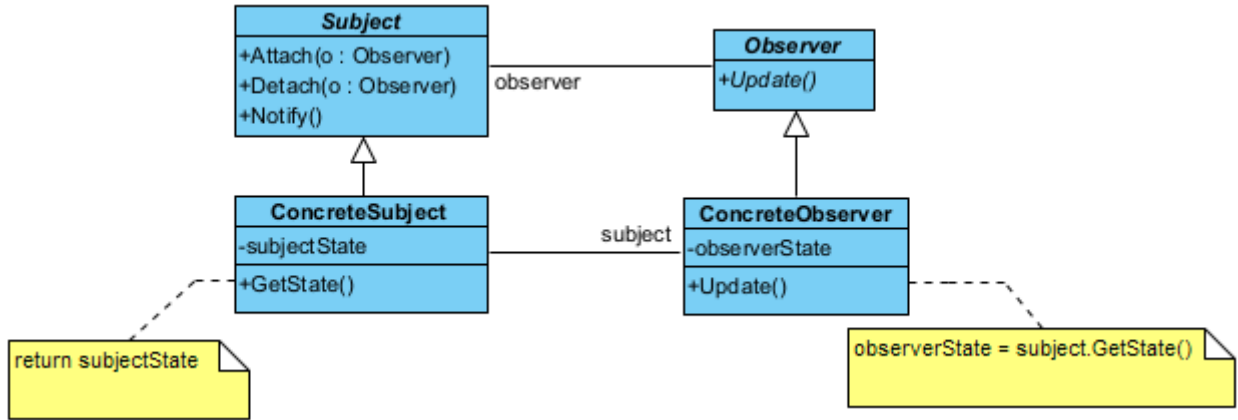
static void Main()
{
 var aggregate = new ConcreteAggregate();
 aggregate.AddItem(10.5);
 aggregate.AddItem(20.7);
 aggregate.AddItem(30.9);
 var iterator = aggregate.CreateIterator();
 Console.WriteLine("Koleksiyon elemanları geziliyor:");
 // Standart Iterator döngü yapısı
 for (iterator.First(); iterator.HasNext();)
 {
 Console.WriteLine($"Değer: {iterator.Next()}");
 }
}
}
/* Program Çıktısı:
Koleksiyon elemanları geziliyor:
Değer: 10.5
Değer: 20.7
Değer: 30.9
*/

```

C# dünyasında yineleyici deseni o kadar temeldir ki, dilin içine `IEnumerable` ve `IEnumerator` arayüzlerine gömülmüştür.

#### §18.4.4 Gözlemci

**Gözlemci deseni** (**observer pattern**), sistemdeki diğer nesnelerin durum değişiklikleri hakkında bir veya daha fazla nesnenin bilgilendirilmesini sağlar.



Şekil 18.16: Gözlemci Deseni UML Diyagramı

Bu tasarım deseni, nesneler arasında bire-çok (**one-to-many**) bir bağımlılık kurmak için kullanılır. Bir nesnenin durumu değiştiğinde, ona bağlı olan tüm diğer nesnelerin (abonelerin) bu değişiklikten otomatik olarak haberdar edilmesini ve güncellenmesini sağlar. Bu desen, modern yazılımlardaki "olay güdümlü" (**event-driven**) sistemlerin ve MVC (**Model-View-Controller**) mimarisinin kalbidir.

- ◆ Bağlam (**Subject**), takip edilen asıl yayıncı nesnedir. Durumu değiştiğinde etrafa haber salar. İçinde bir "Gözlemci Listesi" tutar. Yeni gözlemci eklemek (**attach()**), çıkarmak (**detach()**) ve hepsine haber vermek (**notify()**) için yöntemler barındırır.
- ◆ Gözlemci (**Observer**), güncelleme mesajlarını alabilmek için gerekli olan standart abone arayüzünü tanımlar. Genellikle sadece tek bir **update()** metodu içerir.
- ◆ Somut Yayıncı (**ConcreteSubject**), gerçek veriyi saklayan sınıftır (Örnek olarak bir borsa takip sistemi veya haber sitesi). Kendi içindeki veri değiştiğinde, miras aldığı **notify()** metodunu çağırarak tüm aboneleri tetikler.
- ◆ Somut Gözlemci (**ConcreteObserver**), haberi alan ve buna göre tepki veren sınıftır. **update()** yöntemi tetiklendiğinde, yayıncıdan gelen yeni veriyi alır ve kendi içinde işler (Ekranı yazar, veritabanına kaydeder vb.).

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
using System.Linq;

// 1. Gözlemci: Observer Arayüzü
public interface IObserver
{
 void Update(int state);
}

// 2. Bağlam/Yayıncı: Taban Sınıfı
public abstract class Subject
{
 private readonly List<IObserver> _observers = new List<IObserver>();
 public void Attach(IObserver observer)
 {
 _observers.Add(observer);
 }
 public void Detach(IObserver observer)
 {
 _observers.Remove(observer);
 }
 protected void Notify(int state)
 {
 foreach (var observer in _observers.ToList())
 {
 observer.Update(state);
 }
 }
}

// 3. Somut Bağlam/Yayıncı (Concrete Subject)
public class NewsAgency : Subject
{
 private int _newsId = 0;
 public void SetNews(int id)
 {
 _newsId = id;
 Console.WriteLine($"Haber Ajansı: Yeni haber yayınlandı (ID: {id})");
 Notify(_newsId); // Otomatik bildirim
 }
}

// 4. Somut Gözlemci (Concrete Observer)
public class NewsChannel : IObserver
{
 private readonly string _name;

 public NewsChannel(string name)
 {
 _name = name;
 }
 public void Update(int state)
 {
 Console.WriteLine($" -> {_name} kanalı bilgiyi aldı. Yeni haber ID: {state}");
 }
}

// 5. Client (İstemci)
class Program
{
 static void Main()
 {
 var agency = new NewsAgency();
 var channel1 = new NewsChannel("TRT Haber");
 var channel2 = new NewsChannel("NTV");
 }
}

```

```

var channel3 = new NewsChannel("CNN Türk");
agency.Attach(channel1);
agency.Attach(channel2);
agency.Attach(channel3);
agency.SetNews(101);
Console.WriteLine("-----");
agency.Detach(channel2); // NTV ayrılıyor
agency.SetNews(102);
}
}
/* Program Çıktısı:
Haber Ajansı: Yeni haber yayınlandı (ID: 101)
-> TRT Haber kanalı bilgiyi aldı. Yeni haber ID: 101
-> NTV kanalı bilgiyi aldı. Yeni haber ID: 101
-> CNN Türk kanalı bilgiyi aldı. Yeni haber ID: 101

Haber Ajansı: Yeni haber yayınlandı (ID: 102)
-> TRT Haber kanalı bilgiyi aldı. Yeni haber ID: 102
-> CNN Türk kanalı bilgiyi aldı. Yeni haber ID: 102
*/

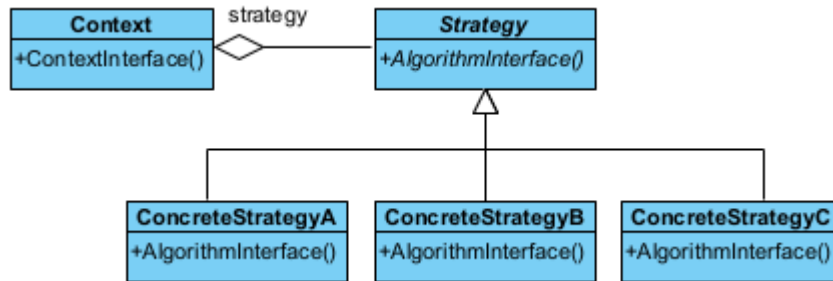
```

Bu desen aşağıdaki durumlarda kullanılır;

- ◊ Bir soyutlama (**abstraction**) sonucu ortaya çıkan bağımsız iki farklı yönün (**aspect**) ortaya çıktığı durumlarda, bu yönler birbirinden bağımsız olarak geliştirilebilir ve değiştirilebilir.
- ◊ Bir değişikliğin başka değişiklikleri gerektirdiği durumlar.
- ◊ Bir nesnenin, diğer nesnelerdeki değişiklikleri kendine vazife etmemesini sağlayan durumlar.

### §18.4.5 Strateji

**Strateji deseni** (**strategy pattern**), belirli bir davranışı gerçekleştirmek için değiştirilebilen kapsüllenmiş (**encapsulated**) algoritmalar kümesini tanımlar.



Şekil 18.17: Strateji Deseni UML Diyagramı

Bu desenin görevi, bir işi yapmanın birden fazla yolu varsa, bu yolları (algoritmaları) sınıflara ayırır ve birbirinin yerine kullanılabilir hale getirir.

- ◊ Bağlam (**Context**), işi yapacak olan sınıf. Ancak işi "nasıl" yapacağını bir strateji nesnesine sorar.
- ◊ Strateji (**Strategy**): Tüm algoritmaların uyması gereken ara yüzdür.
- ◊ Somut Strateji (**ConcreteStrategy**), farklı algoritmalar ile tanımlı stratejilerdir.  
(Örnek olarak "Hızlı Sıralama", "Kabarcık Sıralama" veya "Kredi Kartıyla Öde", "Nakit Öde")

Şimdi desenimizi kodlayalım;

```

using System;
// 1. Strateji Arayüzü
public interface IStrategy
{
 void Execute();
}
// 2. Somut Stratejiler
public class ConcreteStrategyA : IStrategy
{
 public void Execute()
 {
 Console.WriteLine("Algoritma A: Hızlı ama daha fazla bellek kullanıyor.");
 }
}

```



```

 }
}
public class ConcreteStrategyB : IStrategy
{
 public void Execute()
 {
 Console.WriteLine("Algoritma B: Yavaş ama bellek dostu.");
 }
}
public class ConcreteStrategyC : IStrategy
{
 public void Execute()
 {
 Console.WriteLine("Algoritma C: Dengeli bir yaklaşım.");
 }
}
// 3. Bağlam (Context)
public class Context
{
 private IStrategy _strategy;
 public Context(IStrategy strategy)
 {
 _strategy = strategy;
 }
 public void SetStrategy(IStrategy strategy)
 {
 _strategy = strategy;
 }
 public void Run()
 {
 if (_strategy != null)
 {
 _strategy.Execute();
 }
 else
 {
 Console.WriteLine("Hata: Henüz bir strateji belirlenmedi!");
 }
 }
}
// 4. Client (İstemci)
class Program
{
 static void Main()
 {
 // Strateji nesnelerini oluşturuyoruz
 IStrategy s1 = new ConcreteStrategyA();
 IStrategy s2 = new ConcreteStrategyB();
 IStrategy s3 = new ConcreteStrategyC();
 // Bağlam (Context) nesnesini oluştur ve başlangıç stratejisini ata
 Context context = new Context(s1);
 Console.WriteLine("--- Durum 1 ---");
 context.Run();
 // Çalışma zamanında stratejiyi değiştir
 context.SetStrategy(s2);
 Console.WriteLine("--- Durum 2 ---");
 context.Run();
 context.SetStrategy(s3);
 Console.WriteLine("--- Durum 3 ---");
 context.Run();
 }
}

```

```

/* Program Çıktısı:
--- Durum 1 ---
Algoritma A: Hızlı ama daha fazla bellek kullanıyor.
--- Durum 2 ---
Algoritma B: Yavaş ama bellek dostu.
--- Durum 3 ---
Algoritma C: Dengeli bir yaklaşım.
*/

```

Bu desenin **bağlam** nesnesinin, diğer tasarım desenlerinin biri ile iç içe kullanılması halinde, bu desen **sıfır desen** (**null pattern**) olarak adlandırılır. Sıfır desende, **Context** nesnesi akıllıdır ve her zaman ne yapacağını bilir. Bu desen aşağıdaki durumda kullanılır;

- ◊ Aralarında ilişki bulunan birçok sınıfın davranışlara bağlı olarak anlaşmazlığa düşmesini önlemek için,
- ◊ Birden çok algoritmanın olması durumunda,
- ◊ Algoritma, istemcinin bilmeyeceği bir veri yapısına sahip olduğu durumlarda,
- ◊ Bir sınıf kendi yöntemlerinde çok fazla sayıda duruma göre seçim (**conditional choice**) kullanıyorsa.

Aşağıda bu desene ilişkin sıralama algoritmalarının seçildiği gerçek bir örnek verilmiştir;

```

using System;
using System.Collections.Generic;
// 1. --- Abstract Strategy (Strateji Arayüzü) ---
public interface ISortStrategy
{
 void Sort(List<int> data);
}
// 2. --- Concrete Strategy A: Quick Sort ---
public class QuickSort : ISortStrategy
{
 public void Sort(List<int> data)
 {
 Console.WriteLine("QuickSort algoritması kullanılıyor...");
 // C# List.Sort() metodu genellikle QuickSort türevi (Introspective Sort) kullanır.
 data.Sort();
 }
}
// 3. --- Concrete Strategy B: Merge Sort ---
public class MergeSort : ISortStrategy
{
 public void Sort(List<int> data)
 {
 Console.WriteLine("MergeSort algoritması kullanılıyor...");
 // Örnek amaçlı: C# tarafında harici bir kütüphane veya manuel
 // implementasyon yerine LINQ OrderBy kullanımı stabil bir sıralama sağlar.
 var sorted = data.OrderBy(n => n).ToList();
 data.Clear();
 data.AddRange(sorted);
 }
}
// 4. --- Context: Sorter ---
public class Sorter
{
 private ISortStrategy? _strategy;
 // Stratejiyi çalışma zamanında değiştirmemizi sağlar (Property veya Method)
 public void SetStrategy(ISortStrategy strategy)
 {
 _strategy = strategy;
 }
 public void Sort(List<int> data)
 {
 if (_strategy == null)
 {
 Console.WriteLine("Hata: Önce bir strateji belirlenmelidir!");
 }
 }
}

```

```

 return;
 }
 _strategy.Sort(data);
}
}
// 5. --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 List<int> sayilar = new List<int> { 34, 12, 5, 78, 1, 45 };
 Sorter siralayici = new Sorter();

 Console.Write("Orijinal liste: ");
 PrintList(sayilar);
 // 1. QuickSort Stratejisini Seçelim
 siralayici.SetStrategy(new QuickSort());
 siralayici.Sort(sayilar);
 PrintList(sayilar);
 // Listeyi tekrar karıştıralım
 sayilar = new List<int> { 99, 2, 56, 11, 0, 7 };
 Console.Write("\nYeni liste: ");
 PrintList(sayilar);
 // 2. MergeSort Stratejisine Geçelim
 siralayici.SetStrategy(new MergeSort());
 siralayici.Sort(sayilar);
 PrintList(sayilar);
 }
 static void PrintList(List<int> data)
 {
 Console.WriteLine(string.Join(" ", data));
 }
}

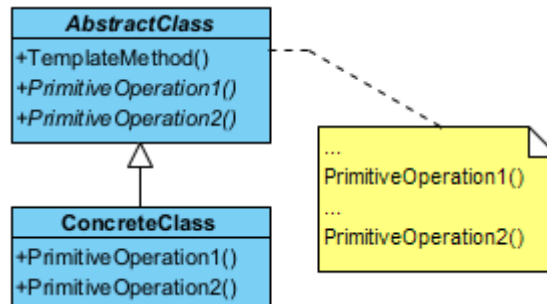
/* Programın Çıktısı:
Orijinal liste: 34 12 5 78 1 45
QuickSort algoritması kullanılıyor...
1 5 12 34 45 78

Yeni liste: 99 2 56 11 0 7
MergeSort algoritması kullanılıyor...
0 2 7 11 56 99
*/

```

#### §18.4.6 Şablon Yöntem

Şablon yöntem deseninde (**template method pattern**), bir algoritmanın çerçevesini belirler ve uygulayıcı sınıfların gerçek davranışı tanımlamasına olanak tanır.



Şekil 18.18: Şablon Yöntem Deseni UML Diyagramı

Bu desen, nesne yönelimli programlamada "algoritma iskeleti" oluşturmak için kullanılır. Bir işin adımlarını (sırasını) belirler ancak bu adımların bazılarının nasıl yapılacağını alt sınıflara bırakır. Kod tekrarını

önlemek ve bir işlemin temel yapısını korumak için mükemmeldir.

- ◊ Soyut Sınıf (**AbstractClass**), algoritmanın ana iskeletini ve şablon yöntemi (**TemplateMethod()**) barındırır. Değişmez adımları kendi içinde tanımlar, değişebilecek adımları ise soyut (**virtual**) olarak işaretler. Şablon yöntem genellikle **final** olarak tanımlanır ki algoritmanın sırası bozulmasın.
- ◊ Somut Sınıf (**ConcreteClass**); Soyut sınıftaki "boşlukları" doldurur. Algoritmanın değişken adımlarını kendi ihtiyacına göre özelleştirir. Algoritmanın genel akışına müdahale edemez, sadece kendisine izin verilen adımları yazar.

Şimdi desenimizi kodlayalım;

```
using System;
// 1. Soyut Sınıf (Abstract Class)
public abstract class AbstractClass
{
 // Şablon Yöntem: Algoritmanın iskeletini sabitler.
 // 'sealed' yapılamaz çünkü kalıtım gereklidir, ancak metodun kendisi
 // alt sınıflar tarafından değiştirilemesin diye sanal (virtual) yapılmaz.
 public void TemplateMethod()
 {
 BaseOperation(); // Ortak adım
 RequiredOperation1(); // Özelleştirilebilir adım
 Hook(); // İsteğe bağlı adım (Hook)
 RequiredOperation2(); // Özelleştirilebilir adım
 }
 // Temel operasyon: Tüm alt sınıflar için aynıdır.
 private void BaseOperation()
 {
 Console.WriteLine("Sistem: Algoritmanın ortak temel adımı çalıştırılıyor.");
 }
 // Alt sınıfların mutlaka uygulaması gereken adımlar
 protected abstract void RequiredOperation1();
 protected abstract void RequiredOperation2();
 // Hook (Kanca): Alt sınıflar isterse geçersiz kılar, istemezse boş kalır.
 // C#'ta boş bir gövde ile 'virtual' tanımlanır.
 protected virtual void Hook() { }
}
// 2. Somut Sınıf (Concrete Class)
public class ConcreteClass : AbstractClass
{
 protected override void RequiredOperation1()
 {
 Console.WriteLine("ConcreteClass: Adım 1 özel olarak uygulandı.");
 }
 protected override void RequiredOperation2()
 {
 Console.WriteLine("ConcreteClass: Adım 2 özel olarak uygulandı.");
 }
 // İsteğe bağlı: Hook'u geçersiz kılarız.
 protected override void Hook()
 {
 Console.WriteLine("ConcreteClass: Hook (Kanca) devreye girdi.");
 }
}
// 3. Client (İstemci)
class Program
{
 static void Main()
 {
 AbstractClass algoritma = new ConcreteClass();
 Console.WriteLine("--- Algoritma Başlatılıyor ---");
 algoritma.TemplateMethod();
 }
}
```

```

/* Program Çıktısı:
--- Algoritma Başlatılıyor ---
Sistem: Algoritmanın ortak temel adımı çalıştırılıyor.
ConcreteClass: Adım 1 özel olarak uygulandı.
ConcreteClass: Hook (Kanca) devreye girdi.
ConcreteClass: Adım 2 özel olarak uygulandı.
*/

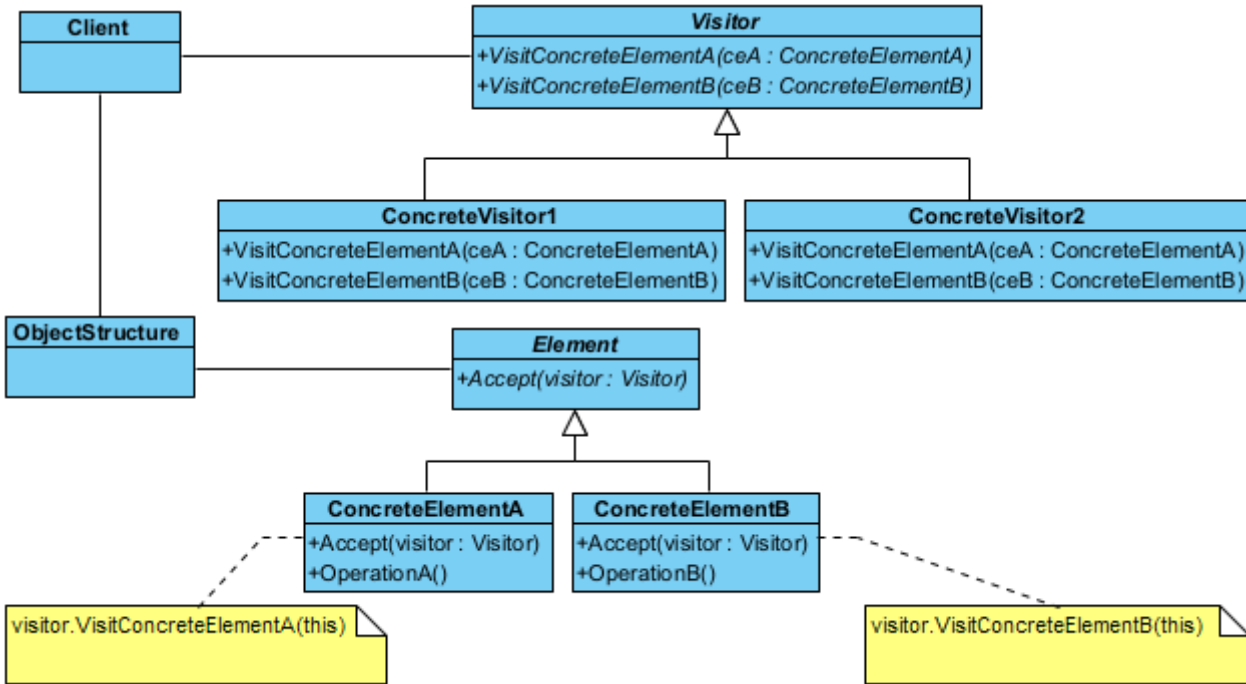
```

Bu desende, kullanılacak algoritmalarından hangilerinin standart, hangilerinin somut sınıflara özgü olacağı belirlenmelidir;

- ◊ İçinde “Bizi çağırmayın! Biz sizi çağıracağız.” sloganını taşıyan yöntemler olan bir soyut bir taban sınıf tasarlanmalıdır.
- ◊ Taban sınıf içinde algoritmanın kabuğunu oluşturacak standart adımları içeren şablon yöntem tanımlanır.
- ◊ Somut sınıflarda detaylandırılacak yöntemlere ilişkin sanal yöntemler taban sınıfta tanımlanır.
- ◊ Şablon yöntem içinde algoritmayı oluşturan yöntemler çağrılır.
- ◊ Şablon yöntem içinde yer almayan tüm detay kodlamalar somut sınıflar için de yapılır.

### §18.4.7 Ziyaretçi

**Ziyaretçi deseni (visitor pattern)**, bir nesne grubunun (koleksiyonun) yapısını değiştirmeden, bu nesneler üzerinde uygulanacak yeni işlemleri dışarıdan eklememizi sağlayan davranışsal bir desendir. Bu desen, "Veri Yapısı" ile "Algoritma"yı birbirinden ayırır. Eğer elinizde sabit bir sınıf hiyerarşisi varsa (örneğin bir dokümandaki paragraflar, resimler, tablolar) ve sürekli bu sınıflar üzerinde yeni işlemler (PDF'e dönüştür, HTML'e dönüştür, kelime say) tanımlamanız gerekiyorsa, her seferinde o sınıfların içine girip kod yazmak yerine ziyaretçi desenini kullanırsınız.



Şekil 18.19: Ziyaretçi Deseni UML Diyagramı

- ◊ Ziyaretçi (**Visitor**), koleksiyondaki her bir somut nesne türü için bir ziyaret yöntemi (**visit()**) tanımlayan soyut bir sınıftır. "Eğer bir Paragraf nesnesine gidersem ne yapacağım?", "Eğer bir Resim nesnesine gidersem ne yapacağım?" sorularının imzasını belirler.
- ◊ Somut Ziyaretçi (**ConcreteVisitor**), gerçek operasyonu gerçekleştirir. Algoritmanın kendisidir. Örneğin **ExportToPDFVisitor** sınıfı, her nesne türünün PDF'e nasıl dönüştürüleceğini bilir.
- ◊ Eleman (**Element**), bir ziyaretçiyi kabul edebilmek için standart bir "kapı" açan soyut bir sınıftır. Ziyaretçi kabul etmek için **accept(Visitor\* v)** yöntemini barındırır.
- ◊ Somut Elemanlar (**ConcreteElement**), veri yapısının asıl parçalarıdır. **accept()** yöntemi çağrıldığında, ziyaretçiye "Ben bir [NesneTürü]'yüm, beni ziyaret et" der (**v->visit(this)**). Buna yazılım dünyasında **çifte sevkیات (double dispatch)** denir.

Şimdi desenimizi kodlayalım;

```
using System;
using System.Collections.Generic;
// 1. Visitor (Ziyaretçi) Arayüzü
public interface IVisitor
{
 // C#'ta aşırı yükleme (overloading) sayesinde metod isimlerini aynı tutabiliriz
 void Visit(ConcreteElementA element);
 void Visit(ConcreteElementB element);
}
// 2. Element (Eleman) Arayüzü
public interface IElement
{
 // Double Dispatch mekanizmasının kalbi: Accept metodu
 void Accept(IVisitor visitor);
}
// 3. Somut Elemanlar
public class ConcreteElementA : IElement
{
 public void Accept(IVisitor visitor)
 {
 // Çalışma zamanında visitor'ın hangi metodunun çağrılacağı burada netleşir
 visitor.Visit(this);
 }
 public void OperationA()
 {
 Console.WriteLine("ConcreteElementA: Özel işlem çalıştırılıyor.");
 }
}

public class ConcreteElementB : IElement
{
 public void Accept(IVisitor visitor)
 {
 visitor.Visit(this);
 }
 public void OperationB()
 {
 Console.WriteLine("ConcreteElementB: Özel işlem çalıştırılıyor.");
 }
}
// 4. Somut Ziyaretçiler
public class ConcreteVisitor1 : IVisitor
{
 public void Visit(ConcreteElementA element)
 {
 Console.WriteLine("Visitor 1: ElementA ziyaret edildi.");
 element.OperationA();
 }
 public void Visit(ConcreteElementB element)
 {
 Console.WriteLine("Visitor 1: ElementB ziyaret edildi.");
 element.OperationB();
 }
}
public class ConcreteVisitor2 : IVisitor
{
 public void Visit(ConcreteElementA element)
 {
 Console.WriteLine("Visitor 2: ElementA üzerinde farklı bir işlem yapılıyor.");
 }
 public void Visit(ConcreteElementB element)
 {

```

```

 Console.WriteLine("Visitor 2: ElementB üzerinde farklı bir işlem yapılıyor.");
 }
}
// 5. Nesne Yapısı (Object Structure)
public class ObjectStructure
{
 private readonly List<IElement> _elements = new List<IElement>();
 public void Add(IElement e) => _elements.Add(e);
 public void Accept(IVisitor v)
 {
 foreach (var element in _elements)
 {
 element.Accept(v);
 }
 }
}
// 6. Client (İstemci)
class Program
{
 static void Main()
 {
 ObjectStructure structure = new ObjectStructure();
 structure.Add(new ConcreteElementA());
 structure.Add(new ConcreteElementB());
 IVisitor v1 = new ConcreteVisitor1();
 IVisitor v2 = new ConcreteVisitor2();
 Console.WriteLine("--- Ziyaretçi 1 Başlıyor ---");
 structure.Accept(v1);
 Console.WriteLine("\n--- Ziyaretçi 2 Başlıyor ---");
 structure.Accept(v2);
 }
}
/* Program Çıktısı:
--- Ziyaretçi 1 Başlıyor ---
Visitor 1: ElementA ziyaret edildi.
ConcreteElementA: Özel işlem çalıştırılıyor.
Visitor 1: ElementB ziyaret edildi.
ConcreteElementB: Özel işlem çalıştırılıyor.

--- Ziyaretçi 2 Başlıyor ---
Visitor 2: ElementA üzerinde farklı bir işlem yapılıyor.
Visitor 2: ElementB üzerinde farklı bir işlem yapılıyor.
*/

```

Bu desen aşağıdaki durumlarda kullanılır;

- ◊ Nesneler birden fazla olup birçok ara yüze sahip olduğunda, bu nesnelerin somut sınıfları ile bir işlem yapmak gerektiğinde,
- ◊ Sınıflar üzerinde yapılacak işlemlerin, sınıflar üzerinde bir iz bırakmamasını sağlamak amacıyla,
- ◊ Nadir değişen sınıf yapılarına, sınıf yapısını değiştirmeden, yeni işlemler eklemek gerektiğinde.

Bu deseni kullanmanın aşağıda sıralanan üstünlükleri sağlar;

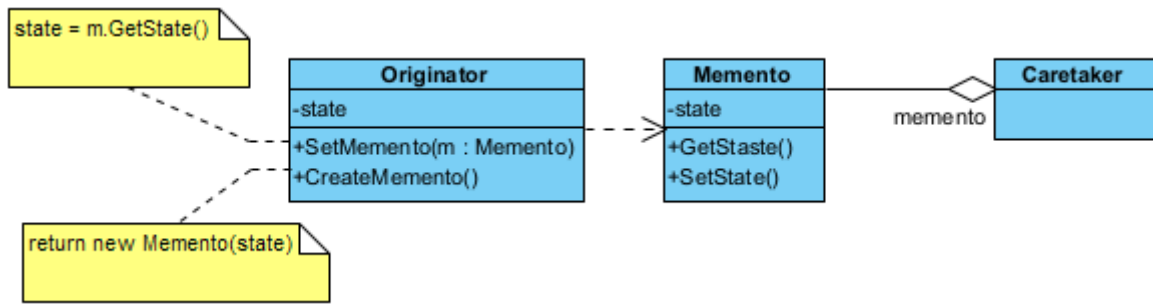
- ◊ Yeni işlemleri (**operation**) sisteme eklemeyi kolaylaştırır.
- ◊ Birbiriyle ilişkili olan işlemleri ilişkili olmayandan ayırmaya yardımcı olur.
- ◊ Yeni **ConcreteElement** nesnelerinin eklenmesini zorlaştırır.
- ◊ Bütün nesne hiyerarşisi baştanbaşa ziyaret edilebilir. Sınıflardaki tüm durumlar (**state**) biriktirilebilir.
- ◊ Bazı durumlarda kapsülleme (**encapsulation**) işlemini engeller. Yani nesnenin iç durumlarından hareketle herkese açık işlemler tanımlanabilir.

### §18.4.8 Hatıra

**Hatıra deseni** (**memento pattern**), nesnelerin bir andaki durumlarını (**state**) saklayıp bir başka zaman da bu duruma geri dönmek gerektiğinde kullanılır. Bir nesnenin iç durumunun yakalanmasına ve dışarıda saklanmasına olanak tanır, böylece daha sonra geri yüklenebilir fakat tüm bunlar **kılıflamayı** (**encapsulation**)



ihlal etmeden yapılır.



Şekil 18.20: Hatıra Deseni UML Diyagramı

Hatıra deseninin amacı, nesnenin iç durumunu (**state**) bozmadan saklamak ve daha sonra geri yüklemektir.

- ♦ Yaratıcı (**Originator**), durumu değişen asıl nesnedir. Kendi "anlık görüntüsünü" (**snapshot**) oluşturur ve geri yükler.
- ♦ Hatıra (**Memento**), sadece veriyi tutan pasif nesnedir. Dışarıdan değiştirilemez; sadece **Originator** içeriğini okuyabilir.
- ♦ Bakıcı (**Caretaker**), hatıraları güvenli bir yerde (liste veya yığın) saklar. İçeriği asla değiştirmez, sadece saklar ve geri verir.

Şimdi desenimizi kodlayalım;

```
using System;
using System.Collections.Generic;
// 1. Originator: Durumu değişen ve "hatıra" üreten asıl nesne
public class Originator
{
 private int _state;
 public void SetState(int state)
 {
 _state = state;
 Console.WriteLine($"Originator: Durum değiştirildi -> {_state}");
 }
 // Mevcut durumu bir Memento nesnesine paketler
 public Memento Save()
 {
 Console.WriteLine($"Originator: Durum yedekleniyor... ({_state})");
 return new Memento(_state);
 }
 // Bir Memento nesnesinden durumu geri yükler
 public void Restore(Memento memento)
 {
 if (memento != null)
 {
 _state = memento.GetState();
 Console.WriteLine($"Originator: Durum geri yüklendi -> {_state}");
 }
 }
}
// --- Memento: Durumu saklayan nesne (İç Sınıf Olarak) ---
// C#'ta memento'yu iç sınıf yaparak Originator dışındakilerin verilere erişmesini
// → kısıtlarız.
public class Memento
{
 private readonly int _state;
 // Constructor internal veya private yapılarak sadece Originator erişimi sağlanır
 internal Memento(int state)
 {
 _state = state;
 }
 internal int GetState() => _state;
}
```

```

 }
}
// 2. Caretaker: Hatıraları saklayan bekçi
public class Caretaker
{
 private readonly Stack<Originator.Memento> _history = new Stack<Originator.Memento>();
 private readonly Originator _originator;
 public Caretaker(Originator originator)
 {
 _originator = originator;
 }
 public void Backup()
 {
 _history.Push(_originator.Save());
 }
 public void Undo()
 {
 if (_history.Count == 0) return;

 Originator.Memento memento = _history.Pop();
 _originator.Restore(memento);
 }
}
// 3. İstemci (Client)
class Program
{
 static void Main()
 {
 Originator player = new Originator();
 Caretaker saveManager = new Caretaker(player);
 player.SetState(100); // Bölüm 1
 saveManager.Backup(); // Kayıt noktası
 player.SetState(50); // Can azaldı
 player.SetState(0); // Öldü!
 Console.WriteLine("--- Geri Yükleniyor ---");
 saveManager.Undo(); // Bölüm 1'e geri dön
 }
}
/* Program Çıktısı:
Originator: Durum değiştirildi -> 100
Originator: Durum yedekleniyor... (100)
Originator: Durum değiştirildi -> 50
Originator: Durum değiştirildi -> 0
--- Geri Yükleniyor ---
Originator: Durum geri yüklendi -> 100
*/

```

Aşağıdaki örnekte bir metin editörünün geçmiş özelliği modellenmiştir;

```

using System;
using System.Collections.Generic;
// --- Memento ---
// Sadece saklanacak veriyi tutan pasif nesne.
public class EditorMemento
{
 // C#'ta 'readonly' kullanarak verinin sonradan değiştirilmesini engelleriz.
 private readonly string _content;
 public EditorMemento(string text)
 {
 _content = text;
 }
 public string GetSavedContent() => _content;
}

```

```
// --- Originator ---
// Durumu kaydedilecek ve geri yüklenecek olan asıl nesne.
public class TextEditor
{
 private string _text = "";
 public void Type(string newText) => _text += newText;
 public string GetText() => _text;
 // Mevcut durumu bir Memento içine paketler.
 public EditorMemento Save()
 {
 return new EditorMemento(_text);
 }
 // Bir Memento'dan durumu geri yükler.
 public void Restore(EditorMemento memento)
 {
 if (memento != null)
 {
 _text = memento.GetSavedContent();
 }
 }
}

// --- Caretaker ---
// Geçmiş (Memento listesini) yöneten sınıf.
public class History
{
 // Geri alma işlemleri için LIFO (Stack) yapısı en doğrusudur.
 private readonly Stack<EditorMemento> _mementos = new Stack<EditorMemento>();
 public void Push(EditorMemento m)
 {
 _mementos.Push(m);
 }
 public EditorMemento Pop()
 {
 if (_mementos.Count == 0) return null;
 return _mementos.Pop();
 }
}

// --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 TextEditor editor = new TextEditor();
 History history = new History();
 // 1. Yazmaya başla ve ilk durumu kaydet
 editor.Type("Merhaba ");
 history.Push(editor.Save());
 Console.WriteLine($"Güncel Metin: {editor.GetText()}");
 // 2. Yazmaya devam et ve ikinci durumu kaydet
 editor.Type("Dünya!");
 history.Push(editor.Save());
 Console.WriteLine($"Güncel Metin: {editor.GetText()}");
 // 3. Bir hata yapalım
 editor.Type(" Yanlış bir ekleme.");
 Console.WriteLine($"Hata Sonrası: {editor.GetText()}");
 // 4. Geri al (Undo)
 // Son eklenen hatıra "Merhaba Dünya!" idi (history.Pop()).
 history.Pop(); // "Merhaba Dünya!" çıktı
 var previousState = history.Pop(); // "Merhaba " çıktı
 if (previousState != null)
 {
 editor.Restore(previousState);
 }
 }
}
```

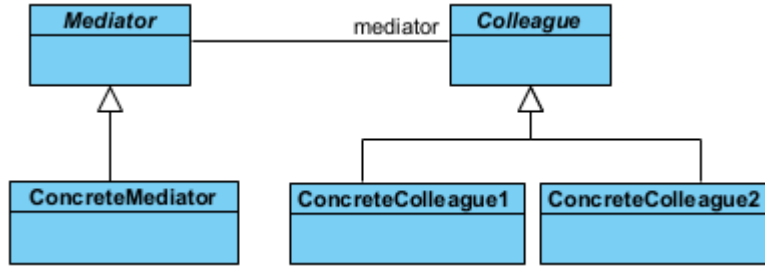
```

 Console.WriteLine($"Geri Alındı: {editor.GetText()}");
 }
}
}
/* Program Çıktısı:
Güncel Metin: Merhaba
Güncel Metin: Merhaba Dünya!
Hata Sonrası: Merhaba Dünya! Yanlış bir ekleme.
Geri Alındı: Merhaba
*/

```

### §18.4.9 Ara Bulucu

**Ara Bulucu deseni** (**mediator pattern**), sınıfların arasındaki iletişimi basitleştirir. Bir iletişim nesnesi tanımlanır, diğer nesneler bu nesne üzerinden birbirleriyle haberleşir. Diğer nesnelerin birbiriyle doğrudan iletişim kurmasına izin verilmez. Amaç, iletişimin kontrol altına alınmasıdır. Bu tasarım deseni, bir dizi nesne



Şekil 18.21: Ara Bulucu Deseni UML Diyagramı

arasındaki karmaşık iletişim ağını basitleştirmek için kullanılır. Nesnelerin birbirleriyle doğrudan konuşmasını engeller ve tüm iletişimi tek bir "merkezi aracı" üzerinden yürütmelerini sağlar. Bu desen, özellikle nesne sayısı arttığında ortaya çıkan "örümcek ağı" gibi karmaşık bağlantıları (**tight coupling**) çözmek için birebirdir.

- ◇ Soyut Aracı (**Mediator**), nesneler arasındaki iletişim protokolünü (arayüzünü) tanımlar. "Hangi olay gerçekleştiğinde kimlere haber verilecek?" sorusunun genel çerçevesini çizer.
- ◇ Somut Aracı (**ConcreteMediator**), iletişim trafiğini yöneten asıl "santral" binasıdır. tüm meslektaşları (**Colleague**) tanır ve aralarındaki koordinasyonu sağlar. Bir bileşenden mesaj geldiğinde, mantık çerçevesinde diğerlerine iletir.
- ◇ Meslektaş (**Colleague**) Sınıflar, kendi işlevini yerine getiren bağımsız sınıflardır. Diğer sınıfların varlığından haberdar değildirler. Sadece "Aracı"ya (**Mediator**) haber verirler. "Ben düğmeye bastım, geri-sini aracı halletsin" mantığıyla çalışırlar.

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
// 1. Mediator (Ara bulucu) Arayüzü
public interface IMediator
{
 void RegisterColleague(Colleague colleague);
 void RelayMessage(string message, Colleague sender);
}
// 2. Colleague (Meslektaş) Taban Sınıfı
public abstract class Colleague
{
 protected string Name;
 protected IMediator Mediator;
 protected Colleague(string name, IMediator mediator)
 {
 Name = name;
 Mediator = mediator;
 }
 public string GetName() => Name;
 public void Send(string message)
 {

```

```

 Mediator.RelayMessage(message, this);
 }
 public virtual void Receive(string message, string senderName)
 {
 Console.WriteLine($"[{Name}] {senderName} kişisinden mesaj aldı: {message}");
 }
}
// 3. Somut Ara bulucu (Concrete Mediator)
public class ChatRoom : IMediator
{
 private readonly List<Colleague> _colleagues = new List<Colleague>();
 public void RegisterColleague(Colleague colleague)
 {
 _colleagues.Add(colleague);
 }
 public void RelayMessage(string message, Colleague sender)
 {
 Console.WriteLine($"--- LOG: {sender.GetName()} bir mesaj yayınladı ---");
 foreach (var colleague in _colleagues)
 {
 // Mesajı gönderen kişiye geri göndermiyoruz
 if (colleague != sender)
 {
 colleague.Receive(message, sender.GetName());
 }
 }
 }
}
// 4. Somut Meslektaş (Concrete Colleague)
public class User : Colleague
{
 public User(string name, IMediator mediator) : base(name, mediator) { }
}
// 5. İstemci (Client)
class Program
{
 static void Main()
 {
 // Arabulucu nesnesi
 IMediator oda = new ChatRoom();
 // Meslektaşlar (Kullanıcılar)
 User ilhan = new User("Ilhan", oda);
 User mehmet = new User("Mehmet", oda);
 User ayse = new User("Ayse", oda);
 // Kayıt işlemleri
 oda.RegisterColleague(ilhan);
 oda.RegisterColleague(mehmet);
 oda.RegisterColleague(ayse);
 // İletişim başlıyor
 ilhan.Send("Selam millet!");
 mehmet.Send("Merhabalar Ilhan.");
 ayse.Send("Nasılsınız?");
 }
}
/* Program Çıktısı:
--- LOG: Ilhan bir mesaj yayınladı ---
[Mehmet] Ilhan kişisinden mesaj aldı: Selam millet!
[Ayse] Ilhan kişisinden mesaj aldı: Selam millet!
--- LOG: Mehmet bir mesaj yayınladı ---
[Ilhan] Mehmet kişisinden mesaj aldı: Merhabalar Ilhan.
[Ayse] Mehmet kişisinden mesaj aldı: Merhabalar Ilhan.
--- LOG: Ayse bir mesaj yayınladı ---

```

```
[Ilhan] Ayse kişisinden mesaj aldı: Nasılsınız?
[Mehmet] Ayse kişisinden mesaj aldı: Nasılsınız?
*/
```

Bu desende, sistemde yalnızca bir adet ara bulucu olacaksa soyut **Mediator** nesnesi tanımlanmayabilir. Ayrıca ara bulucu ile meslektaşlar arasındaki iletişim iki türlü yapılır;

- ◊ Birisinde bu iletişim gözlemci deseni (**observer pattern**) ile sağlanır. **Colleague** sınıfı gözlemci desenindeki bağlam **Subject** sınıfı gibi davranır.
- ◊ Diğerinde ise **Mediator** nesnesine, görüşecekleri meslektaşları belirterek doğrudan ileti gönderirler.

Aşağıda havaalanındaki kule ile uçaklar arasındaki iletişimi modelleyen bir örnek verilmiştir;

```
using System;
using System.Collections.Generic;
// --- Mediator Arayüzü ---
public interface IAirTrafficControl
{
 void SendMessage(string message, Airplane originator);
 void RegisterAirplane(Airplane airplane);
}
// --- Colleague (Meslektaş/Bileşen) ---
public class Airplane
{
 protected IAirTrafficControl Mediator;
 protected string CallSign;
 public Airplane(IAirTrafficControl mediator, string callSign)
 {
 Mediator = mediator;
 CallSign = callSign;
 }
 public virtual void Send(string message)
 {
 Console.WriteLine($"[{CallSign}] Kuleye mesaj gönderiyor: {message}");
 Mediator.SendMessage(message, this);
 }
 public virtual void Receive(string message)
 {
 Console.WriteLine($"[{CallSign}] Mesaj aldı: {message}");
 }
 public string GetCallSign() => CallSign;
}

// --- Concrete Mediator (Somut Aracı) ---
public class AtcTower : IAirTrafficControl
{
 private readonly List<Airplane> _airplanes = new List<Airplane>();
 public void RegisterAirplane(Airplane airplane)
 {
 if (!_airplanes.Contains(airplane))
 {
 _airplanes.Add(airplane);
 }
 }
 public void SendMessage(string message, Airplane originator)
 {
 foreach (var airplane in _airplanes)
 {
 // Mesajı gönderen uçak hariç herkese ilet
 if (airplane != originator)
 {
 airplane.Receive($"Kule'den {originator.GetCallSign()} bilgisi: {message}");
 }
 }
 }
}
```

```

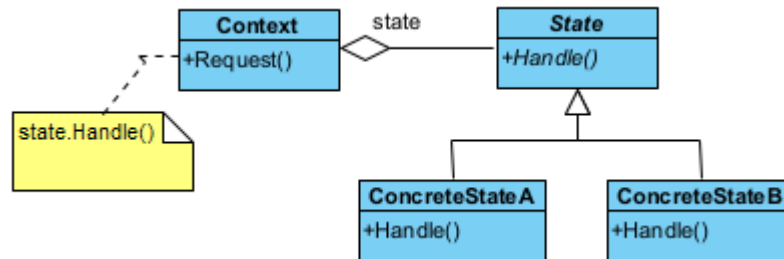
 }
}
// --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 // 1. Aracı (Kule) oluşturulur
 AtcTower tower = new AtcTower();
 // 2. Meslektaşlar (Uçaklar) oluşturulur ve kuleye bildirilir
 Airplane boeing = new Airplane(tower, "THY-123");
 Airplane airbus = new Airplane(tower, "PGS-456");
 Airplane cessna = new Airplane(tower, "OZEL-789");
 tower.RegisterAirplane(boeing);
 tower.RegisterAirplane(airbus);
 tower.RegisterAirplane(cessna);
 // 3. İletişim merkezi üzerinden yürütülür
 boeing.Send("Pist 1'e iniş izni istiyorum.");
 Console.WriteLine(new string('-', 42));
 cessna.Send("Pistten çıkış yapıldı.");
 }
}
/* Program Çıktısı:
[THY-123] Kuleye mesaj gönderiyor: Pist 1'e iniş izni istiyorum.
[PGS-456] Mesaj aldı: Kule'den THY-123 bilgisi: Pist 1'e iniş izni istiyorum.
[OZEL-789] Mesaj aldı: Kule'den THY-123 bilgisi: Pist 1'e iniş izni istiyorum.

[OZEL-789] Kuleye mesaj gönderiyor: Pistten çıkış yapıldı.
[THY-123] Mesaj aldı: Kule'den OZEL-789 bilgisi: Pistten çıkış yapıldı.
[PGS-456] Mesaj aldı: Kule'den OZEL-789 bilgisi: Pistten çıkış yapıldı.
*/

```

### §18.4.10 Durum

**Durum deseni (state pattern)**, nesnenin durumunu davranışına bağlar, nesnenin içsel durumuna göre farklı şekillerde davranmasına olanak tanır.



Şekil 18.22: Durum Deseni UML Diyagramı

Bu desenin amacı, nesnenin iç durumu değiştiğinde davranışını da değiştirmek. Nesne sanki sınıfını değiştiriyormuş gibi görünür.

- ◊ Bağlam (Context), istemcinin (Client) etkileşime girdiği ana sınıftır. Mevcut durum nesnesini bir referans olarak tutar.
- ◊ Soyut Durum (State), tüm somut durumların uyması gereken arayüzü (Interface) tanımlar.
- ◊ Somut Durumlar (ConcreteState), Her sınıf, belirli bir durumdaki davranışını temsil eder (Örn: Oynatılıyor, Duraklatıldı). Bir durumdan diğerine geçiş mantığını genellikle bu sınıflar yönetir.

Şimdi desenimizi kodlayalım;

```

using System;
// 1. State Arayüzü (Soyut Durum)
public abstract class State
{
 public abstract void Handle(Context context);
}

```



```
// 2. Context (Bağlam)
public class Context
{
 private State _currentState;
 public Context(State state)
 {
 TransitionTo(state);
 }
 public void TransitionTo(State state)
 {
 Console.WriteLine($"Context: Durum değiştiriliyor ({state.GetType().Name})...");
 _currentState = state;
 }
 public void Request()
 {
 _currentState.Handle(this);
 }
}

// 3. Somut Durumlar
public class ConcreteStateA : State
{
 public override void Handle(Context context)
 {
 Console.WriteLine("ConcreteStateA: Talep işleniyor. B durumuna geçiş tetikleniyor.");
 context.TransitionTo(new ConcreteStateB());
 }
}

public class ConcreteStateB : State
{
 public override void Handle(Context context)
 {
 Console.WriteLine("ConcreteStateB: Talep işleniyor. A durumuna geri dönülüyor.");
 context.TransitionTo(new ConcreteStateA());
 }
}

// 4. İstemci (Client)
class Program
{
 static void Main()
 {
 // Başlangıç durumu A ile Context oluşturuluyor
 Context context = new Context(new ConcreteStateA());
 // İstekler gönderildikçe durumlar kendi içlerinde birbirini tetikler
 context.Request(); // A -> B geçişi
 context.Request(); // B -> A geçişi
 context.Request(); // A -> B geçişi
 }
}

/* Program Çıktısı:
Context: Durum değiştiriliyor (ConcreteStateA)...
ConcreteStateA: Talep işleniyor. B durumuna geçiş tetikleniyor.
Context: Durum değiştiriliyor (ConcreteStateB)...
ConcreteStateB: Talep işleniyor. A durumuna geri dönülüyor.
Context: Durum değiştiriliyor (ConcreteStateA)...
ConcreteStateA: Talep işleniyor. B durumuna geçiş tetikleniyor.
Context: Durum değiştiriliyor (ConcreteStateB)...
*/
```

Bu desenin bağlam (**Context**) nesnesinin, diğer tasarım desenleri ile iç içe kullanılması halinde, bu desen, **sıfır desen** (**null pattern**) olarak adlandırılır. Ayrıca bu desenle ilgili olarak iki önemli şeyden bahsetmek gerekir:

- ◊ Sahip olunan duruma göre icra edilecek davranış, tanımlanan **State** sınıfına bağlıdır. Yani ilgili davra-

nış somut sınıflardan biri tarafından icra edilir. Bu nedenle somut state (**ConcreteState**) sınıfları içinde tanımlanan yöntemler, durumu kullanan bağlam (**Context**) sınıf içinde tanımlanmak zorundadır.

- ◊ Durumu kullanan bağlam (**Context**) sınıf içinde hangi duruma geçildiğinin anlaşılması için ilgili duruma ilişkin tanımlanan özellik (**property**) içinde **set** yöntemi kullanılmak zorundadır.

Bu desen netice olarak, durum geçişlerini (**state transition**) açığa çıkarır. Bu deseni kullanırken aşağıda verilen durumlar özellikle irdelenmelidir;

- ◊ **Durum geçişleri kim tarafından tanımlanmalıdır?**: Bu desen, hangi şartlarda durumların ayrıştırılacağını belirtmez. Eğer şartlar belirgin ise bu işlem bağlam (**Context**) nesnesi tarafından yapılır. Bu şartlar belirgin değil ve karmaşık ise bu işlem **State** nesnesi ve halefi (**successor**) tarafından yapılır.
- ◊ **Tablo tabanlı alternatif**: Bu durumda durum geçişleri bir tablo aracılığı ile yapılır. Tablo üzerine sağlanacak her bir durum yazılır. Bu durumda, durum geçişlerini sağlayacak değişiklikler yeniden kodlama gerektirmez. Ancak bu yöntemde hız düşer. Durum geçişleri tekdüze (**uniform**) tanımlanmak zorundadır.
- ◊ **State nesnelerini yaratma ve yok etme**: Bu nesneler çalıştırma anında yaratılıp yok edildiklerinden, bu süre zarfında yapılacak işlemler için bir çözüm bulunmalıdır. İki yaklaşım vardır. Birincisi, State nesnesini ihtiyaç olduğunda yaratmak kullanmak ve öldürmektir. İkincisi ise bu nesneleri baştan yaratıp hiç öldürmemektir. Genellikle birinci yöntem tercih edilir.
- ◊ **Dinamik kalıtım (**dynamic inheritance**) kullanma**: Bazı durumlarda nesne, ait olduğu sınıfı çalıştırma anında değiştirir. Bu durumda talep edilen isteğin yerine getirilmesi tam olarak başarılabilir. Ancak birçok nesne yönelimli programlama dili bunu desteklemez.

Bu desen için en gerçekçi örneklerden biri **otomat makineleridir** (**vending machine**). Makinenin davranışı; içinde para olup olmamasına, ürünün kalıp kalmamasına veya ürün seçilip seçilmediğine göre tamamen değişir. Eğer para yoksa "Ürün Seç" düğmesi çalışmaz; para varsa çalışır;

using System;

// 1. State Arayüzü (Soyut Durum)

public abstract class State

{

public abstract void InsertMoney();  
public abstract void EjectMoney();  
public abstract void SelectProduct();  
public abstract void Dispense();

}

// 2. Context: VendingMachine

public class VendingMachine

{

private State \_currentState;  
private int \_count = 0;  
public VendingMachine(int productCount)  
{  
    \_count = productCount;  
    if (\_count > 0)  
        \_currentState = new NoMoneyState(this);  
    else  
        \_currentState = new SoldOutState();  
}

public void SetState(State state)

{

    \_currentState = state;

}

// İstemci tarafındaki komutlar

public void InsertMoney() => \_currentState.InsertMoney();  
public void EjectMoney() => \_currentState.EjectMoney();  
public void SelectProduct() => \_currentState.SelectProduct();  
public void Dispense() => \_currentState.Dispense();  
public void ReleaseProduct()

{

    if (\_count > 0)

    {

        Console.WriteLine(">>> ÜRÜN VERİLDİ. Afiyet olsun!");  
        \_count--;

```

 }
}
public int GetCount() => _count;
}

// 3. Somut Durumlar (Concrete States)
// Para Yok Durumu
public class NoMoneyState : State
{
 private readonly VendingMachine _machine;
 public NoMoneyState(VendingMachine machine) => _machine = machine;
 public override void InsertMoney()
 {
 Console.WriteLine("Para kabul edildi.");
 _machine.SetState(new HasMoneyState(_machine));
 }
 public override void EjectMoney() => Console.WriteLine("Hata: Zaten para yok.");
 public override void SelectProduct() => Console.WriteLine("Hata: Önce para yüklemelisiniz.");
 public override void Dispense() => Console.WriteLine("Hata: Ödeme yapılmadan ürün
 ↪ verilmez.");
}

// Para Var Durumu
public class HasMoneyState : State
{
 private readonly VendingMachine _machine;
 public HasMoneyState(VendingMachine machine) => _machine = machine;
 public override void InsertMoney() => Console.WriteLine("Bilgi: Zaten para yüklü.");
 public override void EjectMoney()
 {
 Console.WriteLine("Para iade ediliyor...");
 _machine.SetState(new NoMoneyState(_machine));
 }
 public override void SelectProduct()
 {
 Console.WriteLine("Ürün seçildi, hazırlanıyor...");
 _machine.SetState(new SoldState(_machine));
 _machine.Dispense(); // Otomatik teslimat tetikleniyor
 }
 public override void Dispense() => Console.WriteLine("Bilgi: Önce ürün seçimi
 ↪ yapmalısınız.");
}

// Ürün Satıldı/Teslimat Durumu
public class SoldState : State
{
 private readonly VendingMachine _machine;
 public SoldState(VendingMachine machine) => _machine = machine;
 public override void InsertMoney() => Console.WriteLine("Lütfen bekleyin: Ürününüz
 ↪ hazırlanıyor.");
 public override void EjectMoney() => Console.WriteLine("Hata: İşlem geri alınamaz, ürün
 ↪ veriliyor.");
 public override void SelectProduct() => Console.WriteLine("Hata: İşlem zaten devam ediyor.");
 public override void Dispense()
 {
 _machine.ReleaseProduct();
 if (_machine.GetCount() > 0)
 _machine.SetState(new NoMoneyState(_machine));
 else
 {
 Console.WriteLine("Dikkat: Son ürün satıldı!");
 _machine.SetState(new SoldOutState());
 }
 }
}

```

```

}
// Stok Yok Durumu
public class SoldOutState : State
{
 public override void InsertMoney() => Console.WriteLine("Hata: Stok bitti, para
 → yükleyemezsiniz.");
 public override void EjectMoney() => Console.WriteLine("Hata: Para yok.");
 public override void SelectProduct() => Console.WriteLine("Hata: Stok bitti.");
 public override void Dispense() => Console.WriteLine("Hata: Stok yok.");
}
// 4. İstemci (Client)
class Program
{
 static void Main()
 {
 VendingMachine machine = new VendingMachine(2);
 Console.WriteLine("--- Senaryo 1: Başarılı Alım ---");
 machine.InsertMoney();
 machine.SelectProduct();
 Console.WriteLine("--- Senaryo 2: Para İadesi ---");
 machine.InsertMoney();
 machine.EjectMoney();
 Console.WriteLine("--- Senaryo 3: Son Ürünü Alma ---");
 machine.InsertMoney();
 machine.SelectProduct();
 Console.WriteLine("--- Senaryo 4: Stok Bittiğinde Deneme ---");
 machine.InsertMoney();
 }
}
/*Program Çıktısı:
--- Senaryo 1: Başarılı Alım ---
Para kabul edildi.
Ürün seçildi, hazırlanıyor...
>>> ÜRÜN VERİLDİ. Afiyet olsun!
--- Senaryo 2: Para İadesi ---
Para kabul edildi.
Para iade ediliyor...
--- Senaryo 3: Son Ürünü Alma ---
Para kabul edildi.
Ürün seçildi, hazırlanıyor...
>>> ÜRÜN VERİLDİ. Afiyet olsun!
Dikkat: Son ürün satıldı!
--- Senaryo 4: Stok Bittiğinde Deneme ---
Hata: Stok bitti, para yükleyemezsiniz.
*/

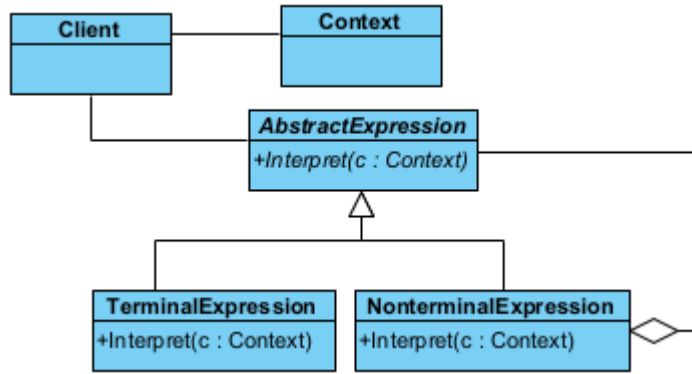
```

### §18.4.11 Yorumlayıcı

**Yorumlayıcı deseni** (**interpreter pattern**), programlama dili yorumlama özelliğini uygulamalarımıza katmanın yolunu gösterir. Yani bir dil bilgisi için bir temsili ve aynı zamanda dil bilgisini anlayıp ona göre hareket etmeyi sağlayacak bir mekanizmayı tanımlar.

Bu desenin amacı, belirli bir gramer yapısına sahip ifadeleri (matematiksel işlemler, SQL komutları vb.) çözümlemektir.

- ◊ Soyut İfade (**AbstractExpression**), tüm düğümlerin sahip olduğu **interpret()** yöntemini tanımlayan ara yüzüdür.
- ◊ Uç İfade (**TerminalExpression**), gramerin en küçük birimleridir (Sayılar veya değişkenler). Daha fazla parçalanamazlar.
- ◊ Uç Olmayan İfade (**NonTerminalExpression**), gramerin kurallarını (Toplama, Çarpma vb.) temsil eder. Genellikle içinde diğer ifadeleri (uç yada veya uç olmayan) barındırır.
- ◊ Bağlam (**Context**), yorumlama sırasında ihtiyaç duyulan küresel bilgileri (Değişken değerleri, sembol tabloları) taşır.



Şekil 18.23: Yorumlayıcı Deseni UML Diyagramı

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
// 1. Context (Bağlam): Değişkenlerin değerlerini saklar.
public class Context
{
 private readonly Dictionary<string, int> _variables = new Dictionary<string, int>();
 public void SetVariable(string name, int value) => _variables[name] = value;
 public int GetVariable(string name)
 {
 if (_variables.TryGetValue(name, out int value)) return value;
 throw new KeyNotFoundException($"Değişken bulunamadı: {name}");
 }
}
// 2. AbstractExpression (Soyut İfade)
public interface IExpression
{
 int Interpret(Context context);
}
// 3. TerminalExpression (Uç İfadeler)
public class NumberExpression : IExpression
{
 private readonly int _number;
 public NumberExpression(int number) => _number = number;
 public int Interpret(Context context) => _number;
}
public class VariableExpression : IExpression
{
 private readonly string _name;
 public VariableExpression(string name) => _name = name;
 public int Interpret(Context context) => context.GetVariable(_name);
}
// 4. NonterminalExpression (Uç Olmayan İfadeler / Operatörler)
public class AddExpression : IExpression
{
 private readonly IExpression _left;
 private readonly IExpression _right;
 public AddExpression(IExpression left, IExpression right)
 {
 _left = left;
 _right = right;
 }
 public int Interpret(Context context)
 {
 return _left.Interpret(context) + _right.Interpret(context);
 }
}

```

```
// 5. İstemci (Client)
class Program
{
 static void Main()
 {
 Context context = new Context();
 context.SetVariable("x", 10);
 context.SetVariable("y", 5);
 // İfade: (x + y) + 20
 // C#'ta nesne ağacı şu şekilde kurulur:
 IExpression expression = new AddExpression(
 new AddExpression(new VariableExpression("x"), new VariableExpression("y")),
 new NumberExpression(20)
);

 Console.WriteLine($"Sonuç: {expression.Interpret(context)}");
 }
}
/* Program Çıktısı:
Sonuç: 35
*/
```

Bu deseni uygularken aşağıda verilen durumlar irdelenmelidir;

- ♦ **Soyut söz dizimin oluşturulması:** Bu desen yorumlanacak dile ilişkin soyut bir söz dizimin nasıl olması gerektiğini açıklamaz. Tabloya dayalı, ağaç yapısı şeklinde el becerisi ile veya doğrudan istemci tarafından yapılabilir.
- ♦ **Yorumlayacak işlemin tanımlanması:** Eğer işlemler ortak bir yapıdaysa yeni bir *Expression* nesnesi tanımlanarak yapılabilir. Daha karmaşık durumlarda ziyaretçi deseni kullanılır. Programlama dillerinin çoğu soyut bir söz dizimine sahiptir ve yorumlanacak öğeler ayrı bir Visitor nesnesi tarafından yorumlanır.
- ♦ **Cümle sonlarını bitiren ortak semboller için sineksiklet deseni kullanılabilir:** Cümle sonlarındaki semboller genellikle bilgi saklamazlar. Yorumlamaya gerek duyulmazlar. Bu nedenle sineksiklet deseni (*flyweight pattern*) yorumlanacak yerler için bir eleme yapar.

Aşağıda bir metindeki ifadeyi hesaplayan bir uygulama örneği verilmiştir;

```
using System;
using System.Collections.Generic;
using System.Linq;
// --- Temel İfade Yapıları ---
public interface IExpression
{
 int Interpret();
}
public class NumberExpression : IExpression
{
 private readonly int _number;
 public NumberExpression(int n) => _number = n;
 public int Interpret() => _number;
}
public class AdditionExpression : IExpression
{
 private readonly IExpression _left, _right;
 public AdditionExpression(IExpression l, IExpression r) { _left = l; _right = r; }
 public int Interpret() => _left.Interpret() + _right.Interpret();
}
public class MultiplicationExpression : IExpression
{
 private readonly IExpression _left, _right;
 public MultiplicationExpression(IExpression l, IExpression r) { _left = l; _right = r; }
 public int Interpret() => _left.Interpret() * _right.Interpret();
}
// --- Dinamik Interpreter Sınıfı ---
```

```

public class Interpreter
{
 public int Evaluate(string expression)
 {
 var expressionTree = BuildExpressionTree(expression);
 return expressionTree.Interpret();
 }
 private IExpression BuildExpressionTree(string rawExpression)
 {
 // Boşluklara göre ayırarak tokenları elde edelim
 string[] tokens = rawExpression.Split(new[] { ' ' },
 ↪ StringSplitOptions.RemoveEmptyEntries);
 Stack<IExpression> nodes = new Stack<IExpression>();
 Stack<char> operators = new Stack<char>();
 // İşlem önceliği sözlüğü: Çarpma (+) dan üstündür
 var precedence = new Dictionary<char, int> { { '+', 1 }, { '*', 2 } };
 void ApplyOp()
 {
 var right = nodes.Pop();
 var left = nodes.Pop();
 char op = operators.Pop();
 if (op == '+')
 nodes.Push(new AdditionExpression(left, right));
 else if (op == '*')
 nodes.Push(new MultiplicationExpression(left, right));
 }
 foreach (string token in tokens)
 {
 if (int.TryParse(token, out int number))
 {
 nodes.Push(new NumberExpression(number));
 }
 else
 {
 char currentOp = token[0];
 while (operators.Count > 0 && precedence[operators.Peek()] >=
 ↪ precedence[currentOp])
 {
 ApplyOp();
 }
 operators.Push(currentOp);
 }
 }
 while (operators.Count > 0) ApplyOp();
 return nodes.Pop();
 }
}
// --- Client (İstemci) ---
class Program
{
 static void Main()
 {
 Interpreter parser = new Interpreter();

 string expr1 = "2 + 3 * 4"; // 14
 string expr2 = "10 * 2 + 5 * 3"; // 35
 string expr3 = "1 + 2 + 3 + 4"; // 10

 Console.WriteLine($"{expr1} = {parser.Evaluate(expr1)}");
 Console.WriteLine($"{expr2} = {parser.Evaluate(expr2)}");
 Console.WriteLine($"{expr3} = {parser.Evaluate(expr3)}");
 }
}

```



}

## 18.5 MVC Mimari Deseni

Model-View-Controller (MVC) mimari deseni (architectural pattern) ilk kez 1979'da *Trygve Reenskaug* tarafından "Applications Programming in Smalltalk-80: How to use Model-View-Controller" makalesiyle tanımlanmıştır. Günümüze kadar bu desenin "Model View Presenter-MVP" gibi birçok türevi geliştirilmiştir.

Bu mimari deseni kullanmak, iş süreci ile ilgili olan katmanın gösterim katmanından bağımsız tasarlanmasını sağlar. Bunun yanında sunum katmanını bağımsız olarak kontrol eden bir nesneye sahip oluruz. Bu katmanlar aslında fiziksel katman değildir. Bu nedenle bu desene uygun bağımsız olan katmanların bakımı kolaylaşır, hem de yeniden kullanımını (**reusing**) artırılmış olunur. Teknik olarak MVC bir sunum katmanı desendir (**UI Pattern**).

MVC desenini Borland firması, Visual Component Library (VCL) ve Component Library for Cross Platform (CLX) bileşenleri altyapısında yıllarca kullanmıştır. Benzer şekilde Apple firması, Cocoa için bu deseni kullanmıştır. Microsoft firması da Document/View mimarisi olarak yıllarca kullanmıştır. Günümüzde .NET Framework Web Uygulamaları bu kapsamda daha kolay ve yönetilebilir olarak geliştirilmektedir.

### §18.5.1 MVC Deseni Temel Yapısı

MVC (**Model-View-Controller**), bir yazılım projesinde kodları görevlerine göre üç ana parçaya ayıran bir mimari desendir. Temel amacı, veriyi (mantık), kullanıcı arayüzünü ve bu ikisi arasındaki iletişimi birbirinden bağımsız hale getirerek kodun yönetimini kolaylaştırmaktır. Bunlar aşağıda detaylı olarak açıklanmıştır;

- ♦ **Model** : Sadece işlevsel davranışları yerine getiren kodlar burada toplanır. Kullanıcıya gösterilen ara yüzde kullanıcının gördüğü ve isteklerini ilettiği kısım dışında kalan konular bu katmanda gerçekleştirilir. Bazen kullanıcı tarafından modelin o andaki durumu (**state**) hakkında bilgi almasına izin verilir. Gösterimler (**view**) bilgiyi modelden aldığından, önceden modele kayıt (**subscribe**) olmak zorundadır. Bunun yanında model üzerinde bilgi değişikliği olduğunda bütün gösterimler bundan haberdar edilir (**notify**). Uygulamanın beynidir. Veritabanı işlemleri, hesaplamalar ve kurallar burada döner. "Veri nedir?" sorusuna cevap verir.
- ♦ **Gösterim (View)**: MVC deseninde birden fazla gösterim olmasına izin verilir. Gösterim, modele ilişkin bir kullanıcı ara yüzüdür. Yani modelden kullanıcıya gösterilecek değerleri alır ve kullanıcıya gösterir. Kullanıcının gördüğü ekrandır (HTML, CSS, butonlar). Sadece veriyi gösterir, arka planda ne olduğuyla ilgilenmez. "Veri nasıl görünmeli?" sorusuna cevap verir.
- ♦ **Denetçi (Controller)**: Denetçi, kullanıcı tarafından etkileşilen gösterimden gelen isteklere göre modeli günceller. Denetçi modeli güncellemek için, modelin değişik yöntemlerini çağırır ve modelin kendi durumunu (**state**) değiştirmesini sağlar. Buna bağlı olarak da model, tüm gösterimleri bundan haberdar eder. Kullanıcıdan gelen istekleri karşılar. Modelden veriyi alır ve hangi gösterimde gösterileceğine karar verir. "Hangi veri nereye gitmeli?" sorusuna cevap verir.

Basit bir restoran senaryosunda MVC'yi bir restorana benzetebiliriz;

- ♦ **Aşçı (Model)**: Mutfağın içindedir. Yemeği hazırlar, malzemeleri kontrol eder. Müşteriyi görmez, sadece siparişe göre yemeği (veriyi) üretir.
- ♦ **Yemek Masası/Sunum (View)**: Müşterinin önüne gelen tabak ve sunumdur. Müşteri yemeğin nasıl piştiğini görmez, sadece sunulan tabağı görür.
- ♦ **Garson (Controller)**: Müşteriden siparişi alır, aşçıya iletir. Yemek piştiğinde ise tabağı alır ve müşteriye servis eder.

MVC kullanmalıyız çünkü;

- ♦ **Kod Karmaşasını Önler**: Tasarımcı sadece **View** ile, yazılımcı sadece **Model/Controller** ile ilgilenir.
- ♦ **Bakım Kolaylığı**: Görünümü değiştirmek istediğinizde veritabanı koduna dokunmanız gerekmez.
- ♦ **Yeniden Kullanılabilirlik**: Aynı **Model** verisini (örneğin kullanıcı listesi), hem bir web sayfasında hem de bir mobil uygulamada farklı **View**'lar ile gösterebilirsiniz.

C# nesne yönelimli bir dil olduğu için MVC yapısını sınıflar arasındaki ilişkiyle kurgulamak, mantığı anlamak açısından çok faydalıdır. Bir "Öğrenci Bilgi Sistemi" üzerinden, bileşenlerin birbirine bağımlılığını (**coupling**) minimize edecek şekilde bir örnek verilebilir. Öğrenci Bilgi Sisteminde her bileşen kendi sorumluluğunu üstlenir: **Model** veriyi tutar, **View** ekrana basar, **Controller** ise akışı yönetir;

1. Veri ve İş Mantığı (**Model**): Sadece veriyi ve bu veriye erişim yöntemlerini içerir. Görselleştirme ile hiç ilgilenmez.

```
// 1. Model: Veriyi ve iş mantığını temsil eder.
public class OgrenciModel
{
```

```

public string Isim { get; set; }
public string Numara { get; set; }
public OgrenciModel(string isim, string numara)
{
 Isim = isim;
 Numara = numara;
}

```

2. Görünüm (View): Verinin kullanıcıya nasıl sunulacağını belirler. Genellikle sadece "Yazdır" veya "Göster" gibi yöntemleri vardır.

// 2. View: Verinin kullanıcıya nasıl sunulacağından sorumludur.

```

public class OgrenciView
{
 public void OgrenciDetaylariniBas(string isim, string numara)
 {
 Console.WriteLine("--- Öğrenci Bilgileri ---");
 Console.WriteLine($"Ad: {isim}");
 Console.WriteLine($"No: {numara}");
 Console.WriteLine("-----");
 }
}

```

3. Denetleyici (Controller): Veri ve Gösterim arasındaki köprüdür. Kullanıcıdan gelen güncellemeleri Veriye yansıtır ve Gösterimi güncellenmeye zorlar.

// 3. Controller: Model ve View arasındaki köprüdür.

```

public class OgrenciController
{
 private readonly OgrenciModel _model;
 private readonly OgrenciView _view;
 public OgrenciController(OgrenciModel model, OgrenciView view)
 {
 _model = model;
 _view = view;
 }
 // Model üzerindeki veriyi güncelleme/okuma proxy metodları
 public void SetOgrenciIsim(string isim) => _model.Isim = isim;
 public string GetOgrenciIsim() => _model.Isim;

 public void SetOgrenciNumara(string numara) => _model.Numara = numara;
 public string GetOgrenciNumara() => _model.Numara;
 // View'u güncelleme komutu
 public void GorunumuGuncelle()
 {
 _view.OgrenciDetaylariniBas(_model.Isim, _model.Numara);
 }
}

```

4. Sistemin Çalıştırılması:

// 4. İstemci (Client)

```

class Program
{
 static void Main()
 {
 // 1. Model'i (Veriyi) oluştur
 OgrenciModel model = new OgrenciModel("Ahmet Öztürk", "2026001");
 // 2. View'u (Arayüzü) oluştur
 OgrenciView view = new OgrenciView();
 // 3. Controller'ı bağla
 OgrenciController controller = new OgrenciController(model, view);
 // İlk hali ekrana bas
 controller.GorunumuGuncelle();
 // Veriyi Controller üzerinden güncelle
 controller.SetOgrenciIsim("Mehmet Demir");
 // Güncel hali tekrar bas
 }
}

```

```

 Console.WriteLine("Veri güncellendikten sonra:");
 controller.GorunumuGuncelle();
 }
}
/* Program Çıktısı:
--- Öğrenci Bilgileri ---
Ad: Ahmet Öztürk
No: 2026001

Veri güncellendikten sonra:
--- Öğrenci Bilgileri ---
Ad: Mehmet Demir
No: 2026001

*/

```

Buradaki mimari ile ilgili şunlar söylenebilir;

- ◊ **Bağımlılık Yönetimi (dependency management)** : MVC'nin ana varlık sebebi **gevşek bağıllık (loose coupling)** oluşturmaktır.
  - **Tek Yönlü Bağımlılık**: İdeal bir MVC yapısında bağımlılık hiyerarşisi şöyledir:  
`View → Controller → Model`  
`Model`, kendisini kimin görüntülediğini veya kimin yönettiğini bilmez. Bu, `Model`'in farklı platformlarda (Web, Mobil, Konsol) tekrar kullanılabilmesini sağlar.
  - **Arayüzler (interfaces)**: `Controller`, `View`'a doğrudan bağımlı olmak yerine bir `IView` arayüzüne bağımlı hale getirilir. Bu sayede birim testleri (**unit test**), gerçek bir ekran olmadan sahte (**mock**) nesnelerle `Controller`'ı test edebilir.
- ◊ **Bellek Yönetimi (memory management)**: MVC yapılarında nesnelerin yaşam döngüsü (**lifetime**), bellek sızıntılarını önlemek için dikkatle yönetilmelidir.
  - **Yaşam Süresi Sorumluluğu**: Genellikle `Controller`, `View` ve `Model`'den daha kısa ömürlüdür. Bir işlem bittiğinde `Controller` bellekten düşebilir ancak `Model` (veri) kalmaya devam edebilir.
  - **Çöp Toplayıcı GC**: `Controller` içinde `Model`'e ait olaylara (**events**) abone olursanız ve aboneliği iptal etmezseniz, dinlemeyi bırakanlar (**lapsed listener**) denilen bellek sızıntısı oluşur. `Controller` yok olsa bile, `Model` içindeki olay referansı `Controller`'ı bellekte tutmaya devam eder.
- ◊ **Gözlemci Deseni (observer pattern) ile İlişkisi**: MVC'nin "gizli sosu" genellikle gözlemci (**observer**) desenidir.
  - **Model-View Senkronizasyonu**: Model'deki veri değiştiğinde View'un otomatik güncellenmesi isteniyorsa, Model bir Subject (Konu), View ise bir Observer (Gözlemci) rolü üstlenir.
  - **Olaylar ve Temsilciler**: İletişim **event** anahtar kelimesiyle çok doğal bir şekilde yapılır. `Model` bir veri değiştiğinde `OnDataChanged` olayını ateşler, `View` bu olayı dinler ve kendini yeniler.
  - **Denetçinin Rolü**: Bazı katı MVC yorumlarında `View` asla `Model`'i dinlemez; tüm değişim bildirimleri `Controller` üzerinden geçer. Ancak modern yaklaşımlarda, `View` doğrudan `Model`'deki (veya `ViewModel`'deki) değişimleri gözlemler.

## §18.5.2 Gözlemci ile MVC Deseni İçin Örnek Proje

Bu deseni bir bardak ve bu bardağı dolduran garson örneğini ele alarak inceleyelim. MVC deseni için bardağı dolduracak ya da boşaltacak olan garson, denetçi; Doldurulacak ve boşaltılacak olan bardak ise modeldir. Bu örnek projede MVC mimarisinin bir yardımcı bileşen kullanmadan bir projede nasıl uygulandığı detaylı olarak anlatılacaktır.

### Projenin Oluşturulması

Bunun için Visual Studio üzerinde ile bir Windows uygulaması örnek olarak oluşturulacaktır. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```

MVCWindowsUygulaması/ → Çözüm (Solution)
├── WinFormsApp1/ → Proje (Windows Forms Application Project)
│ ├── IController.cs → Interface (IController)
│ ├── IModel.cs → Interface (IModel)
│ ├── IView.cs → Interface (IView)
│ ├── Controllers.cs → Class (Controller)
│ ├── Models.cs → Class (Model)
│ ├── ProgressBarView.cs → UserControl (View)
│ └── PieView.cs → UserControl (View)

```

```

├─ Form1.cs → Form (Windows Form)
└─ Program.cs → Ana (Main) Program

```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File→New→Project/Solution...** seçilerek **MVCWindowsUygulaması** adlı çözüm içinde **WinFormsApp1** adlı projemiz **Windows Forms App** seçilerek oluşturulur.
2. Oluşan projede **Form1** adlı bir form ve **Program.cs** dosyasını otomatik olarak oluştur.

### Projede MVC Ara Yüzlerinin Oluşturulması

Proje açık iken **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **IController.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```

public interface IController
{
 //Denetçinin (garsonun) Davranışları
 void AlınanDoldurmaİsteği(int pArtırımOranı);
 void AlınanBoşaltmaİsteği(int pEksilmeOranı);
}

```

Daha sonra **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **IModel.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```

/* Doluluk oranı, en az, en çok ne kadar doldurulacağı ve adı belirlenecek bir bardağın yani
→ modelin, denetçiden (garson) gelen istekleri yerine getirir.*/
public interface IModel
{
 //Özellikler
 string Adı { get; set; }
 int DolulukOranı { get; }
 int EnAz { get; }
 int EnÇok { get; }
 //Davranışlar
 void Doldur(int pArtırımOranı);
 void Boşalt(int pEksilmeOranı);
}

```

Sonra **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **IView.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```

/* Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
→ tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
→ çıkarır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
→ boşaltma isteğine gösterim izin vermemelidir. */
public interface IView
{
 //View Davranışları
 void DoldurmaAktif();
 void DoldurmaPasif();
 void BoşaltmaAktif();
 void BoşaltmaPasif();
}

```

Son olarak **IController** ara yüzümüze hangi model ile çalışacağını söyleyen değişiklikler de yapılır;

```

public interface IController
{
 //Denetçinin (garsonun) Davranışları
 void AlınanDoldurmaİsteği(int pArtırımOranı);
 void AlınanBoşaltmaİsteği(int pEksilmeOranı);
 //MODEL Özelliği
 IModel Model { set; get; }
}

```

### Projeye Gözlemci Deseninin Uygulanması

Bundan sonraki aşama biraz daha ustalık istemektedir. Çünkü modeldeki değişikliklerden gösterimin haberi olmasını istiyoruz. İşte bunun için Gözlemci desenini bu ara yüze uyguluyoruz. Bunun için ilk önce

`IModel` ara yüzü aşağıdaki şekilde değiştirilir;

/\* Doluluk oranı, en az, en çok ne kadar doldurulacağı ve adı belirlenecek bir bardağın yani  
→ modelin, denetçiden (garson) gelen istekleri yerine getirir.\*/

```
using System.Collections; //ArrayList için
public interface IModel
{
 //Özellikler
 string Adı { get; set; }
 int DolulukOranı { get; }
 int EnAz { get; }
 int EnÇok { get; }
 //Davranışlar
 void Doldur(int pArtırımOranı);
 void Boşalt(int pEksilmeOranı);
 //Gözlemci Desenine İlişkin Davranışlar
 ArrayList Observers { get; }
 void AddObserver(IView pView);
 void RemoveObserver(IView pView);
 void NotifyObservers();
}
```

Devamında `IView` ara yüzü de aşağıdaki şekilde değiştirilir;

/\* Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi  
→ tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya  
→ çıkarır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için  
→ boşaltma isteğine gösterim izin vermemelidir. \*/

```
public interface IView
{
 //View Davranışları
 void DoldurmaAktif();
 void DoldurmaPasif();
 void BoşaltmaAktif();
 void BoşaltmaPasif();

 //Gözlemci Deseni Davranışları: Modelden gelen bilgilere göre View güncellenir
 void Update(IModel pModel);
}
```

Son olarak gösterim ara yüzünden alınacak kullanıcı isteklerini denetçiye iletmek için `IView` ara yüzü aşağıdaki gibi değiştirilir;

/\* Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi  
→ tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya  
→ çıkarır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için  
→ boşaltma isteğine gösterim izin vermemelidir. \*/

```
public interface IView
{
 //View Davranışları
 void DoldurmaAktif();
 void DoldurmaPasif();
 void BoşaltmaAktif();
 void BoşaltmaPasif();

 //View üzerinden kullanıcı isteklerini Controller'e iletmek için özellik tanımlama
 IController Control { set; }

 //Gözlemci Deseni Davranışları:
 //Modelden gelen bilgilere göre View güncellenir
 void Update(IModel pModel);
}
```

### Model ve Denetçi Arayüzleri Gerçekleştirecek Sınıfların Tanımlanması

Artık verilen örneği gerçekleştirmek için yüzlerimiz hazır. Bu ara yüzlere bağlı kalarak sırasıyla soyut ve somut bir model (bardak), denetçiyi (hızlı veya çalışan garson) oluşturabiliriz.

Proje açık iken **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **Models.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```
namespace WinFormsApp1
{
 using System.Collections; //ArrayList için
 abstract class SoyutKap : IModel
 {
 private ArrayList views = new ArrayList();
 private string adi;
 private int dolulukOranı;
 private int enAz;
 private int enÇok;
 public SoyutKap(string pAdi, int pEnAz, int pEnÇok,
 int pDolulukOranı)
 {
 this.adi = pAdi;
 this.enAz = pEnAz;
 this.enÇok = pEnÇok;
 this.dolulukOranı = pDolulukOranı;
 }
 public string Adı
 {
 get { return adi; }
 set { adi = value; }
 }
 public int DolulukOranı
 {
 get { return dolulukOranı; }
 }
 public int EnAz
 {
 get { return enAz; }
 }
 public int EnÇok
 {
 get { return enÇok; }
 }
 public void Doldur(int pArtırımOranı)
 {
 this.dolulukOranı += pArtırımOranı;
 if (dolulukOranı >= this.enÇok) dolulukOranı = enÇok;

 this.NotifyObservers();
 }
 public void Boşalt(int pEksilmeOranı)
 {
 this.dolulukOranı -= pEksilmeOranı;
 if (dolulukOranı <= this.enAz) dolulukOranı = enAz;

 this.NotifyObservers();
 }
 public ArrayList Observers
 {
 get { return views; }
 }
 public void AddObserver(IView pView)
 {
 views.Add(pView);
 }
 }
}
```

```

 }
 public void RemoveObserver(IView pView)
 {
 views.Remove(pView);
 }
 public void NotifyObservers()
 {
 foreach (IView view in views)
 {
 view.Update(this);
 }
 }
}
class somutBardak : SoyutKap
{
 public somutBardak(string pAdi) : base(pAdi, 0, 100, 0) { }
}
}

```

Proje açık iken **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **Controllers.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```

namespace WinFormsApp1
{
 class HızlıGarson : IController
 {
 private IModel model;
 public IModel Model
 {
 set { model = value; }
 get { return model; }
 }
 public void AlınanDoldurmaİsteği(int pArtırımOranı)
 {
 if (model != null)
 {
 model.Doldur(pArtırımOranı * 10);
 DurumKontrol();
 }
 }
 public void AlınanBoşaltmaİsteği(int pEksilmeOranı)
 {
 if (model != null)
 {
 model.Boşalt(pEksilmeOranı * 10);
 DurumKontrol();
 }
 }
 public void DurumKontrol()
 {
 foreach (IView view in model.Observers)
 {
 if (model.DolulukOranı >= model.EnÇok)
 view.DoldurmaPasif();
 else
 view.DoldurmaAktif();

 if (model.DolulukOranı <= model.EnAz)
 view.BoşaltmaPasif();
 else
 view.BoşaltmaAktif();
 }
 }
 }
}

```



```

 }
}
class YavaşGarson : IController
{
 private IModel model;
 public IModel Model
 {
 set { model = value; }
 get { return model; }
 }
 public void AlınanDoldurmaİsteği(int pArtırımOranı)
 {
 if (model != null)
 {
 model.Doldur(pArtırımOranı);
 DurumKontrol();
 }
 }
 public void AlınanBoşaltmaİsteği(int pEksilmeOranı)
 {
 if (model != null)
 {
 model.Boşalt(pEksilmeOranı);
 DurumKontrol();
 }
 }
 public void DurumKontrol()
 {
 foreach (IView view in model.Observers)
 {
 if (model.DolulukOranı >= model.EnÇok)
 view.DoldurmaPasif();
 else
 view.DoldurmaAktif();

 if (model.DolulukOranı <= model.EnAz)
 view.BoşaltmaPasif();
 else
 view.BoşaltmaAktif();
 }
 }
}
}

```

### Gösterim Arayüzleri için Kullanıcı Kontrollerinin Tanımlanması

Artık gösterimler için örnek olarak iki farklı **kullanıcı kontrolü** (**user control**) tasarlanacaktır. Bunlardan biri **ProgressBarView** kontrolüdür.

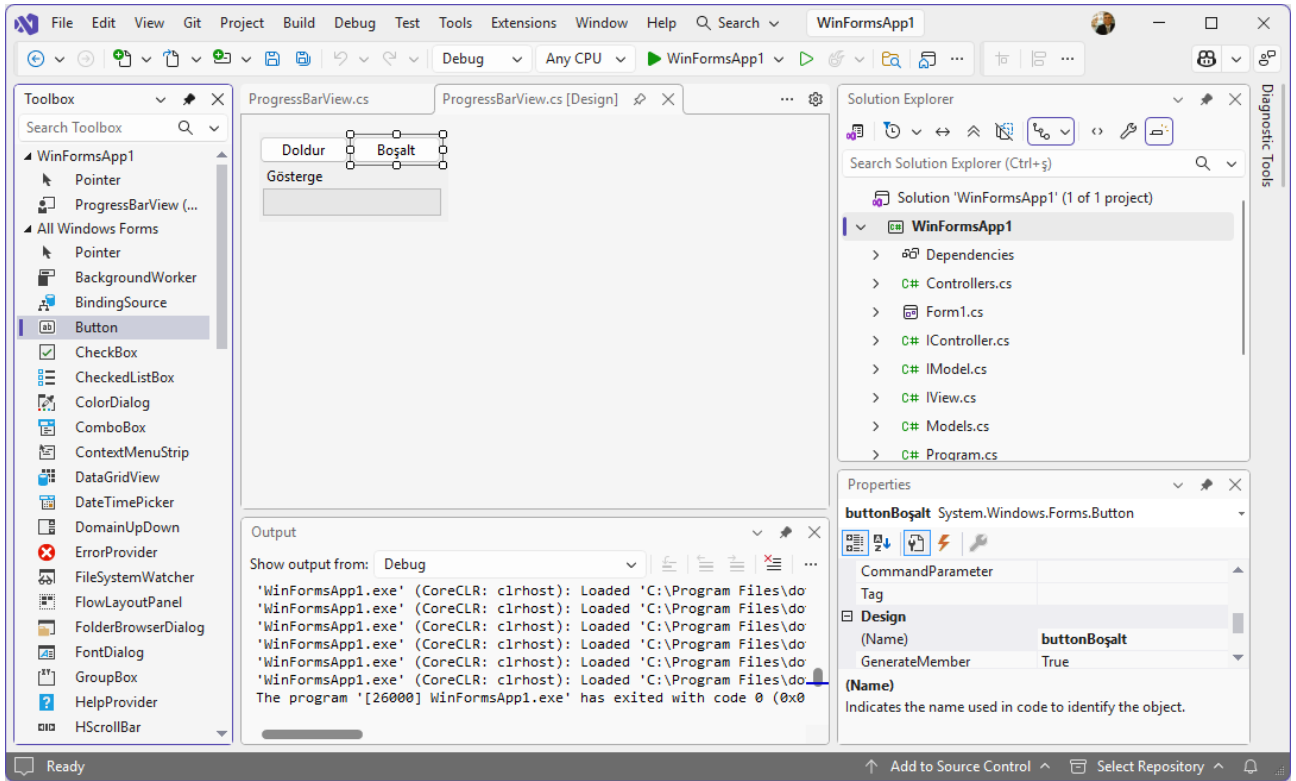
Açık olan projemize kullanıcı kontrolü eklemek için **Solution Explorer** üzerinde projemize sağ tıklanarak **Add → User Control (Windows Forms)...** seçilerek seçilir ve dosya adı **ProgressBarView.cs** belirlenerek kontrolümüz için dosya oluşturulur. Daha sonra **View → Toolbox** tıklanır ve açılan penceren kullanıcı kontrolüne sürükleyip bırak ile iki adet **Button** ve bir adet **ProgressBar** ve bir adet **Label** eklenir (Şekil 18.24);

Eklenen kontrollerin **Name** özellikleri **buttonDoldur**, **buttonBoşalt**, **labelGösterge** ve **progresBarDoluluk** olarak değiştirilir. Sonrasında butonların **onClick()** olayına çift tıklanır. Açılan **ProgressBarView.cs** dosyasında aşağıdaki değişiklikler yapılır;

```

namespace WinFormsApp1
{
 public partial class ProgressBarView : UserControl, IView //IView elle eklendi ve
 ↪ gerçekleştirildi.
 {
 private IController _control; //Elle Eklendi
 public IController Control

```



Şekil 18.24: ProgressBarView Kullanıcı Kontrolü

```

{
 set { //Elle Eklendi
 _control = value;
 }
}

public ProgressBarView()
{
 InitializeComponent();
}

private void buttonDoldur_Click(object sender, EventArgs e)
{
 _control.AlınanDoldurmaİsteği(1); //elle eklendi
}

private void buttonBoşalt_Click(object sender, EventArgs e)
{
 _control.AlınanBoşaltmaİsteği(1); //elle eklendi
}

public void DoldurmaAktif()
{
 this.buttonDoldur.Enabled = true; // elle eklendi
}

public void DoldurmaPasif()
{
 this.buttonDoldur.Enabled = false; //elle eklendi
}

public void BoşaltmaAktif()
{
 this.buttonBoşalt.Enabled = true; //elle eklendi
}

```

```

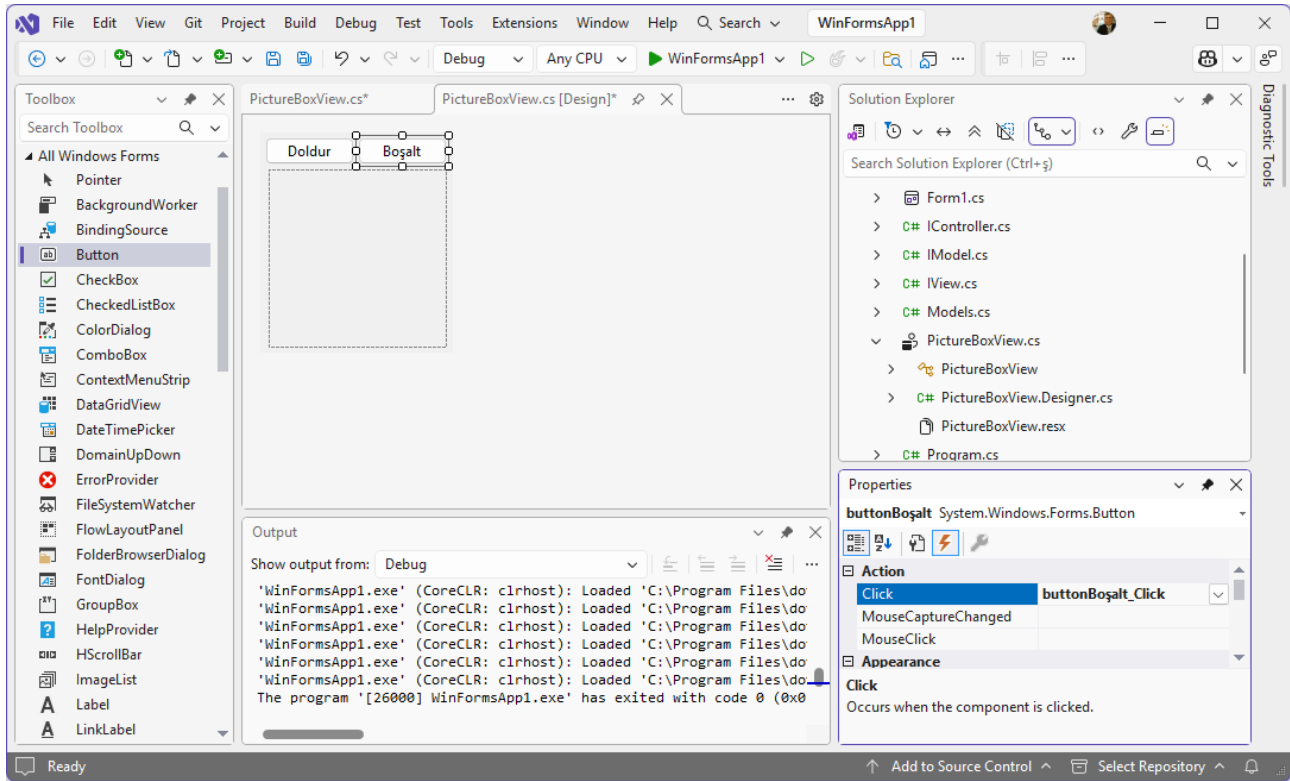
 }

 public void BoşaltmaPasif()
 {
 this.buttonBoşalt.Enabled = false; //elle eklendi
 }

 public void Update(IModel pModel)
 {
 //elle eklendi:
 this.labelGösterge.Text = string.Format("{0} Doluluğu (%):{1}",
 pModel.Adı, pModel.DolulukOranı);
 this.progressBarDoluluk.Value = pModel.DolulukOranı * 100 / pModel.EnÇok;
 }
}
}

```

İkinci kullanıcı kontrolü ise **PieView** kontrolüdür. Bunun için mevcut projemize **PictureBoxView** adında yeni bir kullanıcı kontrolü eklenir. Kullanıcı kontrolüne **Toolbox** üzerinden sürükler bırak ile iki adet **Button**, ve bir adet **PictureBox** eklenir (Şekil 18.25).



Şekil 18.25: PictureBox Kullanıcı Kontrolü

Eklenen kontrollerin **Name** özellikleri **buttonDoldur**, **buttonBoşalt** ve **pictureBoxPie** olarak değiştirilir ve butonların **onClick()** olayına çift tıklanır. Açılan **PieView.cs** dosyasında aşağıdaki değişiklikler yapılır;

```

namespace WinFormsApp1
{
 public partial class PictureBoxView : UserControl, IView //Iview eklendi ve gerçekleştirildi
 {
 private IController _control;
 public IController Control
 {
 set
 {
 _control = value;
 }
 }
 }
}

```

```

public PictureBoxView()
{
 InitializeComponent();
}

private void buttonDoldur_Click(object sender, EventArgs e)
{
 _control.AlınanDoldurmaİsteği(1); //Elle Eklendi
}

private void buttonBoşalt_Click(object sender, EventArgs e)
{
 _control.AlınanBoşaltmaİsteği(1); //Elle Eklendi
}

public void DoldurmaAktif()
{
 this.buttonDoldur.Enabled = true; //Elle Eklendi
}

public void DoldurmaPasif()
{
 this.buttonDoldur.Enabled = false; //Elle Eklendi
}

public void BoşaltmaAktif()
{
 this.buttonBoşalt.Enabled = true; //Elle Eklendi
}

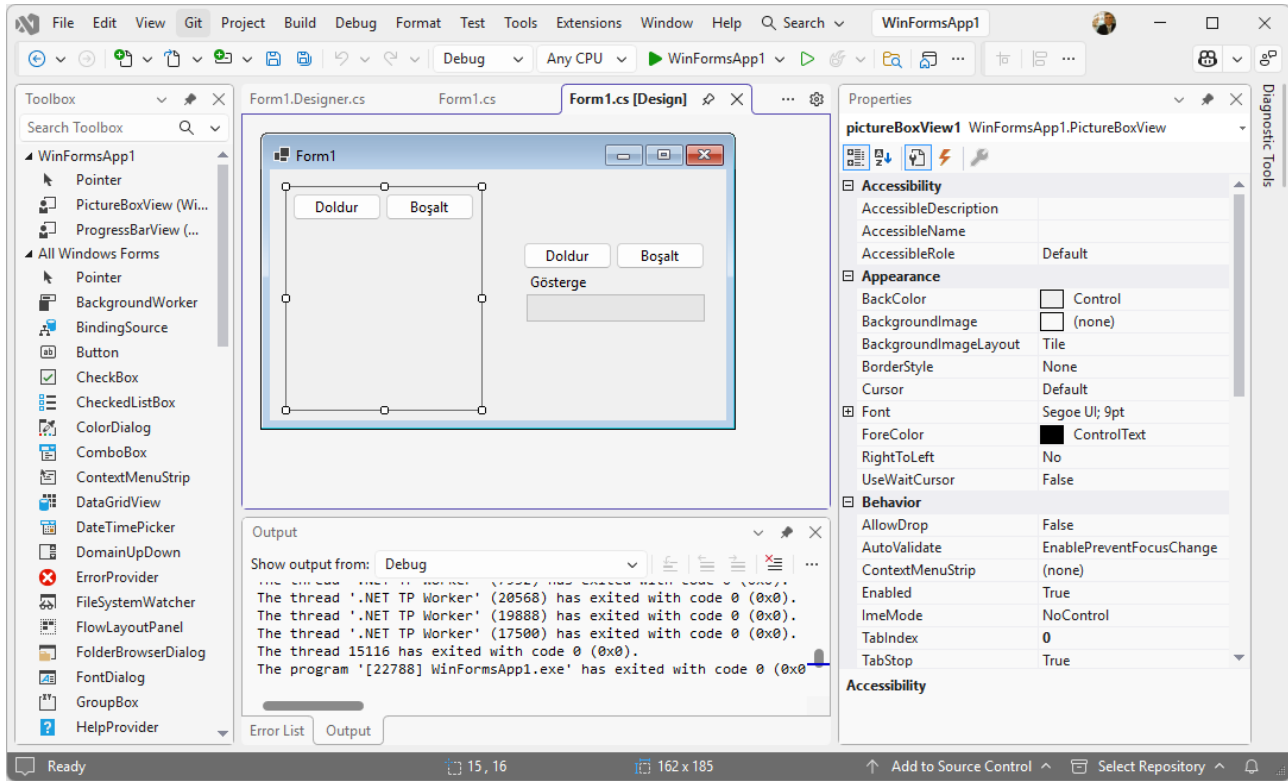
public void BoşaltmaPasif()
{
 this.buttonBoşalt.Enabled = false; //Elle Eklendi
}

public void Update(IModel pModel)
{
 //Elle Eklendi:
 SolidBrush whiteBrush = new SolidBrush(Color.White);
 SolidBrush redBrush = new SolidBrush(Color.Red);
 Graphics graphics = this.pictureBoxPie.CreateGraphics();
 graphics.FillRectangle(whiteBrush, 0, 0,
 this.pictureBoxPie.Width,
 this.pictureBoxPie.Height);
 graphics.FillPie(redBrush, 0, 0,
 this.pictureBoxPie.Width,
 this.pictureBoxPie.Height, 0.0F,
 pModel.DolulukOranı * 360.0F / pModel.EnÇok);
 Font font = new Font(FontFamily.GenericSerif, 12);
 Point point = new Point(this.pictureBoxPie.Width / 2,
 this.pictureBoxPie.Height / 2);
 SolidBrush blackBrush = new SolidBrush(Color.Black);
 graphics.DrawString(string.Format("%{0}",
 pModel.DolulukOranı), font, blackBrush, point);
}
}

```

### Asıl Forma Kullanıcı Kontrollerini Ekleme ve Çalıştırma

Kullanıcı Kontrollerimizi tanımladıktan sonra uygulamanın asıl form olan `Form1.cs` formuna dönüp bu kontrolleri ekleyebiliriz. Bu forma kullanıcı kontrollerimizi **Toolbox** üzerinden sürükleyip bırakarak ekleyebiliriz.



Şekil 18.26: WinFormApp1 Uygulamasında Form1

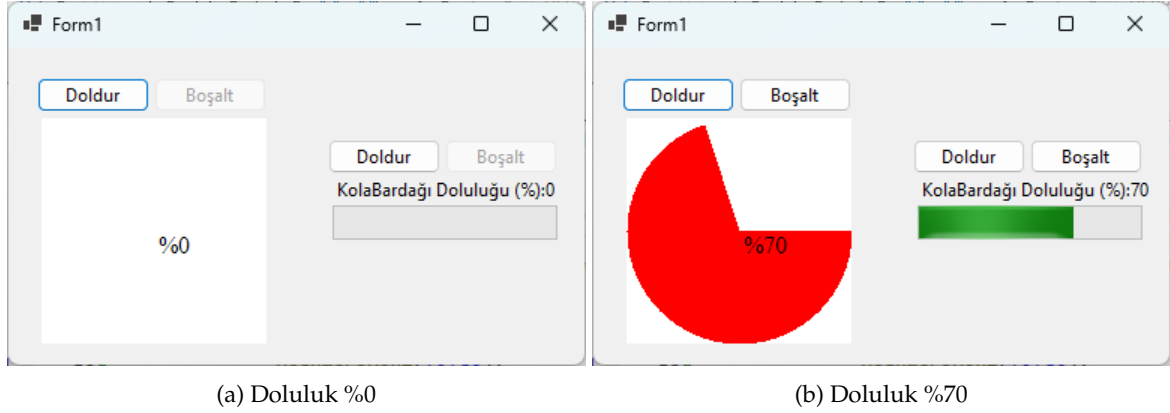
Formun kod kısmına geçerek Form1.cs dosyasında aşağıdaki değişiklikleri yaparak programı son haline getiririz.

```
namespace WinFormsApp1
{
 public partial class Form1 : Form
 {
 public Form1()
 {
 InitializeComponent();
 MVCBaşlat();
 }

 public void MVCBaşlat()
 {
 //Controller
 IController garson = new HızlıGarson();
 //Model
 IModel bardak = new somutBardak("KoLaBardağı");
 //Denetçinin Modeli
 garson.Model = bardak;
 //Her bir Gösterime Denetçi Ataması
 pieView1.Control = garson;
 progressBarView1.Control = garson;
 //Modele Gösterimlerin Kayıt Olması
 bardak.AddObserver(pieView1);
 bardak.AddObserver(progressBarView1);
 }
 }
}
```

Program çalıştırıldığında, MVC deseni ile birbirine gevşek bağlı (**loose couple**) denetleme ara yüzlerinin yazılımlara kolay eklendiğini görebiliriz. Ayrıca programda yapılacak değişiklik isteklerine MVC deseni daha hızlı cevap verecektir. Bunun yanı sıra hazırlanan ara yüzlerle birlikte, soyut sınıfları da başka projelere taşıyıp

yeniden kullanmak (**reusing**) oldukça kolay olacaktır. Böylece ileride olabilecek değişikliklere daha hızlı ayak uydurabilen daha esnek bir projeye sahip olmuş oluruz. Sonuç olarak MVC deseni, kullanıcı ara yüzü işlemleri ile iş mantığının birbirinden ayrılmasını sağlar. Ayrıca Model, Gösterim ve Denetçi bileşenleri ayrı katmanlar halinde Katmanlı Mimari başlığı altında anlatıldığı şekilde tasarlanabilir.



Şekil 18.27: WinFormApp1 Uygulamasının Çalışması

ASP.NET MVC ve Jakarta EE gibi günümüzde en çok kullanılan teknolojiler MVC ara yüzlerini web uygulamasına entegre edilmiş olarak sunmaktadır. ASP.NET MVC altyapısında gösterim bileşeni ASP.NET web sayfası olacak şekilde hazırlanır. Jakarta EE altyapısında ise sunum bileşeni JSP olacak şekilde hazırlanır. Model ve Denetçi bileşenleri ise genellikle nesneye dayalı sınıflar olarak tasarlanırlar.

## 18.6 Düşün ve Çöz

- ◊ **Düşün:** Tekil Nesne (**Singleton**) deseni çok iş parçacıklı (**multi-threaded**) ortamlarda neden tehlikeli olabilir? İş parçacığı-güvenli **Singleton** kodlaması (**implementation**) için "double-checked locking" ve **Lazy<T>** yaklaşımlarını karşılaştırınız.
- ◊ **Düşün:** Fabrika Yöntemi ile Soyut Fabrika deseni arasındaki fark nedir? Bir oyun motoru geliştirirken farklı platformlar (PC, Konsol, Mobil) için hangi deseni kullanırsınız?
- ◊ **Düşün:** Gözlemci deseni ile **event/delegate** mekanizması arasındaki ilişki nedir? Gözlemci deseni yanlış uygulandığında bellek sızıntısına nasıl yol açabilir?
- ◊ **Düşün:** Strateji deseni ve Durum deseni yapısal olarak birbirine benzer görünse de farklı niyetlere hizmet eder. Bu farkı bir varlık (örneğin bir sipariş) üzerinden açıklayınız.
- ◊ **Düşün:** Komut (**Command**) deseni geri alma (**undo**) işlevini nasıl mümkün kılar? Metin editöründeki **Ctrl+Z** mekanizmasını bu deseni nasıl modellersiniz?
- ◊ **Düşün:** Dekoratör deseni kalıtıma alternatif olarak neden tercih edilebilir? Bir kahve siparişi sistemi üzerinden dekoratörün esnekliğini gösteriniz.
- ◊ **Düşün:** MVC deseni ile MVP (**Model-View-Presenter**) ve MVVM (**Model-View-ViewModel**) arasındaki farklar nelerdir? WPF veya MAUI geliştirmesinde hangisi daha yaygın kullanılır?
- ◊ **Çöz:** Bir metin editörü için Komut deseni uygulayınız: Yazma, Silme ve Geri Alma işlemlerini destekleyen bir komut yığını (**command stack**) oluşturun.
- ◊ **Çöz:** Farklı veritabanı motorlarına (SQLite, MSSQL, PostgreSQL) bağlanabilen bir uygulama için Soyut Fabrika deseni kullanarak bağlantı ve sorgu nesnelerini üretin.
- ◊ **Çöz:** Bir bildirim sistemi için Gözlemci deseni uygulayınız: **HavaDurumu** sınıfı sıcaklık değiştiğinde kayıtlı tüm gözlemcileri (**SicaklikGostergesi**, **IklimlendirmeSistemi**) haberdar etsin.
- ◊ **Çöz:** Dekoratör deseni kullanarak bir metin biçimlendirme sistemi oluşturunuz: Kalın, İtalik ve Altı Çizili biçimleri herhangi bir kombinasyonda uygulanabilsin.
- ◊ **Çöz:** Strateji deseni kullanarak aşağıda verilen farklı sıralama algoritmalarını **KabarcıkSiralama**, **HızlıSiralama**, **BirlestirmeSiralama** arasında çalışma zamanında değiştirilebilen bir sıralayıcı tasarlayınız.

## 19. Bölüm

# Kurumsal Yazılımlar

## 19.1 Kurumsal Yazılım İçin Gereklikler

### §19.1.1 İş Analizi ve Gereksinim Analizi

**İş analizi (business analysis)**, kurumun hedeflerine ulaşması için gerekli olan değişimleri yönetme disiplinidir. Sadece bir yazılım projesiyle sınırlı değildir. Büyük resmi inceler. Organizasyonel yapı değişikliği, yeni bir pazara giriş veya maliyet azaltma stratejileri gibi konular da iş analizinin konusudur. "Şu an neredeyiz?", "Nereye gitmek istiyoruz?" ve "Varmak istediğimiz yer için en mantıklı yatırım nedir?" gibi soruların cevabını arar. Bunun için; Pazar analizi, rakip analizi, yatırım getirisi hesaplamaları ve paydaş yönetimi konuları içerir.

**Gereksinim analizi (requirement analysis)**, iş analizinin bir alt kümesidir. İş analizi tarafından belirlenen "ihtiyaç", somut bir çözüme dönüştürülmesi için gereken tüm teknik ve fonksiyonel detayların ayıklandığı süreçtir. "İş hedefine ulaşmak için sistemin veya ürünün tam olarak ne yapması gerekiyor?" sorusunun cevabını verir. Bunun için; Kullanıcı senaryoları yazma, veri modelleri oluşturma, kısıtları belirleme ve onay süreçlerini yönetme gibi konuları irdeler.

Bir bankanın müşteri kaybını azaltmak istediğini varsayalım:

- ◊ **İş Analizi Kısmı:** Analist, verileri inceleyerek müşterilerin mobil uygulamanın yavaşlığı nedeniyle rakip bankaya geçtiğini tespit eder. Çözüm olarak mobil uygulamanın yenilenmesi gerektiğini önerir ve bunun bankaya ne kadar kazandıracağını hesaplar.
- ◊ **Gereksinim Analizi Kısmı:** Analist, yeni uygulamada "hızlı transfer" butonunun nerede olacağını, transferin kaç saniyede tamamlanması gerektiğini, hata mesajlarının nasıl görüneceğini ve hangi güvenlik protokollerinin (örneğin yüz tanıma, parmak izi veya kimlik kartı gibi) kullanılacağını detaylandırır.
- ◊ **Özetle:** İş analizi rotayı çizer ve varış noktasını (hedefi) belirler; gereksinim analizi ise o hedefe ulaşmak için aracın nasıl çalışması gerektiğini tarif eden kullanım kılavuzunu yazar.

İş analizi sürecinde **gereksinim analizi**, paydaşlardan toplanan ham bilgilerin (ihtiyaçlar, istekler, kısıtlar) işlenerek; tutarlı, eksiksiz ve doğrulanmış birer "gereksinim" haline getirildiği kritik aşamadır. Süreci bir mutfağa benzetirsek; paydaşlardan gelen bilgiler "ham malzemeler", gereksinim analizi ise bu malzemelerin ayıklanıp, doğranıp pişirilerek sunuma hazır hale getirilmesidir.

| Özellik      | İş Analizi (Business Analysis)                                         | Gereksinim Analizi (Requirements Analysis)                                  |
|--------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Kapsam       | Çok geniştir; kurumun stratejisini ve tüm iş süreçlerini kapsar.       | Daha dardır; belirli bir çözüm veya proje özelindeki detaylara odaklanır.   |
| Odak Noktası | İş problemlerini tanımlamak, fırsatları belirlemek ve değer yaratmak.  | Toplanan bilgileri yapılandırmak, detaylandırmak ve teknik dile hazırlamak. |
| Zamanlama    | Proje başlamadan önce başlar, proje boyunca ve sonrasında devam eder.  | Genellikle çözümün kapsamı netleştikten sonra (planlama aşamasında) başlar. |
| Çıktı        | İş vakası ( <b>business case</b> ), Fizibilite raporu, Stratejik plan. | Gereksinim dokümanı (BRD/PRD), Kullanıcı hikayeleri, Akış diyagramları.     |

Tablo 19.1: İş Analizi-Gereksinim Analizi Karşılaştırması

### §19.1.2 Gereksinim Analizi Adımları

Bu süreç doğrusal değil, döngüsel bir yapıdadır. Temel adımlar şunlardır:

1. **Modelleme ve Düzenleme:** Toplanan bilgiler kategorize edilir. Bu aşamada görsel modeller kullanılarak karmaşık süreçler basitleştirilir.
  - ◊ Süreç Modelleri: İş akış diyagramları (**flowcharts**).
  - ◊ Veri Modelleri: Varlık-İlişki Diyagramları (ERD).
  - ◊ Kullanım Senaryoları: Use Case diyagramları.



2. **Gereksinimlerin Önceliklendirilmesi:** Tüm gereksinimler aynı anda yapılamaz. Kaynaklar ve zaman kısıtlıdır. Yaygın olarak **MoSCoW** tekniği kullanılır:
  - ◊ **Must have:** Olmazsa olmazlar.
  - ◊ **Should have:** Olsa iyi olur (Yüksek öncelik).
  - ◊ **Could have:** Olsa da olur (Düşük öncelik).
  - ◊ **Won't have:** Bu aşamada yapılmayacaklar.
3. **Doğrulama ve Onay (verification & validation):** Gereksinimlerin kalitesi kontrol edilir:
  - ◊ **Doğrulama:** Gereksinimler standartlara uygun mu? (Net, ölçülebilir, test edilebilir mi?)
  - ◊ **Geçerlilik:** Bu gereksinim gerçekten iş ihtiyacını karşılıyor mu?

|  | Tür                                | Açıklama ve Örnek                                                                     |
|--|------------------------------------|---------------------------------------------------------------------------------------|
|  | <b>İş Gereksinimleri</b>           | Kurumun üst düzey hedefleri: "Yıllık operasyonel maliyeti %10 azaltmak."              |
|  | <b>Paydaş Gereksinimleri</b>       | Belirli bir kullanıcı grubunun ihtiyacı: "Depo sorumlusu stokları anlık görebilmeli." |
|  | <b>Çözüm (Fonksiyonel)</b>         | Sistemin ne yapması gerektiği: "Sistem, stok 10'un altına düştüğünde uyarı vermeli."  |
|  | <b>Çözüm (Fonksiyonel Olmayan)</b> | Sistemin nasıl çalışması gerektiği: "Sistem 2 saniyenin altında yanıt vermelidir."    |

Tablo 19.2: Gereksinim Türleri

### §19.1.3 İyi Bir Gereksinimin Özellikleri

Analiz sonunda ortaya çıkan her bir gereksinim şu kriterleri karşılamalıdır:

- ◊ **Atomik:** Daha küçük parçalara bölünemez olmalı.
- ◊ **Net (unambiguous):** Herkes tarafından aynı şekilde anlaşılmalı.
- ◊ **Test Edilebilir:** Başarılı olup olmadığı ölçülebilmeli.
- ◊ **Tutarlı:** Diğer gereksinimlerle çelişmemeli.
- ◊ **İzlenebilir:** Hangi iş ihtiyacından doğduğu takip edilebilmeli.

Bunun için kullanılan teknikler

1. **Kullanıcı Hikayeleri (user stories):** "Bir [rol] olarak, [amaç] istiyorum, böylece [fayda] sağlayacağım" kalıbı.
2. **Kabul Kriterleri (acceptance criteria):** Bir özelliğin "tamam" sayılması için gereken şartlar.
3. **Prototipleme:** Gereksinimleri somutlaştırmak için ekran taslakları hazırlama.

Gereksinim analizi sadece "yazmak" değildir; paydaşlar arasındaki çatışmaları yönetmek ve "gerçekten neye ihtiyaç duyulduğunu" bulup çıkarmaktır.

İş analizi sonrasında çözüm bir yazılım geliştirme olacaksa buna **kurumsal yazılım (enterprise software)** adı verilir. Bu durumda **gereksinim analizi (requirement analysis)**, geliştirilecek yazılım için başvuru kitabıdır ve **yazılımın kabul ve test noktasında en önemli rolü oynar.**

### §19.1.4 Gereksinim Analizi ve Modelleme Yol Haritası

Bir **kurumsal yazılım** sisteminin inşası için kurumda iş analizi yapılır. İş analizinde aşağıdaki modeller sırasıyla çıkarılır;

- ◊ **İşlevsel model (functional model)** çıkarılır. Kullanıcılara ne (**what**) yapmak istedikleri sorulur ve **use-case** diyagramları oluşturulur.
- ◊ **Yapısal model (structural model)** çıkarılır. İşlemlerin kimler (**who**) tarafından yapılacağı belirlenir ve bu kapsamda sınıf diyagramları oluşturulur.
- ◊ **Davranış modeli (behavioral model)** oluşturulur. İşlemlerin kim tarafından nasıl (**how**) ve hangi sırada yapılacağı belirlenir ve silsile diyagramları oluşturulur.
- ◊ **Mimari modelleme (architectural model)** oluşturulur. Bu modelde nesnelerin nasıl ve nerede (**where**) bir araya geleceği üzerine durulur ve yayılma ve paket diyagramları oluşturulur. Bu modelde iki gereklilik detaylı incelenir;
  - Birincisi iş süreçlerini ya da kullanıcı isteklerini yerine getiren işlevsel gerekliliklerdir (**functional requirements**).
  - İkincisi ise yazılımla birlikte ortaya çıkan performans, güvenlik (**security**), günlük (**logging**) ve

benzeri gerekliliklerdir (**non-functional requirements**).

Ancak güncel çevik süreçlerde yukarıda sırasıyla yapılması gerektiği anlatılan modeller **çoğu zaman eş zamanlı ve artımlı (incremental)** ilerler.

### §19.1.5 Kurumsal Uygulama Geliştirmenin Temel Bileşenleri

Kurumların karmaşık iş süreçlerini dijital ortama taşıyan ve geniş ölçekli operasyonları yöneten sistemler kurumsal uygulamalar (**enterprise application**) olarak adlandırılır. Başarılı bir kurumsal yazılım projesi inşa edebilmek için üç temel unsura ihtiyaç duyulur:

- ◊ **Takım (team)**: Yazılımın fikir aşamasından yayına alınmasına kadar görev yapan; iş analisti, sistem mimarı, yazılım geliştirici, test uzmanı ve proje yöneticisi gibi farklı uzmanlıklara sahip yetkin bir gruptur.
- ◊ **Yazılım Geliştirme Yaşam Döngüsü SDLC (Software Development Life Cycle)**: Projenin yönetim stratejisini belirleyen metodolojilerdir. İhtiyaca göre doğrusal ilerleyen Şelale Modeli (**waterfall model**), yinelemeli bir yapı sunan RUP (**Rational Unified Process**) veya günümüzde en yaygın tercih edilen esnek **Çevik (agile)** modellerden biri seçilir. Bu modeller, karmaşıklığı yönetmek ve riskleri azaltmak için bir yol haritası sunar.
- ◊ **Teknoloji (technology)**: Kurumsal yazılımların ihtiyaç duyduğu güvenlik, veri tabanı yönetimi, ağ iletişimi ve işlem yönetimi gibi standart altyapı hizmetlerini hazır olarak sunan platformlardır. Bu teknolojiler (frameworks), geliştiriciye karmaşık altyapı detaylarıyla uğraşmak yerine doğrudan iş mantığına (**business logic**) odaklanma imkânı tanır. Günümüzde bu alanda en çok tercih edilen ekosistemler Jakarta EE (**Enterprise Edition**) ve .NET (**.NET Framework/Core**) platformlarıdır.

Yazılım geliştirme sırasında bu üç şeyi çok iyi seçmemiz halinde başarılı bir kurumsal yazılım ortaya çıkar. Burada özellikle teknoloji üzerinde durmak gerekir. Jakarta EE ve .NET gibi iki büyük teknolojinin programcıya sunduğu bileşenler arkasında birçok sunucu çalışır. Bunların bazıları aşağıda verilmiştir;

- ◊ **Kurumsal yazılım geliştirmede kullanılan modüler bileşenleri sunan sunucular**: Jakarta Enterprise Beans, COM (**Component Object Model**), COM+ gibi teknolojileri sağlayan .NET Bileşenleri
- ◊ **Kullanıcı, bilgisayar, yazıcı ve benzeri öğeleri içeren kurumsal dizin sunucuları**: JNDI (**Java Naming and Directory Interface**), Microsoft Active Directory
- ◊ **Kurumsal Uygulamada bileşenlerin birbirine ileti (message) gönderip almalarını sağlayan servisler**: Jakarta Messaging API, MSMQ (**Microsoft Message Queue**)
- ◊ **Web Sunucuları üzerinde bileşenlerin çalışmasını sağlayan servisler**: JSP (**Jakarta Server Pages**), ASP.NET Core (.NET platformu için Web Uygulama Çatısı)
- ◊ **Mevcut kurumsal uygulamaları birbirine bağlayan servisler**: Jakarta Connectors, JDBC (**Java Database Connectivity**), Java XML, Jakarta Transactions, ADO.NET (**Active Data Objects for .NET**), WCF (**Windows Communication Foundation**)

Kurumlarda JEE ile .NET teknolojileri arasında seçim yapmak için, kurumun sahip olduğu donanım altyapısı ve yukarıda belirtilen servisleri sağlayan yazılım kaynakları irdelenmelidir.

Tüm projelerde olduğu gibi kurumlarda geliştirilen uygulamalar da proje olarak ele alınır ve yönetilirler. Proje yönetimi, belirli bir hedefe ulaşmak için zaman, bütçe, kaynaklar ve kalite kısıtları altında yürütülen işleri planlama, yürütme, izleme ve tamamlama sürecidir. Kısaca doğru işi, doğru şekilde, doğru zamanda ve doğru kaynaklarla yapmak demektir. Proje yönetimi genellikle beş ana aşamada yürütülür:

1. **Başlatma (initiation)**: Projenin amacı, kapsamı ve paydaşları belirlenir. Proje yöneticisi atanır ve gerekirse fizibilite çalışması yapılır. Proje yöneticisi, proje planı hazırlamak ve güncel tutar, takımı yönetir, iletişimi sağlar, riskleri öngörür ve önlem alır, paydaşlarla iletişimi sürdürür ve teslimatları zamanında gerçekleştirir.
2. **Planlama (planning)**: Proje hedefleri, teslimatlar belirlenir. **Zaman çizelgesi (gantt chart)** ve alt kırılımlar oluşturulur. Kaynak planlaması, risk yönetimi, bütçe, iletişim planı hazırlanır.
3. **Yürütme (execution)**: Takım kurulur, görevler dağıtılır. İşler gerçekleştirilir, yazılım geliştirme, inşa gibi ana faaliyetler yapılır. İletişim ve koordinasyon sağlanır.
4. **İzleme ve Kontrol (monitoring and controlling)**: Proje ilerlemesi takip edilir. Gecikmeler, sapmalar analiz edilir. Riskler yönetilir, gerekirse planlar revize edilir.
5. **Kapanış (closing)**: Proje sonuçlandırılır. Teslimatlar yapılır. Geri bildirim alınır, raporlama yapılır, dersler çıkarılır.

Proje yönetimini yapmak için **Microsoft Project, Jira, Trello, Asana, ClickUp, Notion, Monday.com, Smartsheet ve Wrike** gibi araçlar kullanılır.

Bir yazılım projesini yönetmek için yukarıdaki yönetim aşamalarını içerecek şekilde yazılım geliştirme yaşam döngüsü SDLC (**Software Development Life Cycle**) ile yapılır. Yazılım geliştirme modeli olarak da

adlandırılan SDLC, kaliteli yazılım tasarlamak, geliştirmek ve test etmek için kullanılan yapılandırılmış bir süreçtir. SDLC, bir yazılım mühendisi veya geliştiricisi tarafından çeşitli aşamalarda gerçekleştirilecek görevleri belirtir. Son ürünün müşterinin beklentilerini karşılayabilmesini ve tanımlanan bütçeye uymasını sağlar<sup>1</sup>.

Burada **yazılım mühendisliği (software engineering)** devreye girer. Yazılım mühendisliği, yazılım sistemlerinin planlanması, tasarımı, geliştirilmesi, test edilmesi, dağıtımı ve bakımı ile ilgilenir. Yazılım mühendisi;

- ◊ Yazılım geliştirme veya kısaca programlama yapar,
- ◊ Yazılım mimarisi belirler,
- ◊ Yazılım testini yapar ve kalitesini belirler,
- ◊ Yazılım proje yönetimini yapar,
- ◊ Yazılım yaşam döngüsü sürecini (SDLC) yürütür,
- ◊ Geliştirme ve işletim **DevOps (Development and Operations)** süreci ile sürekli entegrasyon/sürekli dağıtım **CI/CD (Continuous Integration/Continuous Delivery)** süreçlerini yürütür.

## 19.2 Yazılım Geliştirme Modelleri

Günümüzde birçok yazılım geliştirme modeli yani SDLC bulunmaktadır. Bu kitapta bunlardan ilki olan şelale modeli ile günümüzde çok kullanılan çevik modele değinilecektir.

### §19.2.1 Şelale Modeli

**Şelale modeli (waterfall model)** 1970 yılında *Winston W. Royce* tarafından tanımlanmıştır. Bu modelde bir sonraki aşamaya geçmek için, bir önceki aşamanın tamamlanması gerekir. Bir sonraki aşamaya geçmeden, içinde bulunulan aşamanın tüm dokümanları hazırlanır. Bu model, **klasik yaşam döngüsü (classic life cycle)**, doğru sıralı (**linear sequential**) model olarak da adlandırılır. Şelale modelinin aşamaları;

1. **Gereksinim Analizi (requirement analysis)**: Müşteri ihtiyaçları toplanır ve belgelenir. Projenin kapsamı belirlenir, Projenin hedef ve amaçları belirlenir. Projenin kaynakları planlanır. Fonksiyonel ve teknik gereksinimler belirlenir. Gereksinimler gözden geçirilir ve kabul koşulları belirlenir.
2. **Sistem ve Yazılım Tasarımı (design)**: Yazılım mimarisi ve sistem yapısı planlanır. Veri tabanı, ara yüz, iş mantığı gibi bileşenler tasarlanır. Düşük ve yüksek düzey tasarım yapılır. Düşük Seviye Tasarım LLD (**Low Level Design**), yazılım geliştirme sürecinde detaylı sistem bileşenlerinin ve bunların etkileşimlerinin belirtildiği ve bunların uygulandığı bir aşamadır. Yüksek Seviye Tasarım HLD (**High Level Design**), bir sistemin genel yapısının planlandığı uygulamaların geliştirilmesinde ilk adım olup, esas olarak sistemin farklı bileşenlerinin, kodlama ve uygulama hakkında bilgi sahibi olmadan birlikte nasıl çalıştığına odaklanır. Bu, projede yer alan herkesin hedefleri anlamasına yardımcı olur ve geliştirme sırasında iyi iletişim sağlar.
3. **Uygulama ya da Kodlama (implementation)**: Tasarlanan yapıya uygun olarak yazılım geliştirme yapılır. Kod standardı belirlenir yani programlama dilleri burada belirlenir. Kod ölçeklendirme yapılır. Kod ölçeklendirme, uygulamanın daha fazla kullanıcı, daha fazla veri veya daha yoğun trafik ile etkili şekilde çalışabilmesi için yapılan düzenlemelerdir. Sürüm kontrolü belirlenir. Kodun gözden geçirilmesi için süreç belirlenir.
4. **Test Etme (testing)**: Kodlanan yazılım test edilir. Genel sistem testi yanında manuel test ve otomatik testler yapılabilir. Varsa hatalar düzeltilir, kalite kontrolü sağlanır.
5. **Dağıtım (deployment)**: Yazılımın uygun sürümü son kullanıcıya sunulur. Kurulumlar ve eğitimler yapılabilir.
6. **Bakım (maintenance)**: Kullanım sırasında oluşan hatalar düzeltilir. Geri bildirimler sonrası güncellemeler ve iyileştirmeler yapılır.

Bu modelde yazılıma ilişkin dokümantasyon çok net ve kapsamlıdır. Bir aşama tamamlanmadan sıradakine geçilmez. Süreç kontrollüdür. Gereksinimi değişmeyen büyük projelerde kullanılır. Yönetimi ve izlenebilirliği kolaydır. Ancak geri dönmek zordur. Bir hata sonradan fark edilirse süreç baştan sarar. Değişen gereksinimler varsa uygun değildir. Bu nedenle esnek değildir. Teslimat çok geç ortaya çıkar yani müşteri ürünü en sonunda görür.

### §19.2.2 Çevik Model

**Çevik model (agile model)**, şelale modelinin aksine temel olarak proje süresince gelen değişen isteklere hızla uyum sağlamak için tasarlanmıştır. Günümüzde çok tercih edilen modellerden biridir. Bu model, yazılım geliştirme süreçlerinde esnekliği, müşteri memnuniyetini ve hızlı teslimatı ön planda tutan tekrarlamalı (**iterative**) bir yaklaşımdır. Büyük bir projeyi küçük parçalara ayırarak, adım adım geliştirme ve sürekli geri bildirimle iyileştirme sağlar. Bu model, değişen müşteri ihtiyaçlarına hızlıca adapte olabilmek için tasarlanmış, sürekli iletişim, erken teslimat ve kademeli gelişim ilkelerine dayanan bir geliştirme modelidir.

<sup>1</sup><https://www.geeksforgeeks.org/software-development-life-cycle-sdlc/>

Çevik modelde projeyi yönetmek için proje personelinin bazı değerleri (**values**) benimsemesi gerekir;

1. Bireyler ve etkileşimler, süreçler ve araçlardan daha önemlidir.
2. Çalışan yazılım, kapsamlı dokümantasyondan daha önemlidir. Burada, dokümantasyonun "yok sayılması" değil, "kodun kendisinin bir doküman gibi (**self-documenting code**) yazılması çok önemlidir.
3. Müşteri ile iş birliği, sözleşme pazarlığından daha önemlidir.
4. Değişime karşılık vermek, bir planı izlemekten daha önemlidir.

Bu değerler ile **12 adet ilkeye uyulur**. Bunlar **çevik manifestoda** (**agile manifesto**) yer alır;

1. Müşteri memnuniyeti, erken ve sürekli yazılım teslimatıyla sağlanır. Müşteriye sık aralıklarla çalışan yazılım sunarak geri bildirim alınır ve güven kazanılır.
2. Değişen gereksinimler, projenin geç aşamalarında bile kabul edilir. Çevik süreçler, değişime yanıt vererek müşterinin rekabet avantajını korumasına yardımcı olur.
3. Çalışan yazılım, birkaç haftada bir, mümkün olan en kısa sürede teslim edilir. **Kısa döngülerle** (**sprint**) geliştirme yapılır.
4. İş birliği, iş birimlerinden (müşteriden) ve geliştiricilerden oluşan bir ekip arasında sürekli olmalıdır. Müşteri ve geliştirici ekip sürekli iletişim halinde olur.
5. Motivasyonu yüksek bireyler etrafında projeler inşa edilir. Ekip üyelerine güvenilir, destekleyici bir ortam sunulur ve onlara güvenilir.
6. Yüz yüze iletişim, bilgi aktarımının en etkili yoludur. Özellikle aynı fiziksel ortamda çalışan ekiplerde hızlı karar alınmasını sağlar.
7. Çalışan yazılım, ilerlemenin birincil ölçüsüdür. Raporlar ya da belgeler değil, gerçek çalışan yazılım önemlidir.
8. Çevik süreçler sürdürülebilir geliştirmeyi teşvik eder. Ekip, sabit bir hızda uzun süre çalışabilmelidir. Aşırı mesai değil, dengeli tempo hedeflenir.
9. Teknik mükemmellik ve iyi tasarım, çevikliği artırır. Kaliteli kod ve temiz mimari, değişimlere uyumu kolaylaştırır.
10. Basitlik esastır. Yapılmaması gereken işleri yapmaktan kaçınmak. Gereksiz özellik ve aşırı mühendislikten uzak durulur.
11. En iyi mimariler, gereksinimler ve tasarımlar, kendi kendini organize eden ekiplerden çıkar. Ekip içindeki bireyler, en uygun çözümleri birlikte bulur.
12. Ekip düzenli aralıklarla nasıl daha verimli olabileceğini düşünür ve davranışlarını buna göre uyarlar. **Sürekli iyileştirme** (**retrospective**) ile gelişim sağlanır.

Çevik proje yönetiminde ön tanımlı roller vardır;

1. Ürün Sahibi (**Product Owner**): Müşteri ihtiyaçlarını belirler. Ürünün ne olacağına karar verir, iş önceliklerini belirler.
2. Şef (**Scrum Master**): Takımın çevik kurallarına uymasını sağlar. Engelleri kaldırır, kolaylaştırıcı rolündedir.
3. Geliştirme Takımı (**Development Team**): Yazılımı analiz eder tasarlar, kodlar, test eder.

Çevik modelde proje, **sprint** adı verilen kısa geliştirme döngülerine ayrılır. Her sprint sonunda çalışan bir yazılım parçası müşteriye teslim edilir. Bunun için;

- ◇ İş listesi (**Product Backlog**) hazırlanır.
- ◇ Her bir kısa döngü için toplantı (**Sprint Planning**) yapılır.
- ◇ Yazılım takımı, 1 ila 4 haftalık kısa döngü süresince belirlenen işleri yapar.
- ◇ Günlük toplantılar (**Daily Scrum**) yani günlük toplantılar ile ilerleme izlenir.
- ◇ Kısa döngü gözden geçirme toplantısı (**Sprint Review**) ile müşteriye sunum yapılır.
- ◇ Sürekli iyileştirme (**Sprint Retrospective**) ile ekip kendi sürecini değerlendirir ve iyileştirir.

Bir kurumda çevik proje yönetimi yapabilmek için kurumun buna hazırlanması gerekir;

- ◇ **Kurum Kültürü**: Çevik proje yönetimi, hiyerarşik ve katı yönetim anlayışı olan kurumlarda zor olur. Bu yüzden üst yönetimin çevik felsefesini benimsemesi gerekir. Takımlara güven, şeffaflık ve sürekli iyileştirme desteklenmelidir. "Komuta-kontrol" yerine "hizmetkar liderlik" yaklaşımı benimsenmelidir.
- ◇ **Kurum İçi Eğitim**: Tüm ilgili ekipler (yazılım geliştiriciler, testçiler, analistler, ürün sahipleri, yöneticiler) çevik eğitimi almalıdır. **Scrum**, **Kanban**, **Lean** gibi çerçeveler tanıtılmalı ve hangi modelin kuruma uygun olduğuna karar verilmelidir.
- ◇ **Roller**: **Scrum** kullanılıyorsa roller atanmalıdır; **Product Owner**, **Scrum Master**, **Development Team**.



- ◇ **Takım:** Küçük, çapraz fonksiyonel ve çok disiplinli (**multi-disipliner**) takımlar oluşturulmalıdır. Her takım kendi içinde analiz, tasarım, geliştirme ve test yapabilir durumda olmalıdır.
- ◇ **Sprint Yönetimi:** Scrum uygulanıyorsa;
  - **Kısa Döngü Planlama** (Sprint Planning): İşlerin belirlenmesi ve sprint hedefinin tanımlanması.
  - **Günlük Toplantı** (Daily Stand-up): Kısa günlük toplantılarla durum değerlendirmesi.
  - **Kısa Döngü Gözden Geçirmesi** (Sprint Review): Ürün sahipleri ve paydaşlarla yapılan toplantı (**demo**).
  - **Sürekli İyileştirme Toplantısı** (Sprint Retrospective): Ekip içi iyileştirme toplantısı.
- ◇ **Agile süreçleri destekleyen yazılımlar:** **Jira, Trello, Azure DevOps, ClickUp** ve **Asana** gibi araçlarla iş takibi yapılabilir. Ayrıca CI/CD altyapıları kurulmalıdır.
- ◇ **Pilot Proje Deneyimi:** Öncelikle küçük bir projede çevik yöntem uygulanır. Deneyim kazanılır, süreçler test edilir. Başarılar ve hatalar analiz edilerek diğer ekiplerde yaygınlaştırılır.
- ◇ **Sürekli İyileştirme (Kaizen):** Ekipler düzenli olarak süreçlerini gözden geçirip iyileştirme önerileri üretmelidir. Geri bildirim döngüleri (müşteriden ve ekip içinden) önemlidir.
- ◇ **Başarı ve Performans Ölçütleri:** Çevik ekiplerin performansı birey bazında değil, takım bazında ölçülür. Teslim süreleri, müşteri memnuniyeti, iş değeri odaklı metrikler kullanılır.

- ◇ Çevik modelde **müşteri memnuniyeti yüksektir.**
- ◇ Hızlı teslimat ve **erken geri bildirim** alınır.
- ◇ **Değişime açık** ve uyumlu yazılım geliştirilir.
- ◇ Takım içi **iletişim güçlüdür.**
- ◇ Kalite artar, çünkü sürekli test edilir, ancak **sürekli müşteri katılımı gerekir.**
- ◇ **Belirsiz projelerde zaman ve maliyet tahmini zordur.**
- ◇ **Deneyimsiz ekiplerde karmaşıklık ortaya çıkar!**
- ◇ Dokümantasyon önemsiz gibi görülüp ihmal edilebilir. Bu proje **sonrası bakım maliyetini ciddi oranda artırır!**

## 19.3 Modern Kurumsal Yazılımların Özellikleri

Günümüzde geliştirilen modern uygulamaların üç temel özelliği vardır:

1. **Katmanlı mimari (layered architecture):** İstemci/sunucu ya da n-katmanlı modeldir. Katmanlı yapı, sorumlulukları birbirinden ayırarak bağımlılığı azaltır; ancak katman sayısı arttıkça sistemin yapısal **karmaşıklığı (complexity)** yönetilebilirlik açısından dikkatle planlanmalıdır. Sanılanın aksine, katman sayısı arttıkça sistemin toplam karmaşıklığı teknik olarak artar; ancak bu yapı, bileşenler arasındaki bağımlılığı (**coupling**) azaltarak yönetilebilirliği kolaylaştırır. Her katmanın belirli bir görevi olması, bakım ve hata ayıklama süreçlerini hızlandırır.
2. **Bileşen kullanımı (component using):** Yazılımın modüler parçalardan oluşması, kodun farklı projelerde **tekrar kullanılabilirliğini (reusing)** artırır ve bakım maliyetlerini düşürür.
3. **Dağıtık programlama (distributed programming):** Bu yaklaşım sayesinde sistemin **ölçeklenebilirliği (scalability)** artar. Yazılım, iş yükünü birden fazla bilgisayar veya sunucu üzerine çeşitli görevlerle dağıtarak yüksek erişilebilirlik sağlar. Bu yaklaşım, sistemin ölçeklenebilirliğini (**scalability**) ve hata toleransını (**fault tolerance**) maksimize eder. Günümüzde bu özellik kendini en net **mikroservis mimarileri** ve **bulut bilişim (cloud computing)** teknolojilerinde gösterir. Bir sunucuda sorun yaşandığında, yük otomatik olarak diğerlerine dağıtılarak sistemin kesintisiz çalışması sağlanır.

Dağıtık programlamanın günümüzdeki en popüler ve olgun yansıması olan **mikroservis mimarisi (microservices architecture)**, büyük ve yekpare (**monolithic**) bir uygulamanın, her biri kendi başına çalışan, belirli bir iş mantığını yürüten ve birbirleriyle hafif protokoller (genellikle HTTP/REST veya mesaj kuyrukları) üzerinden haberleşen küçük servisler bütününe bölünmesidir.

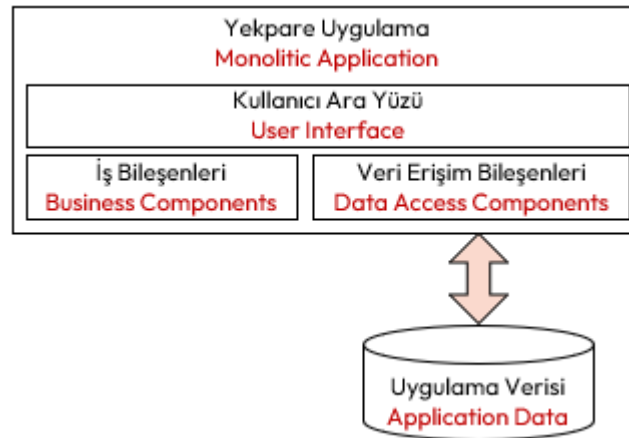
## 19.4 Katmanlı Mimari

Katman kavramı iki türlü düşünülmelidir. Kodların gruplanarak birbirinden ayrıldığı katmanlara kod katmanı (**layer**) adı verilir. Fiziksel olarak birbirinden ayrılmış kodlara ise fiziksel katman (**tier**) adı verilir. Kısaca derlenmiş olarak birbirinden ayrı olan katmanlar fiziksel katman, diğer tip katman ise kod katmanı olarak adlandırılır. Katman sayısı göz önüne alınarak uygulamalar aşağıdaki şekilde sınıflandırılabilir.

### §19.4.1 Yekpare Uygulamalar

Genel olarak hiç katmanı olmayan, tek bir çalıştırılabilir dosyadan oluşan uygulamalar **yekpare (monolithic)** uygulamalar olarak adlandırılır. Aynı bir sınıf kütüphanesi olmayan tüm konsol ve Windows uygulamaları bu sınıfa girer. Şu ana kadar verilen örneklerin çoğu bu kapsamda değerlendirilebilir. Yekpare uygu-

lama, kullanıcı arayüzü, iş mantığı ve veri erişim katmanı gibi tüm bileşenlerin tek bir kod tabanında ve bölünmez bir birim olarak geliştirilip dağıtıldığı geleneksel yazılım mimarisidir. Basit, hızlı geliştirilen ve yönetimi kolay yapılar olsalar da, ölçeklendirme zorluğu ve kod karmaşıklığı gibi dezavantajları bulunur (Şekil 19.1).



Şekil 19.1: Yekpare Uygulama

Yekpare uygulama örneğimizin yapısı aşağıda verilmiştir;

```
KonsolUygulaması/ → Çözüm (Solution)
├─ Yekbare/ → Proje (Console Application Project)
├─ Program.cs → Ana (Main) Program
```

Projeyi oluşturmak için; **Visual Studio** üzerinde **File→New→Project...** seçilerek **KonsolUygulaması** adlı çözüm içinde **Yekpare** adlı projemiz **Console Application** olarak oluşturulur. Oluşan projede **Program.cs** dosyası otomatik olarak oluşur. Bu dosyayı aşağıdaki şekilde değiştiriyoruz;

```
namespace Yekpare
{
 class İşMantığı
 {
 public void İşİşlevi()
 {
 Console.WriteLine("İş İşlevi Çağrıldı.");
 }
 }
 class Client
 {
 static void Main(string[] args)
 {
 İşMantığı işlev = new İşMantığı();
 işlev.İşİşlevi();
 }
 }
}
```

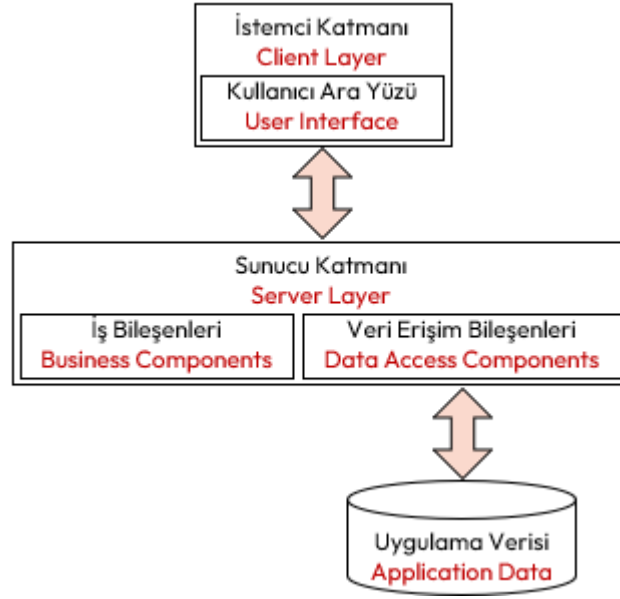
Yekpare uygulamalar, her ne kadar veri tabanı kullansalar da bu tür yazılımlar **tek bir proje olarak kodlanırlar ve derlenir**. Yani **proje tek bir kod tabanından oluşur**. Seksenli yıllara kadar geliştirilen uygulamalar genellikle bu yapıdadır.

Bilgisayara yaptırılan işlerin kapsamının zamanla daha fazla genişlemesi, kurumların iş süreçlerini daha çok içermesi ve uygulamanın işletme ve bakım maliyetlerinin artmasıyla birlikte, bu yazılımları uygun katmanlara bölünmesi ihtiyacı doğurmuştur.

## Ş19.4.2 İki Katmanlı Mimari

İki Katmanlı yani **İstemci-Sunucu (client-server)** mimarisinde uygulama iki katmanlı olup, genellikle veri katmanı bir bilgisayarda diğer katman olan istemci katmanı ise kullanıcı bilgisayarlarında bulunur. Bu tür uygulamalarda kullanıcı ara yüzü bileşenleri **UIC (User Interface Components)** istemci (**client**) katmanını oluşturur. Geri kalan iş süreçleriyle ilgili geliştirilen bileşenler ve veri tabanı ile ilgili iletişimi sağlayacak olan bileşenler ise sunucu (**server**) katmanını oluşturur (Şekil 19.2).

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;



Şekil 19.2: İstemci-Sunucu Mimarisi

```

İstemciSunucuUygulaması/ → Çözüm (Solution)
├── İstemciBileşenleri/ → Proje (Console Application Project)
│ └── Program.cs → Ana (Main) Program
├── SunucuBileşenleri/ → Proje (Class Library)
│ └── Sunucu.cs → Sınıflar (Class)

```

Projeyi oluşturmak için;

1. Visual Studio üzerinde **File→New→Project...** seçilerek **İstemciSunucuUygulaması** adlı çözüm içinde **İstemciBileşenleri** adlı projemiz **Console Application** olarak oluşturulur.
2. Oluşan projede Program.cs dosyası otomatik olarak oluşur.

```
namespace İstemciBileşenleri
```

```
{
 class Client
 {
 static void Main(string[] args)
 {
 }
 }
}
```

3. Daha sonra sunucu katmanını yapmak için **Solution Explorer** üzerinde **KonsolUygulaması** çözümüne sağ tıklayarak **Add→New→Project...** seçerek **SunucuBileşenleri** adlı **Class Library** oluşturulur
4. Proje ile oluşan **Class1.cs** dosyasının adı **Sunucu.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace SunucuBileşenleri
{
 public class Server
 {
 public void Sunucuİşlevi()
 {
 Console.WriteLine("Sunucu İşlevi Çağrıldı.");
 }
 }
}
```

5. İstemci tarafında sunucu bileşenlerini kullanmak için **Solution Explorer** üzerinde **SunucuBileşenleri** projesi sağ tıklayarak **Build** seçilir ve derlenir.
  6. Ardından **İstemciBileşenleri** projesi seçilir ve **Add→Project Reference→Project...** ile sunucu projemiz istemci projemize dahil edilir.
  7. Bunu yaptıktan sonra **İstemciBileşenleri** projesinde aşağıdaki değişiklikler yapılır;
- ```
using Sunucu; //Bunu yapabilmek için:İstemciye Sunucu Referans olarak eklenir.
namespace İstemciBileşenleri
```



```

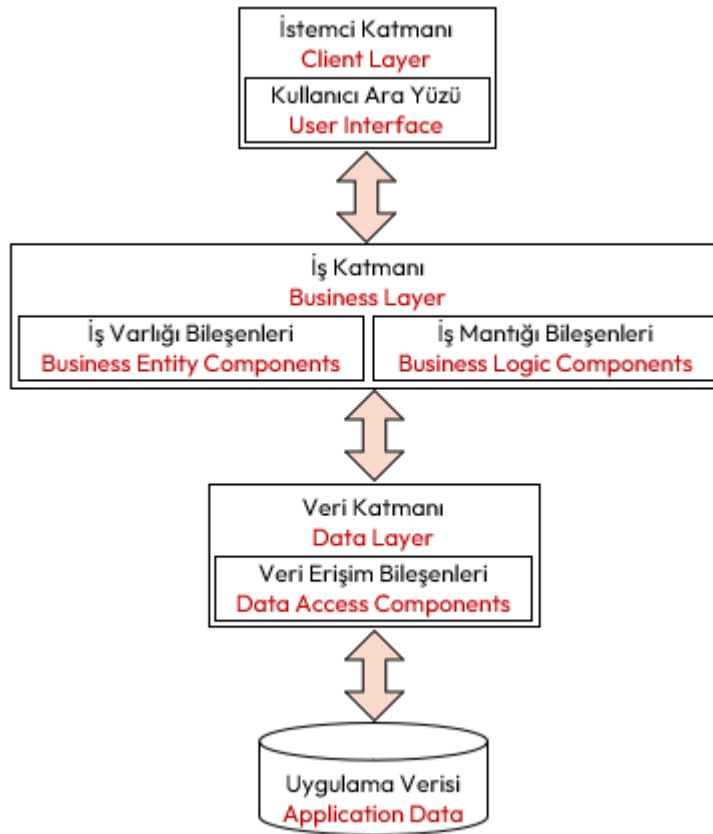
{
    class Client {
        static void Main(string[] args)
        {
            Sunucu.Server sunucu = new Sunucu.Server();
            sunucu.Sunucuİşlevi();
        }
    }
}

```

8. Artık **Solution Explorer** üzerinde **İstemciBileşenleri** projesi sağ tıklanarak **Set as Default Project** seçilir ve program çalıştırılabilir.

§19.4.3 Üç Katmanlı Mimari

Nesne yönelimli yazılım geliştirme ile bildikte nesneler arasındaki iletişimin bir başka katman olarak tasarlandığı uygulamalardır. İki katmanlı mimariden farklı olarak kullanıcı ara yüzü ile iş bileşenleri birbirinden ayrılarak, kullanıcılara istemci yükleme işleminin çok kolaylaştığı uygulamalardır. İstemci-sunucu yazılımlarında zamanla, sunucu tarafında iş süreçlerini modelleyen bileşenler BEC (**Business Entity Components**) ile veri tabanıyla iletişimi sağlayan bileşenler DAC (**Data Access Components**) birbirinden ayrılarak farklı iki katman olarak geliştirilmiştir. Böylece katman sayısı artarak üç katmanlı (**three layer**) mimari hayata geçmiştir (Şekil 19.3).



Şekil 19.3: Üç Katmanlı Mimari

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

```

ÜçKatmanlıUygulama/ → Çözüm (Solution)
├── İstemciKatmanı/ → Proje (Console Application Project)
│   ├── Program.cs → Ana (Main) Program
├── İşKatmanı/ → Proje (Class Library)
│   ├── İşBileşenleri.cs → Sınıflar (Class)
├── VeriKatmanı/ → Proje (Class Library)
│   ├── VeriBileşenleri.cs → Sınıflar (Class)

```

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir; Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File**→**New**→**Project...** seçilerek **ÜçKatmanlıUygulama** adlı çözüm içinde **İstemciKatmanı** adlı projemiz **Console Application** olarak oluşturulur.

2. Oluşan projede `Program.cs` dosyası otomatik olarak oluşur.

```
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
        }
    }
}
```

3. Daha sonra sunucu katmanı yapmak için **Solution Explorer** üzerinde **ÜçKatmanlıUygulama** çözümüne sağ tıklanarak **Add→New→Project...** seçerek **İşBileşenleri** adlı **Class Library** oluşturulur
4. Proje ile oluşan `Class1.cs` dosyasının adı **İşBileşenleri.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace İşBileşenleri
{
    public class İşSınıfı
    {
        public void işYöntemi()
        {
            // İş mantığı burada yer alır
        }
    }
}
```

5. Daha sonra sunucu katmanı yapmak için **Solution Explorer** üzerinde **ÜçKatmanlıUygulama** çözümüne sağ tıklanarak **Add→New→Project...** seçerek **VeriKatmanı** adlı **Class Library** oluşturulur
6. Proje ile oluşan `Class1.cs` dosyasının adı **VeriBileşenleri.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace VeriBileşenleri
{
    public class VeriTabanı
    {
        public void veriEkle(int veri)
        {
            // Veri ekleme işlemleri burada yer alır
            Console.WriteLine($"Veri tabanına veri ekleme {veri} işlemi çalıştı.");
        }
        public void veriSil(int veri)
        {
            // Veri silme işlemleri burada yer alır
            Console.WriteLine($"Veri tabanından veri silme {veri} işlemi çalıştı.");
        }
    }
}
```

7. İstemci tarafında sunucu bileşenlerini kullanmak için **Solution Explorer** üzerinde **VeriKatmanı** projesi sağ tıklanarak **Build** seçilir ve derlenir.
8. Ardından **İşKatmanı** projesi seçilir ve **Add→Project Reference→Project...** ile veri katmanı projemiz istemci projemize dahil edilir.
9. Bunu yaptıktan sonra **İşKatmanı** projesinde aşağıdaki değişiklikler yapılır;

```
using VeriBileşenleri; // İş projesine Veri Projesi Referans olarak eklenir.
namespace İşBileşenleri
{
    public class İşSınıfı
    {
        public void işYöntemi()
        {
            // İş mantığı burada yer alır
            int işlenenVeri = 0;
            VeriTabanı veriTabanı = new VeriTabanı(); //
            veriTabanı.veriEkle(işlenenVeri); //
            //...
```

```

        veriTabanı.veriSil(işlenenVeri); //
        Console.WriteLine("İş sınıfı iş yöntemi çalıştı.");
    }

}

```

- İstemci tarafında iş bileşenlerini kullanmak için **Solution Explorer** üzerinde **İşBileşenleri** projesi sağ tıklanarak **Build** seçilir ve derlenir.
- Ardından **İstemciKatmanı** projesi seçilir ve **Add**→**Project Reference**→**Project...** ile iş katmanı projemiz istemci projemize dahil edilir.
- Bunu yaptıktan sonra **İstemciKatmanı** projesinde aşağıdaki değişiklikler yapılır;


```

using İşBileşenleri; // İstemci projesine İş Projesi Referans olarak eklenir.
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
            İşSınıfı işnesnesi = new İşSınıfı(); //
            işnesnesi.işYöntemi(); //
        }
    }
}

```
- Artık **Solution Explorer** üzerinde **İstemciKatmanı** projesi sağ tıklanarak **Set as Default Project** seçilir ve program çalıştırılabilir.

§19.4.4 Çok Katmanlı Mimari

Yukarıda örneği verilen üç katmanlı uygulamalarda ihtiyaca göre katmanlar daha alt katmanlara ayrılabilir. Örneğin kullanıcı ara yüzü katmanı, aşağıdaki bileşenlerden oluşacak şekilde ikiye ayrılabilir;

- İnsanların yazılıma talimatlarını verdiği insan talimatları bileşenleri HIC (**Human Instruction Components**) ve
- Kullanıcı talimatlarını, kullanıcı ara yüzünden alıp, iş katmanının anlayacağı şekle çeviren kullanıcı süreci bileşenleri UPC (**User Process Components**).

Benzer şekilde İş katmanı, aşağıdaki bileşenlerden oluşacak şekilde üçe ayrılabilir;

- İş akışını sağlayan bileşenler BWC (**Business Workflow Components**),
- İş varlıkları bileşenleri BEC (**Business Entity Components**) ve
- İş varlıklarının toplu olarak işlenmesi ile ilgili iş mantığı bileşenleri BLC (**Business Logic Components**)

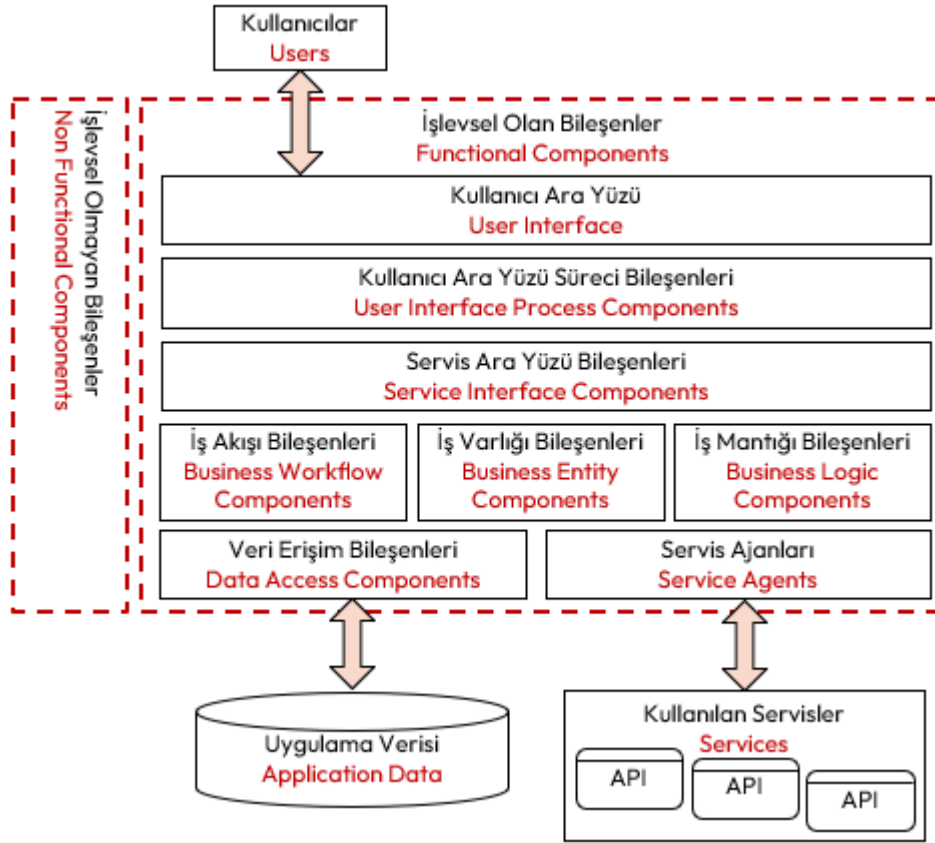
Çok katmanlı mimari, günümüz yazılım mimarisi olup mümkün olduğunca katmanların birbirinden bağımsız tasarlandığı uygulamalardır. Üç katmanlı mimariden daha alt katmanlara ayrılarak geliştirilen yazılımlara, n-katmanlı (**n-layer/n-tier**) yazılım adı verilir. Çok katmanlı yazılım geliştirilirken kullanılan bileşenler ve bu bileşenlerin bulunduğu katmanlar detaylı olarak aşağıdaki şekilde verilmiştir (Şekil 19.4).

Yukarıda verilen şekilde, dikine yazılı olan yerler, yazılımı gerçekleştirilen sistemde bulunmayan, ancak yazılım geliştirilirken ihtiyaç duyulan bileşenleri ve bununla ilgili servisleri temsil etmektedir. Bu nedenle bu bileşenler, yazılımın **niteliksel (işlevsel olmayan) (non-functional)** bölgeleridir. Yani ortada bir yazılım olmayınca hiçbir önemi olmayan bileşenlerdir. Geri kalan bileşenler ise işlevsel (**functional**) bileşenler olarak adlandırılır (Şekil 19.5).

Burada bahsedilen güvenlik (**security**) katmanlara rol bazlı (**role based**) veya kod erişimi (**code access**) olmak üzere iki şekilde ilave edilir.

Sonuç olarak yazılımlarda katman sayısı arttıkça performans (**performance**) düşer ama esneklik (**flexibility**) artar, bakım (**maintainability**) kolaylaşır, yeniden kullanılabilirlik (**reusability**) ve ölçeklenebilirlik (**scalability**) sağlanmış olur.

Çok katmanlı mimari, iş yükünü birden fazla bilgisayar veya sunucu üzerine çeşitli görevlerle dağıtarak yüksek erişilebilirlik sağlar. Bu yaklaşım, sistemin **ölçeklenebilirliğini (scalability)** ve **hata toleransını (fault tolerance)** maksimize eder. Günümüzde bu özellik kendini en net **mikroservis mimarileri** ve **bulut bilişim (cloud computing)** teknolojilerinde gösterir.



Şekil 19.4: Çok Katmanlı Mimari

§19.4.5 Katmanların Tasarlanması

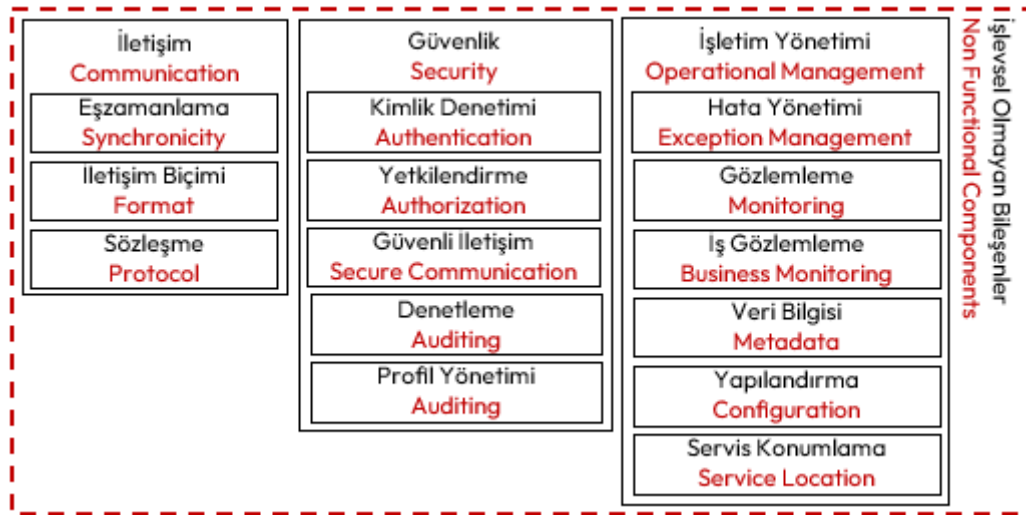
Gereksinim analizinden sonra katmanlar ayrı ayrı tasarlanır;

Business Logic Components (BLC)

Bu katman, sorun alanı bileşenleri PDC (**Problem Domain Components**) olarak da adlandırılır. Uygulamanın, iş modelinde belirtilen, iş kurallarını içeren bileşenlerdir. PDC, gerçek dünyadaki problemin yazılımda modellenmiş haliyle ilgilidir. Sistem çözümünün temel mantığını ve iş kurallarını (**business rules**) kapsar. Gerçek hayat problemini, teknik detaylardan bağımsız şekilde doğru ve tam olarak modellemek amacıyla tanımlanırlar.

```
public class Sipariş //Örnek PDC/BLC Bileşenleri
{
    public Guid Id { get; private set; }
    public List<SiparişKalemi> SiparişKalemleri { get; private set; }
    public DateTime SiparişZamanı { get; private set; }
    public Sipariş()
    {
        Id = Guid.NewGuid();
        SiparişKalemleri = new List<SiparişKalemi>();
        SiparişZamanı = DateTime.UtcNow;
    }
    public void YeniKalemEkle(SiparişKalemi item)
    {
        if (item.Miktar <= 0)
            throw new ArgumentException("Miktar 0 dan fazla olmalıdır!");

        SiparişKalemleri.Add(item);
    }
    public decimal ToplamSiparişÜcretiGetir()
    {
        return SiparişKalemleri.Sum(i => i.BirimFiyat * i.Miktar);
    }
}
```



Şekil 19.5: Niteliksel (İşlevsel Olmayan) Bileşenler

```

}
public class SiparişKalemi
{
    public Guid KalemId { get; set; }
    public int Miktar { get; set; }
    public decimal BirimFiyat { get; set; }
}

```

Business Entity Components (BEC)

Bu katman, veri tabanı tabloları ile eşleşen ve iş nesnelerini temsil eden sınıflar içerir. Sınıflar yalnızca özellik barındırır ve veri taşıma, saklama veya iletme süreçlerinde kullanılır, içinde iş mantığı barındırmaz. Bu sınıflar çok nadir olarak doğrulama (**validation**) yöntemleri içerir. Bu sınıflardan imal edilen nesneler veri transfer nesneleri DTO (**Data Transfer Object**) olarak kullanılır.

```

//Örnek BEC Bileşenleri
public class SiparişDto
{
    public Guid Id { get; set; }
    public List<SiparişKalemiDto> SiparişKalemleri { get; set; }
    public decimal ToplamÜcret { get; set; }
    public DateTime SiparişZamanı { get; set; }
}

public class SiparişKalemiDto
{
    public Guid KalemId { get; set; }
    public int Miktar { get; set; }
    public decimal BirimFiyat { get; set; }
}

```

Problem Domain Components (PDC)

PDC bileşenleri, iş süreçlerini işlerken kullanılabilir; fakat veri tabanına yazılacağı veya servis üzerinden bir istemciye gönderileceği zaman iş varlığına (BEC) dönüştürülür. Dönüştürme için ayrı bileşenler tasarlanır.

```

//PDC den BEC bileşenine dönüştürme
public static class SiparişMapper
{
    public static SiparişDto ToDto(Sipariş sipariş)
    {
        return new SiparişDto
        {
            Id = sipariş.Id,
            SiparişZamanı = sipariş.SiparişZamanı,

```

```

        SiparişKalemleri = sipariş.SiparişKalemleri.Select(i => new SiparişKalemDto
        {
            KalemId = i.KalemId,
            Miktar = i.Miktar,
            BirimFiyat = i.BirimFiyat
        }).ToList(),
        ToplamÜcret = sipariş.ToplamSiparişÜcretiGetir()
    };
}
}

```

Business Workflow Components (BWC)

Bu katman, kurumsal yazılım mimarilerinde iş süreçlerinin (**business processes/workflows**) akışını ve yönetimini tanımlayan bileşenlerdir. Genellikle iş akışlarının adım adım yürütülmesi, durumların yönetilmesi ve süreç bazlı kararların verilmesi gibi konuları kapsar.

```

public class SiparişWorkflow
{
    private readonly İÖdemeServisi ödemeServisi;
    private readonly İStokServisi stokServisi;
    private readonly İGönderimServisi gönderimServisi;

    public OrderWorkflow(İÖdemeServisi ödeme, İStokServisi stok, İGönderimServisi gönderim)
    {
        ödemeServisi = ödeme;
        stokServisi = stok;
        gönderimServisi = gönderim;
    }

    public async Task ProcessOrderAsync(Sipariş sipariş)
    {
        if (!await ödemeServisi.Charge(sipariş))
            throw new Exception("Ödeme Alınamadı!");

        if (!await stokServisi.CheckStock(sipariş.SiparişKalemleri))
            throw new Exception("Yetersiz Stok!");

        await gönderimServisi.DağıtımHazırla(sipariş);

        sipariş.Durum = SiparişDurumu.DağıtımAçıkacak;
    }
}

```

Service Interface Components (SIC)

Servis arayüzü bileşenleri SIC (**Service Interface Components**) katmanı, uygulamanın dış dünya (kullanıcılar, sistemler, API istemcileri vs.) ile iletişim kurduğu giriş noktalarıdır. Diğer bir deyişle, uygulamanın hizmetlerini dışa açan ara yüzlerdir. RESTful API uygulamasının anlatıldığı 19.6.3 başlığını inceleyiniz.

Data Access Components (DAC)

Veri erişim bileşenleri DAC (**Data Access Components**) katmanı, iş varlıklarının (BEC) tuttuğu verilere ilişkin bir kalıcılık sağlamak için veritabanı ya da diğer kalıcı veri kaynaklarına erişimi soyutlayan ve yöneten bileşenlerdir. Kalıcılığı sağlamak için bu bileşenler bazen detaylı bir mantık içerebilir. Bu mantığa, kalıcılık mantığı (**persistence logic**) adı verilir. Bunlar sayesinde uygulamanın diğer katmanları (özellikle uygulama ve iş mantığı katmanları), veri kaynağının teknik detaylarını bilmeden veri okuma/yazma işlemlerini gerçekleştirebilir. Burada veri işlemek için **depo deseni** (**repository pattern**) kullanılır. Bu desende CRUD (**Create-Read-Update-Delete**) işlevlerini de içine alan ara yüzler (**interface**) üzerinden iş bileşenleri (BEC) ihtiyaçları çözülür.

Veri erişim bileşenleri, her BEC ile ilgili veri kaynağında, yeni kayıt (create), mevcut kaydı okuma (read), kaydı güncelleme (update) ve bir kaydı silme (delete) yöntemlerini yerine getirir. Açıklanan bu işlevleri yerine getiren yöntemlerin tümü kısaca CRUD olarak adlandırılır.

CRUD yöntemleri; ABCD (**Add-Browse-Change-Delete**), ACID (**Add-Change-Inquire-Delete**), BREAD (**Browse-Read-Edit-Add-Delete**) ve VADE (**View-Add-Delete-Edit**) olarak da adlandırılır.

```
public interface ISiparişRepository
{
    Task<Sipariş?> GetByIdAsync(Guid id);
    Task AddAsync(Sipariş sipariş);
    Task SaveChangesAsync();
    Task<List<Sipariş>> GetAllAsync();
    Task DeleteAsync(Guid id);
}

public class SiparişRepository : ISiparişRepository
{
    private readonly string _connectionString;
    public SiparişRepository(string connectionString)
    {
        _connectionString = connectionString;
    }
    public async Task<List<Sipariş>> GetAllAsync()
    {
        var sipariş = new List<Sipariş>();
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand("SELECT Id, SiparişZamanı FROM Siparişler", conn);
        await conn.OpenAsync();
        using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            sipariş.Add(new Sipariş
            {
                Id = reader.GetGuid(0),
                SiparişZamanı = reader.GetDateTime(2)
            });
        }
        return sipariş;
    }
    public async Task<Sipariş?> GetByIdAsync(Guid id)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand("SELECT Id, SiparişZamanı FROM Siparişler WHERE Id = @Id", conn);
        cmd.Parameters.AddWithValue("@Id", id);
        await conn.OpenAsync();
        using var reader = await cmd.ExecuteReaderAsync();
        if (await reader.ReadAsync())
        {
            return new Sipariş
            {
                Id = reader.GetGuid(0),
                SiparişZamanı = reader.GetDateTime(2)
            };
        }
        return null;
    }
    public async Task AddAsync(Sipariş order)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand("INSERT INTO Siparişler (Id,SiparişZamanı) VALUES (@Id, @SiparişZamanı)", conn);
        cmd.Parameters.AddWithValue("@Id", order.Id);
        cmd.Parameters.AddWithValue("@SiparişZamanı", order.SiparişZamanı);
    }
}
```



```

        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
    public async Task UpdateAsync(Sipariş order)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand( "UPDATE Siparişler SET SiparişDate = @SiparişZamanı WHERE
        ↳ Id = @Id", conn);
        cmd.Parameters.AddWithValue("@Id", order.Id);
        cmd.Parameters.AddWithValue("@CustomerName", order.CustomerName);
        cmd.Parameters.AddWithValue("@SiparişDate", order.SiparişDate);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
    public async Task DeleteAsync(Guid id)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand( "DELETE FROM Siparişler WHERE Id = @Id", conn);
        cmd.Parameters.AddWithValue("@Id", id);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
}

```

Katmanlar Arası İletişim

Katmanlı mimaride, iş varlığı bileşenleri (BEC) orta katmanda yer aldığından hem kullanıcı katmanında hem de veri erişim katmanında kullanılırlar. Bu nedenle bu katmanlarda iş varlıklarını, verileri, katmanlar arasında taşırlar. Bu taşınma işlemi, genel olarak üç değişik teknoloji ile sağlanır;

1. **Sınıf** : Katmanların birbirine çok yakın olduğu durumda tercih edilir.
2. **Veri Seti (DataSet)**: Katmanlar arası veri iletişimi veri tabanı üzerinden yapılır. Genellikle aynı ağ kullanan katmanlar için tercih edilir.
3. **Serileştirme (serialization)**: Bir nesnenin durumunu (**state**) kalıcı veya taşınabilir bir şekle dönüştürme sürecidir. Katmanların birbirinden uzak olduğu durumda tercih edilir.

Buradaki tercihler uygulamanın hızını etkiler. Dolayısıyla iş varlıkları tasarlanırken yukarıdaki maddelerden doğru olanını tercih etmek gerekir. Serileştirme, 16.8 başlığında işlenmişti.

Katmanlar Arası İstisna İletimi

Çok katmanlı mimaride, katmanların birinde olan istisna, kullanıcı katmanına kadar yeniden fırlatılır (**re-throw**). Aksi takdirde istisna kullanıcı katmanına ulaşmadan program kilitlenir. Kullanıcı istisnanın ne olduğu konusunda bilgi sahibi olamaz. Aşağıda istemci sunucu mimarisinde istisna iletimi örneği verilmiştir.

```

//Sunucu.cs (Class Library)
namespace Sunucu
{
    public class Server {
        public int Bölme(int p1,int p2){
            int ret = 0;
            try {
                ret = p1 / p2;
            } catch (Exception e){
                throw e;
                //hata iletme(rethrow):hata kullanıcı
                //katmanına iletilir.
            }
            return ret;
        }
    }
}

```

Bu sunucu kodunu kullanan istemci uygulaması da aşağıda verilmiştir;

```

//İstemci.cs (Console Application)
using Sunucu; //Bunu yapabilmek için:İstemciye Sunucu Referans olarak eklenir.

```

```
namespace İstemci
{
    class Client {
        static void Main(string[] args){
            Sunucu.Server sunucu = new Sunucu.Server();
            sunucu.Bölme(12,0); //Hata burada oluşur.
        }
    }
}
```

19.5 Bileşen Kullanımı

C# dilinde bileşen kullanımı, yazılımın **modülerlik** (**modularity**) ve **bakım kolaylığı** (**maintainability**) hedeflerine hizmet eder. Bu süreçteki temel araçlar şunlardır:

- ♦ **Arayüz Tabanlı Tasarım** (**interface-based design**): Bileşenlerin birbirine sıkı sıkıya bağlı (**tightly coupled**) olmasını engelleyerek, sistemin esnekliğini artırır.
- ♦ **Paket Yönetimi** (**NuGet**): Hazır bileşenlerin sisteme entegrasyonunu ve sürüm yönetimini standartlaştırır.
- ♦ **Bağımlılık Enjeksiyonu** (**Dependency Injection**): Bileşenlerin yaşam döngülerini merkezi bir yapı üzerinden yöneterek, test edilebilirliği (**testability**) maksimize eder.

Bu yaklaşım, C# geliştiricisine altyapı detaylarıyla uğraşmak yerine, projenin **iş süreçlerine** (**business domain**) odaklanma fırsatı sunar.

§19.5.1 Sınıf Kütüphaneleri ve DLL Yapısı

C#'ta bileşen kullanımının en fiziksel hali **Class Library** (**.dll**) projeleridir. Bir iş mantığını (örneğin; ödeme yönetimi veya iz kaydı) ayrı bir kütüphane olarak derlediğinizde, derlenmiş bu bileşeni projenize sadece referans vererek dahil edebilirsiniz.

Bileşenin iç işleyişi gizlidir; kullanıcı sadece izin verilen **public** metotlarla etkileşime girer (kapsülleme).

§19.5.2 NuGet ve Paket Yönetimi

C# ekosisteminde bileşen kullanımı denilince akla ilk gelen **NuGet** paket yöneticisidir. NuGet, dünya çapındaki geliştiricilerin hazırladığı bileşenlerin "mağazası" gibidir. Kendi yazdığınız bir bileşeni paketleyip şirket içi bir sunucuya veya genel NuGet galerisine yükleyerek binlerce projede saniyeler içinde kullanılabilir hale getirebilirsiniz.

§19.5.3 Arayüzler ile Gevşek Bağlılık

Bileşen tabanlı mimarinin C# tarafındaki kalbi **interface** yapılarıdır. Bir bileşeni kullanırken doğrudan o sınıfa bağımlı olmak yerine bir arayüze bağımlı olursunuz. Örnek: **ILogger** arayüzünü kullanan bir bileşeniniz varsa, yarın öbür gün iz kayıtlarını dosyaya yazan bileşeni söküp, yerine veritabanına yazan bileşeni takabilirsiniz. Kodun geri kalanı bu değişimden etkilenmez.

§19.5.4 Bağımlılık Enjeksiyonu

Modern C#, özellikle .NET Core ve sonrası, bileşen kullanımını **bağımlılık enjeksiyonu** (**dependency injection**) konteyneri ile standart hale getirmiştir. Uygulama çalışma zamanında (**run-time**) hangi bileşenin (servisin) kullanılacağına karar verir. Bu, bileşenlerin birbirine "geçmeli" bir yapıda (Lego parçaları gibi) çalışmasını sağlar.

§19.5.5 Kullanıcı Arayüzü Bileşenleri

C# sadece arka planda değil, kullanıcı arayüzünde UI (**user interface**) de bileşen mantığını kullanır:

- ♦ **Blazor**: Web tarafında tamamen "component-based" bir yapı sunar (örneğin; bir butonu veya tabloyu `<MyCustomTable />` şeklinde bileşen olarak tanımlayıp her yerde kullanabilirsiniz).
- ♦ **WinForms/WPF**: "Custom Control" yapıları ile kendi görsel bileşenlerinizi oluşturmanıza olanak tanır.

19.6 Dağıtık Programlama

Dağıtık Programlama (**distributed programming**), tek bir bilgisayarın kaynaklarının yetmediği veya sistemin kesintisiz çalışmasının kritik olduğu durumlarda, iş yükünün ağ üzerinden birbirine bağlı birden fazla bilgisayara (node) paylaştırılmasıdır. Modern dünyada artık "süper güçlü tek bir bilgisayar" yerine, "birlikte çalışan binlerce ekonomik bilgisayar" felsefesi hakimdir. Dağıtık programlamayı şu üç ana avantajıyla tanımlayabiliriz:

1. Yatay Ölçeklenebilirlik (**horizontal scaling**): Sisteme yeni bir sunucu ekleyerek kapasiteyi artırabilirsiniz.
2. Hata Toleransı (**fault tolerance**): Bir bilgisayar bozulduğunda, diğerleri onun görevini devralır; sistem kapanmaz.
3. Düşük Gecikme (**low latency**): Veriyi kullanıcıya en yakın sunucudan göndererek hızı artırabilirsiniz (**edge computing**).

Mantıksal katmanların birbirinden fiziksel olarak ayrıldığı ve farklı ortamlar üzerinde konumlandırıldığı sistemlere dağıtık uygulamalar (**distributed application**) adı verilir. Dağıtık yazılımlarda iş yükü tek bir kod bloğu veya tek bir merkezi bilgisayar tarafından yürütülmez; bunun yerine her bir katman farklı bir donanım, sunucu veya süreç (**process**) üzerinde koşturulur. İstemci ile sunucu mimarisinin birbirinden ayrıldığı dağıtık yazılımlara olan ihtiyaç, temel olarak şu gerekçelere dayanmaktadır:

- ♦ **Karmaşıklığın Yönetimi**: Bazı büyük ve karmaşık sistemler, işleme güçlerini optimize edebilmek için dağıtık yapılar üzerinde çalışmak zorundadır.
- ♦ **Maliyet ve Performans Avantajı**: Tek bir süper bilgisayar yerine, ağ üzerinden birbirine bağlı çok sayıda ekonomik bilgisayarın işlemci gücünü kullanmak (**parallel processing**), maliyet-performans açısından büyük üstünlük sağlar.
- ♦ **Ölçeklenebilirlik** (**scalability**): Dağıtık mimariler, artan kullanıcı yükü karşısında sisteme yeni düğümler (**node**) eklenerek kapasitenin kolayca artırılmasına imkân tanır.
- ♦ **Donanım Özellikleri**: Bazı özel yazılım bileşenleri, yalnızca belirli donanım yeteneklerine veya işletim sistemlerine sahip sunucularda çalışabilmektedir.
- ♦ **Güvenlik ve İzolasyon**: Kritik verileri barındıran servislerin, güvenlik gerekçesiyle dış dünyaya açık servislerden fiziksel olarak ayrı sunucularda (örneğin; **DMZ** bölgelerinde) tutulması zorunludur.
- ♦ **Hizmet Odaklılık SOA** (**Service Oriented Architecture**): Bazı servislerin sadece birer kaynak veya merkezi API sağlayıcısı olarak konumlandırıldığı, birçok farklı uygulamanın bu merkezi servislerden beslendiği senaryolar.

§19.6.1 Dağıtık Programlama Teknolojileri

Dağıtık uygulamalarda, farklı konumlardaki bileşenlerin birbiriyle veri alışverişi yapabilmesi için tarihsel süreçten günümüze pek çok teknoloji geliştirilmiştir. Bu teknolojiler, iletişimin gerçekleştiği kapsama göre sınıflandırılabilir:

- ♦ **Süreç İçi** (**In-process communication**): Aynı uygulama süreci (**process**) içinde çalışan bileşenler için özel bir iletişim teknolojisine ihtiyaç duyulmaz; doğrudan bellek erişimi ve metod çağrıları kullanılır.
- ♦ **Süreçler Arası İletişim** (**Out-process/IPC-Inter Process Communication**): Aynı işletim sistemi üzerinde fakat farklı süreçlerde çalışan uygulamalar arası iletişimdir. Pano (**clipboard**), Paylaşılan Bellek (**shared memory**), LPC (**Lightweight Procedure Call**) gibi yöntemlerin yanı sıra; COM, DCOM, CORBA ve Java tarafında RMI gibi teknolojiler bu ihtiyacı karşılamak için kullanılmıştır.
- ♦ **Ağlar Arası ve Uzak İletişim** (**Inter-network communication/RPC- remote process communication**): Farklı bilgisayarların bir ağ üzerinden konuşmasını sağlayan yapılardır. Geçmişte .NET Remoting ve JAX-WS gibi protokoller yayginken; WCF (**Windows Communication Foundation**) bu süreçleri standartlaştıran en kapsamlı kütüphanelerden biri olmuştur.

Günümüzde bağlantı hızlarının ve işlem kapasitelerinin artmasıyla birlikte, yukarıdaki teknolojilerin bir kısmı yerini daha hafif, esnek ve yüksek performanslı modern yaklaşımlara bırakmıştır:

- ♦ **RESTful Web Servisleri**: HTTP protokolü üzerinden JSON veya XML formatında veri alışverişi sağlar. ASP.NET Core Web API bu servislerin oluşturulmasında endüstri standardı haline gelmiştir.
- ♦ **gRPC** (**Google Remote Procedure Call**): HTTP/2 protokolü ve **Protocol Buffers** (kısaca **protobuf**) kullanarak çok yüksek performanslı ve ikili (**binary**) formatta iletişim sağlar. Grpc.AspNetCore kütüphanesi ile modern sistemlerde mikroservisler arası hızlı veri aktarımı için tercih edilir.
- ♦ **Mesaj Kuyrukları** (**message queues**): Uygulamalar arasında asenkron (eşzamansız) haberleşmeyi mümkün kılar. RabbitMQ, Apache Kafka, ActiveMQ ve Azure Service Bus sistemin kopukluklara karşı dirençli (**resilience**) olmasını sağlar.
- ♦ **Gerçek Zamanlı İletişim** (**SignalR**): Sunucunun istemciye anlık veri ittiği (push notification), sohbet veya borsa uygulamaları gibi senaryolar için kullanılır.

§19.6.2 Tekdüzen Kaynak Bulucu ve Port

Her bir bilgisayar, ağ üzerinde bir IP (**Internet Protocol**) adresine sahiptir. Bu adresleri ezberlememek için bilgisayarın IP adresi yerine www.ornek.org gibi **alan adı** (domain name) adlı bir metin kullanılır. Bu alan adı ile IP eşleştirmesini **alan adı sunucuları** DNS (**Domain Name Server**) yapar.

İnternete açık bir bilgisayardaki web sayfaları, görseller veya dosyalar gibi her bir kaynağa alan adıyla birlikte kullanılan **tekdüzen kaynak bulucu** URL (**Uniform Resource Locator**) adlı bir metin kullanılır. Kısaca bilgisayarlardaki her bir kaynak da URL olarak gösterilebilir. Aşağıdaki ilk iki satırdaki kaynak aynı kaynaktır

çünkü varsayılan olarak sunucudaki `index.html` dosyası aranır. Bu sunucu ayarı istenirse değiştirilebilir.

```
http://www.ornek.org/
http://www.ornek.org/index.html
http://www.ornek.org/CSS/index.css
http://www.ornek.org/Images/logo.jpg
```

Portlar uygulamaların bilgisayarda çalıştığını gösteren sanal kapılardır. Bilgisayarlar genellikle tek bir bağlantı (**single connection**) ile ağa bağlanırlar. Buradaki tek bağlantıya sahip bilgisayara ağ üzerinden gelen paketlerin, hangi uygulamaya ait olduğunun belirlenmesi ve söz konusu paketin ilgili uygulamaya ulaştırılmasının sağlanması amacıyla portlar kullanılır.

```
http://www.ornek.org:80/
http://www.ornek.org:8080/
ftp://ftp.ornek.com:21/
```

Portlar, 0 ile 65535 arasında olup, ilk 1024 tanesi özel uygulamalar için ayrılmıştır. Örneğin 80 numaralı port **HTTP (Hyper Text Transmission Protocol)** sunucusunu, 25 numaralı port **SMTP (Simple Mail Transport Protocol)** sunucusunu ve 21 numaralı port ise **FTP (File Transfer Protocol)** sunucusunu temsil eder. URL'nin başındaki `http` ve `ftp` ise bilgisayara ulaşılan **port** üzerinde çalışan protokolü göstermektedir.

§19.6.3 Web API ve RESTful Mimari

Web Servis, internet üzerinden farklı uygulamalar arasında platform bağımsız veri alışverişi sağlayan bir yapıdır. C# ekosisteminde web servis oluşturmanın modern ve en verimli yolu **ASP.NET Core Web API** kullanmaktır. Bu platform; yüksek performansı, açık kaynak kodlu oluşu ve platformlar arası (**cross-platform**) çalışma desteği nedeniyle günümüzde endüstri standardı haline gelmiştir.

REST Mimari Stili

REST (**Representational State Transfer**), 2000 yılında *Roy Fielding* tarafından tanımlanmış bir mimari stildir. REST, web sunucuları ile istemciler arasındaki iletişimi sağlayan HTTP protokolünün, özüne sadık kalarak veri transferinde kullanılması esasına dayanır. REST kurallarına uygun olarak tasarlanan API'lere RESTful API denir. Bu mimari her şey bir kaynak (**resource**) olarak kabul edilir. Kaynaklar, URL'deki alan adından sonraki kısma benzeyen, bilgisayarda benzersiz birer URI (**Uniform Resource Identifier**) adını alır.

Yöntem	Görevi	Örnek URI	Açıklama
GET	Veri Oku	/ogrenciler	Tüm öğrencilerin listesini getirir.
GET	Belirli Veriyi Oku	/ogrenciler/5	ID değeri 5 olan öğrenciyi getirir.
POST	Veri Oluştur	/ogrenciler	Yeni bir öğrenci kaydı oluşturur.
PUT	Tam Güncelleme	/ogrenciler/5	Belirtilen kaynağın tüm verilerini günceller.
PATCH	Kısmi Güncelleme	/ogrenciler/5	Kaynağın sadece belirli alanlarını (e-posta gibi) günceller.
DELETE	Veri Sil	/ogrenciler/5	ID değeri 5 olan öğrenci kaydını siler.

Tablo 19.3: HTTP Yöntemlerinin RESTful Servislerde Kullanımı

RESTful Mimari Prensipleri

REST kurallarına uygun olarak tasarlanan API'lere **RESTful API** denir.

- Durumsuzluk (statelessness)**: Her HTTP isteği birbirinden bağımsızdır; sunucu önceki isteklerin bilgisini saklamaz. Oturum veya yetki bilgileri her istekte genellikle JWT (**JSON Web Token**) gibi jetonlar aracılığıyla yeniden iletilir.
- Önbelleklenebilirlik (cacheability)**: Sunucu, yanıtın ne kadar süreyle geçerli olduğunu belirtebilir. Bu, ağ trafiğini azaltır ve performansı artırır.
- Katmanlı Sistem (layered system)**: İstemci, isteğin doğrudan sunucuya mı yoksa bir ara katmana mı (Load Balancer, API Gateway) gittiğini bilmek zorunda değildir. Bu özellik sistemin ölçeklenebilirliğini artırır.
- Veri Formatı**: Veri alışverişinde hafifliği ve hızı nedeniyle JSON tercih edilmekle birlikte, ihtiyaç durumunda XML gibi formatlar da kullanılabilir.

§19.6.4 .NET Core RESTful Web API Örneği

REST, günümüzde istemci (web/mobil) ile sunucu arasındaki iletişimde en yaygın kullanılan standarttır. Okunabilir olması ve HTTP protokolünün tüm nimetlerinden yararlanması en büyük avantajıdır.

.NET Core RESTful Web API örneği olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
WebAPI/ → Çözüm (Solution)
└─ WebApplication1/ → Proje (ASP.NET Core Web API)
```

- Controllers/ → Denetçilerin Olacağı Klasör
 - WeatherForecastController.cs → Class (WeatherForecastController-Controller)
 - KitapKontroller.cs → Class (KitapController-Controller)
- appsettings.json → JSON (Uygulama Ayarları)
- Program.cs → Ana (Main) Program
- WeatherForecast.cs → Class (WeatherForecast-Model)
- WebApplication1.http → HTTP Ayarları

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File→New Project...** seçilerek **WebAPI** adlı çözüm içinde **WebApplication1** adlı projemiz **ASP.NET Core Web API** olarak oluşturulur.
2. Oluşan projede;
Program.cs ve
WeatherForecast.cs ile
Controllers klasöründe bulunan **WeatherForecastController.cs** dosyaları otomatik olarak oluşur.
3. İlk önce **WeatherForecast.cs** dosyası incelendiğinde MVC mimari desenindeki model olduğu görülmektedir.

```
namespace WebApplication1
{
    public class WeatherForecast
    {
        public DateOnly Date { get; set; }

        public int TemperatureC { get; set; }

        public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);

        public string? Summary { get; set; }
    }
}
```

4. Burada sözü geçen **Contollers** dizini altındaki **WeatherController.cs** dosyası MVC mimari desenindeki denetçidir. **WeatherForecastController.cs** dosyası incelendiğinde URL rotası (**route**) üzerinden **GET** isteği ile gelindiğinde 5 adet rastgele hava tahmini üreteceği görülmektedir.

```
using Microsoft.AspNetCore.Mvc;
namespace WebApplication1.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class WeatherForecastController : ControllerBase
    {
        private static readonly string[] Summaries = new[]
        {
            "Freezing", "Bracing", "Chilly", "Cool", "Mild",
            "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
        };

        private readonly ILogger<WeatherForecastController> _logger;

        public WeatherForecastController(ILogger<WeatherForecastController> logger)
        {
            _logger = logger;
        }

        [HttpGet(Name = "GetWeatherForecast")]
        public IEnumerable<WeatherForecast> Get()
        {
            return Enumerable.Range(1, 5).Select(index =>
                new WeatherForecast
                {
                    Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
                    TemperatureC = Random.Shared.Next(-20, 55),
                    Summary = Summaries[Random.Shared.Next(Summaries.Length)]
                }
            );
        }
    }
}
```

- ```

 }).ToArray();
 }
}
}

```
5. Son olarak `Program.cs` dosyasını inceleyelim. `builder.Services.AddControllers();` talimatı ile 18.5 başlığında anlatılan MVC desenindeki `MVCBaşlat()` yöntemi ile yapılan işler burada da web uygulamasının denetçileri `Controllers` klasöründen eklenir. Geri kalan tüm işlemler web sunucusunu `http` isteklerini ayarlamak için ek ayarlardır.
- ```

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();

```
6. Programı çalıştırdığımızda API uygulamamız `http` isteklerini web sunucu üzerinden istekleri kabul etmeye başlar.
- ```

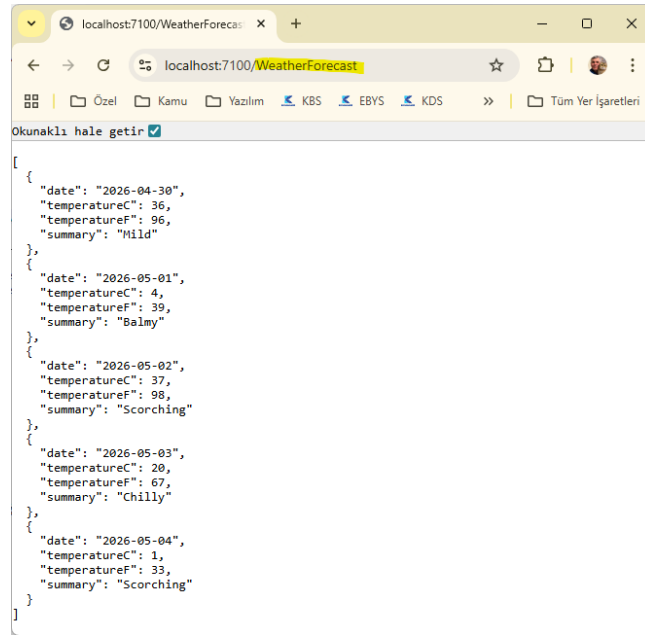
info: Microsoft.Hosting.Lifetime[14]
Now listening on: https://localhost:7100
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5100
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: C:\Users\ilhan\source\repos\WebAPI\WebApplication1

```
7. Çalışan Web API üzerinden veri almak için bir web tarayıcıyı açıp <https://localhost:7100/WeatherForecast> adresinden erişebilirsiniz (Şekil 19.6).
8. Şimdi uygulamayı durdurup yeni bir denetçi ekleyelim. Burada model kullanmayacağız; Bunun için **Solution Explorer** üzerinde `Controllers` sağ tıklanarak **Add→Controller...** seçilir ve `KitapController.cs` adında **MVC Controller-Empty** denetçi oluşturulur.
- ```

using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    public class KitapController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}

```

Şekil 19.6: Web API'ye Tarayıcı Üzerinden Erişim

9. Burada model ve gösterim ihtiyacımız olmadığından `Controllers/KitapController.cs` içinde aşağıdaki değişiklikleri yapalım.

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class KitapController : Controller
    {
        [HttpGet]
        public IActionResult TumKitaplariGetir()
        {
            var kitaplar = new[]
            {
                new { Id = 1, Ad = "Sefiller", Yazar = "Victor Hugo" },
                new { Id = 2, Ad = "Suç ve Ceza", Yazar = "Dostoyevski" }
            };

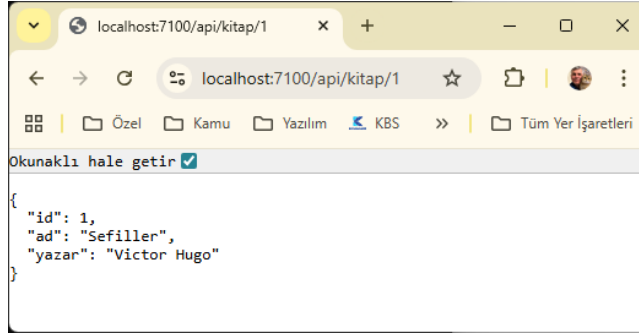
            return Ok(kitaplar);
        }

        [HttpGet("{id}")]
        public IActionResult KitapGetir(int id)
        {
            if (id == 1)
                return Ok(new { Id = 1, Ad = "Sefiller", Yazar = "Victor Hugo" });
            else
                return NotFound("Kitap bulunamadı");
        }
    }
}
```

10. Uygulamayı tekrar çalıştırıp tarayıcıda tüm kitapları getirmek için <https://localhost:7100/api/kitap>, 1 numaralı kitabı getirmek için <https://localhost:7100/api/kitap/1> adresi yazılır (Şekil 19.7).

API'mizden veri almak için örnekteki gibi tarayıcı kullanabiliriz ama bu her zaman yeterli olmaz.

◇ API'mizi test etmek için en yaygın HTTP test aracı olan **Postman**,



Şekil 19.7: Web API ile Kitap Controller'a Tarayıcı Üzerinden Erişim

- ◇ Komut satırından istek gönderme için **cURL**
- ◇ Otomatik API dokümantasyonu da çıkaran **Swagger UI** kullanılabilir.

§19.6.5 Yüksek Performanslı gRPC Uygulama Örneği

gRPC, özellikle mikroservislerin kendi aralarındaki (**internal**) haberleşmesinde tercih edilir. REST'in aksine metin (JSON) değil, ikili formatta veri taşır.

gRPC'de her şey bir **.proto** dosyası (sözleşme) ile başlar. Bu dosyada, servislerin hangi yöntemlere sahip olacağını tanımlanır.

```
syntax = "proto3";
```

```
option csharp_namespace = "FinansServisi.gRPC";
```

```
package finans;
```

```
// Servis tanımı
```

```
service FiyatTakip {
```

```
    // Unary: Tek istek, tek yanıt
```

```
    rpc HisseSenediSorgula(SorguIstegi) returns (FiyatYaniti);
```

```
    // Server Streaming: Bir istek, sürekli veri akışı
```

```
    rpc CanliFiyatTakibi(SorguIstegi) returns (stream FiyatYaniti);
```

```
}
```

```
message SorguIstegi {
```

```
    string sembol = 1;
```

```
}
```

```
message FiyatYaniti {
```

```
    string sembol = 1;
```

```
    double fiyat = 2;
```

```
    string borsa = 3;
```

```
    int64 zaman_damgasi = 4;
```

```
}
```

C# tarafında **gRPC** servisleri, otomatik üretilen sınıfları (**base class**) miras alarak yazılır.

```
using Grpc.Core;
```

```
using FinansServisi.gRPC;
```

```
public class FiyatTakipServisi : FiyatTakip.FiyatTakipBase
```

```
{
```

```
    // Unary Örneği
```

```
    public override Task<FiyatYaniti> HisseSenediSorgula(SorguIstegi request, ServerCallContext  
        context)
```

```
    {
```

```
        return Task.FromResult(new FiyatYaniti
```

```

    {
        Sembol = request.Sembol,
        Fiyat = 150.25,
        Borsa = "BIST",
        ZamanDamgasi = DateTimeOffset.UtcNow.ToUnixTimeSeconds()
    });
}

// Streaming Örneği: Yüksek performanslı veri akışı
public override async Task CanliFiyatTakibi(SorguIstegi request,
    IServerStreamWriter<FiyatYaniti> responseStream, ServerCallContext context)
{
    var rnd = new Random();
    while (!context.CancellationToken.IsCancellationRequested)
    {
        var yaniti = new FiyatYaniti
        {
            Sembol = request.Sembol,
            Fiyat = 150 + rnd.NextDouble() * 10,
            ZamanDamgasi = DateTimeOffset.UtcNow.ToUnixTimeSeconds()
        };

        await responseStream.WriteAsync(yaniti);
        await Task.Delay(1000); // 1 saniyede bir fiyat güncelle
    }
}
}

İstemci uygulamamızda, gRPC kanalı (channel) üzerinden sunucuya bağlanır.
using Grpc.Net.Client;
using FinansServisi.gRPC;

using var channel = GrpcChannel.ForAddress("https://localhost:5001");
var client = new FiyatTakip.FiyatTakipClient(channel);

// 1. Unary Çağrı
var cevap = await client.HisseSenediSorgulaAsync(new SorguIstegi { Sembol = "THYAO" });
Console.WriteLine($"Tekil Sorgu: {cevap.Sembol} -> {cevap.Fiyat}");

// 2. Streaming Çağrı
using var akis = client.CanliFiyatTakibi(new SorguIstegi { Sembol = "ASELS" });
await foreach (var fiyat in akis.ResponseStream.ReadAllAsync())
{
    Console.WriteLine($"Canlı Güncelleme: {fiyat.Sembol} -> {fiyat.Fiyat}");
}

```

§19.6.6 RESTful Web API ve gRPC Seçim Kriterleri

Özellik	RESTful API	gRPC
Protokol	HTTP 1.1 / HTTP 2	HTTP 2 (Zorunlu)
Veri Formatı	JSON (metin Tabanlı)	Protocol Buffers (ikili)
Performans	Orta (serileştirme yükü var)	Çok Yüksek (düşük CPU yükü)
Okunabilirlik	İnsan tarafından okunabilir	Makine dostu (okunamaz)
Kullanım Alanı	İstemci-Sunucu (Public API)	Mikroservisler arası (internal)

Tablo 19.4: Karşılaştırma ve Seçim Kriterleri

19.7 Modern Dağıtık Programlama: Mikroservis Mimarisi

Mikroservis mimarisi (microservices), yekpare (monolithic) yapıların aksine, bir uygulamayı birbirleriyle hafif protokoller üzerinden haberleşen, bağımsız ve küçük servisler bütünü olarak inşa etme yaklaşımıdır. Her bir servis, tek sorumluluk prensibi (single responsibility) doğrultusunda tasarlanır. Bu mimarinin temel çıktıları; hata izolasyonu (fault isolation), teknoloji bağımsızlığı ve yatay ölçeklenebilirlik (horizontal scala-

bility) olarak özetlenebilir.

§19.7.1 Mimari Katmanlar ve Bileşenler

Bir mikroservis uygulaması, monolitik yapıların aksine aşağıdaki temel bileşenlerin bir araya gelmesiyle oluşur:

1. **API Gateway:** İstemciler ile iç servisler arasında bir **proxy** görevi görür. Güvenlik ve yönlendirme burada merkezileştirilir. Uygulamanın tüm dış isteklerin (mobil uygulama, web tarayıcısı vb.) karşılandığı merkezi noktadır. İstemciler tek bir adresle konuşur, **API Gateway** bu isteği arkadaki ilgili mikroservise yönlendirir. **Kimlik doğrulama** (**authentication**), yük dengeleme (**load balancing**), isteklerin yönlendirilmesi ve hız sınırlama (**rate limiting**)
2. **Microservices:** Kendi veritabanına sahip, bağımsız olarak dağıtılabılır (**deployable**) iş birimleridir. Her biri tek bir işi (**single responsibility**) yapan bağımsız birimlerdir. Örnek: Sipariş Servisi (**Order Service**) sadece siparişleri yönetir, Stok Servisi (**stock service**) sadece envanteri tutar. Her servisin kendi veritabanı (**database per service**) vardır. Bu sayede bir servisin veritabanındaki değişiklik diğerlerini etkilemez. Veri yönetiminde ise her servisin kendi veritabanına sahip olması esastır. Bu durum veri bağımsızlığını sağlasa da, dağıtık sistemlerde **veri tutarlılığı** (**data consistency**) yönetimini zorunlu kılar.
3. **Message Broker:** Servisler arası **gevşek bağlılık** (**loose coupling**) sağlamak için asenkron iletişimi yönetir.
4. **Service Registry:** Dinamik servis adreslerinin tutulduğu ve sorgulandığı merkezi rehberdir (**service discovery**).

Bu yapı, özellikle karmaşık kurumsal sistemlerde **bakım kolaylığı** ve **hızlı yayına alma** (**time to market**) avantajları sağlar.

Burada bahsedilen kavramlar mikroservis mimarisi ile kurumsal yazılım geliştirme yapacakların bilmesi gereken kavramlardır. Zaman kritik olan ve belli bir süre içinde tamamlanması gereken süreçleri içeren yazılımlar için bu mimari tercih edilmez. **Bu kavramlar bir kitap dolduracak kadar geniş anlatılabilir.** Buradaki amacım 19.7 başlığı altında anlatılan kavramlar hakkında farkındalık yaratmaktır.

§19.7.2 Temel Prensipler ve Avantajlar

- ◊ **Bağımsız Ölçeklenebilirlik** (**scalability**): Sistemin tamamı yerine, sadece yoğunluk yaşayan spesifik servisler (örneğin; ödeme veya stok servisi) **yatayda ölçeklenebilir** (**horizontal scaling**).
- ◊ **Teknoloji Çeşitliliği** (**polyglot programming**): Her servis, iş ihtiyacına göre en uygun programlama dili (C++, Python, Go, Java, C#, ...) ve veritabanı yapısı ile geliştirilebilir.
- ◊ **Hata İzolasyonu** (**fault isolation**): Dağıtık yapı sayesinde, bir serviste meydana gelen kritik bir hata tüm sistemin çökmesine neden olmaz; sistemin geri kalanı çalışmaya devam eder (**resilience**).

§19.7.3 Mikroservislerde Haberleşme

Mikroservisler arası iletişim iki temel mekanizma ile yönetilir:

1. **Senkron İletişim:** Genellikle **REST** veya **gRPC** protokolleri üzerinden, bir servisin diğerinden yanıt beklediği durumlarda kullanılır.
2. **Asenkron İletişim:** **RabbitMQ**, **Apache Kafka** veya **Redis** gibi **mesaj kuyrukları** (**message brokers**) kullanılarak servislerin birbirine sıkı sıkıya bağlanması (**tight coupling**) engellenir.

§19.7.4 Mikroservislerde Veri Yönetimi ve Tasarım Desenleri

Dağıtık bir mimaride veritabanlarının ayrıştırılması (**decentralized data management**), beraberinde veri tutarlılığı zorluklarını getirir. Bu zorlukları aşmak için modern yazılım mühendisliğinde şu yaklaşımlar kullanılır:

Saga Tasarım Deseni

Saga deseni (**saga pattern**), dağıtık işlemleri (**distributed transactions**) yönetmek için kullanılır. Bir hata durumunda, sistemin tutarlı bir hale geri dönmesi için her başarılı işlemin bir **telafi edici işlemi** (**compensating transaction**) bulunur.

CQRS ve Event Sourcing

- ◊ **CQRS** (**command query responsibility segregation**): Yazma ve okuma modellerinin ayrılması prensibidir. Bu sayede okuma işlemleri, yazma performansını etkilemeden ölçeklenebilir.
- ◊ **Event Sourcing** (**event sourcing**): Sistemin durumunu bir olaylar dizisi olarak saklama yöntemidir. Bu, geçmişe dönük denetim (**auditing**) ve hata analizi için eşsiz bir veri kaynağı sağlar.

Hizmet Yönetimi ve Orkestrasyon

Mikroservislerin canlı ortamdaki karmaşıklığını yönetmek için **Docker** ve **Kubernetes** gibi araçlar standart haline gelmiştir. Bu araçlar; servislerin otomatik ölçeklenmesi, yük dengeleme ve **hata toleransı (fault tolerance)** süreçlerini otomatikleştirir:

- ◊ Konteyner Yapılandırması (**Docker**): Uygulamayı tüm bağımlılıklarıyla birlikte paketler.
- ◊ Orkestrasyon (**Kubernetes**): Yüzlerce konteynerin otomatik olarak ölçeklenmesini, hata durumunda yeniden başlatılmasını ve yönetilmesini sağlar.

§19.7.5 Operasyonel Zorluklar

Dağıtık mimarinin getirdiği esneklik, beraberinde bazı karmaşıklıkları da getirir:

- ◊ **Hizmet Keşfi (service discovery)**: Dinamik olarak değişen servis adreslerinin otomatik olarak tespiti. Dağıtık bir sistemde servislerin IP adresleri dinamik olarak değişebilir. Hizmet keşfi, hangi servisin nerede çalıştığını bilen bir "telefon rehberi" gibidir.
- ◊ **Dağıtık İzlenebilirlik (distributed tracing)**: Bir isteğin onlarca servis arasındaki yolculuğunun takip edilmesi ve hata ayıklama süreçleri.
- ◊ **Ağ Gecikmesi (network latency)**: Servisler arası iletişimin ağ üzerinden yapılması nedeniyle oluşan performans kısıtları.

19.8 Örnek Kurumsal Uygulama

Kurumlar, içinde oldukları zamana bağlı olarak, iş yapma şekillerini ya da süreçlerini değiştirirler. Buna kullandıkları girdilerdeki değişim, istenilen çıktılardaki değişim ya da yasal mevzuattaki değişim sebep olmaktadır. Buna paralel olarak organizasyonu modelleyen ya da benzetim yapan yazılımların bu değişiklikleri karşılayabilir olması gerekir.

Yazılımlar, değişim olmadığı takdirde, sonsuza kadar çalışabilirler. Ancak yazılımda yapılacak her değişimin, canlı bir varlığa atılan bir bıçak darbesi olacağı akıldan çıkarılmamalıdır. Bu nedenle yazılımlar geliştirilirken, değişim yönetiminin ya da hangi sürecin ne kadar değişebileceğinin önceden belirlenmesi gerekir. Değişim yönetimi ne kadar başarılı olur ise yazılım da o kadar başarılı olacaktır.

Değişim yönetiminin yanında, yazılımda daha önceden yapılmış ve başarı sağlamış olan modellerin kullanılması da oldukça önemlidir. Yani yazılım tasarımı ve analizi yapılırken başarılı tecrübeleri içeren **desenlerin (pattern)** kullanılması gerekir. Bir yazılım, ne kadar fazla desenlere uygun yazılır ise, o kadar değişime açık olacaktır. Bütün bunların yanında yazılımların katmanlara ayrılması işletim ve bakım açısından çok önemlidir. Çok katmanlı mimaride yazılan yazılımlar, ölçeklenebilirliği arttırdığı gibi birden fazla donanıma yükü paylaştırdığı için, küçük fiyatlı donanımlarla büyük işler başarılabılır.

Örnek uygulama olarak müşteriden sipariş alan basit bir uygulama ele alınacaktır. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
KurumsalUygulama/ → Çözüm (Solution)
├── Entities/ → Proje (Class Library)
│   ├── Sipariş.cs → Sınıf (Class)
├── DataAccess/ → Proje (Class Library)
│   ├── SiparişRepository.cs → Sınıf (Class)
├── Business/ → Proje (Class Library)
│   ├── SiparişYöneticisi.cs → Sınıf (Class)
├── UI/ → Proje (Console Application))
│   └── Program.cs → Ana (Main) Program
```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File**→**New**→**Project...** seçilerek **KurumsalUygulama** adlı çözüm içinde **Entities** adlı ilk projemiz **Class Library** olarak oluşturulur.
2. Proje ile oluşan **Class1.cs** dosyasını **Sipariş.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştiriyoruz;

```
//İş Varlığı Bileşenleri (BEC)
//Katmanlar arası taşınacak veri yapısını temsil eder
namespace Entities
{
    public class Sipariş
    {
        public Guid Id { get; set; }
        public string MüşteriAdı { get; set; } = string.Empty;
        public DateTime SiparişZamanı { get; set; }
    }
}
```

- ```

 }
}

```
3. Daha sonra **Solution Explorer** üzerinde çözüme sağ tıklanarak **Add→New Project...** seçilerek **DataAccess** adlı ikinci projemiz **Class Library** olarak oluşturulur.
  4. **Solution Explorer** üzerinde **Entities** projesi sağ tıklanarak **Build** seçilir ve derlenir.
  5. **Solution Explorer** üzerinde bu proje sağ tıklanarak **Add→Project Reference** ile **Entities** projesi dahil edilir.
  6. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages ...** tıklanır ve **Microsoft.Data.Sqlite** paketi yüklenir.
  7. **Visual Studio** üzerinde **Tools→SQLite/SQL Server Compact Toolbox** tıklanır ve **DataAccess** klasöründe SQLite veritabanı **Data.db** oluşturulur. Aynı yerden aşağıdaki sorgu çalıştırılarak **Siparişler** tablosunu oluşturmamız gerekir;

```

CREATE TABLE Siparişler (
 Id TEXT PRIMARY KEY, -- GUID string olarak
 MüşteriAdı TEXT NOT NULL,
 SiparişZamanı TEXT NOT NULL -- ISO 8601 tarih formatında saklanabilir
);

```

8. Daha sonra bu tabloyu veri eklemek ve verileri okumak için kullanabiliriz. Proje ile oluşan **Class1.cs** dosyasını **SiparişRepository.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştirilir;

```

//Veri Erişim Bileşenleri (DAC)
//Veri kaynağında CRUD işlemlerini yapar
using Entities;
using Microsoft.Data.Sqlite;

namespace DataAccess
{
 public class SiparişRepository
 {
 private readonly string connectionString;
 public SiparişRepository(string connectionString)
 {
 this.connectionString = connectionString;
 }
 public List<Sipariş> GetAll()
 {
 var orders = new List<Sipariş>();
 using var conn = new SqliteConnection(connectionString);
 conn.Open();
 using var cmd = conn.CreateCommand();
 cmd.CommandText="SELECT Id, MüşteriAdı, SiparişZamanı FROM Siparişler";

 using var reader = cmd.ExecuteReader();
 while (reader.Read())
 {
 orders.Add(new Sipariş
 {
 Id = reader.GetGuid(0),
 MüşteriAdı = reader.GetString(1),
 SiparişZamanı = reader.GetDateTime(2)
 });
 }
 conn.Close();
 return orders;
 }
 public void Add(Sipariş sipariş)
 {
 using var conn = new SqliteConnection(connectionString);
 conn.Open();
 using var cmd = conn.CreateCommand();
 cmd.CommandText = "INSERT INTO Siparişler (Id, MüşteriAdı, SiparişZamanı)"+
 "VALUES(@Id, @Ad, @Zaman)";
 cmd.Parameters.AddWithValue("@Id", sipariş.Id);

```

```

 cmd.Parameters.AddWithValue("@Ad", sipariş.MüşteriAdı);
 cmd.Parameters.AddWithValue("@Zaman", sipariş.SiparişZamanı);
 cmd.ExecuteNonQuery();
 conn.Close();
 }
}
}

```

9. Ardından **Solution Explorer** üzerinde çözüme sağ tıklanarak **Add→New Project...** seçilerek **Business** adlı üçüncü projemizi **Class Library** olarak oluşturuyoruz.
10. Bu projeye de sağ tıklanarak **Add→Project Reference** ile **Entities** projesi dahil edilir.
11. **Solution Explorer** üzerinde **DataAccess** projesi sağ tıklanarak **Build** seçilir ve derlenir.
12. **Business** projesi sağ tıklanarak **Add→Project Reference** ile **DataAccess** projesi dahil edilir.
13. Proje ile oluşan **Class1.cs** dosyasını **SiparişYöneticisi.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştirilir;

```

//İş Mantığı Bileşenleri (BLC)
//İş kurallarını uygular, doğrulama (validaton) yapar
using Entities;
using DataAccess;
namespace Business
{
 public class SiparişYöneticisi
 {
 private readonly SiparişRepository depo;
 public SiparişYöneticisi(SiparişRepository repo)
 {
 depo = repo;
 }
 public List<Sipariş> GetSiparişler()
 {
 return depo.GetAll();
 }
 public void SiparişOluştur(string müşteriAdı)
 {
 if (string.IsNullOrEmpty(müşteriAdı))
 throw new ArgumentException("Müşteri adı gereklidir");
 var yeniSipariş = new Sipariş
 {
 Id = Guid.NewGuid(),
 MüşteriAdı = müşteriAdı,
 SiparişZamanı = DateTime.UtcNow
 };
 depo.Add(yeniSipariş);
 }
 }
}

```

14. Son olarak **Solution Explorer** üzerinde çözüme sağ tıklanarak **Add→New Project...** seçilerek **UI** adlı dördüncü projemizi **Console Application** olarak oluşturuyoruz.
15. Bu proje sağ tıklanarak **Add→Project Reference...** ile **DataAccess** projesi dahil edilir.
16. **Solution Explorer** üzerinde **Business** projesi sağ tıklanarak **Build** seçilir ve derlenir.
17. **UI** projesine sağ tıklanarak **Add→Project Reference...** ile **Business** projesi dahil edilir.
18. Proje ile oluşan **Program.cs** aşağıdaki şekilde değiştirilir;

```

//Kullanıcı Arayıcı (UI-Console Application)
//Kullanıcı ile etkileşim sağlar

using Business;
using DataAccess;

//Çalıştırılabilir .NET uygulamasının olduğu klasör bulunur. UI Altında.
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
//DataAccess Proje klasörü bulunur.

```



```

var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü,
 ↪ @"..\..\..\DataAccess"));
var db = Path.Combine(projeKlasörü, "Data.Db");
//Bağlantı metni oluşturulur.
var connectionString = $"Data Source={db}";

var depo = new SiparişRepository(connectionString);
var yönetici = new SiparişYöneticisi(depo);

// Yeni sipariş ekleme
Console.Write("Müşteri adı girin: ");
string ad = Console.ReadLine() ?? "";
yönetici.SiparişOluştur(ad);
Console.WriteLine("Sipariş eklendi!");

// Siparişleri listeleme
var siparişler = yönetici.GetSiparişler();
foreach (var sipariş in siparişler)
{
 Console.WriteLine($"{sipariş.Id} - {sipariş.MüşteriAdı} - {sipariş.SiparişZamanı}");
}

```

19. Uygulamayı çalıştırmak için **Solution Explorer** üzerinde **UI** projesi sağ tıklanarak **Set As Startup Project** seçilir. Artık programımız çalışır.

## 19.9 Düşün ve Çöz

- ◊ **Düşün:** İş analizi (**business analysis**) ile gereksinim analizi (**requirements analysis**) arasındaki farkı 'ne istiyoruz?' ve 'bunu nasıl yapacağız?' soruları üzerinden açıklayınız. Bu iki disiplin bir yazılım projesinde nasıl iş birliği yapar?
- ◊ **Düşün:** İki katmanlı (**2-tier**), üç katmanlı (**3-tier**) ve çok katmanlı (**n-tier**) mimari arasındaki farklar nelerdir? Her katmanın değiştirilmesinin diğer katmanları etkilememe ilkesi (**separation of concerns**) pratik olarak nasıl sağlanır?
- ◊ **Düşün:** RESTful API tasarımında kaynak odaklı URL yapısı neden önemlidir? HTTP yöntemlerinin (**GET**, **POST**, **PUT**, **DELETE**) doğru kullanımı, API'nin sezgisel kullanılabilirliğini nasıl artırır?
- ◊ **Düşün:** Mikroservis mimarisi yekpare mimariye kıyasla ne gibi avantajlar sunar? Ancak neden her proje için mikroservis önerilmez? Bir startup'ın neden yekpare başlayıp daha sonra mikroservise geçmesi mantıklı olabilir?
- ◊ **Düşün:** Depo deseni (**repository pattern**) veri erişim kodunu iş mantığından nasıl ayırır? Bu ayrımın birim testi (**unit test**) yazılabilirliği üzerindeki etkisi nedir?
- ◊ **Düşün:** API versiyonlamanın önemi nedir? Yayımlanmış bir API'nin geriye dönük uyumluluğunu (**backward compatibility**) bozmadan nasıl geliştirilir? URI tabanlı (**/v1/**, **/v2/**) ve başlık tabanlı versiyonlama arasındaki fark nedir?
- ◊ **Düşün:** Önbellek (**cache**) katmanı bir kurumsal uygulamada hangi sorunları çözer? Veri tutarsızlığı (**cache invalidation**) problemini önlemek için hangi stratejiler kullanılabilir?
- ◊ **Düşün:** **MoSCoW** önceliklendirme tekniği gereksinim yönetiminde nasıl kullanılır? 'Olmazsa olmaz' ile 'olsa iyi olur' arasındaki sınırı çizmek neden zordur ve bu zorluk proje gecikmelerine nasıl yol açar?
- ◊ **Çöz:** Bir kütüphane yönetim sistemi için katmanlı mimari tasarlayın: Sunum Katmanı (**konsol**), İş Katmanı (doğrulama ve iş kuralları) ve Veri Erişim Katmanı (dosya veya SQLite) olsun. Kitap ödünç alma ve iade işlemlerini katmanlar arasında gezin.
- ◊ **Çöz:** Basit bir RESTful API istemcisi yazın: **JSONPlaceholder** gibi açık bir API üzerinden **GET**, **POST** ve **PUT** isteklerini **HttpClient** ile gerçekleştirin; yanıtları JSON'dan nesneye dönüştürün.
- ◊ **Çöz:** Repository deseni kullanarak bir ürün yönetim uygulaması tasarlayın: **IÜrünDepo** arayüzü, bellek içi (**in-memory**) ve dosya tabanlı iki gerçekleştirim yazın; iş katmanı hangi depoyu kullandığını bilmesin.
- ◊ **Çöz:** Bir siparişi işleme koyan, stok azaltan ve müşteriyi bilgilendiren üç bağımsız 'servis' tasarlayın. Bu servisleri yekpare (doğrudan çağrı) ve olaya dayalı (**event-driven**, gözlemci deseni) yaklaşımla uygulayıp karşılaştırın.
- ◊ **Çöz:** Basit bir API versiyonlama sistemi tasarlayın: aynı iş mantığına sahip iki sürümü (**/v1/**, **/v2/**) destekleyen bir yönlendirici (**router**) sınıfı oluşturun; v2'de ek alanlar dönsün.



## 20. Bölüm

# Zaman Uyumlu Programlama

Günümüz yazılım dünyasında uygulamalar yalnızca hesaplama yapmakla kalmaz; ağ üzerinden veri çeker, dosya sistemiyle etkileşime girer, veri tabanına sorgu gönderir. Bu işlemlerin tamamı **giriş-çıkış bağımlı (I/O-bound)** işlemlerdir ve donanımın yanıt vermesini bekler. Bu süre zarfında işlemciyi meşgul tutmak kaynakların israfıdır.

**Eş zamansız programlama (asynchronous programming)**, bir işlemin tamamlanmasını beklerken iş parçacığını serbest bırakmayı ve bu süreçte başka işlerin yapılmasını sağlar. C#, bu paradigmayı `async/await` sözdizimi ile son derece okunabilir bir biçimde sunar.

Bölümün sonunda aşağıdaki soruları rahatça yanıtlayabileceksiniz:

- ◇ Eş zamanlı ve eş zamansız programlama arasındaki fark nedir?
- ◇ `Task` ve `Task<T>` nasıl kullanılır?
- ◇ `async/await` anahtar kelimeleri derleyici tarafından nasıl işlenir?
- ◇ İptal mekanizması nasıl kurulur?
- ◇ Eş zamansız kodda istisnalar nasıl ele alınır?
- ◇ `ValueTask<T>` ile `Task<T>` arasındaki fark nedir?
- ◇ Paralel programlama eş zamansız programlamadan nasıl ayrışır?

## 20.1 Eş Zamansız Programlamanın Temelleri

### §20.1.1 Eş Zamanlı ve Eş Zamansız Arasındaki Fark

**Eş zamanlı (synchronous)** programlamada her işlem, bir önceki tamamlanmadan başlamaz. Bir dosya okunurken ya da ağdan veri çekilirken iş parçacığı (**thread**) bloke olur ve bekler. **Eş zamansız (asynchronous)** programlamada ise işlem başlatılır, iş parçacığı serbest kalır ve işlem tamamlandığında yürütme kaldığı yerden devam eder.

### §20.1.2 G/Ç Bağımlı ve İşlemci Bağımlı İşlemler

Eş zamansız programlamanın hangi tür işlemlerde avantaj sağladığını anlamak için iki temel kategoriye ayırt etmek gerekir.

**G/Ç bağımlı (I/O-bound)** işlemler, işlemciyi değil donanımı meşgul eder: ağ isteği, dosya okuma/yazma, veri tabanı sorgusu. Bu işlemler için `async/await` idealdir.

**İşlemci bağımlı (CPU-bound)** işlemler ise hesaplama yoğunudur: görüntü işleme, şifreleme, fizik simülasyonu. Bu tür işlemler için `Task.Run()` ile ayrı bir iş parçacığına yük aktarılır.

| Tür             | Örnek                     | Önerilen Çözüm                      |
|-----------------|---------------------------|-------------------------------------|
| G/Ç Bağımlı     | HTTP isteği, dosya okuma  | <code>await</code> ile eş zamansız  |
| İşlemci Bağımlı | Şifreleme, görüntü işleme | <code>Task.Run()</code> ile paralel |

Tablo 20.1: G/Ç Bağımlı ve İşlemci Bağımlı İşlemler

### §20.1.3 .NET'te Eş Zamansız Programlamanın Tarihi

.NET'te eş zamansız programlama üç dönemde gelişmiştir:

- ◇ **APM (Asynchronous Programming Model)**: `BeginXxx/EndXxx` çift yöntemi biçimindeydi. Karmaşık ve hata eğilimliydi.
- ◇ **EAP (Event-based Asynchronous Pattern)**: `BackgroundWorker` ve olaylar kullanılıyordu. Okunabilirliği daha iyiydi, ancak yine de karmaşıktı.
- ◇ **TAP (Task-based Asynchronous Pattern)**: C#5.0 ile gelen `async/await` söz dizimi. Günümüzde standart olan yaklaşımdır.

## 20.2 Görev Tabanlı Programlama

`Task` sınıfı, `System.Threading.Tasks` isim uzayında tanımlıdır ve eş zamansız bir işlemi temsil eder. Tıpkı bir söz (promise) gibi düşünülebilir: “Bu işi başlatıyorum, bitince sana bildiririm.”

### §20.2.1 Task Sınıfı

Değer döndürmeyen eş zamansız işlemler için `Task` kullanılır.

```
using System;
using System.Threading.Tasks;

// Değer döndürmeyen görev
Task görev = Task.Run(() =>
{
 Console.WriteLine("Görev arka planda çalışıyor.");
});

görev.Wait(); // Tamamlanmasını bekle (eş zamanlı bekleme)
Console.WriteLine("Görev tamamlandı.");
```

### §20.2.2 Task<T> Sınıfı

Değer döndüren eş zamansız işlemler için `Task<T>` kullanılır. `T` dönüş tipidir.

```
using System;
using System.Threading.Tasks;

Task<int> hesaplaGörevi = Task.Run(() =>
{
 int toplam = 0;
 for (int i = 1; i <= 100; i++)
 toplam += i;
 return toplam;
});

int sonuç = hesaplaGörevi.Result; // Sonucu al (bloke eder!)
Console.WriteLine($"1'den 100'e toplam: {sonuç}");
```

**Uyarı:** `.Result` ve `.Wait()` çağrıları iş parçacığını bloke eder. Eş zamansız bağlamda `await` kullanmak tercih edilmelidir. `.Result` yalnızca eş zamanlı bağlamda veya test kodlarında kullanın.

### §20.2.3 Görev Oluşturma Yöntemleri

```
// 1. Task.Run - İşlemci bağımlı işlemler için
Task<double> karekökGörevi = Task.Run(() => Math.Sqrt(144));

// 2. Task.FromResult - Hazır değeri görev olarak sarmalar
Task<string> hazırGörev = Task.FromResult("Merhaba");

// 3. Task.Delay - Belirtilen süre bekler (Thread.Sleep'in eş zamansız karşılığı)
Task bekleGörevi = Task.Delay(TimeSpan.FromSeconds(2));

// 4. Task.CompletedTask - Tamamlanmış boş görev
Task tamamlananGörev = Task.CompletedTask;
```

### §20.2.4 Görev Durumları

Bir `Task` nesnesi yaşam döngüsü boyunca çeşitli durumlardan geçer.

| Durum                        | Açıklama                     |
|------------------------------|------------------------------|
| <code>Created</code>         | Oluşturuldu, henüz başlamadı |
| <code>WaitingToRun</code>    | Zamanlayıcı kuyruğunda       |
| <code>Running</code>         | Yürütülüyor                  |
| <code>RanToCompletion</code> | Başarıyla tamamlandı         |
| <code>Faulted</code>         | İstisna fırlatıldı           |
| <code>Canceled</code>        | İptal edildi                 |

Tablo 20.2: Task Durum Geçişleri

## 20.3 async ve await Anahtar Kelimeleri

`async` ve `await`, C# 5.0 ile dile eklenen ve TAP modelini son derece okunabilir kılan sözdizimi öğeleridir.

### §20.3.1 Temel Kullanım

```
using System;
using System.Net.Http;
using System.Threading.Tasks;

class HaberServisi
{
 private readonly HttpClient _istemci = new();

 // async anahtar kelimesi yöntemi eş zamansız yapar
 // Task<string> dönüş tipi zorunludur (void yerine)
 public async Task<string> HaberleriGetirAsync()
 {
 // await, görev tamamlanana kadar yöntemi duraklatır
 // ancak iş parçacığını BLOKELEMEZ
 string içerik = await _istemci.GetStringAsync(
 "https://api.example.com/haberler");
 return içerik;
 }
}

class Program
{
 static async Task Main(string[] args)
 {
 HaberServisi servis = new();
 string haberler = await servis.HaberleriGetirAsync();
 Console.WriteLine($"Alınan veri boyutu: {haberler.Length} karakter");
 }
}
```

### §20.3.2 Derleyici Dönüşümü

`async/await` bir söz dizimi şekeridir (*syntactic sugar*). Derleyici bu yapıyı bir durum makinesine (*state machine*) dönüştürür. Aşağıdaki iki kod işlevsel olarak eşdeğerdir:

```
// async/await ile - okunabilir biçim
public async Task<int> DeğerGetirAsync()
{
 int sonuç = await HesaplaAsync();
 return sonuç * 2;
}

// Derleyicinin ürettiğine yakın biçim - karmaşık
public Task<int> DeğerGetirAsync()
{
 return HesaplaAsync().ContinueWith(görev =>
 {
 int sonuç = görev.Result;
 return sonuç * 2;
 });
}
```

### §20.3.3 async Yöntem İmzaları

`async` anahtar kelimesiyle işaretlenmiş yöntemler üç farklı dönüş tipi olabilir.

**Kural:** Olay işleyicileri dışında `async void` kullanmaktan kaçının. İstisnaları yakalamak imkânsız hale gelir.

| Dönüş Tipi   | Kullanım Amacı                | Örnek                             |
|--------------|-------------------------------|-----------------------------------|
| Task         | Değer döndürmeyen eş zamansız | async Task YapAsync()             |
| Task<T>      | Değer döndüren eş zamansız    | async Task<int> HesaplaAsync()    |
| void         | Yalnızca olay işleyicileri    | async void Buton_Tıklandı(...)    |
| ValueTask<T> | Performans kritik durum       | async ValueTask<int> HızlıAsync() |

Tablo 20.3: async Yöntem Dönüş Tipleri

### §20.3.4 Birden Fazla Görevi Bekleme

#### Task.WhenAll ile Hepsini Bekle

Birden fazla görevi paralel başlatıp hepsinin tamamlanmasını bekler.

```
using System;
using System.Threading.Tasks;

class VeriAkümülatörü
{
 public async Task<int[]> TümVerileriGetirAsync()
 {
 // Üç sorgu paralel olarak başlatılır
 Task<int> sorgu1 = VeritabanıSorgusuAsync("SELECT COUNT(*) FROM Müşteriler");
 Task<int> sorgu2 = VeritabanıSorgusuAsync("SELECT COUNT(*) FROM Siparişler");
 Task<int> sorgu3 = VeritabanıSorgusuAsync("SELECT COUNT(*) FROM Ürünler");

 // Üçü de bitene kadar bekle
 int[] sonuçlar = await Task.WhenAll(sorgu1, sorgu2, sorgu3);

 Console.WriteLine($"Müşteriler: {sonuçlar[0]}");
 Console.WriteLine($"Siparişler: {sonuçlar[1]}");
 Console.WriteLine($"Ürünler: {sonuçlar[2]}");

 return sonuçlar;
 }

 private async Task<int> VeritabanıSorgusuAsync(string sorgu)
 {
 await Task.Delay(500); // Veritabanı gecikmesini simüle et
 return new Random().Next(100, 1000);
 }
}
```

#### Task.WhenAny ile İlk Biteni Yakala

Birden fazla görev arasından ilk tamamlananı yakalar. Zaman aşımı (timeout) senaryolarında kullanışlıdır.

```
using System;
using System.Threading.Tasks;

class ZamanAşımıÖrneği
{
 public async Task<string> ZamanAşımıİleGetirAsync()
 {
 Task<string> veriGetirGörevi = VeriyiGetirAsync();
 Task zamanAşımıGörevi = Task.Delay(TimeSpan.FromSeconds(3));

 // Hangisi önce biterse
 Task tamamlanan = await Task.WhenAny(veriGetirGörevi, zamanAşımıGörevi);

 if (tamamlanan == veriGetirGörevi)
 return await veriGetirGörevi; // Veri zamanında geldi
 else
 throw new TimeoutException("Veri 3 saniyede gelmedi.");
 }
}
```

```

private async Task<string> VeriyiGetirAsync()
{
 await Task.Delay(2000); // 2 saniyelik ağ gecikmesi
 return "Sunucudan gelen veri";
}

```

### §20.3.5 ConfigureAwait(false) ile Bağlam Yönetimi

`await` varsayılan olarak mevcut eşitleme bağlamına (`synchronization context`) geri döner. Kütüphane kodunda bu gereksiz yük oluşturur.

```

// Kütüphane kodu – UI bağlamına dönmek gerekmez
public async Task<string> KütüphaneYöntemAsync()
{
 // ConfigureAwait(false) ile UI thread'e geri dönme
 string veri = await HttpİsteğiAsync().ConfigureAwait(false);
 return veri.ToUpper();
}

// Uygulama kodu – UI güncellemesi gerektiğinde ConfigureAwait kullanma
private async void YenileButon_Tıklandı(object sender, EventArgs e)
{
 // UI thread'e geri dönmesi gerekir; ConfigureAwait(false) YOK
 string sonuç = await VeriyiGetirAsync();
 etiket.Text = sonuç; // UI güncellemesi
}

```

**Kural:** Kütüphane ve altyapı kodlarında `ConfigureAwait(false)` kullanın. Uygulama katmanı ve UI kodlarında kullanmayın.

## 20.4 İptal Belirteci

Uzun süren eş zamansız işlemlerin kullanıcı isteğiyle veya zaman aşımıyla iptal edilebilmesi gerekir. .NET bu amaç için `CancellationToken` ve `CancellationTokenSource` çiftini sunar.

### §20.4.1 Temel İptal Mekanizması

```

using System;
using System.Threading;
using System.Threading.Tasks;

class Dosyaİşlemcisi
{
 // Yöntem, dışarıdan iptal belirteci kabul eder
 public async Task BüyükDosyayıİşleAsync(
 string dosyaYolu,
 CancellationToken iptalBelirteci = default)
 {
 Console.WriteLine("Dosya işleme başladı...");

 for (int i = 0; i < 100; i++)
 {
 // Her döngüde iptal istendi mi kontrol et
 iptalBelirteci.ThrowIfCancellationRequested();

 await Task.Delay(100, iptalBelirteci); // İptal belirtecini Delay'e de ver
 Console.WriteLine($"İşlenen satır: {i + 1}/100");
 }

 Console.WriteLine("Dosya işleme tamamlandı.");
 }
}

```

```

class Program
{
 static async Task Main(string[] args)
 {
 using CancellationTokenSource kaynak = new();
 Dosyaİşlemcisi işlemci = new();

 // 2 saniye sonra iptal et
 kaynak.CancelAfter(TimeSpan.FromSeconds(2));

 try
 {
 await işlemci.BüyükDosyayıİşleAsync("veri.txt", kaynak.Token);
 }
 catch (OperationCanceledException)
 {
 Console.WriteLine("İşlem kullanıcı tarafından iptal edildi.");
 }
 }
}
/*Program Çıktısı:
Dosya işleme başladı...
İşlenen satır: 1/100
İşlenen satır: 2/100
...
İşlenen satır: 19/100
İşlem kullanıcı tarafından iptal edildi.
*/

```

## §20.4.2 Manuel İptal

```

using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
 static async Task Main(string[] args)
 {
 using CancellationTokenSource kaynak = new();

 // Kullanıcı tuşa basınca iptal et
 Console.WriteLine("Herhangi bir tuşa basınca işlem iptal edilir.");

 Task işlemGörevi = UzunİşlemAsync(kaynak.Token);

 // Arka planda tuş bekle
 Task tuşGörevi = Task.Run(() =>
 {
 Console.ReadKey(intercept: true);
 kaynak.Cancel(); // İptali tetikle
 });

 try
 {
 await Task.WhenAny(işlemGörevi, tuşGörevi);
 await işlemGörevi; // İstisna varsa yeniden fırlat
 }
 catch (OperationCanceledException)
 {
 Console.WriteLine("\nİşlem iptal edildi!");
 }
 }
}

```

```

static async Task UzunİşlemAsync(CancellationToken iptal)
{
 for (int i = 0; i < 50; i++)
 {
 iptal.ThrowIfCancellationRequested();
 Console.WriteLine($"{i + 1}/50 tamamlandı...");
 await Task.Delay(300, iptal);
 }
 Console.WriteLine("\nİşlem başarıyla tamamlandı.");
}

```

### §20.4.3 Bağlantılı İptal Kaynakları

Birden fazla iptal kaynağını birleştirerek karma koşullarla iptal sağlanabilir.

```

using System;
using System.Threading;
using System.Threading.Tasks;

// Kullanıcı isteği veya zaman aşımı – hangisi önce gelirse iptal et
using CancellationTokenSource kullanıcıKaynağı = new();
using CancellationTokenSource zamanAşımıKaynağı = new(TimeSpan.FromSeconds(5));

// İki kaynağı birleştir
using CancellationTokenSource birleşik =
 CancellationTokenSource.CreateLinkedTokenSource(
 kullanıcıKaynağı.Token,
 zamanAşımıKaynağı.Token);

try
{
 await VeriGetirAsync(birleşik.Token);
}
catch (OperationCanceledException)
{
 if (zamanAşımıKaynağı.IsCancellationRequested)
 Console.WriteLine("Zaman aşımı nedeniyle iptal edildi.");
 else
 Console.WriteLine("Kullanıcı tarafından iptal edildi.");
}

```

## 20.5 İstisna Yönetimi Eş Zamansız Bağlamda

Eş zamansız programlamada istisna yönetimi, eş zamanlı koddan birkaç önemli farkla ayrışır.

### §20.5.1 Temel İstisna Yakalama

```

using System;
using System.Net.Http;
using System.Threading.Tasks;

class HaberServisi
{
 private readonly HttpClient _istemci = new();

 public async Task<string> GüvenliGetirAsync(string url)
 {
 try
 {
 string veri = await _istemci.GetStringAsync(url);
 return veri;
 }
 catch (HttpRequestException hre)
 {
 }
 }
}

```



```

 {
 Console.WriteLine($"Ağ hatası: {hre.Message}");
 return string.Empty;
 }
 catch (TaskCanceledException)
 {
 Console.WriteLine("İstek zaman aşımına uğradı.");
 return string.Empty;
 }
}
}

```

## §20.5.2 Birden Fazla Hatayı Toplama

`Task.WhenAll` ile birden fazla görev paralel çalıştırıldığında, birden fazla istisna oluşabilir. Bu istisnalar `AggregateException` içinde toplanır.

```

using System;
using System.Threading.Tasks;

class Program
{
 static async Task Main(string[] args)
 {
 Task görev1 = HatalıİşlemAsync("Görev 1");
 Task görev2 = HatalıİşlemAsync("Görev 2");
 Task görev3 = BaşarılıİşlemAsync();

 try
 {
 await Task.WhenAll(görev1, görev2, görev3);
 }
 catch (Exception)
 {
 // Her görevin durumunu ayrı ayrı incele
 foreach (Task görev in new[] { görev1, görev2, görev3 })
 {
 if (görev.IsFaulted)
 {
 // AggregateException içindeki gerçek hataya eriş
 Exception gerçekHata = görev.Exception!.InnerException!;
 Console.WriteLine($"Hata: {gerçekHata.Message}");
 }
 }
 }
 }

 static async Task HatalıİşlemAsync(string ad)
 {
 await Task.Delay(100);
 throw new InvalidOperationException($"ad} başarısız oldu.");
 }

 static async Task BaşarılıİşlemAsync()
 {
 await Task.Delay(50);
 Console.WriteLine("Görev 3 başarıyla tamamlandı.");
 }
}

/*Program Çıktısı:
Görev 3 başarıyla tamamlandı.
Hata: Görev 1 başarısız oldu.
Hata: Görev 2 başarısız oldu.

```

\*/

### §20.5.3 async void ve İstisna Tuzağı

`async void` yöntemlerinde fırlatılan istisnalar yakalanamaz ve uygulamayı çökertebilir.

```
// YANLIŞ – istisna yakalanamaz
async void YanlışKullanım()
{
 await Task.Delay(100);
 throw new Exception("Bu istisna kaybolur!"); // Uygulama çöker
}

// DOĞRU – yalnızca olay işleyicilerinde void kullan, içinde yakala
async void Buton_Tıklandı(object sender, EventArgs e)
{
 try
 {
 await EşZamansızİşlemAsync();
 }
 catch (Exception hata)
 {
 MesajGöster(hata.Message); // Hatayı kullanıcıya bildir
 }
}
```

## 20.6 ValueTask<T> ile Performans Optimizasyonu

`Task<T>` her çağrıda öbek belleğinde (heap) yeni bir nesne oluşturur. Yüksek frekanslı yöntemlerde bu durum çöp toplayıcıya (GC) baskı yapar. `ValueTask<T>`, sonucun eş zamanlı olarak hazır olduğu durumlarda öbek bellek tahsisini önler.

### §20.6.1 Ne Zaman ValueTask<T> Kullanılmalı?

- ◊ Yöntem çoğunlukla eş zamanlı (kısık sürede) tamamlanıyorsa
- ◊ Saniyede binlerce kez çağrılan sıcak yollarda (hot path)
- ◊ Bellek tahsisi darboğaz oluşturuyorsa

```
using System;
using System.Threading.Tasks;
using System.Collections.Generic;

class ÖnbellekServisi
{
 private readonly Dictionary<string, string> _önbellek = new();

 // ValueTask<T> – çoğunlukla önbellekten (eş zamanlı) döner
 public ValueTask<string> DeğerGetirAsync(string anahtar)
 {
 // Önbellekte varsa eş zamanlı dön – tahsis yok!
 if (_önbellek.TryGetValue(anahtar, out string? değer))
 return new ValueTask<string>(değer);

 // Yoksa ağdan getir – gerçek eş zamansız işlem
 return new ValueTask<string>(AğdanGetirAsync(anahtar));
 }

 private async Task<string> AğdanGetirAsync(string anahtar)
 {
 await Task.Delay(500); // Ağ gecikmesi
 string değer = $"Değer_{anahtar}";
 _önbellek[anahtar] = değer;
 return değer;
 }
}
```

```

class Program
{
 static async Task Main(string[] args)
 {
 ÖnbellekServisi servis = new();

 // İlk çağrı - ağdan gelir (eş zamansız)
 string v1 = await servis.DeğerGetirAsync("kullanıcı:1");
 Console.WriteLine($"İlk çağrı: {v1}");

 // İkinci çağrı - önbellekten gelir (eş zamanlı, tahsis yok)
 string v2 = await servis.DeğerGetirAsync("kullanıcı:1");
 Console.WriteLine($"İkinci çağrı: {v2}");
 }
}

```

| Özellik                  | Task<T>        | ValueTask<T>           |
|--------------------------|----------------|------------------------|
| Bellek tahsisi           | Her zaman öbek | Eş zamanlı durumda yok |
| Kullanım kolaylığı       | Yüksek         | Biraz daha karmaşık    |
| Birden fazla await       | Desteklenir    | Desteklenmez           |
| Ne zaman tercih edilmeli | Genel kullanım | Sıcak yollar, önbellek |

Tablo 20.4: Task&lt;T&gt; ve ValueTask&lt;T&gt; Karşılaştırması

## 20.7 Paralel Programlama

Eş zamansız programlama G/Ç bağımlı işlemler için tasarlanmışken, **paralel programlama** işlemci bağımlı işlemleri çok çekirdekli işlemcilerde aynı anda yürütmek için tasarlanmıştır.

### §20.7.1 Eş Zamansız ve Paralel Arasındaki Fark

| Özellik      | Eş Zamansız             | Paralel                   |
|--------------|-------------------------|---------------------------|
| Hedef        | G/Ç bağımlı             | İşlemci bağımlı           |
| İş parçacığı | Az (genelde 1)          | Çok (çekirdek sayısı)     |
| Mekanizma    | async/await             | Parallel, PLINQ           |
| Örnek        | HTTP isteği, DB sorgusu | Görüntü işleme, şifreleme |

Tablo 20.5: Eş Zamansız ve Paralel Programlama Karşılaştırması

### §20.7.2 Parallel.For ve Parallel.ForEach

```

using System;
using System.Threading.Tasks;

class Matrisİşlemcisi
{
 // Eş zamanlı versiyon - yavaş
 public static void EşZamanlıTopla(int[] dizi)
 {
 for (int i = 0; i < dizi.Length; i++)
 dizi[i] *= 2;
 }

 // Paralel versiyon - çok çekirdekli işlemcilerde hızlı
 public static void ParalelTopla(int[] dizi)
 {
 Parallel.For(0, dizi.Length, i =>
 {
 dizi[i] *= 2;
 });
 }
}

```

```

class Program
{
 static void Main(string[] args)
 {
 int[] veri = new int[10_000_000];
 for (int i = 0; i < veri.Length; i++)
 veri[i] = i;

 // Paralel.For ile işle
 Parallel.For(0, veri.Length, i =>
 {
 veri[i] = veri[i] * veri[i]; // Her elemanın karesini al
 });

 Console.WriteLine($"İlk 5 eleman: {veri[0]}, {veri[1]}, " +
 $"{veri[2]}, {veri[3]}, {veri[4]}");
 }
}
/*Program Çıktısı:
İlk 5 eleman: 0, 1, 4, 9, 16
*/
\end{csharp}

\subsubsection{Paralel Seçenekleri – \texttt{ParallelOptions}}

\begin{minted}{csharp}
using System;
using System.Threading;
using System.Threading.Tasks;

using CancellationTokenSource kaynak = new();

ParallelOptions seçenekler = new()
{
 MaxDegreeOfParallelism = Environment.ProcessorCount, // Çekirdek sayısı kadar
 CancellationToken = kaynak.Token
};

try
{
 Parallel.For(0, 1000, seçenekler, i =>
 {
 // Her yineleme için iptal kontrolü otomatik yapılır
 Console.Write($"{i} ");
 });
}
catch (OperationCanceledException)
{
 Console.WriteLine("\nParalel işlem iptal edildi.");
}

```

### §20.7.3 PLINQ

Paralel LINQ (Kısaca PLINQ), LINQ sorgularını paralel olarak çalıştırır. Yalnızca `.AsParallel()` çağrısı eklemek yeterlidir.

```

using System;
using System.Linq;
using System.Collections.Generic;

class PLINQÖrneği
{

```

```

static void Main(string[] args)
{
 List<int> sayılar = Enumerable.Range(1, 10_000_000).ToList();

 // Eş zamanlı LINQ
 var eşZamanlıSonuç = sayılar
 .Where(s => s % 2 == 0)
 .Select(s => s * s)
 .Sum();

 // Paralel LINQ – çok çekirdekli işlemcilerde otomatik hızlanır
 var paralelSonuç = sayılar
 .AsParallel()
 .Where(s => s % 2 == 0)
 .Select(s => s * s)
 .Sum();

 Console.WriteLine($"Eş zamanlı: {eşZamanlıSonuç}");
 Console.WriteLine($"Paralel: {paralelSonuç}");
}

```

### PLINQ Sıralı Çıktı

Paralel sorgunun sonuçları sırasız gelebilir. Sıra önemliyse `AsOrdered()` kullanın.

```

// Sırasız (daha hızlı)
var sonuç = Enumerable.Range(1, 100)
 .AsParallel()
 .Select(i => i * i)
 .ToList();

// Sıralı (kaynak sırasını korur, biraz daha yavaş)
var sıralıSonuç = Enumerable.Range(1, 100)
 .AsParallel()
 .AsOrdered()
 .Select(i => i * i)
 .ToList();

```

### §20.7.4 İş Parçacığı Güvenliği ve Paylaşılan Veri

Paralel programlamada birden fazla iş parçacığı aynı veriye eriştiğinde **yarış koşulu** (race condition) oluşabilir.

```

using System;
using System.Threading;
using System.Threading.Tasks;

class YarışKoşuluÖrneği
{
 static int sayaç = 0;

 static void Main(string[] args)
 {
 // YANLIŞ – yarış koşulu var
 Parallel.For(0, 10000, i =>
 {
 sayaç++; // Atom değil! Veri kaybı olabilir
 });
 Console.WriteLine($"Beklenen: 10000, Gerçek: {sayaç}");

 // DOĞRU – Interlocked ile atomik artırım
 sayaç = 0;
 Parallel.For(0, 10000, i =>
 {

```

```

 Interlocked.Increment(ref sayaç); // Atom işlem, güvenli
 });
 Console.WriteLine($"Interlocked ile: {sayaç}");
}
}
/*Program Çıktısı (yaklaşık):
Beklenen: 10000, Gerçek: 9347 <- hatalı
Interlocked ile: 10000 <- doğru
*/

```

### lock ile Kritik Bölge Koruma

```

using System;
using System.Threading;
using System.Threading.Tasks;

class BankaHesabı
{
 private decimal _bakiye;
 private readonly object _kilit = new(); // Kilit nesnesi

 public BankaHesabı(decimal başlangıçBakiye)
 {
 _bakiye = başlangıçBakiye;
 }

 public void ParaYatır(decimal miktar)
 {
 lock (_kilit) // Kritik bölgeye yalnızca bir thread girer
 {
 _bakiye += miktar;
 }
 }

 public bool ParaÇek(decimal miktar)
 {
 lock (_kilit)
 {
 if (_bakiye >= miktar)
 {
 _bakiye -= miktar;
 return true;
 }
 return false;
 }
 }

 public decimal Bakiye
 {
 get { lock (_kilit) { return _bakiye; } }
 }
}

```

## 20.8 Düşün ve Çöz

- ◊ **Düşün:** `async void` neden tehlikelidir? Olay işleyicileri dışında neden kullanılmamalıdır? `async void` içinde istisna fırlatıldığında ne olur? Bunu test etmek için bir örnek yazınız.
- ◊ **Düşün:** `Task.WhenAll` ve `Task.WhenAny` arasındaki fark nedir? Bir hava durumu uygulaması birden fazla kaynaktan veri çekiyorsa hangisini kullanmak daha mantıklıdır? Nedenini açıklayınız.
- ◊ **Düşün:** Aşağıdaki kod kilitlenmeye (deadlock) yol açar mı? Nedenini açıklayınız:

```

static async Task<int> DeğerGetirAsync()
{
 await Task.Delay(1000);
}

```

```

 return 42;
 }

 static void Main()
 {
 int sonuç = DeğerGetirAsync().Result; // Burada ne olur?
 Console.WriteLine(sonuç);
 }

```

- ◇ **Çöz:** Üç farklı sunucudan hava durumu verisi çeken bir uygulama yazınız. Tüm istekleri paralel gönderin. İlk 5 saniyede yanıt gelmeyen sunucular iptal edilsin. Yanıt gelen sunucuların verilerini ekrana yazdırın.
- ◇ **Çöz:** Büyük bir tam sayı listesinin asal sayılarını bulan bir program yazınız. Önce eş zamanlı LINQ ile çalışan versiyonu yazın. Ardından PLINQ versiyonunu yazın. **Stopwatch** kullanarak her iki versiyonun çalışma sürelerini karşılaştırın.
- ◇ **Düşün:** **CancellationToken** bir yöntem zincirinden geçirilirken ( $A \rightarrow B \rightarrow C$ ) en iyi uygulama nedir? Belirtecisi her yöntem parametre olarak geçirmek yerine başka bir yöntem var mıdır? Avantaj ve dezavantajlarını tartışınız.
- ◇ **Çöz:** Bir banka uygulaması düşününüz. Aynı hesaba birden fazla iş parçacığı para yatırıp çekiyor. **lock** ve **SemaphoreSlim** kullanarak iki farklı güvenli versiyon yazın. Her iki yaklaşımın ne zaman daha uygun olduğunu açıklayın.



## 21. Bölüm

# Bellek Yönetimi ve Kaynak Denetimi

Yüksek performanslı kurumsal uygulamalarda doğru yazmak yetmez; *verimli* yazmak da gerekir. .NET'in çöp toplayıcısı (GC) pek çok bellek işini otomatik üstlense de, yanlış yazılmış kod gereksiz tahsisler, uzun çöp toplama duraklamaları ve yönetilmeyen kaynak sızıntılarıyla sistemin performansını ciddi biçimde düşürebilir.

Bu bölümde .NET bellek modelini, sıfır tahsisli (**zero-allocation**) dilim yapılarını, dizi havuzlarını ve yönetilmeyen kaynakların güvenli serbest bırakılmasını öğreneceksiniz. Bölümün sonunda aşağıdaki soruları rahatlıkla yanıtlatabileceksiniz:

- Yığın (**stack**) ve öbek (**heap**) bellek arasındaki fark nedir?
- `Span<T>` neden öbek tahsisi yapmadan dilim oluşturabilir?
- `Memory<T>` ile `Span<T>` ne zaman birbirinin yerine geçemez?
- `ArrayPool<T>` kullanarak çöp toplayıcı (GC) baskısı nasıl azaltılır?
- `IDisposable` deseni doğru nasıl uygulanır?
- Eş zamansız kaynak temizleme için `IAsyncDisposable` ne zaman tercih edilir?
- Bellek sızıntısı hangi araçlarla tespit edilir?

### 21.1 .NET'te Bellek Modeli

#### §21.1.1 Yığın ve Öbek Bellek

.NET çalışma zamanı belleği iki ana bölgede yönetir: **yığın** (**stack**) ve **öbek** (**heap**). Bu iki bölgenin davranışını anlamak, performanslı kod yazmanın temelidir.

**Yığın bellek** (**stack**), iş parçacığına özgüdür ve son giren ilk çıkar (LIFO) mantığıyla çalışır. Yöntem çağrıldığında yerel değişkenler ve parametreler yığına itilir (**push**); yöntem sonlandığında otomatik olarak çıkarılır (**pop**). Tahsis ve serbest bırakma işlemi bir işlemci komutuyla gerçekleşir çöp toplayıcı devreye girmez. Ancak yığın boyutu sınırlıdır (varsayılan olarak 1 MB).

**Öbek bellek** (**heap**), uygulamanın genel bellek havuzudur. **new** anahtar kelimesiyle oluşturulan tüm nesneler buraya tahsis edilir. Çöp toplayıcı bu bölgeyi izler, erişilemeyen nesneleri tespit eder ve belleklerini geri alır. Tahsis esnektir ancak çöp toplayıcı duraklamaları performansı etkileyebilir.

| Özellik               | Yığın                      | Öbek                         |
|-----------------------|----------------------------|------------------------------|
| Tahsis hızı           | Çok hızlı (tek komut)      | Nispeten yavaş               |
| Serbest bırakma       | Otomatik (kapsam sonu)     | Çöp toplayıcı tarafından     |
| Boyut sınırı          | Küçük (~1 MB)              | Büyük (RAM ile sınırlı)      |
| Kapsam                | İş parçacığına özgü        | Uygulama genelinde paylaşılr |
| Depolanan tipler      | Değer tipleri, referanslar | Referans tipi nesneler       |
| Çöp toplayıcı baskısı | Yok                        | Var                          |

Tablo 21.1: Yığın ve Öbek Bellek Karşılaştırması

#### §21.1.2 Değer Tipleri ve Referans Tipleri

```
// Değer tipi - yığında yaşar
int x = 42;
double pi = 3.14;
DateTime bugün = DateTime.Now;

// Referans tipi - öbekte yaşar, yığında yalnızca referans tutulur
string ad = "İlhan";
int[] dizi = new int[100];
List<string> liste = new List<string>();

// Yapı (struct) - değer tipidir, yığında yaşar
// Sınıf (class) - referans tipidir, öbekte yaşar
```

```
struct Nokta { public int X; public int Y; } // yığın
class NesnNokta { public int X; public int Y; } // öbek
```

### §21.1.3 .NET Çöp Toplayıcı Kuşakları

.NET çöp toplayıcı (GC), nesneleri üç kuşakta (**generation**) yönetir. Bu yapı, kısa ömürlü nesnelerin hızla temizlenmesini, uzun ömürlü nesnelerin ise seyrek kontrol edilmesini sağlar.

| Kuşak              | İçerik                       | Çöp Toplama Sıklığı | Boyut    |
|--------------------|------------------------------|---------------------|----------|
| Kuşak 0            | Yeni tahsis edilen nesneler  | Çok sık             | ~256 KB  |
| Kuşak 1            | Kuşak 0'dan hayatta kalanlar | Orta                | ~2 MB    |
| Kuşak 2            | Uzun ömürlü nesneler         | Seyrek              | Sınırsız |
| Büyük Nesne Yığını | 85 KB üzeri nesneler         | Seyrek              | Sınırsız |

Tablo 21.2: Çöp Toplama Kuşakları ve Özellikleri

```
// Hangi kuşakta olduğunu öğrenme
object nesne = new object();
Console.WriteLine($"Kuşak: {GC.GetGeneration(nesne)}"); // Çıktı: 0

GC.Collect(); // Zorla GC (üretim kodunda kullanmayın!)
Console.WriteLine($"Kuşak: {GC.GetGeneration(nesne)}"); // Çıktı: 1

// Bellek baskısını izleme
long kullanılanBellek = GC.GetTotalMemory(forceFullCollection: false);
Console.WriteLine($"Kullanılan bellek: {kullanılanBellek / 1024} KB");
```

### §21.1.4 stackalloc ile Yığın Tahsisi

Küçük, kısa ömürlü diziler için **stackalloc** ile yığında dizi oluşturulabilir. Öbek tahsisi yapılmaz, çöp toplama devreye girmez.

```
using System;

class StackallocÖrneği
{
 static void Main()
 {
 // Yığında 256 bayt ayır - öbek tahsisi yok
 Span<byte> tampon = stackalloc byte[256];

 for (int i = 0; i < tampon.Length; i++)
 tampon[i] = (byte)i;

 Console.WriteLine($"İlk eleman: {tampon[0]}");
 Console.WriteLine($"Son eleman: {tampon[255]}");
 // Yöntem sonunda otomatik serbest bırakılır
 }
}

/*Program Çıktısı:
İlk eleman: 0
Son eleman: 255
*/
```

## 21.2 Yığın Tabanlı Bellek Dilimi

Yığın Tabanlı Bellek Dilimi **Span<T>**, C#7.3+ ile tanıtılan ve **sıfır tahsisli** (**zero-allocation**) bellek dilimleri oluşturmayı sağlayan bir yapıdır (**struct**). Mevcut bir bellek bölgesinin dizi gibi, yığın (**stack**) tahsisi veya yönetilmeyen bellek üzerine bir pencere açar; yeni bellek tahsis etmez.

**Temel kural:** **Span<T>** bir **ref struct**'tır. Bu nedenle yalnızca yığında yaşayabilir; öbekte (**heap**) depolanamaz. Sınıf alanı olamaz, **async** yöntemlerde ve **lambda** ifadelerinde kullanılamaz.

### §21.2.1 Dizi Üzerinde Dilim Oluşturma

```
using System;

class SpanTemelÖrneği
{
 static void Main()
 {
 int[] dizi = { 10, 20, 30, 40, 50, 60, 70, 80 };

 // Tüm dizi – kopyalama yok
 Span<int> tümü = dizi;

 // 2. indeksten başlayan 4 elemanlık dilim – kopyalama yok
 Span<int> dilim = dizi.AsSpan(2, 4); // {30, 40, 50, 60}

 Console.WriteLine("Dilim elemanları:");
 foreach (int eleman in dilim)
 Console.Write($"{eleman} ");
 Console.WriteLine();

 // Dilim üzerinde değişiklik orijinal diziyi etkiler
 dilim[0] = 999;
 Console.WriteLine($"Orijinal dizi[2]: {dizi[2]}"); // 999
 }
}

/*Program Çıktısı:
Dilim elemanları:
30 40 50 60
Orijinal dizi[2]: 999
*/
```

### §21.2.2 Metin İşlemede Span<T> ve Sıfır Tahsis

`string` işlemleri genellikle yeni `string` nesneleri oluşturur ve öbek baskısı yaratır. `ReadOnlySpan<char>` bu sorunu çözer.

```
using System;

class MetinDilimÖrneği
{
 // KÖTÜ: Her Substring çağrısı yeni string nesnesi tahsis eder
 static string KlasikParçala(string kaynak, char ayraç)
 {
 int indeks = kaynak.IndexOf(ayraç);
 return indeks >= 0
 ? kaynak.Substring(0, indeks) // yeni string tahsisi!
 : kaynak;
 }

 // İYİ: Sıfır tahsis – mevcut metin üzerinde pencere açar
 static ReadOnlySpan<char> SpanParçala(ReadOnlySpan<char> kaynak, char ayraç)
 {
 int indeks = kaynak.IndexOf(ayraç);
 return indeks >= 0
 ? kaynak.Slice(0, indeks) // tahsis yok!
 : kaynak;
 }

 static void Main()
 {
 string veri = "İlhan ÖZKAN;Ankara;Türkiye";

 // Klasik yol – üç yeni string nesnesi
```

```

 string[] parçalar = veri.Split(';');
 Console.WriteLine($"Ad: {parçalar[0]}");

 // Span yolu - sıfır ek tahsis
 ReadOnlySpan<char> span = veri.AsSpan();
 ReadOnlySpan<char> ad = SpanParçala(span, ';');
 Console.WriteLine($"Ad (Span): {ad.ToString()}");
 }
}
/*Program Çıktısı:
Ad: İlhan ÖZKAN
Ad (Span): İlhan ÖZKAN
*/

```

### §21.2.3 stackalloc ile Span<T> Birlikte Kullanımı

```

using System;

class YığınTahsisÖrneği
{
 // Küçük tamponu yığında oluştur, GC baskısı sıfır
 static int[] SıralıKopyala(ReadOnlySpan<int> kaynak)
 {
 // Küçük ara tampon - yığında, öbekte değil
 Span<int> geçici = stackalloc int[kaynak.Length];
 kaynak.CopyTo(geçici);

 // Yalnızca sonuç öbeğe gider
 geçici.Sort();
 return geçici.ToArray();
 }

 static void Main()
 {
 int[] veri = { 5, 2, 8, 1, 9, 3 };
 int[] sıralı = SıralıKopyala(veri);

 Console.Write("Sıralı: ");
 foreach (int e in sıralı)
 Console.Write($"{e} ");
 Console.WriteLine();
 }
}
/*Program Çıktısı:
Sıralı: 1 2 3 5 8 9
*/

```

### §21.2.4 Span<T> Kısıtlamaları

```

// DERLEME HATASI - Span sınıf alanı olamaz
class YanlışKullanım
{
 private Span<int> _alan; // HATA: ref struct sınıf alanı olamaz
}

// DERLEME HATASI - Span async yöntemde kullanılamaz
async Task YanlışAsync()
{
 Span<int> span = new int[10]; // HATA: async metodda ref struct
 await Task.Delay(100);
}

// DOĞRU - Bu kısıtlamalar için Memory<T> kullanılır
class DoğruKullanım

```

```

{
 private Memory<int> _alan; // Memory sınıf alanı olabilir

 async Task DoğruAsync()
 {
 Memory<int> bellek = new int[10]; // async içinde çalışır
 await Task.Delay(100);
 }
}

```

## 21.3 Öbek Uyumlu Bellek Dilimi

Öbek Uyumlu Bellek Dilimi `Memory<T>`, `Span<T>`'nin öbek uyumlu kardeşidir. `Span<T>`'nin tüm sıfır-tahsis avantajlarını sunar; ancak `ref struct` kısıtlaması yoktur. Sınıf alanı olabilir, `async` yöntemlerde kullanılabilir ve `lambda` ifadelerine geçilebilir.

### §21.3.1 `Span<T>` ve `Memory<T>` Karşılaştırması

| Özellik                                   | <code>Span&lt;T&gt;</code> | <code>Memory&lt;T&gt;</code> |
|-------------------------------------------|----------------------------|------------------------------|
| Yapı türü                                 | ref struct                 | struct                       |
| Sınıf alanı olabilir mi?                  | Hayır                      | Evet                         |
| <code>async</code> içinde çalışır mı?     | Hayır                      | Evet                         |
| Yığın tahsisi ( <code>stackalloc</code> ) | Evet                       | Hayır                        |
| <code>Span&lt;T&gt;</code> 'e dönüşüm     | —                          | <code>.Span</code>           |
| Performans                                | Biraz daha hızlı           | Çok yakın                    |

Tablo 21.3: `Span<T>` ve `Memory<T>` Karşılaştırması

### §21.3.2 Temel Kullanım

```

using System;
using System.Threading.Tasks;

class MemoryÖrneği
{
 // Memory sınıf alanı olabilir
 private Memory<byte> _tampon;

 public MemoryÖrneği(int boyut)
 {
 _tampon = new byte[boyut];
 }

 // async yöntemde kullanılabilir
 public async Task DoldurAsync(byte değer)
 {
 await Task.Delay(10); // simüle edilmiş async iş

 // .Span ile anlık Span<T>'ye dönüştür
 Span<byte> span = _tampon.Span;
 span.Fill(değer);
 }

 public void YazdırİlkN(int n)
 {
 ReadOnlySpan<byte> görünüm = _tampon.Span.Slice(0, n);
 foreach (byte b in görünüm)
 Console.Write($"{b} ");
 Console.WriteLine();
 }
}

class Program

```

```

{
 static async Task Main()
 {
 MemoryÖrneği örnek = new(16);
 await örnek.DoldurAsync(0xAB);
 örnek.YazdırİlkN(8);
 }
}
/*Program Çıktısı:
171 171 171 171 171 171 171 171
*/

```

### §21.3.3 Memory<T> ile Büyük Veri İşleme

Büyük bir dosyanın parçalar hâlinde (chunked) okunması, her parça için ayrı dizi tahsis etmeden yapılabilir.

```

using System;
using System.IO;
using System.Threading.Tasks;

class DosyaİşlemcisiMemory
{
 // Tek büyük tampon – döngü boyunca yeniden kullanılır
 private readonly Memory<byte> _tampon;

 public DosyaİşlemcisiMemory(int tamponBoyutu = 4096)
 {
 _tampon = new byte[tamponBoyutu];
 }

 public async Task İşleAsync(string dosyaYolu)
 {
 await using FileStream akış = File.OpenRead(dosyaYolu);

 int okunanBayt;
 long toplamBayt = 0;

 // Her döngüde aynı tampon yeniden kullanılır – tahsis yok
 while ((okunanBayt = await akış.ReadAsync(_tampon)) > 0)
 {
 // Yalnızca okunan kısmı işle
 ReadOnlyMemory<byte> parça = _tampon.Slice(0, okunanBayt);
 toplamBayt += okunanBayt;

 // Parça üzerinde işlem yap
 İşleParça(parça.Span);
 }

 Console.WriteLine($"Toplam işlenen: {toplamBayt} bayt");
 }

 private void İşleParça(ReadOnlySpan<byte> parça)
 {
 // Örnek: baytları say
 Console.WriteLine($"Parça boyutu: {parça.Length} bayt");
 }
}

```

## 21.4 Dizi Havuzu ile Tahsis Optimizasyonu

Her istek için yeni dizi oluşturmak çöp toplayıcıya (GC) ciddi baskı uygular. `ArrayPool<T>`, dizileri havuzda (pool) tutarak yeniden kullanımı sağlar. Tahsis yerine *ödünç alma* (rent) ve *iade* (return) vardır.

### §21.4.1 Temel Kullanım

```
using System;
using System Buffers;

class ArrayPoolTemelÖrneği
{
 static void Main()
 {
 // Paylaşılan (shared) havuzdan ödünç al
 int[] dizi = ArrayPool<int>.Shared.Rent(1000);

 try
 {
 // Dikkat: Rent istenen boyuttan BÜYÜK dizi verebilir
 Console.WriteLine($"İstenen: 1000, Alınan: {dizi.Length}");

 // Diziyi kullan
 for (int i = 0; i < 1000; i++)
 dizi[i] = i * 2;

 int toplam = 0;
 for (int i = 0; i < 1000; i++)
 toplam += dizi[i];

 Console.WriteLine($"Toplam: {toplam}");
 }
 finally
 {
 // MUTLAKA iade et – aksi hâlde havuz tükenir
 // clearArray: true – güvenlik için sıfırla (hassas veri varsa)
 ArrayPool<int>.Shared.Return(dizi, clearArray: false);
 }
 }
}

/*Program Çıktısı:
İstenen: 1000, Alınan: 1024
Toplam: 999000
*/
```

### §21.4.2 IDisposable Benzeri Sarmalayıcı ile Güvenli Kullanım

`finally` bloğunu unutmak riski taşır. Sarmalayıcı yapı bu riski ortadan kaldırır.

```
using System;
using System Buffers;

// IDisposable ile ArrayPool dizisini otomatik iade et
readonly struct KiralanmışDizi<T> : IDisposable
{
 private readonly T[] _dizi;
 private readonly int _boyut;

 public KiralanmışDizi(int boyut)
 {
 _dizi = ArrayPool<T>.Shared.Rent(boyut);
 _boyut = boyut;
 }

 // Yalnızca istenen kadar göster
 public Span<T> Span => _dizi.AsSpan(0, _boyut);

 public void Dispose()
 {

```



```

 ArrayPool<T>.Shared.Return(_dizi);
 }
}

class Program
{
 static void Veriİşle(int boyut)
 {
 // using bloğu – kapsam sonunda otomatik iade
 using KiralanmışDizi<double> kira = new(boyut);

 Span<double> veri = kira.Span;
 for (int i = 0; i < veri.Length; i++)
 veri[i] = Math.Sqrt(i);

 Console.WriteLine($"{veri.Length - 1} = {veri[^1]:F4}");
 }

 static void Main()
 {
 Veriİşle(10000);
 // Yöntem sonunda dizi havuza otomatik iade edildi
 }
}
/*Program Çıktısı:
9999 = 99.9950
*/

```

### §21.4.3 Özel Havuz Oluşturma

Varsayılan paylaşılan havuz çoğu senaryo için yeterlidir. Ancak özel gereksinimler için özel havuz oluşturulabilir.

```

using System;
using System.Buffers;

class ÖzelHavuzÖrneği
{
 // Özel havuz: maksimum dizi boyutu ve havuz kapasitesi ayarlanır
 private static readonly ArrayPool<byte> _özelHavuz =
 ArrayPool<byte>.Create(
 maxArrayLength: 1024 * 1024, // Maks 1 MB dizi
 maxArraysPerBucket: 50); // Her boyut kümesinde 50 dizi

 public static void ÖzelHavuzKullan()
 {
 byte[] tampon = _özelHavuz.Rent(512 * 1024); // 512 KB ödünç al
 try
 {
 tampon.AsSpan().Fill(0xFF);
 Console.WriteLine($"Özel havuzdan alınan boyut: {tampon.Length}");
 }
 finally
 {
 _özelHavuz.Return(tampon);
 }
 }
}

```

### §21.4.4 Performans Karşılaştırması

Aşağıdaki iki yöntem işlevsel olarak aynıdır; ancak bellek tahsis davranışları çok farklıdır.

```

using System;
using System.Buffers;

```

```

class PerformansKarşılaştırma
{
 const int İterasyon = 100_000;
 const int DiziBoyutu = 1024;

 // KÖTÜ: Her çağrıda yeni dizi – 100.000 × 1024 bayt öbek tahsis
 static void KlasikYöntem()
 {
 for (int i = 0; i < İterasyon; i++)
 {
 byte[] tampon = new byte[DiziBoyutu]; // her seferinde tahsis
 tampon.AsSpan().Fill(0xAA);
 }
 }

 // İYİ: Havuzdan ödünç al – yeniden kullanım, GC baskısı minimum
 static void HavuzluYöntem()
 {
 for (int i = 0; i < İterasyon; i++)
 {
 byte[] tampon = ArrayPool<byte>.Shared.Rent(DiziBoyutu);
 try
 {
 tampon.AsSpan(0, DiziBoyutu).Fill(0xAA);
 }
 finally
 {
 ArrayPool<byte>.Shared.Return(tampon);
 }
 }
 }
}

```

| Ölçüm                               | Klasik new[] | ArrayPool       |
|-------------------------------------|--------------|-----------------|
| Toplam tahsis                       | ~100 MB      | ~1 KB           |
| Çöp toplayıcıdaki koleksiyon sayısı | Yüksek       | Neredeyse sıfır |
| Ortalama süre                       | Daha yavaş   | Belirgin hızlı  |

Tablo 21.4: Klasik Dizi ve ArrayPool Karşılaştırması (100.000 iterasyon, 1 KB dizi)

## 21.5 Yönetilmeyen Kaynakların Serbest Bırakılması

.NET çöp toplayıcı (GC), yalnızca *yönetilen* (**managed**) belleği temizler. Dosya tutamaçları (**file handle**), ağ soketleri, veri tabanı bağlantıları ve grafik kaynakları gibi *yönetilmeyen* (**unmanaged**) kaynaklar çöp toplayıcı tarafından otomatik serbest bırakılmaz. Bu kaynakları zamanında temizlemek programcının sorumluluğundadır. .NET bu amaç için **IDisposable** arayüzünü sunar.

### §21.5.1 Temel IDisposable Uygulaması

```

using System;

class DosyaYazıcısı : IDisposable
{
 private System.IO.StreamWriter? _yazıcı;
 private bool _temizlendi = false; // Çift temizlemeyi önle

 public DosyaYazıcısı(string dosyaYolu)
 {
 _yazıcı = new System.IO.StreamWriter(dosyaYolu);
 Console.WriteLine($"Dosya açıldı: {dosyaYolu}");
 }
}

```

```

public void Yaz(string metin)
{
 if (!_temizlendi)
 throw new ObjectDisposedException(nameof(DosyaYazıcısı));
 _yazıcı!.WriteLine(metin);
}

public void Dispose()
{
 if (!_temizlendi)
 {
 _yazıcı?.Flush();
 _yazıcı?.Close();
 _yazıcı = null;
 _temizlendi = true;
 Console.WriteLine("Dosya kapatıldı ve kaynaklar serbest bırakıldı.");
 }
 // GC'ye bu nesneyi sonlandırıcı kuyruğuna alma demek
 GC.SuppressFinalize(this);
}
}

class Program
{
 static void Main()
 {
 // using bloğu – kapsam sonunda Dispose otomatik çağrılır
 using (DosyaYazıcısı yazıcı = new("çıkıttı.txt"))
 {
 yazıcı.Yaz("Birinci satır");
 yazıcı.Yaz("İkinci satır");
 } // Burada Dispose() otomatik çağrılır

 Console.WriteLine("Program devam ediyor...");
 }
}
/*Program Çıktısı:
Dosya açıldı: çıkıttı.txt
Dosya kapatıldı ve kaynaklar serbest bırakıldı.
Program devam ediyor...
*/

```

### §21.5.2 Hem Yönetilen Hem Yönetilmeyen Kaynaklar

Bir sınıf hem yönetilen hem yönetilmeyen kaynak tutuyorsa **kullan-at deseni** (**disposable pattern**) uygulanmalıdır. Bu desen sonlandırıcı (**finalizer**) ile birlikte çalışır.

```

using System;
using System.Runtime.InteropServices;

class KarmaşıkKaynak : IDisposable
{
 // Yönetilmeyen kaynak (örnek: yerel kütüphane tutamacı)
 private IntPtr _yönetilmeyenKaynak;

 // Yönetilen kaynak
 private System.IO.MemoryStream? _akış;

 private bool _temizlendi = false;

 public KarmaşıkKaynak()
 {
 // Yönetilmeyen bellek tahsis et (örnek amaçlı)
 }
}

```

```

 _yönetilmeyenKaynak = Marshal.AllocHGlobal(1024);
 _akış = new System.IO.MemoryStream();
 Console.WriteLine("Kaynaklar tahsis edildi.");
 }

 // IDisposable.Dispose - kullanıcı tarafından çağrılır
 public void Dispose()
 {
 Temizle(yönetilenKaynaklarıTemizle: true);
 GC.SuppressFinalize(this); // Sonlandırıcıyı iptal et
 }

 // Sonlandırıcı - GC tarafından çağrılır (yalnızca yönetilmeyen kaynaklar için)
 ~KarmaşıkKaynak()
 {
 Temizle(yönetilenKaynaklarıTemizle: false);
 }

 // Asıl temizleme mantığı
 protected virtual void Temizle(bool yönetilenKaynaklarıTemizle)
 {
 if (!_temizlendi)
 {
 if (yönetilenKaynaklarıTemizle)
 {
 // Yönetilen kaynakları temizle
 _akış?.Dispose();
 _akış = null;
 Console.WriteLine("Yönetilen kaynaklar temizlendi.");
 }

 // Yönetilmeyen kaynakları her durumda temizle
 if (_yönetilmeyenKaynak != IntPtr.Zero)
 {
 Marshal.FreeHGlobal(_yönetilmeyenKaynak);
 _yönetilmeyenKaynak = IntPtr.Zero;
 Console.WriteLine("Yönetilmeyen bellek serbest bırakıldı.");
 }

 _temizlendi = true;
 }
 }
}

```

### §21.5.3 Modern using Bildirimi

C#8.0+ ile gelen `using` bildirimi, parantez ve blok yazmayı ortadan kaldırır. Kaynak, bulunduğu kapsamın sonunda otomatik temizlenir.

```

using System;
using System.IO;

class ModernUsingÖrneği
{
 static void Main()
 {
 // Klasik using bloğu
 using (StreamReader klasik = new("veri.txt"))
 {
 string? satır = klasik.ReadLine();
 Console.WriteLine(satır);
 }
 }
}

```

```

// Modern using bildirimi (C# 8.0+) – daha az girintileme
using StreamReader modern = new("veri.txt");
string? satır2 = modern.ReadLine();
Console.WriteLine(satır2);
// Yöntem sonu veya kapsam sonu geldiğinde Dispose çağrılır

// Birden fazla kaynak
using StreamReader okuyucu = new("kaynak.txt");
using StreamWriter yazıcı = new("hedef.txt");
string? içerik;
while ((içerik = okuyucu.ReadLine()) != null)
 yazıcı.WriteLine(içerik.ToUpper());
}
}

```

### §21.5.4 Hata Durumunda Kaynak Güvenliği

using bloğu içinde istisna fırlatılsa bile Dispose her zaman çağrılır.

```

using System;
using System.IO;

class HataGüvenliğiÖrneği
{
 static void Main()
 {
 try
 {
 using StreamWriter yazıcı = new("çıktı.txt");
 yazıcı.WriteLine("Birinci satır");

 throw new InvalidOperationException("Bir hata oluştu!"); // Hata fırlat

 yazıcı.WriteLine("Bu satır yazılmaz."); // Erişilmez kod
 }
 catch (InvalidOperationException hata)
 {
 Console.WriteLine($"Hata yakalandı: {hata.Message}");
 }
 // İstisna fırlatılmış olsa da yazıcı.Dispose() çağrıldı
 Console.WriteLine("Dosya güvenli kapatıldı.");
 }
}

/*Program Çıktısı:
Hata yakalandı: Bir hata oluştu!
Dosya güvenli kapatıldı.
*/

```

## 21.6 Eş Zamansız Kaynak Temizleme

Bazı kaynakların temizlenmesi eş zamansız işlem gerektirir: ağ bağlantılarının kibarca kapatılması, tampon verilerin diske aktarılması, uzak oturumların sonlandırılması. C#8.0 ile gelen `IAsyncDisposable` ve `await using` bu senaryolar için tasarlanmıştır.

### §21.6.1 Temel Uygulama

```

using System;
using System.Threading.Tasks;

class EşZamansızBağlantı : IAsyncDisposable
{
 private bool _açık = false;

 public async Task BağlanAsync(string sunucu)
 {

```

```

 await Task.Delay(100); // Bağlantı kurma simülasyonu
 _açık = true;
 Console.WriteLine($"Bağlantı kuruldu: {sunucu}");
 }

 public async Task VeriGönderAsync(string veri)
 {
 if (!_açık)
 throw new InvalidOperationException("Bağlantı kapalı.");

 await Task.Delay(50); // Ağ gecikmesi
 Console.WriteLine($"Gönderildi: {veri}");
 }

 // Eş zamansız temizleme
 public async ValueTask DisposeAsync()
 {
 if (_açık)
 {
 // Bekleyen verileri temizle ve bağlantıyı kapat
 await Task.Delay(80); // Kapatma handshake
 _açık = false;
 Console.WriteLine("Bağlantı kibarca kapatıldı.");
 }
 GC.SuppressFinalize(this);
 }
}

class Program
{
 static async Task Main()
 {
 // await using – kapsam sonunda DisposeAsync otomatik çağrılır
 await using EşZamansızBağlantı bağlantı = new();

 await bağlantı.BağlanAsync("sunucu.example.com");
 await bağlantı.VerigönderAsync("Merhaba Sunucu");
 await bağlantı.VerigönderAsync("İkinci paket");
 } // Burada await DisposeAsync() otomatik çağrılır
}

/*Program Çıktısı:
Bağlantı kuruldu: sunucu.example.com
Gönderildi: Merhaba Sunucu
Gönderildi: İkinci paket
Bağlantı kibarca kapatıldı.
*/

```

### §21.6.2 Hem IDisposable Hem IAsyncDisposable Uygulama

Bir sınıf her iki arayüzü de uygulayabilir. Eş zamansız bağlamda `await using`, eş zamanlı bağlamda `using` çalışır.

```

using System;
using System.IO;
using System.Threading.Tasks;

class EsnekKaynak : IDisposable, IAsyncDisposable
{
 private FileStream? _akış;

 public EsnekKaynak(string yol)
 {
 _akış = new FileStream(yol,

```

```

 FileMode.OpenOrCreate, FileAccess.Write,
 FileShare.None, bufferSize: 4096,
 useAsync: true); // Eş zamansız G/Ç etkin
 }

 public async Task YazAsync(byte[] veri)
 {
 if (_akış is null)
 throw new ObjectDisposedException(nameof(EsnekKaynak));
 await _akış.WriteAsync(veri);
 }

 // Eş zamansız temizleme (tercih edilir)
 public async ValueTask DisposeAsync()
 {
 if (_akış is not null)
 {
 await _akış.FlushAsync(); // Tamponu diske yaz
 await _akış.DisposeAsync();
 _akış = null;
 }
 GC.SuppressFinalize(this);
 }

 // Eş zamanlı temizleme (yedek)
 public void Dispose()
 {
 _akış?.Flush();
 _akış?.Dispose();
 _akış = null;
 GC.SuppressFinalize(this);
 }
}

class Program
{
 static async Task Main()
 {
 // Eş zamansız bağlamda
 await using (EsnekKaynak kaynakAsync = new("çıkkt1.bin"))
 {
 await kaynakAsync.YazAsync(new byte[] { 1, 2, 3, 4 });
 }

 // Eş zamanlı bağlamda
 using (EsnekKaynak kaynakSync = new("çıkkt12.bin"))
 {
 // Eş zamanlı kullanım
 }
 }
}

```

### §21.6.3 IDisposable ve IAsyncDisposable Karşılaştırması

| Özellik                                    | IDisposable          | IAsyncDisposable                    |
|--------------------------------------------|----------------------|-------------------------------------|
| Anahtar kelime                             | using                | await using                         |
| Temizleme yöntemi                          | Dispose()            | DisposeAsync()                      |
| Dönüş tipi                                 | void                 | ValueTask                           |
| Eş zamansız giriş-çıkış (IO) destekler mi? | Hayır                | Evet                                |
| Ne zaman tercih edilir?                    | Eş zamanlı kaynaklar | Ağ, dosya, veri tabanı bağlantıları |

Tablo 21.5: IDisposable ve IAsyncDisposable Karşılaştırması



## 21.7 Bellek Sızıntısı Tespiti ve Profilleme

.NET'in çöp toplaması çoğu bellek sorununu otomatik çözer; ancak bazı yaygın hatalar sızıntıya (**memory leak**) yol açar. Bu bölümde sızıntı kalıpları ve tespit yöntemleri incelenmektedir.

### §21.7.1 Yaygın Bellek Sızıntısı Kalıpları

#### *Olay İşleyicisi Sızıntısı*

En yaygın sızıntı kaynağı abonelikten çıkılmayan olay işleyicileridir. İşleyiciyi tutan nesne canlı kaldığı sürece abone olan nesne de çöp toplayıcı (GC) tarafından toplanamaz.

```
using System;

class YayıncıSınıf
{
 public event EventHandler? Olay;

 public void OlayıTetikle() => Olay?.Invoke(this, EventArgs.Empty);
}

class AboneSınıf
{
 private readonly YayıncıSınıf _yayıncı;

 public AboneSınıf(YayıncıSınıf yayıncı)
 {
 _yayıncı = yayıncı;
 // SORUNLU: Bu abone nesne Dispose edilse bile
 // yayıncı bu nesneyi referans tutmaya devam eder
 _yayıncı.Olay += Olayİşleyici;
 }

 private void Olayİşleyici(object? gönderen, EventArgs e)
 {
 Console.WriteLine("Olay alındı.");
 }

 // ÇÖZÜM: IDisposable uygula ve abonelikten çık
 public void Dispose()
 {
 _yayıncı.Olay -= Olayİşleyici; // Abonelikten çık!
 }
}
```

#### *Statik Koleksiyonda Nesne Biriktirme*

```
using System;
using System.Collections.Generic;

class SızıntıÖrneği
{
 // SORUNLU: Statik liste hiç temizlenmez
 private static readonly List<byte[]> _önbellek = new();

 public static void VeriEkle(int boyut)
 {
 // Her çağrıda listeye eklenir, hiç çıkarılmaz
 _önbellek.Add(new byte[boyut]);
 }

 // ÇÖZÜM: Zaman sınırlı önbellek veya WeakReference kullan
 private static readonly List<WeakReference<byte[]>> _zayıfÖnbellek = new();

 public static void ZayıfVeriEkle(byte[] veri)
```

```

{
 // GC gerektiğinde bu nesneleri toplayabilir
 _zayıfÖnbellek.Add(new WeakReference<byte[]>(veri));
}
}

```

### Nesneleri Temizlememe

IDisposable ile nesneleri temizleme yöntemine aşağıda örnek verilmiştir;

```

using System;
using System.Net.Http;

class HttpSızıntısı
{
 // SORUNLU: Her çağrıda yeni HttpClient – soket tüketimi
 public static async System.Threading.Tasks.Task<string> KötüYöntemAsync()
 {
 HttpClient istemci = new(); // using yok!
 return await istemci.GetStringAsync("https://example.com");
 // istemci.Dispose() hiç çağrılmaz – soket açık kalır
 }

 // ÇÖZÜM 1: using ile temizle
 public static async System.Threading.Tasks.Task<string> İyiYöntemAsync()
 {
 using HttpClient istemci = new();
 return await istemci.GetStringAsync("https://example.com");
 }

 // ÇÖZÜM 2 (tercih): IHttpClientFactory ile ömür yönetimi
 // DI konteyneri HttpClient ömrünü otomatik yönetir
}

```

### §21.7.2 Bellek Kullanımını İzleme

```

using System;

class Bellekİzleme
{
 static void Main()
 {
 // Anlık bellek kullanımı
 long başlangıç = GC.GetTotalMemory(forceFullCollection: false);
 Console.WriteLine($"Başlangıç: {başlangıç / 1024:N0} KB");

 // Büyük veri tahsis et
 byte[][] veriler = new byte[100][];
 for (int i = 0; i < 100; i++)
 veriler[i] = new byte[1024 * 1024]; // 100 × 1 MB

 long sonra = GC.GetTotalMemory(forceFullCollection: false);
 Console.WriteLine($"Tahsis sonrası: {sonra / 1024:N0} KB");
 Console.WriteLine($"Fark: {(sonra - başlangıç) / 1024:N0} KB");

 // Referansı bırak
 veriler = null!;
 long temizlik = GC.GetTotalMemory(forceFullCollection: true);
 Console.WriteLine($"GC sonrası: {temizlik / 1024:N0} KB");

 // Kuşak bilgisi
 Console.WriteLine($"Kuşak 0 sayısı: {GC.CollectionCount(0)}");
 Console.WriteLine($"Kuşak 1 sayısı: {GC.CollectionCount(1)}");
 Console.WriteLine($"Kuşak 2 sayısı: {GC.CollectionCount(2)}");
 }
}

```

```

 }
}

```

### §21.7.3 Profilleme Araçları

Bellek sorunlarını tespit etmek için çeşitli araçlar kullanılabilir.

| Araç                           | Tür         | Kullanım Amacı                             |
|--------------------------------|-------------|--------------------------------------------|
| dotnet-counters                | CLI         | Anlık çöp toplama ve bellek metrikleri     |
| dotnet-dump                    | CLI         | Bellek anlık görüntüsü ( <b>snapshot</b> ) |
| dotnet-trace                   | CLI         | Olay izleme ve çöp toplama profili         |
| Visual Studio Diagnostic Tools | IDE         | Anlık görüntü karşılaştırma                |
| JetBrains dotMemory            | Ticari      | Sızıntı analizi, nesne ağacı               |
| BenchmarkDotNet                | Açık kaynak | Tahsis miktarı ölçümü                      |

Tablo 21.6: .NET Bellek Profilleme Araçları

```

// dotnet-counters ile izleme (komut satırı)
// dotnet-counters monitor --process-id <PID> System.Runtime

// BenchmarkDotNet ile tahsis ölçümü
using BenchmarkDotNet.Attributes;
using BenchmarkDotNet.Running;

[MemoryDiagnoser] // Tahsis miktarını ölç
public class TahsisBenchmark
{
 [Benchmark]
 public string KlasikBirleştir()
 {
 string sonuç = "";
 for (int i = 0; i < 100; i++)
 sonuç += i.ToString(); // Her iterasyonda yeni string
 return sonuç;
 }

 [Benchmark]
 public string SpanBirleştir()
 {
 return string.Create(292, 0, (span, _) =>
 {
 int konum = 0;
 for (int i = 0; i < 100; i++)
 {
 i.TryFormat(span.Slice(konum), out int yazılan);
 konum += yazılan;
 }
 });
 }
}

// BenchmarkRunner.Run<TahsisBenchmark>();

```

### §21.7.4 Bellek Yönetimi En İyi Uygulamaları

## 21.8 Düşün ve Çöz

- ◇ **Düşün:** `Span<T>` neden `async` yöntemlerde kullanılamaz? Derleyicinin `async` yöntemi durum makinesine dönüştürme süreciyle nasıl ilişkilidir? Bu kısıtlamayı aşmak için hangi yapıyı kullanırsınız?
- ◇ **Düşün:** Aşağıdaki kodda bellek sızıntısı var mı? Varsa nereden kaynaklanmaktadır ve nasıl düzeltilir?

```

class Uygulama
{
 private static List<Action> _eylemler = new();
}

```

| Uygulama                                                         | Neden?                                                        |
|------------------------------------------------------------------|---------------------------------------------------------------|
| <code>IDisposable</code> nesneleri <code>using</code> ile kullan | Kaynak sızıntısını önler                                      |
| Eş zamansız bağlamda <code>await using</code> kullan             | Kapatma işlemi bloke etmez                                    |
| Olay aboneliğinden <code>Dispose</code> 'da çık                  | Nesne sızıntısını önler                                       |
| Büyük diziler için <code>ArrayPool</code> kullan                 | Çöp toplama baskısını azaltır                                 |
| Dilim işlemleri için <code>Span&lt;T&gt;</code> kullan           | Kopyalamayı ortadan kaldırır                                  |
| <code>async</code> bağlamda <code>Memory&lt;T&gt;</code> kullan  | <code>Span&lt;T&gt;</code> <code>async</code> içinde çalışmaz |
| <code>stackalloc</code> boyutunu küçük tut                       | Yığın taşmasını ( <code>StackOverflow</code> ) önler          |
| Sonlandırıcı ( <code>finalizer</code> ) yazmaktan kaçın          | Çöp toplayıcı nesneyi iki kez işler, yavaşlatır               |

Tablo 21.7: Bellek Yönetimi En İyi Uygulamaları

```
static void KaydetEylem(string ad)
{
 string yerelAd = ad; // yerel değişken
 _eylemler.Add(() => Console.WriteLine(yerelAd));
}
```

- ◇ **Çöz:** Bir CSV (**Comma Seperated Values**) dosyasını satır satır okuyan ve her satırdaki alanları virgüle göre ayıran bir yöntem yazınız. `string.Split` yerine `ReadOnlySpan<char>` ve `Slice` kullanarak öbek tahsisini sıfıra indirin. `BenchmarkDotNet` veya `GC.GetTotalMemory` ile iki versiyonun tahsis miktarını karşılaştırın.
- ◇ **Çöz:** Bir görüntü işleme uygulaması saniyede binlerce küçük bayt dizisi kullanmaktadır. `ArrayPool<byte>` kullanarak dizi havuzunu oluşturun. Havuzlu ve havuzsuz versiyonu çalıştırarak `GC.CollectionCount` değerlerini karşılaştırın.
- ◇ **Düşün:** Tam **Dispose deseni** ne zaman gereklidir? Yalnızca yönetilen kaynaklar tutan bir sınıf için sonlandırıcı (`finalizer`) yazmak neden gereksiz ve zararlıdır?
- ◇ **Çöz:** Bir veri tabanı bağlantısı sınıfı tasarlayınız. Sınıf hem `IDisposable` hem `IAsyncDisposable` uygulansın. `Dispose` çağrıldığında bağlantıyı eş zamanlı, `DisposeAsync` çağrıldığında eş zamansız kapatsın. Çift temizlemeyi `_temizlendi` bayrağıyla önleyin.
- ◇ **Düşün:** `WeakReference<T>` ile normal referans arasındaki fark nedir? Büyük resim önbelleği (**image cache**) tasarlarlarken `WeakReference<T>` kullanmak ne gibi avantajlar sağlar? Ne gibi riskler doğurur?

## 22. Bölüm

# Birim Testi ve Test GÜdümlü Geliştirme

Yazılım geliştirmenin en kritik ama çoğu zaman ihmal edilen boyutu **test**tir. “Kod çalışıyor” demek, kodun *doğru* çalıştığı anlamına gelmez. Kurumsal yazılımlarda bir geliştirici yazdığı kodu aylar sonra tekrar ele alır, ekip arkadaşı aynı modülü değiştirir veya yeni bir gereksinim nedeniyle mevcut davranış güncellenir. Bu değişikliklerin sistemi bozmadığını güvence altına alan tek mekanizma **test**tir.

**Birim testi** (unit test), bir yazılımın en küçük bağımsız parçasını — bir yöntemi, bir sınıfı — izole biçimde doğrular. Birim testleri milisaniyeler içinde çalışır, sürekli entegrasyon (CI) boru hatlarında otomatik yürütülür ve geliştiriciyi anında bilgilendirir.

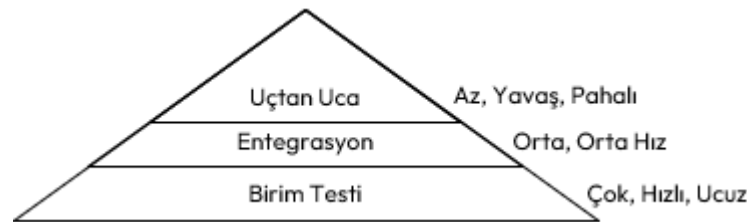
Bölümün sonunda aşağıdaki soruları rahatlıkla yanıtlayabileceksiniz:

- ◇ Birim, entegrasyon ve uçtan uca test arasındaki fark nedir?
- ◇ xUnit.net ile nasıl test yazılır, nasıl çalıştırılır?
- ◇ NUnit’in xUnit’ten farkı nedir, hangi durumda hangisi tercih edilir?
- ◇ Moq ile bağımlılıklar nasıl taklit edilir (mock)?
- ◇ Test GÜdümlü Geliştirme (**Test Driven Development**) döngüsü nasıl işler?
- ◇ Temiz ve bakımı kolay test nasıl yazılır?
- ◇ Eş zamansız kodlar nasıl test edilir?

## 22.1 Test Piramidi ve Test Türleri

### §22.1.1 Test Piramidi

Modern yazılım geliştirmede testler üç katmana ayrılır. Bu yapı **test piramidi** olarak adlandırılır ve her katmanın farklı bir amacı, farklı bir maliyeti vardır.



Şekil 22.1: Test Piramidi

| Tür         | Kapsam              | Hız       | Adet | Örnek                 |
|-------------|---------------------|-----------|------|-----------------------|
| Birim       | Tek sınıf/yöntem    | Çok hızlı | Çok  | Hesapla() doğrular    |
| Entegrasyon | Birden fazla katman | Orta      | Orta | Servis + DB birlikte  |
| Uçtan Uca   | Tüm sistem          | Yavaş     | Az   | Tarayıcı ile tam akış |

Tablo 22.1: Test Türleri Karşılaştırması

### §22.1.2 İyi Bir Birim Testin Özellikleri — FIRST

Kaliteli birim testleri **FIRST** ilkesine uyar:

- ◇ **Fast** (Hızlı): Milisaniyeler içinde biter. Yüzlerce test saniyeler içinde çalışır.
- ◇ **Isolated** (İzole): Başka testlere, ağa, dosyaya veya veri tabanına bağımlı değildir.
- ◇ **Repeatable** (Tekrarlanabilir): Her çalıştırmada aynı sonucu verir.
- ◇ **Self-validating** (Kendini Doğrulayan): Geçti/kaldı sonucunu otomatik üretir, insan yorumuna ihtiyaç duymaz.
- ◇ **Timely** (Zamanında): Üretim kodu yazılmadan önce veya hemen ardından yazılır.

## 22.2 xUnit.net ile Birim Testi

**xUnit.net**, .NET ekosisteminin en yaygın kullanılan modern test çerçevesidir. Microsoft’un kendi projeleri, ASP.NET Core ve Entity Framework Core da xUnit ile test edilmektedir.

### §22.2.1 Proje Kurulumu

Test projesi oluşturmak için komut satırında şu komutlar çalıştırılır:

```
// Yeni bir xUnit test projesi oluştur
// dotnet new xunit -n HesapMakinesi.Testler

// Ana projeye referans ekle
// dotnet add HesapMakinesi.Testler reference HesapMakinesi

// NuGet paketleri (xUnit projesiyle otomatik gelir):
// xunit - test çerçevesi
// xunit.runner.visualstudio - Visual Studio entegrasyonu
// Microsoft.NET.Test.Sdk - .NET test SDK'sı
```

### §22.2.2 Test Edilecek Sınıf

Önce test yazılacak üretim kodunu tanımlayalım. Bu bölüm boyunca bu sınıfı kullanacağız.

```
// HesapMakinesi.cs - Ana proje
namespace HesapMakinesi;

public class HesapMakinesi
{
 public double Topla(double a, double b) => a + b;

 public double Çıkar(double a, double b) => a - b;

 public double Çarp(double a, double b) => a * b;

 public double Böl(double a, double b)
 {
 if (b == 0)
 throw new DivideByZeroException("Sıfıra bölme hatası.");
 return a / b;
 }

 public bool AsalMı(int sayı)
 {
 if (sayı < 2) return false;
 for (int i = 2; i <= Math.Sqrt(sayı); i++)
 if (sayı % i == 0) return false;
 return true;
 }
}
```

### §22.2.3 İlk xUnit Testi — [Fact]

[Fact], parametre almayan, tek bir senaryoyu doğrulayan test yöntemini işaretler.

```
// HesapMakinesiTestleri.cs - Test projesi
using Xunit;

namespace HesapMakinesi.Testler;

public class HesapMakinesiTestleri
{
 // Test sınıfı her test için yeniden örnek oluşturur (xUnit davranışı)
 private readonly HesapMakinesi _hesap = new();

 [Fact]
 public void Topla_İkiPositifSayı_DoğruSonuçDöndürür()
 {
 // Arrange (Hazırla) - test girdileri ve beklenen değer
 double a = 3;
 double b = 4;
```

```

 double beklenen = 7;

 // Act (Uygula) – test edilecek y ntemi  aęır
 double ger ek = _hesap.Topla(a, b);

 // Assert (Doęrula) – beklenen ile ger eęi karřılařtır
 Assert.Equal(beklenen, ger ek);
}

[Fact]
public void Topla_NegatifSayılar_DoęruSonu D nd r r()
{
 double sonu  = _hesap.Topla(-5, -3);
 Assert.Equal(-8, sonu );
}

[Fact]
public void B l_SıfıraB lme_İstisnaD nd r r()
{
 // İstisna fırlatılmasını doęrula
 Assert.Throws<DivideByZeroException>(() =>
 _hesap.B l(10, 0));
}

[Fact]
public void B l_NormalB lme_DoęruSonu D nd r r()
{
 double sonu  = _hesap.B l(10, 4);
 // Kayan nokta karřılařtırmada hassasiyet belirt
 Assert.Equal(2.5, sonu , precision: 5);
}
}

```

###  22.2.4 Test İsimlendirme Kuralı

Test adı    b l mden oluřmalıdır:

[Y ntem]\_[Senaryo]\_[BeklenenSonu ]

| K t  İsim      | İyi İsim                                  |
|----------------|-------------------------------------------|
| Test1()        | Topla_İkiPozitif_DoęruToplamar()          |
| B lmeTest()    | B l_SıfırPaydayla_DivideByZeroFırlatır()  |
| Ařır Y kTest() | AsalM _Bilin AřaęısıSay _YanlıřD nd r r() |

Tablo 22.2: Test İsimlendirme  rnekleri

###  22.2.5 Parametrel  Test — [Theory] ve [InlineData]

Aynı mantıęı farklı girdilerle doęrulamak i in [Theory] ve [InlineData] kullanılır. Bu yapı test tekrarını ortadan kaldırır.

```

public class HesapMakinesiTheoryTestleri
{
 private readonly HesapMakinesi _hesap = new();

 // [Theory] – aynı test, farklı girdilerle  alıřır
 [Theory]
 [InlineData(3, 4, 7)] // 3 + 4 = 7
 [InlineData(-1, -2, -3)] // -1 + -2 = -3
 [InlineData(0, 0, 0)] // 0 + 0 = 0
 [InlineData(100, -50, 50)] // 100 + (-50) = 50
 public void Topla_ eřitliGirdiler_DoęruSonu D nd r r(
 double a, double b, double beklenen)
 {
 double ger ek = _hesap.Topla(a, b);
 }
}

```



```

 Assert.Equal(beklenen, gerçek);
 }

 [Theory]
 [InlineData(2, true)] // 2 asal
 [InlineData(3, true)] // 3 asal
 [InlineData(4, false)] // 4 asal değil
 [InlineData(17, true)] // 17 asal
 [InlineData(1, false)] // 1 asal değil
 [InlineData(0, false)] // 0 asal değil
 public void AsalMı_ÇeşitliSayılar_DoğruSonuçDöndürür(int sayı, bool beklenen)
 {
 bool gerçek = _hesap.AsalMı(sayı);
 Assert.Equal(beklenen, gerçek);
 }
}

```

### §22.2.6 [MemberData] ile Karmaşık Veri Setleri

Karmaşık nesneler veya büyük veri setleri için [MemberData] kullanılır.

```

public class BölmeTestleri
{
 private readonly HesapMakinesi _hesap = new();

 // Statik özellik olarak test verisi tanımla
 public static IEnumerable<object[]> BölmeVerisi =>
 new List<object[]>
 {
 new object[] { 10.0, 2.0, 5.0 },
 new object[] { 9.0, 3.0, 3.0 },
 new object[] { 7.0, 2.0, 3.5 },
 new object[] { -8.0, 4.0, -2.0 },
 };

 [Theory]
 [MemberData(nameof(BölmeVerisi))]
 public void Böl_GeçerliGirdiler_DoğruSonuçDöndürür(
 double bölünen, double bölen, double beklenen)
 {
 double gerçek = _hesap.Böl(bölünen, bölen);
 Assert.Equal(beklenen, gerçek, precision: 10);
 }
}

```

### §22.2.7 xUnit Doğrulama Yöntemleri

```

// Eşitlik doğrulama
Assert.Equal(beklenen, gerçek);
Assert.NotEqual(beklenmeyen, gerçek);

// Boolean doğrulama
Assert.True(koşul);
Assert.False(koşul);

// Null doğrulama
Assert.Null(nesne);
Assert.NotNull(nesne);

// İstisna doğrulama
Assert.Throws<ArgumentException>(() => Yöntem());
var hata = Assert.Throws<InvalidOperationException>(() => Yöntem());
Assert.Contains("beklenen mesaj", hata.Message);

```

```
// Koleksiyon doęrulama
Assert.Empty(koleksiyon);
Assert.NotEmpty(koleksiyon);
Assert.Single(koleksiyon);
Assert.Contains(eleman, koleksiyon);

// T r doęrulama
Assert.IsType<HesapMakinesi>(nesne);
Assert.IsAssignableFrom<IHesapMakinesi>(nesne);

// Yaklařık eřitlik (kayan nokta)
Assert.Equal(3.14, sonu, precision: 2); // 2 ondalık basamak
```

## ř22.2.8 Test Yařam D ng s  — Oluřturucu ve IDisposable

xUnit'te her test y ntemi iin sınıf yeniden oluřturulur. Paylařılan kaynaklar `IClassFixture` ile y netilir.

```
using Xunit;
using Xunit.Abstractions;

// ITestOutputHelper – test ıktısını yakala
public class YařamD ng s Testleri : IDisposable
{
 private readonly ITestOutputHelper _ıktı;
 private readonly HesapMakinesi _hesap;

 // Her test iin alıřır – kurulum
 public YařamD ng s Testleri(ITestOutputHelper ıktı)
 {
 _ıktı = ıktı;
 _hesap = new HesapMakinesi();
 _ıktı.WriteLine("Test bařladı.");
 }

 [Fact]
 public void Topla_ rnek_alıřıyor()
 {
 double sonu = _hesap.Topla(2, 3);
 _ıktı.WriteLine($"Sonu: {sonu}");
 Assert.Equal(5, sonu);
 }

 // Her test sonrası alıřır – temizlik
 public void Dispose()
 {
 _ıktı.WriteLine("Test bitti.");
 }
}

// Paylařılan pahalı kaynak iin Fixture
public class VeritabanıFixture : IDisposable
{
 public string BaęlantıSatırı { get; } = "Server=localhost;Database=Test";

 public VeritabanıFixture()
 {
 // Veri tabanı baęlantısını bir kez kur (t m testler iin)
 Console.WriteLine("Veri tabanı baęlantısı kuruldu.");
 }

 public void Dispose()
 {
 Console.WriteLine("Veri tabanı baęlantısı kapatıldı.");
 }
}
```

```

 }
}

// IClassFixture - tüm testler için fixture'ı paylaş
public class VeritabanıTestleri : IClassFixture<VeritabanıFixture>
{
 private readonly VeritabanıFixture _fixture;

 public VeritabanıTestleri(VeritabanıFixture fixture)
 {
 _fixture = fixture; // Tek örnek, tüm testler paylaşır
 }

 [Fact]
 public void BağlantıSatırı_Boş_Değil()
 {
 Assert.NotEmpty(_fixture.BağlantıSatırı);
 }
}

```

## 22.3 NUnit ile Birim Testi

**NUnit**, .NET dünyasının köklü test çerçevelerinden biridir. JUnit'ten esinlenerek tasarlanmıştır ve özellikle .NET Framework döneminden gelen projelerde yaygın kullanılmaktadır.

### §22.3.1 NUnit Kurulumu

```

// dotnet new nunit -n HesapMakinesi.NUnitTestler
// Paketler:
// NUnit
// NUnit3TestAdapter
// Microsoft.NET.Test.Sdk

```

### §22.3.2 Temel NUnit Testi

```

using NUnit.Framework;

namespace HesapMakinesi.NUnitTestler;

[TestFixture] // NUnit'te test sınıfını işaretler
public class HesapMakinesiNUnitTestleri
{
 private HesapMakinesi _hesap = null!;

 [SetUp] // Her testten önce çalışır (xUnit'te oluşturu)
 public void Kurulum()
 {
 _hesap = new HesapMakinesi();
 }

 [TearDown] // Her testten sonra çalışır (xUnit'te IDisposable)
 public void Temizlik()
 {
 // Kaynakları serbest bırak
 }

 [OneTimeSetUp] // Tüm testlerden önce bir kez (xUnit'te IClassFixture)
 public void SınıfKurulumu()
 {
 Console.WriteLine("Test sınıfı başlatıldı.");
 }

 [Test] // [Fact] karşılığı
 public void Topla_İkiPozitifSayı_DogruSonuç()

```

```

{
 double sonu = _hesap.Topla(3, 4);
 Assert.That(sonu, Is.EqualTo(7)); // NUnit'in akıcı s zdizimi
}

[Test]
public void B l_SıfıraB lme_İstisnaD nd r r()
{
 Assert.Throws<DivideByZeroException>(() => _hesap.B l(10, 0));
}

[TestCase(3, 4, 7)] // [InlineData] karřılıęı
[TestCase(-1, -2, -3)]
[TestCase(0, 0, 0)]
public void Topla_eřitliGirdiler_DoęruSonu(double a, double b, double beklenen)
{
 double gerek = _hesap.Topla(a, b);
 Assert.That(gerek, Is.EqualTo(beklenen));
}
}

```

### §22.3.3 NUnit Akıcı Doęrulama — Assert.That

NUnit'in `Assert.That` s zdizimi son derece okunabilirdir:

```

// Eřitlik
Assert.That(gerek, Is.EqualTo(beklenen));
Assert.That(gerek, Is.Not.EqualTo(beklenmeyen));

// B y kl k karřılařtırma
Assert.That(sayı, Is.GreaterThan(0));
Assert.That(sayı, Is.LessThanOrEqualTo(100));
Assert.That(sayı, Is.InRange(1, 10));

// Null kontrol 
Assert.That(nesne, Is.Null);
Assert.That(nesne, Is.Not.Null);

// String doęrulama
Assert.That(metin, Does.StartWith("Merhaba"));
Assert.That(metin, Does.Contain("d nya"));
Assert.That(metin, Does.Match(@"\d+")); // Regex

// Koleksiyon doęrulama
Assert.That(liste, Is.Empty);
Assert.That(liste, Has.Count.EqualTo(3));
Assert.That(liste, Does.Contain(42));
Assert.That(liste, Is.Ordered);
Assert.That(liste, Is.Unique);

// T r doęrulama
Assert.That(nesne, Is.InstanceOf<HesapMakinesi>());

```

### §22.3.4 xUnit ve NUnit Karřılařtırması

Karřılařtırma tablosu Tablo 22.3'de verilmiřtir.

## 22.4 Moq ile Baęımlılık Taklidi

Gerek d nya sınıfları nadiren tek bařına alıřır. Bir servis veri tabanına eriřir, bir API'ye istek atar veya dosya sistemini kullanır. Birim testlerinde bu *dıř baęımlılıklar* gerek sistemlere eriřmeden taklit edilmelidir. Bu iřleme **mocking** (taklitilik) denir.

**Moq**, .NET iin en yaygın kullanılan taklitilik k t phanesidir.

| Özellik                   | xUnit.net             | NUnit                 |
|---------------------------|-----------------------|-----------------------|
| Test işaretleyici         | [Fact]                | [Test]                |
| Parametreliliği           | [Theory]/[InlineData] | [TestCase]            |
| Sınıf işaretleyici        | Gerek yok             | [TestFixture]         |
| Her test öncesi kurulum   | Oluşturucu            | [SetUp]               |
| Her test sonrası temizlik | IDisposable           | [TearDown]            |
| Paylaşılan kaynak         | IClassFixture         | [OneTimeSetUp]        |
| Doğrulama stili           | Assert.Equal(...)     | Assert.That(Is...)    |
| .NET tavsiyesi            | Microsoft tercih eder | Köklü, geniş topluluk |

Tablo 22.3: xUnit.net ve NUnit Karşılaştırması

### §22.4.1 Taklit İçin Arayüz Tasarımı

Moq yalnızca arayüz veya soyut sınıf üzerinde çalışır. Bu da kodunuzu **gevşek bağlı** (**loosely coupled**) tasarlamaya zorlar — ki bu zaten iyi bir alışkanlıktır.

```
// Üretim kodu – arayüzler üzerinden bağımlılık
namespace Kütüphane;

// Kullanıcı veri erişim arayüzü
public interface IKullanıcıDepo
{
 Kullanıcı? BulById(int id);
 void Kaydet(Kullanıcı kullanıcı);
 bool EmailMevcutMu(string email);
}

// E-posta servis arayüzü
public interface IEpostaServisi
{
 void HoşgeldinGönder(string email, string ad);
}

// Kullanıcı modeli
public record Kullanıcı(int Id, string Ad, string Email, bool Aktif);

// Test edilecek servis – bağımlılıklar arayüz üzerinden enjekte edilir
public class KullanıcıServisi
{
 private readonly IKullanıcıDepo _depo;
 private readonly IEpostaServisi _epostaServisi;

 public KullanıcıServisi(IKullanıcıDepo depo, IEpostaServisi epostaServisi)
 {
 _depo = depo;
 _epostaServisi = epostaServisi;
 }

 public Kullanıcı? BulById(int id)
 {
 if (id <= 0)
 throw new ArgumentException("Id sıfırdan büyük olmalıdır.", nameof(id));
 return _depo.BulById(id);
 }

 public void KaydetVeHoşgeldinGönder(Kullanıcı kullanıcı)
 {
 if (_depo.EmailMevcutMu(kullanıcı.Email))
 throw new InvalidOperationException($"'{kullanıcı.Email}' zaten kayıtlı.");

 _depo.Kaydet(kullanıcı);
 _epostaServisi.HoşgeldinGönder(kullanıcı.Email, kullanıcı.Ad);
 }
}
```

```

 }
}

```

## §22.4.2 Moq Kurulumu ve Temel Kullanım

```

// dotnet add package Moq
using Moq;
using Xunit;

namespace K t phane.Testler;

public class KullanıcıServisTestleri
{
 // Mock nesneleri sınıf d zeyinde tanımla
 private readonly Mock<IKullanıcıDepo> _depoMock;
 private readonly Mock<IEpostaServisi> _epostaMock;
 private readonly KullanıcıServisi _servis;

 public KullanıcıServisTestleri()
 {
 _depoMock = new Mock<IKullanıcıDepo>();
 _epostaMock = new Mock<IEpostaServisi>();

 // Mock nesnelerin .Object  zelliđi ger ek nesne gibi davranır
 _servis = new KullanıcıServisi(
 _depoMock.Object,
 _epostaMock.Object);
 }

 [Fact]
 public void BulById_MevcutId_KullanıcıD nd r r()
 {
 // Arrange – Mock davranışını ayarla
 Kullanıcı beklenen = new(1, "İlhan", "ilhan@ornek.com", true);

 // BulById(1)  ađrıldığında beklenen kullanıcıyı d nd r
 _depoMock
 .Setup(d => d.BulById(1))
 .Returns(beklenen);

 // Act
 Kullanıcı? ger ek = _servis.BulById(1);

 // Assert
 Assert.NotNull(ger ek);
 Assert.Equal("İlhan", ger ek.Ad);
 Assert.Equal("ilhan@ornek.com", ger ek.Email);
 }

 [Fact]
 public void BulById_SıfırId_ArgumentExceptionFırlatır()
 {
 // Id <= 0 i in istisna fırlatılmalı
 Assert.Throws<ArgumentException>(() => _servis.BulById(0));

 // Depo hi   ađrılmamalı (zaten ulaşamaz)
 _depoMock.Verify(d => d.BulById(It.IsAny<int>()), Times.Never);
 }

 [Fact]
 public void BulById_MevcutOlmayanId_NullD nd r r()
 {
 // BulById(99)  ađrıldığında null d nd r
 }
}

```

```

 _depoMock
 .Setup(d => d.BulById(99))
 .Returns((Kullanıcı?)null);

 Kullanıcı? gerçek = _servis.BulById(99);

 Assert.Null(gerçek);
 }
}

```

### §22.4.3 Yöntem Çağrılarını Doğrulama — Verify

**Verify** ile bir yöntemin kaç kez ve hangi parametrelerle çağrıldığı doğrulanır.

```

public class KaydetveHoşgeldinTestleri
{
 private readonly Mock<IKullanıcıDepo> _depoMock = new();
 private readonly Mock<IEpostaServisi> _epostaMock = new();
 private readonly KullanıcıServisi _servis;

 public KaydetveHoşgeldinTestleri()
 {
 _servis = new KullanıcıServisi(_depoMock.Object, _epostaMock.Object);
 }

 [Fact]
 public void KaydetVeHoşgeldinGönder_YeniKullanıcı_KaydedilirVeEmailGönderilir()
 {
 // Arrange
 Kullanıcı kullanıcı = new(0, "Ayşe", "ayse@ornek.com", true);

 _depoMock
 .Setup(d => d.EmailMevcutMu("ayse@ornek.com"))
 .Returns(false); // Email kayıtlı değil

 // Act
 _servis.KaydetVeHoşgeldinGönder(kullanıcı);

 // Assert – Kaydet tam olarak bir kez çağrıldı
 _depoMock.Verify(
 d => d.Kaydet(It.Is<Kullanıcı>(k => k.Email == "ayse@ornek.com")),
 Times.Once);

 // Assert – Hoşgeldin e-postası gönderildi
 _epostaMock.Verify(
 e => e.HoşgeldinGönder("ayse@ornek.com", "Ayşe"),
 Times.Once);
 }

 [Fact]
 public void KaydetVeHoşgeldinGönder_MevcutEmail_İstisnaDöndürür()
 {
 // Arrange
 Kullanıcı kullanıcı = new(0, "Mehmet", "mevcut@ornek.com", true);

 _depoMock
 .Setup(d => d.EmailMevcutMu("mevcut@ornek.com"))
 .Returns(true); // Email zaten kayıtlı

 // Act & Assert
 Assert.Throws<InvalidOperationException>(() =>
 _servis.KaydetVeHoşgeldinGönder(kullanıcı));
 }
}

```



```

// Kaydet asla çağrılmamalı
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.Never);

// Hoşgeldin e-postası asla gönderilmemeli
_epostaMock.Verify(
e => e.HoşgeldinGönder(It.IsAny<string>(), It.IsAny<string>()),
Times.Never);
}
}

```

## §22.4.4 Moq — Gelişmiş Kullanım

```

// It.IsAny<T>() – herhangi bir değer
_depoMock.Setup(d => d.BulById(It.IsAny<int>())).Returns(kullanıcı);

// It.Is<T>(pred) – koşul filtresi
_depoMock.Setup(d => d.BulById(It.Is<int>(id => id > 0))).Returns(kullanıcı);

// Sıralı dönüşler – her çağrıda farklı değer
_depoMock
.SetupSequence(d => d.BulById(1))
.Returns(kullanıcı1) // 1. çağrı
.Returns(kullanıcı2) // 2. çağrı
.Returns((Kullanıcı?)null); // 3. çağrı

// Callback – çağrıldığında yan etki üret
_depoMock
.Setup(d => d.Kaydet(It.IsAny<Kullanıcı>()))
.Callback<Kullanıcı>(k => Console.WriteLine($"Kaydedildi: {k.Ad}"));

// İstisna fırlat
_depoMock
.Setup(d => d.BulById(-1))
.Throws<ArgumentException>();

// Özellik taklit etme
_depoMock
.SetupProperty(d => d.ToplamKayıt, 42);

// Times ile çağrı sayısı doğrulama
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.Once);
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.Exactly(3));
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.AtLeastOnce);
_depoMock.Verify(d => d.Kaydet(It.IsAny<Kullanıcı>()), Times.Never);

```

## 22.5 Eş Zamansız Kod Testi

Bölüm 20’da öğrendiğimiz `async/await` yapıları test edilirken özel dikkat gerektirir. xUnit ve NUnit her ikisi de `async Task` test yöntemlerini destekler.

### §22.5.1 Eş Zamansız Üretim Kodu

```

// Eş zamansız servis
public interface IHaberServisi
{
 Task<IEnumerable<Haber>> HaberleriGetirAsync(CancellationTokent iptal = default);
 Task<Haber?> HaberGetirAsync(int id);
}

public record Haber(int Id, string Başlık, DateTime YayınTarihi);

public class HaberÖzetleyici
{
 private readonly IHaberServisi _servis;

```

```

public HaberÖzetleyici(IHaberServisi servis)
{
 _servis = servis;
}

public async Task<string[]> BaşlıklarıGetirAsync()
{
 var haberler = await _servis.HaberleriGetirAsync();
 return haberler.Select(h => h.Başlık).ToArray();
}

public async Task<Haber> ZorunluHaberGetirAsync(int id)
{
 Haber? haber = await _servis.HaberGetirAsync(id);
 return haber ?? throw new KeyNotFoundException($"Haber bulunamadı: {id}");
}
}

```

## §22.5.2 Eş Zamansız Test Yöntemleri

```

using Moq;
using Xunit;

public class HaberÖzetleyiciTestleri
{
 private readonly Mock<IHaberServisi> _servisMock = new();
 private readonly HaberÖzetleyici _özetleyici;

 public HaberÖzetleyiciTestleri()
 {
 _özetleyici = new HaberÖzetleyici(_servisMock.Object);
 }

 [Fact]
 public async Task BaşlıklarıGetirAsync_ÜçHaber_ÜçBaşlıkDöndürür()
 {
 // Arrange – async döndüren mock
 List<Haber> haberler = new()
 {
 new(1, "C# 13 Yayınlandı", DateTime.Today),
 new(2, "AI Çağında Yazılım", DateTime.Today.AddDays(-1)),
 new(3, ".NET 10 Duyurusu", DateTime.Today.AddDays(-2)),
 };

 _servisMock
 .Setup(s => s.HaberleriGetirAsync(It.IsAny<CancellationToken>()))
 .ReturnsAsync(haberler); // ReturnsAsync – Task döndürür

 // Act – await ile eş zamansız çağır
 string[] başlıklar = await _özetleyici.BaşlıklarıGetirAsync();

 // Assert
 Assert.Equal(3, başlıklar.Length);
 Assert.Contains("C# 13 Yayınlandı", başlıklar);
 }

 [Fact]
 public async Task ZorunluHaberGetirAsync_MevcutOlmayan_KeyNotFoundExceptionFırlatır()
 {
 // Arrange
 _servisMock
 .Setup(s => s.HaberGetirAsync(999))

```

```

.ReturnsAsync((Haber?)null);

// Act & Assert - async istisnayı yakala
await Assert.ThrowsAsync<KeyNotFoundException>(async () =>
await _özetleyici.ZorunluHaberGetirAsync(999));
}

[Fact]
public async Task BaşlıklarıGetirAsync_BoşListe_BoşDiziDöndürür()
{
 _servisMock
 .Setup(s => s.HaberleriGetirAsync(It.IsAny<CancellationTokens>()))
 .ReturnsAsync(Enumerable.Empty<Haber>());

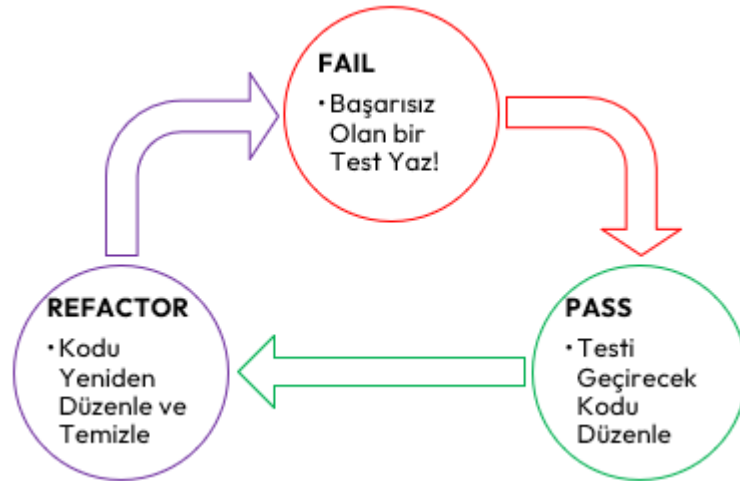
 string[] başlıklar = await _özetleyici.BaşlıklarıGetirAsync();

 Assert.Empty(başlıklar);
}
}

```

## 22.6 Test GÜdümlü Geliştirme — TDD

**Test GÜdümlü Geliştirme** (Test-Driven Development, TDD), kodu yazmadan önce test yazılan bir geliştirme yaklaşımıdır. Üç adımlı döngüsü **Kırmızı**→**Yeşil**→**Yeniden Düzenle** olarak özetlenir.



Şekil 22.2: TDD Döngüsü: Kırmızı → Yeşil → Yeniden Düzenle

### §22.6.1 TDD ile Alışveriş Sepeti Geliştirme

Aşağıda TDD yaklaşımıyla adım adım bir alışveriş sepeti sınıfı geliştirilmektedir.

#### Adım 1 — Kırmızı: İlk Testi Yaz (Kod Yok)

```

// 1. Test yaz - AlışverişSepeti sınıfı henüz YOK
// Bu test derlenmez bile - bu normal!
[Fact]
public void Ekle_TekÜrün_ToplamDoğru()
{
 // Bu aşamada AlışverişSepeti sınıfı yok - derleme hatası
 var sepet = new AlışverişSepeti();
 sepet.Ekle("Kalem", fiyat: 5.0m, adet: 2);

 Assert.Equal(10.0m, sepet.Toplam);
}

```

#### Adım 2 — Yeşil: Testi Geçirecek En Az Kodu Yaz

```

// Testi geçirecek minimum üretim kodu
public class AlışverişSepeti

```

```
{
 private decimal _toplam = 0;

 public decimal Toplam => _toplam;

 public void Ekle(string ürünAdı, decimal fiyat, int adet = 1)
 {
 _toplam += fiyat * adet;
 }
}
// Test artık geçiyor – YEŞİL
```

### Adım 3 — Yeni Test: İndirim Senaryosu

```
[Fact]
public void İndirimUygula_YüzdeCinsinden_ToplamAzalır()
{
 var sepet = new AlışverişSepeti();
 sepet.Ekle("Defter", fiyat: 20.0m, adet: 1);

 sepet.İndirimUygula(yüzde: 10); // %10 indirim

 Assert.Equal(18.0m, sepet.Toplam); // 20 - 2 = 18
}
// Test BAŞARISIZ – İndirimUygula yöntemi yok
```

### Adım 4 — Yeşil: İndirim Yöntemini Ekle

```
public class AlışverişSepeti
{
 private readonly List<(string Ad, decimal Fiyat, int Adet)> _kalemler = new();

 public decimal Toplam => _kalemler.Sum(k => k.Fiyat * k.Adet) * (1 - _indirimOranı);
 private decimal _indirimOranı = 0;

 public void Ekle(string ürünAdı, decimal fiyat, int adet = 1)
 {
 if (fiyat < 0) throw new ArgumentException("Fiyat negatif olamaz.");
 if (adet <= 0) throw new ArgumentException("Adet pozitif olmalıdır.");

 _kalemler.Add((ürünAdı, fiyat, adet));
 }

 public void İndirimUygula(decimal yüzde)
 {
 if (yüzde < 0 || yüzde > 100)
 throw new ArgumentOutOfRangeException(nameof(yüzde), "0-100 arasında olmalı.");
 _indirimOranı = yüzde / 100;
 }

 public int ÜrünSayısı => _kalemler.Sum(k => k.Adet);
}
```

### Adım 5 — Yeniden Düzenleme ve Kapsamlı Test Suite

```
public class AlışverişSepetTestleri
{
 [Fact]
 public void Ekle_TekÜrün_ToplamDoğru()
 {
 var sepet = new AlışverişSepeti();
 sepet.Ekle("Kalem", 5.0m, 2);
 Assert.Equal(10.0m, sepet.Toplam);
 }
}
```

```

[Fact]
public void Ekle_NegatifFiyat_ArgumentExceptionFırlatır()
{
 var sepet = new AlışverişSepeti();
 Assert.Throws<ArgumentException>(() => sepet.Ekle("Kalem", -1.0m));
}

[Fact]
public void İndirimUygula_YüzdeCinsinden_ToplamAzalır()
{
 var sepet = new AlışverişSepeti();
 sepet.Ekle("Defter", 20.0m);
 sepet.İndirimUygula(10);
 Assert.Equal(18.0m, sepet.Toplam);
}

[Fact]
public void İndirimUygula_Tamİndirim_SıfırToplam()
{
 var sepet = new AlışverişSepeti();
 sepet.Ekle("Ürün", 50.0m);
 sepet.İndirimUygula(100);
 Assert.Equal(0.0m, sepet.Toplam);
}

[Theory]
[InlineData(-1)]
[InlineData(101)]
public void İndirimUygula_GeçersizYüzde_ArgumentOutOfRangeExceptionFırlatır(decimal yüzde)
{
 var sepet = new AlışverişSepeti();
 Assert.Throws<ArgumentOutOfRangeException>(() => sepet.İndirimUygula(yüzde));
}

[Fact]
public void ÜrünSayısı_ÇokKalem_ToplamAdetDöndürür()
{
 var sepet = new AlışverişSepeti();
 sepet.Ekle("Kalem", 2.0m, 3);
 sepet.Ekle("Defter", 5.0m, 2);
 Assert.Equal(5, sepet.ÜrünSayısı);
}
}

```

## 22.7 Test Organizasyonu ve En İyi Uygulamalar

### §22.7.1 Test Projesi Klasör Yapısı

Büyük projelerde test organizasyonu kritik öneme sahiptir. Önerilen klasör yapısı aşağıdaki gibidir;

```

Çözüm/ → Çözüm (Solution)
├── src/ → Klasör (Directory)
│ ├── HesapMakinesi/ → Proje (Console Application)
│ │ ├── HesapMakinesi.csproj → Proje Dosyası (Project)
│ │ ├── HesapMakinesi.cs → Ana (Main) Program
│ │ └── Servisler/ → Klasör (Directory)
│ │ └── KullanıcıServisi.cs → Servis Kodu (Class)
│ ├── Kütüphane.Ortak/ → Klasör (Directory)
│ │ └── Ortak.cs → Proje Kodu (Class)
│ └── tests/ → Klasör (Directory)
│ ├── HesapMakinesi.BirimTestleri/ → Proje (Console Application)
│ └── HesapMakinesi.BirimTestleri.csproj → Proje Dosyası (Project)

```

```

├─ HesapMakinesiTestleri.cs → Proje Kodu (Class)
│ └─ Servisler/ → Klasör (Directory)
│ ├── KullanıcıServisTestleri.cs → Servis Kodu (Class)
│ └─ HesapMakinesi.EntegrasyonTestleri/ → Klasör (Directory)
│ └─ VeritabanıEntegrasyonTestleri.cs → Servis Kodu (Class)
└─ Çözüm.sln → Çözüm Dosyası (Solution)

```

### §22.7.2 Testleri Kategorilere Ayırma

[Trait] Özniteliği ile testler kategorilere ayrılabilir;

```

// Testleri kategorilere ayır
[Trait("Kategori", "BirimTesti")]
[Trait("Modül", "HesapMakinesi")]
public class KategorilenmişTestler
{
 [Fact]
 [Trait("Öncelik", "Kritik")]
 public void Kritik_İş_Kuralı_Doğrulama()
 {
 // Kritik iş kuralı testi
 Assert.True(true);
 }
}

// Yalnızca kritik testleri çalıştır:
// dotnet test --filter "Öncelik=Kritik"
// dotnet test --filter "Kategori=BirimTesti"
// dotnet test --filter "FullyQualifiedNames~HesapMakinesi"

```

### §22.7.3 Temiz Test Kodu İlkeleri

```

// KÖTÜ – Her şey tek yöntemde, anlaşılmaz
[Fact]
public void Test1()
{
 var x = new HesapMakinesi();
 Assert.Equal(7, x.Topla(3, 4));
 Assert.Equal(-1, x.Çıkar(3, 4));
 Assert.Throws<DivideByZeroException>(() => x.Böl(1, 0));
}

// İYİ – Her test tek bir şey doğrular
[Fact]
public void Topla_PozitifSayılar_DogruToplam() { ... }

[Fact]
public void Çıkar_BüyüktenKüçüğü_NegatifSonuç() { ... }

[Fact]
public void Böl_SıfırBölen_İstisnaFırlatır() { ... }

// Test yardımcı yöntemleri ile tekrarı önle
public class SepetTestYardımcısı
{
 // Hazır sepet oluşturucu – DRY (Kendini Tekrar Etme) ilkesi
 public static AlışverişSepeti DoluSepetOluştur(
 decimal toplamTutar = 100.0m,
 string ürünAdı = "Test Ürünü")
 {
 var sepet = new AlışverişSepeti();
 sepet.Ekle(ürünAdı, toplamTutar);
 return sepet;
 }
}

```

```
// Testlerde kullan
[Fact]
public void İndirim_DoluSepet_DogruHesaplar()
{
 var sepet = SepetTestYardımcısı.DoluSepetOluştur(100.0m);
 sepet.İndirimUygula(20);
 Assert.Equal(80.0m, sepet.Toplam);
}
```

## §22.7.4 Kod Kapsama Analizi

Testlerin üretim kodunun ne kadarını kapsadığı **kod kapsama** (code coverage) ile ölçülür.

```
// Kod kapsama ile test çalıştır:
// dotnet test --collect:"XPlat Code Coverage"

// Coverlet ile daha ayrıntılı rapor:
// dotnet add package coverlet.collector
// dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=lcov

// coverlet çıktısı:
// +-----+-----+-----+-----+
// | Module | Line | Branch | Method |
// +-----+-----+-----+-----+
// | HesapMakinesi.dll | 95.2% | 87.5% | 100% |
// +-----+-----+-----+-----+
```

**Uyarı:** %100 kod kapsama hedefi, doğruluk garantisi vermez. Önemli olan anlamlı senaryoların (mutlu yol, hata yolları, sınır değerleri) test edilmesidir. Kod kapsamayı bir metrik olarak kullanın, amaç olarak değil.

## 22.8 Düşün ve Çöz

- ◇ **Düşün:** Test GÜdümlü Geliştirme'nin klasik "önce kod sonra test" yaklaşımına göre avantajları ve dezavantajları nelerdir? Bir yazılımcı TDD'yi her zaman uygulamalı mıdır? Hangi durumlarda TDD uygulamak anlamsız olabilir?

- ◇ **Düşün:** Aşağıdaki kodda test edilmesi zor olan nedir? Nasıl yeniden tasarlanmalıdır?

```
public class RaporServisi
{
 public string RaporOluştur()
 {
 // DateTime.Now ve new HttpClient doğrudan kullanılıyor
 var tarih = DateTime.Now;
 using var istemci = new HttpClient();
 var veri = istemci.GetStringAsync("https://api.example.com").Result;
 return $"[{tarih:dd.MM.yyyy}] {veri}";
 }
}
```

- ◇ **Çöz:** Bir banka hesabı sınıfı için TDD ile geliştirme yapın. Aşağıdaki gereksinimleri adım adım (Kırmızı→Yeşil→Yeniden Düzenle) uygulayın:

- Hesap açılışa başlangıç bakiyesiyle oluşturulabilir
- Para yatırılabilir (negatif tutar kabul edilmez)
- Para çekilebilir (bakiye yetersizse hata fırlatır)
- Bakiye sorgulanabilir
- İşlem geçmişi tutulur

- ◇ **Çöz:** *IHavaServisi* arayüzünü uygulayan bir *HavaDurumuServisi* sınıfı yazınız. Bu sınıfı kullanan sınıfta *GünlükPlanlamacı.PlanÖner()* yöntemini geliştirin. Moq ile *IHavaServisi*'ni taklit ederek dört farklı hava durumu senaryosunu test edin (yağmurlu, güneşli, karlı, fırtınalı).

- ◇ **Düşün:** Mock ve stub arasındaki fark nedir? Moq'da *Setup().Returns()* ile stub, *Verify()* ile mock davranışı kurulur. Bir testte hem stub hem mock kullanmak gerekiyorsa bu testin çok şeyi doğrulamaya çalıştığının işareti midir?

- ◇ **Çöz:** *IDepoServisi* arayüzünü hem eş zamanlı hem eş zamansız yöntemlerle tasarlayın. xUnit ve Moq kullanarak bu servisi kullanan *async Task<Ürün[]> AktifÜrünleriGetirAsync()* yöntemini test edin.



Eş zamansız istisna senaryolarını da kapsayın.

- ◇ **Düşün:** %80 kod kapsama hedefi anlamlı mıdır? Kapsama oranı yüksek ancak anlamsız testler mi daha iyidir, yoksa oranı düşük ama kritik senaryoları kapsayan testler mi? Bir projede test kalitesini nasıl ölçersiniz?

## 23. Bölüm

# Varlık Çerçevesi

### 23.1 Nesne-İlişkili Eşleme

Kurumsal yazılımların işlendiği 19 başlığında katmanlı mimarinin bakımı kolaylaştırdığı ve ölçeklenebilirliği artırdığı anlatılmıştı. Ayrıca aynı başlık altında katmanların nasıl tanımlanması gerektiği 19.4.5 başlığında detaylı anlatılmıştı. İş katmanındaki veri taşıyan sınıfların (BEC) kullanılarak veri erişim katmanının tasarlandığı ve iş varlıklarının soyut olarak veri tabanında nasıl saklanacağına veri erişim bileşenleri (DAC) ile tanımlandığı anlatılmıştı. Tüm bunlara örnek olarak 19.8 başlığında bir uygulama örneği verilmişti.

Veri erişim katmanında yer alacak her veri erişim bileşenlerinin (DAC) her biri, yaratma (**create**), veri tabanında kayıt (**record**), okuma (**read**), güncelleme (**update**) ve silme (**delete**) işlemini gerçekleştirilecek şekilde yöntemlere sahiptir. Günümüzde iş varlığı bileşenlerini (BEC) için bu CRUD yöntemlerini otomatik sağlayan ve en az kodlama ile eşleştiren teknolojiler ortaya çıkmıştır. Bu teknolojinin adı **nesne-ilişkili eşlemedir** ORM (**Object-Relational Mapping**). Kısaca iş varlıklarımızı veri tabanındaki tablolardaki kayıtlarla eşler. Yani projemizde CRUD işlevleri için gerekli SQL komutları ORM tarafından otomatik olarak oluşturulur ve bu teknoloji sayesinde daha hızlı ve hatasız yazılım geliştirilir. ORM, çok fazla programlama yapmadan, otomatik bir şekilde iş nesnelerinden ilişkisel veri tabanına veri depolamak için bir araçtır. Üç ana bölümden oluşur:

1. İş bileşenlerini içeren etki alanı sınıf nesneleri
2. İlişkisel veri tabanı nesneleri
3. Etki alanı nesnelerinin ilişkisel veri tabanı nesnelere nasıl eşlendiğine dair eşleme bilgileri.

ORM, veri tabanı tasarımımızı iş bileşeni sınıf tasarımımızdan ayrı tutmamızı sağlar. Bu, uygulamayı sürdürülebilir ve genişletilebilir hale getirir. Ayrıca standart CRUD işlemini (Oluşturma, Okuma, Güncelleme ve Silme) otomatikleştirir, böylece geliştiricinin bunu manuel olarak yazmasına gerek kalmaz. Veri erişimi için, veri erişim katmanında detaylı ADO.NET kodunu yazmak ve yönetmek sıkıcı ve monoton bir iştir. Microsoft, uygulamanız için veri tabanı ile ilgili faaliyetleri otomatikleştirmek amacıyla **varlık çerçevesini** (**Entity Framework**-kısaca **EF**) geliştirmiştir.

Örnek Kurumsal Uygulama 19.8 başlığında verilen örnekte ilk önce veri tabanında tablo oluşturulmuş, daha sonrasında bu tabloya veri ekleme ve veri çekme işlemi kodlanmıştır. Bu yaklaşıma **önce veri tabanı** (**database first**) adı verilir. Önce veri tabanı yaklaşımında, veri tabanı ile iş varlıklar (BEC) eşleşmesi için ilk önce veri tabanı tabloları tanımlanır ve sonrasında bu tablolar ile varlıklar eşleştirilir (**mapping**). Bu da yazılımın devreye alınacağı zaman, veri tabanı yöneticisine bir iş çıkarır. Yazılımın devreye alınmasında ve sonraki uygulama sürümlerinde kodlamayı yapan yazılımcının veri tabanı yöneticisi ile çalışmasını gerekir.

**Önce veri tabanı** yaklaşımında genel olarak **EDMX** (**Entity Data Model XML**) dosyaları kullanılır. Bu dosyalar temel olarak mevcut bir veri tabanı üzerinden görsel olarak .NET nesneleri arasındaki ilişkiyi tanımlayan bir XML belgesidir. Visual Studio içinde **.edmx** dosyası açıldığında genellikle bir grafik tasarım (**designer**) ekranı ile gösterilir. Burada tablolar, ilişkiler ve varlıklar (**entity**) sürükle-bırak ile düzenlenebilir.

**EF Core**, **.edmx** dosyasını desteklemez! Ancak **önce veri tabanı** yaklaşımı **EF Core** tarafında bu işlem **Scaffold-DbContext** komutuyla yapılır. Yeni kurumsal uygulama projelerinde genellikle **Entity Framework Core** tercih edilir ve veritabanında tablo oluşturmadan işe başlanır. Bu yaklaşıma **önce kod** (**code first**) yaklaşımı adı verilir.

Açık olan projemizde **EF Core** bileşenlerini **SQLite** veritabanı ile birlikte dahil etmek için **Tools → NuGet Package Manager → Package Manager Console** üzerinden komut vererek bileşenleri projemize dahil ederiz;

```
Install-Package Microsoft.EntityFrameworkCore
Install-Package Microsoft.EntityFrameworkCore.Sqlite
```

### 23.2 Veri Tabanı Bağlamı

Veri Tabanı Bağlamı (**DbContext**), kullanmayı öğrenmemiz gereken ilk ve en önemli **EF veri tipidir**. Veri tabanı sorguları için bir ara yüz görevi görür ve nesnelerde yaptığınız değişiklikleri takip eder, böylece veriler veri tabanında kalıcı hale getirilebilir.

Nesnelerini kullanarak verileri sorgulamak, eklemek, güncellemek ve silmek için EF kullanmak amacıyla öncelikle modelinizde tanımlanan varlıkları ve ilişkileri bir veri tabanındaki tablolara eşleyen bir model (BEC) oluşturmanız gerekir. Aşağıdaki **Öğrenci** ve **Bölüm** varlığını ele alalım;

```
public class Öğrenci
{
 public int ÖğrenciId { get; set; }
 public string Ad { get; set; }
 public int Yaş { get; set; }
}
public class Bölüm
{
 public int BölümNo { get; set; }
 public string Ad { get; set; }
}
```

Bir modeliniz olduğunda, iş varlığınızın uygulamanızın etkileşime girdiği ilk sınıf **bağlam sınıfıdır** (context class). `Microsoft.EntityFrameworkCore.DbContext` bağlam sınıfını şu amaçlarla kullanabilirsiniz:

- ◊ Sorguları yazıp çalıştırabilirsiniz.
- ◊ Sorgu sonuçlarını varlık nesneleri olarak somutlaştırabilirsiniz
- ◊ Varlık nesnelerinde yapılan değişiklikleri izleyebilirsiniz.
- ◊ Nesne değişikliklerini veri tabanında kalıcı hale getirebilirsiniz.
- ◊ Nesneleri bellekte kullanıcı arayüzü (UI) bileşenlerine bağlayabilirsiniz.

Bağlam sınıfıyla çalışmanın önerilen yolu, `DbContext` sınıfından türetilen bağlam sınıfında, belirtilen varlıkların koleksiyonlarını temsil eden `DbSet` özelliklerini açığa çıkaran bir sınıf tanımlamaktır. **EF Core**, önce kod (code first) yaklaşımını kullanır. Bu nedenle bağlam sınıfını programcı yazar;

```
public class OkulContext : DbContext
{
 public DbSet<Öğrenci> Öğrenciler { get; set; }
 public DbSet<Bölüm> Bölümler { get; set; }
}
```

Bir bağlam sınıfınız olduğunda, bu özellikler `DbSet` veri tipine sahip olduğundan bağlamdaki varlıkları sorgulayabilir, ekleyebilir veya silebilirsiniz. Bir bağlam nesnesindeki `DbSet` özelliğine eriştiğinizde, belirtilen tipteki tüm varlıkları döndüren bir başlangıç sorgusunu çalışır. Bu sorgu sonucu;

- ◊ Numaralandırılan nesne listesi `foreach` ile gezilebilir.
- ◊ Numaralandırılan nesne listesi `ToArray()`, `ToDictionary()` ve `ToList()` gibi yöntemlerle koleksiyonlara çevrilebilir.
- ◊ `First` veya `Any` gibi LINQ genişleme yöntemleri (LINQ extension method) ile sorgulanabilir.
- ◊ Bir varlık, bağlamda yüklü değilse `DbEntityEntry.Reload` ve `Database.ExecuteSqlCommand` ile yeniden yüklenebilir.

Bağlam nesnesinin ömrü, örneğin (instance) oluşturulmasıyla başlar ve örneğin atılması veya çöp toplanmasıyla sona erer. Bunun için `using` anahtar kelimesi kullanılır;

```
using (var context = new AppDbContext())
{
 var öğrenciler = context.Öğrenciler;
 foreach (var o in öğrenciler)
 {
 Console.WriteLine($"Ad: {o.Ad}, Yaş: {o.Yaş}");
 }
}
```

Burada veritabanından veriler çekilirken üç yöntem kullanılır;

### §23.2.1 Önden Yükleme

**Önden yükleme** (eager loading), ana veriyle birlikte ilişkili verilerin de tek bir SQL sorgusunda (JOIN kullanarak) veritabanından çekilmesidir. **EF Core**'da bu işlem `.Include()` metodu ile yapılır. İlişkili verilere mutlaka ihtiyaç duyacağınızdan emininseniz ve veritabanına gidiş-dönüş (round-trip) sayısını azaltmak istiyorsanız tercih edilir.

```
using (var context = new KitaplikContext())
{
```

```
// Yazarı çekerken kitaplarını da aynı sorguda getirir
var yazar = context.Authors
.Include(a => a.Books)
.FirstOrDefault(a => a.AuthorId == 1);
foreach (var kitap in yazar.Books)
{
 Console.WriteLine(kitap.Title);
}
}
```

### §23.2.2 Tembel Yükleme

**Tembel yükleme** (**lazy loading**), ilişkili verilerin sadece onlara erişildiği anda (kod içerisinde özellik çağrıldığında) veritabanından çekilmesidir.

EF Core'da bu özelliği kullanmak için; `Microsoft.EntityFrameworkCore.Proxies` paketini kurmalı ve navigasyon özelliklerini **virtual** yapmalısınız. İlişkili verilere her zaman ihtiyaç duyulmadığı durumlarda tercih edilir. Ancak döngü içerisinde kullanılırsa, her yazar için ayrı bir SQL sorgusu atılması gerektiğinden "N+1 Sorgu Problemine" yol açabilir.

```
// Context yapılandırmasında: options.UseLazyLoadingProxies(); yazmalıyız.
using (var context = new KitaplikContext())
{
 // Sadece yazar bilgisi çekilir
 var yazar = context.Authors.FirstOrDefault(a => a.AuthorId == 1);
 // Aşağıdaki satır çalıştığı anda EF Core arka planda yeni bir SQL sorgusu atarak kitapları
 // → getirir
 foreach (var kitap in yazar.Books)
 {
 Console.WriteLine(kitap.Title);
 }
}
```

### §23.2.3 Açıkça Yükleme

**Açıkça Yükleme** (**explicit loading**), ilişkili verilerin ana veri çekildikten sonra, ihtiyaç duyulduğu anda manuel olarak (açık bir komutla) yüklenmesidir. Lazy Loading'e benzer ancak verinin ne zaman çekileceği tamamen programcının kontrolündedir. Nesne çekildikten sonra belirli bir mantığa göre ilişkili veriyi çekip çekmemeye karar verecekseniz idealdir.

```
using (var context = new KitaplikContext())
{
 var yazar = context.Authors.FirstOrDefault(a => a.AuthorId == 1);
 // Belirli bir koşula göre kitapları daha sonra yüklüyoruz
 if (yazar.IsActive)
 {
 context.Entry(yazar)
 .Collection(a => a.Books)
 .Load(); // Veritabanına şimdi gidilir
 }
}
```

### §23.2.4 Varlıkları Yüklerken Değişiklik Takibini Kapatma

EF Core'da `AsNoTracking()` yöntemi, sorgu sonucunda dönen nesnelerin **değişiklik için takip edilmesini** kısıtlar. Özellikle sadece veri okuma (**read-only**) işlemlerinde bellek (RAM) ve işlemci (CPU) yükünü azaltmak için kullanılır.

```
using (var context = new MyDbContext())
{
 // Verileri takip mekanizmasına (Change Tracker) almadan çeker
 var musteriler = context.Customers
 .AsNoTracking()
 .ToList();
 // Performans avantajı: Bellekte nesnelerin bir kopyası (snapshot) tutulmaz.
}
```

## 23.3 Önce Kod

**Önce kod** (**code first**) yaklaşımında ise geliştirici, modeli (BEC) belirtmek için kod yazar. EF, geliştirici tarafından kodlanan varlık sınıflarına ve ek model yapılandırmasına bağlı olarak ver modellerini ve veri tabanı eşlemelerini **çalışma zamanında** (**run-time**) üretir. Kısaca EF, **kodlanan varlık sınıflarından veri tabanını otomatik olarak oluşturur**. Yani eşleme (**mapping**) otomatik olarak yapılır.

Bu yaklaşım, veri tabanının tasarlandığı **.edmx** dosyası kullanmadan veri tabanı tablosundaki kayıtlar ile varlıklarımız otomatik olarak eşleme yapılır. Ayrıca mevcut veri tabanı tablolarını da kullanabilirsiniz, buna **mevcut veri tabanı ile önce kod** (**code first with existing database**) denir. Aşağıdaki gibi bir BEC varlığımız olsun;

```
public class Öğrenci
{
 public int ÖğrenciId { get; set; }
 public string Ad { get; set; }
 public string Soyad { get; set; }
 public int Yaş { get; set; }
}
```

Şimdi varlık sınıfımız ve veri tabanınız arasındaki köprü olan bağlamımız olan **DbContext** sınıfı tasarlayacağız. Bu sınıfa da varlık sınıfına ait bir **DbSet<>** özelliği eklenir;

```
public class AppDbContext : DbContext
{
 public DbSet<Öğrenci> Öğrenciler { get; set; }
}
```

Artık veri tabanına ilk bağlandığımız anda **Öğrenci** tablosu veri tabanında oluşacaktır; Bunu yapmak için aşağıdaki program yazılabilir,

```
using (var context = new AppDbContext())
{
 var yeniÖğrenciler = new[]
 {
 new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
 new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
 new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
 };
 context.Öğrenciler.Add(yeniÖğrenciler[0]);
 context.Öğrenciler.Add(yeniÖğrenciler[1]);
 context.Öğrenciler.Add(yeniÖğrenciler[2]);
 // ya da context.Öğrenciler.AddRange(yeniÖğrenciler);
 context.SaveChanges();
}
```

**Bu program bu haliyle çalışmaz!** Çünkü veri tabanı bağlantı kısmı eksiktir. Ama genel eşleşme iskeleti bu şekildedir. **SQLite** veri tabanı kullanarak buna çalışan bir örnek verelim. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
EFCoreUygulama/ → Çözüm (Solution)
├─ CodeFirst1/ → Proje (Console Application)
│ └─ Program.cs → Ana (Main) Program
```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File→New→Project...** seçilerek **EFCoreUygulama** adlı çözüm içinde **CodeFirst1** adlı ilk projemizi **Console Application** olarak oluşturulur.
2. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır; **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.Sqlite** paketleri yüklenir.
3. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```
using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.Sqlite; // Install Nuget package

using (var context = new AppDbContext())
{
```

```

context.Database.EnsureCreated();
Console.WriteLine("SQLite Veri Tabanı Üzerinde Tablolar Oluşturuldu!");
var yeniÖğrenciler = new[]
{
 new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
 new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
 new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
};
context.Öğrenciler.AddRange(yeniÖğrenciler);
context.SaveChanges();

var öğrenciler = context.Öğrenciler;
Console.WriteLine("Veritabanındaki Öğrenciler:");
foreach (var o in öğrenciler)
{
 Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
}

var öğrenci = context.Öğrenciler.FirstOrDefault(u => u.Ad == "İlhan");
if (öğrenci != null)
{
 context.Öğrenciler.Remove(öğrenci);
 context.SaveChanges();
 Console.WriteLine($"'{öğrenci.Ad}' adlı öğrenci silindi.");
}
context.Database.CloseConnection();
//context.Database.EnsureDeleted();
}

public class AppDbContext : DbContext
{
 public DbSet<Öğrenci> Öğrenciler { get; set; }

 protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
 {
 var çalışmaKlasörü=AppDomain.CurrentDomain.BaseDirectory;
 //Üç klasör geri git. Orada CodeFirst.sqlite veritabanını oluştur.
 var dbKlasörü =Path.Combine(çalışmaKlasörü, "..", "..", "..", "CodeFirst.sqlite");
 //Bunu yapınca Solution Explorer üzerinde veritabanı görünür.
 optionsBuilder.UseSqlite($"Data Source={dbKlasörü}");
 }
}

public class Öğrenci
{
 public int ÖğrenciId { get; set; }
 public string Ad { get; set; }
 public string Soyad { get; set; }
 public int Yaş { get; set; }
}

```

4. Program çalışıp `context.Database.EnsureCreated()`; icra edildiği anda bağlantı kurulan `SQLite` veri tabanında `Öğrenciler` tablosu oluşur (Şekil 23.1a).

5. Aynı projede `CodeFirst.sqlite` veritabanını silindikten sonra bir başka sınıf olan `Kullanıcı` sınıfı eklenebilir.

```

using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.Sqlite; // Install Nuget package

var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
//Üç klasör geri git. Orada CodeFirst.sqlite veritabanını oluştur.
var dbKlasörü = Path.Combine(çalışmaKlasörü, "..", "..", "..", "CodeFirst.sqlite");
//Bunu yapınca Solution Explorer üzerinde veritabanı görünür.
var options = new DbContextOptionsBuilder<AppDbContext>()

```

```

.UseSqlite($"Data Source={dbKlasörü}")
.Options;

using (var context = new AppDbContext(options))
{
 context.Database.EnsureCreated();
 Console.WriteLine("SQLite Veri Tabanı Üzerinde Tablolar Oluşturuldu!");
 var yeniÖğrenciler = new[]
 {
 new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
 new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
 new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
 };
 context.Öğrenciler.AddRange(yeniÖğrenciler);
 context.SaveChanges();

 var öğrenciler = context.Öğrenciler;
 Console.WriteLine("Veritabanındaki Öğrenciler:");
 foreach (var o in öğrenciler)
 {
 Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
 }

 var öğrenci = context.Öğrenciler.FirstOrDefault(u => u.Ad == "İlhan");
 if (öğrenci != null)
 {
 context.Öğrenciler.Remove(öğrenci);
 context.SaveChanges();
 Console.WriteLine($"'{öğrenci.Ad}' adlı öğrenci silindi.");
 }
 var kullanıcılar = context.Kullanıcılar;
 Console.WriteLine("Veritabanındaki Kullanıcılar:");
 foreach (var o in kullanıcılar)
 {
 Console.WriteLine($"Adı: {o.Ad}, Id: {o.Id}");
 }

 context.Database.CloseConnection();
 //context.Database.EnsureDeleted(); Veritabanı Siler
}

public class AppDbContext : DbContext
{
 public DbSet<Öğrenci> Öğrenciler { get; set; }
 public DbSet<Kullanıcı> Kullanıcılar { get; set; }

 public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
}

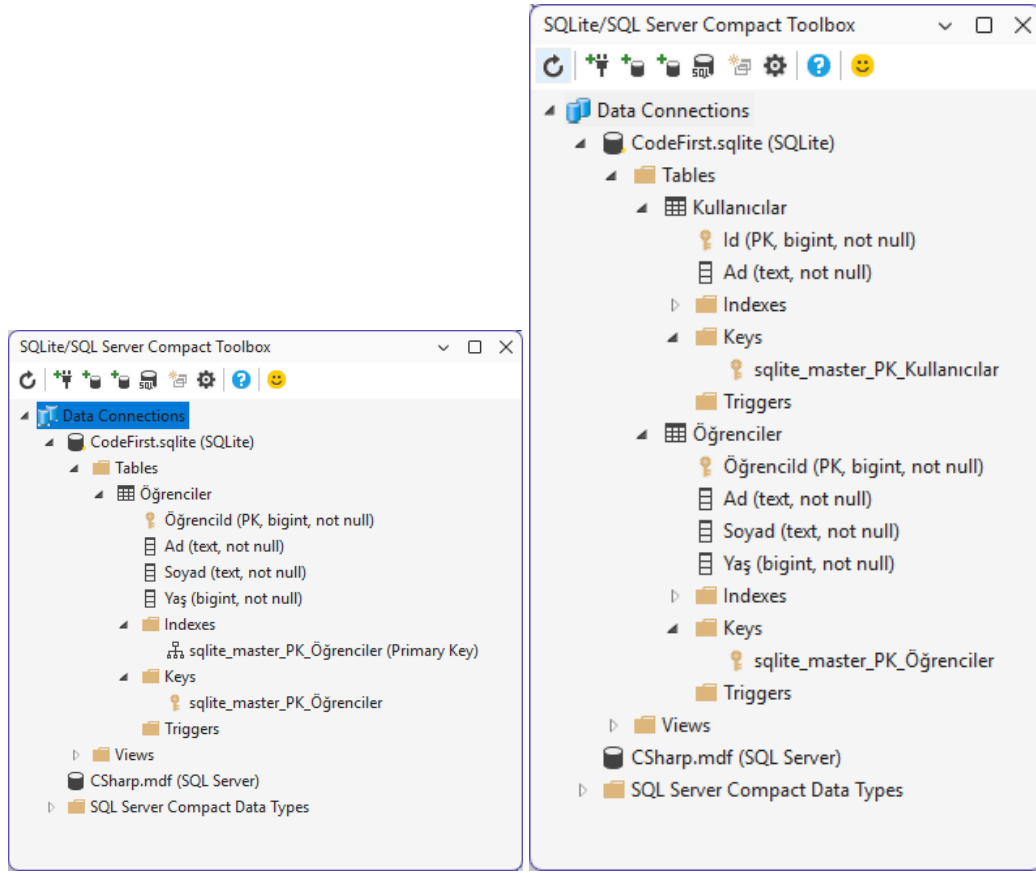
public class Öğrenci
{
 public int ÖğrenciId { get; set; }
 public string Ad { get; set; }
 public string Soyad { get; set; }
 public int Yaş { get; set; }
}

public class Kullanıcı
{
 public int Id { get; set; }
 public string Ad { get; set; }
}

```

6. Program çalışıp `context.Database.EnsureCreated()`; icra edildiği anda bağlantı kurulan SQLite veri





(a) Öğrenci Varlığı Eklenince

(b) Öğrenci ve Kullanıcı Varlığı Eklenince

Şekil 23.1: CodeFirst.sqlite Veritabanında EF Core Tarafından Oluşturulan Tablolar

tabanında **Öğrenciler** yanında **Kullanıcılar** tablosu oluşur (Şekil 23.1b).

7. Bu sefer veritabanına bağlanma **OnConfiguring()** yöntemi ile değil başka şekilde başlatılmıştır. Yeni programda **AppDbContext** nesnesine **DbContextOptionsBuilder** parametresi verilerek başlatılmıştır.

**CodeFirst.sqlite** veritabanında oluşan tabloların isimleri **AppDbContext** sınıfında tanımlanan **DbSet<>** özelliklerinden gelmiştir. Ayrıca veri tabanı incelendiğinde **ÖğrenciId** ve **Id** kolonlarının **birincil anahtar** (**primary key**) olduğu da görülmektedir. Bu alanlar **User** sınıfındaki **Id** ve **Öğrenci** sınıfındaki **ÖğrenciId** özellik kimliklerinin sonunda **Id** karakterleri içeriyor olmasından kaynaklanmaktadır. Varlık sınıfları tasarlanırken göz önüne alınan bunun gibi birçok **kimliklendirme kuralı** (**convention**) vardır.

## 23.4 Önce Kod Kimliklendirme Kuralları

Varlık sınıfında özellik ve alan kimliklendirmesi oluşturulacak veri tabanındaki tabloları etkiler. Dolayısıyla yukarıda anlatılan **kimliklendirme kuralları** (**convention**) **EF Core**'un kalbini oluşturur.

### §23.4.1 Var Olan Bir Kimliklendirme Kuralını Geçersiz Kılma

**ConfigureConventions** ile tanımlanmış herhangi bir kuralı (**convention**), **OnModelCreating()** yöntemini geçersiz kılarak kaldırabilirsiniz.

Aşağıda veri tabanında, varlığa karşılık oluşturulan, tablo isimleri sonuna "s"/"ler" eki getirilme kaldırılmıştır.

```
public class DersContext : DbContext
{
 public DbSet<Saat> Saatler { set; get; }
 protected override void OnModelCreating(DbModelBuilder modelBuilder)
 {
 modelBuilder.Conventions.Remove<PluralizingTableNameConvention>();
 // PluralizingTableNameConvention adlı kural kaldırılmıştır.
 }
}
```

```
}
```

### §23.4.2 Birincil Anahtar Kuralı

Son örneğimizde olduğu gibi varsayılan olarak, bir sınıftaki bir özelliğin;

- ◊ Kimliği "ID" ise
- ◊ Kimlik "ID" karakterleri ile bitmişse
- ◊ Sınıf adını "ID" ile takip edilmişse

bu özellik **birincil anahtardır** (**primary key**).

Kimlikteki "ID" karakterleri büyük/küçük harfe duyarlı değildir. "iD", "Id" olabilir. Ayrıca birincil anahtar olacak özelliğin veri tipi sayısal (**int**, **long**) veya **GUID** olabilir;

```
public class Okul
{
 public int okulId { get; set; } // Primary key
 //...
}
```

### §23.4.3 Veri Tipi Keşfi

EF Core (**Entity Framework Core**) içerisinde **veri tipi keşfi** (**type discovery**), modelinizin veritabanı şemasına nasıl dönüştürüleceğini belirleyen kritik bir mekanizmadır. Bir sınıfın veritabanında bir tabloya karşılık gelip gelmeyeceğine EF Core belirli kurallar çerçevesinde karar verir.

İşte EF Core'un veri tiplerini keşfetme ve modele dahil etme hiyerarşisi:

#### DbSet<T> Özellikleri

En temel keşif yöntemi, **DbContext** sınıfınız içinde tanımladığınız **DbSet<TEntity>** özellikleridir. EF Core, bu özellikleri tarar ve belirtilen tüm veri tiplerini modelin bir parçası olarak kabul eder.

```
public class UygulamaDbContext : DbContext
{
 // 'Kitap' veri tipi ve tüm bağlı olduğu alt veri tipleri keşfedilir.
 public DbSet<Kitap> Kitaplar { get; set; }
}
```

#### Gezinme Özellikleri

EF Core sadece **DbSet** içindekileri değil, bu sınıfların işaret ettiği diğer sınıfları da özyinelemeli (**recursive**) olarak keşfeder. EF Core'da buna **gezinme özelliği** (**navigation property**) adı verilir. Gezinme özelliği özyinelemeli(**recursive discovery**) olarak yapılır.

Aşağıdaki örnekten gidecek olursak, eğer **Kitap** sınıfınız içinde bir **Yazar** referansı varsa, **Yazar** için bir **DbSet** tanımlamasanız bile EF Core onu keşfeder ve bir tablo oluşturur.

```
public class Kitap
{
 public int Id { get; set; }
 public string Ad { get; set; }
 // EF Core bu navigasyon özelliğini takip ederek 'Yazar' türünü keşfeder.
 public Yazar Yazar { get; set; }
}
```

Buna ilişkin bir gerçek bir örnek **Microsoft SQL Server LocalDB** üzerinden verilecektir. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
EFCoreUygulama/ → Çözüm (Solution)
├── CodeFirst2/ → Proje (Console Application)
│ ├── SQLServer.mdf → MSSQL Veritabanı (Data)
│ ├── SQLServer_log.ldf → MSSQL Veritabanı (Log)
│ └── Program.cs → Ana (Main) Program
```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File**→**New**→**Project...** seçilerek **EFCoreUygulama** adlı çözüm içinde **CodeFirst2** adlı ilk projemizi **Console Application** olarak oluşturulur.
2. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır ve **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.Sqlserver** paketleri yüklenir.

3. **Visual Studio** üzerinde **Tools→Connect To Database** tıklanarak *projenin klasöründe SQLServer.mdf* adında bir *Microsoft SQL Server LocalDB* veritabanı oluşturulur.

4. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```
using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.SqlServer; // Install Nuget package

var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü=Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
var db = Path.Combine(projeKlasörü, "SQLServer.mdf");

string connectionString = $"Server=(LocalDB)\MSSQLLocalDB; AttachDbFilename={db};
→ Integrated Security=True;Connect Timeout=30;";
var options = new DbContextOptionsBuilder<AppDbContext>()
.UseSqlServer(connectionString)
.Options;

using (var context = new AppDbContext(options))
{
 context.Database.EnsureCreated();
 Console.WriteLine("SQServer Veri Tabanı Üzerinde Tablolar Oluşturuldu!");
 var yerleşkeler = context.Yerleşkeler;
 Console.WriteLine("Yerleşkeler:");
 foreach (var y in yerleşkeler)
 {
 Console.WriteLine($"Id: {y.Id}, Yerleşke Adı: {y.YerleşkeAdı}:");
 foreach (var b in y.Birimler)
 {
 Console.WriteLine($"Id: {b.Id}, Birim Adı: {b.BirimAdı}:");
 }
 }
 context.Database.CloseConnection();
}

public class AppDbContext : DbContext
{
 public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
 public DbSet<Yerleşke> Yerleşkeler { set; get; }
}

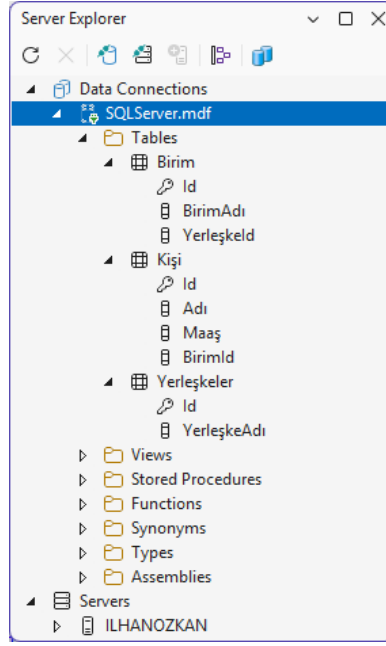
public class Yerleşke
{
 public int Id { set; get; }
 public string YerleşkeAdı { set; get; }
 public virtual ICollection<Birim> Birimler { set; get; }
}

public class Birim
{
 public int Id { set; get; }
 public string BirimAdı { set; get; }
 public virtual ICollection<Kişi> Personel { set; get; }
}

public class Kişi
{
 public int Id { set; get; }
 public string Adı { set; get; }
 public decimal Maaş { set; get; }
}

public class Müdür : Kişi
{
 public string MüdürÖzelliği { set; get; }
}

public class Şef : Kişi
{
 public string ŞefÖzelliği { set; get; }
```



Şekil 23.2: EF Core Veri Tipi Keşfi

```

}
public class Muhasebeci : Kişi
{
 public string MuhasebeciÖzelliği { set; get; }
}

```

5. Program çalıştığında oluşacak veri tabanı içeriği Şekil 23.2’de verilmiştir. Şekilde görüldüğü üzere `AppDbContext` sınıfında yalnızca `DbSet<Yerleşke> Yerleşkeler set; get;` özelliği tanımlanmasına rağmen veritabanında tüm varlıklara ilişkin tablolar oluşturulmuştur.

### OnModelCreating İçinde Manuel Veri Tipi Dahil Etme

Bazen bir sınıfın ne `DbSet` olarak tanımlanmasını ne de bir navigasyon özelliğiyle erişilmesini istersiniz. Bu durumda `ModelBuilder` kullanarak türü manuel olarak modele zorla dahil edebilirsiniz.

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
 // Veri tipi manuel olarak keşif sürecine dahil edilir
 modelBuilder.Entity<LogKaydi>();
}

```

### Keşfedilen Veri Tiplerinin Yapılandırılması

EF Core bir veri tipini keşfettiğinde, onun nasıl bir tabloya dönüşeceğini şu üç yöntemle belirler (Öncelik sırasına göre):

1. **Fluent API:** `OnModelCreating` içindeki tanımlamalar en yüksek önceliğe sahiptir.
2. **Veri Öznitelikleri (data annotations):** Sınıf üzerindeki `[Table]`, `[Key]`, `[Required]` gibi öznitelikler (attribute).
3. **Kimliklendirme Kuralları (conventions):** Varsayılan kurallar (Örn: `Id` kimlikli özelliklerin birincil anahtar yapılması gibi).

### Veri Tipi Keşfini Engelleme (NotMapped)

Keşfedilmesini istemediğiniz sınıfları veya özellikleri dışarıda bırakmak için şu yöntemleri kullanabilirsiniz:

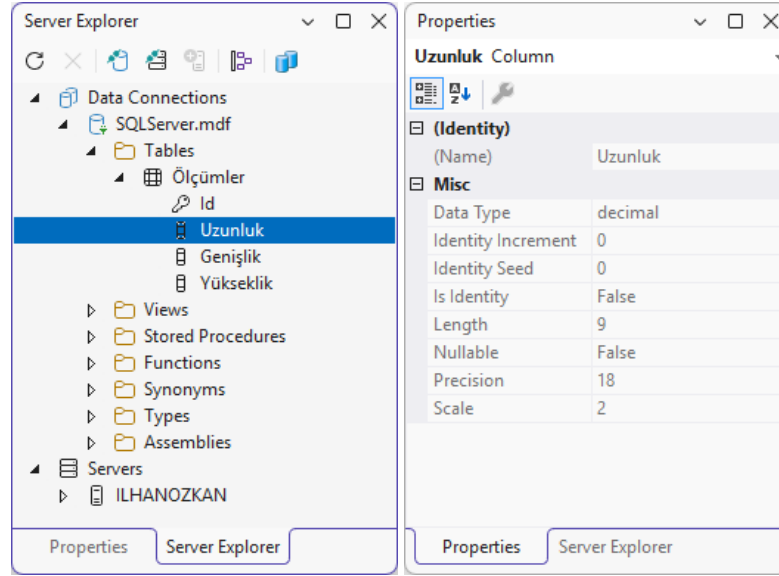
1. **Öznitelik ile:** Sınıfın üzerine `[NotMapped]` eklemek.
2. **Fluent API ile:** `modelBuilder.Ignore<GeciciTur>();` kullanmak.

### Özet

EF Core, keşfettiği veri tiplerinde, sınıfta fiziksel olarak bulunmayan ancak veritabanında olması gereken sütunları da keşfedebilir. Örneğin, bir **yabancı anahtar (foreign key)** özelliğini sınıfa yazmazsanız, EF Core bunu bir gölge özellik (**shadow property**) olarak arka planda keşfeder ve yönetir.

| Yöntem                      | Keşif Biçimi                                   | Öncelik   |
|-----------------------------|------------------------------------------------|-----------|
| <code>DbSet&lt;T&gt;</code> | Doğrudan tanımlama.                            | Orta      |
| Navigasyon                  | Özyinelemeli (Bağımlı veri tipleri).           | Orta      |
| Fluent API                  | Manuel ( <code>modelBuilder.Entity()</code> ). | En Yüksek |
| Annotations                 | Öznitelikler ( <code>[Table]</code> ).         | Yüksek    |
| Conventions                 | Varsayılan kimliklendirme kuralları.           | Düşük     |

Tablo 23.1: EF Core Tür Keşfi ve Yapılandırma Hiyerarşisi



(a) Ölçüm Tablosu

(b) Ölçüm Tablosu Uzunluk Özelliği

Şekil 23.3: Veritabanında Ölçüm için EF Core Tarafından Oluşturulan Tablo

#### Ş23.4.4 Ondalık Kimliklendirme Kuralı

**Ondalık kimliklendirme kuralı** (**decimal convention**), ondalık sayıların (özellikle para birimleri ve hassas ölçümler) veritabanında hangi hassasiyetle (**precision**) ve ölçekle (**scale**) saklanacağını belirleyen kurallardır. Varsayılan olarak EF Core, **decimal** veri tipindeki bir alanı veritabanına aktarırken genellikle **decimal(18,2)** formatını kullanır. Bu, toplamda 18 basamak ve virgülden sonra 2 basamak anlamına gelir.

```
public class Ölçüm
{
 public int Id { set; get; }
 public decimal Uzunluk { set; get; }
 public decimal Genişlik { set; get; }
 public decimal Yükseklik { set; get; }
}
```

Bu varlığın veritabanında oluşmuş hali Şekil 23.3'de verilmiştir.

Finansal uygulamalarda bu varsayılan değer genellikle yetersiz kalır. Üç şekilde yapılır.

#### Genel Kural

EF Core 6.0+ sonrasında projenizdeki tüm **decimal** alanların aynı formatta (örneğin (18,4)) olmasını istiyorsanız, her varlık (entity) için tek tek ayar yapmak yerine genel bir kural tanımlayabilirsiniz.

```
protected override void ConfigureConventions(ModelConfigurationBuilder configurationBuilder)
{
 // Projedeki tüm decimal özellikler varsayılan olarak (18,4) olacak
 configurationBuilder.Properties<decimal>()
 .HavePrecision(18, 4);
}
```

#### Fluent API ile Yapılandırma

Sadece belirli bir sınıfın belirlenen bir özelliği için kural koymak istediğinizde kullanılır. Bu yöntem global kuralları ezer.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
 modelBuilder.Entity<Kitap>()
 .Property(k => k.Fiyat)
 .HasPrecision(18, 2); // Toplam 18 basamak, 2 basamak virgülden sonra
}
```

### Öznitelikler Kullanarak

Sınıf içindeki özelliklerin üzerine doğrudan öznitelik eklenerek (**data annotations**) yapılır.

```
public class Kitap
{
 public int Id { get; set; }
 [Precision(14, 3)]
 public decimal Agirlik { get; set; }
}
```

### §23.4.5 İlişki Kuralı

İki varlık arasındaki ilişki, **gezinme özelliğini** (**navigation property**) kullanarak otomatik olarak yapılır. Bu gezinme özelliği basit bir referans tipi veya koleksiyon tipi olabilir.

```
public class Kişi
{
 public int Id { set; get; }
 public string Adı { set; get; }
 public decimal Maaş { set; get; }

 //Telefon için gezinme(navigability) özelliği
 public ICollection<Telefon> Telefonlar { set; get; }
}

public class Telefon
{
 public int Id { set; get; }
 public string TelefonNumarası { set; get; }
 public string TelefonTürü { set; get; }
}
```

Buna ilişkin örnek 23.4.3 başlığında verilmiştir.

### §23.4.6 Yabancı Anahtar Kimliklendirme Kuralı

Eğer A sınıfı B sınıfıyla ilişkiliyse ve B sınıfı, A'nın birincil anahtarıyla aynı ada ve tipe sahip bir özelliğe sahipse, EF Core otomatik olarak bu özelliğin **yabancı anahtar** (**foreign key**) olduğunu varsayar.

```
public class Birim
{
 public int Id { set; get; }
 public string BirimAdı { set; get; }
 public virtual ICollection<Kişi> Personel { set; get; }
}

public class Kişi
{
 public int Id { set; get; }
 public string Adı { set; get; }
 public decimal Maaş { set; get; }
 public ICollection<Telefon> Telefonlar { set; get; }
 public int BirimId { set; get; } //Foreign Key
 public virtual Birim Birim { set; get; } //Foreign Key
}

public class Telefon
{
 public int Id { set; get; }
 public string TelefonNumarası { set; get; }
 public string TelefonTürü { set; get; }
}
```

Yukarıdaki sınıfların veri tabanındaki karşılığı aşağıdaki şekilde verilmiştir. **Server Explorer** üzerinde **Kişiler** tablosu sağ tıklanıp **Open Table Definition** seçildiğinde tablonun SQL kodu aşağıdaki gibi olduğu görülecektir;

```
CREATE TABLE [dbo].[Kişiler] (
 [Id] INT IDENTITY (1, 1) NOT NULL,
 [Adı] NVARCHAR (MAX) NOT NULL,
 [Maaş] DECIMAL (18, 2) NOT NULL,
 [BirimId] INT NOT NULL,
 CONSTRAINT [PK_Kişiler] PRIMARY KEY CLUSTERED ([Id] ASC),
 CONSTRAINT [FK_Kişiler_Birim_BirimId] FOREIGN KEY ([BirimId]) REFERENCES [dbo].[Birim] ([Id])
 → ON DELETE CASCADE
);
```

## 23.5 Önce Kod Veri Öznitelikleri

Veri öznitelikleri (**data annotations**), EF Core'ü veri tabanında tabloları ve kolonları oluştururken ve işlerken yönlendirir. Aşağıda çok kullanılan özniteliklere yer verilmiştir<sup>1</sup>.

### §23.5.1 [Column] Özniteliği

EF'ye, özelliğin adını kullanmak yerine belirli bir sütun adını kullanmasını söyler.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
 public int Id { set; get; }
 [Column("Adı ve Soyadı")] //Kolon adı Adı veyine "Adı ve Soyadı" olsun
 public string Adı { set; get; }
}
```

### §23.5.2 [Required] Özniteliği

Bir etki alanı sınıfının (BEC) bir özelliğine uygulandığında, veri tabanı sütunu **NOT NULL** olarak oluşturacaktır.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
 public int Id { set; get; }

 [Required] //Adı Girilmeden Kişiye kayıt atılmaz!.
 public string Adı { set; get; }
}
```

### §23.5.3 [MaxLength] ve [MinLength] Öznitelikleri

[MaxLength(int)] veya [MinLength(int)] özniteliği bir sınıfının (BEC) metin (**string**) alanına veya metin özelliğine uygulanabilir. EF Core, bir sütunun boyutunu belirtilen değere ayarlayacaktır.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
 public int Id { set; get; }
 [MinLength(3), MaxLength(100)] //Adı enz 3 krakter en fazla 100 karakter olabilir!.
 public string Adı { set; get; }
}
```

### §23.5.4 [ForeignKey(string)] Özniteliği

Aşağıdaki Örneği ele alalım;

```
using System.ComponentModel.DataAnnotations.Schema;
public class Müşteri
{
 public int Id { get; set; }
 [Required, MaxLength(100)]
```

<sup>1</sup><https://learn.microsoft.com/en-us/ef/ef6/modeling/code-first/data-annotations>



```

public string MüşteriAdı { get; set; }
public int PostaAdresID { get; set; }
[ForeignKey("PostaAdresID")]
public virtual Adres PostaAdresi { get; set; }
}
public class Adres
{
 public int Id { get; set; }
 public string Sokak { get; set; }
 public string DışKapıNo { get; set; }
 public string İçKapıNo { get; set; }
 public string İlçe { get; set; }
 public string İl { get; set; }
 public long UlusalAdresKodu { get; set; }
}

```

`ForeignKey` nitelikleri olmadan, EF Core bunları karıştırabilir ve `PostaAdresi` alırken başka `ID` içeren özellikleri kullanabilir. Bunu önlemek için bu nitelik kullanılır.

### §23.5.5 [InverseProperty(string)] Özniteliği

`InverseProperty`, iki varlık arasında birden fazla iki yönlü ilişki olduğunda iki yönlü ilişkileri tanımlamak için kullanılabilir. EF Core'e diğer taraftaki özelliklerle hangi gezinme (`navigation`) özelliklerini eşleştirmesi gerektiğini söyler. EF Core, iki varlık arasında birden fazla çift yönlü ilişki olduğunda diğer taraftaki hangi gezinme özelliğinin hangi özelliklerle eşleştiğini bilmez. İlgili sınıftaki karşılık gelen gezinme özelliğinin adını parametresi olarak alması gerekir.

```

using System.ComponentModel.DataAnnotations; //gerekli
public class Birim
{
 public int Id { set; get; }
 public string BirimAdı { set; get; }
 public virtual ICollection<Kişi> AsılPersonel { set; get; }
 public virtual ICollection<Kişi> GörevliPersonel { set; get; }
}
public class Kişi
{
 public int Id { set; get; }
 public string Adı { set; get; }
 [InverseProperty("AsılPersonel")]
 public virtual Birim AnaBirim { get; set; }
 [InverseProperty("GörevliPersonel")]
 public virtual Birim GörevliOlduğuBirim { get; set; }
}

```

Ağaç yapısında nesneler için bu nitelik kullanılabilir;

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
public class AğaçDüğümü
{
 public int ID { get; set; }
 public int ParentID { get; set; }
 [ForeignKey("ParentID")]
 public AğaçDüğümü EbeveynDüğüm { get; set; }
 [InverseProperty("ParentNode")]
 public virtual ICollection<AğaçDüğümü> ÇocukDüğümler { get; set; }
}

```

### §23.5.6 [ComplexType] Özniteliği

Alan odaklı tasarımda DDD (**Domain Driven Design**) değer nesneleri (`value object`) gibi karmaşık tipler kendi başlarına izlenemezler ancak bir varlığın parçası olarak izlenirler. Böyle durumda bu nitelik kullanılır. Veri tabanını oluşturan kod içeriği aşağıda verilmiştir;

```
[ComplexType]
public class BlogDetay
{
 public DateTime? OluşturmaZamanı { get; set; }
 [MaxLength(250)]
 public string Açıklama { get; set; }
}
public class Blog
{
 public int Id { get; set; }
 public string Başlık { get; set; }
 public string İçerik { get; set; }
 public BlogDetay Detay { get; set; }
}
```

### §23.5.7 [Range(min,max)] Özniteliği

İş varlığına ilişkin sayısal alanlarda sınır belirlemek için kullanılır;

```
public class MezuniyetDerecesi
{
 public int DereceID { get; set; }
 [Range(0, 4)]
 public Nullable<decimal> Derece { get; set; }
}

public class Ogrenci
{
 public int Id { get; set; }
 [Required, MaxLength(50)]
 public string Adı { get; set; }
 [Required, MaxLength(50)]
 public string Soyadı { get; set; }
 [EmailAddress, MaxLength(150)]
 public string EPosta { get; set; }
 public MezuniyetDerecesi MezuniyetDerecesi { get; set; }
}
```

### §23.5.8 [NotMapped] Özniteliği

Önce kod yaklaşımında EF Core, desteklenen bir veri türünde olan her genel özellik için bir sütun oluşturur. [NotMapped] niteliği, bir veri tabanı tablosunda sütun istemediğimiz tüm özelliklere uygulanmalıdır.

```
public class Öğrenci
{
 public int Id { get; set; }
 [Required, MaxLength(50)]
 public string Adı { get; set; }
 [Required, MaxLength(50)]
 public string Soyadı { get; set; }

 public string TamAdı => string.Format("{0} {1}", Adı, Soyadı);

 [Required, MaxLength(150)]
 public string EPosta { get; set; }
 [NotMapped]
 public float Ortalama { set; get; }
}
```

### §23.5.9 [Table] Özniteliği

İş varlığına ilişkin veri tabanında tablo adı belirlemek için kullanılır.

```
[Table("Personel")]
public class Kişi
{
}
```

```

 public int Id { set; get; }
 public string Ad1 { set; get; }
}

```

### §23.5.10 [Index] Özniteliği

Veri tabanında oluşan kolonlar için dizin (**index**) oluşturmak için kullanılır;

```

public class Kişi
{
 public int Id { set; get; }
 public string Ad1 { set; get; }

 [Index]
 public int Yaş { set; get; }
}

```

İstenirse dizine ad da verilebilir;

```

[Index("IX_Kişi_Yaş1")]
public int Yaş { set; get; }

```

İstenirse tekil (**unique**) dizin de oluşturulabilir;

```

[Index(IsUnique = true)]
public int Yaş { set; get; }

```

Ayrıca birden fazla kolon kullanılarak da dizin oluşturulabilir;

```

[Index("IX_Kişi_AdveYaş",1)]
public string Ad1 { set; get; }
[Index("IX_Kişi_AdveYaş ",2)]
public int Yaş { set; get; }

```

### §23.5.11 [Key] Özniteliği

**Anahtar** (**key**), bir veri tabanı tablosundaki her satırın (kaydın) benzersiz bir şekilde tanımlayan bir alandır. Varlık sınıfında **ID** içeren bir özellik yoksa bu nitelik kullanılabilir;

```

public class Personel
{
 [Key]
 public int SicilNo { set; get; }
 public string Ad1 { set; get; }
}

```

Aslında bu niteliği kullanmadan aşağıdaki gibi yapılabilirdi;

```

public class Personel
{
 public int PersonelId { set; get; } //Personel ID anahtar olarak görülür
 public string Ad1 { set; get; }
}

```

### §23.5.12 [StringLength(int)] Özniteliği

Varlık nesnesinde metinlere karakter sınırlaması getirmek için kullanılır;

```

public class Personel
{
 public int PersonelId { set; get; }

 [StringLength(100)]
 public string Ad1 { set; get; }
}

```

### §23.5.13 [DatabaseGenerated] Özniteliği

Bazı durumlarda anahtar alanın otomatik artan bir kolon olması istenmez. Aşağıdaki **İl** varlığını düşünelim;

```

public class İl
{

```

```

 public int İlId { get; set; } // İlId veritabanında otomatik artıtılan (IDENTITY)
 public string İlAdı { get; set; }
}

```

Eğer il trafik kodlarını **anahtar** (key) olarak otomatik artmayan bir sütun istiyorsak varlığı aşağıdaki şekilde tanımlamalıyız;

```

public class İl
{
 [Key]
 [DatabaseGenerated(DatabaseGeneratedOption.None)]
 public int TrafikKodu { get; set; }
 public string İlAdı { get; set; }
}

```

Veri tabanında hesaplanan bir kolon varsa da bu nitelik kullanılabilir. Aşağıdaki gibi bir tablo oluşturmak istiyorsak;

```

CREATE TABLE [Personel] (
 [Adı] varchar(100) PRIMARY KEY,
 [DoğumTarihi] Date NOT NULL,
 [Yaş] AS DATEDIFF(year, [DoğumTarihi], GETDATE())
)

```

Bu tabloya ilişkin varlığı aşağıdaki gibi kodlamalıyız;

```

[Table("Personel")]
public class Kişi
{
 [Key, StringLength(100)]
 public string Adı { get; set; }
 public DateTime DoğumTarihi { get; set; }

 [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
 public int Yaş { get; set; }
}

```

## 23.6 Mevcut Veri Tabanıyla Önce Kod

**Önce kod** (code first) yaklaşımıyla hali hazırda var olan tablolar içeren veri tabanı ile çalışılabilir. Bunun için içerisinde il listesi olan bir veri tabanını `SQLServerMevcut.mdf` örnek alalım. Veri tabanında aşağıdaki tablo var olsun;

```

CREATE TABLE [dbo].[İller] (
 [İlTrafikKodu] INT NOT NULL,
 [İlAdı] NVARCHAR (MAX) NOT NULL,
 CONSTRAINT [PK_İller] PRIMARY KEY CLUSTERED ([İlTrafikKodu] ASC)
);

```

Bu tablonun içinde de il listemiz işlemiş olsun. Bu tabloyu kullanacak önce kod projesi için oluşacak uygulamanın yapısı aşağıda verilmiştir;

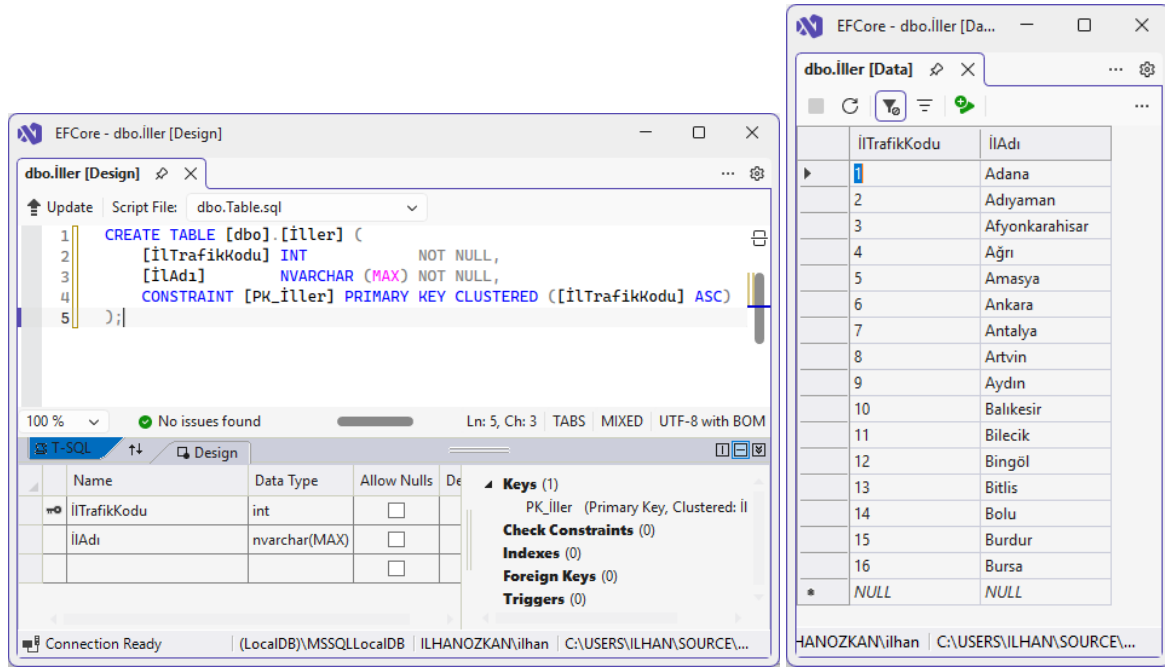
```

EFCoreUygulama/ → Çözüm (Solution)
├── CodeFirst3/ → Proje (Console Application)
│ ├── SQLServerMevcut.mdf → MSSQL Veritabanı (Data)
│ ├── SQLServerMevcut_log.ldf → MSSQL Veritabanı (Log)
│ └── Program.cs → Ana (Main) Program

```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File** → **New** → **Project...** seçilerek **EFCoreUygulama** adlı çözüm içinde **CodeFirst3** adlı ilk projemizi **Console Application** olarak oluşturulur.
2. **Visual Studio** üzerinde **Tools** → **Connect To Database** tıklanarak *projenin klasöründe* **SQLServerMevcut.mdf** adında bir *Microsoft SQL Server LocalDB* veritabanı oluşturulur.
3. **Visual Studio** üzerinde **Tools** → **Server Explorer** açılarak 23.4 şeklinde gösterilen tablo oluşturulur ve içerisine birkaç kayıt atılır. **Maksat veritabanı önce hazırlanmış olsun.**
4. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır; **Microsoft.EntityFrameworkCore** ile **Microsoft.EntityFrameworkCore.SqlServer** paketleri yüklenir.



(a) Mevcut Veritabanında Tablo Oluşturma

(b) Oluşturulan Tabloya Veri Girme

Şekil 23.4: SQLServerMevcut.mdf Veritabanında Önceden Oluşturulan İller Tablosu

5. Proje ile oluşan `Program.cs` dosyasını aşağıdaki şekilde değiştirilir;

```
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\"));
var db = Path.Combine(projeKlasörü, "SQLServerMevcut.mdf");

string connectionString = $"Server=(LocalDB)\MSSQLLocalDB; AttachDbFilename={db};
 Integrated Security=True;Connect Timeout=30;";
var options = new DbContextOptionsBuilder<AppDbContext>()
 .UseSqlServer(connectionString)
 .Options;

using (var context = new AppDbContext(options))
{
 var il = context.İller.FirstOrDefault();
 if (il != null)
 {
 Console.WriteLine($"İl: {il.İlAdı}, Trafik Kodu: {il.İlTrafikKodu}");
 //İl: Adana, Trafik Kodu: 1
 }
 else
 {
 Console.WriteLine("İl bulunamadı.");
 }
 il.İlAdı = "Adana 1 Numara"; //Veri Tabanında "Adana", "Adana 1 Numara" olarak
 //değiştirilir.
 context.SaveChanges();
 context.Database.CloseConnection();
}

public class AppDbContext : DbContext
{
 public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
}
```

```

 public DbSet<İl> İller { set; get; }
 }

 public class İl
 {
 [Key]
 public int İlTrafikKodu { get; set; }
 public string İlAdı { get; set; }
 }

```

6. Program çalıştırıldığında daha önce hazırlanmış ve içi dolu olan **İller** tablosundan veri çekerek gösterecektir. Ayrıca Veri Tabanında **İlTrafikKodu, 1** olan "Adana", "Adana 1 Numara" olarak değişecektir.

## 23.7 Veri Göçü

Gerçek olarak uygulanan projelerde, özellikler uygulandıkça veri modelleri değişir: yeni varlıklar veya özellikler eklenir ve kaldırılır ve veri tabanı şemalarının uygulama ile senkronize tutulabilmesi için buna göre değiştirilmesi gerekir. EF Core'daki **göç (migration)** özelliği, veri tabanındaki verileri korurken veri tabanı şemasını uygulamanın veri modeliyle senkronize tutmak için artımlı olarak güncellemenin bir yolunu sağlar.

EF Core'daki göç özelliğini kullanabilmek için; yukarıdaki gibi **daha önce EF Core** ile yönetilmiş **veri tabanı nesnelerinizi içeren** geçerli bir **DbContext** içeren bir uygulamanız olması gerekir. Göç özelliklerini kullanmak için EF Core ile yönetilen bir proje oluşturalım. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```

EFCoreUygulama/ → Çözüm (Solution)
├─ CodeFirst4/ → Proje (Console Application))
│ └─ DataBaseFirst.mdf → MSSQL Veritabanı (Data)
│ └─ DataBaseFirst_log.ldf → MSSQL Veritabanı (Log)
└─ Program.cs → Ana (Main) Program

```

Projeyi oluşturmak için;

1. Visual Studio üzerinde **File→New→Project...** seçilerek **EFCoreUygulama** adlı çözüm içinde **CodeFirst4** adlı ilk projemizi **Console Application** olarak oluşturulur.
2. **Solution Explorer** üzerinde projemize sağ tıklanarak **Add→New Item...** seçilir ve **DataBaseFirst.mdf** adında bir *Microsoft SQL Server LocalDB* veritabanı *proje klasöründe* oluşturulur.
3. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır ve **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.SqlServer** paketleri yüklenir.
4. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```

using (var context = new AppDbContext())
{
 context.Database.EnsureCreated();
 var iller = context.İller.ToList();
 Console.WriteLine("İller:");
 foreach (var il in iller)
 {
 Console.WriteLine($"Trafik Kodu: {il.İlTrafikKodu}, İl Adı: {il.İlAdı}");
 }
 context.Database.CloseConnection();
}

public class AppDbContext : DbContext
{
 protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
 {
 var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
 var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
 var db = Path.Combine(projeKlasörü, "DataBaseFirst.mdf");
 string connectionString = $"Server=(LocalDB)\\MSSQLLocalDB;
 → AttachDbFilename={db};Integrated Security=True;Connect Timeout=30;";
 }
}

```

```

 optionsBuilder.UseSqlServer(connectionString);
 }
 protected override void OnModelCreating(ModelBuilder modelBuilder)
 {
 modelBuilder.Entity<Personel>()
 .HasOne(p => p.Adres)
 .WithMany()
 .HasForeignKey(p => p.PostaAdresID);
 modelBuilder.Entity<Adres>()
 .HasOne(a => a.İl)
 .WithMany()
 .HasForeignKey(a => a.İlTrafikKodu);
 modelBuilder.Entity<İl>().HasData(
 new İl { İlTrafikKodu = 1, İlAdı = "Adana" },
 new İl { İlTrafikKodu = 2, İlAdı = "Adıyaman" },
 new İl { İlTrafikKodu = 3, İlAdı = "Afyonkarahisar" },
 new İl { İlTrafikKodu = 4, İlAdı = "Ağrı" },
 new İl { İlTrafikKodu = 5, İlAdı = "Amasya" },
 new İl { İlTrafikKodu = 6, İlAdı = "Ankara" },
 new İl { İlTrafikKodu = 7, İlAdı = "Antalya" },
 new İl { İlTrafikKodu = 8, İlAdı = "Artvin" },
 new İl { İlTrafikKodu = 9, İlAdı = "Aydın" },
 new İl { İlTrafikKodu = 10, İlAdı = "Balıkesir" },
 new İl { İlTrafikKodu = 11, İlAdı = "Bilecik" },
 new İl { İlTrafikKodu = 12, İlAdı = "Bingöl" },
 new İl { İlTrafikKodu = 13, İlAdı = "Bitlis" },
 new İl { İlTrafikKodu = 14, İlAdı = "Bolu" },
 new İl { İlTrafikKodu = 15, İlAdı = "Burdur" },
 new İl { İlTrafikKodu = 16, İlAdı = "Bursa" }
);
 }

 public DbSet<Personel> Personel { set; get; }
 public DbSet<İl> İller { set; get; }
}
public class Personel
{
 public int PersonelId { get; set; }
 [StringLength(100)]
 public string Adı { get; set; }
 public string Soyadı { get; set; }
 public int PostaAdresID { get; set; }
 [ForeignKey("PostaAdresID")]
 public virtual Adres Adres { get; set; }
}
public class Adres {
 public int AdresId { get; set; }
 [StringLength(200)] //Cadde-Sokak-Bulvar-Meydan
 public string CSBM { get; set; }
 [StringLength(10)]
 public string DışKapıNo { get; set; }
 [StringLength(10)]
 public string İçKapıNo { get; set; }
 [StringLength(100)]
 public string İlçe { get; set; }
 [StringLength(50)]
 public int İlTrafikKodu { get; set; }
 [ForeignKey("İlTrafikKodu")]
 public virtual İl İl { get; set; }
 public long PostaKodu { get; set; }
}

```



```
public class İl
{
 [Key]
 [DatabaseGenerated(DatabaseGeneratedOption.None)]
 public int İlTrafikKodu { get; set; }
 public string İlAdı { get; set; }
}
```

5. Proje bir kez çalıştırılarak **EF Core** tarafından veri tabanında ilgili tablolar ve verilerin oluşması sağlanır. Burada dikkat edilmesi gereken **AppDbContext** sınıfının yapıcısını **parametresiz olarak kullanmaktır**. Bunu yapmak için **onConfiguring()** yöntemi bu nedenle bu sınıfa bulunur. Ayrıca il listesini de bir başka veritabanına göç sırasında oluşturmak istiyorsak **OnModelCreating()** yöntemini de ekleyip göç edilen veri tabanına istediğimiz verileri kaydedebiliriz.

6. Projemizin ileride olabilecek yeni veritabanlarına göç etmesini istiyorsak; **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır; **Microsoft.EntityFrameworkCore.Tools** ve **Microsoft.EntityFrameworkCore.Design** paketleri yüklenir.

7. Ardından projemizde mevcut veri tabanının ve modelin ilk anlık görüntüsünü saklamak için; **Visual Studio** üzerinde **Tools**→**NuGet Package Manager**→**Package Manager Console** tıklanır ve açılan pencerede **Add-Migration Başlangıç** komutu yazılır ve çalıştırılır. Burada dikkat edilmesi gereken göç edilecek projenin varsayılan proje (**default project**) ayarının mevcut projemizi (**CodeFirst4**) göstermesidir;

```
PM> Add-Migration Başlangıç
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

**.NET Core CLI** ile oluşturulan projelerde bu işlemi yapabilmek için **proje klasöründe** aşağıdaki komut çalıştırılır.

```
$ dotnet ef migrations add Başlangıç
```

8. Komut sonrasında projemizde **Migrations** adında klasör oluşur. Bu klasör içinde veritabanının anlık görüntüsünden yeni bir veritabanı oluşturacak olan **20250530083726\_Başlangıç.cs** (Dosya Adı:Zaman damgası ve Göç adı) adlı göç dosyası oluşur.

Ayrıca varlıklarımızın o anki bir fotoğrafı olan **AppDbContextModelSnapshot.cs** dosyası bulunur. Projenin yapısı aşağıdaki gibi olur;

```
EFSQLUygulama/ → Çözüm (Solution)
├── CodeFirst4/ → Proje (Console Application)
│ ├── Migrations/ → Klasör
│ │ ├── 20250530122325_Başlangıç.cs → Zaman Damgalı Göç
│ │ └── AppDbContextModelSnapshot.cs → AppDbContextModel Yedeği
│ ├── DataBase4.mdf → MSSQL Veritabanı (Data)
│ ├── DataBase4.ldf → MSSQL Veritabanı (Log)
│ └── Program.cs → Ana (Main) Program
```

9. **Migrations** klasöründe oluşan ilk dosya **20260429054001\_Başlangıç.cs** göç dosyası aslında anlık görüntü alınan veritabanını, bir başka veri tabanına geçtiğimizde bu anlık görüntüyü yeni veritabanında oluşturacak kod dosyasıdır. Şimdi **20260429054001\_Başlangıç.cs** göç dosyasının içeriğini inceleyelim;

```
using Microsoft.EntityFrameworkCore.Migrations;
```

```
#nullable disable
```

```
#pragma warning disable CA1814 // Prefer jagged arrays over multidimensional
```

```
namespace CodeFirst4.Migrations
{
 /// <inheritdoc />
 public partial class Başlangıç : Migration
 {
 /// <inheritdoc />
 protected override void Up(MigrationBuilder migrationBuilder)
 {
 migrationBuilder.CreateTable(
```

```

name: "İller",
columns: table => new
{
 İlTrafikKodu = table.Column<int>(type: "int", nullable: false),
 İlAdı = table.Column<string>(type: "nvarchar(max)", nullable: false)
},
constraints: table =>
{
 table.PrimaryKey("PK_İller", x => x.İlTrafikKodu);
});

migrationBuilder.CreateTable(
name: "Adres",
columns: table => new
{
 AdresId = table.Column<int>(type: "int", nullable: false)
 .Annotation("SqlServer:Identity", "1, 1"),
 CSBM = table.Column<string>(type: "nvarchar(200)", maxLength: 200,
 → nullable: false),
 DışKapıNo = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
 → nullable: false),
 İçKapıNo = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
 → nullable: false),
 İlçe = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
 → nullable: false),
 İlTrafikKodu = table.Column<int>(type: "int", maxLength: 50, nullable:
 → false),
 PostaKodu = table.Column<long>(type: "bigint", nullable: false)
},
constraints: table =>
{
 table.PrimaryKey("PK_Adres", x => x.AdresId);
 table.ForeignKey(
 name: "FK_Adres_İller_İlTrafikKodu",
 column: x => x.İlTrafikKodu,
 principalTable: "İller",
 principalColumn: "İlTrafikKodu",
 onDelete: ReferentialAction.Cascade);
});

migrationBuilder.CreateTable(
name: "Personel",
columns: table => new
{
 PersonelId = table.Column<int>(type: "int", nullable: false)
 .Annotation("SqlServer:Identity", "1, 1"),
 Adı = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
 → nullable: false),
 Soyadı = table.Column<string>(type: "nvarchar(max)", nullable: false),
 PostaAdresID = table.Column<int>(type: "int", nullable: false)
},
constraints: table =>
{
 table.PrimaryKey("PK_Personel", x => x.PersonelId);
 table.ForeignKey(
 name: "FK_Personel_Adres_PostaAdresID",
 column: x => x.PostaAdresID,
 principalTable: "Adres",
 principalColumn: "AdresId",
 onDelete: ReferentialAction.Cascade);
});

```

```

migrationBuilder.InsertData(
 table: "İller",
 columns: new[] { "İlTrafikKodu", "İlAdı" },
 values: new object[,]
 {
 { 1, "Adana" },
 { 2, "Adıyaman" },
 { 3, "Afyonkarahisar" },
 { 4, "Ağrı" },
 { 5, "Amasya" },
 { 6, "Ankara" },
 { 7, "Antalya" },
 { 8, "Artvin" },
 { 9, "Aydın" },
 { 10, "Balıkesir" },
 { 11, "Bilecik" },
 { 12, "Bingöl" },
 { 13, "Bitlis" },
 { 14, "Bolu" },
 { 15, "Burdur" },
 { 16, "Bursa" }
 });

migrationBuilder.CreateIndex(
 name: "IX_Adres_İlTrafikKodu",
 table: "Adres",
 column: "İlTrafikKodu");

migrationBuilder.CreateIndex(
 name: "IX_Personel_PostaAdresID",
 table: "Personel",
 column: "PostaAdresID");
}

/// <inheritdoc />
protected override void Down(MigrationBuilder migrationBuilder)
{
 migrationBuilder.DropTable(
 name: "Personel");

 migrationBuilder.DropTable(
 name: "Adres");

 migrationBuilder.DropTable(
 name: "İller");
}
}
}

```

Gördüğünüz gibi, `Up()` yöntemi `İller`, `Adres` ve `Personel` adlı üç tablo ve alanları oluşturulur. Ek olarak, tablolar arasındaki ilişkiler ve veritabanı dizinler (`index`) de oluşturulur. Diğer tarafta `Down()` yöntemi göç aktivitelerini (`Up` yöntemi ile yapılanları) tersine çevirmeye çalışır. Göçünüz konusunda kendinize güveniyorsanız, göçü istediğiniz veri tabanına uygulayabilirsiniz.

10. `Migrations` klasöründe oluşan ikinci dosya `AppDbContextModelSnapshot.cs` dosyası da mevcut varlıklarımızın anlık görüntüsünü tutan bir dosyadır. Bir başka veri tabanına geçtiğimizde veritabanını bu anlık görüntüsünü ile kullanmak için oluşturulur. Şimdi `AppDbContextModelSnapshot.cs` göç dosyasının içeriğini inceleyelim;

```

// <auto-generated />
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Infrastructure;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;

```

```
#nullable disable
```

```
namespace CodeFirst4.Migrations
{
 [DbContext(typeof(AppDbContext))]
 partial class AppDbContextModelSnapshot : ModelSnapshot
 {
 protected override void BuildModel(ModelBuilder modelBuilder)
 {
 #pragma warning disable 612, 618
 modelBuilder
 .HasAnnotation("ProductVersion", "10.0.7")
 .HasAnnotation("Relational:MaxIdentifierLength", 128);

 SqlServerModelBuilderExtensions.UseIdentityColumns(modelBuilder);

 modelBuilder.Entity("Adres", b =>
 {
 b.Property<int>("AdresId")
 .ValueGeneratedOnAdd()
 .HasColumnType("int");

 SqlServerPropertyBuilderExtensions.UseIdentityColumn(b.Property<int>("AdresId"));

 b.Property<string>("CSBM")
 .IsRequired()
 .HasMaxLength(200)
 .HasColumnType("nvarchar(200)");

 b.Property<string>("DışKapıNo")
 .IsRequired()
 .HasMaxLength(10)
 .HasColumnType("nvarchar(10)");

 b.Property<long>("PostaKodu")
 .HasColumnType("bigint");

 b.Property<int>("İlTrafikKodu")
 .HasMaxLength(50)
 .HasColumnType("int");

 b.Property<string>("İlçe")
 .IsRequired()
 .HasMaxLength(100)
 .HasColumnType("nvarchar(100)");

 b.Property<string>("İçKapıNo")
 .IsRequired()
 .HasMaxLength(10)
 .HasColumnType("nvarchar(10)");

 b.HasKey("AdresId");

 b.HasIndex("İlTrafikKodu");

 b.ToTable("Adres");
 });

 modelBuilder.Entity("Personel", b =>
 {
 b.Property<int>("PersonelId")
 .ValueGeneratedOnAdd()

```

```
.HasColumnType("int");

SqlServerPropertyBuilderExtensions
 .UseIdentityColumn(b.Property<int>("PersonelId"));

b.Property<string>("Adı")
 .IsRequired()
 .HasMaxLength(100)
 .HasColumnType("nvarchar(100)");

b.Property<int>("PostaAdresID")
 .HasColumnType("int");

b.Property<string>("Soyadı")
 .IsRequired()
 .HasColumnType("nvarchar(max)");

b.HasKey("PersonelId");

b.HasIndex("PostaAdresID");

b.ToTable("Personel");
});

modelBuilder.Entity("İl", b =>
{
 b.Property<int>("İlTrafikKodu")
 .HasColumnType("int");

 b.Property<string>("İlAdı")
 .IsRequired()
 .HasColumnType("nvarchar(max)");

 b.HasKey("İlTrafikKodu");

 b.ToTable("İller");

 b.HasData(
 new
 {
 İlTrafikKodu = 1,
 İlAdı = "Adana"
 },
 new
 {
 İlTrafikKodu = 2,
 İlAdı = "Adıyaman"
 },
 new
 {
 İlTrafikKodu = 3,
 İlAdı = "Afyonkarahisar"
 },
 new
 {
 İlTrafikKodu = 4,
 İlAdı = "Ağrı"
 },
 new
 {
 İlTrafikKodu = 5,
 İlAdı = "Amasya"
 }
);
});
```

```

 },
 new
 {
 İlTrafikKodu = 6,
 İlAdı = "Ankara"
 },
 new
 {
 İlTrafikKodu = 7,
 İlAdı = "Antalya"
 },
 new
 {
 İlTrafikKodu = 8,
 İlAdı = "Artvin"
 },
 new
 {
 İlTrafikKodu = 9,
 İlAdı = "Aydın"
 },
 new
 {
 İlTrafikKodu = 10,
 İlAdı = "Balıkesir"
 },
 new
 {
 İlTrafikKodu = 11,
 İlAdı = "Bilecik"
 },
 new
 {
 İlTrafikKodu = 12,
 İlAdı = "Bingöl"
 },
 new
 {
 İlTrafikKodu = 13,
 İlAdı = "Bitlis"
 },
 new
 {
 İlTrafikKodu = 14,
 İlAdı = "Bolu"
 },
 new
 {
 İlTrafikKodu = 15,
 İlAdı = "Burdur"
 },
 new
 {
 İlTrafikKodu = 16,
 İlAdı = "Bursa"
 }
});

modelBuilder.Entity("Adres", b =>
{
 b.HasOne("İl", "İl")
 .WithMany()

```

```

 .HasForeignKey("İlTrafikKodu")
 .OnDelete(DeleteBehavior.Cascade)
 .IsRequired();

 b.Navigation("İl");
 });

 modelBuilder.Entity("Personel", b =>
 {
 b.HasOne("Adres", "Adres")
 .WithMany()
 .HasForeignKey("PostaAdresID")
 .OnDelete(DeleteBehavior.Cascade)
 .IsRequired();

 b.Navigation("Adres");
 });
#pragma warning restore 612, 618
 }
}
}

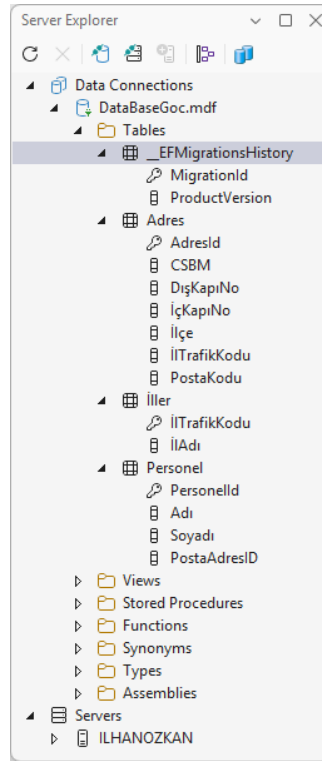
```

11. Şimdi mevcut projemizde veritabanını değiştirerek mevcut projemizde kullandığımız veri göçü yapacağız. Mevcut projemizi açıp;
12. **Visual Studio** üzerinde projemize sağ tıklanarak **Tools→Connect To Database...** seçilir ve göç edilecek **DatabaseGoc.mdf** adında bir *Microsoft SQL Server LocalDB* veritabanı *proje klasöründe* oluşturulur.
13. **Program.cs** içinde **yeni veri tabanı bağlantımızı sağlayacak** değişikliği **AppDbContext** sınıfının veri tabanına bağlanmayı sağlayan **onConfiguring()** yönteminde yapıyoruz.  
**protected override void OnConfiguring**(DbContextOptionsBuilder optionsBuilder)  
 {  
     **var** çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;  
     **var** projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));  
     **var** db = Path.Combine(projeKlasörü, "DatabaseGoc.mdf"); // Göç edilecek Veritabanı  
     **string** connectionString = \$"Server=(LocalDB)\MSSQLLocalDB; AttachDbFilename={db};  
         Integrated Security=True;Connect Timeout=30;";  
     optionsBuilder.UseSqlServer(connectionString);  
 }
14. **Visual Studio** üzerinde **Tools→NuGet Package Manager→Package Manager Console** tıklanır ve açılan pencerede **Update-Database** komutu yazılır ve çalıştırılır;  
 PM> Update-Database  
 Build started...  
 Build succeeded.  
 Acquiring an exclusive lock for migration application. See ...  
 Applying migration '20260429054001\_Başlangıç'.  
 Done.  
 PM>
15. Artık projemiz için yeni veri tabanında **EF Core** için tablolar otomatik olarak oluşturmuştur (Şekil 23.5). **.NET Core CLI** ile oluşturulan projelerde bu işlemi yapabilmek için **proje klasöründe** aşağıdaki komut çalıştırılır.  
 \$ dotnet ef database update

Bir **EF Core** projesinde **göç (migration)** işlemleri şu şekilde kullanılır;

- ♦ Bir veri modeli değişikliği yapıldığında, geliştirici veri tabanı şemasını senkronize tutmak için gerekli güncellemeleri açıklayan karşılık gelen bir göç eklemek için **EF Core Tools** kullanır. **EF Core**, farklılıkları belirlemek için geçerli modeli eski modelin bir anlık görüntüsüyle karşılaştırır ve göç kaynak dosyaları oluşturur; dosyalar projenizin kaynak denetiminde diğer kaynak dosyaları gibi izlenebilir.
- ♦ Yeni bir göç oluşturulduktan sonra, çeşitli şekillerde bir veri tabanına uygulanabilir. **EF Core**, uygulanan tüm göçleri özel bir geçmiş tablosunda kaydeder ve hangi göçlerin uygulandığını ve hangilerinin uygulanmadığını bilmesini sağlar.
- ♦ **Add-Migration** ve **Update-Database** komutları için aktiviteleri ayarlamak için kullanılacak çeşitli seçenekler mevcuttur. Bunları görmek ve incelemek için;





Şekil 23.5: Göç Sonrası Veri Tabanı

`get-help Add-Migration` ve `get-help Update-Database` komutları kullanılabilir.

Geliştirmenin yapıldığı ortamın dışında çalışan uygulamalar genellikle veri tabanı güncellemeleri (**update**) gerektirir. Veri tabanı yamalarını (**patch**) oluşturmak için `AddMigration` komutunu kullandıktan sonra, güncellemeleri test ve diğer ortamlarda çalıştırmanız gerekir. Bunun yanında uygulamanın müşteri ortamına bağlantı izni yok ya da bu ortamda **Visual Studio** kurulmamış ise bu komutu çalıştıramayız. Bunun çözüm örneğimizde olduğu gibi `DbContext` sınıfına `OnModelCreating` yöntemini eklemektir. Göç sırasında yeni veri tabanında tabloları verilerle doldurma işlemine **veri ile besleme** (**seeding data**) adı verilir.

Şu ana kadar anlatılan göç işlemleri **Package Manager Console** açılarak `Add-Migration` **Başlangıç** ve `Update-Database` gibi komutları elle girerek yaptık. Bunu bir yardımcı program yazarak otomatik olarak da yapabiliriz. `context.Database.Migrate()` yöntemi, **EF Core** içerisinde, önce kod (kızılcode first) yaklaşımıyla oluşturulan model değişikliklerini yani göç (migration) dosyalarını veritabanına otomatik olarak uygulamak için kullanılır. Aşağıda örneği verilmiştir;

```
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

// Main Metodu İçeriği
try
{
 using (var context = new AppDbContext())
 {
 Console.WriteLine("Veritabanı kontrol ediliyor/güncelleniyor...");

 // EnsureCreated() yerine Migrate() kullanıyoruz. Bu komut bekleyen migration varsa
 // → uygular, veritabanı yoksa oluşturur.
 context.Database.Migrate();

 var iller = context.İller.ToList();
 Console.WriteLine("\nKayıtlı İller:");
 foreach (var il in iller)
```

```

 {
 Console.WriteLine($"Trafik Kodu: {il.İlTrafikKodu}, İl Adı: {il.İlAdı}");
 }
 }
}
catch (Exception ex)
{
 Console.WriteLine($"Bir hata oluştu: {ex.Message}");
}

public class AppDbContext : DbContext
{
 public DbSet<Personel> Personel { set; get; }
 public DbSet<İl> İller { set; get; }
 public DbSet<Adres> Adresler { set; get; } // Adres tablosu eksikti, eklendi.

 protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
 {
 var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
 var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
 var db = Path.Combine(projeKlasörü, "DataBaseGoc.mdf");
 string connectionString = $"Server=(LocalDB)\\MSSQLLocalDB;
 ↳ AttachDbFilename={db};Integrated Security=True;Connect
 ↳ Timeout=30;TrustServerCertificate=True;";
 optionsBuilder.UseSqlServer(connectionString);
 }

 protected override void OnModelCreating(ModelBuilder modelBuilder)
 {
 // Fluent API ilişkileri
 modelBuilder.Entity<Personel>()
 .HasOne(p => p.Adres)
 .WithMany()
 .HasForeignKey(p => p.PostaAdresID);

 modelBuilder.Entity<Adres>()
 .HasOne(a => a.İl)
 .WithMany()
 .HasForeignKey(a => a.İlTrafikKodu);

 // Seed Data (Örnek Veriler)
 modelBuilder.Entity<İl>().HasData(
 new İl { İlTrafikKodu = 1, İlAdı = "Adana" },
 new İl { İlTrafikKodu = 6, İlAdı = "Ankara" },
 new İl { İlTrafikKodu = 34, İlAdı = "İstanbul" }
);
 }
}

public class Personel
{
 public int PersonelId { get; set; }
 [StringLength(100)]
 public string Adı { get; set; }
 public string Soyadı { get; set; }
 public int PostaAdresID { get; set; }
 [ForeignKey("PostaAdresID")]
 public virtual Adres Adres { get; set; }
}

public class Adres {
 public int AdresId { get; set; }

```

```

[StringLength(200)]
public string CSBM { get; set; }
[StringLength(10)]
public string DışKapıNo { get; set; }
[StringLength(10)]
public string İçKapıNo { get; set; }
[StringLength(100)]
public string İlçe { get; set; }

public int İlTrafikKodu { get; set; }

[ForeignKey("İlTrafikKodu")]
public virtual İl İl { get; set; }
public long PostaKodu { get; set; }
}

public class İl
{
 [Key]
 [DatabaseGenerated(DatabaseGeneratedOption.None)]
 public int İlTrafikKodu { get; set; }
 public string İlAdı { get; set; }
}

```

Bu yöntem, özellikle üretim (**production**) ortamlarında veya CI/CD süreçlerinde, **Package Manager Console** üzerinden **Update-Database** gibi manuel komut çalıştırmak yerine, uygulamanın açılışında veritabanı şemasını güncel tutmak için kullanılır. Veritabanında **\_\_EFMigrationsHistory** tablosunu kontrol eder, projedeki göç dosyalarıyla karşılaştırır ve eksik olanları çalıştırarak veritabanı şemasını (tablolar, sütunlar vb.) günceller. Manuel SQL yazma veya CLI komutları çalıştırma ihtiyacını ortadan kaldırır. **MigrateAsync()** yöntemi asenkron çalışan versiyonudur. Performansı artırmak için tercih edilebilir. Çoklu sunucu ortamlarında (**load balancer**) aynı anda birden fazla örnek çalışıyorsa, göç (**migration**) çakışmalarını önlemek için dikkatli kullanılmalıdır.

## 23.8 Düşün ve Çöz

- ◊ **Düşün:** ORM (**Object-Relational Mapping**) nedir ve veri erişim katmanını neden kolaylaştırır? ADO.NET ile doğrudan SQL yazmaya kıyasla Entity Framework'ün getirdiği soyutlama maliyeti var mıdır?
- ◊ **Düşün:** 'Önce kod' (**code first**) ile 'önce veri tabanı' (**database first**) yaklaşımları arasındaki farklar nelerdir? Hangi senaryoda hangisi tercih edilir? Mevcut bir veri tabanının olduğu ve mevcut bir kod tabanının olduğu iki farklı senaryo düşününüz.
- ◊ **Düşün:** **DbContext** sınıfının yaşam döngüsü (**lifetime**) neden önemlidir? Bir Web API'de **DbContext**'i **Singleton** olarak kaydetmenin neden sorun yarattığını; **Scoped** olarak kaydetmenin neden doğru olduğunu açıklayınız.
- ◊ **Düşün:** EF Core'un değişiklik takibi (**change tracking**) mekanizması nasıl çalışır? **AsNoTracking()** kullanmanın salt okunur sorgularda performans avantajı nedir? Her zaman **AsNoTracking()** kullanmak doğru mudur?
- ◊ **Düşün:** Göç (**migrations**) sistemi veri tabanı şemasını kodla yönetmenizi sağlar. Bir ekip projesinde göç çakışmaları nasıl oluşur ve nasıl çözülür? Göcü geri alma (**rollback**) işlemi nasıl yapılır?
- ◊ **Düşün:** EF Core'da **Eager Loading** (Include), **Lazy Loading** ve **Explicit Loading** arasındaki farklar nelerdir? N+1 sorgusunu önlemek için hangisi tercih edilir? **Lazy Loading**'in avantajı ve tuzağı nedir?
- ◊ **Düşün:** EF Core ile ham SQL sorgusu (**FromSqlRaw**, **ExecuteSqlRaw**) ne zaman kullanılır? ORM'nin ürettiği SQL'den daha performanslı özel sorgular yazmayı ne zaman düşünmelisiniz?
- ◊ **Çöz:** Öğrenci, Ders ve Kayıt tablolarını içeren bir okul veri tabanı oluşturun (**code first**, SQLite). Göç uygulayın. Bir öğrencinin tüm derslerini listeleyen, yeni ders kaydı yapan ve kaydı silen CRUD yöntemleri yazınız.
- ◊ **Çöz:** EF Core ile bir ürün kataloğu uygulaması geliştirin: Ürün ve Kategori arasında bire-çok ilişki kurun. **Eager Loading** (Include) kullanarak bir kategorideki tüm ürünleri getirin; **AsNoTracking** ile performansı karşılaştırın.
- ◊ **Çöz:** EF Core değişiklik takibini (**change tracking**) gösteren bir senaryo oluşturun: bir nesneyi okuyun, özelliğini değiştirin, **SaveChanges** öncesinde **ChangeTracker.Entries()** ile değişiklikleri listeleyin, ardından kaydedin.

- ◇ **Çöz:** Bir veritabanı tohum verisi (**seed data**) sistemi tasarlayın: `DbContext.OnModelCreating()` içinde Ürün ve Kategori tablolarını örnek verilerle doldurun; her migration uygulandığında bu veriler yoksa eklensin.
- ◇ **Çöz:** EF Core ile bir sipariş yönetimi uygulaması yazın: `Müşteri`, `Sipariş` ve `SiparişKalemi` arasında ilişkiler kurun. İş (**transaction**) kullanarak bir siparişi tüm kalemleriyle atomik olarak ekleyin; herhangi bir kalemde hata olursa geri alma (**rollback**) yapın.

# Dizin

---

- .NET Class Library, 21
- .NET Remoting, 319
- ABCD, 316
- abstract, 107, 109, 118
- abstract class, 107, 125, 144
- abstract factory pattern, 233
- abstract method, 107, 125, 126, 144
- abstraction, 49, 107, 150
- abstraction ladder, 18
- access, 35
- access modifier, 91, 109, 116, 122, 144
- accumulator, 3
- ACID, 316
- actual parameter, 13
- adapter pattern, 244
- ADO.NET, 21, 304
- aggregation, 144, 149, 150
- agile manifesto, 306
- agile model, 305
- alias, 130
- analysis pattern, 232
- anonymous method, 133, 134
- anonymous type, 38
- anti-pattern, 232
- AOT, 161
- architectural model, 303
- architectural pattern, 232
- argument, 13
- arithmetic logic unit, 3
- arithmetic operator, 41
- array, 14, 54, 79, 182
- array size, 79
- arrow operator, 180
- as operator, 128
- assembly, 23, 25, 160
- assembly code, 6
- assignment operator, 45
- association, 144, 148
- asynchronous, 163, 331
- attribute, 160, 390
- attributes, 159, 220
- base case, 97
- base class, 115, 126, 145
- base keyword, 115
- BCL, 21
- BEC, 310, 312
- behavior, 89, 101, 110, 113
- behavioral model, 303
- behavioral patterns, 232, 258
- BigInteger, 74
- binary, 12, 13
- binary executable native code, 23
- BIOS, 7
- bitwise copy, 241
- bitwise operator, 43
- BLC, 312
- boxing, 158, 176, 182, 198
- BREAD, 316
- break statement, 75
- bridge pattern, 256
- buffer overflow, 178
- builder pattern, 238
- built-in attributes, 159
- bus, 1, 3
- business analysis, 302
- BWC, 312
- byte, 2
- call, 14, 94
- call by value, 95
- callback, 176
- calling convention, 100
- calling overhead, 49
- camel case, 36
- capture, 135
- cast, 176
- cast operator, 174
- casting, 173, 174
- catch an exception, 164, 166
- chain of responsibility pattern, 258
- change management, 49
- character literal, 50
- checked, 174
- child class, 115
- CI, 363
- CI/CD, 305, 307
- CIL, 21
- class, 89, 144
- class diagram, 113, 144
- class library, 17, 49
- classic life cycle, 305
- classifier, 89
- client-server, 308
- clock, 2
- closure, 135
- CLR, 21
- CLS, 22
- code block, 31, 35
- code coverage, 379
- code first, 381, 384, 397
- code first with existing database, 384
- code segment, 24
- coefficient, 36
- collection, 182
- COM, 304
- COM+, 304
- comma operator, 71, 93, 98
- command, 223
- command pattern, 260
- compile-time, 38, 39, 55, 161, 162
- compiler, 6, 19, 21
- complexity, 307
- component, 110
- component using, 307
- composite pattern, 246
- composition, 144, 148
- computer program, 6
- concrete, 107, 233
- concrete class, 107, 126
- concurrency, 226
- condition, 60, 69
- conditional choice, 59, 60
- conditional code, 60
- conditional code register, 3
- conditional ternary operator, 67
- connection, 223
- connection string, 222
- console, 7
- const, 37
- const specifier, 120
- constraints, 183
- constructor, 34, 101, 110, 116, 156, 161
- constructor chaining, 104
- constructor overloading, 102
- context class, 382
- contextual keywords, 32
- continue statement, 75
- control abstraction, 107–109
- control expression, 65
- control flow, 59
- control operation, 59, 61
- control structure, 14, 94
- control unit, 3
- convention, 387
- conventions, 390
- copy constructor, 116
- CQRS, 326
- creational patterns, 232
- CRUD, 315
- CSV, 362
- CTS, 21, 22
- current instruction code, 4
- current instruction register, 3
- cursor, 49

- custom attributes, 159
- DAC, 310, 315
- dangling else, 64
- data, 1
- data abstraction, 107, 109
- data annotations, 390, 392, 393
- data consistency, 326
- data memory register, 24
- data pattern, 232
- data segment, 24, 119
- data structure, 14, 79, 94, 182
- data type, 11, 13, 33, 35
- database, 222
- database first, 381
- database per service, 326
- DbContext, 381
- DDD, 394
- de-reference operator, 177, 179
- deadlock, 196
- debug, 28
- decimal convention, 391
- declaration, 35, 176
- decode, 3, 4
- decorator pattern, 249
- deep copy, 241
- default, 91, 101
- default argument, 97
- default constructor, 101, 103
- definition, 35
- delegate, 132, 134
- dependency, 145, 146
- dependency as a parameter, 145
- dependency as an instantiation, 145
- dependency injection, 106, 146, 153, 318
- derived class, 115, 126, 145
- deserialization, 213, 219
- design pattern, 171, 232
- destructor, 188
- destructors, 124
- deviation, 81
- DevOps, 305
- DIP, 153
- directory, 213
- disposable pattern, 124, 354
- distributed application, 319
- distributed programming, 307, 318
- distributed tracing, 327
- distributed transactions, 326
- DNS, 319
- do-while statement, 69
- Docker, 327
- dot operator, 91, 129, 155
- DTO, 314
- dynamic variables, 39
- eager loading, 382
- EDMX, 381
- else code, 62
- empty product, 97
- encapsulation, 107, 109, 110, 150, 161
- encoding, 216
- engineer, 1
- enterprise application, 304
- enterprise software, 303
- Entity Framework, 159
- entry, 270
- enumeration, 40
- escape sequences, 51
- event, 135
- event sourcing, 326
- event-driven programming, 132
- exception, 163, 170, 195
- exception object, 163
- executable code, 23
- execute, 3, 4, 223
- explicit, 176
- explicit conversion, 173
- explicit loading, 383
- expression, 34, 59, 60, 65, 67, 133
- extension method, 125, 202, 203
- facade pattern, 241
- factory method pattern, 235
- fault isolation, 325, 326
- fetch, 3
- field, 90, 101, 120
- field variable, 91
- FIFO, 132, 191, 201
- file, 213
- file-scoped namespaces, 130
- finalizer, 116, 362
- finalizers, 124
- flag, 3
- flag register, 24
- floating point number, 12
- flow, 213
- flowchart, 59
- flyweight pattern, 252
- for statement, 70
- foreach loop, 86
- foreign key, 390, 392
- format specifier, 54–56
- format string, 49
- framework, 110
- FTP, 320
- function, 13
- function pointer, 132
- functional model, 303
- functional programming, 132
- functional requirements, 303
- gant chart, 304
- GC, 18, 21, 29, 34, 124, 176, 177, 198, 339, 345, 346, 359
- general class, 115
- general purpose registers, 24
- generalization, 145
- generic, 182
- generic programming, 182
- global namespace, 130
- goto, 68
- gRPC, 220, 319
- hash table, 189
- heap, 34, 35, 38, 79, 90, 155, 158, 176, 197
- heap segment, 24
- HIC, 312
- high cohesion, 150
- high-level programming language, 9, 10
- HLD, 305
- horizontal scalability, 326
- HTTP, 320
- IDE, 26
- identifier, 35, 189
- identifier definition, 35, 59, 68
- if statement, 60
- if-else statement, 62
- image cache, 362
- immutability, 110
- imperative programming, 15, 89
- implementation pattern, 232
- implicit, 179
- implicit conversion, 173
- index, 79, 396
- indexer, 200
- infinite loop, 76
- information, 1
- information hiding, 107, 109
- inheritance, 113, 116, 145, 148, 150, 155, 160
- initial value, 36
- initialization, 35, 101
- initialize, 156
- input output, 49
- input-output operation, 59
- instance, 89, 91, 101, 107, 124, 125, 144
- instance member, 119
- instantiation, 90, 91, 121
- instruction, 1, 3, 18
- instruction code, 4, 6, 23, 32
- instruction cycle, 3
- instruction set, 4
- integer, 79
- integer literal, 36
- integrated circuit, 1
- interface, 107, 122, 144, 182, 202
- interface implementation, 122

- interface realization, 122, 144, 156
- interface-based design, 318
- interfaces, 291
- interpreter, 19
- interpreter pattern, 285
- invoke, 94
- IP, 319
- IPC, 319
- is operator, 128
- is-a, 160
- ISP, 152
- iteration, 75, 80, 97, 203
- iterator, 185
- iterator pattern, 263
  
- jagged array, 79, 86
- Jakarta EE, 304
- JAX-WS, 319
- JNDI, 304
- join, 209
- JSBC, 304
- JSON, 160, 161, 219, 220
- JSP, 304
- jump, 59, 75
- Just-in-time Compiler, 23
- JWT, 320
  
- Kafka, 319
- key, 189, 396
- kontrol flow, 89
- Kubernetes, 327
  
- label, 68
- lambda expression, 133
- lambda operator, 133
- latency, 327
- layer, 307
- layered architecture, 307
- lazy copy, 241
- lazy evaluation, 186
- lazy initialization, 192
- lazy loading, 383
- leaf class, 126
- library, 110
- LIFO, 24, 191, 345
- link, 35
- linked list, 14
- LINQ, 32, 133, 135, 202
- LINQ extension method, 382
- literal, 36–38
- LLD, 305
- load balancer, 410
- local variable, 35, 93
- log, 222
- logical operator, 44
- logical sequence, 10, 59, 68–70, 76
- logical sequences, 32
- loop, 97
- loop code, 69, 75
  
- loose coupling, 148, 150, 160, 291, 326
- low-level programming, 9, 15
- LPC, 319
- LSP, 152
  
- machine code, 4
- machine language, 6
- main function, 14, 17
- managed code, 23, 93, 176
- managed compilation, 23
- mapping, 384
- matrix, 84
- mediator pattern, 278
- member hiding, 117
- memberwise copy, 241
- memento pattern, 274
- memory, 1
- memory data register, 4
- memory leak, 359
- message brokers, 326
- message queues, 319
- message-passing, 89, 91, 94
- method, 90, 91, 132
- method definition, 93
- method overloading, 99
- method signature, 99
- methos name, 93
- microservices, 325
- microservices architecture, 307
- migration, 399, 407
- mnemonic, 6
- monolithic, 307
- MSIL, 21
- MSMQ, 304
- multi-dimentional array, 85
- multi-threaded, 194
- multi-threading, 193
- mutable, 122
- MVC, 265, 289
- MVP, 301
- MVVM, 301
  
- namespace, 54
- namespaces, 129
- NAOT, 24
- native code, 23, 24
- navigation property, 388
- nested if, 61
- nested loop, 73
- new operator, 34, 90
- new specifier, 117
- non-functional requirements, 304
- NoSQL, 222
- NuGet, 318
- null, 33, 39
- null character, 34, 37
- null coalescing operator, 39
- null pattern, 269
- null reference, 34
  
- nullable, 39, 40, 128
  
- object, 89, 101
- object initializer, 110
- observer pattern, 265, 291
- OCP, 151
- offset, 158
- OMG, 143
- OMT, 143
- OOP, 15, 16, 59, 89, 143
- OOSE, 143
- opcode, 4
- operand, 4, 41
- operating system, 7
- operator, 59
- operator overloading, 119
- operator precedence, 46
- ORM, 160, 381
- overload, 200
- overloaded constructor, 103
- override, 118, 126
- ownership, 146, 148, 149
  
- parameterized constructor, 103
- parent class, 115
- partial class, 127
- PascalCase, 92
- pass by reference, 96
- patch, 408
- pattern, 232
- pattern matching, 128
- pattern oriented programming, 232
- PDC, 313
- peek, 191
- physical sequence, 59, 69, 70, 76
- plugin, 161
- pointer, 35, 79, 132, 176
- polyglot programming, 326
- polymorphism, 150, 174
- pop, 96, 191
- postfix, 42
- precision, 391
- prefix, 42
- primary constructor, 106
- primary key, 388
- primitive data type, 14
- private, 91, 107, 116, 144, 148, 156
- private association, 146, 148
- private class, 115
- processor, 1
- program counter, 3
- programming, 6
- project, 91
- property, 107, 113, 200
- protected, 91, 144
- protobuf, 220, 319
- prototype pattern, 240
- proxy pattern, 254



- public, 91, 107, 144, 148
- public association, 146, 148, 149
- punch card, 9
- range based loop, 86
- range operator, 87
- re-throw, 170, 171
- read-only, 38
- readonly keyword, 120
- readonly specifier, 120
- record, 159
- recursion, 97
- recursive, 97, 98, 214, 246
- recursive discovery, 388
- ref keyword, 96
- reference type, 34, 79, 157
- reflection, 160, 161
- register, 3
- relational loop, 59, 69
- relational pattern, 67
- relationship, 144
- release, 28
- repository pattern, 315
- required specifier, 106
- requirement analysis, 302, 303, 305
- REST, 320
- RESTful API, 315
- RESTful API, 320
- return, 13
- reusing, 49, 232, 307
- role, 122
- root class, 126
- round-trip, 382
- route, 321
- RPC, 319
- RSP, 152
- run-time, 39, 55, 87, 148, 162, 174, 182, 384
- RUP, 304
- saga pattern, 326
- scalability, 307, 326
- scale, 391
- schema, 222
- scope, 36
- scopoe, 35
- SDLC, 304
- sealed, 156
- sealed class, 126
- sealed method, 126
- segment, 24
- sentinel value, 73
- seperation of concern, 150
- sequential operation, 59, 61, 67
- serialization, 213, 218, 219, 317
- server, 308
- service discovery, 326, 327
- set, 79
- shallow copy, 241
- shared code, 116
- shared memory, 319
- shared state, 194
- SIC, 315
- sign, 12
- SignalR, 319
- signature, 122, 132
- signed, 12
- single responsibility, 325
- singleton class, 127
- singleton pattern, 236
- size, 79
- slice, 87
- SMTP, 320
- snap change, 116
- software engineering, 143, 305
- SOLID principles, 150
- solution, 91
- solution pattern, 232
- source generators, 162
- spaghetti code, 11
- sprint, 306
- SQL, 222
- SRP, 150
- stack, 33–35, 90, 98, 155, 158, 176, 193, 197
- stack overflow, 96
- stack pointer, 24
- stack register, 24
- stack segment, 24, 95, 96
- stack unwinding, 170, 191
- state, 89, 101, 108, 185
- state machine, 186
- state pattern, 281
- statement, 10, 15, 31, 41, 133
- static, 118, 127, 144
- static class, 119
- static keyword, 119
- static member, 119
- stream, 213
- string, 50, 122, 175, 393
- string interpolation, 52
- string literal, 50
- struct, 121, 156
- structural model, 303
- structural pattern, 232, 241
- structural programming, 14, 69
- structure, 14
- switch expression, 67
- switch statement, 65
- synchronous, 163, 331
- syntactic sugar, 333
- TDD, 363
- technician, 1
- test pattern, 232
- text segment, 24
- this keyword, 103
- thread, 192, 331
- thread-safe, 48, 122, 192, 238
- three layer, 310
- throw an exception, 164, 166
- tier, 307
- tight coupling, 146, 148, 326
- transaction, 225
- two-dimentional array, 84
- type discovery, 388
- type inference, 184
- type safety, 196
- UIC, 308
- UML, 113, 143
- unary operator, 42
- unboxing, 176, 182
- unhandled exception, 170
- unique, 81, 396
- unit test, 160
- unmanaged, 176
- unmanaged code, 23, 176
- unmanaged resources, 124
- unsigned, 12
- unsigned right shift, 44
- UPC, 312
- URI, 320
- URL, 319
- user defined class, 89
- user interface, 26
- user-defined-class, 89
- using, 145
- utility class, 119, 127
- VADE, 316
- value object, 394
- value type, 33, 157, 159
- variable, 12, 14, 33
- variable declaration, 35
- variable scope, 35
- variable type, 13
- variance, 81
- verbatim string, 51, 213
- virtual, 118
- visibility, 91
- visitor pattern, 272
- void, 93
- waterfall model, 305
- WCF, 21, 304, 319
- Web API, 319, 320, 325
- while statement, 69
- white space, 31
- XML, 27, 219
- zero-allocation, 345, 346